

QSpace tensor library (v3.2)

Generated by Doxygen 1.9.4

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	5
2.1 Class List	5
3 File Index	9
3.1 File List	9
4 Class Documentation	11
4.1 __wb_msgbuf Struct Reference	11
4.2 Wb::ARGV Class Reference	11
4.3 Mx::Array< T > Class Template Reference	12
4.4 clebsch::binomial_t Class Reference	13
4.5 Wb::bitset< T > Class Template Reference	13
4.6 Wb::Bitset Class Reference	13
4.7 blockSpaceQS< TQ, TD > Class Template Reference	14
4.8 Wb::cc_quad Class Reference	14
4.9 CData< TQ, TD > Class Template Reference	15
4.10 cdata< TD > Class Template Reference	17
4.11 cgc_contract_id< TM > Class Template Reference	18
4.12 cgdStatus Class Reference	19
4.13 cgrType Class Reference	20
4.14 Wb::citerator< T, T2 > Class Template Reference	21
4.15 Wb::citerator< T, T2 * > Class Template Reference	21
4.16 Wb::CleanUp Class Reference	22
4.17 Wb::Clock Class Reference	22
4.18 Wb::Clock_resume Class Reference	22
4.19 clebsch::coefficients Class Reference	23
4.20 CPAT< TQ, TD > Class Template Reference	23
4.21 CPAT_Set< TQ, TD > Class Template Reference	24
4.22 cpu_time Class Reference	25
4.23 CRef< TQ > Class Template Reference	25
4.24 CStore< TQ > Class Template Reference	27
4.25 ctrlIdx Class Reference	29
4.26 clebsch::decomposition Class Reference	30
4.27 CG::FileLock Class Reference	30
4.28 genRG_base< TQ, TD > Class Template Reference	31
4.29 genRG_struct< TQ, TD > Class Template Reference	31
4.30 Wb::gpara__ Class Reference	32
4.31 groupIndex< T > Class Template Reference	33
4.32 icFlags Class Reference	33
4.33 clebsch::index_adapter Class Reference	34

4.34 IndexLabels Class Reference	34
4.35 indexSparseRef< TD > Class Template Reference	35
4.36 Wb::IOstat Class Reference	36
4.37 is_pointer_< T > Class Template Reference	36
4.38 is_pointer_< T * > Class Template Reference	37
4.39 is_pointer_< T ** > Class Template Reference	37
4.40 is_pointer_< T *** > Class Template Reference	37
4.41 iTag Class Reference	37
4.42 Wb::iterator< T, T2 > Class Template Reference	38
4.43 Wb::iterator< T, T2 * > Class Template Reference	39
4.44 iterSparseCMref< TD > Class Template Reference	39
4.45 Wb::line_stat__ Class Reference	40
4.46 Wb::LogException Struct Reference	40
4.47 Wb::matFile Class Reference	41
4.48 MEM_SET< T > Class Template Reference	42
4.49 MEM_SET< T * > Class Template Reference	42
4.50 MHash< TM > Class Template Reference	42
4.51 Wb::mpfr__< P > Class Template Reference	43
4.52 Wb::MREC Class Reference	45
4.53 mxArray_buf Class Reference	46
4.54 MXPut Class Reference	46
4.55 mxStruct Class Reference	47
4.56 Wb::myname__ Class Reference	48
4.57 NRGData< TQ, TD > Class Template Reference	48
4.58 NRGIndex< TQ, TD > Class Template Reference	50
4.59 Wb::num2Fmt< T > Class Template Reference	51
4.60 opFlags< T > Class Template Reference	51
4.61 OPTS Class Reference	52
4.62 OrthoCGS< TQ > Class Template Reference	52
4.63 pairPatternCG< T > Class Template Reference	53
4.64 Wb::Path Class Reference	53
4.65 clebsch::pattern Class Reference	54
4.66 Wb::Profile Class Reference	54
4.67 QBlock< TQ, TD, TE > Class Template Reference	55
4.68 QDir Class Reference	55
4.69 QHash< TQ > Class Template Reference	56
4.70 QHash_ Class Reference	57
4.71 QMap< TQ > Class Template Reference	57
4.72 QSet< TQ > Class Template Reference	58
4.73 qset< TQ > Class Template Reference	59
4.74 QSpace< TQ, TD > Class Template Reference	61
4.75 QType Class Reference	67

4.76 QVec Class Reference	69
4.77 Wb::rand Class Reference	70
4.78 RCHandle Class Reference	71
4.79 RIXS_para Class Reference	71
4.80 RSpecData< TS > Class Template Reference	72
4.81 RStore< TQ, TD > Class Template Reference	72
4.82 wblog::SBUF Class Reference	73
4.83 sparseCollector2D< TD > Class Template Reference	73
4.84 sparseIndex2D< T > Class Template Reference	74
4.85 Spectral< TS > Class Template Reference	74
4.86 Wb::sptr< T > Class Template Reference	75
4.87 STAT_DGEMM Class Reference	76
4.88 wblog::stdio_buf Class Reference	76
4.89 Wb::String Class Reference	76
4.90 SVDDData< TQ, TD > Class Template Reference	77
4.91 DY::Symmetry Class Reference	77
4.92 TDSpectral< TS > Class Template Reference	78
4.93 Wb::tmpFile Class Reference	79
4.94 Wb::UseClock Class Reference	79
4.95 wbarry< T > Class Template Reference	79
4.96 wbarrRef< T > Class Template Reference	86
4.97 wbcomplex Class Reference	87
4.98 wbdebug_dummy Class Reference	88
4.99 wbhist Class Reference	88
4.100 wbindex Class Reference	88
4.101 wbIndex Class Reference	89
4.102 Wb::WbListMTRACK Class Reference	90
4.103 wblogbuf_struct Class Reference	91
4.104 wblogbuffer Class Reference	91
4.105 wbMatrix< T > Class Template Reference	92
4.106 wbMsg Class Reference	99
4.107 wbperm Class Reference	99
4.108 wbRecs< T > Class Template Reference	101
4.109 wbREF< TX > Class Template Reference	101
4.110 wbSigHandler Class Reference	101
4.111 wbsparray< TD > Class Template Reference	102
4.112 wbstring Class Reference	107
4.113 wbsys Class Reference	109
4.114 wbtopy Class Reference	110
4.115 WbUtil< T > Class Template Reference	110
4.116 wbvector< T > Class Template Reference	111
4.117 clebsch::weight Class Reference	117

4.118 DY::weight_info< TQ > Class Template Reference	118
4.119 DY::Weights< TQ > Class Template Reference	118
4.120 X3Map< TQ, TD > Class Template Reference	119
4.121 x3map< TQ, TD > Class Template Reference	119
4.122 XMAP Class Reference	120
4.123 XMAP_CMat Class Reference	121
4.124 XMAP_Node Class Reference	121
5 File Documentation	123
5.1 clebsch.hh	123
5.2 cpat.hh	176
5.3 gathered.hh	190
5.4 memlib.hh	196
5.5 memtrack.hh	200
5.6 mexlib.hh	201
5.7 mpsortho.hh	207
5.8 mxlib.hh	209
5.9 nomex.hh	215
5.10 nrgdata.hh	217
5.11 QSpace.hh	246
5.12 QSpace_aux.hh	285
5.13 spectral.hh	288
5.14 util.hh	308
5.15 wbarrray.hh	314
5.16 wbarrray_blas.hh	341
5.17 wbbitsset.hh	345
5.18 wblas.hh	347
5.19 wbclock.hh	351
5.20 wbcomplex.hh	354
5.21 wbeigs.hh	359
5.22 wfft.hh	360
5.23 wbindex.hh	361
5.24 wbio.h	375
5.25 wblib.h	376
5.26 wlibx.h	384
5.27 wblog.h	385
5.28 wbMatIO.hh	389
5.29 wbMatrix.hh	391
5.30 wbMatrix_blas.hh	423
5.31 wbmpfr.hh	424
5.32 wbmshg.hh	430
5.33 wbomp.hh	432

5.34 wbopts.hh	434
5.35 wbperm.hh	440
5.36 wbsort.hh	444
5.37 wbsparray.hh	445
5.38 wbstring.hh	468
5.39 wbutil.hh	473
5.40 wbvector.hh	475
5.41 xmap.hh	498
Index	501

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

__wb_msgbuf	11
Wb::ARGV	11
Mx::Array< T >	12
clebsch::binomial_t	13
Wb::bitset< T >	13
blockSpaceQS< TQ, TD >	14
Wb::cc_quad	14
cgdStatus	19
cgrType	20
Wb::citerator< T, T2 >	21
Wb::citerator< T, T2 * >	21
Wb::CleanUp	22
Wb::Clock	22
Wb::Clock_resume	22
clebsch::coefficients	23
CPAT< TQ, TD >	23
CPAT_Set< TQ, TD >	24
cpu_time	25
CRef< TQ >	25
CStore< TQ >	27
clebsch::decomposition	30
exception	
Wb::LogException	40
CG::FileLock	30
genRG_base< TQ, TD >	31
genRG_struct< TQ, TD >	31
Wb::gpara__	32
groupIndex< T >	33
icFlags	33
clebsch::index_adapter	34
indexSparseRef< TD >	35
Wb::IOstat	36
is_pointer_< T >	36
is_pointer_< T * >	37
is_pointer_< T ** >	37

is_pointer_< T *** >	37
iTag	37
Wb::iterator< T, T2 >	38
Wb::iterator< T, T2 * >	39
iterSparseCMref< TD >	39
Wb::line_stat__	40
Wb::matFile	41
MEM_SET< T >	42
MEM_SET< T * >	42
MHash< TM >	42
Wb::mpfr__< P >	43
Wb::MREC	45
mxArray_buf	46
MXPut	46
mxStruct	47
Wb::myname__	48
NRGData< TQ, TD >	48
NRGIndex< TQ, TD >	50
Wb::num2Fmt< T >	51
opFlags< T >	51
OPTS	52
OrthoCGS< TQ >	52
pairPatternCG< T >	53
pairPatternCG< double >	53
Wb::Path	53
clebsch::pattern	54
Wb::Profile	54
QBlock< TQ, TD, TE >	55
QBlock< TQ, TD, double >	55
QHash< TQ >	56
QHash_	57
QMap< TQ >	57
QSet< TQ >	58
CData< TQ, RTD >	15
CData< TQ, TD >	15
QSpace< TQ, TD >	61
QType	67
Wb::rand	70
RCStore	71
RIXS_para	71
RSpecData< TS >	72
RStore< TQ, TD >	72
wblog::SBUF	73
sparseCollector2D< TD >	73
sparseIndex2D< T >	74
Spectral< TS >	74
Wb::sptr< T >	75
Wb::sptr< double >	75
Wb::sptr< int >	75
Wb::sptr< TD >	75
Wb::sptr< TS >	75
STAT_DGEMM	76
wblog::stdio_buf	76
SVDDData< TQ, TD >	77
DY::Symmetry	77
TDSpectral< TS >	78
Wb::tmpFile	79
Wb::UseClock	79

wbarray< T >	79
wbarray< double >	79
wbarray< int >	79
wbarray< TD >	79
wbarray< TS >	79
wbarrRef< T >	86
wbcomplex	87
wbdebug_dummy	88
wbhist	88
Wb::WbListMTRACK	90
wblogbuf_struct	91
wblogbuffer	91
wbMatrix< T >	92
wbMatrix< const wbvector< CRef< TQ > > * >	92
wbMatrix< CRef< TQ > >	92
wbMatrix< double >	92
wbMatrix< int >	92
wbMatrix< SPIDX_T >	92
wbMatrix< TQ >	92
wbMatrix< TS >	92
wbMatrix< unsigned >	92
wbMatrix< wbcomplex >	92
wbMatrix< widx_t >	92
wbMsg	99
wbRecs< T >	101
wbREF< TX >	101
wbSigHandler	101
wbsparray< TD >	102
cdata< TD >	17
wbsparray< RTD >	102
cdata< RTD >	17
wbsys	109
wbtop	110
WbUtil< T >	110
wbvector< T >	111
wbvector< char >	111
QDir	55
Wb::Bitset	13
Wb::String	76
wbstring	107
wbvector< CRef< TQ > >	111
wbvector< double >	111
wbvector< int >	111
wbvector< iTag >	111
IndexLabels	34
wbvector< mxArray * >	111
wbvector< NRGIndex< TQ, TD > >	111
wbvector< pairPatternCG< double > >	111
wbvector< QBlock< TQ, TD, double > >	111
wbvector< QSpace< TQ, TD > >	111
wbvector< QType >	111
QVec	69
wbvector< RSpecData< TS > >	111
wbvector< RTD >	111
wbvector< size_t >	111
wbvector< SPIDX_T >	111
wbvector< sSPIDX_T >	111

<code>wbvector< TD ></code>	111
<code>wbvector< TM ></code>	111
<code>cgc_contract_id< TM ></code>	18
<code>wbvector< TQ ></code>	111
<code>qset< TQ ></code>	59
<code>wbvector< TS ></code>	111
<code>wbvector< unsigned ></code>	111
<code>ctrlIdx</code>	29
<code>wbvector< unsigned long ></code>	111
<code>wbvector< warray< TD > * ></code>	111
<code>wbvector< wbMatrix< unsigned > ></code>	111
<code>wbvector< wbSparrayTD ></code>	111
<code>wbvector< wbstring ></code>	111
<code>wbvector< wbvector< CPAT_Set< TQ, TD > > ></code>	111
<code>wbvector< wbvector< QSpace< TQ, TD > > ></code>	111
<code>wbvector< widx_t ></code>	111
<code>wbIndex</code>	89
<code>wbindex</code>	88
<code>wbvector< wperm_t ></code>	111
<code>wbperm</code>	99
<code>wbvector< XMAP_Node ></code>	111
<code>XMAP</code>	120
<code>clebsch::weight</code>	117
<code>DY::weight_info< TQ ></code>	118
<code>DY::Weights< TQ ></code>	118
<code>X3Map< TQ, TD ></code>	119
<code>x3map< TQ, TD ></code>	119
<code>XMAP_CMat</code>	121
<code>XMAP_Node</code>	121

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

__wb_msgbuf	11
Wb::ARGV	11
Mx::Array< T >	12
clebsch::binomial_t	13
Wb::bitset< T >	13
Wb::Bitset	13
blockSpaceQS< TQ, TD >	14
Wb::cc_quad	14
CData< TQ, TD >	15
cdata< TD >	17
cgc_contract_id< TM >	18
cgdStatus	19
cgrType	20
Wb::citerator< T, T2 >	21
Wb::citerator< T, T2 * >	21
Wb::CleanUp	22
Wb::Clock	22
Wb::Clock_resume	22
clebsch::coefficients	23
CPAT< TQ, TD >	23
CPAT_Set< TQ, TD >	24
cpu_time	25
CRef< TQ >	25
CStore< TQ >	27
ctrlIdx	29
clebsch::decomposition	30
CG::FileLock	30
genRG_base< TQ, TD >	31
genRG_struct< TQ, TD >	31
Wb::gpara__	32
groupIndex< T >	33
icFlags	33
clebsch::index_adapter	34
IndexLabels	34
indexSparseRef< TD >	35

Wb::IOstat	36
is_pointer_< T >	36
is_pointer_< T * >	37
is_pointer_< T ** >	37
is_pointer_< T *** >	37
iTag	37
Wb::iterator< T, T2 >	38
Wb::iterator< T, T2 * >	39
iterSparseCMref< TD >	39
Wb::line_stat__	40
Wb::LogException	40
Wb::matFile	41
MEM_SET< T >	42
MEM_SET< T * >	42
MHash< TM >	42
Wb::mpfr__< P >	43
Wb::MREC	45
mxArray_buf	46
MXPut	46
mxStruct	47
Wb::myname__	48
NRGData< TQ, TD >	48
NRGIndex< TQ, TD >	50
Wb::num2Fmt< T >	51
opFlags< T >	51
OPTS	52
OrthoCGS< TQ >	52
pairPatternCG< T >	53
Wb::Path	53
clebsch::pattern	54
Wb::Profile	54
QBlock< TQ, TD, TE >	55
QDir	55
QHash< TQ >	56
QHash_	57
QMap< TQ >	57
QSet< TQ >	58
qset< TQ >	59
QSpace< TQ, TD >	61
QType	67
QVec	69
Wb::rand	70
RCStore	71
RIXS_para	71
RSpecData< TS >	72
RStore< TQ, TD >	72
wblog::SBUF	73
sparseCollector2D< TD >	73
sparseIndex2D< T >	74
Spectral< TS >	74
Wb::sptr< T >	75
STAT_DGEMM	76
wblog::stdio_buf	76
Wb::String	76
SVDDData< TQ, TD >	77
DY::Symmetry	77
TDSpectral< TS >	78
Wb::tmpFile	79

Wb::UseClock	79
wbarray< T >	79
wbarrRef< T >	86
wbcomplex	87
wbdebug_dummy	88
wbhist	88
wbindex	88
wbIndex	89
Wb::WbListMTRACK	90
wblogbuf_struct	91
wblogbuffer	91
wbMatrix< T >	92
wbMsg	99
wbperm	99
wbRecs< T >	101
wbREF< TX >	101
wbSigHandler	101
wbsparray< TD >	102
wbstring	107
wbsys	109
wbtop	110
WbUtil< T >	110
wbvector< T >	111
clebsch::weight	117
DY::weight_info< TQ >	118
DY::Weights< TQ >	118
X3Map< TQ, TD >	119
x3map< TQ, TD >	119
XMAP	120
XMAP_CMat	121
XMAP_Node	121

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

clebsch.hh	??
cpat.hh	??
gathered.hh	??
memlib.hh	??
memtrack.hh	??
mexlib.hh	??
mpsortho.hh	??
mxlib.hh	??
nomex.hh	??
nrgdata.hh	??
QSpace.hh	??
QSpace_aux.hh	??
spectral.hh	??
util.hh	??
wbarray.hh	??
wbarray_blas.hh	??
wbbitset.hh	??
wbblas.hh	??
wbclock.hh	??
wbcomplex.hh	??
wbeigs.hh	??
wbfft.hh	??
wbindex.hh	??
wbio.h	??
wblib.h	??
wblibx.h	??
wblog.h	??
wbMatIO.hh	??
wbMatrix.hh	??
wbMatrix_blas.hh	??
wbmpfr.hh	??
wbmsg.hh	??
wbomp.hh	??
wbopts.hh	??
wbperm.hh	??

wbsort.hh	??
wbsparray.hh	??
wbstring.hh	??
wbutil.hh	??
wbvector.hh	??
xmap.hh	??

Chapter 4

Class Documentation

4.1 `__wb_msgbuf` Struct Reference

Public Attributes

- long **type**
- time_t **time**
- char **text** [128]
- double **val**

The documentation for this struct was generated from the following file:

- `wbmsg.hh`

4.2 `Wb::ARGV` Class Reference

Public Member Functions

- **ARGV** (`va_list *vl=NULL`)

The documentation for this class was generated from the following file:

- `wblib.h`

4.3 Mx::Array< T > Class Template Reference

Public Member Functions

- **Array** (unsigned ndim, size_t *dims, char t_=0, char c_=-1)
- **Array** (size_t d1, size_t d2, char t_=0, char c_=-1)
- **Array** (const [wbvector](#)< size_t > &S, char t_=0, char c_=-1)
- **Array** (const mxArray *a, char t_=0, char c_=-1)
- **Array** (const char *F, int L, const mxArray *a, char t_=0, char c_=-1)
- [wbstring toStr](#) (char vflag=0, unsigned l=64) const
- [wbstring toStrT](#) (char vflag=0) const
- void **init_xstr** ()
- const char * **xStr** () const
- mxArray * **Return** (T *&dd)
- mxArray * **Return** ()
- [Array](#) & **init** (unsigned ndim, size_t *dims, char t_=-1, char c_=-1)
- [Array](#) & **init** (const mxArray *a, char t_=-1, char c_=-1)
- template<class Tb >
mxArray * **copyFromTR** (const Tb *b, char tcheck=0)
- template<class Tb >
int **copy_from** (const Tb *b, char tcheck=0)
- template<class Tb >
size_t **copyTo** (Tb *b, char tcheck=0) const
- size_t **copy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- size_t **rcopy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- size_t **zcopy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- template<class Tx >
size_t **cpy_to** (T *b, const [wbperm](#) *P, char tcheck, const Tx *ad_) const
- size_t **ncopy_to** (T *b, size_t n, char tcheck=0)
- size_t **copy_to** (CGD_TYPE *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (CGR_TYPE *b, const [wbperm](#) *P, char tcheck) const
- size_t **zcopy_to** ([wbcomplex](#) *b, const [wbperm](#) *P, char ck) const
- size_t **copy_to** ([wbcomplex](#) *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (double *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (int *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (unsigned *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (size_t *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (char *b, const [wbperm](#) *P, char tcheck) const

Public Attributes

- mxArray * **ax**
- T * **data**
- size_t **len**
- unsigned **rank**
- bool **trans**
- char **cmplx**
- char **mxref**
- std::string **xstr**

The documentation for this class was generated from the following files:

- mxlib.hh
- mexlib.cc

4.4 clebsch::binomial_t Class Reference

Public Member Functions

- `int operator() (int n, int k)`

The documentation for this class was generated from the following file:

- `ClebschGordan_Arne.cpp`

4.5 Wb::bitset< T > Class Template Reference

Public Member Functions

- `bitset` (const `bitset< T >` &b)
- `bitset` & `setall` (bool t)
- `bitset` & `set` (unsigned i)
- `bitset` & `unset` (unsigned i)
- bool `isset` (unsigned i)
- unsigned `nnz` () const
- bool `operator==` (const `bitset` &b) const
- bool `operator!=` (const `bitset` &b) const
- `wbstring toStr` () const
- `mxArray * toMx` () const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- void `add2MxStruct` (`mxArray *S`, unsigned i) const
- void `put` (const char *vname, const char *ws="base") const
- void `put` (const char *file, int line, const char *vname, const char *ws="base") const

Public Attributes

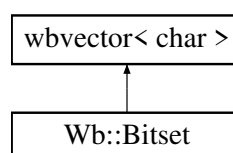
- `T data`

The documentation for this class was generated from the following files:

- `wbbitset.hh`
- `wbbitset.cc`

4.6 Wb::Bitset Class Reference

Inheritance diagram for `Wb::Bitset`:



Public Member Functions

- **Bitset** (unsigned n=0)
- **Bitset** (const [Bitset](#) &b)
- **Bitset** & **init** (unsigned n=0)
- **Bitset** & **init** (unsigned n, const char *b)
- **Bitset** & **setall** (bool t)
- **Bitset** & **set** (unsigned i)
- **Bitset** & **unset** (unsigned i)
- bool **isset** (unsigned i)
- unsigned **nnz** () const
- bool **operator==** (const [Bitset](#) &b) const
- bool **operator!=** (const [Bitset](#) &b) const
- [wbstring](#) **toStr** () const
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- void **put** (const char *vname, const char *ws="base") const
- void **put** (const char *file, int line, const char *vname, const char *ws="base") const

Public Attributes

- unsigned **nbits**

Protected Member Functions

- [Bitset](#) & **NEW_BITSET** (unsigned n, const char *d=NULL)

The documentation for this class was generated from the following files:

- wbbitsset.hh
- wbbitsset.cc

4.7 blockSpaceQS< TQ, TD > Class Template Reference

The documentation for this class was generated from the following file:

- clebsch.hh

4.8 Wb::cc_quad Class Reference

Public Member Functions

- **cc_quad** (long double l=0)
- [cc_quad](#) **sqrt** () const
- [cc_quad](#) **fabs** () const
- [cc_quad](#) **abs** () const
- [cc_quad](#) **operator+** (const [cc_quad](#) &b) const
- [cc_quad](#) **operator-** (const [cc_quad](#) &b) const
- [cc_quad](#) **operator*** (const [cc_quad](#) &b) const
- [cc_quad](#) **operator/** (const [cc_quad](#) &b) const
- [cc_quad](#) **operator+=** (const [cc_quad](#) &b)
- [cc_quad](#) **operator-=** (const [cc_quad](#) &b)
- [cc_quad](#) **operator*=** (const [cc_quad](#) &b)
- [cc_quad](#) **operator/=** (const [cc_quad](#) &b)

Public Attributes

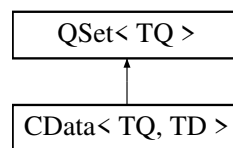
- long double **f**

The documentation for this class was generated from the following file:

- wbmpfr.hh

4.9 CData< TQ, TD > Class Template Reference

Inheritance diagram for CData< TQ, TD >:



Public Member Functions

- **CData** (const [CData](#) &C)
- **CData** (const [QSet](#)< TQ > &Q)
- **CData** (const [CRef](#)< TQ > &R)
- void **init** ()
- [CData](#) & **init** (const [CRef](#)< TQ > &S)
- [CData](#) & **initAbelian** (const [QSet](#)< TQ > &Q)
- [CData](#) & **initScalar** (const [QSet](#)< TQ > &Q)
- [CData](#) & **init3** (const [QType](#) &t_, SPIDX_T D1, SPIDX_T D2, SPIDX_T D, unsigned M=1)
- [CData](#) & **init3** (const [QType](#) &t_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const [qset](#)< TQ > &J, unsigned M=1)
- [CData](#) & **init3_abelian** (const [QType](#) &t_, const TQ &J1, const TQ &J2, const TQ &J)
- template<class T2 >
[CData](#) & **init** (const [CData](#)< TQ, T2 > &B)
- int **Reflinit** (const char *F, int L, const [CData](#)< TQ, TD > &B, char lflag=0, char bare=0)
- [CData](#) & **Reflinit_auxtr** (const char *F, int L, const [wbvector](#)< SPIDX_T > &S, const [wbvector](#)< RTD > &cgt)
- int **Reduce2Ref** (const char *F=0, int L=0, char force=0)
- int **LoadRef** (const char *F=0, int L=0, char force=1)
- int **Load_CRef** (const char *F, int L, [CData](#) &Cr, const char *cgd_file) const
- [CData](#) & **initX3** (const char *F, int L, const [QSet](#)< TQ > &Q, [wbvector](#)< char > &cflags, [wbperm](#) &P, unsigned loadRC=0, unsigned char flag=0)
- int **init_mxCRef** (const char *F, int L, const mxArray *S, unsigned k)
- TQ * **qptr** (unsigned k) const
- [CData](#) & **operator=** (const [CData](#) &B)
- [CData](#) & **operator=** (const [QSet](#)< TQ > &B)
- [CData](#) & **save2** ([CData](#) &B)
- bool **olderThan** (const [CData](#) &B) const
- bool **operator!=** (const [CData](#) &B) const
- bool **operator==** (const [CData](#) &B) const
- int **gotDiff** (const [CData](#) &B, char lflag=0) const

- bool **sameAs** (const [CData](#) &B, char lflag=0) const
- bool **operator!=** (const [QSet](#)< TQ > &B) const
- bool **operator==** (const [QSet](#)< TQ > &B) const
- bool **operator!=** (const [cgdStatus](#) &b) const
- bool **operator==** (const [cgdStatus](#) &b) const
- bool **isEmpty** (char lflag=0) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **isScalar** (char dflag=0) const
- TD **getScalar** (const char *F=NULL, int L=0) const
- unsigned **rank** (const char *F=NULL, int L=0) const
- unsigned **rankS** () const
- bool **isrank** (unsigned r, unsigned *r_=NULL) const
- template<class T >
[wbvector](#)< T > & **getSize** ([wbvector](#)< T > &S, char bare=0, const [wbperm](#) *cgp=NULL) const
- SPIDX_T **dim** () const
- bool **isReflnit** (char check=1) const
- bool **checkValidStat** (const char *F=0, int L=0) const
- int **checkSameStat** (const char *F, int L, const [CData](#) &B, char lflag=1) const
- bool **sameType** (const [CData](#) &S, const char *F=NULL, int L=0) const
- int **cmpOM** (const [CData](#) &B) const
- template<class DB >
bool **sameSizeR** (const [CData](#)< TQ, DB > &B, unsigned *r=NULL, const char *F=NULL, int L=0) const
- template<class DB >
bool **sameSizeR** (const [cdata](#)< DB > &B, unsigned *r=NULL, const char *F=NULL, int L=0) const
- template<class T2 >
int **sameSizeR** (const char *F, int L, const [wbvector](#)< T2 > &S) const
- bool **isSymmmetric** () const
- unsigned **checkOM** (const char *F=NULL, int L=0) const
- unsigned **gotOM** (const char *F=NULL, int L=0) const
- unsigned **getOM** (const char *F=NULL, int L=0) const
- [CData](#) & **initOM** (const char *F, int L)
- int **gotFullOM** (const char *F=NULL, int L=0) const
- int **completeOM_DegQ** (const char *F, int L, const [wbperm](#) &p, unsigned level=0)
- [CData](#) & **Project** (const char *F, int L, [cdata](#)< TD > X, [wbvector](#)< TD > &cgw)
- double **normDiff** (const char *F, int L, const [CData](#) &S, double eps=1E14) const
- double **norm2** (unsigned k=-1) const
- [wbvector](#)< RTD > & **trace** (const char *F, int L, [wbvector](#)< RTD > &cgt) const
- [wbvector](#)< RTD > **trace** (const char *F=NULL, int L=0) const
- [CData](#) & **trace** (const char *F, int L, [ctrlDx](#) &i1, [ctrlDx](#) &i2, [CData](#)< TQ, TD > &X) const
- [CData](#) & **AddMultiplicity** (const char *F, int L, const [cdata](#)< TD > &c)
- [CData](#) & **AddMultiplicity** (const char *F, int L, const [CData](#) &C)
- [cdata](#)< TD > & **getMultiplicity** (unsigned im, [cdata](#)< TD > &c) const
- bool **isBasicCG** (const char *F=NULL, int L=0) const
- SPIDX_T **getBasicCGSize** (unsigned *m=NULL) const
- [wbsparray](#)< TD > & **getBasicCG** (unsigned k, [wbsparray](#)< TD > &a) const
- int **getCG_set** (const char *F, int L, [wbIndex](#) &Idx, [wbsparray](#)< TD > &a) const
- [CData](#) & **reduceto1JSymbol** (const char *F, int L, [CData](#)< TQ, TD > &C) const
- int **checkConsistency** (const char *F=NULL, int L=0) const
- int **checkNormSign** (const char *F=NULL, int L=0, char xflag=0) const
- int **checkQ** (const char *F, int L, const [QSet](#)< TQ > &Q, const char *istr=NULL) const
- bool **checkAdditivityZ** (const char *F, int L, const [wbMatrix](#)< double > &Z1, const [wbMatrix](#)< double > &Z2, const [wbMatrix](#)< double > &Z, TD eps=CG_EPS2) const
- double **SkipTiny** (const char *F=NULL, int L=0)
- [CData](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [CData](#) & **permute** ([CData](#) &B, const [wbperm](#) &P, char iflag=0) const

- [CData](#) & [Conj](#) ()
- [CData](#) & [Conj](#) (unsigned i)
- [size_t](#) [memSize](#) () const
- void [info](#) (const char *istr=NULL, const char *F=NULL, int L=0) const
- [wbstring](#) [dStr](#) () const
- [wbstring](#) [qStr](#) () const
- [wbstring](#) [toStr](#) (char vflag=0) const
- [wbstring](#) [sizeStr](#) (char xflag=0) const
- mxArray * [toMx](#) () const
- mxArray * [mxCreateStruct](#) (unsigned m, unsigned n) const
- void [add2MxStruct](#) (mxArray *S, unsigned i, char tst=0) const
- mxArray * [mxCreateCell](#) (unsigned m, unsigned n) const
- void [add2MxCell](#) (mxArray *S, unsigned i, char tst=0) const
- void [put](#) (const char *vname, const char *ws="caller") const
- void [put](#) (const char *F, int L, const char *vname, const char *ws="caller") const
- void [setuser_BUF](#) (unsigned char c=1)
- void [setuser_BUF](#) (unsigned char c, unsigned l)
- void [setuser_BUF_ref](#) (unsigned char c=1)
- bool [gotuser_BUF](#) () const
- bool [gotuser_BUF_ref](#) () const

Public Attributes

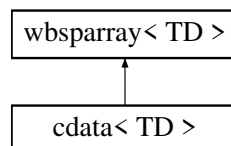
- [cdata](#)< TD > [cgd](#)
- [cgdStatus](#) [cstat](#)

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_aux.cc
- clebsch_io.cc

4.10 cdata< TD > Class Template Reference

Inheritance diagram for cdata< TD >:



Public Member Functions

- [cdata](#) & **init** (SPIDX_T l=0)
- [cdata](#) & **init** (SPIDX_T D1, SPIDX_T D2, SPIDX_T D, unsigned M=1)
- [cdata](#) & **init** (const [wbvector](#)< SPIDX_T > &S)
- [cdata](#) & **init** (const [cdata](#) &C)
- template<class TQ >
[cdata](#) & **init** (const [CRef](#)< TQ > &R, char full=1)
- template<class T2 >
[cdata](#) & **init** ([wbsparray](#)< T2 > &C)
- [cdata](#) & **init** (const char *F, int L, const mxArray *a, unsigned k, const [QType](#) &q, unsigned r=-1)
- [cdata](#) & **operator=** (const [cdata](#) &C)
- [cdata](#) & **initScalar** (double x=1, unsigned r=-1)
- [cdata](#) & **initIdentity** (const [QType](#) &q, SPIDX_T d, TD dval=1)
- [cdata](#) & **initIdentity** (const [wbvector](#)< SPIDX_T > &S)
- bool **isScalar** () const
- unsigned **len** () const
- unsigned **getOM** (const char *F=NULL, int L=0, const char *istr=NULL) const
- [cdata](#) & **Conj** ()
- [cdata](#) & **permute** ([cdata](#) &B, const [wbperm](#) &P0, char iflag=0) const
- [cdata](#) & **Permute** (const [wbperm](#) &P0, char iflag=0)
- [cdata](#) & **Kron** (const [cdata](#) &B)
- TD **contract** (const char *F, int L, const [ctrlidx](#) &ica, const [cdata](#) &B, const [ctrlidx](#) &icb, [cdata](#) &C, [cdata](#) *Cx=NULL, [wbperm](#) *P=NULL, char normalize=1) const
- double **SkipTiny** (const char *F=NULL, int L=0)
- TD **NormSignC** (const char *F=NULL, int L=0, unsigned m=0, TD eps=CG_EPS1, TD eps2=CG_EPS2)
- void **info** () const
- template<class TQ >
[cdata](#)< TD > & **init** (const [CRef](#)< TQ > &R, char full)

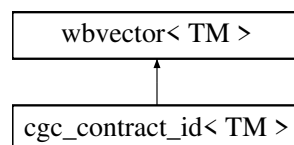
Additional Inherited Members

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_aux.cc
- clebsch_io.cc

4.11 [cgc_contract_id](#)< TM > Class Template Reference

Inheritance diagram for [cgc_contract_id](#)< TM >:



Public Member Functions

- `template<class TQ , class TD >`
`cgc_contract_id` (const `CData`< TQ, TD > &A, const `ctrlidx` &ica, const `CData`< TQ, TD > &B, const `ctrlidx` &icb)
- `template<class TQ , class TD >`
`void extract` (`CData`< TQ, TD > &A, `ctrlidx` &ica, `CData`< TQ, TD > &B, `ctrlidx` &icb, char xflag=1) const
- `size_t memSize` () const
- `void wblog_` (const char *F=0, int L=0, const char *istr=0) const
- `wbstring toStr` (char vflag=7) const

Additional Inherited Members

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_aux.cc`

4.12 cgdStatus Class Reference

Public Member Functions

- `cgdStatus` (CGD_TYPE t_)
- `cgdStatus` (double t_)
- `cgdStatus` (const char *F, int L, mxArray *a)
- `template<class TQ >`
`cgdStatus` (const `CData`_TQ &C)
- `cgdStatus & init` ()
- `cgdStatus & init` (const CGD_TYPE &t_)
- `cgdStatus & init0` (const CGD_TYPE &t_)
- `cgdStatus & update_m` ()
- `cgdStatus & update_m` (const CGD_TYPE &t_)
- `cgdStatus & operator=` (const `cgdStatus` &b)
- `cgdStatus & init` (const `cgdStatus` &b)
- `CGR_TYPE init` (const char *F, int L, const mxArray *a)
- `int sameID` (const `cgdStatus` &b) const
- `int cmp` (const char *F, int L, const `cgdStatus` &b) const
- `bool olderThan` (const `cgdStatus` &b) const
- `bool newerThan` (const `cgdStatus` &b) const
- `bool hasID` (char lflag=0) const
- `bool operator==` (const CGD_TYPE t_) const
- `bool operator!=` (const CGD_TYPE t_) const
- `bool operator==` (const `cgdStatus` &b) const
- `bool operator!=` (const `cgdStatus` &b) const
- `bool isSet` () const
- `bool isEmpty` () const
- `int invalid` () const
- `bool sameAs` (const `cgdStatus` &b, char lflag=0) const
- `const char * tstr` () const
- `wbstring toStr` (char vflag=0) const
- `wbstring u2Str` (const char *istr=NULL) const
- `mxArray * toMx` () const
- `mxArray * toMx` (CGR_TYPE rt) const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- `mxArray * add2MxStruct` (mxArray *S, unsigned k) const

Public Attributes

- CGD_TYPE **t**
- double **ctime**
- double **mtime**
- unsigned **ID**
- unsigned char **user** [4]

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.13 cgrType Class Reference

Public Member Functions

- **cgrType** (CGR_TYPE t_=CGR_DEFAULT)
- **cgrType** (double t_)
- template<class T >
cgrType & **init** (const char *F, int L, T t_)
- **cgrType** & **operator=** (const **cgrType** &q)
- **cgrType** & **operator=** (const CGR_TYPE &t_)
- bool **operator<** (const CGR_TYPE t_) const
- bool **operator<=** (const CGR_TYPE t_) const
- bool **operator>** (const CGR_TYPE t_) const
- bool **operator>=** (const CGR_TYPE t_) const
- bool **operator==** (const CGR_TYPE t_) const
- bool **operator!=** (const CGR_TYPE t_) const
- bool **operator<** (const **cgrType** &q) const
- bool **operator<=** (const **cgrType** &q) const
- bool **operator>** (const **cgrType** &q) const
- bool **operator>=** (const **cgrType** &q) const
- bool **operator==** (const **cgrType** &q) const
- bool **operator!=** (const **cgrType** &q) const
- const char * **tostr** () const
- **wbstring** **toStr** () const

Public Attributes

- CGR_TYPE **t**

The documentation for this class was generated from the following file:

- clebsch.hh

4.14 Wb::citerator< T, T2 > Class Template Reference

Public Member Functions

- **citerator** (const T *p_)
- **citerator** & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- const T2 & **operator[]** (size_t k)
- const T2 & **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- const T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.15 Wb::citerator< T, T2 * > Class Template Reference

Public Member Functions

- **citerator** (const T *p_)
- **citerator** & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- const T2 * **operator[]** (size_t k)
- const T2 * **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- const T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.16 Wb::CleanUp Class Reference

Public Attributes

- `int i`

The documentation for this class was generated from the following file:

- `wbllib.h`

4.17 Wb::Clock Class Reference

Public Member Functions

- **Clock** (`const char *s=0`)
- `void Init ()`
- `void init` (`const char *s=NULL`)
- `void reset ()`
- `void start ()`
- `void resume ()`
- `int stop` (`const char *F=NULL, int L=0`)
- `double gettime` (`char cflag=0`) `const`
- `mxAarray * toMx` (`const char *F=NULL, int L=0`)
- `mxAarray * toMxS` (`const char *F=NULL, int L=0`)
- `void info` (`const char *istr=NULL`) `const`

Public Attributes

- `double ttot`
- `double ctot`
- `double tneg`
- `CHRONO_CLOCK tref`
- `clock_t cref`
- `size_t nz`
- `size_t ncall`
- `wbstring name`

The documentation for this class was generated from the following files:

- `wbclock.hh`
- `wbclock.cc`

4.18 Wb::Clock_resume Class Reference

Public Member Functions

- **Clock_resume** (`Clock *t_`)
- `int stop` ()
- `int Switch` (`Clock *t_`)

Public Attributes

- [Clock](#) * t

The documentation for this class was generated from the following file:

- wblock.hh

4.19 clebsch::coefficients Class Reference

Public Member Functions

- **coefficients** (const [weight](#) &irrep, const [weight](#) &factor1, const [weight](#) &factor2)
- double **operator()** (int factor1_state, int factor2_state, int multiplicity_index, int irrep_state) const

Public Attributes

- const int **N**
- const [weight](#) **factor1**
- const [weight](#) **factor2**
- const [weight](#) **irrep**
- const int **factor1_dimension**
- const int **factor2_dimension**
- const int **irrep_dimension**
- const int **multiplicity**

The documentation for this class was generated from the following file:

- ClebschGordan_Arne.cpp

4.20 CPAT< TQ, TD > Class Template Reference

Public Member Functions

- **CPAT** (const [CPAT](#)< TQ, TD > &D)
- void **init** (TD oz, const mxArray **, unsigned n, unsigned ic1, unsigned r1, unsigned ic2, unsigned r2, const char twoSite, const char ldir)
- int **getCtrlIndex** (const [QSpace](#)< TQ, TD > &PSI)
- [QSpace](#)< TQ, TD > & **HZTimes** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &HZPsi, const char='N', char disp=1)
- [QSpace](#)< TQ, TD > & **HZApplyDiag** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &EPsi, TD a, TD b, char lflag=0, char disp=1)
- void **check_contraction** (const [QSpace](#)< TQ, TD > &PSI, char disp=1)
- void **initCounter** ()
- [QSpace](#)< TQ, TD > & **HZTimes_tst** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &HZPsi, const char='N', char disp=1)
- void **getQRRange** (unsigned k, [wbMatrix](#)< TQ > &QQ, [wbvector](#)< unsigned > &SS, [QSpace](#)< TQ, TD > *B=NULL) const
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base") const
- void **PermlC** (const [wbperm](#) &P)

Public Attributes

- [wbvector](#) < [wbvector](#) < [CPAT_Set](#) < TQ, TD > > > **CP**
- TD **z**
- unsigned **sameCtrlIdx**
- unsigned **nr_check_ctr**
- unsigned **HPsi_counter**

The documentation for this class was generated from the following file:

- cpat.hh

4.21 CPAT_Set< TQ, TD > Class Template Reference

Public Member Functions

- **CPAT_Set** (const [CPAT_Set](#) < TQ, TD > &D)
- int **init** (const mxArray *a, unsigned ic1, unsigned r1, unsigned ic2, unsigned r2, const char twoSite, const char ldir)
- int **getCtrlIndex** (const [QSpace](#) < TQ, TD > &PSI)
- int **getCtrlIndex** (const [wbvector](#) < [QSpace](#) < TQ, TD > > &, const char='N')
- int **getCtrlIndex** (const unsigned i, const [QSpace](#) < TQ, TD > &, char='N')
- void **contract** (const [QSpace](#) < TQ, TD > &PSI, [wbvector](#) < [QSpace](#) < TQ, TD > > &C, char='N') const
- void **contract** (const [wbvector](#) < [QSpace](#) < TQ, TD > > &PSI1, [wbvector](#) < [QSpace](#) < TQ, TD > > &C, char='N') const
- void **contract** (const unsigned k, const [QSpace](#) < TQ, TD > &PSI, [QSpace](#) < TQ, TD > &C, char='N') const
- void **contract_diag** (const [QSpace](#) < TQ, TD > &PSI, [wbvector](#) < [QSpace](#) < TQ, TD > > &C, TD a, TD b, char lflag) const
- void **contract_diag** (const [wbvector](#) < [QSpace](#) < TQ, TD > > &PSI1, [wbvector](#) < [QSpace](#) < TQ, TD > > &C, TD a, TD b, char lflag) const
- void **contract_diag** (const unsigned k, const [QSpace](#) < TQ, TD > &PSI, [QSpace](#) < TQ, TD > &C, TD a, TD b, char lflag) const
- mxArray * **toMx** () const

Public Attributes

- [wbvector](#) < [QSpace](#) < TQ, TD > > > **A**
- char **opA**
- unsigned **ic**
- [wbperm](#) **P**
- [wbvector](#) < [QSpace](#) < TQ, TD > > > **CQn**
- [wbvector](#) < [QSpace](#) < TQ, TD > > > **CQt**
- [wbvector](#) < [wbMatrix](#) < unsigned > > > **CIn**
- [wbvector](#) < [wbMatrix](#) < unsigned > > > **CI**

The documentation for this class was generated from the following file:

- cpat.hh

4.22 `cpu_time` Class Reference

Public Member Functions

- `void init ()`
- `cpu_time since ()`
- `wbstring toStr (char flag='c')`

Static Public Member Functions

- `static double time (char flag)`

Public Attributes

- `double tcpu`
- `double tsys`

The documentation for this class was generated from the following files:

- `util.hh`
- `util.cc`

4.23 `CRef< TQ >` Class Template Reference

Public Member Functions

- `CRef (const CRef &S)`
- `CRef (const CDATA_TQ &C, unsigned i=0, unsigned m=-1)`
- `CRef & operator= (const CRef &S)`
- `CRef & init (CGR_TYPE rt_=CGR_DEFAULT)`
- `CRef & init (const CRef &S)`
- `CRef & init (const char *F, int L, const CRef &S, unsigned i, const double &x)`
- `CRef & init (const char *F, int L, const mxArray *a, unsigned k, char refC=0, const QSet< TQ > *Q=NULL, char xflag=0)`
- `CRef< TQ > & initIdentityR (const char *F, int L, const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0)`
- `CRef< TQ > & initIdentity1J (const char *F, int L, const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0)`
- `CRef< TQ > & Reduce2Identity (char xflag=0)`
- `wbvector< RTD > & trace (const char *F, int L, wbvector< RTD > &cgt) const`
- `wbvector< double > & trace (const char *F, int L, wbvector< double > &cgt) const`
- `wbvector< double > trace (const char *F, int L) const`
- `double trace () const`
- `double trace (const char *F, int L, const ctrlIdx &i1, const ctrlIdx &i2, CRef *Rt=NULL) const`
- `CRef & initAbelian ()`
- `CRef & initCtrScalar (double x=1)`
- `CRef & initBase (const CDATA_TQ *cgr_, unsigned i=0, unsigned m=-1)`
- `CRef & initBase (const CDATA_TQ &C, unsigned i=0, unsigned m=-1)`
- `int LoadRef (const char *F=0, int L=0, char force=1) const`

- int **Reduce2Ref** (const char *F=0, int L=0, char force=0) const
- bool **isEmpty** () const
- bool **isSymmetric** (const char *F=0, int L=0) const
- bool **isReflnit** (const char *F=0, int L=0, char check=1) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **isScalar** (char dflag=0) const
- bool **isScalarCtr** () const
- int **check_cgw** (const char *F=0, int L=0) const
- int **checkQ** (const char *F, int L, const TQ *q, unsigned r, unsigned stride, const QType &t, const IndexLabels &it) const
- unsigned **numel** () const
- template<class T >
wbvector< T > & **getSize** (wbvector< T > &S, char bare=0) const
- unsigned **Size** (unsigned k, unsigned r=-1) const
- unsigned **qdim** (unsigned k, const QType *qt=NULL) const
- int **checkAbelian** (const char *F=NULL, int L=0) const
- bool **isDiagCSC** (RTD eps=1E-14) const
- bool **isIdentityCG** (double *nrm=NULL, double eps=1E-14) const
- char **sameUptoFac** (const CRef &B, double *fac=NULL, double eps=1E-12) const
- char **sameAs** (const CRef &B, double eps=1E-12) const
- bool **sameQSet** (const QSet< TQ > &Q) const
- bool **sameQSet** (const CRef &B) const
- bool **sameQDir** (const char *F, int L, const QDir &qd) const
- bool **sameQDir** (const char *F, int L, const CRef &B) const
- bool **sameQDir** (const IndexLabels &b) const
- int **gotSameCData** (const char *F, int L, const CRef &B, char c2flag=0) const
- double **NormStd** (const char *F, int L, unsigned r=-1)
- double **normExt** (const char *F=NULL, int L=0) const
- int **SortDegQ** (const char *F=NULL, int L=0, QSet< TQ > *QS=NULL)
- bool **isSortedDegQ** (QSet< TQ > *QS, wbperm &pxt, char &cxt) const
- bool **isSortedDegQ** (QSet< TQ > *QS=NULL) const
- char **got3** (const char *F, int L, const QType &q, const qset< TQ > &J12, const qset< TQ > &J3) const
- bool **operator==** (const CRef &B)
- bool **operator!=** (const CRef &B)
- RTD **scalarProd** (const char *F, int L, const CRef &B) const
- double **normDiff2** (const char *F, int L, const CRef &B) const
- double **normDiff** (const char *F, int L, const CRef &B) const
- RTD **sumTimesEI** (const char *F, int L, const CRef &B) const
- int **check** (const char *F, int L) const
- int **check** (const char *F, int L, const QType &q, const QDir &qdir) const
- double **norm2** (char xflag=0) const
- double **NormSignR** (const char *F=NULL, int L=0, double eps=CG_SKIP_DEPS1, double eps2=CG_SKIP_DEPS2)
- unsigned **getOM** (const char *F=NULL, int L=0) const
- unsigned **checkOM** (const char *F=NULL, int L=0) const
- unsigned **rankS** (char lflag=0) const
- unsigned **rank** (const char *F=NULL, int L=0, char lflag=0) const
- CRef & **Permute** (const wbperm &P, char iflag=0, char isnew=0)
- CRef & **permute** (const wbperm &P, CRef &B, char iflag=0, char isnew=0) const
- CRef & **Conj** ()
- int **HConjOpScalar** ()
- bool **gotConj** () const
- bool **gotPerm** (const char *F=NULL, int L=0) const
- bool **anyTrafo** (char pflag=1) const
- bool **anyTrafo** (char &ip, char &ic, char pflag=1) const

- bool **gotnoTrafos** () const
- unsigned **getP** (unsigned k) const
- [wbperm](#) & **getP** ([wbperm](#) &p, char iflag=0) const
- char **stat** () const
- bool **affectsQDir** () const
- [qset](#)< TQ > & **adapt** ([qset](#)< TQ > &q, char iflag=0) const
- [QSet](#)< TQ > & **adapt** ([QSet](#)< TQ > &Q, char iflag=0) const
- [ctrlIdx](#) & **adapt** ([ctrlIdx](#) &l, char iflag=0) const
- [QDir](#) & **get_qdir** ([QDir](#) &qd) const
- [QDir](#) **get_qdir** () const
- double **safeCpy** (const char *F, int L, const [CRef](#) &B, [CRef](#) &C) const
- [CRef](#) & **save2** ([CRef](#) &S)
- [wbstring](#) **qdir2Str** (char vflag=0) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **statStr** (char vflag=0) const
- [wbstring](#) **qStr** () const
- [wbstring](#) **QStr** () const
- mxArray * **toMx** (char flag=0) const
- mxArray * **toMX** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n, char flag=0) const
- void **add2MxStruct** (mxArray *S, unsigned i, char flag=0) const

Public Attributes

- const CDATA_TQ * **cgb**
- [wbvector](#)< double > **cgw**
- [wbperm](#) **cgp**
- char **conj**
- [cgrType](#) **rtype**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.24 CStore< TQ > Class Template Reference

Public Types

- typedef MAP32::iterator **IMAP32**
- typedef MAP31::iterator **IMAP31**

Public Member Functions

- void **clear** ()
- void **init** ()
- const CDATA_TQ & **getIdentityC** (const char *F, int L, const QType &t, const TQ *qs, unsigned dim=-1, unsigned loadRC=LB_GEN)
- const CDATA_TQ & **getIdentity1J** (const char *F, int L, CRef< TQ > &C, const QType &t, const TQ *qs, unsigned dim=-1, unsigned loadRC=LB_GEN)
- void **add_CData_abelian** (const QType &q, const TQ &J1, const TQ &J2)
- void **add_CData** (const QType &q, const qset< TQ > &J1, const qset< TQ > &J2)
- void **add2BUF** (const char *F, int L, const CDATA_TQ &S, char nflag=0)
- unsigned **add3** (const QType &q, const mxArray *S, char loadC=0)
- unsigned **add2BUF** (const QType &q, const mxArray *S, unsigned k=-1)
- CDATA_TQ & **getBUF** (const char *F, int L, const QSet< TQ > &Q, unsigned loadRC=0)
- size_t **reduceMemUsage** (const char *F=NULL, int L=0, double mfac=-1)
- void **getQfinal** (const char *F, int L, const QType &type, const qset< TQ > &J1, const qset< TQ > &J2, wbMatrix< TQ > &J, wbMatrix< const wbvector< CRef< TQ > > * > *ss=NULL, char loadC=0)
- void **getQfinal** (const char *F, int L, const QVec &type, const qset< TQ > &q1, const qset< TQ > &q2, wbMatrix< TQ > &QQ, wbMatrix< const wbvector< CRef< TQ > > * > *SS=NULL, char loadC=0)
- int **getQfinal_zdim** (const char *F, int L, const QVec &qvec, const wbMatrix< TQ > &Q1, const wbMatrix< TQ > &Q2, const wbMatrix< TQ > &Q, wbvector< widx_t > &s1, wbvector< widx_t > &s2, wbvector< widx_t > &s, wbvector< unsigned > *m=NULL, wbMatrix< const wbvector< CRef< TQ > > * > *S3=NULL)
- void **getQfinal** (const char *F, int L, const QVec &type, const wbMatrix< TQ > &Q1, const wbMatrix< TQ > &Q2, QMap< TQ > &M, unsigned char cgflag=0)
- void **Info** (const char *F=0, int L=0, char vflag=0) const
- int **checkInit** (const char *F, int L, const QType &t) const
- unsigned **LoadStore** (const char *F, int L, const char *file)
- mxArray * **toMx** (const QType &q=QTYPE_UNKNOWN) const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **put** (const char *vname, const char *ws="caller") const
- int **find_map3** (const QType &q, const qset< TQ > &J1, const qset< TQ > &J2) const
- int **find_map3** (const QType &q, const qset< TQ > &J12) const
- int **valid_mp3_data** (const QType &t, const qset< TQ > &J1, const qset< TQ > &J2, const qset< TQ > &J3, char rflag=0) const
- CDATA_TQ * **BUF_find** (const QSet< TQ > &Q)
- int **Reduce2Ref** (const char *F, int L, const QSet< TQ > &Q, char xflag=0)

Public Attributes

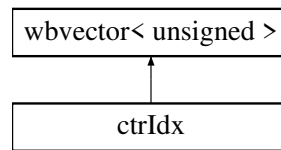
- map< QType, map< qset< TQ >, map< qset< TQ >, wbvector< CRef< TQ > > > > > **map3**
- unordered_map< QSet< TQ >, CDATA_TQ, QHash< TQ > > **BUF**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc
- clebsch_old.cc

4.25 ctrlIdx Class Reference

Inheritance diagram for ctrlIdx:



Public Member Functions

- **ctrlIdx** (unsigned l=0, unsigned *d=NULL, char c=0)
- **ctrlIdx** (const char *F, int L, const char *s, char c=0)
- **ctrlIdx** (const char *F, int L, const mxArray *a)
- **ctrlIdx** (const [ctrlIdx](#) &l)
- **ctrlIdx** (const [wbvector](#)< unsigned > &l, char c=0)
- **ctrlIdx & init** (unsigned l=0, unsigned *d=NULL, char c=0)
- **template<class T >**
ctrlIdx & initT (unsigned l=0, T *d=NULL, char c=0)
- **ctrlIdx & init** (const [wbvector](#)< unsigned > &l, char c=0)
- **ctrlIdx & init1** (unsigned k, char c=0)
- **ctrlIdx & init2** (unsigned k1, unsigned k2, char c=0)
- **template<class TQ, class TD >**
ctrlIdx & initOp (const char *F, int L, [iTag](#) t1, const [QSpace](#)< TQ, TD > &B, const char *sopt)
- **int init** (const char *F, int L, const char *s, unsigned offset=1)
- **ctrlIdx & init** (const char *F, int L, const mxArray *a)
- **ctrlIdx & init** (const mxArray *a)
- **ctrlIdx & Index** (unsigned l, char c=0)
- **int checkUniques** (const char *F=0, int L=0, const unsigned r=-1) const
- **ctrlIdx & Sort** ([wbperm](#) &P, unsigned r=-1)
- **ctrlIdx & sort** ([ctrlIdx](#) &l, [wbperm](#) &P, unsigned r=-1) const
- **ctrlIdx & Sort_** ([ctrlIdx](#) &J, unsigned r=-1)
- **ctrlIdx & invert** (unsigned r, [ctrlIdx](#) &l) const
- **ctrlIdx & Invert** (unsigned r)
- **ctrlIdx & Conj** ()
- **int extend2Perm** (widx_t N, [wbperm](#) &P, char toend=0) const
- **wbperm & extend2Perm** (unsigned ra, unsigned ma, const [ctrlIdx](#) &icb, unsigned rb, unsigned mb, const [wbperm](#) &pab, char conj, unsigned mc, [wbperm](#) &Pbc, [ctrlIdx](#) &kb, [ctrlIdx](#) &kcb, char Bflag=1) const
- **ctrlIdx & save2** ([ctrlIdx](#) &J)
- **mxArray * toMx** () const
- **wbstring toStr** (char xflag=0) const

Public Attributes

- char **conj**
- [IndexLabels](#) **newtags**

Additional Inherited Members

The documentation for this class was generated from the following files:

- wbindex.hh
- wbindex.cc

4.26 clebsch::decomposition Class Reference

Public Member Functions

- **decomposition** (const [weight](#) &factor1, const [weight](#) &factor2)
- int **size** () const
- const [clebsch::weight](#) & **operator**() (int j) const
- int **multiplicity** (const [weight](#) &irrep) const

Public Attributes

- const int **N**
- const [weight](#) **factor1**
- const [weight](#) **factor2**

The documentation for this class was generated from the following file:

- ClebschGordan_Arne.cpp

4.27 CG::FileLock Class Reference

Public Member Functions

- template<class TQ >
FileLock (const [QType](#) &q_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const char *ext="mp3")
- template<class TQ >
int **init** (const [QType](#) &q_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const char *ext="mp3")
- int **GetLock** (const char *F=NULL, int L=0, char flag='W')
- int **ReleaseLock** (const char *F=NULL, int L=0)

Public Attributes

- int **fid**
- [QType](#) **q**
- [wbstring](#) **fname**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.28 `genRG_base< TQ, TD >` Class Template Reference

Public Member Functions

- `bool isEmpty () const`
- `double normDiff (const char *F, int L, const genRG\_base &B, TD eps=1E12) const`
- `void checkOrthoCommRel (const char *F, int L, const genRG\_base &B) const`
- `genRG\_base & Sort ()`
- `double SkipTiny (const char *F=0, int L=0)`
- `genRG\_base & ApplyQFac (const wbvector< double > &qfac, const wbarray< double > *JM=NULL)`
- `genRG\_base & operator= (const genRG\_base &S)`
- `genRG\_base & init ()`
- `SPIDX_T dim () const`
- `void compareStdSU2 (const char *F=NULL, int L=0) const`
- `genRG\_base & save2 (genRG\_base &S)`
- `wbstring info () const`
- `mxArray * toMx () const`
- `size_t memSize () const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i) const`

Public Attributes

- `QType q`
- `wbvector< wbSpararrayTD > Sp`
- `wbvector< wbSpararrayTD > Sz`
- `qset< TQ > J`
- `wbMatrix< double > Z`
- `double err`
- `wbstring istr`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch.cc`
- `clebsch_aux.cc`
- `clebsch_io.cc`

4.29 `genRG_struct< TQ, TD >` Class Template Reference

Public Member Functions

- `genRG\_struct & checkInit (const char *F, int L, const QType &q0)`
- `genRG\_struct & initBase (const char *F=NULL, int L=0, qset< TQ > *qs=NULL)`
- `genRG\_struct & initBase_SUN (const char *F, int L, qset< TQ > *qs=NULL)`
- `genRG\_struct & initBase_SpN (const char *F, int L, qset< TQ > *qs=NULL)`
- `genRG\_struct & initBase_SON (const char *F, int L, qset< TQ > *qs=NULL)`
- `genRG\_struct & initBase_SEN (const char *F, int L, qset< TQ > *qs=NULL)`
- `void initSU2 (const qset< TQ > &J)`
- `void initCommRel (const char *F, int L, const genRG\_base< TQ, TD > &R, char vflag=0)`

- double **checkCommRel** (const char *F, int L, const [genRG_base](#)< TQ, TD > &R) const
- void **printStatus** (const char *F, int L, const char *istr=NULL)
- int **get1J_gen** (const char *F, int L, const [qset](#)< TQ > &q, char flag=0)
- int **getTensorProdReps_gen** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, unsigned flag=TP3_LDM)
- void **getTensorProdReps** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2)
- unsigned **getTensorProdReps** (const [qset](#)< TQ > &q, unsigned flag=0)
- unsigned **getTensorProdReps** (unsigned dmax=-1, char sdig=0, char vflag=0)
- void **put** (const char *vname, const char *ws="caller") const
- mxArray * **toMx** (char bflag=1) const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const

Public Attributes

- [QType](#) **q**
- map< [qset](#)< TQ >, [genRG_base](#)< TQ, TD > > **RSet**
- [qset](#)< TQ > **qs**
- [wbvector](#)< [pairPatternCG](#)< double > > **CR**
- [wbMatrix](#)< double > **DZ**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.30 Wb::gpara__ Class Reference

Public Member Functions

- void **init** ()

Public Attributes

- char **x**

The documentation for this class was generated from the following file:

- wblib.h

4.31 groupIndex< T > Class Template Reference

Public Member Functions

- **groupIndex** (const [wbvector](#)< T > &P_, const [wbvector](#)< T > &D_, const wperm_t **p=NULL)
- **groupIndex** (const [wbvector](#)< T > &DI, T ng=-1)
- void **init** ()
- [groupIndex](#) & **init** (const [wbvector](#)< T > &P_, const [wbvector](#)< T > &D_)
- [groupIndex](#) & **initX** ([wbvector](#)< wperm_t > &P_, [wbvector](#)< T > &D_, const wperm_t **p=NULL)
- [groupIndex](#) & **init** (const [wbvector](#)< T > &DI, T ng=-1)
- [groupIndex](#) & **init** (T ng)
- [groupIndex](#) & **init_g** (T ig, T nz)
- [groupIndex](#) & **setD** (T i, T nz)
- T **init_ic** ()
- [groupIndex](#) & **setup** (char xflag=0)
- T **dim** (T ig)
- T **numGroups** () const
- void **getGroup** (T ig, T *&p0, T &d) const
- T **getGroup** (T ig, T &j, T &j2) const
- T **getGroup** (T ig) const
- template<class T2 >
void **getRecs0** (const [wbMatrix](#)< T2 > &, [wbMatrix](#)< T2 > &, T m) const
- T **groupIter** (T &ig, T &j, T &j2) const
- mxArray * **toMx** () const

Public Attributes

- [wbvector](#)< T > **D**
- [wbperm](#) **P**
- [wbvector](#)< T > **I0**
- [wbvector](#)< T > **Ig**

The documentation for this class was generated from the following files:

- wbindex.hh
- wbindex.cc

4.32 icFlags Class Reference

Public Member Functions

- **icFlags** (const mxArray *ain=NULL)
- [icFlags](#) & **checkQSpace** (const mxArray *a)
- char **check_arg** (const mxArray *ain)
- [icFlags](#) & **init** (const mxArray *ain)
- char **set** (const mxArray *a, const char *istr="")
- template<class TQ, class TD >
void **apply** ([QSpace](#)< TQ, TD > &X)

Public Attributes

- const mxArray * **a**
- [wbstring](#) **ktags**
- [wbstring](#) **otags**
- char **conj**

The documentation for this class was generated from the following file:

- contractQS.cc

4.33 clebsch::index_adapter Class Reference

Public Member Functions

- **index_adapter** (const [clebsch::decomposition](#) &decomp)
- int **size** () const
- int **operator()** (int j) const
- int **multiplicity** (int irrep) const

Public Attributes

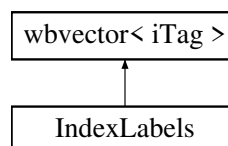
- const int **N**
- const int **factor1**
- const int **factor2**

The documentation for this class was generated from the following file:

- ClebschGordan_Arne.cpp

4.34 IndexLabels Class Reference

Inheritance diagram for IndexLabels:



Public Member Functions

- **IndexLabels** (unsigned n=0)
- **IndexLabels** (const char *F, int L, const char *s)
- **IndexLabels** (const char *s)
- unsigned **init** (const char *F, int L, const char *s)
- unsigned **init** (const char *s)
- **IndexLabels** & **init** (unsigned n=0)
- **IndexLabels** & **init** (const **IndexLabels** &b)
- **IndexLabels** & **operator=** (const **IndexLabels** &b)
- **IndexLabels** & **MatchSet** (const **IndexLabels** &b)
- unsigned **init** (const char *F, int L, const mxArray *a)
- **IndexLabels** & **init3** ()
- **IndexLabels** & **init** (size_t l1, const **iTag** *d1, size_t l2, const **iTag** *d2)
- unsigned **Init** (const char *F, int L, const char *s, unsigned k=-1, char lflag=-1)
- **IndexLabels** & **initOp** (**iTag** t, const **IndexLabels** &b, const char *ot="op")
- **IndexLabels** & **SetConj** (const char *F, int L, unsigned k)
- **IndexLabels** & **SetFlag** (const char *F, int L, unsigned k, unsigned l)
- **IndexLabels** & **UnsetFlag** (const char *F, int L, unsigned k, unsigned l)
- **IndexLabels** & **UnsetFlags** ()
- **IndexLabels** & **Rmltags** ()
- bool **operator==** (const **IndexLabels** &b) const
- bool **operator!=** (const **IndexLabels** &b) const
- int **findRegEx** (const char *r) const
- unsigned **match** (const **IndexLabels** &b, **ctrlIdx** &ia, **ctrlIdx** &ib, const char tflag=0) const
- bool **sameConj** (const **IndexLabels** &b) const
- char **same** (unsigned i, unsigned j) const
- bool **isOp** (unsigned r=-1) const
- bool **isOpX** (unsigned r=-1) const
- bool **isAtensor** (unsigned r=-1) const
- bool **got_qdir** (const char *qdir, char fflag=0) const
- unsigned **SkipLast** (unsigned n=1)
- **wbstring** **toStr** (char vflag=0) const
- const char * **toStrk** (char *sout, unsigned l, unsigned k, const char *st="(itag?)" , char cflag=0) const
- mxArray * **toMx** () const
- int **got_op_labels** (const char *F=NULL, int L=0) const
- unsigned **gotLabels** () const

Additional Inherited Members

The documentation for this class was generated from the following files:

- `wbindex.hh`
- `wbindex.cc`

4.35 `indexSparseRef< TD >` Class Template Reference

Public Member Functions

- **indexSparseRef** (const **wbsparray**< TD > &A_, unsigned M2=1)
- **indexSparseRef** (const char *F, int L, const **wbsparray**< TD > &A_, unsigned M2=1)
- **indexSparseRef** & **init** (const char *F, int L, const **wbsparray**< TD > &A_, unsigned M2=1)
- SPIDX_T **numCols** () const
- SPIDX_T **numRows** () const
- SPIDX_T **nnz** (SPIDX_T k) const
- TD **LContractVec** (const **wbsparray**< TD > &b, SPIDX_T k, char tnorm=0)
- mxArray * **toMx** () const

Public Attributes

- unsigned **M**
- [wbvector](#)< SPIDX_T > **JA**
- [wbvector](#)< sSPIDX_T > **JJ**
- const [wbsparray](#)< TD > * **A**

The documentation for this class was generated from the following file:

- wbsparray.hh

4.36 Wb::IOstat Class Reference

Public Member Functions

- [IOstat](#) & **init** ()
- void **gotread** (size_t s=0)
- void **gotwrite** (size_t s=0)
- bool **isEmpty** ()
- void **print** (const char *istr, const char *hstr=0, const char *xstr1=0, const char *xstr2=0)

Public Attributes

- unsigned **nread**
- unsigned **nwrite**
- size_t **sread**
- size_t **swrite**
- size_t **aux1**
- size_t **aux2**

The documentation for this class was generated from the following files:

- util.hh
- util.cc

4.37 is_pointer_< T > Class Template Reference

Public Member Functions

- char **q** ()

The documentation for this class was generated from the following file:

- wblibx.h

4.38 `is_pointer_< T * >` Class Template Reference

Public Member Functions

- `char q ()`

The documentation for this class was generated from the following file:

- `wbllibx.h`

4.39 `is_pointer_< T ** >` Class Template Reference

Public Member Functions

- `char q ()`

The documentation for this class was generated from the following file:

- `wbllibx.h`

4.40 `is_pointer_< T *** >` Class Template Reference

Public Member Functions

- `char q ()`

The documentation for this class was generated from the following file:

- `wbllibx.h`

4.41 `iTag` Class Reference

Public Member Functions

- `iTag` (const char *F, int L, const char *s)
- `iTag` (const char *s)
- `iTag` (const `iTag` &x)
- `iTag` & `init` (const char *F, int L, const char *s, unsigned n=-99)
- `iTag` & `init` (const char *s=NULL)
- `iTag` & `init_qdir` (char c)
- `template<class TQ, class TD >`
`iTag` & `init` (const char *F, int L, const `wbvector< QSpace< TQ, TD > >` &A, `wbindex` ia)
- `iTag` & `init` (const char *F, int L, const `mxArray` *a)
- `iTag` & `operator=` (unsigned n)

- `iTag & operator=` (const char *s)
- `bool operator==` (const `iTag` &x) const
- `bool operator!=` (const `iTag` &x) const
- `bool operator!` () const
- `char sameAs` (const `iTag` &x, char lflag=0) const
- `bool isConj` (const `iTag` &x, char full=1) const
- `bool isConjx` (const `iTag` &x, char lflag) const
- `int isValid` () const
- `bool isEmpty` () const
- `operator bool` () const
- `iTag & tSet` (const `iTag` &x)
- `iTag & Set_` (const `iTag` &x, char keep=0)
- `iTag & Conj` ()
- `iTag & setConj` ()
- `bool isConj` () const
- `iTag & deConj` ()
- `iTag conj` () const
- `iTag deconj` () const
- `iTag & SetFlag` (unsigned k)
- `iTag & UnsetFlag` (unsigned k)
- `iTag & UnsetFlags` ()
- `iTag & Rmltag` ()
- `iTag & AppendChar` (char q, const char *qs=NULL)
- `iTag & PrependChar` (char q, const char *qs=NULL)
- `int CheckFirstChar` (char q, const char *qs=NULL)
- `iTag & SetK` ()
- `iTag & SetD` ()
- `iTag & SetM` ()
- `iTag & AddTilde` ()
- `unsigned to_str` (char *s, unsigned len, char cflag=1) const
- `wbstring toStr` (char cflag=1) const
- `mxAarray * toMx` () const

Public Attributes

- `IDT t`

The documentation for this class was generated from the following files:

- `wbindex.hh`
- `wbindex.cc`

4.42 Wb::iterator< T, T2 > Class Template Reference

Public Member Functions

- `iterator` (T *p_)
- `iterator & operator++` ()
- `bool end` ()
- `size_t iter` () const
- `size_t numel` () const
- `T2 & operator[]` (size_t k)
- `T2 & at` ()

Public Attributes

- `size_t i`
- `size_t n`
- `T * p`

The documentation for this class was generated from the following file:

- `util.hh`

4.43 Wb::iterator< T, T2 * > Class Template Reference

Public Member Functions

- `iterator (T *p_)`
- `iterator & operator++ ()`
- `bool end ()`
- `size_t iter () const`
- `size_t numel () const`
- `T2 * operator[] (size_t k)`
- `T2 * at ()`

Public Attributes

- `size_t i`
- `size_t n`
- `T * p`

The documentation for this class was generated from the following file:

- `util.hh`

4.44 iterSparseCMref< TD > Class Template Reference

Public Member Functions

- `iterSparseCMref (const char *F, int L, wbsparray< TD > &A)`
- `SPIDX_T next (wbsparray< TD > &a)`
- `wbsparray< TD > & getIter (SPIDX_T i, wbsparray< TD > &a, char ref=0) const`
- `TD norm2 (SPIDX_T i, char tnorm=0) const`
- `SPIDX_T numIter () const`
- `bool gotLast () const`
- `SPIDX_T index () const`
- `mxArray * toMx () const`

Public Attributes

- SPIDX_T **it**
- [wbvector](#)< SPIDX_T > **IO**
- [wbvector](#)< SPIDX_T > **IX**
- [wbMatrix](#)< SPIDX_T > **IDX**

The documentation for this class was generated from the following file:

- wbsparray.hh

4.45 Wb::line_stat__ Class Reference

Public Member Functions

- void **add** (double dt_, double dx)

Public Attributes

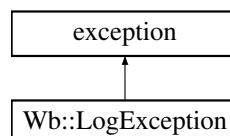
- double **dt**
- double **aux**
- size_t **ncall**

The documentation for this class was generated from the following file:

- wblock.hh

4.46 Wb::LogException Struct Reference

Inheritance diagram for Wb::LogException:



Public Member Functions

- **LogException** (WBLOG_TYPE l=OK, const char *s=NULL)
- **LogException** (const [LogException](#) &e)
- [LogException](#) & **operator=** (const [LogException](#) &e)
- [LogException](#) & **operator++** ()
- [LogException](#) & **operator+=** (const [LogException](#) &e)
- bool **operator!** () const
- **operator bool** () const
- [LogException](#) & **init** ()
- const char * **what** () const throw ()
- void **report** (const char *F=0, int L=0, const char *func=0)

Public Attributes

- `WBLOG_TYPE` **type**
- `int * nrefs`
- `char istr [64]`
- `unsigned count`
- `int ith`
- `int nth`

The documentation for this struct was generated from the following files:

- `wblog.h`
- `wblog.c`

4.47 Wb::matFile Class Reference

Public Member Functions

- **matFile** (const char *F, int L, const char *fname, const char *rw)
- **matFile** (const [matFile](#) &X)
- void **close** ()
- void **init** ()
- int **Open** (const char *F, int L, const char *FName, const char *RW)
- int **open** (const char *file, int line, const char *FName, const char *RW, char xflag=0)
- [matFile](#) & **put** (const char *file, int line, const char *vname, const mxArray *a)
- [matFile](#) & **put** (const char *vname, const mxArray *a)
- int **existsVar** (const char *vn) const
- int **deleteVar** (const char *vn) const
- int **deleteVar** (const char *f, int L, const char *vn) const

Public Attributes

- `MATFile * mfp`
- [wbstring](#) **F**
- [wbstring](#) **fname**
- [wbstring](#) **rw**
- `int L`

The documentation for this class was generated from the following file:

- `wbMatIO.hh`

4.48 MEM_SET< T > Class Template Reference

Public Member Functions

- **MEM_SET** (T *data __attribute__((unused)), size_t len __attribute__((unused)), const T *a __attribute__((unused))=NULL)
- **MEM_SET** (int *data, size_t len, const int *a)
- **MEM_SET** (unsigned *data, size_t len, const unsigned *a)
- **MEM_SET** (unsigned long *data, size_t len, const unsigned long *a)
- **MEM_SET** (long *data, size_t len, const long *a)
- **MEM_SET** (char *data, size_t len, const char *a)
- **MEM_SET** (double *data, size_t len, const double *a)
- **MEM_SET** (float *data, size_t len, const float *a)
- **MEM_SET** (wbcomplex *data, size_t len, const wbcomplex *a)

The documentation for this class was generated from the following file:

- memlib.hh

4.49 MEM_SET< T * > Class Template Reference

Public Member Functions

- **MEM_SET** (T **data, size_t len, T *const *a=NULL)

The documentation for this class was generated from the following file:

- memlib.hh

4.50 MHash< TM > Class Template Reference

Public Member Functions

- size_t **operator()** (const cgc_contract_id< TM > &S) const

The documentation for this class was generated from the following file:

- clebsch.hh

4.51 Wb::mpfr__ < P > Class Template Reference

Public Member Functions

- **mpfr__** (double val=0)
- **mpfr__** (const char *F, int L, double val)
- **mpfr__** (const [mpfr__](#) &b)
- void **Cleanup** (const char *F=NULL, int L=0) const
- **mpfr__** & **init** (const char *F, int L, double val, char lflag=0)
- **mpfr__** & **init_s** (const char *F, int L, const char *s, unsigned base=10)
- **mpfr__** & **init_d** (const char *F, int L, double val, char dcheck=1)
- **mpfr__** & **set_pi** ()
- **mpfr__** & **set_log2** ()
- **mpfr__** & **set_e** ()
- **mpfr__** & **set_nan** ()
- **mpfr__** & **set_inf** (int s=+1)
- **mpfr__** & **set_zero** (int s=+1)
- int **isnan** () const
- int **isinf** () const
- int **iszero** () const
- int **isnumber** () const
- int **isnonzero** () const
- int **sign** ()
- void **swap** ([mpfr__](#) < P > &b)
- int **cmp** (const [mpfr__](#) &b) const
- int **cmp** (const double &val) const
- int **cmp** (const int &val) const
- int **cmp** (const long &val) const
- int **cmp** (const unsigned &val) const
- int **cmp** (const unsigned long &val) const
- bool **operator>** (const [mpfr__](#) &x) const
- bool **operator>** (const double &x) const
- bool **operator>** (const int &x) const
- bool **operator>** (const long &x) const
- bool **operator>** (const unsigned &x) const
- bool **operator>** (const unsigned long &x) const
- bool **operator<** (const [mpfr__](#) &x) const
- bool **operator<** (const double &x) const
- bool **operator<** (const int &x) const
- bool **operator<** (const long &x) const
- bool **operator<** (const unsigned &x) const
- bool **operator<** (const unsigned long &x) const
- bool **operator>=** (const [mpfr__](#) &x) const
- bool **operator>=** (const double &x) const
- bool **operator>=** (const int &x) const
- bool **operator>=** (const long &x) const
- bool **operator>=** (const unsigned &x) const
- bool **operator>=** (const unsigned long &x) const
- bool **operator<=** (const [mpfr__](#) &x) const
- bool **operator<=** (const double &x) const
- bool **operator<=** (const int &x) const
- bool **operator<=** (const long &x) const
- bool **operator<=** (const unsigned &x) const
- bool **operator<=** (const unsigned long &x) const

- `bool operator!= (const mpfr__ &x) const`
- `bool operator!= (const double &x) const`
- `bool operator!= (const int &x) const`
- `bool operator!= (const long &x) const`
- `bool operator!= (const unsigned &x) const`
- `bool operator!= (const unsigned long &x) const`
- `bool operator== (const mpfr__ &x) const`
- `bool operator== (const double &x) const`
- `bool operator== (const int &x) const`
- `bool operator== (const long &x) const`
- `bool operator== (const unsigned &x) const`
- `bool operator== (const unsigned long &x) const`
- `mpfr__ & operator= (double val)`
- `mpfr__ & operator= (const mpfr__ &b)`
- `mpfr__ & times (const mpfr__ &b, mpfr__ &c) const`
- `mpfr__ & divby (const mpfr__ &b, mpfr__ &c) const`
- `mpfr__ & operator*= (const mpfr__ &b)`
- `mpfr__ & operator/= (const mpfr__ &b)`
- `mpfr__ & operator+= (const mpfr__ &b)`
- `mpfr__ & operator-= (const mpfr__ &b)`
- `mpfr__ & operator*= (double val)`
- `mpfr__ & operator/= (double val)`
- `mpfr__ & operator+= (double val)`
- `mpfr__ & operator-= (double val)`
- `mpfr__ operator* (const mpfr__ &b) const`
- `mpfr__ operator/ (const mpfr__ &b) const`
- `mpfr__ operator+ (const mpfr__ &b) const`
- `mpfr__ operator- (const mpfr__ &b) const`
- `mpfr__ operator* (int n) const`
- `mpfr__ operator/ (int n) const`
- `mpfr__ operator+ (int n) const`
- `mpfr__ operator- (int n) const`
- `mpfr__ operator* (unsigned n) const`
- `mpfr__ operator/ (unsigned n) const`
- `mpfr__ operator+ (unsigned n) const`
- `mpfr__ operator- (unsigned n) const`
- `mpfr__ operator* (long n) const`
- `mpfr__ operator/ (long n) const`
- `mpfr__ operator+ (long n) const`
- `mpfr__ operator- (long n) const`
- `mpfr__ operator* (unsigned long n) const`
- `mpfr__ operator/ (unsigned long n) const`
- `mpfr__ operator+ (unsigned long n) const`
- `mpfr__ operator- (unsigned long n) const`
- `mpfr__ & Sqrt ()`
- `mpfr__ & Exp ()`
- `mpfr__ & Log ()`
- `mpfr__ & Log2 ()`
- `mpfr__ & Atan ()`
- `mpfr__ & Tanh ()`
- `mpfr__ & Atanh ()`
- `mpfr__ & Rand ()`
- `mpfr__ & Log10 ()`
- `mpfr__ & Round ()`
- `mpfr__ sqrt () const`

- [mpfr__exp](#) () const
- [mpfr__log](#) () const
- [mpfr__log2](#) () const
- [mpfr__atan](#) () const
- [mpfr__log10](#) () const
- [mpfr__round](#) () const
- **operator double** () const
- **operator wbcomplex** () const
- **operator bool** () const
- double **getval_d** () const
- bool **operator<** (double val) const
- bool **operator<=** (double val) const
- bool **operator>** (double val) const
- bool **operator>=** (double val) const
- int **prec** (unsigned base=2) const
- [mpfr__eps](#) ()
- [mpfr__](#) & **FlipSign** ()
- [mpfr__](#) & **Abs** ()
- [mpfr__abs](#) () const
- [mpfr__fabs](#) () const
- [mpfr__abs2](#) () const
- void **print** (const char *vname="", const char *istr=0, char sep=0) const
- [size_t to_str](#) (char *s, [size_t](#) n_, int base=10) const
- [wbstring to_str](#) (int base=10) const
- void **toStr** (char *s, unsigned n, int base=10, int nd=-1, char align=0) const
- [wbstring](#) & **toStr** ([wbstring](#) &sout, int base=10, int nd=-1, char align=0) const

Public Attributes

- [mpfr_t f](#)

Static Public Attributes

- static unsigned long **num_vars** =0

The documentation for this class was generated from the following files:

- [wbmpfr.hh](#)
- [wbmpfr.cc](#)

4.52 Wb::MREC Class Reference

Public Member Functions

- template<class T >
const char * **init** (const char *F __attribute__((unused)), int L __attribute__((unused)), [size_t](#) i, const T *p __attribute__((unused)), [size_t](#) l)
- void **println** ([size_t](#) i, void *p) const

Public Attributes

- `size_t id`
- `size_t len`
- `unsigned unit`

The documentation for this class was generated from the following file:

- `memtrack.cc`

4.53 mxArray_buf Class Reference

Public Member Functions

- **mxArray_buf** (const char *n=NULL)
- void **add** (void *p, mxArray *a)
- mxArray * **find** (const char *F, int L, void *p)
- mxArray * **Return** (const char *F, int L, void *p)
- int **erase** (void *p)

Public Attributes

- `unordered_map< void *, mxArray * > BUF`
- `char * name`

The documentation for this class was generated from the following file:

- `mexlib.hh`

4.54 MXPut Class Reference

Public Member Functions

- **MXPut** (const char *vn=0, const char *ws=0)
- **MXPut** (const char *file, int line, const char *vn="ans", const char *ws=0)
- void **init** ()
- void **init** (const char *file, int line, const char *vn=NULL)
- void **put** (const char *ws=0, const char *vn=0)
- void **save2** (mxArray *&S0)
- mxArray * **toMx** ()
- void **add2Struct** (mxArray *S0)
- **MXPut** & **addFL** ()
- template<class T >
MXPut & **add** (const T &x, const char *fn)
- template<class T >
MXPut & **addP** (const T *x, const char *fn)
- template<> **MXPut** & **addP** (const mxArray *a, const char *fn)
- template<> **MXPut** & **add** (const double &x, const char *fn)

- `template<> MXPut & add (const float &x, const char *fn)`
- `template<> MXPut & add (const long double &x, const char *fn)`
- `template<> MXPut & add (const wbcplx &x, const char *fn)`
- `template<> MXPut & add (const char &x, const char *fn)`
- `template<> MXPut & add (char *const &s, const char *fn)`
- `template<> MXPut & add (const char *const &s, const char *fn)`
- `template<> MXPut & add (const unsigned &x, const char *fn)`
- `template<> MXPut & add (const int &x, const char *fn)`
- `template<> MXPut & add (const bool &x, const char *fn)`
- `template<> MXPut & add (const long &x, const char *fn)`
- `template<> MXPut & add (const unsigned long &x, const char *fn)`

Public Attributes

- `mxArray * S`
- `char wsp [16]`
- `char * vname`
- `char * F`
- `unsigned L`

Static Public Attributes

- `static const unsigned wlen =16`

The documentation for this class was generated from the following file:

- `mexlib.hh`

4.55 mxStruct Class Reference

Public Member Functions

- `mxStruct (const char *file, int line)`
- `void init (const char *file, int line)`
- `void put (const char *name)`
- `void save2Struct (mxArray *S0, const char *name)`
- `void checkNewField (const char *f=NULL, int l=0)`
- `mxStruct & operator() (const char *fn)`
- `template<class T > mxStruct & operator= (const T &x)`
- `mxStruct & operator= (mxArray *a)`
- `template<class T > mxStruct & operator= (const wvector< T > &x)`
- `template<class T > mxStruct & operator= (const wbMatrix< T > &x)`
- `template<> mxStruct & operator= (const wperm &P)`
- `template<> mxStruct & operator= (const windex &I)`

Public Attributes

- mxArray * **S**
- [wbstring](#) **F**
- [wbstring](#) **fId**
- unsigned **L**

The documentation for this class was generated from the following file:

- mexlib.hh

4.56 Wb::myname__ Class Reference

Public Attributes

- char **data** [64]

The documentation for this class was generated from the following file:

- wblib.h

4.57 NRGData< TQ, TD > Class Template Reference

Public Member Functions

- **NRGData** (const char *s=NULL)
- **NRGData** (const [NRGData](#) &A)
- void **initName** (const char *n)
- void **init** (const char *F, int L, const mxArray *a, [wbvector](#)< [QSpace](#)< TQ, TD > > &C0, int k=0, char force=0, const char *vname0=0)
- void **init** (const char *F, int L, const char *XX, int iter=-1)
- void **initXX** (const char *F, int L, int m=4, int iter=-1)
- void **initX** (const char *F, int L, int m=2, int iter=-1)
- void **init** (char flag=0)
- void **setupMX** (const char *F, int L, int N, char store_=1, char locMX_=1)
- void **PUT** (const char *F, int L, const char *vn, const char *ws="caller")
- void **check_dloc** (const char *F, int L, const char *fun, unsigned dloc, unsigned iter) const
- template<class TD_>
int **checkOpConsistency** (const char *F, int L, const [NRGData](#)< TQ, TD_ > &B) const
- bool **isQSpaceVec** (const char *nm, int iter=-1)
- bool **checkVec** (const char *F, int L, const char *XX, unsigned R, const char *istr="")
- bool **checkVecVec** (const char *XX, unsigned r)
- void **forceCalc** ()
- void **getRelevantOps** ([wbindex](#) &l) const
- [wbvector](#)< [QSpace](#)< TQ, TD > > & **getOpsXX** (const char *t)
- const [QSpace](#)< TQ, TD > & **getQSpace** (const char *t, unsigned i=0) const
- [QSpace](#)< TQ, TD > & **getQSpace** (const char *t, unsigned i=0)
- const [QSpace](#)< TQ, TD > & **getQSpace** (char t1, char t2, unsigned i=0) const

- [QSpace](#)< TQ, TD > & **getQSpace** (char t1, char t2, unsigned i=0)
- const [QSpace](#)< TQ, TD > & **getQSpace** (char t) const
- [QSpace](#)< TQ, TD > & **getQSpace** (char t)
- void **info** (const char *vstr=NULL) const
- void **displdx** (const char *F, int L) const
- void **info** (const char *F, int L, const char *istr=" * ") const
- void **disp_otype** (const char *F=0, int L=0) const
- const mxArray * **getMxInfo** (const char *n)
- template<class TX >
void **getMxInfo** (const char *F, int L, const char *v, TX &X)
- mxArray * **getMxData** (const char *nm, int iter=-1, char ionly=0)
- const char * **wnrg** ()
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base")
- void **put** (const char *F, int L, const char *vname, const char *ws="base")
- template<class T >
int **getPara** (const char *name, T &x, int iter=-1, char i=0)
- bool **opExists** (char tflag=1)
- void **initOp** (unsigned n=1, char calc_=1, char store_=0)
- bool **initOp** (const char *F, int L, const [wbvector](#)< [QSpace](#)< TQ, TD > > &C0, char tflag=1, [NRGData](#)< TQ, TD > *CR=NULL, const char *istr="")
- bool **initOp** (const char *F, int L, const [wbvector](#)< [wbvector](#)< [QSpace](#)< TQ, TD > > &CC, [NRGData](#)< TQ, TD > *CR=NULL, const char *istr="", char tflag=1)
- void **initSym** (const char *tag)
- void **updatePara** (const char *F, int L, const char *tag, mxArray *, int iter=-1) const
- void **updatePara** (const char *F, int L, const char *tag, int iter=-1, const char *tag2=NULL, char all=-1) const
- void **updateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A, const [NRGData](#)< TQ, TD > &B, unsigned nop, int iter=-1)
- template<class TC >
void **updateOpProd** (const char *F, int L, unsigned iter, const [NRGData](#)< TQ, TD > &B, const [NRGData](#)< TQ, TC > &C)
- void **updateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A, unsigned nop, int iter=-1)
- template<class TA >
void **backPropagateOp** (const char *F, int L, const [NRGData](#)< TQ, TA > &A0, const [NRGData](#)< TQ, TA > &A2)
- template<class TD_ >
void **initBG** (const char *F, int L, const [NRGData](#)< TQ, TD_ > &B, const [NRGData](#)< TQ, TD_ > &H1, const [NRGData](#)< TQ, TD_ > &H2, int m=3)
- void **unsetAllRefs** ()
- unsigned **skipOpZeros** (double eps=DEPS, char bflag='b')
- void **saveOp** (const char *, int, int iter=-1, unsigned m=4) const
- void **times** (TD fac, const char *flag)
- void **setupIO** (const char *, int, const char *, const mxArray *, const char *istr=0)
- void **updateInfo** (const char *F, int L, const char *, mxArray *, char keep=-1) const
- void **updateInfof** (const char *F, int L, const char *vtag, mxArray *, char keep=-1) const
- const char * **getFileName** (unsigned k, char lenient=0) const
- template<class TD >
void **updateOpProd** (const char *F, int L, unsigned iter, const [NRGData](#)< TQ, TS > &B, const [NRGData](#)< TQ, TD > &C)
- template<class TD >
void **initBG** (const char *F, int L, const [NRGData](#)< TQ, TD > &B, const [NRGData](#)< TQ, TD > &H0, const [NRGData](#)< TQ, TD > &H2, int m)
- template<class TD >
void **backPropagateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A0, const [NRGData](#)< TQ, TD > &A2)

Public Attributes

- [wbstring](#) **name**
- [wbvector](#)< [NRGIndex](#)< TQ, TD > > **nrgldx**
- char **calc**
- char **store**
- int **kloc**
- char **locMX**
- [wbvector](#)< [wbvector](#)< [QSpace](#)< TQ, TD > > > **CI**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **KK**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **KD**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **DK**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **DD**
- [QSpace](#)< TQ, TD > **K**
- [QSpace](#)< TQ, TD > **D**
- [wbvector](#)< char > **issym**
- mxArray * **MX**
- [wbstring](#) **NAME**
- mxArray * **aux**

The documentation for this class was generated from the following file:

- nrgdata.hh

4.58 [NRGIndex](#)< TQ, TD > Class Template Reference

Public Member Functions

- **NRGIndex** (const [NRGIndex](#) &l)
- void **init** ()
- void **init** (int i, [NRGData](#)< TQ, TD > *d)
- void **init** (int i, const char *t)
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tst=0) const
- void **put** (const char *vname, const char *ws="base") const

Public Attributes

- int **idx**
- [NRGData](#)< TQ, TD > * **nrg**
- [wbstring](#) **tag**

The documentation for this class was generated from the following file:

- nrgdata.hh

4.59 Wb::num2Fmt< T > Class Template Reference

Public Member Functions

- **num2Fmt** (const char *F, int L, int m_=-2, int p_=-2, const char *t_="")
- int **check_init** ()
- **operator wstring** () const
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()

The documentation for this class was generated from the following files:

- gathered.hh
- gathered.cc

4.60 opFlags< T > Class Template Reference

Public Member Functions

- **opFlags** (const [opFlags](#) &f)
- **opFlags** (const char *F, int L, char f)
- **opFlags** (const [opFlags](#) &f1, const [opFlags](#) &f2)
- [opFlags](#)< T > & **operator=** (char f)
- [opFlags](#)< T > & **operator=** (const [opFlags](#)< T > &x)
- **opFlags** (char f)
- **operator char** ()
- [opFlags](#)< T > & **init** (const char *F, int L, char f)
- [opFlags](#)< T > & **init** (char f)
- [opFlags](#)< T > & **init** (const [opFlags](#)< T > &f1, const [opFlags](#)< T > &f2)
- char **trans** () const
- char **conj** () const
- char **idle** () const
- void **applyOrRef** (const char *F, int L, const [warray](#)< T > &A0, [warray](#)< T > &A, const T fac=1) const
- template<class T2 >
T2 & **applyOrRef** (const char *F, int L, const T2 &A0, const T fac=1) const
- template<class T2 >
const T2 & **applyConjOrRef** (const char *F, int L, const T2 &A0, const T fac=1) const
- template<class T2 >
char **apply** (const char *F, int L, T2 &A, const T fac=1) const
- char **toChar** () const
- mxArray * **toMx** () const
- void **info** (const char *F, int L, const char *istr="")

Public Attributes

- char **p**

The documentation for this class was generated from the following file:

- wbararray_blas.hh

4.61 OPTS Class Reference

Public Member Functions

- **OPTS** (const mxArray **ain, const unsigned n)
- **OPTS** (const char *fname)
- void **init** (const mxArray **ain, const unsigned n)
- void **init** (const char *fname)
- void **init** ()
- int **len** ()
- template<class T >
bool **getOpt** (const char *, int, const char *s, T &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *, int, const char *s, [wbvector](#)< T > &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *, int, const char *s, [wbMatrix](#)< T > &x, char force=0, const char *istr=0)
- bool **getOpt** (const char *, int, const char *s, const char *istr=0)
- template<class T >
bool **getOpt** (const char *s, T &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *s, [wbvector](#)< T > &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *s, [wbMatrix](#)< T > &x, char force=0, const char *istr=0)
- bool **getOpt** (const char *s, const char *istr=0)
- void **checkAnyLeft** (const char *MAT=NULL, int L=0)
- template<> bool **getOpt** (const char *F, int L, const char *s, [wbstring](#) &x, char force, const char *istr)
- template<> bool **getOpt** (const char *F, int L, const char *s, mxArray *a, char force, const char *istr)
- template<> bool **getOpt** (const char *F __attribute__((unused)), int L __attribute__((unused)), const char *s, [iTag](#) &t, char force, const char *istr)

The documentation for this class was generated from the following file:

- wbopts.hh

4.62 OrthoCGS< TQ > Class Template Reference

Public Member Functions

- [OrthoCGS](#) & **init** ()
- [OrthoCGS](#) & **init** (const [CRef](#)< TQ > &R)
- template<class TA , class TB >
int **contractCG_ortho** (const char *F, int L, const [QSpace](#)< TQ, TA > &A, [C_UVEC](#) &la, const [ctrlIdx](#) &ica, const [QSpace](#)< TQ, TB > &B, [C_UVEC](#) &lb, const [ctrlIdx](#) &icb, unsigned isym, char cg_preview)
- template<class TD >
unsigned **getCGS_ortho** (const char *F, int L, [QSpace](#)< TQ, TD > &A, [C_UVEC](#) &la, unsigned isym)
- unsigned **getOM** () const

Public Attributes

- [QSet](#)< TQ > **Q**
- const CDATA_TQ * **cgb**
- [wbperm](#) **cgp**
- char **conj**
- [cgrType](#) **rtype**
- [wbarray](#)< double > **cgQ**
- [wbarray](#)< double > **cgR**

The documentation for this class was generated from the following files:

- QSpace_aux.hh
- QSpace_aux.cc

4.63 pairPatternCG< T > Class Template Reference

Public Member Functions

- [pairPatternCG](#) & **init** (unsigned i0=0, unsigned j0=0, unsigned n=0)
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const

Public Attributes

- unsigned **i**
- unsigned **j**
- [wbvector](#)< unsigned > **k**
- [wbvector](#)< T > **fac**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_io.cc

4.64 Wb::Path Class Reference

Public Member Functions

- **Path** (const char *s)
- **Path** (const char *F, int L, const char *s)
- int **init** ()
- int **init** (const char *F, int L, const char *s, const char *delim=":", unsigned n_=-1)
- int **append** (const char *F, int L, const char *s, const char *delim=":")
- char * **operator[]** (unsigned i)
- const char * **operator[]** (unsigned i) const
- const char * **last** () const
- unsigned **numel** () const
- [Wb::iterator](#)< [Path](#), char * > **begin** ()
- [Wb::citerator](#)< [Path](#), char * > **cbegin** () const
- void **print** (const char *istr="") const

Public Attributes

- `wbstring` `buf`
- `wbvector` < unsigned > `idx`

The documentation for this class was generated from the following file:

- `wbutil.hh`

4.65 `clebsch::pattern` Class Reference

Public Member Functions

- `pattern` (const `pattern` &pat)
- `pattern` (const `weight` &irrep, int index=0)
- `int` & `operator()` (int k, int l)
- `const int` & `operator()` (int k, int l) const
- `bool` `operator++` ()
- `bool` `operator--` ()
- `int` `index` () const
- `clebsch::weight` `get_weight` () const
- `double` `lowering_coeff` (int k, int l) const
- `double` `raising_coeff` (int k, int l) const

Public Attributes

- `const int` `N`

The documentation for this class was generated from the following file:

- `ClebschGordan_Arne.cpp`

4.66 `Wb::Profile` Class Reference

Public Member Functions

- `Profile` (const char *F, int L, const char *istr=0)
- `void` `tic` ()
- `void` `toc` (int L, double x=0)

Public Attributes

- `CHRONO_CLOCK` `tref`

The documentation for this class was generated from the following file:

- `wbclock.hh`

4.67 QBlock< TQ, TD, TE > Class Template Reference

Public Member Functions

- [QBlock](#) & **init** ()
- [QBlock](#) & **operator=** (const [QBlock](#) &B)
- [QBlock](#) & **init_bare_refA** (const [QBlock](#) &B, unsigned w=0)
- void **toBlockMatrix** ([wbarray](#)< TD > &MM, unsigned K, const [wbperm](#) &P=[wbperm](#)())
- void **initFromBlockMatrix** ([QSpace](#)< TQ, TD > &B, const [wbarray](#)< TD > &MM, const [IndexLabels](#) &it, char flag, const [wbperm](#) *cgp=NULL)
- void **updateBlockDim** (unsigned dim2)
- double **get_s2t** (const [wbvector](#)< char > &mark)
- void **print** (const char *F=NULL, int L=0, const [IndexLabels](#) *t=NULL) const
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="caller")

Public Attributes

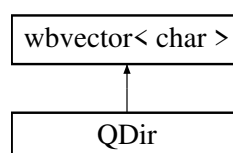
- [QSpace](#)< TQ, TD > **A**
- [wbindex](#) **l0**
- [wbindex](#) **lk**
- [wbindex](#) **lt**
- [wbMatrix](#)< TQ > **Q1**
- [wbMatrix](#)< TQ > **Q2**
- [wbMatrix](#)< widx_t > **S1**
- [wbMatrix](#)< widx_t > **S2**
- [wbMatrix](#)< widx_t > **D1**
- [wbMatrix](#)< widx_t > **D2**
- [wbMatrix](#)< int > **idx**
- [wbarray](#)< TD > **U**
- [wbarray](#)< TD > **V**
- [wbvector](#)< TE > **S**
- double **qdim**
- double **sfac**

The documentation for this class was generated from the following files:

- mpsortho.hh
- mpsortho.cc

4.68 QDir Class Reference

Inheritance diagram for QDir:



Public Member Functions

- **QDir** (unsigned l, const char *d=NULL, char ref=0)
- **QDir** (const [IndexLabels](#) &it)
- **QDir** (const char *F, int L, const char *s)
- **QDir** (const char *s)
- **QDir** (const char *F, int L, const mxArray *a)
- **QDir & init** (unsigned l=0, const char *d=NULL, char ref=0)
- **QDir & operator=** (const char *s)
- **QDir & init** (const char *F, int L, const char *s)
- **QDir & init** (const char *s)
- **QDir & init** (const [IndexLabels](#) &it)
- **QDir & init** (const char *F, int L, const mxArray *a)
- **QDir & init_iout** (const char *F, int L, unsigned r, unsigned iout)
- unsigned **isSorted** () const
- bool **operator==** (const char *s) const
- bool **operator!=** (const char *s) const
- bool **operator==** (const [QDir](#) &q) const
- bool **operator!=** (const [QDir](#) &q) const
- bool **operator<** (const [QDir](#) &q) const
- [QDir](#) & **Conj** ()
- char & **Conj** (const unsigned &i)
- unsigned **nconj** ()
- unsigned **nconj** (const [ctrlIdx](#) &lx)
- [wbstring](#) **toStr** () const
- [wbstring](#) **toTag** () const
- mxArray * **toMx** () const

Additional Inherited Members

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc

4.69 QHash< TQ > Class Template Reference

Public Member Functions

- size_t **operator()** (const [QSet](#)< TQ > &S, const char *istr=0) const

The documentation for this class was generated from the following file:

- clebsch.hh

4.70 QHash_ Class Reference

Public Member Functions

- `size_t operator() (const qset< double > &x) const`

The documentation for this class was generated from the following file:

- `wbhash.cc`

4.71 QMap< TQ > Class Template Reference

Public Member Functions

- `void init ()`
- `QMap< TQ > & init (const char *F, int L, const QVec &qv, const wbMatrix< TQ > &q1, const wbMatrix< TQ > &q2, const wbMatrix< wbMatrix< TQ > > &QQ, const wbMatrix< wbMatrix< const wbvector< CRef< TQ > > * > > *SS=NULL)`
- `QMap< TQ > & init (const char *F, int L, const mxArray *a)`
- `template<class TD > int getCGZlist (const char *F, int L, wbMatrix< TQ > &qc, wbvector< TD > &dc, wbvector< widx_t > *IC=NULL, const wbvector< widx_t > *Ix=NULL, const qset< TQ > *q3=NULL, wbIndex *m3=NULL, const wbvector< TD > *cc=NULL, double eps1=1E-10, double eps2=1E-14) const`
- `void Skip (const wbindex &l)`
- `template<class TD > void getIdentityQ (const char *F, int L, const wbvector< widx_t > &Sa, const wbvector< widx_t > &Sb, QSpace< TQ, TD > &A, char vflag=1) const`
- `bool isConsistent (const char *F=NULL, int L=0, const QVec *qv=NULL) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tst=0) const`
- `mxArray * toMx () const`
- `void put (const char *vname, const char *ws="caller") const`
- `void put (const char *F, int L, const char *vname, const char *ws="caller") const`

Public Attributes

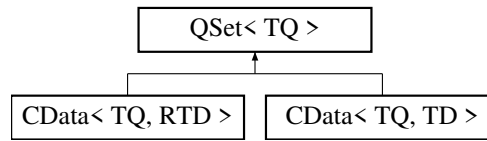
- `wbMatrix< TQ > Q1`
- `wbMatrix< TQ > Q2`
- `wbMatrix< TQ > Q`
- `wbvector< widx_t > I1`
- `wbvector< widx_t > I2`
- `wbvector< widx_t > D`
- `wbMatrix< widx_t > II`
- `wbMatrix< const wbvector< CRef< TQ > > * > cg3`
- `QVec qvec`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_aux.cc`
- `clebsch_io.cc`

4.72 QSet< TQ > Class Template Reference

Inheritance diagram for QSet< TQ >:



Public Member Functions

- **QSet** (const [QSet](#) &B)
- **QSet** (const [QType](#) &t_, const [qset](#)< TQ > &qset_, unsigned iout=0, char isref=0)
- **QSet** (const [QType](#) &t_, const [IndexLabels](#) &it, unsigned r)
- **QSet** (const [CRef](#)< TQ > &A)
- **QSet** (const char *F, int L, const mxArray *a)
- **QSet** (const mxArray *a)
- **QSet** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *P=NULL)
- **QSet** (const char *F, int L, const char *s)
- **QSet** & **init** (const [CRef](#)< TQ > &A)
- **QSet** & **init** (const char *F, int L, const mxArray *a, unsigned k=0)
- **QSet** & **init** (const [QType](#) t_=QTYPE_UNKNOWN)
- **QSet** & **init** (const [QSet](#) &B)
- **QSet** & **init2ref** (const [QSet](#) &B)
- **QSet** & **init** (const [QType](#) &t_, const TQ *q, const char *qd)
- **QSet** & **init** (const [QType](#) &t_, const [qset](#)< gTQ > &qset_, unsigned iout=0, char isref=0)
- **QSet** & **init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, unsigned iout=0)
- **QSet** & **init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, const [IndexLabels](#) &it)
- **QSet** & **init1J** (const [QType](#) &t_, const TQ *q, char iflag=0)
- **QSet** & **init2** (const [QType](#) &t_, const TQ *q=NULL)
- **QSet** & **init2** (const [QType](#) &t_, const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const char *qds="++")
- **QSet** & **init1** (const [QType](#) &t_, TQ *q=NULL)
- **QSet** & **init3** (const [QType](#) &t_)
- **QSet** & **init3** (const [QType](#) &t_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const [qset](#)< TQ > &J)
- **QSet** & **init3** (const [QType](#) &t_, const TQ &J1, const TQ &J2, const TQ &J)
- **QSet** & **init** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *Pcgd=NULL)
- **QSet** & **init_str** (const char *F, int L, const char *s)
- **QSet** & **Conj** ()
- **QSet** & **Conj** (unsigned i)
- unsigned **rank** (const char *F=NULL, int L=0) const
- **QSet** & **permute** ([QSet](#) &B, const [wbperm](#) &P, char iflag=0) const
- **QSet** & **Permute** (const [wbperm](#) &P, char iflag=0)
- **QSet** & **save2** ([QSet](#) &X)
- **QSet** & **operator=** (const [QSet](#) &B)
- bool **operator==** (const [QSet](#) &B) const
- bool **operator!=** (const [QSet](#) &B) const
- char **cmp** (const [QSet](#)< TQ > &B) const
- bool **operator<** (const [QSet](#)< TQ > &B) const
- bool **isEmpty** (char iflag=0) const
- bool **gotRCData** () const

- int **isZero** () const
- unsigned **isStd3** () const
- unsigned **checkStd3** () const
- int **init2opt3** (const **QSet**< TQ > &Q, **wbperm** &P, **wbvector**< char > &cflags)
- **QSet** & **Convert** (const char *s, **wbvector**< char > &cflags)
- **wbstring** **sizeStr** () const
- bool **isScalar** () const
- bool **is1JSymbol** () const
- bool **isAbelian** () const
- int **checkQ_abelian** (const char *F, int L) const
- int **checkQ_SU2** (const char *F, int L) const
- **QSet** & **Sort** (**wbperm** *cgp=NULL, char *conj=NULL, char iflag=0)
- bool **isSorted** () const
- size_t **memSize** () const
- **wbstring** **QStrS** (const **wbperm** *cgp=NULL, char sep='|') const
- **wbstring** **QStr** () const
- **wbstring** **toStr** (const char *istr=NULL) const
- **wbstring** **toTag** () const
- mxArray * **toMx** () const

Public Attributes

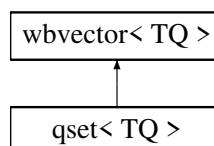
- **QType** t
- **qset**< TQ > qs
- **QDir** qdir

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.73 qset< TQ > Class Template Reference

Inheritance diagram for qset< TQ >:



Public Member Functions

- **qset** (unsigned l, const TQ *d0=NULL, char ref=0)
- **qset** (const [qset](#)< TQ > &q)
- **qset** (const [QType](#) &Q, const TQ *qq)
- **qset** (const [QVec](#) &Q, const TQ *qq)
- **qset** (const char *F, int L, const mxArray *a)
- **qset** (const char *F, int L, const mxArray *a, const [QVec](#) *Q, char check_type=1)
- **qset** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2)
- **qset** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const [qset](#)< TQ > &q3)
- **qset** (const [qset](#)< TQ > &q1, char op, const [qset](#)< TQ > &q2)
- [qset](#) & **init** (const [qset](#)< TQ > &q1)
- [qset](#) & **init** (const char *F, int L, const char *s, [QType](#) *t=NULL)
- [qset](#) & **init** (const char *F, int L, const mxArray *a, const [QVec](#) *Q=NULL, char check_type=1)
- [qset](#) & **init** (const [QType](#) &Q, const TQ *qq)
- [qset](#) & **init** (const [QVec](#) &Q, const TQ *qq)
- [qset](#) & **init** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2)
- [qset](#) & **init** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const [qset](#)< TQ > &q3)
- [qset](#) & **init** ()
- [qset](#) & **init** (unsigned l, const TQ *d0=NULL)
- [qset](#) & **init** (unsigned l1, const TQ *d1, unsigned l2, const TQ *d2)
- [qset](#) & **init** (unsigned l1, const TQ *d1, unsigned l2, const TQ *d2, unsigned l3, const TQ *d3)
- [qset](#)< TQ > & **init** (const TQ q1)
- [qset](#)< TQ > & **init** (const TQ q1, const TQ q2)
- [qset](#)< TQ > & **init** (const TQ q1, const TQ q2, const TQ q3)
- [qset](#)< TQ > & **initStride** (const TQ *d_, unsigned n, unsigned r, unsigned QDIM)
- bool **operator**< (const [qset](#)< TQ > &b) const
- int **checkNQs** (const [QVec](#) &qq, const char *F=NULL, int L=0)
- bool **isConsistent** (const [QVec](#) &qq, int rank=-1, const char *F=NULL, int L=0) const
- unsigned **QDim** (const [QVec](#) &q, int rank=-1) const
- [qset](#)< TQ > & **times** (TQ fac, [qset](#)< TQ > &x) const
- [wbstring toStr](#) () const
- [wbstring toStr](#) (const [QType](#) &t __attribute__((unused))) const
- bool **operator**< (const [qset](#)< double > &b) const
- bool **operator**< (const [qset](#)< float > &b) const

Additional Inherited Members

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc

4.74 QSpace< TQ, TD > Class Template Reference

Public Member Functions

- **QSpace** (unsigned n=0)
- **QSpace** (unsigned m, unsigned n, unsigned nq)
- **QSpace** (const **QSpace** &B)
- template<class TQ_, class TD_>
QSpace (const **QSpace**< TQ_, TD_ > &B, char xflag=**WbUtil**< TD_ >::isComplex() ? 1 :0)
- **QSpace** (const char *F, int L, const mxArray *a, char ref=0)
- **QSpace** (const mxArray *a, char ref=0)
- **QSpace** & **clearQSpace** ()
- void **clearDATA** ()
- unsigned **rank** (const char *F=NULL, int L=0) const
- unsigned **len** () const
- unsigned **nsym** () const
- TQ * **Qref** (unsigned r, unsigned k)
- const TQ * **Qref** (unsigned r, unsigned k) const
- void **init** ()
- void **init** (unsigned m, unsigned n, unsigned nq)
- void **init** (const **wbMatrix**< TQ > &Q, unsigned qdim, const **QVec** *qt=NULL)
- **QSpace** & **init** (const **QSpace** &B)
- template<class TB >
QSpace & **initT** (const **QSpace**< TQ, TB > &B)
- template<class TR >
QSpace & **getReal** (**QSpace**< TQ, TR > &R) const
- **QSpace**< TQ, double > **getReal** () const
- template<class TI >
QSpace & **getImag** (**QSpace**< TQ, TI > &I) const
- **QSpace**< TQ, double > **getImag** () const
- void **init** (const char *file, int line, const mxArray *S, char ref=0)
- void **init** (const char *file, int line, const mxArray *S, char ref, unsigned k)
- **QSpace** & **setupDATA** (const char *file=NULL, int line=0)
- int **gotCGS** (const char *F=NULL, int L=0) const
- int **gotCGX** (const char *F=NULL, int L=0) const
- **QSpace** & **Enlarge** (unsigned m)
- **QSpace** & **Trim** (unsigned n)
- **QSpace** & **initScalar** (const TD &x)
- void **init2ref** (const mxArray *S)
- void **init2ref** (const **QSpace** &)
- void **initOpZ_WET** (const char *F, int L, const **QVec** &qvec, const **wbMatrix**< TQ > &Q1, const **wbMatrix**< TQ > &Q2, const **wbMatrix**< TQ > &Q, const **wbarray**< TD > &D3, const double eps1=1E-10, const double eps2=1E-13)
- void **reduceMatEI** (const char *F, int L, const **wbMatrix**< TQ > &Q1, const **wbvector**< TD > &dd, const **wbMatrix**< TQ > &Q2, const **wbvector**< TD > &dc, **wbvector**< unsigned > &l1, **wbvector**< unsigned > &l2, **wbvector**< TD > &dr) const
- void **unRef** ()
- void **initIdentity** (const **QSpace** &A, char dflag=0)
- void **initIdentity** (const **wbMatrix**< TQ > &Q)
- **QSpace** & **setupCGR** (const char *F=NULL, int L=0)
- **QSpace** & **initIdentityCGS** (const char *F=NULL, int L=0)
- **QSpace** & **initIdentityCGS** (const char *F, int L, unsigned k, unsigned r=0)
- template<class TD2 >
QSpace & **initIdentityCGS** (const char *F, int L, const **QSpace**< TQ, TD2 > &H)

- `template<class TA >`
`QSpace< TQ, TD > & initIdentityCG` (const `wbvector< QSpace< TQ, TA > >` &A, const `wbindex` &ia, char zflag=0)
- `template<class TA , class TB >`
`QSpace< TQ, TD > & initIdentityCG` (const `wbvector< QSpace< TQ, TA > >` &A, const `wbindex` &ia, const `wbvector< QSpace< TQ, TB > >` &B, const `wbindex` &ib, char vflag='v')
- `template<class TA , class TB >`
`QSpace< TQ, TD > & initIdentityCG` (const `wbvector< QSpace< TQ, TA > >` &A, const `wbvector< QSpace< TQ, TB > >` &B, const char *pstr=NULL)
- `QSpace & initDiagonal` (const `wbMatrix< TQ >` &Q, const `wbvector< unsigned >` &S, const `wbvector< TD >` &D, char rflag)
- `QSpace & init2DiffOp` (const `QSpace` &A, unsigned ia, const `QSpace` &B, unsigned ib)
- `template<class TB >`
`QSpace & initQT_safe` (const `QSpace< TQ, TB >` &B)
- `template<class TA >`
`QSpace & initQ` (const `QSpace< TQ, TA >` &A, char full=1)
- `template<class TA , class TB >`
`QSpace & initQ` (const char *F, int L, const `QSpace< TQ, TA >` &A, char full=1, const `QSpace< TQ, TB >` *B=NULL)
- void `initQ` (unsigned n, unsigned r, const `QSpace` &A)
- void `initQ` (const `wbMatrix< TQ >` &Qk, const `QSpace` &A)
- void `init_itags` (const char *F, int L, const char *s, unsigned r=-1, char lflag=-1)
- `QSpace & SetTag` (unsigned k, const `iTag` &b)
- `QSpace & SetTags` (const `iTag` &b)
- `QSpace & SetConjTags` ()
- `QSpace & SetConjTags` (unsigned k1, unsigned k2)
- `QSpace & SetConjTag` (unsigned k)
- `QSpace & SetConjTag` (const char *F, int L, unsigned k)
- `QSpace & SetFlag` (const char *F, int L, unsigned k, unsigned l)
- `QSpace & UnsetFlag` (const char *F, int L, unsigned k, unsigned l)
- `QSpace & UnsetFlags` ()
- `QSpace & Rmltags` ()
- `template<class TB >`
 unsigned `matchITags` (const char *F, int L, const `QSpace< TQ, TB >` &B, `ctrlidx` &ia, `ctrlidx` &ib) const
- unsigned `RemoveZLabels` (const char *F, int L, `wbMatrix< TQ >` &QQ, `wbMatrix< TQ >` *Z=NULL) const
- void `RemoveZLabels` (const char *F, int L, `wbMatrix< TQ >` *Z)
- void `ExpandDiagonal` (unsigned i1=1, unsigned i2=2)
- `size_t getDataSize` (`wbMatrix< size_t >` &S) const
- `size_t getDataSize` (`wbvector< size_t >` &S) const
- `size_t getDataSize` () const
- `size_t getCGSSize` () const
- `wbstring totSize2Str` () const
- `wbstring itags2Str` () const
- unsigned `map2Vec` (const char *F, int L, `wbvector< TD >` &V, const `wbvector< unsigned >` &S=`wbvector< unsigned >` ())
- unsigned `map2Vec` (TD *v, char lflag=0)
- void `ExpandQ` (`CPAT< TQ, TD >` &CP, double nrm, `wbvector< unsigned >` &xflag, char disp=0)
- void `ResetRec` (unsigned k)
- void `setData` (const TD c)
- void `setRand` (double nrm=0)
- void `markUnequalZero` (const TD m=1, const TD eps=0)
- void `PrependSingletons` (unsigned R)
- void `swap` (`QSpace` &B)
- `QSpace & save2` (`QSpace` &B)
- unsigned `Reduce2AbelianOp` (const char *F=NULL, int L=0)

- char **allAbelian** () const
- char **isNonAbelian** () const
- bool **permitsOM** () const
- bool **hasOM** () const
- bool **allAdditive** () const
- bool **isEmpty** () const
- bool **isConsistent** (int &r, const char *F=NULL, int L=0, char level=2) const
- bool **isConsistent** (const char *F=NULL, int L=0, char level=2) const
- bool **isConsistentR** (int r, const char *F=NULL, int L=0, char level=2) const
- bool **isScalar** () const
- bool **isComplex** () const
- bool **isDiagBlock** (unsigned k) const
- bool **isBlockDiagMatrix** (const char *F, int L, char dflag=0) const
- bool **isBlockDiagMatrix** (char dflag=0) const
- bool **isDiagMatrix** (const double eps=1e-14) const
- bool **isIdentityMatrix** (const TD eps=1e-14) const
- char **hasIdentityCGS** (const TD eps=1e-14) const
- bool **isR3Op** () const
- int **isOperator** (unsigned *dr, char xflag=0) const
- bool **isOperator** () const
- bool **isAtensor** (unsigned r=-1) const
- int **checkAbelianOp** () const
- bool **isAbelianOp** () const
- bool **isQSym** (const char *F=0, int L=0, char dflag=0) const
- bool **isHConj** (const char *F=0, int L=0, double eps=1e-14, char vflag=0) const
- bool **isAHerm** (const char *F=0, int L=0, double eps=1e-14, char vflag=0) const
- int **isSingleton** (unsigned k) const
- template<class T2 >
bool **hasSameQ** (const QSpace< TQ, T2 > &B) const
- bool **hasSameQ** (unsigned r1, unsigned k1, unsigned r2, unsigned k2) const
- template<class T2 >
bool **hasSameRank** (const QSpace< TQ, T2 > &B) const
- char **checkQ** (const char *F=NULL, int L=0) const
- template<class TB >
void **checkQ** (const char *F, int L, const QSpace< TQ, TB > &B, char aflag=0) const
- char **checkQ_CGR** (const char *F, int L, unsigned k=-1) const
- int **check_CGR_cgw** (const char *F=0, int L=0) const
- void **checkScalarCGS** (const char *F=0, int L=0) const
- template<class TB >
bool **sameType** (const QSpace< TQ, TB > &B, int r=-1, const char *istr=NULL) const
- QDir & **getQDir** (QDir &qdir) const
- bool **sameQDir** (const char *F, int L, const QSpace &B) const
- bool **hasQOverlap** (const QSpace &A) const
- template<class TB >
int **hasQOverlap** (unsigned ica, const QSpace< TQ, TB > &B, unsigned icb, const char type=0) const
- int **getQOverlapU** (const char *F, int L, const QSpace &B, wbMatrix< TQ > &QQ, wbMatrix< int > &IQ, char olonly=0) const
- int **getQOverlap** (const char *F, int L, const QSpace &B, wbMatrix< TQ > &QQ, wbMatrix< int > &IQ, wbvector< widx_t > &DQ) const
- bool **findDimQ** (const TQ *q, unsigned &d) const
- bool **findDimQ** (const TQ *q, unsigned k, unsigned &d) const
- bool **operator==** (const QSpace &B) const
- bool **operator!=** (const QSpace &B) const
- template<class T >
void **operator*=** (const T fac)

- `template<class T >`
`QSpace & times` (const T fac, `QSpace &B`) const
- `QSpace & operator=` (const `QSpace &A`)
- `void operator+=` (const `QSpace &Q`)
- `void operator-=` (const `QSpace &Q`)
- `QSpace & Cat` (const char *F, int L, const `QSpace< TQ, TD > &A`, const `QSpace< TQ, TD > &B`, TD afac=1, TD bfac=1, char uflag=1)
- `QSpace & plus` (const `QSpace &B`, `QSpace &C`, TD bfac=1, char vflag=0) const
- `QSpace & minus` (const `QSpace &B`, `QSpace &C`, TD bfac=1, char vflag=0) const
- `QSpace & Plus` (const `QSpace &A`, TD bfac=1)
- `QSpace & Minus` (const `QSpace &A`, TD bfac=1)
- `QSpace & plus_plain` (const `QSpace &B`, `QSpace &C`, TD bfac=1) const
- `QSpace & NormCG` (char skipzeros=1)
- `TD norm2` () const
- `TD norm` () const
- `TD normDiff2` (const `QSpace &B`) const
- `TD scalarProd` (const `QSpace &B`) const
- `void checkNorm` (const char *F, int L, double nrm, char vflag=1, double eps=1E-12) const
- `QSpace & TimesEI` (const `QSpace &B`, char conj=0)
- `QSpace & TimesEI_plain` (const `QSpace &B`, char conj=0)
- `QSpace & TimesEI_OM` (const `QSpace &B`, char conj=0)
- `TD sumData` () const
- `template<class TM >`
`wbMatrix< TM > & diag` (`wbMatrix< TM > &xd`) const
- `QSpace & TensorProd` (const `QSpace &B`, `QSpace &C`, const char aflag='N', const char bflag='N') const
- `template<class TC , ENABLE_IF_COMPLEX(TC) >`
`QSpace< TQ, TC > & TensorProd` (const `QSpace &B`, `QSpace< TQ, TC > &C`, const char aflag='N', const char bflag='N') const
- `template<class TC , ENABLE_IF_COMPLEX(TC) >`
`QSpace< TQ, TC > & TensorProd` (const `QSpace< TQ, TC > &B`, `QSpace< TQ, TC > &C`, const char aflag='N', const char bflag='N') const
- `void TensorProdUnity` (const `wbMatrix< TQ > &Q`, `QSpace &C`) const
- `void EigenSymmetric` (`wbMatrix< double > &Etot`) const
- `void EigenSymmetric` (`QSpace &Ak`, `QSpace< TQ, double > &Ek`, `wbMatrix< double > &Etot`) const
- `void EigenSymmetric` (`QSpace &Ak`, `QSpace< TQ, double > &Ek`) const
- `void EigenSymmetric` (`QSpace &Ak`, `QSpace &Ad`, `QSpace< TQ, double > &Ek`, `QSpace< TQ, double > &Et`, `wbMatrix< double > &Etot`, `wbMatrix< unsigned > &DD`, int &Nkeep, double Etrunc=0, double *E0=NULL, const `wbperm &P=wbperm()`, double eps=0., double b=0., int dmax=-1, const char *sort=NULL) const
- `void GroupIndices` (unsigned K)
- `void Append` (unsigned k)
- `void Append` (const `wbvector< TQ > &`, const `wbarray< TD > &D`)
- `QSpace & Append` (const char *F, int L, const `QSpace &A`, char uflag=1)
- `int Append2AndDestroy` (const char *F, int L, `QSpace &A`, char unique=0)
- `template<class TA , ENABLE_IF_COMPLEX(TA) >`
`int Append2AndDestroy` (const char *F, int L, `QSpace< TQ, TA > &A`, char unique=0)
- `void TransferSpace` (const char *F, int L, `QSpace &B`, unsigned k, const `QSpace< TQ, double > &W`, double eps)
- `void TransferSpace` (const char *F, int L, `QSpace &B`, unsigned k)
- unsigned `makeUnique` (char omFlag=-1)
- unsigned `makeUnique_plain` (char omFlag=-1)
- unsigned `makeUnique_ortho` (char vflag=0)
- bool `isZero` (const double eps=0., char flag=0) const
- unsigned `SkipZeroData` (double eps=0., char bflag=0, char cgflag=1, char all=0)
- unsigned `skipZeroOffDiag` (double eps=0., char bflag=0)

- unsigned **SkipEmptyData** (const char *F=0, int L=0)
- bool **gotZeroData** (unsigned i, const double eps=0.) const
- void **Set** (unsigned k, const [wbvector](#)< TQ > &Q, const [wbarray](#)< TD > &D)
- void **SetQ** (unsigned i, unsigned k, const TQ *d)
- [QSpace](#) & **Select** (const [wbindex](#) &I, char data_only=0)
- template<class TM >
[QSpace](#) & **SelectMarked** (const [wbvector](#)< TM > &mark)
- void **Sort** (const [QSpace](#) &A)
- void **Sort** ()
- unsigned **getOM** ([wbMatrix](#)< unsigned > &OM, [wbvector](#)< unsigned > *om1=NULL, [wbMatrix](#)< TQ > *Q=NULL) const
- unsigned **cgsDimScalar** (unsigned k) const
- unsigned **cgsDim** (unsigned k, unsigned r) const
- unsigned **getDim** (unsigned k, widx_t *D=NULL) const
- unsigned **getDIM** (unsigned k) const
- void **getDim** ([wbvector](#)< widx_t > &D, [wbvector](#)< widx_t > *DD=NULL) const
- [wbvector](#)< widx_t > **getDim** () const
- void **getQDim** (unsigned k, [wbMatrix](#)< TQ > &Q, [wbvector](#)< widx_t > &S, [wbMatrix](#)< widx_t > *SC=NULL) const
- void **getQDim** ([wbMatrix](#)< TQ > &Q, [wbvector](#)< widx_t > &S, [wbMatrix](#)< widx_t > *SC=NULL) const
- void **getDRange** (TD &dmin, TD &dmax) const
- void **getDistQtot** ([wbMatrix](#)< TQ > &Qtot, [wbvector](#)< double > &w) const
- widx_t **getDQtot** ()
- [wbMatrix](#)< TQ > & **getQsub** (const [wbindex](#) &I, [wbMatrix](#)< TQ > &QI) const
- [wbMatrix](#)< TQ > & **getQsub** (const unsigned k, [wbMatrix](#)< TQ > &Qk, char iflag=0) const
- void **getQsub** (const [wbindex](#) &I, [wbMatrix](#)< TQ > &QI, [wbMatrix](#)< TQ > &Q2) const
- [wbMatrix](#)< TQ > **getQsub** (const unsigned k) const
- [wbMatrix](#)< TQ > & **getQtot** ([wbMatrix](#)< TQ > &Qtot, char uflag=0) const
- [wbMatrix](#)< TQ > & **getQsum** (const [wbindex](#) &J, [wbMatrix](#)< TQ > &Q) const
- [wbMatrix](#)< TQ > & **getQsum** (const [wbMatrix](#)< TQ > &Q0, [wbMatrix](#)< TQ > &Q) const
- [wbMatrix](#)< TQ > & **getQfinal** (const [wbindex](#) &J, [wbMatrix](#)< TQ > &Q) const
- [QSpace](#) & **getSub** (const [wbindex](#) &I, [QSpace](#) &A, char ref=0) const
- [QSpace](#) & **getSubInit** (const [wbindex](#) &I, [QSpace](#) &A) const
- void **saveSub2** ([QSpace](#) &A, const [wbindex](#) &I)
- void **toFull** ([wbarray](#)< TD > &A, [wbvector](#)< [wbMatrix](#)< TQ > > &Q, [wbvector](#)< [wbvector](#)< unsigned > > &S) const
- void **toFull2** ([wbarray](#)< TD > &A, [wbMatrix](#)< TQ > &Q, [wbMatrix](#)< widx_t > &S) const
- void **ExtendQ** (const [QSpace](#) &B)
- void **takeMainQS** (char disp=1)
- void **setQ** (unsigned k, const [QSpace](#) &B, unsigned k0)
- void **setQRec** (unsigned r, unsigned j, const TQ *q0, const TQ *q2=NULL)
- bool **hasEqualQ** (unsigned i1, unsigned i2) const
- double **maxDiff** (const [QSpace](#) &B) const
- void **PermuteQ** (const [wbperm](#) &P)
- void **permuteQ** (const [wbperm](#) &P, [wbMatrix](#)< TQ > &) const
- [QSpace](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [QSpace](#) & **permute** (const [wbperm](#) &P, [QSpace](#) &B, char iflag=0) const
- void **permute** (const char *pstr, [QSpace](#) &B, unsigned offset=1) const
- void **Permute** (const char *pstr, unsigned offset=1)
- void **PermuteFirstTo** (unsigned k)
- void **permuteFirstTo** (unsigned k, [QSpace](#) &C)
- void **PermuteLastTo** (unsigned k)
- void **permuteLastTo** (unsigned k, [QSpace](#) &C)
- void **FlipQ** ()
- [QSpace](#) & **trace** (const char *F, int L, [ctrlidx](#) &i1, [ctrlidx](#) &i2, [QSpace](#) &C) const

- **TD trace** () const
- void **contractMat** (unsigned ia, const [QSpace](#) &B, unsigned ib, [QSpace](#) &C) const
- template<class TB , class TC >
int **contract_getIdxSet** (const char *F, int L, const [ctrlIdx](#) &ica, const [QSpace](#)< TQ, TB > &B, const [ctrlIdx](#) &icb, [wbindex](#) &la, [wbindex](#) &lb, [wbvector](#)< widx_t > &Dc, [QSpace](#)< TQ, TC > &C) const
- template<class TB , class TC >
int **contract** (const char *F, int L, const [ctrlIdx](#) &ica, const [QSpace](#)< TQ, TB > &B, const [ctrlIdx](#) &icb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1, char cg_preview=0) const
- template<class TB , class TC >
int **contract** (const char *F, int L, unsigned ia, const [QSpace](#)< TQ, TB > &B, unsigned ib, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1) const
- template<class TB , class TC >
int **contract** (const char *F, int L, unsigned ia, const [QSpace](#)< TQ, TB > &B, const char *sb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1) const
- template<class TB , class TC >
int **contract** (const char *F, int L, const char *sa, const [QSpace](#)< TQ, TB > &B, unsigned ib, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1) const
- template<class TB , class TC >
int **contract** (const char *F, int L, const char *sa, const [QSpace](#)< TQ, TB > &B, const char *sb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1) const
- template<class TB , class TC >
int **contract** (const char *sa, const [QSpace](#)< TQ, TB > &B, const char *sb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1) const
- template<class TB , class TC >
int **contract** (const [ctrlIdx](#) &ica, const [QSpace](#)< TQ, TB > &B, const [ctrlIdx](#) &icb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1) const
- template<class TB , class TC >
int **contract** (unsigned ia, const [QSpace](#)< TQ, TB > &B, unsigned ib, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char cgflag=-1) const
- mxArray * **toMx** (bool setcls=0) const
- mxArray * **save2Mx** (char vflag=0)
- mxArray * **QIDX_toMx** () const
- mxArray * **INFO_toMx** () const
- mxArray * **DATA_toMx** () const
- mxArray * **DATA_save2Mx** (char vflag=0)
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, bool setcls=0, char chk=0) const
- void **save2MxStruct** (mxArray *S, unsigned i, char tst=0, char vflag=0)
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tst=0) const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **put** (const char *vname, const char *ws="caller") const
- [wbstring](#) **sizeStrQ** (const char *F=NULL, int L=0) const
- [wbstring](#) **sizeStr** (char vflag=0) const
- [wbstring](#) **qStr** () const
- [wbstring](#) **qrec2Str** (unsigned k) const
- void **disp_cgs** (const char *F, int L, const char *istr) const
- void **info** (const char *vname=NULL, unsigned nlt=0, unsigned nlb=0, unsigned nind=2) const
- void **print** (const char *vname, char vflag=1) const
- void **initOType** (const char *F, int L, const mxArray *a)
- [wbstring](#) **otype2Str** (const char *fmt="%s") const
- [QSpace](#) & **ConjOpScalar** ()
- [QSpace](#) & **SortDegQ** (const char *F=0, int L=0)
- [QSpace](#) & **Conj** ()
- [QSpace](#) & **dConj** ()
- [QSpace](#) & **HConj** ()

- [QSpace](#) & **hconj** ([QSpace](#) &B) const
- [QSpace](#) & **transp** ([QSpace](#) &B) const
- const [QSpace](#) & **opA** (char tflag, [QSpace](#) &X) const
- template<class TD2 >
[QSpace](#)< TQ, TD > & **initIdentityCGS** (const char *F, int L, const [QSpace](#)< TQ, TD2 > &H)

Public Attributes

- [wbMatrix](#)< TQ > **QIDX**
- [wbvector](#)< [wbarray](#)< TD > * > **DATA**
- [wbMatrix](#)< [CRef](#)< TQ > > **CGR**
- [QVec](#) **qtype**
- unsigned **QDIM**
- QS_TYPES **otype**
- [IndexLabels](#) **itags**
- Wb::MTYPE **mt**
- char **isref**
- double **ctime**

The documentation for this class was generated from the following files:

- QSpace.hh
- cpat.hh
- QSpace.cc

4.75 QType Class Reference

Public Member Functions

- **QType** (QTYPE_SET t=QTYPE_UNKNOWN, unsigned s=0)
- **QType** (const char *s)
- **QType** (const [QType](#) &q)
- **QType** (const char *F, int L, const mxArray *a)
- [QType](#) & **init** ()
- [QType](#) & **initx** (QTYPE_SET t, unsigned m=0)
- [QType](#) & **init** (QTYPE_SET t, unsigned m=0)
- int **init_s** (const char *s)
- [QType](#) & **init** (const char *s)
- [QType](#) & **init** (const char *F, int L, const char *s)
- [QType](#) & **init** (const char *F, int L, const mxArray *a)
- [QType](#) & **init** (const [QType](#) &q)
- bool **isUnknown** () const
- bool **isKnown** () const
- bool **validType** (const char *F=0, int L=0) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **isNonAbelian** (const char *F=NULL, int L=0, char lflag=0) const
- bool **isSU2** () const
- bool **isSUN** () const
- bool **isSU** (unsigned N) const
- bool **isSpN** () const

- bool **isSON** () const
- bool **isSEN** () const
- bool **isU1** () const
- bool **isParity** () const
- bool **isAdditive** () const
- int **permitsOM** () const
- int **permitsOM** (unsigned r) const
- unsigned **qlen** () const
- unsigned **qrank** () const
- bool **isLargeD** (unsigned d) const
- bool **isLargeD** (const [wbvector](#)< unsigned > &dd) const
- unsigned **getDimDef** () const
- template<class TQ >
size_t **wdim** (const TQ *qs) const
- template<class TQ >
size_t **wdim** (const [qset](#)< TQ > &qs) const
- unsigned **maxDimLocal** () const
- unsigned **maxDimLocal0** () const
- void **printDimInfo** (const char *F=NULL, int L=0) const
- template<class T >
unsigned **qdim** (const T *q) const
- template<class T >
[wbvector](#)< unsigned > & **QDim** (const T *q, unsigned r, unsigned stride, [wbvector](#)< unsigned > &S) const
- bool **isSelfDual** () const
- template<class TQ >
char **setDual** (TQ *q) const
- template<class TQ >
unsigned **getDual** (const TQ *q0, TQ *q) const
- template<class TQ >
bool **isDual** (const TQ *q, const TQ *x) const
- [QType](#) & **operator=** (const char *s)
- [QType](#) & **operator=** (const [QType](#) &q)
- [QType](#) & **operator=** (QTYPE_SET t)
- bool **operator<** (const [QType](#) &q) const
- char **cmp** (const [QType](#) &q) const
- bool **operator>** (const [QType](#) &q) const
- bool **operator<=** (const [QType](#) &q) const
- bool **operator==** (const [QType](#) &q) const
- bool **operator!=** (const [QType](#) &q) const
- [wbstring](#) **toStr** (const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- mxArray * **toMx** (const char tflag=0) const
- template<class TQ >
unsigned **qdim** (const TQ *q) const
- template<class TQ >
[wbvector](#)< unsigned > & **QDim** (const TQ *q, unsigned r, unsigned stride, [wbvector](#)< unsigned > &S) const

Public Attributes

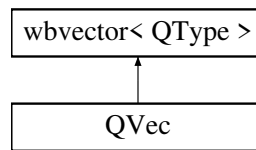
- QTYPE_SET **type**
- unsigned **sub**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.76 QVec Class Reference

Inheritance diagram for QVec:



Public Member Functions

- **QVec** (unsigned l, const [QType](#) *d0, char ref=0)
- **QVec** (const char *F, int L, const mxArray *a, unsigned d=0)
- **QVec & init** (const char *F, int L, const mxArray *a, unsigned d=0)
- **QVec & init** (unsigned l=0)
- **QVec & init** (const char *F, int L, const char *s, unsigned d=0)
- unsigned **Qlen** () const
- unsigned **Qrank** () const
- unsigned **Qlen** ([wbvector](#)< unsigned > &dd) const
- unsigned **Qlen** ([wbvector](#)< unsigned > &dd, [wbvector](#)< unsigned > &dz) const
- unsigned **Qpos** ([wbvector](#)< unsigned > &dc) const
- unsigned **Qlenz** () const
- [wbvector](#)< gTQ > & **getScalar** ([wbvector](#)< gTQ > &qs, char zflag=0)
- unsigned **largestRank** () const
- template<class TQ >
unsigned **QDim** (const TQ *q) const
- template<class TQ >
unsigned **QDim** (const TQ *qq, unsigned k) const
- template<class TQ >
[wbvector](#)< unsigned > & **QDim** (const TQ *qq, [wbvector](#)< unsigned > &S) const
- template<class TQ >
[wbvector](#)< unsigned > & **QDim** (const TQ *qq, unsigned k, unsigned r, [wbvector](#)< unsigned > &S) const
- template<class TQ >
[wbMatrix](#)< TQ > & **getQsub** (const [wbMatrix](#)< TQ > &Q, const [wbindex](#) &l, [wbMatrix](#)< TQ > &Ql, [wbMatrix](#)< TQ > *Qx=NULL) const
- bool **allU1** () const
- void **ReduceU1** ()
- char **allAbelian** () const
- bool **anyAbelian** () const
- bool **anyParity** () const
- bool **allNonAbelian** () const
- char **isNonAbelian** () const
- bool **allAdditive** () const
- int **permitsOM** () const
- int **permitsOM** (unsigned r) const
- bool **sameAs** (const [QVec](#) &b, char lflag=1) const
- bool **sameType** (const [QVec](#) &b) const
- [wbstring](#) **toStr** (const char vflag=0) const

Additional Inherited Members

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc

4.77 Wb::rand Class Reference

Public Member Functions

- double **get_dfac** ()
- template<class T >
T & **rand_** (T &x)
- template<class T >
T & **rand_** (T &x, double fac)
- template<class T >
T & **rands** (T &x)
- template<class T >
T & **randb** (T &x)
- template<class T >
void **scale_rand** (T *x, size_t n, double fac)
- template<> double & **rand_** (double &x)
- template<> int & **rand_** (int &x)
- template<> unsigned & **rand_** (unsigned &x)
- template<> long & **rand_** (long &x)
- template<> size_t & **rand_** (size_t &x)
- template<> int & **rand_** (int &x, double fac)
- template<> unsigned & **rand_** (unsigned &x, double fac)
- template<> long & **rand_** (long &x, double fac)
- template<> size_t & **rand_** (size_t &x, double fac)
- template<> double & **randb** (double &x)
- template<> int & **randb** (int &x)
- template<> double & **rands** (double &x)
- template<> int & **rands** (int &x)
- template<> void **scale_rand** (int *x, size_t n, double fac)

The documentation for this class was generated from the following file:

- util.hh

4.78 RCStore Class Reference

Public Member Functions

- int **setupDirs** (const char *F=NULL, int L=0)
- template<class TQ , class TD >
int **save_CData** (const char *F, int L, const [CData](#)< TQ, TD > &A)
- template<class TQ , class TD >
int **load_CData** (const char *F, int L, const [QSet](#)< TQ > &Q, [CData](#)< TQ, TD > &A, char bflag=0)
- template<class TQ >
int **save_Std3** (const char *F, int L, const [QType](#) &t, const [qset](#)< TQ > &J12, const [wbvector](#)< double > *c2eps=NULL) const
- template<class TQ >
int **load_Std3** (const char *F, int L, const [QType](#) &t, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, unsigned loadRC=0) const
- template<class TQ , class TD >
int **save_RSet** (const char *F, int L, const [genRG_base](#)< TQ, TD > &R, char xflag=0)
- template<class TQ >
int **load_RSet** (const char *F, int L, const [QType](#) &t, const [qset](#)< TQ > &J, char vflag=0)
- int **save_XMap** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc) const
- int **load_XMap** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc) const
- template<class TQ >
int **get_file_name** (const char *F, int L, [wbstring](#) &file, const [QSet](#)< TQ > &Q, const char *ext=NULL, char mflag=0) const
- template<class TQ , class TD >
int **get_file_name** (const char *F, int L, [wbstring](#) &file, const [genRG_base](#)< TQ, TD > &R, const char *ext=NULL, char mflag=0) const
- int **get_file_name** (const char *F, int L, [wbstring](#) &file, const [cgc_contract_id](#)< MTI > &idc, const char *ext=NULL, char mflag=0, [QType](#) *t=NULL) const
- size_t **rclog** (const [QType](#) &t, const char *F, int L, const char *fmt,...)
- size_t **rclog** (const [QType](#) &t, const char *F, int L, char vflag, const char *fmt,...)
- size_t **vrclog** (const [QType](#) &t, const char *F, int L, const char *fmt, va_list args)
- void **Info** (const char *F=0, int L=0)
- int **is_new** (const [QType](#) &t)

Public Attributes

- [Wb::Path](#) root

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc

4.79 RIXS_para Class Reference

Public Member Functions

- void **init** ()
- void **setE0** (int it, double e0, double e2)
- void **check** (const char *F=0, int L=0, char init=0)
- [Wb::String](#) **toStr** () const
- mxArray * **toMx** () const

Public Attributes

- [wbcomplex](#) **Einc**
- double **Eoffset**
- double **sigma**
- double **afac**
- char **wrxs**
- double **E0**
- double **E2**
- unsigned **dloc**
- int **iter**

The documentation for this class was generated from the following file:

- nrgdata.hh

4.80 RSpecData< TS > Class Template Reference

Public Member Functions

- [RSpecData](#) & **Append** (const double *op, const TS *cp, unsigned n)
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const

Public Attributes

- [wbvector](#)< double > **om**
- [wbvector](#)< TS > **cc**

The documentation for this class was generated from the following file:

- spectral.hh

4.81 RStore< TQ, TD > Class Template Reference

Public Member Functions

- **RStore** (const char *s=NULL)
- size_t **bufsize** (const [QType](#) &q)
- bool **isEmpty** (const [QType](#) &q)
- [genRG_struct](#)< TQ, TD > & **Buf** (const [QType](#) &q)
- [genRG_base](#)< TQ, TD > * **find_RSet** (const [QType](#) &q, const TQ *qptr)
- [genRG_struct](#)< TQ, TD > & **getRSet** (const char *F, int L, const [QType](#) &q, [qset](#)< TQ > *qvec=NULL)
- [genRG_base](#)< TQ, TD > & **getR** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- [genRG_base](#)< TQ, TD > & **getR** (const char *F, int L, const CData_TQ &S, unsigned k) const
- [wbMatrix](#)< TQ > **getZ** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- unsigned **qdim** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- mxArray * **toMx** (const [QType](#) &q=QTYPE_UNKNOWN, char bflag=1) const
- unsigned **LoadStore** (const char *F, int L, const char *file)
- unsigned **add** (const [QType](#) &q, const mxArray *S, unsigned k)
- void **Info** (const char *F=0, int L=0) const

Public Attributes

- `map< QType, genRG_struct< TQ, TD > > buf`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_aux.cc`
- `clebsch_io.cc`
- `clebsch_old.cc`

4.82 wblog::SBUF Class Reference

Public Member Functions

- **SBUF** (unsigned l0=wblog::SLEN)
- **SBUF** & **init** (unsigned l0=0)
- `int cat` (const char *s)
- `int catf` (const char *fmt,...)
- `void error_bounds` (const char *F, int L, const char *fct, const char *fmt)
- `void increase_size` (const char *F, int L, unsigned n)
- `void flush` (FILE *fid=stdout, char fflag=0)
- `void print` (const char *F=0, int L=0, const char *istr=0)

Public Attributes

- unsigned **l**
- unsigned **slen**
- char * **sbuf**

The documentation for this class was generated from the following files:

- `wblog.h`
- `wblog.c`

4.83 sparseCollector2D< TD > Class Template Reference

Public Member Functions

- **sparseCollector2D** (SPIDX_T m=0, SPIDX_T n=0)
- `void clear` ()
- `void init` (SPIDX_T m=0, SPIDX_T n=0)
- **sparseCollector2D** & **add** (const char *F, int L, SPIDX_T i, SPIDX_T j, const TD &a)
- `void getData` (**wbMatrix**< SPIDX_T > &IJ, **wbvector**< TD > &X)
- `mxAarray * toMx` () const

The documentation for this class was generated from the following file:

- `wbsparray.hh`

4.84 `sparseIndex2D< T >` Class Template Reference

Public Member Functions

- `sparseIndex2D` & `Setup` (char use_rmaj=0)
- `T cidx` (T j) const
- `T ridx` (T ic) const
- `wbstring sizeStr` () const

Public Attributes

- char `rmaj`
- `wbMatrix< T > IJ`
- `wbvector< T > S`
- `wbperm P`
- `wbvector< T > cldx`

The documentation for this class was generated from the following files:

- `wbindex.hh`
- `wbindex.cc`

4.85 `Spectral< TS >` Class Template Reference

Public Member Functions

- void `init` ()
- void `init` (const char *istr, unsigned Nops, double delta=0., double T_=0, unsigned Niter=0)
- void `initBuf` ()
- void `calcSpectralMoments` (const char *F, int L, unsigned n)
- void `Add` (const `wbarray< double > &Om`, const `wbarray< TS > &CC`, unsigned s, char dblock=0, char mflag=0)
- void `Add2buf` (const `wbarray< double > &Om`, const `wbarray< TS > &CC`, unsigned s)
- void `crossAddBuf` (const char minor=1, char vflag=1)
- void `savelter` (const char *F, int L, unsigned iter)
- void `getOmega` (`wbvector< double > &om`, unsigned &linfit) const
- void `getOmega` (`wbvector< double > &om`) const
- void `applyIROPfac` (const char *F, int L, const `wbvector< double > &fac`)
- void `getRawData` (`wbvector< double > &om`, `wbMatrix< TS > &A`, unsigned linfit=0, const char *isbuf="") const
- void `getPARTIAL` (`wbarray< TS > &A1`, `wbarray< TS > &A2`) const
- void `pairRawData` ()
- void `detailedBalance` (const double T, `wbvector< double > &om`, `wbMatrix< TS > &aa`) const
- void `dispSumRule` ()
- void `dispSumRule` (const `wbvector< double > &om`, const `wbMatrix< TS > &A`)
- void `getSmoothSpec` (`wbvector< double > &om`, `wbMatrix< TS > &A`, char vflag=1)
- void `getSmoothSpec` (`wbMatrix< TS > &A`, const `wbvector< double > &om`, char vflag=1)
- void `KKreal` (`wbMatrix< wbcomplex > &GZ`, `wbvector< double > &OM`, `wbvector< double > &DOM`) const

Public Attributes

- [wbMatrix](#)< TS > **Ap**
- [wbMatrix](#)< TS > **An**
- [wbMatrix](#)< TS > **Ap_buf**
- [wbMatrix](#)< TS > **An_buf**
- [wbvector](#)< TS > **Ar**
- [wbMatrix](#)< TS > **A0**
- [wbMatrix](#)< TS > **mspec**
- [wbarray](#)< TS > **AP**
- [wbarray](#)< TS > **AN**
- [wbstring](#) **istr**
- double **Temp**
- double **sigma**
- double **sigma2**
- double **alpha**
- double **Emin**
- double **Emax**
- double **eps**
- unsigned **nlog**

The documentation for this class was generated from the following file:

- spectral.hh

4.86 Wb::sptr< T > Class Template Reference

Public Member Functions

- void **init_def** ()
- T * **malloc_base** (size_t n, char init=1, MTYPE mt=MEM_DEF)
- T * **malloc_data** (size_t n, const T *d0=NULL, MTYPE mt=MEM_DEF)
- int **rm_dref** (char mflag=0)
- unsigned **add_dref** ()
- T * **save_dref** (const char *F=0, int L=0)
- T * **init2ref** (size_t n, const T *d0)
- int **isRef** () const
- **operator bool** () const
- bool **operator!** () const
- int **check_consistency** (char force=1) const
- [wbstring](#) **toStr** () const

Public Attributes

- MTYPE **mtype**
- size_t **len**
- unsigned **nref**
- T * **data**

The documentation for this class was generated from the following files:

- memlib.hh
- memlib.cc

4.87 STAT_DGEMM Class Reference

Public Member Functions

- void **account** (unsigned n0)

Public Attributes

- [wbvector](#)< unsigned > **nn**
- [wbvector](#)< unsigned long > **ss**

The documentation for this class was generated from the following file:

- wbbblas.hh

4.88 wblog::stdio_buf Class Reference

Public Member Functions

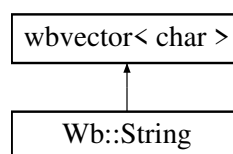
- **stdio_buf** ([SBUF](#) &S, FILE *f_, int t_)
- **stdio_buf** (const [stdio_buf](#) &b)
- void **print_stdout** (FILE *f=NULL) const

The documentation for this class was generated from the following file:

- wblog.h

4.89 Wb::String Class Reference

Inheritance diagram for Wb::String:



Public Member Functions

- **String** (unsigned n)
- **String** (const char *F, int L, unsigned n)
- int **pushf** (const char *fmt,...)

Additional Inherited Members

The documentation for this class was generated from the following files:

- wbstring.hh
- wbstring.cc

4.90 SVDDData< TQ, TD > Class Template Reference

Public Member Functions

- void **blockSVD** (const [QSpace](#)< TQ, TD > &PSI, const unsigned K)
- unsigned **dmrgTruncate** (unsigned Nkmin, unsigned Nkeep, double stol, [iTag](#) tx, [wbMatrix](#)< double > &SV, [QSpace](#)< TQ, TD > &UQ, [QSpace](#)< TQ, TD > *SQ, [QSpace](#)< TQ, TD > &VQ, mxArray *Sout=NULL, char vflag=0)
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base") const
- void **put** (const char *F, int L, const char *vname, const char *ws="base") const

Public Attributes

- [wbvector](#)< [QBlock](#)< TQ, TD, double > > **QB**

The documentation for this class was generated from the following files:

- mpsortho.hh
- mpsortho.cc

4.91 DY::Symmetry Class Reference

Public Member Functions

- **Symmetry** (const [QType](#) &q_)
- **Symmetry** (const mxArray *a)
- [Symmetry](#) & **init** (const [QType](#) &q)
- template<class TQ >
size_t **getWeightsFT** (const char *F, int L, const [qset](#)< TQ > &qs, [Weights](#)< TQ > &R)
- template<class TQ >
int **getInnerMultFT** (const [qset](#)< TQ > &qs, const [weight_info](#)< TQ > &w, [Weights](#)< TQ > &R, char vflag=0)
- template<class TQ >
int **Get** (const [qset](#)< TQ > &qk, map< [qset](#)< TQ >, [weight_info](#)< TQ > > &W, char vflag=0)
- template<class TQ >
int **wExpand** (const char *F, int L, [Weights](#)< TQ > &R) const
- mxArray * **toMx** () const

Public Attributes

- [QType](#) **q**
- unsigned **n**
- [wbarray](#)< double > **R**
- [wbMatrix](#)< int > **A**
- [wbvector](#)< int > **n2**
- [wbarray](#)< int > **M**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.92 TDSpectral< TS > Class Template Reference

Public Member Functions

- void **init** (const char *F, int L, const char *istr0, const [wbvector](#)< double > &t0, unsigned Nnrg, unsigned Nop, double Emin=1E-8, double Emax=10., unsigned nl=512, char vflag=1)
- void **checkAvgOm** ()
- [wbarray](#)< TS > **getAvgOm** ()
- void **Add** (const [wbarray](#)< double > &Om, const [wbarray](#)< TS > &CC, unsigned iter, unsigned s)
- void **addPartialFourier** ()
- void **getOmega** ([wbvector](#)< double > &om, unsigned &linfit) const
- void **getOmega** ([wbvector](#)< double > &om) const
- void **applyIROPfac** (const char *F, int L, const [wbvector](#)< double > &fac, char rixs=0)
- void **getSpecData** ([wbvector](#)< double > &om, [wbarray](#)< TS > &A, unsigned linfit=0) const
- void **getSpecData** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, unsigned linfit=0)
- void **mergeNRGData** ()
- void **getSmoothSpec_t** (double sigma, double eps, char bswitch=LOG_GAUSS_BRD, char vflag=1)
- void **Fourier** ([wbvector](#)< double > &om, [wbMatrix](#)< [wbcomplex](#) > &ST, double alpha, double Lambda, char vflag=1)
- void **Fourier** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, const [wbvector](#)< double > &tt, [wbMatrix](#)< [wbcomplex](#) > &at, double alpha=0., char vflag=1)
- void **dispSumRule** ()
- mxArray * **toMx** () const
- int **init** (const mxArray *S, double &emin, double &emax, unsigned &nlog)

Public Attributes

- [wbarray](#)< TS > **AP**
- [wbarray](#)< TS > **AN**
- [wbMatrix](#)< TS > **Ap**
- [wbMatrix](#)< TS > **An**
- [wbMatrix](#)< [wbcomplex](#) > **At**
- [wbvector](#)< double > **tt**
- [wbstring](#) **istr**
- unsigned **raw**
- [wbvector](#)< [RSpecData](#)< TS > > **RAW**

The documentation for this class was generated from the following file:

- spectral.hh

4.93 Wb::tmpFile Class Reference

Public Member Functions

- **tmpFile** (const char *F, int L, const char *f=NULL, const char *x=NULL, const char *mode=NULL)
- void **init** (const char *F, int L, const char *f=NULL, const char *x=NULL, const char *mode=NULL)
- int **blogf** (const char *F, int L, const char *fmt,...)
- int **sepline** (const char *w="-", unsigned len=-1)
- void **flush** ()

Public Attributes

- FILE * **fid**
- int **ncall**
- [wbstring](#) **fout**

The documentation for this class was generated from the following files:

- wbio.h
- wbio.c

4.94 Wb::UseClock Class Reference

Public Member Functions

- **UseClock** ([Wb::Clock](#) *c)
- void **done** ()
- void **stop** ()
- void **resume** ()

The documentation for this class was generated from the following file:

- wbclock.hh

4.95 wbarry< T > Class Template Reference

Public Member Functions

- **wbarry** (size_t d1)
- **wbarry** (size_t d1, size_t d2)
- **wbarry** (size_t d1, size_t d2, size_t d3)
- **wbarry** (size_t d1, size_t d2, size_t d3, size_t d4)
- **wbarry** (char *sidx)
- **wbarry** (const [wbvector](#)< unsigned > &S)
- **wbarry** (const [wbvector](#)< size_t > &S)
- **wbarry** (const [wbarry](#) &a, const char ref=0)

- `template<class T2 >`
`wbarray` (const `wbarray`< T2 > &a)
- `wbarray` (const `mxArray` *a, char ref=0, char vec=0)
- `wbarray` (const char *F, int L, const `mxArray` *a, char ref=1, char vec=0)
- `wbarray` (const `wbMatrix`< T > &a, char ref=0)
- `wbarray` & `initX` ()
- `wbarray` & `init` (size_t d1=0)
- `wbarray` & `init` (size_t d1, size_t d2)
- `wbarray` & `init` (size_t d1, size_t d2, size_t d3)
- `wbarray` & `init` (size_t d1, size_t d2, size_t d3, size_t d4)
- `wbarray` & `initp` (size_t d1, const T *d)
- `wbarray` & `initp` (size_t d1, size_t d2, const T *d)
- `wbarray` & `initp` (size_t d1, size_t d2, size_t d3, const T *d)
- `wbarray` & `init2ref` (const `wbarray` &a)
- `wbarray` & `init` (const `wbarray` &a, char ref=0)
- `wbarray` & `init` (const `wbMatrix`< T > &a, char ref=0)
- `wbindex` & `ind2sub` (size_t k, `wbindex` &l) const
- void `adjustMMat` (const char *F, int L, char flag, const T fac, `wbarray` &A) const
- void `adjustDMat` (const char *F, int L, char flag, const T fac, `wbarray` &A) const
- void `adjustVec` (const char *F, int L, char &flag, `wbarray` &A, char pos) const
- void `adjustVec` (const char *F, int L, char &flag, char pos)
- void `adjustCMat` (const char *F, int L, T cfac, size_t s1, size_t s2, char cforce)
- void `set` (const `wbarray` &a, const T fac=1)
- void `setBlock` (const `wbvector`< WBINDEX > &D, const `wbindex` &IB, const `wbarray` &A)
- void `addBlock` (const WBINDEX &l, const `wbarray` &A, const char dflag=0)
- `wbarray` & `addBlock` (size_t i, size_t j, size_t n, size_t m, `wbarray` &a) const
- `wbarray` & `getBlock` (size_t i, size_t j, size_t n, size_t m, `wbarray` &a) const
- `wbarray` & `BlockDiag` (const `wbvector`< `wbarray` > &D)
- `wbarray` & `blockTrace` (size_t D, `wbarray` &a, size_t D2=-1) const
- void `copyStride` (T *dd, size_t stride) const
- void `addStride` (T *dd, size_t stride, const T afac=1) const
- `template<class T0 >`
`wbarray` & `init_T` (const T0 *dd)
- `template<class T0 >`
`wbarray` & `init_T` (const char *F, int L, const T0 *dd)
- `wbarray` & `initT` (const `wbarray`< T > &a)
- `template<class T0 >`
`wbarray` & `initT` (const `wbarray`< T0 > &a)
- `template<class T0 >`
`wbarray` & `initT` (const char *F, int L, const `wbarray`< T0 > &a)
- `wbarray` & `init` (char *s, T *d=NULL)
- `wbarray` & `init` (const WBINDEX &S, T *d=NULL)
- `template<class IT >`
`wbarray` & `init` (const `wbvector`< IT > &S_, T *d=NULL)
- `wbarray` & `init` (const char *F, int L, const `mxArray` *a, char ref=1, char vec=0)
- `wbarray` & `init` (const char *F, int L, const `mxArray` *a, `wbperm` &P0)
- `wbarray` & `init` (const `mxArray` *a, char ref=0, char vec=0)
- `wbarray` & `init` (const `mxArray` *a, `wbperm` &P0)
- `wbarray` & `init2ref` (const T *x, const WBINDEX &S)
- `wbarray` & `init2ref` (size_t d1, const T *x)
- `wbarray` & `init2ref` (size_t d1, size_t d2, const T *x)
- `wbarray` & `init2ref` (size_t d1, size_t d2, size_t d3, const T *x)
- int `isRef` () const
- `wbarray` & `Instantiate` (const char *F=NULL, int L=0)
- `wbarray` & `unRef` ()

- void **init2Vec** (const T *d0, size_t len, char ref=0)
- void **init2ref** (const wbarry &A, size_t s1, size_t s2)
- void **initTst** ()
- void **init2Vec** (const wvector< T > &a)
- wbarry & **initIdentity** (size_t d, char dflag=0, T dval=1)
- wbarry & **initIdentity** (const WBINDEX &S, char dflag=0)
- wbarry & **initIdentityB** (size_t d1, size_t d2, size_t k=0)
- wbarry & **initIdentityB3** (size_t d1, size_t d2, size_t D, size_t i0, T one=1)
- wbarry & **Expand2Projector** (const T eps=0)
- wbarry & **ExpandDiagonal** (unsigned i1, unsigned i2)
- wbarry & **ExpandDiagonal** ()
- void **Diag2Vec** ()
- wbarry & **Reduce2Diag** (T eps=0)
- wbarry & **initDiag** (unsigned n, const T *d)
- wbarry & **initDiag** (const wvector< T > &d)
- void **initVec** (size_t d, const T val=0)
- void **resize** (const wvector< size_t > &S, wbarry &B) const
- void **Resize** (const wvector< size_t > &S, const T *d0=NULL)
- void **Resize** (size_t s1, size_t s2)
- wbarry & **Enlarge** (size_t dim, size_t D, size_t istart=0)
- wbarry & **save2** (wbarry &)
- wbarry & **save2** (wbMatrix< T > &)
- wbarry & **Append2** (const char *F, int L, wbarry &B, unsigned dim)
- wbarry & **Append2** (wbarry &B, unsigned dim)
- void **Append2** (const char *F, int L, wbarry &B, unsigned dim, wbarry< double > &W, double eps)
- void **Append2** (wbarry &B, unsigned dim, wbarry &W, double eps)
- void **rand** (const UVEC &d)
- void **setRand** (const char pflag=0)
- wbarry & **Reset** (const T *dref)
- wbarry & **set** (const T &c)
- wbarry & **operator=** (const wbarry &a)
- template<class T0 >
wbarry< T > & **operator=** (const wbarry< T0 > &a)
- void **SetRef** (const widx_t *l, size_t len, const T *d0)
- void **SetRef** (size_t k, const T *d0)
- const T * **ref** (const widx_t *l, size_t n) const
- T * **ref** (const widx_t *l, size_t n)
- T * **col** (size_t k)
- const T * **col** (size_t k) const
- wvector< T > & **getCol** (size_t k, wvector< T > &v) const
- T * **ref** (const WBINDEX &l)
- const T * **ref** (const WBINDEX &l) const
- const T * **ref** (widx_t i) const
- T * **ref** (widx_t i)
- const T * **ref** (widx_t i1, widx_t i2) const
- T * **ref** (widx_t i1, widx_t i2)
- const T * **ref** (widx_t i1, widx_t i2, widx_t i3) const
- T * **ref** (widx_t i1, widx_t i2, widx_t i3)
- const T & **operator[]** (size_t i) const
- T & **operator[]** (size_t i)
- const T & **operator()** (size_t i) const
- T & **operator()** (size_t i)
- const T & **operator()** (size_t i, size_t j) const
- T & **operator()** (size_t i, size_t j)
- const T & **operator()** (size_t i, size_t j, size_t k) const

- T & **operator()** (size_t i, size_t j, size_t k)
- const T & **operator()** (const WBINDEX &l) const
- T & **operator()** (const WBINDEX &l)
- const T & **element** (const WBINDEX &l) const
- T & **element** (const WBINDEX &l)
- size_t **numel** () const
- [wbstring](#) **sizeStr** () const
- void **squeeze** ()
- void **skipSingletons** ()
- void **skipSingleton** (const char *F, int L, unsigned n=1)
- void **addSingletons** (unsigned r, const char *F=NULL, int L=0)
- void **prependSingletons** (unsigned r)
- size_t **size** (unsigned i) const
- size_t **dim** (unsigned i) const
- size_t **dim0** (unsigned i) const
- size_t **dim1** () const
- size_t **dim2** () const
- void **getMatSize** (const char *F, int L, size_t &dim1, size_t &dim2) const
- void **getMatSize** (size_t &dim1, size_t &dim2, unsigned m) const
- unsigned **rank** () const
- bool **isRank** (unsigned r) const
- bool **isOpS** (size_t *n=NULL) const
- bool **isOpS** (unsigned &n) const
- [wbarray](#) & **SkipTiny_float** (const T eps=1E-14)
- double **SkipTiny_imag** (double eps=1E-14)
- double **SkipTiny** (const double eps_=1E-14)
- bool **isMatrix** () const
- bool **isSMatrix** (unsigned d=-1) const
- bool **isSquare** () const
- char **isEmpty** () const
- bool **isComplex** () const
- bool **isZero** (double eps=0., char flag=0) const
- bool **isFinite** ()
- **operator bool** () const
- bool **operator!** () const
- [wbarray](#) & **Symmetrize** (const char *F, int L, double *delta=NULL, char cflag=1, char tflag=0, char fflag=0)
- [wbarray](#) & **swapRows** (size_t i1, size_t i2)
- [wbarray](#) & **swapCols** (size_t j1, size_t j2)
- [wbarray](#) & **setCol** (size_t k, const T *d, const char *F=NULL, int L=0)
- [wbarray](#) & **setCol** (size_t k, size_t k0, const char *F=NULL, int L=0)
- [wbarray](#) & **setRow** (size_t k, const T *d, const char *F=NULL, int L=0)
- [wbvector](#)< T > & **colNorm2** ([wbvector](#)< T > &a) const
- T **colNorm2** (size_t k) const
- unsigned **QRdecomp** (const char *F, int L, [wbarray](#)< T > &Q, [wbarray](#)< T > &R, T eps=1E-15)
- int **Householder** (const char *F, int L, size_t k, T *u, WBPERM *P=NULL, T eps=1E-15)
- unsigned **eigTriDiag** (const char *F, int L, [wbvector](#)< T > &E, [wbarray](#)< T > *U=NULL, T eps=1E-15) const
- [wbarray](#) & **ColProject** (size_t k1, size_t k2, char nflag=0, char tnorm=0)
- [wbarray](#) & **ColPermute** (const [wbperm](#) &P, char iflag=0)
- bool **isOrthogonalCol** (size_t k, char tnorm=0) const
- [wbarray](#) & **SignCol** (size_t k, const T *d0, char tnorm=0)
- [wbarray](#) & **FlipSignCol** (size_t k)
- [wbarray](#) & **SignConventionCol** (size_t k=-1, double eps=1E-12)
- T **NormalizeCol** (size_t k, char tnorm=0, char qflag=0)
- [wbarray](#) & **NormalizeCols** (const char *F, int L, double *amin=NULL, double *amax=NULL, char tnorm=0)

- [wbarry](#) & **OrthoNormalizeCols** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, double eps=1E-14, unsigned np=1)
- [wbarry](#) & **balanceOp** (const char *F, int L, T &xref, double &xscale)
- bool **isProptold** (T &x, T eps=1E-14) const
- bool **isDiagMatrix** (double eps=1E-14) const
- bool **isIdentityMatrix** (double eps=1E-14) const
- bool **isDiagMatrix** (double *eps) const
- bool **isIdentityMatrix** (double *eps) const
- size_t **nnz** (const T eps=0, WBINDEX *I=NULL) const
- double **sparsity** (const T eps=0) const
- bool **isHConj** (const [wbarry](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isHConj** (double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (const [wbarry](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (double eps=1E-12, double *xref=NULL) const
- char **hasGroupSize** (size_t s1, size_t s2) const
- double **aMax** (size_t *i=NULL) const
- double **maxDiff** (const [wbarry](#) &B) const
- T **froNorm2** (const [wbarry](#) &B) const
- T **norm2** () const
- T **norm** () const
- T **normCol2** (size_t k) const
- T **scalarProd** (const [wbarry](#) &B) const
- T **sum** () const
- [wbarry](#) & **sum** (const WBINDEX &I, [wbarry](#) &) const
- [wbarry](#) & **sum** (const char *sidx, [wbarry](#) &A) const
- char **fitsSize** (const [wbarry](#) &, char strict=0) const
- size_t **getSize** (unsigned i, char lflag=1) const
- bool **hasSameSize** (const size_t *s, unsigned n) const
- bool **hasSameSize** (size_t d1, size_t d2) const
- bool **hasSameSize1** (const size_t *s, unsigned n) const
- template<class TB >
bool **hasSameSize** (const [wbarry](#)< TB > &, char lflag=1) const
- template<class TB >
bool **hasSameSize** (unsigned ia, const [wbarry](#)< TB > &B, unsigned ib, char lflag=1) const
- char **sameUptoFac** (const [wbarry](#) &B, T *fac=NULL, double eps=1E-12) const
- bool **hasRank** (unsigned r) const
- bool **operator==** (const [wbarry](#) &B) const
- bool **operator!=** (const [wbarry](#) &B) const
- T **normDiff2** (const [wbarry](#) &B) const
- T **normDiff** (const [wbarry](#) &B) const
- T **normDiff2** (const [wbvector](#)< T > &B) const
- T **normDiff** (const [wbvector](#)< T > &B) const
- void **TimesEI** (const [wbarry](#) &, char conj=0)
- [wbarry](#) & **timesEI** (const [wbarry](#) &B, [wbarry](#) &C, T bfac=1, T cfac=0, char conj=0) const
- T **sumTimesEI** (const [wbarry](#) &, char conj=0) const
- T **weightedAvg** (const [wbarry](#) &) const
- void **TensorProd** (const [wbarry](#) &B, const char aflag='N', const char bflag='N', const char kflag=0)
- void **tensorProd** (const [wbarry](#) &, [wbarry](#) &, const char aflag='N', const char bflag='N', const char kflag=0) const
- void **kron** (const [wbarry](#) &B, [wbarry](#) &X, char aflag='N', char bflag='N') const
- void **Kron** (const [wbarry](#) &B, char aflag='N', char bflag='N')
- [wbarry](#) & **Cat** (unsigned dim, const [wbvector](#)< [wbarry](#) * > &ap)
- [wbarry](#) & **Cat** (unsigned dim, const [wbvector](#)< [wbarry](#) > &ap)
- [wbarry](#) & **BlockCat** (const [wbarry](#)< [wbarry](#) > &aa, WBINDEX *D1=NULL, WBINDEX *D2=NULL)

- [wbararray](#) & **BlockCat** (const [wbararray](#)< const [wbararray](#) * > &ap, WBINDEX *D1=NULL, WBINDEX *D2=NULL)
- void **operator+=** (T x)
- void **operator-=** (T x)
- void **operator*=** (const T a)
- void **operator+=** (const [wbararray](#) &B)
- void **operator-=** (const [wbararray](#) &B)
- [wbararray](#) & **plus** (const [wbararray](#) &B, [wbararray](#) &C, T bfac=1, char iflag=0) const
- [wbararray](#) & **Plus** (const [wbararray](#) &B, T bfac=1, char iflag=0)
- bool **equal** (const [wbararray](#) &, const T &eps) const
- bool **unequal** (const [wbararray](#) &, const T &eps) const
- bool **equal** (const [wbararray](#) &, const double &eps, double &maxdiff) const
- bool **unequal** (const [wbararray](#) &, const double &eps, double &maxdiff) const
- bool **allEqual** (const T &x)
- bool **allLE** (const T &x)
- char **isVector** () const
- bool **isScalar** () const
- bool **requiresDataPerm** (const [wbperm](#) &P) const
- void **markUnequalZero** (const T m=1, const T eps=0)
- void **toMatrixRef** ([wbararray](#) &A, const UVEC &I, int pos, [wbperm](#) &P) const
- bool **toMatrixRef** (const char *F, int L, [wbararray](#) &A, const UVEC &I, int pos, char &tflag) const
- void **toMatrixRef** ([wbararray](#) &A, unsigned K, const [wbperm](#) &P0=[wbperm](#)()) const
- void **toMatrixRef** ([wbararray](#) &A, unsigned i, int pos, [wbperm](#) &P) const
- void **toMatrixRef** ([wbararray](#) &A, const char *sidx, int pos, [wbperm](#) &P) const
- [wbararray](#) & **Reshape** (const [wbvector](#)< size_t > &)
- [wbararray](#) & **Reshape** (size_t s1, size_t s2)
- [wbararray](#) & **Reshape** (size_t s1, size_t s2, size_t s3)
- [wbararray](#)< T > & **Reshape1** (const size_t *s, unsigned n)
- [wbararray](#) & **Reshape1** (size_t s1, size_t s2)
- [wbararray](#) & **Reshape1** (size_t s1, size_t s2, size_t s3)
- void **GroupIndices** (size_t K)
- [wbararray](#) & **groupIndices_DREF** ([wbararray](#) &A, unsigned k1, unsigned k2) const
- void **GroupIndices** (unsigned k1, unsigned k2)
- void **groupIndices_P** (const WBINDEX &, int, [wbperm](#) &, size_t *s=NULL, size_t *s=NULL) const
- [wbararray](#) & **transpose** (const char *F, int L, [wbararray](#) &A) const
- [wbararray](#) & **Transpose** (const char *F=0, int L=0)
- [wbararray](#) & **flipLR** (const char *F, int L, [wbararray](#) &A) const
- [wbararray](#) & **FlipLR** (const char *F=0, int L=0)
- [wbararray](#) & **flipUD** (const char *F, int L, [wbararray](#) &A) const
- [wbararray](#) & **FlipUD** (const char *F=0, int L=0)
- [wbararray](#) & **MatPermute** (const [wbperm](#) &P, char iflag=0)
- [wbararray](#) & **Permute** (const char *s, char iflag=0)
- [wbararray](#) & **Permute** (const WBPERM &, char iflag=0)
- [wbararray](#) & **permute** ([wbararray](#) &, const char *s, char iflag=0) const
- [wbararray](#) & **permute** ([wbararray](#) &, const WBPERM &, char iflag=0) const
- [wbararray](#) & **RPermute** (C_UVEC &P, char iflag=0)
- [wbararray](#) & **select0** (const WBPERM &, unsigned dim, [wbararray](#) &) const
- [wbararray](#) & **Select0** (const UVEC &I, unsigned dim)
- [wbararray](#) & **select0** (size_t i1, size_t i2, unsigned dim, [wbararray](#) &Q) const
- [wbararray](#) & **selectSqueeze** ([wbararray](#) &A, unsigned p, unsigned dim) const
- T **min** () const
- T **max** () const
- T **aMin** (char zflag=0, size_t *k=NULL) const
- void **info** (const char *s="") const
- void **info** (const char *F, int L, const char *s="", unsigned k=-1, unsigned nlt=0, unsigned nlb=0) const

- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **print** (const char *istr="", const char *fmt="") const
- int **printdata** (const char *istr="ans", const char *fmt="") const
- void **print_ref** (const char *bstr=" ***", const char *nl="\n") const
- **wbstring toStr** () const
- void **contractDiag** (unsigned ic, const **wbarray** &B, T a, T b, **wbarray** &C, char lflag=0, char bflag='N') const
- template<class TB, class TC >
wbarray< TC > & **contractMat** (unsigned, const **wbarray**< TB > &, **wbarray**< TC > &, unsigned=1) const
- **wbarray** & **ContractMat** (unsigned i1, const **wbarray** &B, unsigned i2=1)
- template<class TB, class TC >
wbarray< TC > & **comm** (const **wbarray**< TB > &, **wbarray**< TC > &, char aflag='N', char bflag='N') const
- template<class TB, class TC >
wbarray< TC > & **acommm** (const **wbarray**< TB > &, **wbarray**< TC > &, char aflag='N', char bflag='N') const
- template<class TB, class TC >
wbarray< TC > & **contract** (const char *F, int L, const **ctrlidx** &, const **wbarray**< TB > &, const **ctrlidx** &, **wbarray**< TC > &, const **wbperm** &pfinal=**wbperm**() , const T &afac=1.) const
- template<class TB >
wbarray< T > & **Contract** (char *, const **wbarray**< TB > &, char *, const **wbperm** &pfinal=**wbperm**())
- template<class TB >
wbarray< T > & **Contract** (const **wbvector**< unsigned > &, const **wbarray**< TB > &, const **wbvector**< unsigned > &, const **wbperm** &pfinal=**wbperm**())
- template<class TB, class TC >
wbarray< TC > & **contract** (const char *F, const int L, const char *, const **wbarray**< TB > &, const char *, **wbarray**< TC > &, const **wbperm** &pfinal=**wbperm**()) const
- template<class TB, class TC >
wbarray< TC > & **contract** (char *i1, const **wbarray**< TB > &B, char *i2, **wbarray**< TC > &Q, const **wbperm** &pfinal=**wbperm**()) const
- template<class TB, class TC >
wbarray< TC > & **contract** (const **wbvector**< unsigned > &i1, const **wbarray**< TB > &B0, const **wbvector**< unsigned > &i2, **wbarray**< TC > &Q0, const **wbperm** &P) const
- template<class TB, class TC >
wbarray< TC > & **contract** (unsigned i1, const **wbarray**< TB > &B, unsigned i2, **wbarray**< TC > &C, const **wbperm** &pfinal=**wbperm**()) const
- **wbarray operator+** (const **wbarray** &B) const
- **wbarray operator-** (const **wbarray** &B) const
- **wbarray operator*** (const **wbarray** &B) const
- T **trace** () const
- **wbarray** & **trace** (unsigned i1, unsigned i2, **wbarray** &C) const
- **wbarray** & **trace** (const **ctrlidx** &I1, const **ctrlidx** &I2, **wbarray**< T > &C) const
- mxArray * **toMx** () const
- mxArray * **toMx_Struct** () const
- mxArray * **toMx_base** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **put** (const char *vname="ans", const char *ws="base") const
- void **put** (const char *F, int L, const char *vname="ans", const char *ws="base") const
- void **getReal** (**wbarray**< double > &R) const
- void **getImag** (**wbarray**< double > &I) const
- void **Conj** ()
- void **set** (const **wbarray**< double > &R, const **wbarray**< double > &I)
- const **wbarray** & **opA** (char tflag, **wbarray** &aux)
- bool **isSym_aux** (const char *file, int line, const char *fct, const **wbarray** &B, double eps, double *xref, const char symflag='s', const char lflag=1) const
- double **SkipTiny** (const double eps)
- **wbarray**< **wbcomplex** > & **SkipTiny_float** (**wbcomplex** ref)
- double **SkipTiny_imag** (double eps)

- bool **isComplex** () const
- int **printdata** (const char *istr, const char *fmt0) const
- template<class TB , class TC >
wbarray< TC > & **contract** (const char *F, const int L, const **ctrlidx** &ica, const **wbarray**< TB > &B0, const **ctrlidx** &icb, **wbarray**< TC > &C0, const **wbperm** &P, const TA &afac) const
- void **getReal** (**wbarray**< double > &R) const
- void **getImag** (**wbarray**< double > &I) const
- void **Conj** ()
- void **set** (const **wbarray**< double > &R_, const **wbarray**< double > &I_)
- mxArray * **toMx** () const
- mxArray * **toMx** () const
- mxArray * **toMx** () const
- **wbcomplex norm2** () const
- **wbcomplex normCol2** (size_t k) const
- **wbcomplex scalarProd** (const **wbarray**< **wbcomplex** > &B) const

Public Attributes

- **wbvector**< size_t > **SIZE**
- Wb::MTYPE **mtype**
- **Wb::sptr**< T > * **sptr**
- T * **data**

Protected Member Functions

- bool **isDiag_aux** (const T eps, const char *task) const
- bool **isDiag_aux** (double *eps, const char *task) const
- size_t **serial_index** (const widx_t *l, size_t n) const

The documentation for this class was generated from the following files:

- wbarray.hh
- wbarray.cc

4.96 **wbarrRef**< T > Class Template Reference

Public Member Functions

- **wbarrRef** (const **wbarrRef**< T > &r)
- **wbarrRef**< T > & **operator=** (const **wbarrRef**< T > &r)
- void **print** (const char *vname="ans")

Public Attributes

- double **dc**
- double **fac**
- bool **tflag**
- **wbarray**< T > * **D**

The documentation for this class was generated from the following file:

- wbarray.hh

4.97 wbcomplex Class Reference

Public Member Functions

- **wbcomplex** (const [wbcomplex](#) &z)=default
- **wbcomplex** (double r0, double i0=0.)
- **wbcomplex** & **operator=** (const [wbcomplex](#) &z)=default
- **wbcomplex** & **operator=** (const double &dbl)
- **operator bool** () const
- **bool operator!** () const
- **operator double** () const
- **wbcomplex** & **init** (double r0, double i0)
- **wbcomplex** & **set** (double r0, double i0)
- **wbcomplex** & **setRand** (double fac=2., double shift=-1.)
- **wbcomplex** & **init_expi** (double ph)
- **mxComplexDouble** & **explicit_copy2** (**mxComplexDouble** &zm) const
- **bool isinf** () const
- **bool isnan** () const
- **bool operator==** (const [wbcomplex](#) &z) const
- **bool operator!=** (const [wbcomplex](#) &z) const
- **bool operator==** (const double &dbl) const
- **bool operator!=** (const double &dbl) const
- **bool operator>** (const [wbcomplex](#) &z) const
- **bool operator>=** (const [wbcomplex](#) &z) const
- **bool operator<** (const [wbcomplex](#) &z) const
- **bool operator<=** (const [wbcomplex](#) &z) const
- **wbcomplex** & **operator+=** (const [wbcomplex](#) &z)
- **wbcomplex** & **operator-=** (const [wbcomplex](#) &z)
- **wbcomplex** & **operator*=** (const [wbcomplex](#) &z)
- **wbcomplex** & **operator/=** (const [wbcomplex](#) &z)
- **wbcomplex** & **operator*=** (double dbl)
- **wbcomplex** & **operator+=** (double dbl)
- **wbcomplex** & **operator-=** (double dbl)
- **wbcomplex operator+** (const [wbcomplex](#) &z) const
- **wbcomplex operator-** (const [wbcomplex](#) &z) const
- **wbcomplex operator*** (const [wbcomplex](#) &z) const
- **wbcomplex operator/** (const [wbcomplex](#) &z) const
- **wbcomplex operator+** (const double &dbl) const
- **wbcomplex operator-** (const double &dbl) const
- **wbcomplex operator*** (const double &dbl) const
- **wbcomplex operator/** (const double &dbl) const
- **wbcomplex operator/=** (double dbl)
- **wbcomplex times** (const [wbcomplex](#) &) const
- **wbcomplex ConjProd** (const [wbcomplex](#) &z) const
- **void Conj** ()
- **wbcomplex conj** () const
- **double abs2** () const
- **double abs** () const
- **wbcomplex sqrt** () const
- **wbcomplex sign** () const
- **double arg** () const
- **double arg_degree** ()
- **wbcomplex round** () const
- **wbstring toStr** (const char *fmt="% .4g") const
- **void put** (const char *vname="ans", const char *ws="base") const

Public Attributes

- double **r**
- double **i**

The documentation for this class was generated from the following files:

- wbcomplex.hh
- wbcomplex.cc

4.98 wdebug_dummy Class Reference

The documentation for this class was generated from the following file:

- memtrack.cc

4.99 wbhist Class Reference

Public Member Functions

- **wbhist** (const mxArray *a)
- **wbhist** & **init** (const mxArray *a)
- **wbhist** & **add** (const char *F, int L, const mxArray *ax, const mxArray *ay)

Public Attributes

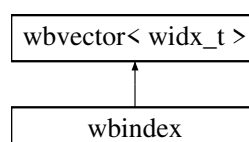
- long unsigned **ndat**
- long unsigned **ncalls**
- **wbvector**< double > **xbdr**
- **wbvector**< double > **xbin**
- **wbvector**< double > **ybin**

The documentation for this class was generated from the following file:

- wbhist.cc

4.100 wbindex Class Reference

Inheritance diagram for wbindex:



Public Member Functions

- **wbindex** (widx_t n=0, char iflag=0)
- **wbindex** (const [ctrlidx](#) &ic)
- **wbindex** (widx_t n, const widx_t *d)
- **wbindex** (const char *s, unsigned offset=0)
- **wbindex** & **init** (widx_t n=0, const widx_t *d=NULL)
- **wbindex** & **init** (const [ctrlidx](#) &ic)
- **wbindex** & **init** (const char *F, int L, const mxArray *a, widx_t offset=0)
- **widx_t init** (const char *F, int L, const mxArray *a, const char *istr, char check_type=1)
- **wbindex** & **init** (const mxArray *a, widx_t offset=0)
- **wbindex** & **initGroup** (const [wbvector](#)< widx_t > &P, [wbvector](#)< widx_t > &D)
- **wbindex** & **initGroup0** (const [wbvector](#)< widx_t > &P, [wbvector](#)< widx_t > &D)
- void **initStr** (const char *s, widx_t offset=0)
- bool **isIndex** (widx_t n=0)
- **wbindex** & **Index** (widx_t n)
- **wbindex** & **Index_ex** (widx_t n, widx_t ix)
- **wbindex** & **Index** (widx_t i1, widx_t i2)
- **wbindex** & **Index** (widx_t i1, widx_t i2, int p)
- **wbindex** & **BlockIndex** (const [wbvector](#)< widx_t > &D, char sub=0)
- **wbindex** & **BlockIndex** (widx_t n, widx_t D)
- **wbindex** & **BlockIndexI** (widx_t n, widx_t D)
- **wbindex** & **IndexTranspose** (widx_t m, widx_t n)
- **wbindex** & **operator=** (const char *s)
- int **extend2Perm** (widx_t N, [wbperm](#) &P, char toend=0) const
- void **toPerm** ([wbperm](#) &P) const
- **wbindex** & **invert** (widx_t N, [wbindex](#) &I2, char uflag=1) const
- **wbindex** & **Invert** (widx_t N, char uflag=1)
- **wbindex** **flipldx** (widx_t ndim) const
- char **isUnique** (widx_t imax, widx_t offset=0) const
- mxArray * **toMx** (const char tflag=0) const

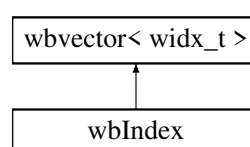
Additional Inherited Members

The documentation for this class was generated from the following files:

- [wbindex.hh](#)
- [wbindex.cc](#)

4.101 **wbIndex** Class Reference

Inheritance diagram for **wbIndex**:



Public Member Functions

- **wbIndex** (const [wbvector](#)< widx_t > &S)
- template<class TI >
wbIndex (widx_t n, const TI *S)
- **wbIndex** (const [wbIndex](#) &I)
- [wbIndex](#) & **init** ()
- [wbIndex](#) & **init** (const [wbvector](#)< widx_t > &S)
- [wbIndex](#) & **reset** ()
- template<class TI >
[wbIndex](#) & **init** (unsigned n, const TI *S)
- [wbIndex](#) & **set** ([wbvector](#)< widx_t > &I)
- bool **isValid** () const
- void **checkValid** (const char *F, int L) const
- bool **operator++** ()
- bool **operator--** ()
- bool **isEmpty** () const
- bool **gotmore** () const
- bool **isdiag** () const
- widx_t **numel** () const
- widx_t **serial** (const [wbvector](#)< widx_t > *S2=NULL)
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStr** () const
- mxArray * **toMx** () const

Public Attributes

- [wbvector](#)< widx_t > **SIZE**

Additional Inherited Members

The documentation for this class was generated from the following file:

- [wbindex.hh](#)

4.102 Wb::WbListMTRACK Class Reference

Public Member Functions

- template<class T >
void **rm_ptr** (const char *F, const int &L, T *&p)
- template<class T >
void **add_ptr** (const char *F, int L, T *&p, size_t I)
- template<class T >
bool **exists** (T *&p)
- template<class T >
void **tst_exists** (const char *F, int L, T *&p)
- void **checkTotSize** ()
- char * **totStr** (char mflag=0)
- void **printSize** (const char *F, int L, char mflag=0)

Public Attributes

- map< void *, [MREC](#) > **M**
- unsigned long long **idx**
- unsigned long long **totsize**
- unsigned long long **maxsize**

The documentation for this class was generated from the following file:

- memtrack.cc

4.103 wblogbuf_struct Class Reference

Public Member Functions

- **wblogbuf_struct** (const char *s)
- **wblogbuf_struct** (const char *F, int L, const char *s)
- **wblogbuf_struct** (const [wblogbuf_struct](#) &S)
- void **unset** ()

Public Attributes

- char * **file**
- int **line**
- char * **text**

The documentation for this class was generated from the following file:

- wbsting.hh

4.104 wblogbuffer Class Reference

Public Member Functions

- void **push** (const char *F, int L, const char *s)
- void **flush** ()

The documentation for this class was generated from the following file:

- wbsting.hh

4.105 `wbMatrix< T >` Class Template Reference

Public Member Functions

- **wbMatrix** (size_t r=0, size_t c=0)
- **wbMatrix** (size_t r, size_t c, T *d, const char ref=0)
- **wbMatrix** (const [wbMatrix](#) &M)
- template<class T2 >
wbMatrix (const [wbMatrix](#)< T2 > &M)
- **wbMatrix** (const mxArray *a, char tflag=0, char ref=0)
- **wbMatrix** (const char *F, int L, const mxArray *a)
- void **copyStride** (T *dd, size_t stride) const
- [wbMatrix](#) & **operator=** (const [wbMatrix](#) &B)
- template<class TB >
[wbMatrix](#) & **operator=** (const [wbMatrix](#)< TB > &B)
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const T &x) const
- bool **operator==** (const [wbMatrix](#) &B) const
- bool **operator!=** (const [wbMatrix](#) &B) const
- bool **deepEqualP** (const [wbMatrix](#) &B) const
- bool **anyEqual** (const T &x) const
- bool **allEqual** (const T &x) const
- bool **anyUnequal** (const T &x) const
- bool **anyLT** (const T &x) const
- bool **anyGT** (const T &x) const
- bool **col_anyGT** (size_t j, const T &x) const
- [wbMatrix](#) & **operator+=** (const [wbMatrix](#) &B)
- [wbMatrix](#) **operator+** (const [wbMatrix](#) &B) const
- [wbMatrix](#) **operator+** (const T &x) const
- void **operator-=** (const [wbMatrix](#) &B)
- [wbMatrix](#) **operator-** (const [wbMatrix](#) &B) const
- void **operator*=** (const T c)
- [wbMatrix](#) & **operator+=** (const T c)
- void **operator-=** (const T c)
- [wbMatrix](#) **operator*** (const T x) const
- [wbMatrix](#) & **add** (const [wbMatrix](#) &B, const T &c=1, const char zflag=0)
- [wbMatrix](#) & **minus** (const [wbMatrix](#) &B, const T &c=1, const char zflag=0)
- [wbMatrix](#) & **times** (T fac, [wbMatrix](#) &X) const
- T **prod** () const
- T **min** () const
- T **max** () const
- T **amax** () const
- T **sum** () const
- T **cumsum0prodRec** ([wbvector](#)< T > &cs, char xflag=0) const
- T **cumsum0prodRec** ([wbvector](#)< T > &cs, unsigned i1, unsigned i2, char xflag=0) const
- T **norm** () const
- T **norm2** () const
- double **normReal** () const
- double **normImag** () const
- size_t **maxRec** (char lex=1, size_t *m=NULL) const
- size_t **maxRec_float** (char lex=1, size_t *m=NULL, T xref=-1) const
- T **recMax** (size_t r, size_t *k=NULL) const
- T **colMax** (size_t c, size_t *k=NULL) const

- `T colMax (size_t c, unsigned &k) const`
- `wbMatrix & NormalizeCol (size_t k)`
- `double recMaxA (size_t r, size_t *k=NULL) const`
- `size_t skipNanRecs (wbindex &l)`
- `wbMatrix & init (size_t r=0, size_t c=0, const T *d=NULL)`
- `wbMatrix & init (size_t r, size_t c, const T *d0, size_t stride)`
- `wbMatrix & init2val (size_t r, size_t c, const T &d)`
- `wbMatrix & initDef (size_t r=0, size_t c=0, T *d=NULL)`
- `template<class T2 >`
`wbMatrix< T > & initT (size_t r, size_t c, T2 *d)`
- `template<class T2 >`
`wbMatrix< T > & initT (const wbMatrix< T2 > &B)`
- `template<class T2 >`
`wbMatrix< T > & initT (const char *F, int L, const wbMatrix< T2 > &B)`
- `template<class TA, class TB >`
`wbMatrix< T > & init (const char *F, int L, const wbMatrix< TA > &B, const wbvector< TB > &V, char dim=2)`
- `template<class T2 >`
`wbMatrix< T > & initDim (const wbMatrix< T2 > &B)`
- `wbMatrix & init (const wbMatrix &B)`
- `wbMatrix & init (const char *F, int L, const mxArray *a, char tflag=0, char ref=0, char tcheck=1)`
- `void init (const mxArray *a)`
- `void init0 (const mxArray *a)`
- `void initTST ()`
- `wbMatrix & init2ref (size_t r, size_t c, const T *d)`
- `wbMatrix & init2ref (const wbvector< T > &v, const char tflag=0)`
- `wbMatrix & init2ref (const wbMatrix &M)`
- `wbMatrix & unRef ()`
- `wbMatrix & save2 (wbMatrix &A, char ref=0)`
- `void ref_check (const char *F, int L, const char *fct)`
- `wbMatrix & ResizeXRecs (size_t r)`
- `wbMatrix & Resize (size_t r, size_t c, const T *dx=NULL)`
- `wbMatrix & resize (size_t m, size_t n, wbMatrix &M) const`
- `wbMatrix & Resize2Mult (const WBINDEX &M)`
- `wbMatrix & transpose (wbMatrix &M) const`
- `wbMatrix & Transpose ()`
- `wbMatrix & Reshape (size_t r, size_t c)`
- `template<class T2 >`
`wbMatrix< T > & AddCols (size_t n, const T2 *d0=NULL)`
- `template<class T2 >`
`wbMatrix< T > & AddRows (size_t n, const T2 *d0=NULL)`
- `bool isEmpty () const`
- `bool isNormal () const`
- `bool isSquare () const`
- `bool isSquare (size_t d) const`
- `bool isScalar () const`
- `bool isVector () const`
- `bool isDiag () const`
- `bool isIdentity (const double &eps, double &maxdiff) const`
- `bool isHConj (const wbMatrix &B, double eps=1E-12, double *xref=NULL) const`
- `bool isHConj (double eps=1E-12, double *xref=NULL) const`
- `bool isAHerm (const wbMatrix &B, double eps=1E-12, double *xref=NULL) const`
- `bool isAHerm (double eps=1E-12, double *xref=NULL) const`
- `bool isComplex () const`
- `bool isUnique () const`

- bool **isUniqueSorted** (char dir=0, char lex=1) const
- int **isSorted** (char dir=0, char lex=1, size_t m=-1) const
- bool **quickCheckSorted** (char dir, char lex=1, const char *F=NULL, int L=0, size_t n=2) const
- bool **thisIsInt** () const
- int **getDiff** (const [wbMatrix](#) &B, [wbindex](#) &la, [wbindex](#) *lb=NULL) const
- int **getDiffSorted** (const [wbMatrix](#) &B, [wbindex](#) &la, [wbindex](#) *lb=NULL) const
- size_t **length** () const
- size_t **totSize** () const
- size_t **numel** () const
- void **flipSign** ()
- [wbMatrix](#) & **Symmetrize** ()
- void **set** (const T &x)
- [wbMatrix](#) & **setRand** (double fac=1., double shift=0.)
- void **setDiagRand** (double fac=1., double shift=0.)
- double **maxRelDiff** (const [wbMatrix](#) &M) const
- template<class T2 >
double **normDiff** (const [wbMatrix](#)< T2 > &M) const
- T **normDiff2** (const [wbMatrix](#) &M, size_t *k=NULL) const
- T **normDiff** (const [wbMatrix](#) &M, size_t *k=NULL) const
- bool **hasSameSize** (const [wbMatrix](#) B) const
- WBINDEX & **toIndex** (WBINDEX &I, const WBINDEX *S=NULL) const
- WBIDXMAT & **toIndex2D** (const WBINDEX &S, const [wbindex](#) &ic, WBIDXMAT &IJ, char pos=1) const
- [wbMatrix](#) & **SkipTiny_float** (T ref __attribute__((unused))=-1)
- void **reset** ()
- size_t **nnz** () const
- [wbMatrix](#) & **CAT** (C_UINT d12, const [wbMatrix](#) **M, C_UINT len)
- [wbMatrix](#) & **CAT** (C_UINT d12, const [wbvector](#)< [wbMatrix](#) > &M0)
- [wbMatrix](#) & **CAT** (C_UINT d12, [wbvector](#)< [wbMatrix](#) const * > M)
- [wbMatrix](#) & **CAT** (C_UINT d12, const [wbvector](#)< [wbvector](#)< T > > &M)
- [wbMatrix](#) & **CAT** (C_UINT d12, const [wbvector](#)< [wbvector](#)< T > const * > &M)
- [wbMatrix](#) & **cat** (C_UINT d12, const [wbMatrix](#) &)
- [wbMatrix](#) & **cat** (C_UINT d12, const [wbMatrix](#) &, const [wbMatrix](#) &)
- [wbMatrix](#) & **cat** (C_UINT d12, const [wbMatrix](#) &, const [wbMatrix](#) &, const [wbMatrix](#) &)
- [wbMatrix](#) & **Cat** (C_UINT d12, const [wbMatrix](#) &, const [wbMatrix](#) &)
- [wbMatrix](#) & **Cat** (C_UINT d12, const [wbMatrix](#) &, const [wbMatrix](#) &, const [wbMatrix](#) &)
- [wbMatrix](#) & **Cat** (C_UINT d12, const [wbMatrix](#) &, const [wbMatrix](#) &, const [wbMatrix](#) &, const [wbMatrix](#) &)
- [wbMatrix](#)< T > & **CAT** (const unsigned dim, [wbvector](#)< [wbMatrix](#)< T > const * > M, [wbvector](#)< WBINDEX const * > I)
- [wbMatrix](#) & **Cat** (const unsigned dim, const [wbMatrix](#) &M1, const [wbindex](#) &I1, const [wbMatrix](#) &M2, const [wbindex](#) &I2)
- void **split** (C_UINT d12, [wbvector](#)< [wbMatrix](#) * > &M) const
- void **split** (C_UINT d12, [wbMatrix](#) **M, C_UINT len) const
- void **split** (C_UINT d12, [wbMatrix](#) &M1, [wbMatrix](#) &M2, [wbMatrix](#) &M3) const
- void **swap** ([wbMatrix](#) &B)
- const T & **operator()** (size_t i, size_t j, const T &x) const
- T & **operator()** (size_t i, size_t j, const T &x)
- T & **operator()** (size_t i, size_t j)
- const T & **operator()** (size_t i, size_t j) const
- const T & **el** (size_t i, size_t j) const
- T & **el** (size_t i, size_t j)
- const T & **els** (long i, long j) const
- T & **els** (long i, long j)
- const T * **operator()** (size_t i) const
- T * **operator()** (size_t i)
- const T & **operator[]** (size_t i) const

- **T & operator[]** (size_t i)
- **const T * ref** (size_t i, size_t j=0) const
- **T * ref** (size_t i, size_t j=0)
- **const T * rec** (size_t i) const
- **T * rec** (size_t i)
- **void getDiag** (wbvector< T > &D) const
- **wbvector< T > getDiag** () const
- **void appendRows** (size_t n, const T *=NULL)
- **void appendRow** (const wbvector< T > &v)
- **mxAarray * toMx** (char raw=0) const
- **mxAarray * toMxP** (char flag=0) const
- **mxAarray * toMx_base** (char raw=0) const
- **mxAarray * toMx_base_d** (char raw=0) const
- **mxAarray * toMx_Struct** () const
- **mxAarray * toMx_StructP** () const
- **mxAarray * toMx_CellP** () const
- **mxAarray * toMxT** (char raw=0) const
- **mxAarray * mxCreateStruct** (unsigned m, unsigned n) const
- **void add2MxStruct** (mxAarray *S, unsigned i, char tst=0) const
- **void toMxStruct** (mxAarray *S, const char *vname) const
- **void mat2mx** (mxAarray *a) const
- **void mat2mxs** (mxAarray *a, char field_nr) const
- **void info** (const char *istr="ans") const
- **void print** (const char *istr="", char mflag=0) const
- **void Print** (const char *istr="", char mflag=0) const
- **void recPrint** (size_t i, const char *istr="", char mflag=0) const
- **void printdata** (const char *istr, const char *dbl_fmt="%8.4g", const char *rsep="\n") const
- **wbstring toStr** (const char *fmt="", const char *sep=" ", const char *rsep=" ", size_t stride=0, const char *sep2="") const
- **wbstring rec2Str** (size_t k, const char *fmt="", const char *sep=" ", size_t stride=0, const char *sep2="") const
- **wbstring sizeStr** () const
- **void put** (const char *F, int L, const char *vname="ans", const char *ws="caller", const char raw=0) const
- **void put** (const char *vname="ans", const char *ws="caller", const char raw=0) const
- **void putg** (const char *vname="ans", const char raw=0) const
- **void putx** (const char *vname="ans", const char raw=0) const
- **void put0** (const char *vname="ans", const char *ws="caller") const
- **void getReal** (wbMatrix< double > &R) const
- **void getImag** (wbMatrix< double > &I) const
- **void set** (const wbMatrix< double > &R, const wbMatrix< double > &I)
- **wbMatrix & Conj** ()
- **void setRec** (size_t r, const T x)
- **void setRec** (size_t r, const T x1, const T x2)
- **void recSet** (size_t idest, size_t isrc)
- **void recSet** (size_t, const wbvector< T > &)
- **void recSet** (size_t, const wbMatrix &, size_t)
- **void recSet** (size_t i, const wbvector< T > &, const wbvector< T > &)
- **void recSetP** (size_t, const T *, size_t n=-1)
- **void recSetP** (size_t, const T *, size_t, const T *, size_t)
- **void recSetP** (size_t, const T *, const wbindex &i1, const T *, const wbindex &i2)
- **void recSetB** (size_t, size_t, size_t D, const T *, const T *=NULL, const T bfac=1)
- **void recSetB** (size_t, const wbindex &, size_t D, const T *)
- **void recSet** (size_t k, const wbvector< T > &a, const wbindex &ia, const wbvector< T > &b, const wbindex &ib)
- **void recSet** (size_t k, const T *a, const wbindex &ia, const T *b, const wbindex &ib)

- `template<class T2 >`
`void setCol (size_t k, const wbvector< T2 > &v)`
- `template<class T2 >`
`void setCol (size_t k, const T2 *v, size_t stride=1)`
- `void setCol (size_t k, const T &x)`
- `template<class T2 >`
`void setColT (size_t k, const wbvector< T2 > &v)`
- `template<class T2 >`
`void setColT (size_t k, const T2 *v, size_t stride=1)`
- `void setLastCol (const T &x)`
- `wbMatrix & scaleCol (size_t k, const T &fac)`
- `wbMatrix & scaleRow (size_t k, const T &fac)`
- `void recAddP (size_t, const T *)`
- `int recLess (size_t, size_t, char lex=1) const`
- `int recLessE (size_t, size_t, char lex=1) const`
- `int recEqual (size_t, size_t) const`
- `int recEqual (size_t j1, const T *d) const`
- `T recDiff2 (size_t i1, size_t i2, size_t n=-1) const`
- `T recDiff2P (size_t i1, const T *b, size_t n=-1) const`
- `bool recAllEqual (const T *v)`
- `bool recAllEqual ()`
- `bool recIsZero (size_t r)`
- `char recCompare (size_t, size_t, size_t m=-1, char lex=1, T eps=0) const`
- `char recCompare (size_t, const wbvector< T > &r2, char lex=1) const`
- `char recCompareP (size_t, const T *, size_t m=-1, char lex=1, T eps=0) const`
- `void set2Rec (size_t i)`
- `wbMatrix & Set2Recs (const WBINDEX &I)`
- `wbvector< T > & getRec (size_t j0, wbvector< T > &v) const`
- `wbvector< T > getRec (size_t j0) const`
- `size_t UnionRecs (const wbMatrix &B, wbindex &Ia)`
- `bool gotRecOverlap (const wbMatrix &B) const`
- `bool gotRecOverlapSA (const wbMatrix &B) const`
- `size_t findValsCol (const size_t k, const wbvector< T > &v, wbindex &Ia) const`
- `wbMatrix & getRecs (const WBINDEX &, wbMatrix &M) const`
- `wbMatrix getRecs (const WBINDEX &I) const`
- `wbMatrix & getRecs (size_t j1, size_t j2, wbMatrix &M, size_t m=0) const`
- `wbMatrix getRecs (size_t i1, size_t i2, size_t m=0) const`
- `wbvector< T > & getCol (const size_t c, wbvector< T > &v) const`
- `wbvector< T > getCol (const size_t c) const`
- `wbMatrix getCols (const size_t j1, const size_t j2) const`
- `wbMatrix & getCols (size_t j1, size_t j2, wbMatrix &M) const`
- `template<class TI >`
`wbMatrix & getCols (size_t n, const TI *I, wbMatrix< T > &M) const`
- `wbMatrix & getCols (const wbperm &p, wbMatrix &M) const`
- `wbMatrix & getCols (const wbindex &I, wbMatrix &M, char iflag=0) const`
- `wbMatrix & set2Cols (const size_t j1, const size_t j2)`
- `wbMatrix & set2Cols (const wbindex &J, char iflag=0)`
- `wbMatrix & recPermute (const wbperm &P, wbMatrix &M, char iflag=0) const`
- `wbMatrix & recPermute (const wbperm &P, char iflag=0)`
- `wbMatrix & colPermute (const wbperm &P, wbMatrix &M, char iflag=0) const`
- `wbMatrix & colPermute (const wbperm &P, char iflag=0)`
- `wbMatrix & FlipRecs ()`
- `wbMatrix & FlipCols ()`
- `wbMatrix & cols2Front (const WBINDEX &I1, wbMatrix &M) const`
- `wbMatrix & cols2End (const WBINDEX &I1, wbMatrix &M) const`

- void **blockPermute** (const [wbperm](#) &P, [wbMatrix](#) &M) const
- void **BlockPermute** (const [wbperm](#) &P)
- [wbvector](#)< T > & **recProd** ([wbvector](#)< T > &p) const
- T **recProd** (size_t) const
- T **recSum** (size_t) const
- [wbvector](#)< T > & **recSum** ([wbvector](#)< T > &) const
- [wbvector](#)< T > & **recSumA** ([wbvector](#)< T > &) const
- [wbvector](#)< T > **recSum** () const
- T **colSum** (size_t) const
- [wbMatrix](#) & **repmat** (size_t m, size_t n, [wbMatrix](#) &Q, size_t pad1=0, size_t pad2=0) const
- [wbMatrix](#) & **Repmat** (size_t m, size_t n, size_t pad1=0, size_t pad2=0)
- void **ColKron** (const [wbMatrix](#) &Q, int d=-1)
- [wbMatrix](#) & **getBlocks** (size_t j1, size_t j2, const size_t D, [wbMatrix](#) &M) const
- [wbMatrix](#) & **getBlock** (size_t j1, const size_t D, [wbMatrix](#) &M) const
- [wbMatrix](#) **getBlock** (size_t j1, const size_t D) const
- template<class T2 >
[wbMatrix](#)< T > & **getBlock** (size_t k, size_t D, const [wbMatrix](#)< T2 > &R, size_t k2, size_t D2, [wbMatrix](#) &M) const
- [wbMatrix](#) & **getBlock** (size_t i, size_t j, size_t n, size_t m, [wbMatrix](#) &M) const
- [wbMatrix](#) & **blockSum** (size_t D, [wbMatrix](#) &) const
- void **blockSum** (size_t, size_t, const size_t, [wbMatrix](#)< double > &) const
- void **blockSum** (const [wbindex](#) &J, const size_t, [wbMatrix](#)< double > &) const
- void **setBlock** (size_t k, const size_t D, const T *)
- void **SetBlock** (size_t i0, size_t j0, const [wbMatrix](#) &M)
- WBIDXMAT & **toBlockIndex** (widx_t D, WBIDXMAT &I, [wbvector](#)< [wbMatrix](#) > *QI=NULL) const
- template<class T2 >
void **toBlockIndex** (widx_t D, [wbMatrix](#)< T2 > &R, widx_t DR, WBIDXMAT *I=NULL, WBIDXMAT *IS=NULL, WBIDXMAT *SS=NULL, [wbvector](#)< [wbMatrix](#) > *QI=NULL) const
- void **groupRecs** ([wbperm](#) &P, WBINDEX &d, size_t m=-1, char lex=1, [wbindex](#) *Ig=NULL)
- void **groupRecs** ([wbindex](#) &Ig, widx_t m=-1, char lex=1)
- void **groupRecs** ([groupIndex](#)< widx_t > &IG, size_t m=-1, char lex=1, [wbindex](#) *Ig=NULL)
- void **groupRecs** ([groupIndex](#)< widx_t > &IG, size_t m, char lex, const wperm_t **p)
- void **groupRecs** ([wbperm](#) &P, WBINDEX &d, [wbMatrix](#) &B, size_t m=-1, char lex=1, [wbindex](#) *Ig=NULL) const
- void **groupRecs** ([wbperm](#) &P, WBINDEX &d, const WBINDEX *I, char lex=1)
- template<class T2 >
void **groupRecs_rdeg** ([wbperm](#) &P, WBINDEX &D, const [wbMatrix](#)< T2 > &R, WBINDEX *Ib=NULL, WBINDEX *I2=NULL, WBINDEX *Sb=NULL, char iflag=0)
- void **groupRecs** ([wbperm](#) &P, WBINDEX &D, size_t nc, WBINDEX &Ib, WBINDEX &I2, WBINDEX &Sb, char iflag=0)
- void **groupSortedRecs** (WBINDEX &d, char keepall=0, size_t m=-1, char lex=1)
- void **groupSortedRecs** (WBINDEX &d, const WBINDEX *I)
- void **groupSortedRecs** (WBINDEX &d, size_t mc, char lex, WBINDEX &Ib, WBINDEX &I2, WBINDEX &Sb)
- void **makeUnique** ()
- void **makeUnique** ([wbperm](#) &P, WBINDEX &D)
- void **makeUnique** ([wbindex](#) &I)
- void **findUnique** ([wbindex](#) &I) const
- size_t **findUniqueRecSorted1** (size_t n=-1, T eps=T(1E-12)) const
- void **makeUnique_ig** ([wbindex](#) &Iu)
- void **makeUnique** ([groupIndex](#)< widx_t > &IG)
- [wbMatrix](#) & **SortRecs** ([wbperm](#) &P, char dir=+1, char lex=1)
- [wbMatrix](#) & **sortRecs** ([wbMatrix](#) &B, [wbperm](#) &P, char dir=+1, char lex=1)
- [wbMatrix](#) & **SortRecs** (char dir=+1, char lex=1)
- [wbMatrix](#) & **sortRecs_float** ([wbperm](#) &P, char dir=+1)
- [wbMatrix](#) & **sortRecs_float** ()

- char **findRecsInSet** (const [wbMatrix](#) &S, [wbvector](#)< int > &I) const
- int **findRecSorted** (const T *r, size_t n=-1, char lex=1) const
- int **findRec** (const T *r, size_t n=-1, char lex=1) const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- WBINDEX & **toIndex** (WBINDEX &I, const WBINDEX *S) const
- WBIDXMAT & **toIndex2D** (const WBINDEX &S, const [wbindex](#) &Ic, WBIDXMAT &IJ, char pos) const
- template<class T1 >
[wbMatrix](#)< T > & **getCols** (size_t n, const T1 *I, [wbMatrix](#)< T > &M) const
- template<class T2 >
void **groupRecs_rdeg** ([wbperm](#) &P, WBINDEX &D, const [wbMatrix](#)< T2 > &R, [wbvector](#)< widx_t > *Ib, [wbvector](#)< widx_t > *I2, [wbvector](#)< widx_t > *Sb, char iflag)
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const

Public Attributes

- T * **data**
- size_t **dim1**
- size_t **dim2**
- char **isdiag**
- char **isref**

Protected Member Functions

- void **NEW_DATA** (T *d=NULL)
- void **RENEW** (const size_t &d1, const size_t &d2, const T *dd=NULL, char iflag=1)

The documentation for this class was generated from the following files:

- [wblib.h](#)
- [wbMatrix.hh](#)
- [wbMatrix.cc](#)

4.106 wbMsg Class Reference

Public Member Functions

- **wbMsg** (const char *file, int line, const char *istr="Message queue", int key_=0)
- int **Rec** (const char *, int, [wbMsgBuf](#) &rbuf, unsigned flag=IPC_NOWAIT)
- int **Snd** (const char *, int, [wbMsgBuf](#) &sbuf, const char *istr, double val=0., unsigned type=1)

Public Attributes

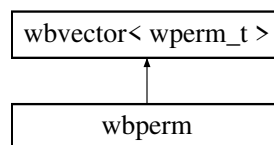
- [wbstring](#) **info**
- int **msqid**
- int **msgflg**
- key_t **key**

The documentation for this class was generated from the following file:

- wbmsg.hh

4.107 wbpemr Class Reference

Inheritance diagram for wbpemr:



Public Member Functions

- **wbpemr** (wperm_t n=0, char reverse=0)
- **wbpemr** (const [wbpemr](#) &P, char iflag=0, unsigned r=-1)
- **wbpemr** (const [wbpemr](#) &P, size_t r)
- template<class T >
 wbpemr (const [wbvector](#)< T > &b, wperm_t offset=0)
- **wbpemr** (const [wbindex](#) &P)
- **wbpemr** (const char *s, wperm_t offset=1)
- wperm_t **el1** (wperm_t i) const
- [wbpemr](#) & **operator=** (const char *s)
- int **initStr** (const char *F, int L, const char *s, wperm_t offset=1)
- [wbpemr](#) & **init** (const [wbpemr](#) &P, char iflag=0, unsigned r=-1)
- [wbpemr](#) & **init_trafo** (const [wbpemr](#) &p1_, const [wbpemr](#) &p2_)
- [wbpemr](#) & **init_pad** (const [wbpemr](#) &P, wperm_t r)
- [wbpemr](#) & **init** (wperm_t n=0, char reverse=0)
- [wbpemr](#) & **init** (wperm_t n, const wperm_t *d)
- [wbpemr](#) & **init** (const char *F, int L, const mxArray *a)
- [wbpemr](#) & **init** (const char *F, int L, const mxArray *a, wperm_t offset)

- `template<class T >`
`wbperm & init` (const char *F, int L, const `wbvector`< T > &b, wperm_t offset=0)
- `wbperm & initTranspose` (wperm_t r)
- `wbperm & initPerm2` (wperm_t r)
- `wbperm & initFirstTo` (wperm_t k, wperm_t N)
- `wbperm & initLastTo` (wperm_t k, wperm_t N)
- `wbperm & init2Front` (const WPERM &l, wperm_t N)
- `wbperm & init2End` (const WPERM &l, wperm_t N)
- `wbperm & init2FrontB` (wperm_t m, wperm_t N)
- `wbperm & init2EndB` (wperm_t m, wperm_t N)
- `wbperm & init2Front` (wperm_t k, wperm_t N)
- `wbperm & init2End` (wperm_t k, wperm_t N)
- `wbperm & Cycle` (wperm_t k1, wperm_t k2)
- `wbperm & initCycle` (wperm_t n, wperm_t k1, wperm_t k2)
- `wbperm & Index` (wperm_t n, char reverse=0)
- `wbperm & Index` (wperm_t i1, wperm_t i2)
- `char isValidPerm` (const char *F, int L, wperm_t r=-1) const
- `char isValidPerm` (wperm_t r=-1) const
- `wbperm & Invert` ()
- `wbperm & invert` (`wbperm` &iP) const
- `wbperm & Rotate` (swperm_t l)
- `wbperm & initRotate` (wperm_t n, swperm_t k)
- `wbperm & Permute` (const `wbperm` &P, char iflag, wperm_t r)
- `wbperm & Permute` (const `wbperm` &P, char iflag=0)
- `wbperm & Extend` (wperm_t r)
- `bool sameAs` (const `wbperm` &b) const
- `bool isIdentityPerm` () const
- `bool isIdentityPerm` (const char *F, int L, wperm_t r=-1) const
- `bool isReversePerm` () const
- `bool isCyclic2F` (wperm_t m, wperm_t n=-1, char iflag=0) const
- `bool isOpTranspose` () const
- `wbperm & times` (const `wbperm` &p2, `wbperm` &Pout) const
- `wbperm & flipIdx` (`wbperm` &P) const
- `wbperm & FlipIdx` ()
- `wbperm flipIdx` () const
- `void ToRaw` ()
- `void ToRaw` (wperm_t n)
- `wbstring toStr` () const
- `mxAarray * toMx` (const char tflag=0) const

Additional Inherited Members

The documentation for this class was generated from the following files:

- `wbperm.hh`
- `wbindex.hh`
- `wbperm.cc`

4.108 **wbRecs**< T > Class Template Reference

Public Member Functions

- **wbRecs** (const [wbMatrix](#)< T > &A, [wbperm](#) &P, char dir_=1, char lex_=1)
- bool **operator()** (const size_t &pa, const size_t &pb) const

The documentation for this class was generated from the following file:

- [wbMatrix.hh](#)

4.109 **wbREF**< TX > Class Template Reference

Public Member Functions

- **wbREF** (const TX &A, char flag)
- TX & **Ref** ()

The documentation for this class was generated from the following file:

- [gathered.hh](#)

4.110 **wbSigHandler** Class Reference

Public Member Functions

- **wbSigHandler** (const char *F, int L)
- int **check911** (char tflag=0)

Static Public Member Functions

- static int **check911_** (const char *F=NULL, int L=0)

Static Public Attributes

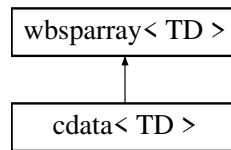
- static unsigned **NC0** =3
- static unsigned **icount** =wbSigHandler::NC0
- static unsigned **acount** =wbSigHandler::NC0

The documentation for this class was generated from the following file:

- [gathered.hh](#)

4.111 `wbsparray< TD >` Class Template Reference

Inheritance diagram for `wbsparray< TD >`:



Public Member Functions

- `wbsparray` (SPIDX_T d1)
- `wbsparray` (SPIDX_T d1, SPIDX_T d2)
- `wbsparray` (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- `wbsparray` (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- `wbsparray` (const `wbvector`< SPIDX_T > &d, SPIDX_T l=0)
- `wbsparray` (char *sidx, SPIDX_T l=0)
- `wbsparray` (const `wbarray` &a)
- `wbsparray` (const char *F, int L, const `mxArray` *a)
- `wbsparray` (const `mxArray` *a)
- `wbarray` & `init` (const char *F, int L, const `wbarray`< TD > &A, TD eps=0)
- template<class TA >
`wbarray`< TA > & `toFull` (`wbarray`< TA > &A) const
- template<class TA >
`wbvector`< TA > & `toFull` (`wbvector`< TA > &A) const
- template<class DB >
`wbsparray` & `init` (const `wbsparray`< DB > &a)
- SPIDX_T `catRecs` (SPIDX_T i0, const `wbsparray` &a, SPIDX_T id)
- `wbsparray` & `init` (const `wbvector`< SPIDX_T > &S0, SPIDX_T l=0)
- template<class I2 >
`wbsparray` & `init` (const `wbvector`< I2 > &S_, SPIDX_T l=0)
- `wbsparray` & `init` ()
- `wbsparray` & `init` (SPIDX_T d1)
- `wbsparray` & `init` (SPIDX_T d1, SPIDX_T d2)
- `wbsparray` & `init` (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- `wbsparray` & `init` (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- `wbsparray` & `initz` ()
- `wbsparray` & `initz` (SPIDX_T d1, SPIDX_T n)
- `wbsparray` & `initz` (SPIDX_T d1, SPIDX_T d2, SPIDX_T n)
- `wbsparray` & `initz` (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T n)
- `wbsparray` & `init_rzd` (SPIDX_T r, SPIDX_T n)
- `wbsparray` & `init_nnz` (SPIDX_T n=0, char kflag=0)
- `wbsparray` & `setRand` (double x=1, char sflag=0)
- `wbsparray` & `setRand_data` (double x=1, char sflag=0)
- template<class DB >
`wbsparray` & `init` (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const `wbarray`< DB > &B)
- template<class DB >
`wbsparray` & `init` (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const `wbarray`< DB > &B)
- template<class DB >
`wbsparray` & `init` (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const `wbarray`< DB > &B)
- `wbarray` & `initEmpty` (const `wbarray` &a)

- [wbsparray](#) & **init** (const char *F, int L, const mxArray *a, unsigned k=0)
- [wbsparray](#) & **init2ref** (const [wbsparray](#) &a)
- [wbsparray](#) & **unRef** ()
- [wbsparray](#) & **initCAT** (const char *F, int L, [wbvector](#)< [wbsparray](#) * > &X, [wbvector](#)< SPIDX_T > *I0=NULL, char del=0)
- [wbsparray](#) & **initCAT** (const char *F, int L, [wbvector](#)< [wbsparray](#) > &X, const [wbvector](#)< SPIDX_T > *I0=NULL)
- [wbsparray](#) & **initCAT** (const char *F, int L, const [wbvector](#)< const [wbsparray](#) * > &X, unsigned dim)
- [wbsparray](#) & **initCAT** (const char *F, int L, const [wbvector](#)< [wbsparray](#) > &X, unsigned dim)
- [wbsparray](#) & **Cat** (const char *F, int L, const [wbsparray](#) &B, unsigned dim)
- [wbsparray](#) & **init_kron** (const [wbvector](#)< SPIDX_T > &sa, const [wbvector](#)< SPIDX_T > &sb, SPIDX_T nab, char kflag)
- void **setSize_kron** (const char *F, int L, const [wbvector](#)< SPIDX_T > &sa, const [wbvector](#)< SPIDX_T > &sb, char kflag=0)
- void **setSize_kron** (const char *F, int L, char kflag=0)
- [wbsparray](#) & **setRec** (SPIDX_T i, SPIDX_T j)
- template<class T2 >
[wbsparray](#) & **setRec** (SPIDX_T i, SPIDX_T i1, SPIDX_T i2, const T2 &x)
- [wbsparray](#) & **setRecP** (SPIDX_T i, const SPIDX_T *I, const TD &x)
- [wbsparray](#) & **setRec_kron** (SPIDX_T I, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2, unsigned kflag=0)
- [wbsparray](#) & **Resize** (SPIDX_T I)
- [wbsparray](#) & **REDSIZE_NNZ** (SPIDX_T I)
- [wbsparray](#) & **Resize** (const [wbvector](#)< SPIDX_T > &S)
- [wbsparray](#) & **operator=** (const [wbsparray](#) &a)
- template<class DB >
[wbsparray](#) & **operator=** (const [wbsparray](#)< DB > &a)
- [wbsparray](#) & **save2** ([wbsparray](#) &a, char ref=0)
- SPIDX_T **sub2ind** (SPIDX_T k) const
- void **ind2sub** (SPIDX_T k, SPIDX_T *I) const
- void **ind2sub** (SPIDX_T k, [wbindex](#) &I) const
- [wbsparray](#)< TD > & **make2D** (const [wbindex](#) &ic, [wbsparray](#)< TD > &B, char pos=1) const
- [sparseIndex2D](#)< SPIDX_T > & **toIndex2D** (const [wbindex](#) &ic, [sparseIndex2D](#)< SPIDX_T > &a, char pos, char use_rmaj=0) const
- [wbsparray](#) & **initDiag** (SPIDX_T d)
- [wbsparray](#) & **initDiag** (const [wbvector](#)< TD > &dd)
- [wbsparray](#) & **initDiag2** (SPIDX_T d, TD dval=1)
- bool **isDiagMatrix** (TD eps=1E-14) const
- bool **isDiag** (const char *F=0, int L=0) const
- [wbvector](#)< TD > & **getDiag** (const char *F, int L, [wbvector](#)< TD > &dd) const
- TD **getDiag** (const char *F, int L, SPIDX_T k) const
- [wbsparray](#) & **getDiag** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **diag2reg** (const char *F=0, int L=0, unsigned r=-1)
- [wbsparray](#) & **diag2reg** (const char *F, int L, [wbsparray](#) &A, unsigned r=-1) const
- [wbsparray](#) & **toDiag** (const char *F=0, int L=0)
- int **toDiagSafe** (const char *F, int L, TD eps=1E-14)
- [wbsparray](#) & **initIdentity** (SPIDX_T d, TD dval=1)
- [wbsparray](#) & **initIdentity** (SPIDX_T d1, SPIDX_T d2, sSPIDX_T m=0, TD dval=1)
- [wbsparray](#) & **initIdentity** (const [wbvector](#)< SPIDX_T > &S0, int m=-1, sSPIDX_T k=0, char fflag='f')
- [wbsparray](#) & **initIdentityB3** (SPIDX_T d1, SPIDX_T d2, SPIDX_T D, sSPIDX_T i0=0, char fflag='f')
- unsigned **rank** (const char *F=NULL, int L=0) const
- SPIDX_T **dim** () const
- SPIDX_T **dim** (unsigned k) const
- SPIDX_T **dim0** (unsigned k) const
- SPIDX_T **matDim** (unsigned k=1)
- SPIDX_T **maxDim** (unsigned r=-1) const

- TD **norm2** (char tnorm=0) const
- [wbstring](#) **sizeStr** () const
- TD **SkipTinyCols** (TD eps=1E-14)
- TD **SkipTiny** (TD eps=1E-14)
- bool **isVector** (const char *F=0, int L=0) const
- bool **isMatrix** (const char *F=0, int L=0) const
- bool **isRank** (unsigned r) const
- bool **isSMatrix** (const char *F=0, int L=0, SPIDX_T *d=NULL) const
- bool **isSMatrix** (const char *F, int L, SPIDX_T d) const
- bool **isEmpty** () const
- bool **isEmpty_** () const
- **operator bool** () const
- bool **operator!** () const
- bool **isZero** (TD eps=0, char flag=0) const
- SPIDX_T **nnz** () const
- SPIDX_T **nnz** (const TD eps1, const TD eps2=1E9) const
- SPIDX_T **nnz_diag** () const
- SPIDX_T **numel** () const
- TD **norm** (char tnorm=0) const
- template<class T2 >
[wbvector](#)< T2 > & **norm2vec** (unsigned k, [wbvector](#)< T2 > &A, char tnorm=0) const
- void **swap** ([wbsparray](#) &B)
- [wbsparray](#) & **TimesEI** (const [wbsparray](#) &B, char tnorm=0)
- TD **dotProd** (const char *F, int L, const [wbsparray](#) &B, char tnorm=0) const
- TD **froNorm2** (const [wbsparray](#) &B) const
- double **normDiff2** (const [wbsparray](#) &B) const
- double **normDiff** (const [wbsparray](#) &B) const
- TD **Normalize** (char tnorm=0, char qflag=0)
- TD **NormalizeCol** (SPIDX_T k, char tnorm=0, char qflag=0)
- [wbsparray](#) & **QRdecomp** (const char *F, int L, [wbsparray](#) &Q, char posR=0, TD eps=1E-14)
- [wbsparray](#) & **OrthoNormalizeColsQR** (const char *F=NULL, int L=0, TD eps=1E-14)
- [wbsparray](#) & **OrthoNormalizeColsGS** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, TD eps=1E-14, unsigned np=1)
- [wbsparray](#) & **setCol** (const char *F, int L, SPIDX_T k, const [wbsparray](#) &v)
- [wbsparray](#) & **setCol** (SPIDX_T k, const [wbsparray](#) &v)
- double **sparsity** () const
- int **checkTrailingSingletons** (const char *F=0, int L=0, unsigned *r=NULL) const
- int **skipTrailingSingletons** (const char *F=0, int L=0, unsigned r=-1)
- int **skipTrailingSingletons** (const char *F, int L, [wbsparray](#) &c, unsigned r=-1) const
- bool **hasSingletons** (const [wbindex](#) &l) const
- [wbsparray](#) & **skipSingletons** (const char *F, int L, const [wbindex](#) &l, [wbsparray](#) &A) const
- [wbsparray](#) & **AddTrailingSingletons** (const char *F, int L, unsigned r)
- [wbsparray](#) & **initScalar** (const TD x=0, unsigned r=-1)
- char **isScalar** (const char *F=0, int L=0, char zflag=0) const
- [wbsparray](#) & **toScalar** (const char *F=0, int L=0, unsigned r=-1)
- TD **getScalar** (const char *F=0, int L=0) const
- char **checkSize** (const char *F=0, int L=0, const char *fct=0, const char *istr=0) const
- char **checkSize2** (const char *F, int L, const [wbsparray](#) &B) const
- void **check_IDX_range** (const char *F=0, int L=0) const
- bool **isProptold** () const
- bool **isProptold** (TD &q, TD eps=1E-14) const
- bool **isIdentityMatrix** (TD eps=1E-14) const
- char **isIdentity** (TD eps=1E-14) const
- char **isIdentity** (const [wbperm](#) &P, TD *dval=NULL, TD eps=1E-14) const
- bool **hasSize** (SPIDX_T d1) const

- bool **hasSize** (SPIDX_T d1, SPIDX_T d2) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const
- template<class T >
 wbvector< T > & **getSize** (**wbvector**< T > &S) const
- char **fitsSize** (const **wbsparray** &, char strict=0) const
- char **sameSize** (const **wbsparray** &, char strict=0) const
- char **sameSizeUp2Singletons** (const **wbvector**< SPIDX_T > &S, char strict=0) const
- char **matchWithSingletons** (const char *F, int L, const **wbvector**< SPIDX_T > &S, **wbvector**< int > &M) const
- char **sameUpToFac** (const **wbsparray** &B, TD *fac=NULL, char lex=-1, TD eps=1E-12) const
- bool **operator==** (const **wbsparray** &B) const
- bool **operator!=** (const **wbsparray** &B) const
- **wbsparray** & **Plus** (const char *F, int L, const **wbsparray** &B, TD bfac=1, TD afac=1, TD eps=0)
- TD **sum** () const
- TD **aMin** () const
- TD **aMax** () const
- SPIDX_T **findRecSortedP** (SPIDX_T *idx, SPIDX_T n, char lex=-1)
- **wbsparray** & **Eldiv** (const char *F, int L, const **wbvector**< TD > &x_)
- **wbsparray** & **operator+=** (const **wbsparray** &B)
- **wbsparray** & **operator-=** (const **wbsparray** &B)
- **wbsparray** & **Conj** ()
- **wbsparray** & **operator*=** (TD x)
- **wbsparray** & **operator/=** (TD x)
- const TD & **operator[]** (SPIDX_T i) const
- TD & **operator[]** (SPIDX_T i)
- TD **value** (const SPIDX_T *idx, SPIDX_T n, char lex=-1) const
- **wbstring** **info2Str** (const char *istr=0) const
- size_t **memSize** () const
- void **info** (const char *istr="") const
- void **info** (const char *F, int L, const char *="") const
- void **print** (const char *istr="", const char *fmt="") const
- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **printdata** (const char *istr="ans", const char *fmt="") const
- void **sperror_this** (const char *F, int L, const char *fct=0, const char *istr=0) const
- TD **trace** () const
- template<class T2 >
 wbvector< T2 > & **trace** (unsigned k, **wbvector**< T2 > &A) const
- bool **isSym_aux** (const char *file, int line, const char *fct, const **wbsparray** &B, TD eps, TD *xref, const char symflag='s', const char lflag=1) const
- bool **isHConj** (const **wbsparray** &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isHConj** (TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (const **wbsparray** &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (TD eps=1E-12, TD *xref=NULL) const
- **wbsparray** & **Sort** (char lex=-1)
- double **Compress** (const char *F=0, int L=0, TD eps=0, char lex=-1, const **wbperm** *P=NULL)
- bool **isCompressed** (char lex=-1, char dir=0) const
- int **splitSparseCM** (const char *F, int L, **wbvector**< **wbsparray** > &X, TD eps=0, char ref=0, char tnorm=0)
- **wbsparray** & **reshape** (const **wbvector**< SPIDX_T > &S, **wbsparray** &A) const
- **wbsparray** & **Reshape** (const **wbvector**< SPIDX_T > &S)
- **wbsparray** & **Reshape** (SPIDX_T s1, SPIDX_T s2)
- **wbsparray** & **Reshape** (SPIDX_T s1, SPIDX_T s2, SPIDX_T s3)
- **wbvector**< TD > & **getCol** (SPIDX_T k, **wbvector**< TD > &v) const
- **wbvector**< TD > & **getRow** (SPIDX_T k, **wbvector**< TD > &v) const
- **wbsparray** & **getBlock** (const char *F, int L, SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2, **wbsparray**< TD > &B) const

- [wbsparray](#) & **getCol** (SPIDX_T k, [wbsparray](#)< TD > &b)
- [wbsparray](#) & **splitBlock** (const char *F, int L, sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2, [wbsparray](#)< TD > &B)
- [wbsparray](#) & **SplitQR** (sSPIDX_T k, sSPIDX_T k_, [wbsparray](#)< TD > &X)
- TD **getHouseholderVec** (const char *F, int L, SPIDX_T k, SPIDX_T j, [wbsparray](#)< TD > &b, TD eps2=0) const
- [wbsparray](#) & **IDX_Shift** (sSPIDX_T i1, sSPIDX_T i2)
- [wbsparray](#)< TD > & **LProject1** (const [wbsparray](#)< TD > &b, [wbsparray](#)< TD > &X, TD xfac=1, char tnorm=0) const
- SPIDX_T **LContractVec_omp** (const [wbsparray](#)< TD > &b, [wbvector](#)< TD > &x, char tnorm) const
- [wbsparray](#) & **splitLast** (SPIDX_T k, [wbsparray](#) &a) const
- [wbsparray](#) & **select0** (const WBINDEX &, unsigned dim, [wbsparray](#) &) const
- [wbsparray](#) & **Select0** (const WBINDEX &I, unsigned dim)
- [wbsparray](#) & **Permute** (const char *s, char iflag=0)
- [wbsparray](#) & **Permute** (const [wbperm](#) &, char iflag=0)
- [wbsparray](#) & **permute** (const char *s, [wbsparray](#) &, char iflag=0) const
- [wbsparray](#) & **permute** (const [wbperm](#) &, [wbsparray](#) &, char iflag=0) const
- [wbsparray](#) & **transpose** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **Transpose** (const char *F=0, int L=0)
- [wbsparray](#) & **htranspose** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **hTranspose** (const char *F=0, int L=0)
- [wbsparray](#) & **ColPermute** (const [wbperm](#) &P, char iflag=0)
- [wbsparray](#) & **MatPermute** (const [wbperm](#) &P, char iflag=0)
- [wbsparray](#) & **mtimes** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char sflag=0, const TD eps=0)
- char **contract_scalar** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal=[wbperm](#)(), const TD &afac=1., const TD &cfac=0.) const
- [wbsparray](#) & **contract_diag_diag** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal, const TD &afac, const TD &cfac) const
- char **contract_check_2full** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal, const TD &afac, const TD &cfac, char flag, TD eps, SPIDX_T *nnz_↵ C=NULL) const
- [wbsparray](#) & **contract** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal=[wbperm](#)(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- [wbsparray](#) & **contract** (const char *F, int L, const char *ica, const [wbsparray](#) &B, const char *icb, [wbsparray](#) &C, const [wbperm](#) &pfinal=[wbperm](#)(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- [wbarray](#)< TD > & **contract** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbarray](#)< TD > &C, const [wbperm](#) &pfinal, const TD &afac, const TD &cfac) const
- [wbsparray](#) & **contract** (const char *F, int L, unsigned i1, const [wbvector](#)< TD > &B, [wbsparray](#) &C) const
- [wbsparray](#) & **comm** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char aflag='N', char bflag='N') const
- [wbsparray](#) & **tensorProdX** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char conjA=0, const TD &afac=1, char conjB=0, const TD &cfac=0, const [wbperm](#) *Pfinal=NULL) const
- [wbsparray](#) & **tensorProd** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char aflag='N', char bflag='N', char kflag=0) const
- [wbsparray](#) & **Kron** (const [wbsparray](#) &B, char aflag='N', char bflag='N', char kflag='k')
- [wbsparray](#) & **kron** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char aflag='N', char bflag='N', char kflag='k') const
- mxArray * **toMxSp** () const
- mxArray * **toMx** (int m=-99) const
- mxArray * **IDtoMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *a, unsigned i, int ma=-99) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *a, unsigned i) const

- mxArray * **toMx0** (int m=-99) const
- mxArray * **mxCreateStructX** (unsigned m, unsigned n, int ma=-99) const
- void **add2MxStructX** (mxArray *a, unsigned i, int ma=-99) const
- mxArray * **toMX** () const
- mxArray * **mxCreateSTRUCT** (unsigned m, unsigned n) const
- mxArray * **add2MxSTRUCT** (mxArray *a, unsigned i) const
- void **put** (const char *vname, unsigned ma=-99, const char *ws="caller") const
- void **put0** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **put** (const char *F, int L, const char *vname, unsigned ma=-99, const char *ws="caller", char flag=0) const
- template<class T2 >
[wbvector](#)< T2 > & **toFull** ([wbvector](#)< T2 > &A) const
- template<class DB >
[wbsparray](#)< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const [wbsparray](#)< DB > &B)
- template<class DB >
[wbsparray](#)< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const [wbsparray](#)< DB > &B)
- template<class DB >
[wbsparray](#)< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const [wbsparray](#)< DB > &B)

Public Attributes

- [wbvector](#)< SPIDX_T > **SIZE**
- [wbMatrix](#)< SPIDX_T > **IDX**
- [wbvector](#)< TD > **D**
- char **isref**

Protected Member Functions

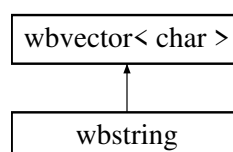
- unsigned **get2DSize** (const char *F, int L, SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const
- unsigned **get2DIndex** ([wbMatrix](#)< SPIDX_T > &IJ, SPIDX_T &d1, SPIDX_T &d2, int m=-1) const

The documentation for this class was generated from the following files:

- wblib.h
- wbsparray.hh
- wbsparray.cc

4.112 wbsting Class Reference

Inheritance diagram for wbsting:



Public Member Functions

- **wbstring** (unsigned n=0)
- **wbstring** (unsigned n, const char c)
- **wbstring** (const char *i, unsigned n)
- **wbstring** (const [wbstring](#) &s)
- **wbstring** (const char *s1)
- **wbstring** (const char *s1, const char *s2, const char *s3=NULL, const char *s4=NULL)
- **wbstring** (const mxArray *a)
- [wbstring](#) & **init** (const mxArray *a)
- **wbstring** (const char *F, int L, const mxArray *a)
- [wbstring](#) & **init** (unsigned n=0)
- [wbstring](#) & **init** (const char *s)
- [wbstring](#) & **init** (const char *F, int L, const mxArray *a)
- [wbstring](#) & **initrep** (const char *s, unsigned n)
- [wbstring](#) & **initrep** (char c, unsigned n)
- void **set** (char c)
- [wbstring](#) & **operator=** (const char *i)
- bool **gotTerm0** ()
- bool **strchr** (const char *s)
- [wbstring](#) & **push** (const char *F, int L, const char *s, char extend=1)
- [wbstring](#) & **push** (const char *s, char extend=1)
- [wbstring](#) & **pushf** (const char *F, int L, const char *fmt,...)
- [wbstring](#) & **printf** (const char *F, int L, const char *fmt,...)
- [wbstring](#) & **cpy** (const char *F, int L, const char *s)
- [wbstring](#) & **time** ()
- [wbstring](#) & **time** (const time_t &t)
- [wbstring](#) & **time_sys** ()
- [wbstring](#) & **time_sys** (const time_t &t)
- [wbstring](#) & **Upper** ()
- [wbstring](#) & **Lower** ()
- [wbstring](#) **toupper** () const
- [wbstring](#) **tolower** () const
- [wbstring](#) & **Chomp** ()
- char **getopt** (char c)
- char **getopt** ()
- const char * **basename** (const char c='/') const
- mxArray * **toMx** () const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- template<class T >
char * **init2Fmt** (const T &x __attribute__((unused)), int n=-1, int p=-1)
- template<class T >
[wbstring](#) & **operator<<** (const T *x)
- template<class T >
[wbstring](#) & **operator<<** (const T &x)
- [wbstring](#) **operator+** (const [wbstring](#) &s) const
- [wbstring](#) **operator+** (const char *s) const
- [wbstring](#) & **operator+=** (const char *s)
- [wbstring](#) & **operator+=** (const [wbstring](#) &s)
- unsigned **cat** (unsigned &k, const char *s)
- [wbstring](#) **toStr** () const
- char & **operator[]** (unsigned i) const
- unsigned **strlen_** () const
- bool **isEmpty** () const
- bool **operator!** () const

- **operator bool** () const
- unsigned **isName** (int L=-1) const
- int **operator==** (const char *s) const
- int **operator!=** (const char *s) const
- int **operator==** (const [wbstring](#) &s) const
- int **operator!=** (const [wbstring](#) &s) const
- int **operator<** (const [wbstring](#) &s) const
- int **operator<=** (const [wbstring](#) &s) const
- int **operator>** (const [wbstring](#) &s) const
- int **operator>=** (const [wbstring](#) &s) const
- int **firstLocation** (const [wbstring](#) &, unsigned k=0)
- int **firstLocation** (const char *, unsigned offset=0)
- std::istream & **getline** (std::istream &istr, char c='\n')
- template<> [wbstring](#) & **operator<<** (const char *s)
- template<> [wbstring](#) & **operator<<** (const char &x)
- template<> [wbstring](#) & **operator<<** (const int &x)
- template<> [wbstring](#) & **operator<<** (const unsigned &x)
- template<> [wbstring](#) & **operator<<** (const long &x)
- template<> [wbstring](#) & **operator<<** (const unsigned long &x)
- template<> [wbstring](#) & **operator<<** (const double &x)
- template<> [wbstring](#) & **operator<<** (const float &x)
- template<> [wbstring](#) & **operator<<** (const [wbcomplex](#) &x)

Additional Inherited Members

The documentation for this class was generated from the following files:

- [wbstring.hh](#)
- [wbstring.cc](#)

4.113 wbsys Class Reference

Public Member Functions

- [wbstring](#) **MemTot2Str** () const
- [wbstring](#) **MemFree2Str** () const
- [wbstring](#) **SwapTot2Str** () const
- [wbstring](#) **SwapFree2Str** () const
- size_t **getMemTot** () const
- size_t **getMemFree** () const
- size_t **getSwapTot** () const
- size_t **getSwapFree** () const
- char **checkSwapSpace** (const char *F=NULL, int L=0) const
- int **getNumCores** () const

The documentation for this class was generated from the following files:

- [util.hh](#)
- [util.cc](#)

4.114 wbttop Class Reference

Public Member Functions

- [wbstring](#) **VmSize2Str** () const
- `size_t` **getVmSize** () const
- [wbstring](#) **VmPeak2Str** () const
- `size_t` **getVmPeak** () const
- `int` **runningLarge** (const char *F=NULL, int L=0, double th0=0.80, double fac=1.2)
- `void` **MemStat** (const char *F=0, int L=0, char lflag=0) const

Public Attributes

- `pid_t` **pid**
- `pid_t` **ppid**
- `size_t` **sysmem**

The documentation for this class was generated from the following files:

- util.hh
- util.cc

4.115 WbUtil< T > Class Template Reference

Public Member Functions

- `T` **eps** ()
- [wbcomplex](#) **eps** ()
- `void` **adjust_tnorm** (char &tnorm __attribute__((unused)))
- `bool` **hasConj** ()
- `bool` **hasNoConj** ()
- `bool` **isFloat** ()
- `bool` **isFloat** ()
- `bool` **isFloat** ()
- `bool` **isFloat** ()
- `bool` **isInt** ()
- `bool` **isInt** ()
- `bool` **isInt** ()
- `bool` **isInt** ()
- `bool` **isInt** ()
- `bool` **isInt** ()
- `bool` **isInt** ()
- `double` **eps** ()
- `float` **eps** ()
- `long double` **eps** ()

Static Public Member Functions

- static void **adjust_tnorm** (char &tnorm)
- static bool **hasConj** ()
- static bool **hasNoConj** ()
- static constexpr bool **isPOD** ()
- static bool **isFloat** ()
- static bool **isInt** ()
- static constexpr char **isComplex** ()
- static char **isReal** ()

The documentation for this class was generated from the following file:

- util.hh

4.116 `wbvector< T >` Class Template Reference

Public Member Functions

- **wbvector** (size_t l, const T *d0=NULL, char ref_flag=0)
- **wbvector** (const [wbvector](#) &v)
- template<class T2 >
wbvector (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const T *d0)
- [wbvector](#) & **init** (size_t l, const T *d0, char ref)
- [wbvector](#) & **init2val** (size_t l, const T &x)
- [wbvector](#) & **init_stride** (const T *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#) & **init_data** (const T *d0)
- template<class T0 >
[wbvector](#) & **init_data** (const T0 *d0)
- template<class T0 >
[wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< T > & **init** (const [wbsparray](#)< T > &S)
- template<class T2 >
[wbvector](#)< T > & **initT** (const [wbsparray](#)< T2 > &S)
- template<class T2 >
[wbvector](#)< T > & **initT** (size_t l, const T2 *v)
- template<class T2 >
[wbvector](#)< T > & **initT** (const [wbvector](#)< T2 > &v)
- template<class T2 >
[wbvector](#)< T > & **initT_** (const [wbvector](#)< T2 > &v)
- template<class T2 >
[wbvector](#)< T > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const T *d0)
- [wbvector](#) & **unRef** ()

- [wbvector](#) & **init** (size_t l1, const T *d1, size_t l2, const T *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init_mpr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- template<class TB >
[wbvector](#)< T > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const T &x)
- [wbvector](#) & **setMin** (const T &x)
- [wbvector](#) & **setMax** (const T &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const T &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const T &x)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const T &x) const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const T &x) const
- size_t **find1** () const
- size_t **count** (const T &x) const
- size_t **icount** (const T &x) const
- size_t **nnz** () const
- size_t **nnz** (T eps) const
- size_t **numZeros** (T eps=T(1E-14)) const
- size_t **numEQ** (const T &) const
- size_t **numGT** (const T &) const
- size_t **numLT** (const T &) const
- [wbindex](#) & **find** (const T &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const T &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findGT** (const T &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const T &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findRange** (const T &x1, const T &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (T r) const
- int **findClosest** (T r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const T *x)
- void **set** (size_t offset, const T *x, size_t n)
- void **reset** ()
- void **cpy** (const T *v)

- `void cpy` (const T *x, size_t m)
- `wbvector` & `save2` (`wbvector` &v)
- `void swap` (`wbvector` &v)
- `wbvector` & `swap` (size_t i, size_t j)
- `wbvector` & `Collect` (T &A1, T &A2)
- `wbvector` & `Collect` (T &A1, T &A2, T &A3)
- `wbvector` & `Skip` (size_t)
- `wbvector` & `Skip` (const WBINDEX &, char rflag=0)
- `wbvector` & `Move2_FE` (const WBINDEX &I, char FE_flag)
- `wbvector` & `Move2front` (const WBINDEX &I)
- `wbvector` & `Move2end` (const WBINDEX &I)
- `wbvector` & `Move2front` (size_t k)
- `wbvector` & `Move2end` (size_t k)
- `wbvector` & `mvLastTo` (size_t k)
- `wbvector` & `mvFirstTo` (size_t k)
- `bool checkSameLength` (const `wbvector` &a, const char *F=NULL, int L=0, const char *istr="") const
- `bool checkSameLength` (const `wbvector` &a, const `wbvector` &b, const char *F=NULL, int L=0, const char *istr="") const
- `template<class T_>`
`void operator+=` (const T_ &c)
- `template<class T_>`
`void operator-=` (const T_ &c)
- `template<class T_>`
`void operator*=` (const T_ &c) const
- `template<class T_>`
`void operator/=` (const T_ &c)
- `template<class T_>`
`wbvector operator*` (const T_ &c) const
- `template<class T_>`
`wbvector operator+` (const T_ &c) const
- `template<class T_>`
`wbvector operator-` (const T_ &c) const
- `wbvector operator+` (const `wbvector` &b) const
- `wbvector operator-` (const `wbvector` &b) const
- `void operator+=` (const `wbvector` &b)
- `void operator-=` (const `wbvector` &b)
- `wbvector` & `Plus` (const T *b)
- `wbvector` & `Plus` (const T *b, T bfac)
- `wbvector` & `Plus` (const `wbvector` &b, T bfac=1)
- `void plus` (const `wbvector` &b, `wbvector` &c, T bfac=1, T cfac=0) const
- `wbvector` & `times` (T fac, `wbvector` &x) const
- `wbvector` & `RevertSigns` ()
- `int isum` () const
- `T sum` () const
- `T sum` (size_t i1, const char *estr) const
- `T sum` (size_t i1, size_t i2) const
- `T cumsum_` (`wbvector` &cs, char xflag=0) const
- `T Cumsum` (char zflag=0) const
- `template<class T2>`
`T cumsum0prod` (`wbvector` < T2 > &b, `wbvector` < T > &cs, char xflag=0) const
- `wbvector` & `Eldiv` (const `wbvector` &v) const
- `wbvector` & `tensorProd` (const `wbvector` &v2, `wbvector` &vv) const
- `T scalarProd` (const `wbvector` &v) const
- `T scalarProd` (const T *v) const
- `T norm` () const

- **T norm2** () const
- double **normDiff** (const [wbvector](#) &v, T fac=1) const
- **T normDiff2** (const [wbvector](#) &v, T fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- **T GSProject** (const T *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **add** (const T *d)
- [wbvector](#) & **subtract** (const T *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &I)
- [wbvector](#) & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const T &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual** (const T &a) const
- bool **allEqual2** (const T &a, T eps=1E-15) const
- bool **allUnequal** (const T &a) const
- bool **any** () const
- bool **anyUnequal** (const T &a) const
- bool **anyEqual** (const T &a, size_t *l=NULL) const
- bool **anyGT** (const T &a) const
- bool **anyGE** (const T &a) const
- bool **anyLT** (const T &a) const
- bool **anyLE** (const T &a) const
- bool **allIn** (const T &a1, const T &a2) const
- bool **anySmallVals** (const T eps=1E-15) const
- bool **isEqual** (const T *v) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const T &x, size_t *k=NULL) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const T &x, size_t *k=NULL) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- int **operator<** (const T &r) const
- T & **el** (size_t i) const
- T **elx** (size_t i, const T &x) const
- const T & **operator[]** (size_t i) const
- T & **operator[]** (size_t i)
- T & **end** () const
- T & **last** () const
- T & **p2last** () const
- T * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)

- `void setIdx` (double shift=0., double fac=1.)
- `int set2Group` (const `wbperm` &P, const `WBINDEX` &D, const `T *S0=NULL`, char iflag=0)
- `wbvector` & `ResizeX` (size_t n)
- `wbvector` & `Resize` (size_t n, const `T *d=NULL`)
- `wbvector` & `resize` (size_t n, `wbvector` &X) const
- `wbvector` & `cat` (const `wbvector` &v2)
- `wbvector` & `Cat` (const `wbvector` &v1, const `wbvector` &v2)
- `wbvector` & `Cat` (const `wbvector< wbvector >` &vv)
- `wbvector` & `Cat` (const `wbvector< wbvector * >` &vv)
- `template<class T1 , class T2 , class T3 , class T4 >`
`wbvector` & `Cat` (const `T1 *v1`, size_t l1, const `T2 *v2`, size_t l2, const `T3 *v3=0`, size_t l3=0, const `T4 *v4=0`, size_t l4=0)
- `template<class T1 , class T2 , class T3 , class T4 >`
`wbvector` & `Cat` (const `wbvector< T1 >` *v1, const `wbvector< T2 >` *v2, const `wbvector< T3 >` *v3=0, const `wbvector< T3 >` *v4=0)
- `wbvector` & `Append` (const `wbvector` &v2)
- `wbvector` & `Append` (const `T` &x)
- `wbvector` & `Append` (size_t d2, const `T *x`)
- `wbvector` `append` (const `wbvector` &v) const
- `wbvector` & `append` (const `wbvector` &v2, `wbvector` &vout) const
- `void append` (const `T *x`, size_t d2, `wbvector` &vout) const
- `int recLess` (const `T *v`) const
- `int recLessE` (const `T *v`) const
- `int recCompare` (const `T *v`) const
- `T max` () const
- `T max` (size_t *k) const
- `T max` (unsigned &k) const
- `T min` () const
- `T min` (size_t *k) const
- `T min` (unsigned &k) const
- `T aMax` (size_t *k=NULL) const
- `T aMax` (unsigned &k) const
- `T aMin` (char zflag=0, size_t *k=NULL) const
- `T maxneg` (size_t *k=NULL) const
- `T minpos` (size_t *k=NULL) const
- `T avg` () const
- `T std` () const
- `void range` (`T &xmin`, `T &xmax`) const
- `T range` () const
- `wbvector` & `Permute` (const `wbperm` &P, char iflag=0)
- `wbvector` & `permute` (`wbvector` &, const `wbperm` &P, char iflag=0) const
- `wbvector` `permute` (const `wbperm` &P, char iflag=0) const
- `wbvector` & `blockPermute` (const `wbperm` &P, `wbvector` &, char iflag=0) const
- `wbvector` & `BlockPermute` (const `wbperm` &P, char iflag=0)
- `int blockCompare` (const `wbperm` &pa, const `wbvector` &b, const `wbperm` &pb) const
- `void get` (const `WBINDEX` &l0, `wbvector` &v) const
- `void get` (const `wbvector< size_t >` &l, `T *v`) const
- `void get` (size_t i1, size_t i2, `wbvector` &v) const
- `void selectSU` (const `WBINDEX` &l)
- `void select` (const `WBINDEX` &l, `T *r`) const
- `wbvector` & `select` (const `WBINDEX` &l, `wbvector` &a) const
- `wbvector` & `Select` (const `WBINDEX` &l)
- `wbvector` & `BlockSelect` (const `WBINDEX` &l, const `WBINDEX` &D)
- `wbvector` `select` (const `WBINDEX` &l) const
- `wbvector` & `getl` (size_t k, `wbvector` &v) const

- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- T **prod** () const
- T **prod** (const T &p0) const
- T **prod** (size_t i1, size_t i2) const
- template<class T1 >
T **prod** (const T1 *l, size_t n) const
- template<class T1 , class T2 >
T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (T &p1, T &p2, const T &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMx_StructP** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const T &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n=-1, const char *sep=" ") const
- [wbstring](#) **toStrD** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", const char stride=0, const char *sep2="") const
- bool **isNormal** () const
- bool **isNormal** () const
- template<class T1 , class T2 , class T3 , class T4 >
[wbvector](#)< T > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- template<class T1 , class T2 , class T3 , class T4 >
[wbvector](#)< T > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- template<class TD >
[wbvector](#)< T > & **initT** (const [wbsparray](#)< TD > &S)
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStrf** (const char *fmt0, const char *sep, const char stride, const char *sep2) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const

- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &I)
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const

Public Attributes

- T * **data**
- size_t **len**
- char **isref**

Protected Member Functions

- void **NEW** (size_t n, const T *d=NULL)
- [wbvector](#) & **RENEW** (size_t n, const T *d=NULL)
- [wbvector](#) & **RENEW_VAL** (size_t n, const T &x)

The documentation for this class was generated from the following files:

- wblib.h
- wbvector.hh
- wbvector.cc

4.117 clebsch::weight Class Reference

Public Member Functions

- **weight** (int N)
- **weight** (int N, int index)
- [clebsch::weight](#) & **operator=** (const [clebsch::weight](#) &w)
- int & **operator()** (int k)
- const int & **operator()** (int k) const
- bool **operator<** (const [weight](#) &w) const
- bool **operator==** (const [weight](#) &w) const
- [clebsch::weight](#) **operator+** (const [weight](#) &w) const
- int **index** () const
- long long **dimension** () const

Public Attributes

- const int **N**

The documentation for this class was generated from the following file:

- ClebschGordan_Arne.cpp

4.118 DY::weight_info< TQ > Class Template Reference

Public Member Functions

- **weight_info** (const [weight_info](#) &w)
- **weight_info** (unsigned m_, unsigned n)
- [weight_info](#) & **init** (unsigned m_, unsigned n)
- [weight_info](#) & **init** (const [weight_info](#) &w)
- [weight_info](#) & **operator=** (const [weight_info](#) &w)
- [weight_info](#) & **save2** ([weight_info](#) &w)

Public Attributes

- int **m**
- [qset](#)< TQ > **p**

The documentation for this class was generated from the following file:

- clebsch.hh

4.119 DY::Weights< TQ > Class Template Reference

Public Member Functions

- **Weights** (const [qset](#)< TQ > &q)
- [Weights](#) & **init** (const [qset](#)< TQ > &q)
- size_t **dim** () const
- void **print** (const char *F, int L, const [QType](#) &q, char vflag=0) const
- mxArray * **toMx** () const

Public Attributes

- unsigned **n**
- [qset](#)< TQ > **qm**
- map< [qset](#)< TQ >, [weight_info](#)< TQ > > **W**
- map< [qset](#)< TQ >, const [weight_info](#)< TQ > * > **X**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.120 X3Map< TQ, TD > Class Template Reference

Public Member Functions

- void **clear** ()
- int **init** (const char *F, int L, const mxArray *S, unsigned k=0)
- [x3map](#)< TQ, TD > & **getXBUF** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc)
- int **contains** (const char *F, int L, const [x3map](#)< TQ, TD > &x, const [cgc_contract_id](#)< MTI > &idc)
- int **contract** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [CRef](#)< TQ > &C, char tflag=0)
- [CRef](#)< TQ > & **contractDegQ** (const char *F, int L, const [CRef](#)< TQ > &A, [CRef](#)< TQ > &B)
- unsigned **add** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc, const mxArray *S, unsigned k)
- mxArray * **toMx** () const
- void **Info** (const char *F=0, int L=0) const

Public Attributes

- unordered_map< [cgc_contract_id](#)< MTI >, [x3map](#)< TQ, TD >, [MHash](#)< MTI > > **XBUF**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.121 x3map< TQ, TD > Class Template Reference

Public Member Functions

- **x3map** (const char *F, int L, const [CRef](#)< TQ > &A_, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B_, const [ctrlIdx](#) &icb, [wbperm](#) &cgp, char xflag=0)
- [x3map](#) & **init** ()
- int **initCtr** (const char *F, int L, const [CRef](#)< TQ > &A_, const [ctrlIdx](#) &ica_, const [CRef](#)< TQ > &B_, const [ctrlIdx](#) &icb_, [wbperm](#) &cgp, char xflag=1)
- [x3map](#) & **init** (const char *F, int L, const mxArray *S, unsigned k=0, [ctrlIdx](#) *ica=NULL, [ctrlIdx](#) *icb=NULL, char recalc=1)
- bool **isEmpty** () const
- void **print** (const char *F=NULL, int L=0, const char *istr="", char vflag=0) const
- [wbstring](#) **toStr** (char vflag=0) const
- [wbstring](#) **x3Str** (char vflag=0) const
- size_t **memSize** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned l, const [cgc_contract_id](#)< MTI > *idc=NULL) const
- mxArray * **toMx** (const [cgc_contract_id](#)< MTI > *idc=NULL) const

Public Attributes

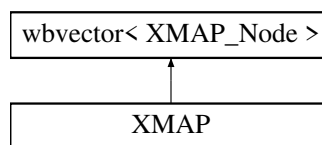
- [CData](#)< TQ, RTD > **a**
- [CData](#)< TQ, RTD > **b**
- [CData](#)< TQ, RTD > **c**
- const CDATA_TQ * **cgb**
- [wbperm](#) **pab**
- char **conj**
- int **zflag**
- [cgrType](#) **rtype**
- [wbararray](#)< TD > **x3**
- [wbararray](#)< TD > & **X3**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.122 XMAP Class Reference

Inheritance diagram for XMAP:



Public Member Functions

- **XMAP** (unsigned n)
- **free** ()
- void **init** (unsigned n)
- template<class T >
void **set** (unsigned k, const wbArray< T > &A, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z)
- void **set** (unsigned k, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z)

Public Attributes

- char **isref**

Additional Inherited Members

The documentation for this class was generated from the following file:

- xmap.hh

4.123 XMAP_CMat Class Reference

Public Attributes

- `wbCMat * M`
- unsigned `ic`
- unsigned `i1`
- unsigned `i2`
- `wbcomplex z`

The documentation for this class was generated from the following file:

- `bicgMatC.cc`

4.124 XMAP_Node Class Reference

Public Member Functions

- `free ()`
- void `init` (const `mxArray *a`)
- `wbvector`< unsigned > `size` () const
- unsigned `rank` () const
- bool `hasSameSize` (const `XMAP_Node` &B) const
- template<class T >
void `set` (const `wbArray`< T > &A, unsigned `ic`, unsigned `i1`, unsigned `i2`, `wbcomplex z`, char `ref=1`)
- void `set` (unsigned `ic`, unsigned `i1`, unsigned `i2`, `wbcomplex z`, char `ref=1`)
- template<class T >
void `set` (unsigned `k`, const `wbArray`< T > &A, unsigned `ic_`, unsigned `i1_`, unsigned `i2_`, `wbcomplex z_`, char `ref`)

Public Attributes

- `wbArray`< `wbcomplex` > * `Z`
- `wbArray`< double > * `R`
- char `tflag`
- `wbperm P`
- unsigned `ic`
- unsigned `i1`
- unsigned `i2`
- unsigned `isref`
- `wbcomplex z`

The documentation for this class was generated from the following file:

- `xmap.hh`

Chapter 5

File Documentation

5.1 clebsch.hh

```
1 #ifndef __WB_CLEBSCH_GORDAN_HH__
2 #define __WB_CLEBSCH_GORDAN_HH__
3
4 //=====//
5 // CStore => global CG container: CStore<gTQ> gCS; // CG_GLOBAL
6 //
7 //   global container for Clebsch Gordan coefficients
8 //   - based on CData for single (Q1,Q2->Q) mapping.
9 //   - based on QVec, qset, cdata below.
10 //
11 // cdata : wbararray<double>    // full data arrays!
12 // cdata : wbSparray // switching to sparse // Wb,Dec22,11
13 //
14 //   actual storage of ClebschGordan coefficients
15 //   - of rank 3(+1) for standard CG (3+1=4 in case of multiplicity)
16 //   - cdata also includes (contracted) derivatives of ANY rank
17 //
18 // QVec : wbvector<QType>
19 //
20 //   single set of types of quantum numbers with QType
21 //   - the most basic symmetry unit
22 //
23 // qset : wbvector<TQ>
24 //
25 //   actual set of quantum numbers interpreted through QVec
26 //
27 // Wb,Sep20,09 ; Wb,May01,10
28 //
29 // ----- //
30 // NB! sign convention on CGCs // tag: DEF_CGD_NORM
31 //
32 // using CRef, ensure that CData is simply normalized to 1
33 // internally within gCS. This ensures that for each OM set
34 // * overlaps can simply be computed based on cgw data!
35 // * OM within gCS can be also determined through cdata.norm2().
36 // return value: number of trailing dimensions to be interpreted
37 // as OM space.
38 //
39 // Furthermore, keep |cgw|^2=1 as far as possible,
40 // so A.norm2() of some QSpace A is simply carried by |A.data|^2.
41 // @QSpace/display.m also assumes this convention throughout.
42 //
43 // => data already carries correct 3j weight factors
44 //   for simple contraction [e.g. norm2()]
45 // => identity A tensors have entries sqrt(d_out)
46 //   which already indicates multiplet weight
47 //
48 // Other (not so useful) attempts to normalize cgw
49 // (with corresponding inverse factor multiplied onto data{:}):
50 //
51 // * |cgw| := sqrt(max(size(cdata))) // see also EXT_CGD_NORM
52 //
53 //   (+) rank-2 tensor are exact identities (or unaries for 1J symbols)
54 //   (+) rank-3 tensors are mostly normalized to max(|CGC|)
55 //       e.g. (SS|0) is normalized the same way as (S0|S)
56 //       whereas their standard normalization [see NormStd()]
57 //       would be sqrt(d_out), here 1 or (2S+1), respectively!
58 //
```

```

59 //      (-) standard rank-3 identity A tensors do not have cgd = Id
60 //      (-) does not work specifially well for rank>3 tensors.
61 //      e.g. consider
62 //          contract( (1,1|2), 3, (1,1|2), 3 ) (i)
63 //          vs. contract( (1,1|0), 3, (1,1|0), 3 ) (ii)
64 //          while the output CData space is the same (1,1,1,1)
65 //          in either case, already the normalization of the
66 //          CData requires *different* normalization factors 1/sqrt(d3)
67 //          depending on the *contracted* 3rd index containing a multiplet
68 //          of dimension d3. This information is no longer available
69 //          once the CGCs have been contracted!
70 //
71 // Wb,Oct16,14 ; Wb,Nov04,14
72 //
73 //=====//
74 // RStore => global constructor class for non-abelian Lie groups
75 //
76 // RStore <TQ,TD> gRS; // global
77 // * map[QType] :=> genRG_struct buf[]
78 //
79 //     genRG_struct<TQ,TD>
80 //     * map[qset] :=> genRG_base<TQ,RTD> RSet[]
81 //
82 //     genRG_base<TQ,RTD>
83 //     * basic symmetry settings such as
84 //     generators, commutator relations (CR), ...
85 //
86 // Wb,Jun07,10
87 //
88 // ===== //
89 /* CHANGE LOG
90
91 // Wb,Aug28,20 // PERMIT_RANK1_Q0 ----- //
92
93     Permit rank-1 QSpace if scalar, i.e. for q=0 only;
94     e.g. for SU(2), can contract (S,0|S) at '13'
95     for any spin-S multiplet, resulting in a non-zero weight
96     since by DEF_CGD_NORM with d=2S+1 the multiplet dimension,
97     trace('S,0;S,'13') = 1/sqrt(d) * d = sqrt(d)
98     [e.g. see @QSpace/getIdentity3.m with --rho flag]
99     however, contract('S,S;S,'13') = 0, i.e., rank-1 QSpace is
100     not permitted for non-zero (non-scalar) symmetry label q!=0.
101
102     NB! a rank-1 QSpace may be generated, e.g., by contraction
103     to obtain expectation values [HAM/calc_SdotS.m] after using
104     @QSpace/getIdentity3.m
105
106     NB! contract('0,S,S*,'13','S,S*','12')
107         = sqrt(d) * trace('S,0;S,'13')
108         = 1.
109     since 2nd line acquires additional sqrt(d) factor from 'S,S*'
110     => X-symbol has x3=1 - ok!
111
112     => permit rank-1 QSpace as output, but not necessarily as input
113     yet with exceptions, such as getDimQS so that at least QSpace
114     can be displayed using @QSpace/display.m
115
116 // Wb,June 2014 ----- //
117
118     * removed CG_SU2 (merged with treatment of QTYPE_SUN)
119     * removed zflag from qset [e.g. nq = n(q) + n(qz)]
120     * removed zflags from QType and qset
121     * removed qset::RemoveZLabels (still got QSpace::RemoveZLabels!)
122     * removed following variables from CData as these are redundant
123       with data in RStore => genRG_struct => genRG_base:
124       - qset<TQ> Q1,Q2,Q; // (Q1,Q2) -> Q
125       - wbMatrix<TQ> Z1,Z2,Z; // (Z1,Z2) -> Z
126       use gRS container to retrieve these where required
127     * renamed CData => cdata; CGSpace => CData
128
129     => see Archive/clebsch_140627.[ch]* for version including zflag
130
131 //=====*/
132
133 // #define RTD long double // "long double" appears unreliable!
134
135 #ifndef RTD
136     #ifdef __WB_MPFR_HH__
137
138         #define MTI unsigned long
139         #define RTD Wb::quad
140
141     #else
142
143         #define MTI unsigned long
144         #define RTD double
145

```

```

146 #endif
147 #endif
148
149 #ifndef gTQ
150
151     #define gTQ int
152 #endif
153
154 #ifndef gTD
155 #define gTD double
156 #endif
157
158 #define CData_TQ      CData<TQ,RTD>
159 #define cdata__      cdata<RTD>
160 #define cgsparray    wbsparray<RTD>
161
162     template <class TQ, class TD> class CData;
163     template <class TQ> class CRef;
164
165     class QVec;
166
167     int MAX_OM=9;
168
169     int get_CG_VERBOSE(const char *F=0, int L=0);
170     int CG_VERBOSE=get_CG_VERBOSE();
171
172     unsigned get_CG_FIXIT(const char *F=0, int L=0);
173     unsigned CG_FIXIT=get_CG_FIXIT();
174
175     #define FIX_CID3  1
176     #define FIX_CTR   2
177
178     unsigned get_CG_PREVIEW(const char *F=0, int L=0);
179     unsigned CG_PREVIEW=get_CG_PREVIEW();
180
181     int QSP_NUM_THREADS=0;
182
183     #define RC_LOAD_ERR_AB_MSG "RC_LOAD_ERROR (A or B)"
184
185     wstring RC_LOG;
186
187     #define CG_SKIP_DEPS1 1E-12
188     #define CG_SKIP_DEPS2 1E-14
189
190     #define CG_SKIP_REPS 1E-17
191
192 #ifdef __WB_MPFR_HH__
193     #define CG_SKIP_EPS1 1E-16
194     #define CG_SKIP_EPS2 1E-18
195
196     #define CG_EPS 1E-38
197     #define CG_EPS0 1E-36
198     #define CG_EPS1 1E-16
199     #define CG_EPS2 1E-18
200
201 #else
202
203     #define CG_EPS 2.22E-16
204     #define CG_EPS0 1E-15
205
206     #define CG_SKIP_EPS1 1E-14
207     #define CG_SKIP_EPS2 1E-16
208
209     #define CG_EPS1 1E-10
210     #define CG_EPS2 1E-12
211
212 #endif
213
214     double cg_eps1=CG_EPS1;
215     double cg_eps2=CG_EPS2;
216
217     enum QTYPE_SET {
218         QTYPE_UNKNOWN,
219         QTYPE_P,
220         QTYPE_ZN,
221         QTYPE_U1,
222         QTYPE_ASEP,
223         QTYPE_SUN,
224         QTYPE_SpN,
225         QTYPE_SON,
226         QTYPE_SEN,
227         QTYPE_NUM_TYPES
228     };
229
230     const char* QTYPE_STR[QTYPE_NUM_TYPES]= {
231         "???",
232         "P",

```

```

233     "ZN",
234     "A",
235     "!!?",
236     "SUN",
237     "Sp2N",
238     "SO2N",
239     "SO2N_1"
240 };
241
242 #define CGC_ALL_ABELIAN "A*"
243
244 enum CGD_TYPE { CGD_DEFAULT,
245     CGD_ABELIAN,
246     CGD_IDENTITY,
247     CGD_REF_INIT,
248     CGD_BSZ_INIT,
249     CGD_FROM_CGR,
250     CGD_FROM_CTR,
251     CGD_FROM_DEC,
252     CGD_STD3,
253     CGD_1JSY_ST3,
254     CGD_1JSY_GEN,
255     CGD_STD3_X,
256     CGD_GEN_X,
257     CGD_EXPLICIT,
258 CGD_NUM_TYPES };
259
260 #define CID_RANK1_Q0 60
261
262 const char* CGD_TYPE_STR[CGD_NUM_TYPES]= { "",
263     "d:iA",
264     "d:iE",
265     "d:iR",
266     "d:iS",
267     "d:cR",
268     "d:cC",
269     "d:cD",
270     "d:c3",
271     "d:rlJ",
272     "d:clJ",
273     "d:c3x",
274     "d:cgx",
275     "d:iX",
276 };
277
278 enum CGR_TYPE { CGR_DEFAULT,
279     CGR_ABELIAN,
280     CGR_CTR_SCALAR,
281     CGR_CTR_ZERO,
282 CGR_NUM_TYPES };
283
284 const char* CGR_TYPE_STR[CGR_NUM_TYPES]= { "",
285     "r:iA",
286     "r:cS",
287     "r:c0",
288 };
289
290 namespace Mx {
291
292 template <>
293 size_t Mx::Array<CGD_TYPE>::copy_to(CGD_TYPE* b, const wbperm *P, char tcheck)
294 const { return rcopy_to(b,P,tcheck); };
295
296 template <>
297 size_t Mx::Array<CGR_TYPE>::copy_to(CGR_TYPE* b, const wbperm *P, char tcheck)
298 const { return rcopy_to(b,P,tcheck); };
299
300 };
301
302 #ifdef WB_SPARSE_CLOCK
303 Wb::Clock wbc_sparse_gss("cgs::getSymStates");
304 #endif
305
306 template <class T> class qset;
307 template <class T> class CStore;
308
309 template <class T> class QMap;
310 template <class TQ, class TD> class blockSpaceQS;
311
312 #include <map>
313
314 namespace CG {
315
316     template <class TD> inline
317     int signFirstVal(const char *F, int L,
318         const TD *d, SPIDX_T n,
319         double eps1=CG_EPS1, double eps2=CG_EPS2

```

```

320 );
321
322 template <class TD> inline
323 int rangeSignConvention(const char *F, int L,
324     TD *d, SPIDX_T n,
325     double eps1=CG_EPS1, double eps2=CG_EPS2
326 );
327
328 template<class T>
329 double FixRational(const char *F, int L,
330     T* d, SPIDX_T n, unsigned niter=0,
331     double eps1=-1, double eps2=-1
332 );
333 };
334
335 namespace DY {
336
337     template<class TQ>
338     size_t wdim_A(unsigned n, const TQ *qs);
339
340     template<class TQ>
341     size_t wdim_B(unsigned n, const TQ *qs);
342
343     template<class TQ>
344     size_t wdim_C(unsigned n, const TQ *qs);
345
346     template<class TQ>
347     size_t wdim_D(unsigned n, const TQ *qs);
348
349 };
350
351 namespace CG {
352
353     template <class TQ> inline
354     unsigned gotQAlpha(const TQ* q, unsigned n) {
355
356         if (!WbUtil<TQ>().isInt()) return 0;
357
358         if (int(n)<0) wblog(FL,"ERR %s() got n=%d !?",FCT,n);
359
360         TQ qmax=61;
361         #if __APPLE__
362             qmax=35;
363         #endif
364
365         for (unsigned i=0; i<n; ++i) { if (q[i]<0 || q[i]>qmax) return 0; }
366
367         return 1;
368     };
369
370     template <> inline
371     unsigned gotQAlpha(const char* s, unsigned n) {
372
373         unsigned i=0;
374
375         #if __APPLE__
376             for (; i<n; ++i) { if (!isalnum(s[i]) || s[i]>90) break; }
377         #else
378             for (; i<n; ++i) { if (!isalnum(s[i])) break; }
379         #endif
380
381         if (int(n)<0) { return i; }
382         else { return (i<n ? (i+1) : 0); }
383
384         return 1;
385     };
386
387     template <class TQ> inline
388     unsigned cstr2qset(const char *s, TQ *q, unsigned n=-1) {
389
390         unsigned i=0;
391         for (; i<n; ++i) {
392             if (s[i]>='0' && s[i]<='9') { q[i]=s[i]-'0'; } else
393             if (s[i]>='A' && s[i]<='Z') { q[i]=s[i]-'A'+10; } else
394             #if !(defined __APPLE__)
395             if (s[i]>='a' && s[i]<='z') { q[i]=s[i]-'a'+36; } else
396             #endif
397                 break;
398         }
399
400         if (int(n)<0) { return i; }
401         else { return (i<n ? (i+1) : 0); }
402     };
403
404     template <class TQ> inline
405     int qset2cstr(const TQ *q, char *s, unsigned n) {
406

```

```

407     for (unsigned i=0; i<n; ++i) {
408         if (q[i]<0) { return (i+1); }
409         if (q[i]<10) { s[i]='0'+q[i]; } else
410         if (q[i]<36) { s[i]='A'-10+q[i]; } else
411         #if !(defined __APPLE__)
412             if (q[i]<62) { s[i]='a'-36+q[i]; } else
413         #endif
414             { return (i+1); }
415     }
416     return 0;
417 };
418
419 };
420
421 class QType {
422 public:
423     QType(QTYPE_SET t=QTYPE_UNKNOWN, unsigned s=0)
424         : type(t), sub(s) { if (t || s) validType(FL); };
425
426     QType(const char *s) : type(QTYPE_UNKNOWN), sub(0) { init(0,0,s); };
427     QType(const QType &q) : type(q.type), sub(q.sub) { };
428
429     QType(const char *F, int L, const mxArray *a)
430         : type(QTYPE_UNKNOWN), sub(0) { init(F,L,a); };
431
432     ~QType() { init(); };
433
434     QType& init() { type=QTYPE_UNKNOWN; sub=0; return *this; };
435
436     QType& initx(QTYPE_SET t, unsigned m=0) {
437         if (!t && !m) { init(); return *this; }
438         type=t; sub=m; validType(FL);
439         return *this;
440     };
441
442     QType& init(QTYPE_SET t, unsigned m=0) {
443         if (!t && !m) { init(); return *this; }
444         type=t;
445         if (type==QTYPE_SpN) { sub=m/2;
446             if (m%2) wblog(FL,"ERR %s() invalid sub=%d with SpN",FCT,m);
447         }
448         else { sub=m; }
449         validType(FL); return *this;
450     };
451
452     int init_s(const char *s);
453
454     QType& init(const char *s) { return init(FL,s); }
455
456     QType& init(const char *F, int L, const char *s) {
457         if (!s || !s[0]) { type=QTYPE_UNKNOWN; sub=0; }
458         else {
459             if (init_s(s)) wblog(FL,"ERR invalid QType '%s'",s);
460             else validType(F_L);
461         }
462         return *this;
463     };
464
465     QType& init(const char *F, int L, const mxArray *a) {
466         wbsttring s(F_L,a);
467         return init(F_L,s.data);
468     };
469
470     QType& init(const QType &q) {
471         type=q.type; sub=q.sub; return *this;
472     };
473
474     bool isUnknown() const {
475         return (type==QTYPE_UNKNOWN && sub==0);
476     };
477
478     bool isKnown() const {
479         validType();
480         return (type!=QTYPE_UNKNOWN);
481     };
482
483     bool validType(const char *F=0, int L=0) const;
484
485     bool isAbelian(const char *F=NULL, int L=0) const {
486         if (type<=QTYPE_UNKNOWN || type>=QTYPE_NUM_TYPES) wblog(F_L,
487             "ERR invalid type %d, '%s'",type,STR_(this));
488         return ((type && type<QTYPE_ASEP)? 1 : 0);
489     };
490
491
492
493

```



```

494     };
495
496     bool isNonAbelian(const char *F=NULL, int L=0, char lflag=0) const {
497         if (!lflag && !isUnknown()) {
498             if (type<=QTYPE_UNKNOWN || type>=QTYPE_NUM_TYPES) wblog(FL,
499                 "ERR invalid type %d, '%s'",type,STR_(this)); }
500         return ((type && type>QTYPE_ASEP)? 1 : 0);
501     };
502
503     bool isSU2() const { return (type==QTYPE_SUN && sub==1); };
504     bool isSUN() const { return (type==QTYPE_SUN); };
505     bool isSU(unsigned N) const { return (type==QTYPE_SUN && sub+1==N); };
506
507     bool isSpN() const { return (type==QTYPE_SpN); };
508     bool isSON() const { return (type==QTYPE_SON); };
509     bool isSEN() const { return (type==QTYPE_SEN); };
510
511     bool isU1() const { return (type==QTYPE_U1 && sub==0); };
512     bool isParity() const { return (type==QTYPE_P && sub==0); };
513
514     bool isAdditive() const {
515         if (type==QTYPE_U1) {
516             if (sub) wblog(FL,
517                 "WRN invalid type %s (%d,%d)",STR_(this),type,sub);
518             return 1;
519         }
520         return 0;
521     };
522
523     int permitsOM() const {
524         return (isAbelian() ? 0 : (sub<=1 ? 1 : 2));
525     };
526
527     int permitsOM(unsigned r) const {
528         return (isAbelian() || r<3 || (sub<2 && r<4) ? 0 : 1);
529     };
530
531     unsigned qlen() const {
532         if (type && type<QTYPE_ASEP) { return 1; }
533         if (type>=QTYPE_NUM_TYPES) wblog(FL,
534             "ERR invalid type '%s' (%d)",STR_(this),type);
535         return sub;
536     };
537
538     unsigned qrank() const {
539         if (type && type<QTYPE_ASEP) { return 0; }
540         if (type>=QTYPE_NUM_TYPES) wblog(FL,
541             "ERR invalid type '%s' (%d)",STR_(this),type);
542         return sub;
543     };
544
545     bool isLargeD(unsigned d) const {
546         if (type && type<QTYPE_ASEP) { return (d>1); }
547         switch (type) {
548             case QTYPE_SUN: return (d>12000 || d>(sub!=1 ? exp10(qlen()) : 20));
549             case QTYPE_SON:
550             case QTYPE_SEN:
551             case QTYPE_SpN: return (d>12000 || d>exp10(qlen()));
552
553             case QTYPE_UNKNOWN : return (d>100);
554             default:
555                 wblog(FL,"ERR invalid type '%s' (%d)",STR_(this),type);
556         }
557         return 1;
558     };
559
560     bool isLargeD(const wbvector<unsigned> &dd) const {
561         for (unsigned i=0; i<dd.len; ++i) {
562             if (isLargeD(dd[i])) return 1;
563         }
564         return 0;
565     };
566
567     unsigned getDimDef() const {
568         unsigned d=1;
569         if (type && type>QTYPE_ASEP) {
570             switch (type) {
571                 case QTYPE_SUN: d=sub+1; return d;
572                 case QTYPE_SpN: d=2*sub; return d;
573                 case QTYPE_SON: d=2*sub+1; return d;
574                 case QTYPE_SEN: d=2*sub; return d;
575                 default:
576                     wblog(FL,"ERR invalid type '%s' (%d)",STR_(this),type);
577             }
578         }
579         return d;
580     };

```

```

581
582 template<class TQ>
583 size_t wdim(const TQ *qs) const {
584     unsigned d=1;
585     if (type && type>QTYPE_ASEP) {
586         switch (type) {
587             case QTYPE_SUN: d=DY::wdim_A(sub,qs); break;
588             case QTYPE_SpN: d=DY::wdim_C(sub,qs); break;
589             case QTYPE_SON: d=DY::wdim_B(sub,qs); break;
590             case QTYPE_SEN: d=DY::wdim_D(sub,qs); break;
591             default:
592                 wblog(FL,"ERR invalid type '%s' (%d)",STR_(this),type);
593         }
594     }
595     return d;
596 };
597
598 template<class TQ>
599 size_t wdim(const qset<TQ> &qs) const { return wdim(qs.data); }
600
601 unsigned maxDimLocal() const {
602     unsigned d=1;
603     if (type) { d=maxDimLocal0(); if (d<20) d=20; }
604     return d;
605 };
606
607 unsigned maxDimLocal0() const {
608     unsigned d=2;
609     if (type>QTYPE_ASEP) {
610
611         d=pow(double(getDimDef()),2.25+double(sub)/4.);
612     }
613     return d;
614 };
615
616 void printDimInfo(const char *F=NULL, int L=0) const {
617     wblog(FL_L," * %s: sub=%d, dim_def=%d, dloc<=%d",
618         STR_(this), sub, getDimDef(), maxDimLocal());
619 };
620
621 template <class T>
622 unsigned qdim(const T* q) const;
623
624 template <class T>
625 wbvector<unsigned>& QDim(
626     const T* q, unsigned r, unsigned stride,
627     wbvector<unsigned> &S) const;
628
629 bool isSelfDual() const {
630     switch (type) {
631         case QTYPE_P :
632         case QTYPE_SpN :
633         case QTYPE_SON : return 1;
634
635         case QTYPE_SEN : return (qlen()%2 ? 0 : 1);
636
637         case QTYPE_U1 :
638         case QTYPE_ZN :
639         case QTYPE_SUN : return 0;
640
641         default:
642             wblog(FL,"ERR dual not yet defined for '%s'",STR_(this));
643     }
644     return 0;
645 };
646
647 template<class TQ>
648 char setDual(TQ* q) const {
649     switch (type) {
650         case QTYPE_P : { return 0; }
651         case QTYPE_U1 : { q[0]=-q[0]; return 1; }
652         case QTYPE_ZN : { q[0]=(sub-q[0])%sub; return 1; }
653
654         case QTYPE_SUN: {
655             unsigned n=qlen(); if (n<2) return 0;
656             else {
657                 unsigned i=0, l=n-1; TQ x;
658                 for (n/=2; i<n; ++i) { x=q[i]; q[i]=q[l-i]; q[l-i]=x; }
659                 return 1;
660             }
661         }
662         case QTYPE_SpN:
663         case QTYPE_SON: return 0;
664
665         case QTYPE_SEN: {
666             unsigned n=qlen();
667             if (n<3) wblog(FL,"ERR %s() got %s !?",FCT,STR_(this));

```

```

668         if ((n%2)==0) { return 0; }
669     else {
670         unsigned l=n-2; n-=1;
671         if (q[l]==q[n]) { return 0; }
672     else {
673         TQ x=q[l]; q[l]=q[n]; q[n]=x;
674         return 1;
675     }
676     }
677 }
678
679 default:
680     wblog(FL,"ERR dual not yet defined for '%s'",STR_(this));
681 }
682 return 0;
683 };
684
685 template<class TQ>
686 unsigned getDual(const TQ* q0, TQ* q) const {
687     unsigned n=qlen(); memcpy(q,q0,n*sizeof(TQ));
688     setDual(q); return n;
689 };
690
691 template<class TQ>
692 bool isDual(const TQ *q, const TQ *x) const {
693     switch (type) {
694     case QTYPE_P : { return x[0]== q[0]; }
695     case QTYPE_U1 : { return x[0]==-q[0]; }
696     case QTYPE_ZN : { return x[0]==TQ((sub-q[0])%sub); }
697
698     case QTYPE_SUN: {
699         unsigned i=0, l=qlen();
700         if (!l) wblog(FL,"ERR %s() got n=%d !?",FCT,l);
701         for (--l; i<=l; ++i) { if (x[i]!=q[l-i]) return 0; }
702         return 1;
703     }
704     case QTYPE_SpN:
705     case QTYPE_SON: {
706         if (x!=q) {
707             unsigned i=0, n=qlen();
708             for (; i<n; ++i) { if (x[i]!=q[i]) return 0; }
709             }; return 1;
710     }
711     case QTYPE_SEN: {
712         unsigned n=qlen();
713         if (n<3) wblog(FL,"ERR %s() got %s !?",FCT,STR_(this));
714
715         if (x==q) {
716             return (n%2 ? x[n-2]==x[n-1] : 1);
717         }
718         else {
719             unsigned i=0, l=n-(n%2 ? 2 : 0);
720             for (; i<l; ++i) { if (x[i]!=q[i]) { return 0; } }
721             if (i<n) {
722                 if (x[i]!=q[i+1] || x[i+1]!=q[i]) { return 0; }
723             }; return 1;
724         }
725     }
726
727     default:
728         wblog(FL,"ERR dual not yet defined for '%s'",STR_(this));
729 }
730 return 0;
731 };
732
733 QType& operator=(const char *s) { return init(FL,s); };
734 QType& operator=(const QType &q) { return init(q); };
735
736 QType& operator=(QTYPE_SET t) {
737     type=t; sub=0; if (t) validType(FL);
738     return *this;
739 };
740
741 bool operator<(const QType &q) const {
742     if (type!=q.type)
743         return (type<q.type);
744     else return (sub<q.sub);
745 };
746
747 char cmp(const QType &q) const {
748     if (type<q.type) return -1;
749     if (type>q.type) return +1;
750     if (sub <q.sub ) return -1;
751     if (sub >q.sub ) return +1; else return 0;
752 };
753
754 bool operator> (const QType &q) const { return !((*this)<=q); };

```

```

755 bool operator<=(const QType &q) const {
756     if (type!=q.type) return (type<q.type);
757     else return (sub<=q.sub);
758 };
759
760 bool operator==(const QType &q) const {
761     return (type==q.type && sub==q.sub);
762 };
763 bool operator!=(const QType &q) const {
764     return (type!=q.type || sub!=q.sub);
765 };
766
767 wstring toStr(const char tflag=0) const {
768     wstring s(8);
769     char e=0; s[0]=0;
770
771     if (type==QTYPE_U1 ) { s=(tflag ? QTYPE_STR[type] : "U(1)" ); } else
772     if (type==QTYPE_P ) { s=(tflag ? QTYPE_STR[type] : "Parity"); } else
773     if (!type && !sub) { s=(tflag ? "???" : "unknown");}
774
775     if (s[0]) {
776         if (sub) wblog(FL,
777             "ERR invalid QType %s with sub=%d",s.data,sub);
778         return s;
779     }
780
781     if (sub<1 || sub>99) { e|=1; }
782     else if (type==QTYPE_ZN) { if (sub<2) e|=2;
783         if (tflag)
784             { snprintf(s.data,s.len,"Z%d", sub); }
785         else { snprintf(s.data,s.len,"Z(%d)",sub); }
786     }
787     else if (type==QTYPE_SUN) {
788         if (tflag)
789             { snprintf(s.data,s.len,"SU%d", sub+1); }
790         else { snprintf(s.data,s.len,"SU(%d)",sub+1); }
791     }
792     else if (type==QTYPE_SpN) { if (sub<2) e|=4;
793         if (!tflag)
794             { snprintf(s.data,s.len,"Sp(%d)",2*sub); }
795         else { snprintf(s.data,s.len,"Sp%d", 2*sub);
796             if (tflag=='u') { s[1]='P'; }
797         }
798     }
799     else if (type==QTYPE_SON) {
800         if (!tflag)
801             { snprintf(s.data,s.len,"SO(%d)",2*sub+1); }
802         else { snprintf(s.data,s.len,"SO%d", 2*sub+1); }
803     }
804     else if (type==QTYPE_SEN) { if (sub<2) e|=8;
805         if (!tflag)
806             { snprintf(s.data,s.len,"SO(%d)",2*sub); }
807         else { snprintf(s.data,s.len,"SO%d", 2*sub); }
808     }
809     else {
810         if (type) wblog(FL,"ERR %s() invalid type=%d",FCT,type);
811         s=(tflag ? "???" : "unknown");
812     }
813
814     if (e) wblog(FL,
815         "ERR invalid QType %s with sub=%d", type < QTYPE_NUM_TYPES ?
816         QTYPE_STR[type] : "(type out of bounds)", sub
817     );
818
819     return s;
820 };
821
822 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
823 void add2MxStruct(mxArray *S, unsigned i) const;
824
825 mxArray* toMx(const char tflag=0) const {
826     const size_t n=32; wstring s(n);
827     size_t l=snprintf(s.data,n,"%s",toStr(tflag).data);
828     if (l>=n) wblog(FL,
829         "WRN %s() QType too long (%s; %d) !?",FCT,s.data,l);
830     return s.toMx();
831 };
832
833 QTYPE_SET type;
834 unsigned sub;
835
836 private:
837
838 int atoi(const char *s, int &k, unsigned n) {
839     if (s[0]!='(') { return Wb::atoi(s,k,n); }
840     else {
841         unsigned l=strlen(++s);

```

```

842         if (l && s[l-1]!='') {
843             char x[l]; memcpy(x,s,l); x[l-1]=0;
844             return Wb::atoi(x,k,n);
845         }
846     }
847     return -6;
848 }
849 };
850
851 QType SU2("SU2");
852
853 bool operator!(const QType &q) {
854     if (q.type == QTYPE_UNKNOWN) {
855         if (q.sub) wblog(FL,
856             "ERR %s() invalid QType %s",FCT,STR(q));
857         return 1;
858     }
859     return 0;
860 };
861
862 map<QType, time_t> load_cstore;
863
864 QType load_store_qtype(const char *F, int L, const Wb::matFile &M);
865 void load_RCStore(const char *F, int L, const QVec &qvec);
866
867 class QVec : public wbvector<QType> {
868 public:
869
870     QVec() : wbvector<QType>() {};
871
872     QVec(unsigned l, const QType* d0, char ref=0)
873         : wbvector<QType>(l,d0,ref) {};
874
875 #ifndef NOMEX
876     QVec(const char *F, int L, const mxArray *a, unsigned d=0)
877         : wbvector<QType>() { init(F,L,a,d); };
878
879     QVec& init(const char *F, int L, const mxArray *a, unsigned d=0){
880         if (!a) { init(); return *this; }
881         if (mxIsChar(a)) { wbststring s(a); return init(F,L,s.data,d); }
882         else wblog(FL,"ERR %s() invalid QVec (%s) !?",FCT,mxGetClassName(a));
883         return *this;
884     };
885 #endif
886
887     QVec& init(unsigned l=0) {
888         wbvector<QType>::init(l); return *this;
889     };
890
891     QVec& init(const char *F, int L, const char *s, unsigned d=0);
892
893     unsigned Qlen() const;
894
895     unsigned Qrank() const;
896
897     unsigned Qlen(wbvector<unsigned> &dd) const;
898     unsigned Qlen(
899         wbvector<unsigned> &dd,
900         wbvector<unsigned> &dz
901     ) const;
902
903     unsigned Qpos(wbvector<unsigned> &dc) const;
904
905     unsigned Qlenz() const {
906         unsigned i=0, n=0;
907         for (; i<len; ++i) { n+=(data[i].qlen() + data[i].qrank()); }
908         return n;
909     }
910
911     wbvector<gTQ>& getScalar(wbvector<gTQ> &qs, char zflag=0) {
912         unsigned l=0, i=0;
913         for (; i<len; ++i) {
914             if (data[i].isParity()) {
915                 if (data[i].qlen()!=1 || data[i].qrank()!=0) wblog(FL,
916                     "ERR %s() %s @ %d/%d !?",
917                     FCT,STR(data[i]),data[i].qlen(),data[i].qrank());
918                 break;
919             }
920         }
921         if (i==len) {
922             qs.init(zflag ? Qlenz() : Qlen());
923         }
924         else if (!zflag) {
925             wbvector<unsigned> dd; qs.init(Qlen(dd));
926             for (i=0; i<len; ++i) {
927                 if (data[i].isParity()) { qs[l]=1; }; l+=dd[i];
928             }

```

```

929     }
930     else {
931         wbvector<unsigned> dd,dz;
932         unsigned l1=Qlen(dd,dz), l2=dz.sum(); qs.init(l1+l2);
933
934         for (i=0; i<dd.len; ++i) {
935             if (data[i].isParity()) { qs[l]=1; }
936             l+=dd[i]; if (zflag!='z') l+=dz[i];
937         }
938     }
939     return qs;
940 };
941
942 unsigned largestRank() const {
943     unsigned i=0, q=0, r=0;
944     for (; i<len; ++i) { q=data[i].qrank(); if (r<q) { r=q; } }
945     return r;
946 };
947
948 template <class TQ>
949 unsigned QDim(const TQ *q) const;
950
951 template <class TQ>
952 unsigned QDim(const TQ *qq, unsigned k) const;
953
954 template <class TQ>
955 wbvector<unsigned>& QDim(
956     const TQ *qq, wbvector<unsigned> &S
957 ) const;
958
959 template <class TQ>
960 wbvector<unsigned>& QDim(
961     const TQ *qq, unsigned k, unsigned r, wbvector<unsigned> &S
962 ) const;
963
964 template <class TQ>
965 wbMatrix<TQ>& getQsub(
966     const wbMatrix<TQ>& Q, const windex &I,
967     wbMatrix<TQ>& QI,
968     wbMatrix<TQ> *Qx=NULL
969 ) const;
970
971 bool allU1() const {
972     for (unsigned i=0; i<len; ++i) { if (!data[i].isU1()) return 0; }
973     return 1;
974 };
975
976 void ReduceU1() {
977     if (len) {
978         for (unsigned i=0; i<len; ++i) { if (!data[i].isU1())
979             return;
980         }; init();
981     }
982 };
983
984 char allAbelian() const {
985     char q=1;
986     for (unsigned i=0; i<len; ++i) {
987         if (!data[i].isAbelian()) { q=0; break; }
988         if (data[i]!=QTYPE_U1) q|=2;
989     }
990     return q;
991 };
992
993 bool anyAbelian() const {
994     if (!len) return 1;
995     for (unsigned i=0; i<len; ++i) {
996         if (data[i].isAbelian()) return 1; }
997     return 0;
998 };
999
1000 bool anyParity() const {
1001     if (len) {
1002         for (unsigned i=0; i<len; ++i) {
1003             if (data[i].isParity()) return 1; }
1004     }
1005     return 0;
1006 };
1007
1008 bool allNonAbelian() const { return !anyAbelian(); };
1009
1010 char isNonAbelian() const {
1011     if (!len) return 0;
1012     for (unsigned i=0; i<len; ++i) {
1013         if (data[i].isNonAbelian()) { return data[i].permitsOM(); }
1014     }
1015     return 0;
1016 };

```

```

1016
1017     bool allAdditive() const {
1018         for (unsigned i=0; i<len; ++i) {
1019             if (!data[i].isAdditive()) return 0;
1020         }
1021         return 1;
1022     };
1023
1024     int permitsOM() const {
1025         int q=0; if (len) {
1026             int qi; unsigned i=0;
1027             for (; i<len; ++i) { if (q<(qi=data[i].permitsOM())) { q=qi; }}
1028         }
1029         return q;
1030     };
1031
1032     int permitsOM(unsigned r) const {
1033         for (unsigned i=0; i<len; ++i) { if (data[i].permitsOM(r)) return 1; }
1034         return 0;
1035     };
1036
1037     bool sameAs(const QVec &b, char lflag=1) const {
1038         if (len==b.len || !lflag) { return (*this)==b; }
1039         if (len && !b.len) return allU1();
1040         if (b.len && !len) return b.allU1();
1041         return 0;
1042     };
1043
1044     bool sameType (const QVec &b) const {
1045         if (len!=b.len) return 0;
1046         for (unsigned i=0; i<len; i++)
1047             if (data[i].type!=b.data[i].type) return 0;
1048         return 1;
1049     };
1050
1051     wstring toStr(const char vflag=0) const;
1052
1053 };
1054
1055 namespace DY {
1056
1057 template <class TQ>
1058 class weight_info {
1059 public:
1060
1061     weight_info() : m(0) {};
1062
1063     weight_info(const weight_info& w) { init(w); };
1064
1065     weight_info(unsigned m_, unsigned n) : m(m_) {
1066         p.init(n); };
1067
1068     weight_info& init(unsigned m_, unsigned n) {
1069         m=m_; p.init(n); return *this;
1070     };
1071
1072     weight_info& init(const weight_info& w) {
1073         m=w.m; p.init(w.p);
1074         return *this;
1075     };
1076
1077     weight_info& operator=(const weight_info& w) { return init(w); }
1078
1079     weight_info& save2(weight_info& w) {
1080         m=w.m; w.m=0; w.p.save2(p);
1081         return *this;
1082     };
1083
1084     int m;
1085
1086     qset<TQ> p;
1087
1088 protected:
1089 private:
1090 };
1091
1092
1093 template <class TQ>
1094 class Weights {
1095 public:
1096     Weights() : n(0) {};
1097
1098     Weights(const qset<TQ> &q) : n(qs.len), qm(qs) {
1099         if (!n) wblog(FL,"ERR %s() got empty qset !?",FCT);
1100         weight_info<TQ> &w=W[q];
1101         w.init(1,n);
1102     }

```

```

1103         X[qs]=&w;
1104     };
1105
1106     Weights& init(const qset<TQ> &qs) {
1107         if (!n=qs.len()) wblog(FL,"ERR %s() got empty qset !?",FCT);
1108         qm.init(qs); W.clear(); X.clear();
1109         weight_info<TQ> &w=W[qs];
1110         w.init(1,n);
1111         X[qs]=&w;
1112         return *this;
1113     };
1114
1115     size_t dim() const {
1116         size_t mtot=0, merr=0;
1117         for (auto I=W.begin(); I!=W.end(); ++I) {
1118             if (I->second.m>0) { mtot+=I->second.m; }
1119             else { ++merr; }
1120         }
1121         if (merr) wblog(FL,
1122             "WRN %s() got %d undetermined weight spaces !?",FCT,merr);
1123         return mtot;
1124     };
1125
1126     void print(
1127         const char *F, int L, const QType &q,
1128         char vflag=0) const;
1129
1130     mxArray* toMx() const;
1131
1132     unsigned n;
1133     qset<TQ> qm;
1134
1135     map <
1136         qset<TQ>,
1137         weight_info<TQ>
1138     > W;
1139
1140     map <
1141         qset<TQ>,
1142         const weight_info<TQ>*
1143     > X;
1144
1145     protected:
1146     private:
1147 };
1148
1149 class Symmetry {
1150
1151     public:
1152
1153     Symmetry(const QType &q_) : q(q_) { init(q); };
1154     Symmetry(const mxArray *a) { q.init(FL,a); init(q); };
1155
1156     Symmetry& init(const QType &q);
1157
1158     template <class TQ>
1159     size_t getWeightsFT(
1160         const char *F, int L, const qset<TQ> &qs,
1161         Weights<TQ> &R
1162     );
1163
1164     template <class TQ>
1165     int getInnerMultFT(
1166         const qset<TQ> &qs, const weight_info<TQ> &w,
1167         Weights<TQ> &R, char vflag=0
1168     );
1169
1170     template <class TQ>
1171     int Get(
1172         const qset<TQ> &qk, map < qset<TQ>, weight_info<TQ> > &W,
1173         char vflag=0
1174     );
1175
1176     template <class TQ>
1177     int wExpand(const char *F, int L, Weights<TQ> &R) const;
1178
1179     mxArray* toMx() const;
1180
1181     QType q;
1182     unsigned n;
1183
1184     wbararray<double> R;
1185
1186     wbMatrix<int> A;
1187
1188     wbvector<int> n2;
1189

```



```

1190     wbararray<int> M;
1191
1192     protected:
1193     private:
1194
1195 };
1196
1197 };
1198
1199 class QDir: public wbvector<char> {
1200
1201     public:
1202
1203     QDir() : wbvector<char>() {};
1204
1205     QDir(unsigned l, const char* d=NULL, char ref=0)
1206         : wbvector<char>(l,d,ref) {};
1207
1208     QDir(const IndexLabels& it) { init(it); };
1209     QDir(const char *F, int L,const char *s) { init(F,L,s); };
1210     QDir(const char *s) { init(FL,s); };
1211
1212 #ifndef NOMEX
1213     QDir(const char *F, int L, const mxArray *a) { init(F,L,a); };
1214 #endif
1215
1216     QDir& init(unsigned l=0, const char* d=NULL, char ref=0) {
1217         wbvector<char>::init(l,d,ref);
1218         return *this;
1219     };
1220
1221     QDir& operator=(const char *s) { return init(FL,s); };
1222
1223     QDir& init(const char *F, int L,const char *s);
1224     QDir& init(const char *s) { return init(FL,s); };
1225     QDir& init(const IndexLabels& it);
1226
1227     QDir& init(const char *F, int L,const mxArray *a) {
1228         wbsttring s(F,L,a); init(F,L,s.data);
1229         return *this;
1230     };
1231
1232     QDir& init_iout(const char *F, int L,
1233         unsigned r, unsigned iout
1234     );
1235
1236     unsigned isSorted() const {
1237         unsigned i=0, j=0;
1238
1239         #ifndef WB_SKIP_ASSERT
1240             for (; i<len; ++i) if (!data[i]) {
1241                 wblog(FL,"ERR %s() invalid qdir '%s'",FCT,STR_(this));
1242                 i=0;
1243             }
1244         #endif
1245
1246         for (; i<len; ++i) { if (data[i]>0) break; };
1247         if (i==len) return 1;
1248         if (i) return 0;
1249
1250         for (; i<len; ++i) { if (data[i]<0) break; }
1251         if (i==len) return (len+1);
1252
1253         for (j=i+1; i<len; ++i) { if (data[i]>0) break; }
1254         if (i==len) return j;
1255
1256         return 0;
1257     };
1258
1259     bool operator==(const char *s) const;
1260     bool operator!=(const char *s) const { return !((*this)==s); };
1261
1262     bool operator==(const QDir &q) const {
1263         return this->wbvector<char>::operator==(q); };
1264     bool operator!=(const QDir &q) const { return !((*this)==q); };
1265
1266     bool operator<(const QDir &q) const {
1267         return wbvector<char>::operator<(q); };
1268
1269     QDir& Conj() {
1270         for (unsigned i=0; i<len; ++i) { data[i]=-data[i]; }
1271         return *this;
1272     };
1273
1274     char& Conj(const unsigned &i) {
1275         if (i>=len) wblog(FL,
1276             "ERR %s() index out of bounds (%d/%d)",FCT,i,len);
1277         return (data[i]=-data[i]);

```

```

1277     };
1278
1279     unsigned nconj() {
1280         unsigned i=0, n=0;
1281         for (; i<len; ++i) { if (data[i]<0) ++n; }
1282         return n;
1283     };
1284
1285     unsigned nconj(const ctrIdx &Ix) {
1286         unsigned n=0, i=0;
1287         wbvector<char> q(*this); const unsigned* ix=Ix.data;
1288
1289         for (; i<Ix.len; ++i) {
1290             if (ix[i]>=len) wblog(FL,"ERR %s() index out of bounds"
1291                 "(%d/%d: %d/%d)",FCT,i,Ix.len, ix[i],len);
1292             q[ix[i]]=0;
1293         }
1294         if (Ix.conj)
1295             { for (i=0; i<len; ++i) { if (q[i]>0) ++n; }}
1296         else { for (i=0; i<len; ++i) { if (q[i]<0) ++n; }}
1297
1298         return n;
1299     };
1300
1301     wbstring toStr() const;
1302     wbstring toTag() const;
1303
1304     mxArray* toMx() const { return toStr().toMx(); };
1305
1306 protected:
1307 private:
1308 };
1309
1310
1311 class cgdStatus {
1312
1313 public:
1314
1315     cgdStatus() : t(CGD_DEFAULT), ctime(0), mtime(0), ID(0) {
1316         memset(user,0,4); };
1317
1318     cgdStatus(CGD_TYPE t_) : cgdStatus() { t=t_; init_time(); };
1319
1320     cgdStatus(double t_) : cgdStatus() {
1321         if (double(t)!=t_ || t>=CGD_NUM_TYPES) wblog(FL,
1322             "ERR %s() invalid CData type %g",FCT,t_);
1323         t=CGD_TYPE(t_);
1324     };
1325
1326     cgdStatus(const char *F, int L, mxArray *a)
1327         : cgdStatus() { t=CGD_DEFAULT; init(F,L,a); };
1328
1329 template <class TQ>
1330 cgdStatus(const CDATA_TQ& C) { init(C.cstat); }
1331
1332     cgdStatus& init() {
1333         t=CGD_DEFAULT; ctime=0; mtime=0; ID=0;
1334         memset(user,0,2);
1335         return *this;
1336     };
1337
1338     cgdStatus& init(const CGD_TYPE &t_) {
1339         t=t_; init_time(); return *this;
1340     };
1341
1342     cgdStatus& init0(const CGD_TYPE &t_) {
1343         t=t_; ctime=0; mtime=0; ID=0;
1344         memset(user,0,2);
1345         return *this;
1346     };
1347
1348     cgdStatus& update_m() { init_time('m'); return *this; };
1349     cgdStatus& update_m(const CGD_TYPE &t_) {
1350         t=t_; init_time('m'); return *this;
1351     };
1352
1353     cgdStatus& operator=(const cgdStatus &b) { return init(b); };
1354
1355     cgdStatus& init(const cgdStatus &b) {
1356         if (ID && b.ID && (ID!=b.ID || ctime!=b.ctime)) wblog(FL,
1357             "ERR got initialized cstat with ID mismatch\n%s\n%s",
1358             STR2_(this,'V'),STR2(b,'V'));
1359         t=b.t; ctime=b.ctime; mtime=b.mtime; ID=b.ID;
1360
1361         user[0]=b.user[0];
1362         user[1]=b.user[1];
1363

```

```

1364     return *this;
1365 };
1366
1367 CGR_TYPE init(const char *F, int L, const mxArray *a);
1368
1369 int sameID(const cgdStatus &b) const {
1370     return (ID==b.ID && ctime==b.ctime);
1371 };
1372
1373 int cmp(const char *F, int L, const cgdStatus &b) const {
1374     if (ID!=b.ID || ctime!=b.ctime) { if (F) wblog(F,L,
1375         "ERR %s() got CData status mismatch!%N %s%N<> %s",
1376         FCT, toStr('V').data, b.toStr('V').data);
1377         else return -11;
1378     }
1379     if (mtime<ctime || b.mtime<b.ctime) { if (F) wblog(F,L,
1380         "ERR %s() invalid mtime",FCT);
1381         else return -12;
1382     }
1383     return (mtime<b.mtime ? -1 : (mtime==b.mtime ? 0 : +1));
1384 };
1385
1386 bool olderThan(const cgdStatus &b) const { return (cmp(F,L,b)<0); };
1387 bool newerThan(const cgdStatus &b) const { return (cmp(F,L,b)>0); };
1388
1389 bool hasID(char lflag=0) const {
1390     if (!lflag)
1391         return (ctime && mtime && ID);
1392     else return (ctime || mtime || ID);
1393 };
1394
1395 bool operator==(const CGD_TYPE t_) const { return (t==t_); };
1396 bool operator!=(const CGD_TYPE t_) const { return (t!=t_); };
1397
1398 bool operator==(const cgdStatus &b) const {
1399     return (ctime==b.ctime && mtime==b.mtime && ID==b.ID);
1400 };
1401
1402 bool operator!=(const cgdStatus &b) const { return !((*this)==b); };
1403
1404 bool isSet() const { return (ctime || mtime || ID); };
1405 bool isEmpty() const {
1406     return (!ctime && !mtime && !ID && t==CGD_DEFAULT);
1407 };
1408
1409 int invalid() const {
1410     int e=0;
1411     if ((ctime!=0) ^ (mtime!=0)) { e|=1; }
1412     if (t==CGD_EXPLICIT) {
1413         if (ctime || ID>99) { e|=8; }
1414     }
1415     else {
1416         if ((ctime!=0) ^ (ID!=0)) { e|=2; }
1417         if ((ctime!=0) ^ (t!=CGD_DEFAULT)) { e|=4; }
1418     }
1419     return e;
1420 };
1421
1422 bool sameAs(const cgdStatus &b, char lflag=0) const {
1423     if (ctime!=b.ctime || ID!=b.ID) return 0;
1424
1425     if (lflag>2) {
1426         if (lflag=='l') lflag=1; else
1427         if (lflag=='L') lflag=2; else
1428             wblog(FL,"WRN %s() unexpected lflag=%d",FCT,lflag);
1429     }
1430     if (lflag<2 && mtime!=b.mtime) return 0;
1431     if (lflag<1 && t!=b.t) return 0;
1432     return 1;
1433 };
1434
1435 const char* tstr() const {
1436     if (t>=CGD_NUM_TYPES) wblog(FL,"ERR %s() "
1437         "cgd type out of bounds (%d/%d)",FCT,t,CGD_NUM_TYPES);
1438     return CGD_TYPE_STR[t];
1439 };
1440
1441 wbsting toStr(char vflag=0) const;
1442 wbsting u2Str(const char *istr=NULL) const;
1443
1444 mxArray* toMx() const;
1445 mxArray* toMx(CGR_TYPE rt) const;
1446
1447 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
1448 mxArray* add2MxStruct(mxAarray *S, unsigned k) const;
1449
1450 CGD_TYPE t;

```

```

1451
1452     double ctime;
1453     double mtime;
1454
1455     unsigned ID;
1456
1457     unsigned char user[4];
1458
1459 protected:
1460 private:
1461
1462     cgdStatus& init_time(char flag=0);
1463     wbsting to_vstr(double t) const;
1464
1465     void setID() {
1466         unsigned i=0, n=3;
1467         for (; i<n; ++i) {
1468             ID= (::rand() & ((1<<20)-1)); if (ID) break;
1469         }
1470         if (!ID) wblog(FL, "ERR %s() got ID=%d (%d/%d) !?", FCT, ID, i, n);
1471     };
1472 };
1473
1474 bool operator!(const cgdStatus &q) {
1475     return (q.t==CGD_DEFAULT && !q.ctime && !q.mtime && !q.ID);
1476 };
1477
1478 class cgrType {
1479 public:
1480
1481     cgrType(CGR_TYPE t_=CGR_DEFAULT) : t(t_) {};
1482
1483     cgrType(double t_) { init(FL, t_); };
1484
1485     template<class T>
1486     cgrType& init(const char *F, int L, T t_) {
1487         t=CGR_TYPE(t_);
1488         if (T(t)!=t_ || t>=CGR_NUM_TYPES) wblog(F_L,
1489             "ERR %s() invalid CRef type %g", FCT, double(t_))
1490         );
1491         return *this;
1492     };
1493 };
1494
1495     cgrType& operator=(const cgrType &q) { t=q.t; return *this; };
1496     cgrType& operator=(const CGR_TYPE &t_) { t=t_; return *this; };
1497
1498     bool operator< (const CGR_TYPE t_) const { return (t< t_); };
1499     bool operator<= (const CGR_TYPE t_) const { return (t<=t_); };
1500     bool operator> (const CGR_TYPE t_) const { return (t> t_); };
1501     bool operator>= (const CGR_TYPE t_) const { return (t>=t_); };
1502     bool operator== (const CGR_TYPE t_) const { return (t==t_); };
1503     bool operator!= (const CGR_TYPE t_) const { return (t!=t_); };
1504
1505     bool operator< (const cgrType &q) const { return (q.t< t); };
1506     bool operator<= (const cgrType &q) const { return (q.t<=t); };
1507     bool operator> (const cgrType &q) const { return (q.t> t); };
1508     bool operator>= (const cgrType &q) const { return (q.t>=t); };
1509     bool operator== (const cgrType &q) const { return (q.t==t); };
1510     bool operator!= (const cgrType &q) const { return (q.t!=t); };
1511
1512     const char* tostr() const {
1513         if (t>=CGR_NUM_TYPES) wblog(FL, "ERR %s() "
1514             "cgrType out of bounds (%d/%d)", FCT, t, CGR_NUM_TYPES);
1515         return CGR_TYPE_STR[t];
1516     };
1517
1518     wbsting toStr() const { return tostr(); }
1519
1520     CGR_TYPE t;
1521
1522 protected:
1523 private:
1524 };
1525
1526 template <class TQ>
1527 class qset : public wbvector<TQ> {
1528 public:
1529     qset() : wbvector<TQ>() {};
1530
1531     qset(unsigned l, const TQ* d0=NULL, char ref=0)
1532         : wbvector<TQ>(l, d0, ref) {};
1533
1534     qset(const qset<TQ> &q)
1535         : wbvector<TQ>(q.len, q.data) {};
1536
1537

```

```

1538     qset(const QType &Q, const TQ *qq)
1539     : wvector<TQ>() { init(Q,qq); };
1540
1541     qset(const QVec &Q, const TQ *qq)
1542     : wvector<TQ>() { init(Q,qq); };
1543
1544     qset(const char *F, int L, const mxArray *a) : wvector<TQ>() {
1545         if (mxIsChar(a)) { wstring s(a); init(F_L,s.data); }
1546         else init(F,L,a,NULL,1);
1547     };
1548
1549     qset(const char *F, int L, const mxArray *a,
1550           const QVec *Q, char check_type=1) : wvector<TQ>() {
1551         init(F,L,a,Q,check_type);
1552     };
1553
1554     qset(const qset<TQ> &q1, const qset<TQ> &q2)
1555     : wvector<TQ>() { init(q1,q2); };
1556
1557     qset(const qset<TQ> &q1, const qset<TQ> &q2, const qset<TQ> &q3)
1558     : wvector<TQ>() { init(q1,q2,q3); };
1559
1560     qset(const qset<TQ> &q1, char op, const qset<TQ> &q2);
1561
1562     qset& init(const qset<TQ> &q1) {
1563         wvector<TQ>::init(q1);
1564         return *this;
1565     };
1566
1567     qset& init(const char *F, int L, const char *s, QType *t=NULL);
1568     qset& init(
1569         const char *F, int L, const mxArray *a,
1570         const QVec *Q=NULL, char check_type=1
1571     ){
1572         if (mxIsChar(a)) { wstring s(a); init(F_L,s.data); }
1573         else wvector<TQ>::init(F,L,a,NULL,check_type);
1574
1575         if (Q) {
1576             unsigned n=Q->Qlen();
1577             if ((this->len)%n) wblog(FL,
1578                 "ERR %s() got invalid qset.len=%d @ %d",FCT,this->len,n
1579             );
1580         }
1581         return *this;
1582     };
1583
1584     qset& init(const QType &Q, const TQ *qq) {
1585         wvector<TQ>::init(Q.Qlen(),qq);
1586         return *this;
1587     };
1588
1589     qset& init(const QVec &Q, const TQ *qq) {
1590         init(Q.Qlen(),qq);
1591         return *this;
1592     };
1593
1594     qset& init(const qset<TQ> &q1, const qset<TQ> &q2) {
1595         init(q1.len, q1.data, q2.len, q2.data);
1596         return *this;
1597     };
1598
1599     qset& init(const qset<TQ> &q1, const qset<TQ> &q2, const qset<TQ> &q3) {
1600         init(q1.len, q1.data, q2.len, q2.data, q3.len, q3.data);
1601         return *this;
1602     };
1603
1604     qset& init() { wvector<TQ>::init(); return *this; };
1605
1606     qset& init(unsigned l, const TQ *d0=NULL) {
1607         wvector<TQ>::RENEW(l,d0);
1608         return *this;
1609     };
1610
1611     qset& init(
1612         unsigned l1, const TQ *d1,
1613         unsigned l2, const TQ *d2
1614     );
1615
1616     qset& init(
1617         unsigned l1, const TQ *d1,
1618         unsigned l2, const TQ *d2,
1619         unsigned l3, const TQ *d3
1620     );
1621
1622     qset<TQ>& init(const TQ q1) { return init(1U,&q1); }
1623
1624     qset<TQ>& init(const TQ q1, const TQ q2) {

```

```

1625         return init(1,&q1,1,&q2);
1626     };
1627     qset<TQ>& init(const TQ q1, const TQ q2, const TQ q3) {
1628         return init(1,&q1,1,&q2,1,&q3);
1629     };
1630
1631     qset<TQ>& initStride(
1632         const TQ* d_, unsigned n, unsigned r, unsigned QDIM
1633     ){
1634         init(n*r); if (d_ && this->len) {
1635             Wb::cpyStride(this->data,d_,n,NULL,r,-1,QDIM);
1636         }
1637         return *this;
1638     };
1639
1640     bool operator<(const qset<TQ> &b) const;
1641
1642     int checkNQs(const QVec &qq, const char *F=NULL, int L=0);
1643
1644     bool isConsistent(
1645         const QVec &qq, int rank=-1, const char *F=NULL, int L=0) const;
1646
1647     unsigned QDim(const QVec &q, int rank=-1) const;
1648
1649     qset<TQ>& times(TQ fac, qset<TQ> &x) const {
1650         x=*this; x*=fac; return x;
1651     };
1652
1653     wstring toStr() const { return wbvector<TQ>::toStr(); };
1654
1655     wstring toStr(const QType &t __attribute__((unused))) const {
1656         if (CG::gotQAlpha(this->data, this->len)) {
1657             wstring sout(this->len+1);
1658             if (CG::qset2cstr(this->data,sout.data,this->len)) wblog(FL,
1659                 "ERR %s() failed to obtain compact qset string !?",FCT);
1660             sout.data[this->len]=0;
1661             return sout;
1662         }
1663         return wbvector<TQ>::toStr();
1664     };
1665 };
1666
1667 template <class TQ>
1668 bool qset<TQ>::operator<(const qset<TQ> &b) const {
1669
1670     if (!this->len || !b.len) wblog(FL,
1671         "ERR %s() got empty object (%d/%d)",FCT,this->len,b.len);
1672
1673     return wbvector<TQ>::operator<(b);
1674 };
1675
1676 template <>
1677 bool qset<double>::operator<(const qset<double> &b) const {
1678
1679     if (!len || !b.len) wblog(FL,
1680         "ERR %s() got empty object (%d/%d)",FCT,len,b.len);
1681     if (len!=b.len) wblog(FL,
1682         "ERR %s() got length mismatch (%d/%d)",FCT,len,b.len);
1683
1684     double *d=b.data;
1685     for (unsigned i=0; i<len; i++) {
1686         if (float(data[i])<float(d[i])) return 1;
1687         if (float(data[i])>float(d[i])) return 0;
1688     }
1689     return 0;
1690 };
1691
1692 template <>
1693 bool qset<float>::operator<(const qset<float> &b) const {
1694     wblog(FL,"ERR %s() double precision preferred over '%s' (0x%lX)",
1695         TSTR(float),&b);
1696     return 0;
1697 };
1698
1699 template <class TQ>
1700 class QSet {
1701 public:
1702
1703     QSet() : t(QTYPE_UNKNOWN) {};
1704
1705     QSet(const QSet &B) { t=B.t; qs=B.qs; qdir=B.qdir; };
1706
1707     QSet(const QType &t_, const qset<TQ> &qs_,
1708         unsigned iout=0,
1709         char isref=0
1710     )

```

```

1712     ){
1713         init(t,qs,iout,isref);
1714     };
1715
1716     QSet(const QType &t_, const IndexLabels &it, unsigned r){
1717         t=t_; qdir.init(it); qs.init(r*t_.qlen());
1718     };
1719
1720     QSet(const CRef<TQ> &A) { init(A); };
1721
1722     QSet(const char *F, int L, const mxArray *a) { init(F,L,a); };
1723     QSet(const mxArray *a) { init(0,0,a); };
1724
1725     QSet(const char *F, int L,
1726           const CRef<TQ> &A, const ctrIdx &ica,
1727           const CRef<TQ> &B, const ctrIdx &icb, wbperm *P=NULL
1728     ){ init(F,L,A,ica,B,icb,P); };
1729
1730     QSet(const char *F, int L, const char *s) { init_str(F,L,s); };
1731
1732     QSet& init(const CRef<TQ> &A) {
1733         if (!A.cgb) init();
1734         else { init(*A.cgb);
1735             if (A.gotPerm()) Permute(A.cgp);
1736             if (A.gotConj()) Conj();
1737         }
1738         return *this;
1739     };
1740
1741     QSet& init(const char *F, int L, const mxArray *a, unsigned k=0);
1742
1743     QSet& init(const QType t_ = QTYPE_UNKNOWN) { t=QTYPE_UNKNOWN;
1744         t=t_; if (qs.len() ) qs.init();
1745         if (qdir.len) qdir.init();
1746         return *this;
1747     };
1748
1749     QSet& init(const QSet &B) {
1750         t=B.t; qs=B.qs; qdir=B.qdir; return *this; };
1751
1752     QSet& init2ref(const QSet &B) {
1753         t=B.t; qs.init2ref(B.qs); qdir.init2ref(B.qdir); return *this;
1754     };
1755
1756     QSet& init(const QType &t_, const TQ* q, const char *qd);
1757
1758     QSet& init(
1759         const QType &t_, const qset<gTQ>&qs_,
1760         unsigned iout=0,
1761         char isref=0
1762     );
1763
1764     QSet& init(
1765         const QType t_, const TQ* qs_, unsigned r, unsigned N,
1766         unsigned iout=0
1767     );
1768
1769     QSet& init(
1770         const QType t_, const TQ* qs_, unsigned r, unsigned N,
1771         const IndexLabels &it
1772     );
1773
1774     QSet& init1J(const QType &t_, const TQ *q, char iflag=0) {
1775         unsigned m=t_.qlen();
1776         t=t_; qdir.init2val(2, iflag ? -1 : +1);
1777
1778         qs.init(2*m); memcpy(qs.data, q, m*sizeof(TQ));
1779         t.getDual(q, qs.data+m);
1780
1781         return *this;
1782     };
1783
1784     QSet& init2(const QType &t_, const TQ *q=NULL) {
1785         unsigned m=t_.qlen();
1786         t=t_; qdir.init_iout(FL,2,2); qs.init(2*m);
1787         if (q) { unsigned l=m*sizeof(TQ);
1788             memcpy(qs.data, q, l);
1789             memcpy(qs.data+m, q, l);
1790         }
1791         return *this;
1792     };
1793
1794     QSet& init2(const QType &t_,
1795         const qset<TQ> &q1, const qset<TQ> &q2, const char *qds="++") {
1796
1797         unsigned m=t_.qlen(), l=m*sizeof(TQ);
1798         if (q1.len<m || q2.len<m) wblog(FL,

```

```

1799         "ERR %s() invalid qset length (%s: %d,%d/%d)",
1800         FCT, STR(t_), q1.len, q2.len, m
1801     );
1802     t=t_; qdir=qds; qs.init(2*m);
1803     memcpy(qs.data, q1.data, 1);
1804     memcpy(qs.data+m, q2.data, 1);
1805     return *this;
1806 };
1807
1808 QSet& init1(const QType &t_, TQ *q_=NULL) {
1809     t=t_; qs.init(t_.qlen(), q_);
1810     qdir.init2val(1, +1);
1811     return *this;
1812 };
1813
1814 QSet& init3(const QType &t_) {
1815     t=t_; qs.init(3*t_.qlen());
1816     qdir.init_iout(FL, 3, 3); return *this;
1817 };
1818
1819 QSet& init3(const QType &t_,
1820     const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J
1821 ) {
1822     unsigned n=t_.qlen();
1823     if (J1.len!=n || J1.len!=J2.len || J1.len!=J.len) {
1824         if (J1.len!=n || J2.len!=n || J.len!=n) wblog(FL,
1825             "ERR %s() severe length inconsistency [%d %d %d; %d]", FCT,
1826             J1.len, J2.len, J.len, n
1827         );
1828     }
1829     qs.init(J1, J2, J); t=t_;
1830     qdir.init_iout(FL, 3, 3);
1831
1832     return *this;
1833 };
1834
1835 QSet& init3(const QType &t_,
1836     const TQ &J1, const TQ &J2, const TQ &J
1837 ) {
1838     unsigned n=t_.qlen();
1839     if (n!=1) wblog(FL,
1840         "ERR %s() got QType %s (%d/1) !?", FCT, STR(t_), n);
1841     qs.init(3U); qs[0]=J1; qs[1]=J2; qs[2]=J;
1842     qdir.init_iout(FL, 3, 3); t=t_;
1843     return *this;
1844 };
1845
1846 QSet& init(const char *F, int L,
1847     const CRef<TQ> &A, const ctrIdx &ica,
1848     const CRef<TQ> &B, const ctrIdx &icb, wbperm *Pcgd=NULL
1849 );
1850
1851 QSet& init_str(const char *F, int L, const char *s);
1852
1853 QSet& Conj() { qdir.Conj(); return *this; };
1854 QSet& Conj(unsigned i);
1855
1856 unsigned rank(const char *F=NULL, int L=0) const {
1857     unsigned n=t_.qlen();
1858     if (n*qdir.len!=qs.len) wblog(F_L,
1859         "ERR %s() invalid qs (%d*%d == %d?)", FCT, n, qdir.len, qs.len);
1860     return qdir.len;
1861 };
1862
1863 QSet& permute(QSet &B, const wbperm &P, char iflag=0) const;
1864
1865 QSet& Permute(const wbperm &P, char iflag=0) {
1866     QSet<TQ> X; permute(X, P, iflag);
1867     return X.save2(*this);
1868 };
1869
1870 QSet& save2(QSet &X) {
1871     X.t=t; qdir.save2(X.qdir); qs.save2(X.qs);
1872     return X;
1873 };
1874
1875 QSet& operator=(const QSet &B) { return init(B); };
1876
1877 bool operator==(const QSet &B) const {
1878     return (t==B.t && qs==B.qs && qdir==B.qdir);
1879 };
1880
1881 bool operator!=(const QSet &B) const {
1882     return (t!=B.t || qs!=B.qs || qdir!=B.qdir);
1883 };
1884
1885 char cmp(const QSet<TQ> &B) const {

```



```

1886     char i=0;
1887     if ((i=t.cmp(B.t))) { return i; }
1888     if ((i=qdir.cmp(B.qdir))) { return i; }
1889     return (i=qs.cmp(B.qs));
1890 };
1891
1892 bool operator<(const QSet<TQ> &B) const { return cmp(B)<0; }
1893
1894 bool isEmpty(char lflag=0) const {
1895     if (qs.len || qdir.len) {
1896         if ((t==QTYPE_UNKNOWN && !lflag) || qs.len!=qdir.len*t.qlen())
1897             wblog(FL,"ERR %s() got invalid QSet: %s (%d/%d*%d)",
1898                 FCT,STR_(this), qs.len, qdir.len, t.qlen());
1899         return 0;
1900     }
1901     return (t==QTYPE_UNKNOWN || lflag ? 1 : 0);
1902 };
1903
1904 bool gotRCData() const;
1905
1906 int isZero() const;
1907
1908 unsigned isStd3() const;
1909
1910 unsigned checkStd3() const;
1911
1912 int init2opt3(
1913     const QSet<TQ> &Q, wbperm &P, wbvector<char> &cflags);
1914
1915 QSet& Convert(const char *s, wbvector<char> &cflags);
1916
1917 wbstring sizeStr() const;
1918
1919 bool isScalar() const;
1920 bool is1JSymbol() const;
1921
1922 bool isAbelian() const { return (this->t.isAbelian()); };
1923
1924 int checkQ_abelian(const char *F, int L) const;
1925
1926 int checkQ_SU2(const char *F, int L) const;
1927
1928 QSet& Sort(wbperm *cgp=NULL, char *conj=NULL, char iflag=0);
1929 bool isSorted() const;
1930
1931 size_t memSize() const {
1932     return ( sizeof(t)
1933         + sizeof(qs ) + qs .len*sizeof(TQ)
1934         + sizeof(qdir) + qdir.len*sizeof(char)
1935     );
1936 };
1937
1938 wbstring QStrS(const wbperm *cgp=NULL, char sep='|') const;
1939 wbstring QStr() const;
1940
1941 wbstring toStr(const char *istr=NULL) const;
1942 wbstring toTag() const;
1943
1944 mxArray* toMx() const;
1945
1946 QType t;
1947 qset<TQ> qs;
1948
1949 QDir qdir;
1950
1951 protected:
1952 private:
1953
1954     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
1955     void add2MxStruct(mxAarray *S, unsigned i) const;
1956 };
1957
1958 template <class TQ>
1959 unsigned QSet<TQ>::isStd3() const {
1960
1961     unsigned q=0; { if (qdir!="++-") return q; }
1962
1963     unsigned m=t.qlen(), l=t.maxDimLocal(),
1964         d1=t.qdim(qs.data ),
1965         d2=t.qdim(qs.data+ m),
1966         d3=t.qdim(qs.data+2*m), dmax=MAX(1U<<6,1); dmax*=dmax;
1967
1968     if (qdir.len*m!=qs.len) wblog(FL,
1969         "ERR %s %d*%d=? %d",FCT,STR_(this),qdir.len,m,qs.len);
1970
1971     q=(d1*d2<=dmax || d1<=1 || d2<=1) ? 1 : 2;
1972

```

```

1973     if (d1>2*d3 && (d2*d3<=dmax || d2<=1 || d3<=1)) q|=4;
1974     if (d2>2*d3 && (d1*d3<=dmax || d1<=1 || d3<=1)) q|=8;
1975
1976     return q;
1977 };
1978
1979 template <class TQ>
1980 unsigned QSet<TQ>::checkStd3() const {
1981
1982     unsigned q=0; { if (qdir.len!=3) return q; }
1983     unsigned m=t.qlen(), l=t.maxDimLocal(),
1984             d1=t.qdim(qs.data),
1985             d2=t.qdim(qs.data+m),
1986             d3=t.qdim(qs.data+2*m), dmax=MAX(1U<<6,1); dmax*=dmax;
1987
1988     if (qdir.len*m!=qs.len) wblog(FL,
1989     "ERR %s having %d*%d =? %d",FCT,STR_(this),qdir.len,m,qs.len);
1990
1991     if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<0);
1992
1993     if (!q) { dmax=1U<<16;
1994     if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<1);
1995     }
1996     if (!q) { dmax=1U<<19;
1997     if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<2);
1998     }
1999
2000     if (d1<=1) q|=(1<<3);
2001     if (d2<=1) q|=(1<<4);
2002     if (d3<=1) q|=(1<<5);
2003
2004     if (d1<d2) {
2005         if (d2<=d3) q|=(3<<6);
2006         else q|=(2<<6);
2007     } else {
2008         if (d1<=d3) q|=(3<<6);
2009         else q|=(1<<6);
2010     }
2011
2012     return q;
2013 };
2014
2015 template <class TQ>
2016 int QSet<TQ>::init2opt3(
2017     const QSet<TQ> &Q, wbpem &iP, wbpem<char> &cflags) {
2018
2019     unsigned i3=Q.checkStd3(), w=(i3>>6);
2020     unsigned m=Q.t.qlen();
2021     wbpem P;
2022
2023     if (!(i3&63)) { return -1; }
2024
2025     if (!w || w>3 || m*Q.qdir.len!=Q.qs.len) wblog(FL,
2026     "ERR %s() got %s\n(%d -> %d) !?",FCT,STR(Q),BITS(i3),BITS(w));
2027     if (w==1) P.init("231"); else
2028     if (w==2) P.init("132"); else
2029     if (w==3) P.init();
2030
2031     Q.permute(*this,P);
2032     Convert("++-",cflags);
2033
2034     if (Wb::cmpRange(qs.data,qs.data+m,m)>0) {
2035         wbpem p2="213"; Permute(p2);
2036         P.Permute(p2); cflags.Permute(p2);
2037     }
2038
2039     P.invert(iP);
2040
2041     return 0;
2042 };
2043
2044 template <class TQ>
2045 QSet<TQ> & QSet<TQ>::Convert(const char *s, wbpem<char> &cflags) {
2046
2047     unsigned i=0, r=(!s || !s[0] ? 0 : strlen(s));
2048
2049     if (r!=qdir.len) wblog(FL,
2050     "ERR %s() size mismatch %s [0/%d]",FCT,STR_(this),qdir.len);
2051     if (!s || !s[0]) { cflags.init(); return *this; }
2052
2053     cflags.init(r);
2054
2055     for (; i<r; ++i) {
2056         if ((s[i]=='+' && qdir[i]<0) || (s[i]=='-' && qdir[i]>0)) {
2057             Conj(i); cflags[i]=1;
2058         }
2059         else if (s[i]!='+' && s[i]!='-')

```

```

2060         wblog(FL, "ERR %s() invalid qdir='%s' !?", FCT, s);
2061     }
2062
2063     return *this;
2064 };
2065
2066 template <class TQ>
2067 QSet<TQ>& QSet<TQ>::Conj(unsigned i) {
2068
2069     unsigned m=t.qlen();
2070     if (i>=qdir.len || !qdir[i]) wblog(FL, "ERR %s() "
2071         "index out of bounds %s (%d/%d) !?", FCT, STR_(this), i+1, qdir.len);
2072     if (m*qdir.len!=qs.len) wblog(FL, "ERR %s() "
2073         "invalid %s (%d*%d=? %d) !?", FCT, STR_(this), m, qdir.len, qs.len);
2074
2075     qdir[i]=-qdir[i];
2076     t.setDual(qs.data+i*m);
2077
2078     return *this;
2079 };
2080
2081 template <class TQ>
2082 wstring QSet<TQ>::sizeStr() const {
2083     unsigned m=t.qlen(), r=qdir.len;
2084
2085     if (r*m!=qs.len) wblog(FL,
2086         "ERR %s: %d != %d*%d !?", STR_(this), qs.len, r, m);
2087     try {
2088         if (qs.len) { wvector<size_t> dd(r);
2089             for (unsigned i=0; i<r; ++i) { dd[i]=t.qdim(qs.data+i*m); }
2090             return dd.toStrf("", "x");
2091         }
2092     }
2093     catch (...) {
2094         wblog(FL, "ERR %s() failed to obtain size(Q)\n%s", FCT, STR_(this));
2095     }
2096     return "";
2097 };
2098
2099 template <class TQ>
2100 class QHash {
2101
2102 public:
2103
2104     size_t operator()(const QSet<TQ> &S, const char *istr=0) const {
2105
2106         unsigned i, j=1, r=S.qdir.len, nq=S.qs.len,
2107             l=1+(1+nq+r)/sizeof(long),
2108             lb=1*sizeof(long);
2109         unsigned long h=5381U, x[l];
2110         char *s=(char*)x;
2111
2112         const TQ *qs=S.qs.data;
2113
2114         if ((!r && nq) || (r && nq*r)) wblog(FL,
2115             "ERR %s() got QSet inconsistency (%d/%d)", FCT, nq, r);
2116         x[l-1]=0;
2117
2118         s[0]=S.t.type;
2119         s[1]=S.t.sub;
2120
2121         for (i=0; i<nq; ++i) {
2122             s[++j]=char(qs[i]);
2123         }; ++j;
2124
2125         if ((j+r)>lb) {
2126             wblog(FL, "ERR %s() string/id out of bounds (%d/(%d*%d))",
2127                 FCT, j, l, sizeof(long));
2128         }
2129
2130         memcpy(s+j, S.qdir.data, r);
2131
2132         if (istr && istr[0]) {
2133             unsigned n=strlen(istr); if (n>lb) {
2134                 wblog(FL, "ERR %s() istr '%s' out of bounds (%d/%d*%d)",
2135                     FCT, istr, n, l, sizeof(long));
2136             }
2137             for (j=lb-n, i=0; i<n; ++i) { s[j+i] ^= istr[i]; }
2138         }
2139
2140         for (i=0; i<l; ++i) { h ^= ((h<<6) + (h>>2)) + x[i]; }
2141
2142         return h;
2143     };
2144 };
2145
2146 template <class TD>

```

```

2147 class cdata : public wbsparray<TD> {
2148
2149 public:
2150
2151     cdata& init(SPIDX_T l=0) {
2152         wbsparray<TD>::init(l);
2153         return *this;
2154     };
2155
2156     cdata& init(
2157         SPIDX_T D1, SPIDX_T D2, SPIDX_T D,
2158         unsigned M=1
2159     ){
2160         if (M==1)
2161             wbsparray<TD>::init(D1,D2,D);
2162         else wbsparray<TD>::init(D1,D2,D,M);
2163         return *this;
2164     };
2165
2166     cdata& init(const wbvector<SPIDX_T> &S) {
2167         wbsparray<TD>::init(S); return *this;
2168     };
2169
2170     cdata& init(const cdata &C) {
2171         wbsparray<TD>::init(C);
2172         return *this;
2173     };
2174
2175     template <class TQ>
2176     cdata& init(const CRef<TQ> &R, char full=1);
2177
2178     template <class T2>
2179     cdata& init(wbsparray<T2> &C) {
2180         wbsparray<TD>::init(C);
2181         return *this;
2182     };
2183
2184     cdata& init(
2185         const char *F, int L, const mxArray *a, unsigned k,
2186         const QType &q, unsigned r=-1
2187     );
2188
2189     cdata& operator=(const cdata &C) {
2190         wbsparray<TD>::init(C);
2191         return *this;
2192     };
2193
2194     cdata& initScalar(double x=1, unsigned r=-1) {
2195         wbsparray<TD>::initScalar(x,r);
2196         return *this;
2197     };
2198
2199     cdata& initIdentity(const QType &q, SPIDX_T d, TD dval=1);
2200     cdata& initIdentity(const wbvector<SPIDX_T> &S);
2201
2202     bool isScalar() const {
2203         return wbsparray<TD>::isScalar(FL,'z');
2204     };
2205
2206     unsigned len() const { return this->SIZE.len; };
2207
2208     unsigned getOM(
2209         const char *F=NULL, int L=0, const char *istr=NULL) const;
2210
2211     cdata& Conj() { return *this; };
2212
2213     cdata& permute(cdata &B, const wbperm &P0, char iflag=0) const;
2214     cdata& Permute(const wbperm &P0, char iflag=0) {
2215         cdata X; this->save2(X);
2216         return X.permute(*this,P0,iflag);
2217     };
2218
2219     cdata& Kron(const cdata &B){
2220         wbsparray<TD>::Kron(B,'N','N','k');
2221         return *this;
2222     };
2223
2224     TD contract(
2225         const char *F, int L, const ctrIdx &ica,
2226         const cdata &B, const ctrIdx &icb, cdata &C,
2227         cdata *Cx=NULL, wbperm *P=NULL, char normalize=1
2228     ) const;
2229
2230     double SkipTiny(const char *F=NULL, int L=0);
2231
2232     TD NormSignC(
2233         const char *F=NULL, int L=0, unsigned m=0,

```

```

2234         TD eps=CG_EPS1,
2235         TD eps2=CG_EPS2
2236     );
2237
2238     void info() const {
2239         printf("\n data: %s\n",this->sizeStr().data);
2240     };
2241
2242     protected:
2243     private:
2244 };
2245
2246 template <class TQ, class TD>
2247 class CData : public QSet<TQ> {
2248
2249     public:
2250
2251         CData() : cstat(CGD_DEFAULT) { init(); };
2252
2253         CData(const CData &C)
2254             : QSet<TQ>((QSet<TQ>&)C), cgd(C.cgd), cstat(C.cstat) {};
2255
2256         CData(const QSet<TQ> &Q) : QSet<TQ>(Q) {};
2257
2258         CData(const CRef<TQ> &R) { init(R); }
2259
2260     void init() {
2261         this->t.init(); this->qs.init(); this->qdir.init();
2262         cgd.init(); cstat.init();
2263     };
2264
2265     CData& init(const CRef<TQ> &S);
2266
2267     CData& initAbelian(const QSet<TQ> &Q) {
2268         this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
2269         cstat.init(CGD_ABELIAN); cgd.init();
2270         return *this;
2271     };
2272
2273     CData& initScalar(const QSet<TQ> &Q) {
2274         if (!Q.t.isAbelian() && Q.qs.norm2()) {
2275             wblog(FL,"ERR %s() got QSet %s",FCT,STR(Q)); }
2276         this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
2277         cstat.init(CGD_EXPLICIT); cgd.initScalar(1,Q.qdir.len);
2278         return *this;
2279     };
2280
2281     CData& init3(const QType &t_,
2282                 SPIDX_T D1, SPIDX_T D2, SPIDX_T D,
2283                 unsigned M=1
2284     ){
2285         QSet<TQ>::init3(t_);
2286         cgd.init(D1,D2,D, M); cstat=CGD_DEFAULT;
2287         return *this;
2288     };
2289
2290     CData& init3(const QType &t_,
2291                 const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J,
2292                 unsigned M=1
2293     ){
2294         SPIDX_T D1,D2,D;
2295         QSet<TQ>::init3(t_,J1,J2,J);
2296         cstat.init(CGD_DEFAULT);
2297
2298         D1=t_.qdim(J1.data);
2299         D2=t_.qdim(J2.data);
2300         D =t_.qdim(J .data); cgd.init (D1,D2,D,M);
2301
2302         return *this;
2303     };
2304
2305     CData& init3_abelian(const QType &t_,
2306                        const TQ &J1, const TQ &J2, const TQ &J
2307     ){
2308         QSet<TQ>::init3(t_,J1,J2,J);
2309         cgd.init();
2310         cstat.init(CGD_ABELIAN);
2311         return *this;
2312     };
2313
2314     template <class T2>
2315     CData& init(const CData<TQ,T2> &B) {
2316         this->t=B.t; this->qs.init(B.qs); this->qdir=B.qdir;
2317         cgd.wbsspararray<TD>::init(B.cgd); cstat=B.cstat;
2318         return *this;
2319     };
2320

```

```

2321 int RefInit(
2322     const char *F, int L, const CData<TQ,TD> &B,
2323     char lflag=0, char bare=0);
2324
2325 CData& RefInit_auxtr(
2326     const char *F, int L,
2327     const wvector<SPIDX_T> &S, const wvector<RTD> &cgt
2328 );
2329
2330 int Reduce2Ref(const char *F=0, int L=0, char force=0);
2331 int LoadRef(const char *F=0, int L=0, char force=1);
2332
2333 int Load_CRef(
2334     const char *F, int L, CData &Cr, const char *cgd_file) const;
2335
2336 CData& initX3(
2337     const char *F, int L, const QSet<TQ> &Q,
2338     wvector<char> &cflags, wperm &P,
2339     unsigned loadRC=0, unsigned char flag=0);
2340
2341 int init_mxCRef(
2342     const char *F, int L, const mxArray *S, unsigned k
2343 );
2344
2345 TQ* qptr(unsigned k) const {
2346     unsigned n=this->t.qlen();
2347     if (this->qs.len%k || k>this->qs.len/n) wblog(FL,
2348         "ERR %s() index out of bounds (%d/%d <> %d)",
2349         FCT, this->qs.len, n, k);
2350     return (this->qs.data+k*n);
2351 };
2352
2353 CData& operator=(const CData &B) {
2354     this->t=B.t; this->qs=B.qs; this->qdir=B.qdir;
2355     cgd=B.cgd; cstat=B.cstat;
2356     return *this;
2357 };
2358
2359 CData& operator=(const QSet<TQ> &B) {
2360     this->t=B.t; this->qs=B.qs; this->qdir=B.qdir;
2361     return *this;
2362 };
2363
2364 CData& save2(CData &B);
2365
2366 bool olderThan(const CData &B) const {
2367     if ((QSet<TQ>&)*this!=(QSet<TQ>&B) wblog(FL,
2368         "ERR %s() got incompatible CData\n %s\n<> %s",
2369         FCT, STR_(this), STR(B)
2370     );
2371     checkSameStat(FL,B);
2372     return cstat.olderThan(B.cstat);
2373 };
2374
2375 bool operator!=(const CData &B) const { return !((*this)==B); };
2376 bool operator==(const CData &B) const {
2377     return (
2378         this->t==B.t && this->qs==B.qs && this->qdir==B.qdir &&
2379         cgd==B.cgd && cstat.sameAs(B.cstat,0)
2380     );
2381 };
2382
2383 int gotDiff(const CData &B, char lflag=0) const;
2384 bool sameAs(const CData &B, char lflag=0) const {
2385     return !gotDiff(B,lflag);
2386 };
2387
2388 bool operator!=(const QSet<TQ> &B) const {
2389     return (this->QSet<TQ>::operator!=(B));
2390 };
2391 bool operator==(const QSet<TQ> &B) const {
2392     return (this->QSet<TQ>::operator==(B));
2393 };
2394
2395 bool operator!=(const cgdStatus &b) const { return (cstat!=b); };
2396 bool operator==(const cgdStatus &b) const { return (cstat==b); };
2397
2398 bool isEmpty(char lflag=0) const {
2399     return (QSet<TQ>::isEmpty(lflag) && cgd.isEmpty());
2400 };
2401
2402 bool isAbelian(const char *F=NULL, int L=0) const {
2403     if (this->t.isAbelian()) {
2404         if (cstat.t==CGD_ABELIAN) {
2405             if (cgd.D.len || cgd.SIZE.len) wblog(F_L,
2406                 "ERR %s() invalid abelian CData\n%s", FCT, STR_(this));
2407             return 1;

```

```

2408         }
2409         if (!isScalar() || (cgd.D.len && (cgd.D.len>1 || cgd.D[0]!=1)))
2410             wblog(F_L, "ERR %s() invalid abelian CData\n%s", STR_(this));
2411         return 1;
2412     }
2413     return 0;
2414 };
2415
2416 bool isScalar(char dflag=0) const;
2417
2418 TD getScalar(const char *F=NULL, int L=0) const {
2419     if (!isScalar()) wblog(F_L,
2420         "ERR %s() got non-scalar CData\n%s", FCT, STR_(this));
2421     return (cgd.D.len ? cgd.D.data[0] : TD(1));
2422 };
2423
2424 unsigned rank(const char *F=NULL, int L=0) const;
2425
2426 unsigned rankS() const { return cgd.rank(); };
2427
2428 bool isrank(unsigned r, unsigned *r_=NULL) const {
2429     unsigned l=rank(F_L); if (r_) { (*r_)=l; }
2430     return (l==r);
2431 };
2432
2433 template <class T>
2434 wbvector<T>& getSize(
2435     wbvector<T> &S, char bare=0, const wbperm *cgp=NULL) const;
2436
2437 SPIDX_T dim() const;
2438
2439 bool isRefInit(char check=1) const {
2440     if (cstat!=CGD_REF_INIT) { return 0; }
2441     if (check) {
2442         const unsigned r=cgd.SIZE.len, m=cgd.D.len;
2443         if (!r || (m>1 && m!=cgd.SIZE[r-1])) wblog(F_L,
2444             "ERR %s() invalid CData size ref (%s @ D.len=%d)",
2445             FCT, SSTR(cgd), cgd.D.len
2446         );
2447     }
2448     return 1;
2449 };
2450
2451 bool checkValidStat(const char *F=0, int L=0) const {
2452     if (this->t.isAbelian()) {
2453         if (cstat.hasID('l') || cstat.t!=CGD_ABELIAN) {
2454             if (F) wblog(F_L, "ERR %s() "
2455                 "abelian\n%s\n%s", FCT, STR_(this), STR2(cstat, 'V'));
2456             return 0;
2457         }
2458     }
2459     else if (cstat.inValid() || cstat.t<=CGD_ABELIAN) {
2460         if (F) wblog(F_L, "ERR %s() "
2461             "non-abelian\n%s\n%s", FCT, STR_(this), STR2(cstat, 'V'));
2462         return 0;
2463     }
2464     return 1;
2465 };
2466
2467 int checkSameStat(
2468     const char *F, int L, const CData &B, char lflag=1) const {
2469     int e=0;
2470     if (!lflag) { if (cstat.t!=B.cstat.t) e=1; }
2471     if (cstat.ID!=B.cstat.ID || cstat.ctime!=B.cstat.ctime) e|=2;
2472
2473     if (e && F) {
2474         wblog(F_L, "WRN %s() cstat mismatch (e=%d)", FCT, e);
2475         wblog(F_L, " > %s\n > %s\n... having\n > %s\n > %s",
2476             STR_(this), STR(B), STR2(cstat, 'V'), STR2(B.cstat, 'V'));
2477         if (e>1) wblog(F_L, "ERR %s() e=%d", FCT, e);
2478     }
2479     return e;
2480 };
2481
2482 bool sameType(const CData &S, const char *F=NULL, int L=0) const;
2483
2484 int cmpOM(const CData &B) const;
2485
2486 template <class DB>
2487 bool sameSizeR(
2488     const CData<TQ,DB> &B,
2489     unsigned *r=NULL, const char *F=NULL, int L=0) const;
2490
2491 template <class DB>
2492 bool sameSizeR(
2493     const cdata<DB> &B,

```

```

2495     unsigned *r=NULL, const char *F=NULL, int L=0) const;
2496
2497     template <class T2>
2498     int sameSizeR(const char *F, int L, const wbvector<T2> &S) const;
2499
2500     bool isSymmmetric() const;
2501
2502     unsigned checkOM(const char *F=NULL, int L=0) const {
2503         if (this->qdir.len) {
2504             unsigned r=this->rank(F_L);
2505             if (this->t.permitsOM(r)) {
2506                 if (this->cgd.SIZE.len>r) {
2507                     SPIDX_T i=this->cgd.SIZE[r]; if (!i)
2508                         wblog(FL, "ERR %s() got OM=%d !?", FCT, i);
2509                     return i;
2510                 }
2511                 return 1;
2512             }
2513             else if (this->cgd.SIZE.len>r) wblog(FL,
2514                 "ERR %s() CData with invalid OM\n%s", FCT, STR_(this));
2515         }
2516         return 0;
2517     };
2518
2519     unsigned gotOM(const char *F=NULL, int L=0) const {
2520         if (this->qdir.len) {
2521             unsigned l=cgd.SIZE.len, r=this->rank(F_L);
2522             if (l==r+1) {
2523                 SPIDX_T i=cgd.SIZE[r];
2524                 if (!i || (i>1 && !this->t.permitsOM(r))) wblog(FL,
2525                     "ERR %s() invalid OM setting (i=%d,r=%d) !?\n%s",
2526                     FCT, i, r, STR_(this));
2527                 return i;
2528             }
2529             else if (l && l!=r) wblog(FL,
2530                 "ERR %s() invalid OM setting\n%s", FCT, STR_(this)
2531             );
2532         }
2533         return 0;
2534     };
2535
2536     unsigned getOM(const char *F=NULL, int L=0) const {
2537         if (cgd.SIZE.len) {
2538             unsigned m=gotOM(F, L); return (m ? m : 1);
2539         }
2540
2541         if (this->qdir.len==2 && cgd.isDiag()) { return 1; }
2542         if (cgd.isEmpty() && cstat==CGD_ABELIAN) { return 1; }
2543
2544         if (!isEmpty()) {
2545             this->rank(F_L);
2546             cgd.wbarray<TD>::checkSize(F_LF);
2547         }
2548         return 0;
2549     };
2550
2551     CData& initOM(const char *F, int L);
2552
2553     int gotFullOM(const char *F=NULL, int L=0) const {
2554         unsigned r=this->qdir.len, m=getOM(F_L);
2555
2556         if (isEmpty()) return -1;
2557
2558         if (isAbelian(F_L)) return 1;
2559         if (r<=2 && m!=1) wblog(FL,
2560             "ERR %s() %s got om=%d !?", FCT, STR_(this), m);
2561
2562         if (r<2) return -2;
2563         if (r>3) return -4;
2564
2565         if (r==2) {
2566             if (cstat.t!=CGD_IDENTITY && cstat.t!=CGD_1JSY_ST3 &&
2567                 cstat.t!=CGD_1JSY_GEN)
2568                 wblog(FL, "WRN %s() got %s !?", FCT, STR_(this));
2569             return 2;
2570         }
2571         else {
2572             if (cstat.t==CGD_STD3 || cstat.t==CGD_STD3_X
2573                 || this->t.sub<2) return +3;
2574             if (cstat.t!=CGD_FROM_CTR && cstat.t!=CGD_FROM_DEC &&
2575                 cstat.t!=CGD_GEN_X) wblog(FL,
2576                 "WRN %s() got %s !?", FCT, STR_(this));
2577             return -3;
2578         }
2579     };
2580
2581     };

```



```

2582
2583 int completeOM_DegQ(
2584     const char *F, int L, const wbperm &p,
2585     unsigned level=0
2586 );
2587
2588 CData& Project(
2589     const char *F, int L, cdata<TD> X,
2590     wbvector<TD> &cgw
2591 );
2592
2593 double normDiff(
2594     const char *F, int L, const CData &S,
2595     double eps=1E14
2596 ) const;
2597
2598 double norm2(unsigned k=-1) const;
2599
2600 wbvector<RTD>& trace(
2601     const char *F, int L, wbvector<RTD> &cgt) const;
2602
2603 wbvector<RTD> trace(const char *F=NULL, int L=0) const {
2604     wbvector<RTD> cgt; return trace(F,L,cgt);
2605 };
2606
2607 CData& trace(const char *F, int L,
2608     ctrIdx &i1, ctrIdx &i2, CData<TQ,TD> &X) const;
2609
2610 CData& AddMultiplicity(const char *F, int L, const cdata<TD> &c);
2611
2612 CData& AddMultiplicity(const char *F, int L, const CData &C) {
2613     if (this->t!=C.t || this->qs!=C.qs) wblog(F_L,
2614         "ERR %s() incompatible CData\n[%s] <> [%s] (%s, %s)",
2615         FCT, STR_(this), STR(C), sizeStr().data, C.sizeStr().data
2616     );
2617     return AddMultiplicity(F_L,C.cgd);
2618 };
2619
2620 cdata<TD>& getMultiplicity(unsigned im, cdata<TD> &c) const;
2621
2622 bool isBasicCG(const char *F=NULL, int L=0) const;
2623
2624 SPIDX_T getBasicCGSize(unsigned *m=NULL) const;
2625
2626 wbsparray<TD>&
2627 getBasicCG(unsigned k, wbsparray<TD> &a) const;
2628
2629 int getCG_set(
2630     const char *F, int L, wbIndex &Idx, wbsparray<TD> &a) const;
2631
2632 CData& reducetolJSymbol(const char *F, int L, CData<TQ,TD> &C) const;
2633
2634 int checkConsistency(const char *F=NULL, int L=0) const;
2635 int checkNormSign(const char *F=NULL, int L=0, char xflag=0) const;
2636
2637 int checkQ(const char *F, int L,
2638     const QSet<TQ> &Q, const char* istr=NULL) const;
2639
2640 bool checkAdditivityZ(const char *F, int L,
2641     const wbMatrix<double> &Z1,
2642     const wbMatrix<double> &Z2, const wbMatrix<double> &Z,
2643     TD eps=CG_EPS2
2644 ) const;
2645
2646 double SkipTiny(const char *F=NULL, int L=0);
2647
2648 CData& Permute(const wbperm &P, char iflag=0) {
2649     CData<TQ,TD> X; this->save2(X);
2650     return X.permute(*this,P,iflag);
2651 };
2652
2653 CData& permute(CData &B, const wbperm &P, char iflag=0) const {
2654     QSet<TQ>::permute((QSet<TQ>&)B,P,iflag);
2655
2656     cgd.permute(B.cgd,P,iflag);
2657     B.cstat=cstat;
2658
2659     return B;
2660 };
2661
2662 CData& Conj() { QSet<TQ>::Conj(); return *this; }
2663 CData& Conj(unsigned i);
2664
2665 size_t memSize() const {
2666     return (
2667         QSet<TQ>::memSize() + cgd.memSize() + sizeof(cstat)
2668     );

```

```

2669     };
2670
2671     void info(const char *istr=NULL, const char *F=NULL, int L=0) const;
2672
2673     wbststring dStr() const { return this->qdir.toStr(); };
2674     wbststring qStr() const { return this->t.toStr(); };
2675
2676     wbststring toStr(char vflag=0) const;
2677     wbststring sizeStr(char xflag=0) const;
2678
2679     mxArray* toMx() const;
2680
2681     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
2682     void add2MxStruct(mxAarray *S, unsigned i, char tst=0) const;
2683
2684     mxArray* mxCreateCell(unsigned m, unsigned n) const {
2685         wblog(FL,"ERR %s(%d,%d)",FCT,m,n); return 0; };
2686     void add2MxCell(mxAarray *S, unsigned i, char tst=0) const {
2687         wblog(FL,"ERR %s(%lX,%d,%d)",FCT,S,i,tst); };
2688
2689     void put(const char *vname, const char *ws="caller"
2690 ) const { put(0,0,vname,ws); };
2691
2692     void put(const char *F, int L,
2693         const char *vname, const char *ws="caller"
2694 ) const {
2695         mxArray *a=toMx();
2696         int i=mexPutVariable(ws,vname,a);
2697
2698         if (i) wblog(F_L,
2699             "ERR failed to write variable '%s' (%d)",vname,i);
2700         if (F) wblog(F_L,"I/O putting '%s' to %s",vname,ws);
2701         mxDestroyArray(a);
2702     };
2703
2704     #define UBUF_ID 2
2705     #define UBUF_REF 2
2706
2707     void setUser_BUF(unsigned char c=1) { cstat.user[UBUF_ID] = c; };
2708
2709     void setUser_BUF(unsigned char c, unsigned l) { unsigned k=UBUF_ID;
2710         if (l>7 || (!l && c==1)) wblog(FL,
2711             "WRN %s() got u[%d]=%d (%d/%d)",FCT,k,cstat.user[k],l,c);
2712
2713         if (c) { cstat.user[k] |= (c<l); }
2714         else { c=(l<l); cstat.user[k] &= ~c; };
2715     };
2716
2717     void setUser_BUF_ref(unsigned char c=1) {
2718         if (c) { c=(1<UBUF_REF); cstat.user[UBUF_ID] |= c; }
2719         else {
2720             c=(1<UBUF_REF)-1; cstat.user[UBUF_ID] &= c;
2721         };
2722     };
2723
2724     bool gotuser_BUF() const { return cstat.user[UBUF_ID]; };
2725
2726     bool gotuser_BUF_ref() const { unsigned char c=(1<UBUF_REF);
2727         return (cstat.user[UBUF_ID] & c);
2728     };
2729
2730     cdata<TD> cgd;
2731
2732     cgdStatus cstat;
2733
2734 private:
2735 };
2736
2737 template <class TQ>
2738 class CRef {
2739
2740 public:
2741
2742     CRef() : cgb(NULL), conj(0) {};
2743
2744     CRef(const CRef &S) { *this=S; };
2745
2746     CRef(const CDATA_TQ &C, unsigned i=0, unsigned m=-1)
2747         : CRef() { initBase(C,i,m); };
2748
2749     CRef& operator=(const CRef &S) {
2750         cgb=S.cgb; rtype=S.rtype;
2751         cgw=S.cgw; cgp=S.cgp; conj=S.conj;
2752         return *this;
2753     };
2754
2755     CRef& init(CGR_TYPE rt_=CGR_DEFAULT) {

```

```

2756     cgb=NULL; cgw.init(); cgp.init(); conj=0; rtype=rt_;
2757     return *this;
2758 };
2759
2760 CRef& init(const CRef &S) { *this=S; return *this; };
2761
2762 CRef& init(
2763     const char *F, int L, const CRef &S, unsigned i, const double &x
2764 ){
2765     *this=S; if (i>=cgw.len) {
2766         unsigned m=getOM(FL); if (i>=m) wblog(FL,
2767             "ERR %s() index out of bounds (OM %d/%d)",FCT,i,m);
2768         cgw.init(m);
2769     }
2770     else cgw.set(0);
2771
2772     cgw[i]=x; return *this;
2773 };
2774
2775 CRef& init(
2776     const char *F, int L, const mxArray *a, unsigned k,
2777     char refC=0, const QSet<TQ> *Q=NULL,
2778     char xflag=0
2779 );
2780
2781 CRef<TQ>& initIdentityR(const char *F, int L,
2782     const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0);
2783
2784 CRef<TQ>& initIdentitylJ(const char *F, int L,
2785     const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0);
2786
2787 CRef<TQ>& Reduce2Identity(char xflag=0);
2788
2789 wbvector<RTD>& trace(const char *F, int L, wbvector<RTD> &cgt) const;
2790
2791 #ifndef QS_SKIP_MPFR
2792 wbvector<double>& trace(const char *F, int L, wbvector<double> &cgt) const {
2793     wbvector<RTD> cgt_; trace(F,L,cgt_);
2794     return cgt_.initT(cgt_);
2795 };
2796 #endif
2797
2798 wbvector<double> trace(const char *F, int L) const {
2799     wbvector<double> cgt;
2800     return trace(F,L,cgt);
2801 };
2802
2803 double trace() const;
2804
2805 double trace(const char *F, int L,
2806     const ctrIdx &i1, const ctrIdx &i2, CRef *Rt=NULL) const;
2807
2808 CRef& initAbelian(){
2809     cgb=NULL; cgw.init(); rtype=CGR_ABELIAN;
2810     cgp.init(); conj=0;
2811     return *this;
2812 };
2813
2814 CRef& initCtrScalar(double x=1){
2815     cgb=NULL; cgw.init(1,&x); rtype=CGR_CTR_SCALAR;
2816     cgp.init(); conj=0;
2817     return *this;
2818 };
2819
2820 CRef& initBase(const CDATA_TQ* cgr_, unsigned i=0, unsigned m=-1);
2821
2822 CRef& initBase(const CDATA_TQ &C, unsigned i=0, unsigned m=-1);
2823
2824 int LoadRef(const char *F=0, int L=0, char force=1) const {
2825     if (!cgb) {
2826         wblog(FL,"WRN %s() got cgb=null !?\n%s",FCT,STR_(this));
2827         return 0;
2828     }
2829     return ((CDATA_TQ*)cgb)->LoadRef(F,L,force);
2830 };
2831
2832 int Reduce2Ref(const char *F=0, int L=0, char force=0) const {
2833     if (isRefInit(F,L,force)) { return 0; }
2834     if (!cgb) wblog(FL,"ERR %s() got cgb=null !?\n%s",FCT,STR_(this));
2835     return ((CDATA_TQ*)cgb)->Reduce2Ref(F,L,force);
2836 };
2837
2838 bool isEmpty() const {
2839     if (!cgb || cgb->isEmpty()) {
2840         if (cgw.len>1 || cgp.len) wblog(FL,
2841             "ERR %s() invalid abelian cgrep (%d)",FCT,STR_(this));
2842         return (rtype==CGR_DEFAULT && !cgw.len);
2843     }
2844 }

```

```

2843     }
2844     return 0;
2845 };
2846
2847 bool isSymmetric(const char *F=0, int L=0) const;
2848
2849 bool isRefInit(const char *F=0, int L=0, char check=1) const {
2850     if (cgb && cgb->isRefInit(check)) {
2851         if (rtype!=CGR_DEFAULT) wblog(F,L,
2852             "ERR %s() REF_INIT mismatch\n%s",FCT,STR_(this));
2853         return 1;
2854     }
2855     return 0;
2856 };
2857
2858 bool isAbelian(const char *F=NULL, int L=0) const;
2859
2860 bool isScalar(char dflag=0) const;
2861
2862 bool isScalarCtr() const {
2863     if (cgb || rtype!=CGR_CTR_SCALAR || cgw.len!=1) return 0;
2864     return 1;
2865 };
2866
2867 int check_cgw(const char *F=0, int L=0) const {
2868     if (!cgw.isFinite()) {
2869         if (F) wblog(F,L,"ERR %s() got invalid cgw",FCT);
2870         return 1;
2871     }
2872     return 0;
2873 };
2874
2875 int checkQ(
2876     const char *F, int L, const TQ *q, unsigned r, unsigned stride,
2877     const QType &t, const IndexLabels &it) const;
2878
2879 unsigned numel() const;
2880
2881 template <class T>
2882 wbvector<T>& getSize(wbvector<T> &S, char bare=0) const;
2883
2884 unsigned Size(unsigned k, unsigned r=-1) const;
2885
2886 unsigned qdim(unsigned k, const QType *qt=NULL) const;
2887
2888 int checkAbelian(const char *F=NULL, int L=0) const;
2889
2890 bool isDiagCSC(RTD eps=1E-14) const;
2891 bool isIdentityCG(double *nrm=NULL, double eps=1E-14) const;
2892
2893 char sameUptoFac(const CRef &B, double *fac=NULL, double eps=1E-12) const;
2894 char sameAs(const CRef &B, double eps=1E-12) const;
2895
2896 bool sameQSet(const QSet<TQ> &Q) const;
2897 bool sameQSet(const CRef &B) const;
2898 bool sameQDir(const char *F, int L, const QDir &qd) const;
2899
2900 bool sameQDir(const char *F, int L, const CRef &B) const {
2901     QDir qdb; B.get_qdir(qdb);
2902     return sameQDir(F,L,qdb);
2903 };
2904
2905 bool sameQDir(const IndexLabels &b) const;
2906
2907 int gotSameCData(
2908     const char *F, int L, const CRef &B, char c2flag=0
2909 ) const;
2910
2911 double NormStd(const char *F, int L, unsigned r=-1);
2912
2913 double normExt(const char *F=NULL, int L=0) const;
2914
2915 int SortDegQ(const char *F=NULL, int L=0, QSet<TQ> *QS=NULL);
2916
2917 bool isSortedDegQ(QSet<TQ> *QS, wbperm &pxt, char &cxt) const;
2918 bool isSortedDegQ(QSet<TQ> *QS=NULL) const {
2919     wbperm pxt; char cxt=0;
2920     return isSortedDegQ(QS,pxt,cxt);
2921 };
2922
2923 char got3(const char *F, int L,
2924     const QType &q, const qset<TQ> &J12, const qset<TQ> &J3
2925 ) const;
2926
2927 bool operator==(const CRef &B) { return (sameAs(B)==0 ? 1 : 0); };
2928 bool operator!=(const CRef &B) { return (sameAs(B)==0 ? 0 : 1); };
2929

```

```

2930     RTD scalarProd(const char *F, int L, const CRef &B) const;
2931
2932     double normDiff2(const char *F, int L, const CRef &B) const;
2933     double normDiff (const char *F, int L, const CRef &B) const {
2934         return Sqrt(normDiff2(F,L,B)); };
2935
2936     RTD sumTimesEl(const char *F, int L, const CRef &B) const;
2937
2938     int check(const char *F, int L) const;
2939     int check(const char *F, int L,
2940         const QType &q, const QDir &qdir) const;
2941
2942     double norm2(char xflag=0) const;
2943
2944     double NormSignR(
2945         const char *F=NULL, int L=0,
2946         double eps =CG_SKIP_DEPS1,
2947         double eps2=CG_SKIP_DEPS2
2948     );
2949
2950     unsigned getOM(const char *F=NULL, int L=0) const;
2951     unsigned checkOM(const char *F=NULL, int L=0) const {
2952         return (cgb ? cgb->checkOM(F,L) : 0);
2953     };
2954
2955     unsigned rankS(char lflag=0) const;
2956     unsigned rank(const char *F=NULL, int L=0, char lflag=0) const;
2957
2958     CRef& Permute(const wBperm &P, char iflag=0, char isnew=0);
2959
2960     CRef& permute(
2961         const wBperm &P, CRef &B, char iflag=0, char isnew=0
2962     ) const { B=(*this); return B.Permute(P,iflag,isnew); };
2963
2964     CRef& Conj() {
2965         if (cgb) { conj=(conj ? 0 : 1); }
2966         return *this;
2967     };
2968
2969     int HConjOpScalar();
2970
2971     bool gotConj() const { return (conj!=0); };
2972
2973     bool gotPerm(const char *F=NULL, int L=0) const {
2974         if (cgp.len) {
2975             if (!cgb) wBlog(F_L,"ERR %s() "
2976                 "invalid perm (len=%d; cgb=NULL)",FCT, cgp.len);
2977             if (cgp.len!=cgb->qdir.len) wBlog(F_L,"ERR %s() "
2978                 "invalid perm (len=%d/%d)",FCT, cgp.len, cgb->qdir.len);
2979             return !cgp.isIdentityPerm(F_L,cgb->qdir.len);
2980         }
2981         return 0;
2982     };
2983
2984     bool anyTrafo(char pflag=1) const {
2985         if (conj || cgp.len) return 1;
2986         if (pflag && !cgp.isIdentityPerm()) return 1;
2987         return 0;
2988     };
2989
2990     bool anyTrafo(char &ip, char &ic, char pflag=1) const {
2991         if (pflag)
2992             { ip=(cgp.len && !cgp.isIdentityPerm()); }
2993         else { ip=(cgp.len ? 1 : 0); }
2994         ic=(conj!=0);
2995         return (ip || ic);
2996     };
2997
2998     bool gotnoTrafos() const {
2999         return !anyTrafo(); };
3000
3001     unsigned getP(unsigned k) const {
3002         if (cgp.len) {
3003             if (k>=cgp.len) wBlog(F_L,
3004                 "ERR %s() index out of bounds (%d/%d)",FCT,k,cgp.len);
3005             return cgp.data[k];
3006         }
3007         return k;
3008     };
3009
3010     wBperm& getP(wBperm &p, char iflag=0) const {
3011         if (!cgp.len)
3012             return p.init(rank(F_L));
3013         else return p.init(cgp,iflag);
3014     };
3015
3016     char stat() const { char i=0;

```

```

3017         if (rtype!=CGR_DEFAULT) i ^= 1;
3018         if (conj) i ^= 2;
3019         if (cgp.len && !cgp.isIdentityPerm()) i ^= 4;
3020         return i;
3021     };
3022
3023     bool affectsQDir() const;
3024
3025     qset<TQ>& adapt(qset<TQ> &q, char iflag=0) const {
3026         if (cgp.len) q.BlockPermute(cgp,iflag ? 0 : 'i');
3027         return q;
3028     };
3029
3030     QSet<TQ>& adapt(QSet<TQ> &Q, char iflag=0) const {
3031         if (cgp.len) Q.Permute(cgp,iflag ? 0 : 'i');
3032         if (conj) Q.qdir.Conj();
3033         return Q;
3034     };
3035
3036     ctrIdx& adapt(ctrIdx &I, char iflag=0) const {
3037         if (cgp.len && !cgp.isIdentityPerm()) {
3038             unsigned i=0; wvector<unsigned> x(I.len,I.data);
3039             if (I.len>cgp.len) wblog(FL,"ERR %s() "
3040                 "unexpected index length (%d/%d)",FCT,I.len,cgp.len);
3041             if (iflag==0) {
3042                 for (; i<I.len; ++i) { I[i]= cgp.el(x[i]); }
3043             }
3044             else {
3045                 wbperm cpi(cgp,'i');
3046                 for (; i<I.len; ++i) { I[i]= cpi.el(x[i]); }
3047             }
3048             if (conj) I.Conj();
3049             return I;
3050         };
3051
3052     QDir& get_qdir(QDir &qd) const {
3053         if (!cgb) { qd.init(); return qd; }
3054
3055         if (cgp.len)
3056             cgb->qdir.permute(qd,cgp);
3057         else qd=cgb->qdir;
3058
3059         if (conj) qd.Conj();
3060
3061         return qd;
3062     };
3063
3064     QDir get_qdir() const { QDir qd;
3065         return get_qdir(qd);
3066     };
3067
3068     double safeCpy(const char *F, int L, const CRef &B, CRef &C) const;
3069
3070     CRef& save2(CRef &S) {
3071         if (this!=&S) {
3072             S.cgb=cgb;
3073             cgw.save2(S.cgw); S.conj=conj; conj=0;
3074             cgp.save2(S.cgp); S.rtype=rtype; rtype=0;
3075         }
3076         return S;
3077     };
3078
3079     wstring qdir2Str(char vflag=0) const {
3080
3081         if (!cgb || !cgb->qdir.len) return "";
3082
3083         if (vflag<=0) return get_qdir().toStr();
3084         if (vflag==1 || vflag=='v') {
3085             unsigned l, n=2*(cgb->qdir.len)+5; char s[n];
3086             l=snprintf(s,n,"%s => %s",
3087                 cgb->qdir.toStr().data(), STR(get_qdir()));
3088             if (l>=n) wblog(FL,
3089                 "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
3090             return s;
3091         }
3092         else {
3093             unsigned l, n=2*(cgb->qdir.len)+5; char s[n];
3094             l=snprintf(s,n,"%s[%s]%s",
3095                 cgb->qdir.toStr().data(), (cgp+1).toStr(1,"").data(),
3096                 gotConj()?"*":"");
3097             if (l>=n) wblog(FL,
3098                 "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
3099             return s;
3100         }
3101     };
3102
3103     wstring toStr() const;

```

```

3104     wbsting sizeStr() const;
3105
3106     wbsting statStr(char vflag=0) const;
3107
3108     wbsting qStr() const { return cgb ? cgb->qStr() : ""; };
3109     wbsting QStr() const {
3110         return cgb ? cgb->QStrS(&cgp) : "";
3111     };
3112
3113     mxArray* toMx(char flag=0) const;
3114
3115     mxArray* toMX() const { return toMx('f'); };
3116
3117     mxArray* mxCreateStruct(unsigned m, unsigned n, char flag=0) const;
3118     void add2MxStruct(mxAarray *S, unsigned i, char flag=0) const;
3119
3120     const CDATA_TQ *cgb;
3121
3122     wbsting<double> cgw;
3123
3124     wbsting cgp;
3125
3126     char conj;
3127
3128     cgrType rtype;
3129 };
3130
3131 #define MAP32 std::map < qset<TQ>, wbsting< CRef<TQ> > >
3132 #define MAP31 std::map < qset<TQ>, MAP32 >
3133
3134 #define MP3_LDC 1
3135 #define MP3_LDR 2
3136
3137 #define TP3_ITER 3
3138 #define TP3_LDM 4
3139 #define TP3_LDR 8
3140 #define TP3_TST 16
3141 #define TP3_LOAD (TP3_LDM|TP3_LDR)
3142
3143 #define LB_LOAD 4
3144 #define LB_UPD 8
3145 #define LB_CALC 16
3146 #define LB_REF 32
3147
3148 #define LB_GEN (LB_LOAD | LB_CALC )
3149 #define LB_GET (LB_LOAD | LB_CALC | LB_UPD )
3150 #define LB_ANY (31<2)
3151
3152 #define QF_LOADC 128
3153
3154 #define CX3_SAVE 1
3155 #define CX3_FORCE 2
3156
3157 template <class TQ>
3158 class CStore {
3159 public:
3160
3161     CStore() {
3162         BUF.max_load_factor(0.2);
3163     };
3164
3165     void clear() {
3166         map3.clear();
3167         BUF.clear();
3168     };
3169
3170     typedef typename MAP32::iterator iMAP32;
3171     typedef typename MAP31::iterator iMAP31;
3172
3173     void init() {};
3174
3175     const CDATA_TQ& getIdentityC(const char *F, int L,
3176         const QType &t, const TQ *qs, unsigned dim=-1,
3177         unsigned loadRC=LB_GEN
3178     );
3179     const CDATA_TQ& getIdentity1J(
3180         const char *F, int L, CRef<TQ> &C,
3181         const QType &t, const TQ *qs, unsigned dim=-1,
3182         unsigned loadRC=LB_GEN
3183     );
3184
3185     void add_CData_abelian(
3186         const QType &q, const TQ &J1, const TQ &J2);
3187
3188     void add_CData(
3189         const QType &q, const qset<TQ> &J1, const qset<TQ> &J2);
3190

```

```

3191
3192 void add2BUF(const char *F, int L, const CData_TQ &S, char nflag=0);
3193
3194 unsigned add3(const QType &q, const mxArray* S, char loadC=0);
3195 unsigned add2BUF(const QType &q, const mxArray* S, unsigned k=-1);
3196
3197 CData_TQ& getBUF(
3198     const char *F, int L, const QSet<TQ> &Q, unsigned loadRC=0
3199 );
3200
3201 size_t reduceMemUsage(const char *F=NULL, int L=0, double mfac=-1);
3202
3203 void getQfinal(const char *F, int L, const QType &type,
3204     const qset<TQ> &J1, const qset<TQ> &J2, wbMatrix<TQ> &J,
3205     wbMatrix< const wbvector<CRef<TQ> >* > *ss=NULL,
3206     char loadC=0
3207 );
3208
3209 void getQfinal(const char *F, int L, const QVec &type,
3210     const qset<TQ> &q1, const qset<TQ> &q2, wbMatrix<TQ> &QQ,
3211     wbMatrix< const wbvector< CRef<TQ> >* > *SS=NULL,
3212     char loadC=0
3213 );
3214
3215 int getQfinal_zdim(
3216     const char *F, int L, const QVec &qvec,
3217     const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2, const wbMatrix<TQ> &Q,
3218     wbvector<wid_x_t> &s1, wbvector<wid_x_t> &s2, wbvector<wid_x_t> &s,
3219     wbvector<unsigned> *m=NULL,
3220     wbMatrix< const wbvector< CRef<TQ> >* > *S3=NULL
3221 );
3222
3223 void getQfinal(const char *F, int L, const QVec &type,
3224     const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2, QMap<TQ> &M,
3225     unsigned char cgflag=0
3226 );
3227
3228 void Info(const char *F=0, int L=0, char vflag=0) const;
3229
3230 int checkInit(const char *F, int L, const QType &t) const;
3231
3232 unsigned LoadStore(const char *F, int L, const char *file);
3233
3234 mxArray* toMx(const QType &q=QTYPE_UNKNOWN) const;
3235
3236 void put(const char *F, int L,
3237     const char *vname, const char *ws="caller"
3238 ) const {
3239     mxArray *a=toMx(); int i=mexPutVariable(ws,vname,a);
3240
3241     if (i) wblog(F_L,
3242         "ERR failed to write variable '%s' to %s (%d)",vname,i,ws);
3243     if (F) wblog(F_L,"I/O putting '%s' to %s",vname,ws);
3244     mxDestroyArray(a);
3245 };
3246
3247 void put(const char *vname, const char *ws="caller") const {
3248     put(0,0,vname,ws);
3249 };
3250
3251 int find_map3(const QType &q, const qset<TQ> &J1, const qset<TQ> &J2
3252 ) const {
3253     qset<TQ> J12(J1,J2);
3254     if (J1.len!=J2.len || J1.len!=q.qlen()) wblog(FL,
3255         "ERR %s() qset length inconsistency (%d/%d/%d)",
3256         FCT,J1.len,J2.len,q.qlen());
3257     return find_map3(q,J12);
3258 };
3259
3260 int find_map3(const QType &q, const qset<TQ> &J12) const {
3261     if (J12.len!=2*q.qlen()) {
3262         if (!q.isKnown()) wblog(FL,
3263             "ERR %s() got uninitialized symmetry",FCT); else
3264             wblog(FL,"ERR %s() "
3265                 "qset length inconsistency (%d/2*%d)",FCT,J12.len,q.qlen()
3266             );
3267     }
3268     auto it1 = map3.find(q);
3269     if (it1!=map3.end()) {
3270         auto it2 = it1->second.find(J12);
3271         if (it2!=it1->second.end()) { return it2->second.size(); }
3272     }
3273     return -1;
3274 };
3275
3276 int valid_mp3_data(const QType &t,
3277     const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J3,

```



```

3278     char rflag=0
3279 ) const;
3280
3281 CDATA_TQ* BUF_find(const QSet<TQ> &Q);
3282
3283 int Reduce2Ref(
3284     const char *F, int L, const QSet<TQ> &Q,
3285     char xflag=0
3286 );
3287
3288 map <QType,
3289     map <qset<TQ>,
3290         map <qset<TQ>,
3291             wvector< CRef<TQ> >
3292         >
3293     >
3294 > map3;
3295
3296 unordered_map <
3297     QSet<TQ>,
3298     CDATA_TQ,
3299     QHash<TQ>
3300 > BUF;
3301
3302 protected:
3303 private:
3304
3305 iMAP31 get_mp3_data(const char *F, int L,
3306     const QType &q, const qset<TQ> &J1, const qset<TQ> &J2,
3307     char force=0
3308 ){
3309     if (J1.len!=J2.len) wblog(FL,
3310         "ERR qset length inconsistency (%d/%d)",J1.len,J2.len);
3311
3312     iMAP31 iJ12;
3313     qset<TQ> J12(J1,J2);
3314
3315     auto itype = map3.find(q);
3316     if (itype==map3.end()) {
3317         checkInit(FL,q); itype = map3.find(q);
3318     }
3319
3320     if (itype!=map3.end()) {
3321         iJ12 = itype->second.find(J12);
3322         if (iJ12!=itype->second.end()) {
3323             if (!iJ12->second.size()) wblog(FL,
3324                 "ERR %s() empty map3 data !?",FCT);
3325             return iJ12;
3326         }
3327     }
3328
3329     if (!force) {
3330         add_CData(q,J1,J2);
3331         return get_mp3_data(F,L,q,J1,J2,'f');
3332     }
3333
3334     wblog(F,L,
3335         "ERR failed to find given target space\n[%s] x [%s] => [%s]",
3336         J1.toStr().data, STR(J2), STR(J12));
3337     return iJ12;
3338 };
3339
3340 void print_Coeff(const char *F, int L, const QType &type,
3341     const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J,
3342     double c
3343 ){
3344     wblog(F,L," * CG coeff. %-8s [%s, %s; %s] = %.6g",
3345         type.toStr(), STR(J1), STR(J2),
3346         J.toStr().data,c);
3347 }
3348 };
3349
3350 CStore<gTQ> gCS;
3351
3352 CData<gTQ,double> EMPTY_CGSTORE;
3353
3354 mxArray* map3_CreateStructMatrix(unsigned m, unsigned n);
3355
3356 template <class TQ>
3357 void map3_add2MxStruct(mxArray *S, unsigned k,
3358     const QType &q, const qset<TQ> &J12);
3359
3360 gTQ get_qtot_abelian(
3361     const QType &t, const gTQ *q0, const widx_t n,
3362     const widx_t *idx=NULL, const widx_t stride=1
3363 );
3364

```

```

3365 template <class TM>
3366 class cgc_contract_id : public wbvector<TM> {
3367
3368 public:
3369
3370     cgc_contract_id() { wbvector<TM>::init(); };
3371
3372     template <class TQ, class TD>
3373     cgc_contract_id(
3374         const CData<TQ,TD> &A, const ctrIdx &ica,
3375         const CData<TQ,TD> &B, const ctrIdx &icb
3376     );
3377
3378     template <class TQ, class TD>
3379     void extract(
3380         CData<TQ,TD> &A, ctrIdx &ica,
3381         CData<TQ,TD> &B, ctrIdx &icb,
3382         char xflag=1
3383     ) const;
3384
3385     size_t memSize() const {
3386         return ( sizeof(*this) + this->len*sizeof(TM) );
3387     };
3388
3389     void wblog_(const char* F=0, int L=0, const char *istr=0) const {
3390         #ifdef QS_USING_OMP
3391             Wb::ompGuard myLK(wblog_lock,1);
3392         #endif
3393
3394         CData<gTQ,RTD> a,b; ctrIdx ica,icb;
3395         this->extract(a,ica,b,icb,'l');
3396
3397         wblog(F_L,"TST cgc_contract_id() %s",istr ? istr : "");
3398         wblog(F_L,"b a: %-32s @ %s\n b: %-32s @ %s\n\n    %s\n    %s\n",
3399             STR(a), STR(ica), STR(b), STR(icb),
3400             STR2(a.cstat,'V'),STR2(b.cstat,'V')
3401         );
3402     };
3403
3404     wbstring toStr(char vflag=7) const {
3405         wbstring sout;
3406
3407         CData<gTQ,RTD> a,b; ctrIdx ica,icb;
3408         this->extract(a,ica,b,icb, vflag & 6 ? 'l' : 0);
3409
3410         if (vflag&4) {
3411             sout.init(256); snprintf(sout.data,sout.len,
3412                 "%-20s @%s #05x | %-20s @%s #05x",
3413                 STR2(a,1), STR(ica), a.cstat.ID,
3414                 STR2(b,1), STR(icb), b.cstat.ID
3415             );
3416         }
3417         else if (vflag&2) {
3418             sout.init(128); snprintf(sout.data,sout.len,
3419                 "%-20s @%s | %-20s @%s",
3420                 STR2(a,1), STR(ica), STR2(b,1), STR(icb)
3421             );
3422         }
3423         else {
3424             sout.init(64); snprintf(sout.data,sout.len,
3425                 "%s_%s * %s_%s",STR2(a,0), STR(ica), STR2(b,0), STR(icb));
3426         }
3427         return sout;
3428     };
3429
3430 protected:
3431 private:
3432
3433     template <class T1, class T2>
3434     unsigned set_val(const char *F, int L, T1* &s, const T2 &x) {
3435         s[0]=x;
3436         if (T2(s[0])!=x) wblog(F_L,"ERR got non-%s data entry (%g)",
3437             TSTR(T1), double(x)
3438         );
3439         s+=1; return 1;
3440     };
3441
3442     template <class T1, class T2>
3443     unsigned set_vec(const char *F, int L, T1* &s, const wbvector<T2> &x) {
3444         s[0]=x.len;
3445         if (unsigned(s[0])!=x.len) wblog(F_L,"ERR unexpected size "
3446             "(out of bounds %s @ %d)",TSTR(T1),x.len
3447         );
3448         set_range(F_L,++s,x.len,x.data);
3449         return (1+x.len);
3450     };
3451

```

```

3452     template <class T1, class T2>
3453     unsigned set_range(const char *F, int L, T1* &s, unsigned n, const T2 *x) {
3454         for (unsigned i=0; i<n; ++i) {
3455             s[i]=x[i];
3456             if (T2(s[i])!=x[i]) wblog(F_L,"ERR got non-%s data entry (%g)",
3457                 TSTR(T1), double(x[i])
3458             );
3459         }
3460         s+=n; return n;
3461     };
3462
3463     template <class TQ, class TD>
3464     unsigned set_CData(const char *F, int L,
3465         char* &s, unsigned n, const CData<TQ,TD> &A, char tflag=1
3466     ){
3467         char *s0=s;
3468         unsigned r=A.rank(), l=4+A.qs.len+r+(4+r)*sizeof(unsigned);
3469
3470         wvector<size_t> S;
3471
3472         if (l>n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
3473
3474         if (tflag) {
3475             set_val(FL,s, A.t.type);
3476             set_val(FL,s, A.t.sub );
3477         }
3478
3479         set_vec(FL,s, A.qs);
3480         set_vec(FL,s, A.qdir);
3481
3482         unsigned *u=(unsigned*)s;
3483
3484         set_vec(FL,u, A.getSize(S,'b'));
3485
3486         s+=(u-(unsigned*)s)*sizeof(unsigned); l=s-s0;
3487         if (l>n) wblog(F_L,
3488             "ERR %s() string out of bounds (%d/%d)",FCT,l,n
3489         );
3490
3491         return l;
3492     };
3493
3494     template <class T1, class T2>
3495     void get_range(
3496         const char *F, int L, const T1* &s, unsigned n, T2 *x) const {
3497
3498         for (unsigned i=0; i<n; ++i) {
3499             x[i]=s[i];
3500             if (T1(x[i])!=s[i]) wblog(F_L,"ERR got non-%s data entry (%g)",
3501                 TSTR(T1), double(x[i])
3502             );
3503         }
3504         s+=n;
3505     };
3506
3507     template <class T1, class T2>
3508     void get_vec(
3509         const char *F, int L, const T1* &s, wvector<T2> &x) const {
3510         if (!s || int(s[0])<0) wblog(FL,
3511             "ERR %s() got invalid vec.len=%d",FCT,s?s[0]:-1);
3512         x.init(s[0]); ++s;
3513         get_range(F,L,s,x.len,x.data);
3514     };
3515
3516     template <class T1, class T2>
3517     void get_val(
3518         const char *F, int L, const T1* &s, T2 &x) const {
3519         get_range(F,L,s,1,&x);
3520     };
3521
3522     template <class TQ, class TD>
3523     unsigned get_CData(const char *F, int L,
3524         const char* &s, CData<TQ,TD> &A, char tflag=1) const {
3525
3526         A.init();
3527         if (tflag) {
3528             A.t.initx(QTYPE_SET(s[0]),unsigned(s[1]));
3529             s+=2;
3530         }
3531
3532         get_vec(F_L,s, A.qs);
3533         get_vec(F_L,s, A.qdir);
3534
3535         const unsigned *u = (const unsigned*)s;
3536         get_vec(F_L,u, A.cgd.SIZE);
3537         A.cstat.init0(CGD_BSZ_INIT);
3538

```

```

3539         s+=(u-(unsigned*)s)*sizeof(unsigned);
3540
3541         return 0;
3542     };
3543
3544 };
3545
3546 template <class TM>
3547 class MHash {
3548
3549 public:
3550
3551     size_t operator()(const cgc_contract_id<TM> &S) const {
3552
3553         MTI h=5381U, *const x=S.data;
3554         for (unsigned i=0; i<S.len; ++i) {
3555             h ^= ((h<<6) + (h>>2)) + x[i];
3556             h ^= ((h<<6) + (h>>2)) + x[i];
3557         }
3558         return h;
3559     };
3560 };
3561
3562 template <class TQ, class TD>
3563 class x3map {
3564
3565 public:
3566
3567     x3map() : cgb(NULL), conj(0), zflag(0), rtype(CGR_DEFAULT)
3568     #ifndef QS_USING_MPFR
3569     , X3(x3)
3570     #endif
3571     {};
3572
3573     x3map(const char *F, int L,
3574           const CRef<TQ> &A_, const ctrIdx &ica,
3575           const CRef<TQ> &B_, const ctrIdx &icb, wbperm &cgp, char xflag=0
3576           ) : x3map() { initCtr(F,L,A_,ica,B_,icb, cgp, xflag); };
3577
3578     x3map& init() {
3579         a.init(); b.init(); c.init(); pab.init(); x3.init();
3580         #ifdef QS_USING_MPFR
3581             X3.init();
3582         #endif
3583         cgb=NULL; conj=0; zflag=0; rtype=CGR_DEFAULT;
3584         return *this;
3585     };
3586
3587     int initCtr(const char *F, int L,
3588               const CRef<TQ> &A_, const ctrIdx &ica_,
3589               const CRef<TQ> &B_, const ctrIdx &icb_, wbperm &cgp,
3590               char xflag=1
3591               );
3592
3593     x3map& init(
3594         const char *F, int L, const mxArray* S, unsigned k=0,
3595         ctrIdx *ica=NULL, ctrIdx *icb=NULL, char recalc=1
3596         );
3597
3598     bool isEmpty() const {
3599         return (
3600             a.isEmpty() && b.isEmpty() && c.isEmpty() &&
3601             !pab.len && x3.isEmpty()
3602         );
3603     };
3604
3605     void print(
3606         const char *F=NULL, int L=0, const char *istr="", char vflag=0
3607         ) const;
3608
3609     wbststring toStr(char vflag=0) const;
3610     wbststring x3Str(char vflag=0) const;
3611
3612     size_t memSize() const {
3613         return ( sizeof(cgb) + sizeof(char)
3614             + a.memSize() + b.memSize() + c.memSize()
3615             + sizeof(pab) + pab.len*sizeof(widx_t)
3616             + sizeof(x3) + x3.numel()*sizeof(TD)
3617             + sizeof(rtype)
3618         );
3619     };
3620
3621     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
3622
3623     void add2MxStruct(
3624         mxArray *S, unsigned l,
3625         const cgc_contract_id<MTI> *idc=NULL) const;

```

```

3626
3627     mxArray* toMx(const cgc_contract_id<MTI> *idc=NULL) const {
3628         mxArray *S=mxCreateStruct(1,1);
3629         add2MxStruct(S,0,idc); return S;
3630     };
3631
3632     CData<TQ,RTD> a, b, c;
3633
3634     const CDATA_TQ* cgb;
3635
3636     wbpem pab;
3637
3638     char conj;
3639
3640     int zflag;
3641
3642     cgrType rtype;
3643
3644     wbararray<TD> x3;
3645
3646 #ifdef QS_USING_MPFR
3647     wbararray<RTD> X3;
3648 #else
3649     wbararray<TD> &X3;
3650 #endif
3651
3652     protected:
3653     private:
3654 };
3655
3656 template <class TQ, class TD>
3657 class X3Map {
3658
3659     public:
3660
3661     X3Map() {
3662         XBUF.max_load_factor(0.2);
3663     };
3664
3665     void clear() {
3666         XBUF.clear();
3667     };
3668
3669     int init(const char *F, int L, const mxArray *S, unsigned k=0);
3670
3671     x3map<TQ,TD> & getXBUF(
3672         const char *F, int L,
3673         const cgc_contract_id<MTI> &idc
3674     );
3675
3676     int contains(const char *F, int L,
3677         const x3map<TQ,TD> &x, const cgc_contract_id<MTI> &idc);
3678
3679     int contract(const char *F, int L,
3680         const CRef<TQ> &A, const ctrIdx &ica,
3681         const CRef<TQ> &B, const ctrIdx &icb, CRef<TQ> &C,
3682         char tflag=0
3683     );
3684
3685     CRef<TQ> & contractDegQ(const char *F, int L,
3686         const CRef<TQ> &A, CRef<TQ> &B);
3687
3688     unsigned add(
3689         const char *F, int L, const cgc_contract_id<MTI> &idc,
3690         const mxArray * S, unsigned k
3691     );
3692
3693     mxArray* toMx() const;
3694
3695     void Info(const char *F=0, int L=0) const;
3696
3697     unordered_map <
3698         cgc_contract_id<MTI>,
3699         x3map<TQ,TD>,
3700         MHash<MTI>
3701     > XBUF;
3702
3703     protected:
3704     private:
3705 };
3706
3707     X3Map<gTQ,double> gXS;
3708
3709 template <class TQ>
3710 class QMap {
3711
3712     public:

```

```

3713
3714 void init() {
3715     Q1.init(); Q2.init(); Q.init();
3716     I1.init(); I2.init(); II.init(); D.init(); qvec.init();
3717 };
3718
3719 QMap<TQ>& init(const char *F, int L,
3720             const QVec &qv, const wbMatrix<TQ> &q1, const wbMatrix<TQ> &q2,
3721             const wbMatrix< wbMatrix<TQ> > &QQ,
3722             const wbMatrix< wbMatrix< const wbvector< CRef<TQ> >* > > *SS=NULL
3723 );
3724
3725 QMap<TQ>& init(const char *F, int L, const mxArray *a);
3726
3727 template<class TD>
3728 int getCGZlist(
3729     const char *F, int L,
3730     wbMatrix<TQ> &qc,
3731     wbvector<TD> &dc,
3732     wbvector<widxt_t> *IC=NULL,
3733     const wbvector<widxt_t>* Ix=NULL,
3734     const qset<TQ> *q3=NULL,
3735     wbindex *m3=NULL,
3736     const wbvector<TD>* cc=NULL,
3737     double eps1=1E-10,
3738     double eps2=1E-14
3739 ) const;
3740
3741 void Skip(const wbindex &I);
3742
3743 template<class TD>
3744 void getIdentityQ(const char *F, int L,
3745                 const wbvector<widxt_t> &Sa, const wbvector<widxt_t> &Sb,
3746                 QSpace<TQ,TD> &A, char vflag=1
3747 ) const;
3748
3749 bool isConsistent(
3750     const char *F=NULL, int L=0, const QVec *qv=NULL
3751 ) const;
3752
3753 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
3754 void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
3755
3756 mxArray* toMx() const {
3757     mxArray *S=mxCreateStruct(1,1);
3758     add2MxStruct(S,0); return S;
3759 };
3760
3761 void put(const char *vname, const char *ws="caller") const
3762 { put(0,0,vname,ws); }
3763
3764 void put(const char *F, int L,
3765         const char *vname, const char *ws="caller"
3766 ) const {
3767     mxArray *a=toMx(); int i=mexPutVariable(ws,vname,a);
3768
3769     if (i) wblog(F,L,
3770         "ERR failed to write variable '%s' (%d)",vname,i);
3771     if (F) wblog(F,L,"I/O putting '%s' to %s",vname,ws);
3772     mxDestroyArray(a);
3773 };
3774
3775 wbMatrix<TQ> Q1,Q2,Q;
3776 wbvector<widxt_t> I1,I2,D;
3777 wbMatrix<widxt_t> II;
3778
3779 wbMatrix< const wbvector< CRef<TQ> >* > cg3;
3780
3781 QVec qvec;
3782
3783 };
3784
3785 template <class TQ>
3786 bool QMap<TQ>::isConsistent(const char *F, int L, const QVec *qv) const {
3787
3788     unsigned d=qvec.Qlen();
3789
3790     if (Q1.dim2!=d || Q2.dim2!=d) {
3791         if (!F) return 0; else wblog(F,L,
3792             "ERR QMap: invalid qset record length (%d,%d/%d)",
3793             Q1.dim2, Q1.dim2, d);
3794     }
3795     if (qv && (!qvec.sameType(*qv) || d!=(qv->Qlen()))) {
3796         if (!F) return 0; else wblog(F,L,
3797             "ERR QMap: Q type mismatch: %s; %s (%d/%d)",
3798             qv->toStr().data, STR(qvec), qv->Qlen(), d);
3799     }

```

```

3800     return 1;
3801 };
3802
3803 template <class T>
3804 class pairPatternCG {
3805 public:
3806
3807     pairPatternCG& init(unsigned i0=0, unsigned j0=0, unsigned n=0) {
3808         i=i0; j=j0; k.init(n); fac.init(n);
3809         return *this;
3810     };
3811
3812     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
3813     void add2MxStruct(mxArray *S, unsigned i) const;
3814
3815     unsigned i,j;
3816     wvector<unsigned> k;
3817     wvector<T> fac;
3818
3819 protected:
3820 private:
3821 };
3822
3823 template <class TQ, class TD>
3824 class genRG_base{
3825 public:
3826
3827     genRG_base() : err(0) {};
3828
3829     bool isEmpty() const {
3830         return (!q && !Sp.len && !Sz.len && !J.len && Z.isEmpty());
3831     };
3832
3833     double normDiff(const char *F, int L,
3834                     const genRG_base& B,
3835                     TD eps=1E12
3836                 ) const;
3837
3838     void checkOrthoCommRel(
3839         const char *F, int L, const genRG_base &B) const;
3840
3841     genRG_base& Sort();
3842     double SkipTiny(const char *F=0, int L=0);
3843
3844     genRG_base& ApplyQFac(
3845         const wvector<double> &qfac, const wbararray<double> *JM=NULL);
3846
3847     genRG_base& operator=(const genRG_base& S) {
3848         q=S.q; J=S.J; Z=S.Z; Sp=S.Sp; Sz=S.Sz;
3849         err=S.err; istr=S.istr;
3850         return *this;
3851     };
3852
3853     genRG_base& init() {
3854         q=0; J.init(); Z.init(); Sp.init(); Sz.init();
3855         err=0; istr.init();
3856         return *this;
3857     };
3858
3859     SPIDX_T dim() const {
3860         if (!Sz.len || Sz.len!=Sp.len) wblog(FL,
3861             "ERR %s() got empty generators for %s [%s] !?",
3862             FCT,STR(q),STR(J),Sz.len,Sp.len);
3863         SPIDX_T d=Sz[0].dim();
3864         if (d!=Sp[0].dim()) wblog(FL,"ERR %s() got "
3865             "inconsistent Sp/z dimensions (%d/%d) !?",FCT,Sp[0].dim(),d);
3866         return d;
3867     };
3868
3869     void compareStdSU2(const char *F=NULL, int L=0) const;
3870
3871     genRG_base& save2(genRG_base& S) {
3872         S.q=q; J.save2(S.J); Z.save2(S.Z); q=0;
3873         Sp.save2(S.Sp); Sz.save2(S.Sz);
3874 #ifdef CG_CHECK_MW_PERM
3875         P0.save2(S.P0);
3876 #endif
3877         S.err=err; istr.save2(S.istr);
3878         return S;
3879     };
3880
3881     wstring info() const {
3882         unsigned n=128; char s[n]; char sep[2]=" ";
3883         if (!q.isAbelian() && J.wvector<TQ>::allIn(0,9)) sep[0]=0;
3884         unsigned l=snprintf(s,n,"%s [%s]",
3885             q.toStr().data, J.toStrf(" ",sep,q.qlen(),";").data);

```

```

3887         if (l>=n) wblog(FL,
3888             "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
3889         return s;
3890     };
3891
3892     mxArray* toMx() const {
3893         mxArray *a=mxCreateStruct(1,1);
3894         add2MxStruct(a,0);
3895         return a;
3896     };
3897
3898     size_t memSize() const {
3899         size_t mtot=sizeof(QType) + J.len*sizeof(TQ);
3900         unsigned i=0;
3901         for ( ; i<Sp.len; ++i) mtot+=Sp[i].memSize();
3902         for (i=0; i<Sz.len; ++i) mtot+=Sz[i].memSize();
3903         return mtot;
3904     };
3905
3906     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
3907     void add2MxStruct(mxArray *S, unsigned i) const;
3908
3909     QType q;
3910
3911     wbvector< wbSpararrayTD > Sp;
3912     wbvector< wbSpararrayTD > Sz;
3913
3914     qset<TQ> J;
3915
3916     wbMatrix<double> Z;
3917
3918     #ifdef CG_CHECK_MW_PERM
3919         wbperm P0;
3920     #endif
3921
3922     double err;
3923     wbstring istr;
3924
3925     protected:
3926     private:
3927 };
3928
3929 template <class TQ, class TD>
3930 class genRG_struct {
3931     public:
3932
3933     genRG_struct() {};
3934
3935     genRG_struct& checkInit(const char *F, int L, const QType &q0) {
3936         if (q0.isNonAbelian()) {
3937             unsigned n=RSet.size();
3938             if (n>0) {
3939                 if (!q0.type) wblog(FL,"ERR %s() got %d elements "
3940                     "in buf for '%s'",FCT,n,STR(q));
3941                 return *this;
3942             }
3943             q=q0; initBase(F,L);
3944         }
3945         return *this;
3946     };
3947
3948     genRG_struct& initBase(
3949         const char *F=NULL, int L=0, qset<TQ> *qs=NULL);
3950
3951     genRG_struct& initBase_SUN(const char *F, int L, qset<TQ> *qs=NULL);
3952     genRG_struct& initBase_SpN(const char *F, int L, qset<TQ> *qs=NULL);
3953     genRG_struct& initBase_SON(const char *F, int L, qset<TQ> *qs=NULL);
3954     genRG_struct& initBase_SEN(const char *F, int L, qset<TQ> *qs=NULL);
3955
3956     void initSU2(const qset<TQ> &J);
3957
3958     void initCommRel(const char *F, int L,
3959         const genRG_base<TQ,TD> &R, char vflag=0);
3960
3961     double checkCommRel(
3962         const char *F, int L, const genRG_base<TQ,TD> &R
3963     ) const;
3964
3965     void printStatus(const char *F, int L, const char *istr=NULL);
3966
3967     int get1J_gen(const char *F, int L, const qset<TQ> &q, char flag=0);
3968
3969     int getTensorProdReps_gen(
3970         const qset<TQ> &q1, const qset<TQ> &q2,
3971         unsigned flag=TP3_LDM);
3972
3973     void getTensorProdReps(

```



```

3974     const qset<TQ> &q1, const qset<TQ> &q2
3975 ) {
3976     q.validType(FL);
3977     int n=getTensorProdReps_gen(q1,q2,TP3_TST);
3978     if (n>=0) return;
3979
3980     wblog(FL,"ERR %s() "
3981           "failed to access RSets of input spaces (%d) !?",FCT,n);
3982
3983 };
3984
3985 unsigned getTensorProdReps(const qset<TQ> &q, unsigned flag=0);
3986 unsigned getTensorProdReps(unsigned dmax=-1, char sdig=0, char vflag=0);
3987
3988 void put(const char *vname, const char *ws="caller") const
3989 { put(0,0,vname,ws); }
3990
3991 mxArray* toMx(char bflag=1) const;
3992 void put(const char *F, int L,
3993         const char *vname, const char *ws="caller"
3994 ) const;
3995
3996 QType q;
3997
3998 map <qset<TQ>,
3999     genRG_base<TQ,TD>
4000 > RSet;
4001
4002 qset<TQ> qs;
4003
4004 wbvector< pairPatternCG<double> > CR;
4005
4006 wbMatrix<double> DZ;
4007
4008 protected:
4009 private:
4010 };
4011
4012 template <class TQ>
4013 TQ getTensorProdReps_abelian(const QType &q, const TQ &J1, const TQ &J2) {
4014     TQ J=0;
4015
4016     switch (q.type) {
4017     case QTYPE_U1: J=J1+J2; break;
4018
4019     case QTYPE_ZN: J=int(J1+J2)%q.sub;
4020
4021         if (TQ(int(J1)%q.sub)!=J1 || TQ(int(J2)%q.sub)!=J2)
4022             wblog(FL,"ERR %s() invalid %s labels (%g,%g)",
4023                 FCT, STR(q), double(J1), double(J2));
4024             break;
4025
4026     case QTYPE_P: J=J1*J2;
4027
4028         if (fabs(double(J1))!=1 || fabs(double(J2))!=1)
4029             wblog(FL,"ERR %s() invalid %s labels (%g,%g)",
4030                 FCT,STR(q),double(J1),double(J2));
4031             break;
4032
4033     default: wblog(FL,"ERR %s() got invalid symmetry %s",FCT,STR(q));
4034     }
4035 }
4036
4037 return J;
4038 };
4039
4040 template <class TQ, class TD>
4041 class RStore {
4042 public:
4043
4044     RStore(const char *s=NULL) {
4045         if (s) {
4046             if (!strcmp(s,"info")) {
4047                 #ifdef __WB_MPFR_HH__
4048                     wblog(FL,"<i> %s() uses type %s (=mpfr@%d) ",
4049                         FCT, TSTR(TD), TD().prec());
4050                 #else
4051                     wblog(FL,"<i> %s() uses type %s",FCT,TSTR(TD));
4052                 #endif
4053             }
4054             else wblog(FL,"ERR %s() invalid usage (%s)",FCT,TSTR(TD));
4055         }
4056     };
4057
4058     size_t bufsize(const QType &q) {
4059         auto it=buf.find(q);
4060

```

```

4061     return (it==buf.end() ? 0 : it->second.RSet.size());
4062 };
4063
4064 bool isEmpty(const QType &q) {
4065     return (bufsize(q)>0 ? 1 : 0);
4066 };
4067
4068 genRG_struct<TQ,TD>& Buf(const QType &q) {
4069     auto it=buf.find(q);
4070     if (it!=buf.end()) {
4071         genRG_struct<TQ,TD> &B=(it->second);
4072         if (!B.q.isKnown()) {
4073             if (B.RSet.size()) wblog(FL,
4074                 "ERR %s() got RSet with unknown qtype !? (len=%d; %s)",
4075                 FCT,B.RSet.size(),STR(B.q));
4076             B.checkInit(FL,q);
4077         }
4078         return B;
4079     }
4080     else {
4081         genRG_struct<TQ,TD> &B=buf[q];
4082         B.checkInit(FL,q);
4083         return B;
4084     }
4085 };
4086
4087 genRG_base<TQ,TD>* find_RSet(const QType &q, const TQ *qptr){
4088     genRG_struct<TQ,TD> &B=Buf(q);
4089     qset<TQ> qs(q.qlen(),qptr,'r');
4090
4091     auto it = B.RSet.find(qs);
4092     return (it!=B.RSet.end() ? &(it->second) : NULL);
4093 };
4094
4095 genRG_struct<TQ,TD>& getRSet(
4096     const char *F, int L, const QType &q,
4097     qset<TQ> *qvec=NULL
4098 ){
4099     genRG_struct<TQ,TD> &B=Buf(q);
4100     if (qvec) { (*qvec)=B.qs; }
4101     return B;
4102 };
4103
4104 genRG_base<TQ,TD>& getR(
4105     const char *F, int L, const QType &qtype, const TQ *q) const;
4106
4107 genRG_base<TQ,TD>& getR(
4108     const char *F, int L, const CDATA_TQ &S,
4109     unsigned k
4110 ) const { return getR(F_L, S.t, S.qptr(k)); };
4111
4112 wbMatrix<TQ> getZ(
4113     const char *F, int L, const QType &qtype, const TQ *q
4114 ) const {
4115     const genRG_base<TQ,TD> &R = getR(F_L,qtype,q);
4116     return R.Z;
4117 };
4118
4119 unsigned qdim(
4120     const char *F, int L, const QType &qtype, const TQ *q
4121 ) const {
4122     if (qtype.isAbelian()) return 1;
4123     const genRG_base<TQ,TD> &R = getR(F_L,qtype,q);
4124
4125     if (qtype.isSU2() && TQ(R.Z.dim1)!=q[0]+1) wblog(FL,"ERR %s() "
4126         "got multiplet dim mismatch (%d/%d)",FCT,R.Z.dim1,int(q[0]+1));
4127
4128     return R.Z.dim1;
4129 };
4130
4131
4132 mxArray* toMx(const QType &q=QTYPE_UNKNOWN, char bflag=1) const;
4133
4134 unsigned LoadStore(const char *F, int L, const char *file);
4135 unsigned add(const QType &q, const mxArray* S, unsigned k);
4136
4137 void Info(const char *F=0, int L=0) const;
4138
4139 map<QType,
4140     genRG_struct<TQ,TD>
4141 > buf;
4142
4143 protected:
4144 private:
4145 };
4146
4147 RStore<gTQ,RTD> gRS;

```

```

4148
4149 #ifdef QS_USING_OMP
4150     Wb::ompLockNest RCS_lock("RCStore");
4151 #endif
4152
4153     map<QType,int> g_sym;
4154
4155 class RCStore {
4156
4157 public:
4158
4159     RCStore() { };
4160
4161     int setupDirs(const char *F=NULL, int L=0) {
4162
4163         if (root.idx.len) { return -root.idx.len; }
4164
4165         #ifdef QS_USING_OMP
4166             Wb::ompGuard dLK(RCS_lock);
4167             if (root.idx.len) { return -root.idx.len; }
4168         #endif
4169
4170         int i1,i2=0,e=0;
4171
4172         const char rcs[]="RC_STORE", rsy[]="RC_SYNC";
4173
4174         i1=setup_root_directory(F_L,rcs,root);
4175         if (!i1) wblog(FL,"ERR missing env variable '%s' !?",rcs);
4176
4177         i2=setup_root_directory(F_L,rsy,root,'+');
4178
4179         if (i1<0 || i2<0) { root.init(); wblog(FL,
4180             "ERR check environmental variables"); }
4181         if (i2>1) { root.init(); wblog(FL,
4182             "ERR %s() only one RC_SYNC directory allowed",FCT); }
4183
4184         for (auto it=root.begin(); it.end(); ++it) {
4185             if (!strncmp(it.at(),"/home/",7)) { e=1;
4186                 wblog(FL,"WRN %s() RC_%s points to HOME (%d/%d)\n"
4187                     "(hint: use /data, instead)",FCT,
4188                     !it.i || it.i+1<it.n?"STORE":"SYNC",it.i+1,it.n);
4189                 root.print("RC_STORE+SYNC"); break;
4190             }
4191         }
4192         if (!e && CG_VERBOSE>6) root.print("RC_STORE+SYNC");
4193
4194         if (!i2 && i1) {
4195             root.append(FL,root.last(),":");
4196         }
4197         return root.idx.len;
4198     };
4199
4200 template <class TQ, class TD>
4201 int save_CData(const char *F, int L, const CData<TQ,TD> &A);
4202
4203 template <class TQ, class TD>
4204 int load_CData(
4205     const char *F, int L, const QSet<TQ> &Q, CData<TQ,TD> &A,
4206     char bflag=0);
4207
4208 template <class TQ>
4209 int save_Std3(
4210     const char *F, int L, const QType &t, const qset<TQ> &J12,
4211     const wvector<double> *c2eps=NULL) const;
4212
4213 template <class TQ>
4214 int load_Std3(const char *F, int L,
4215     const QType &t, const qset<TQ> &J1, const qset<TQ> &J2,
4216     unsigned loadRC=0) const;
4217
4218 template <class TQ, class TD>
4219 int save_RSet(
4220     const char *F, int L, const genRG_base<TQ,TD> &R, char xflag=0);
4221
4222 template <class TQ>
4223 int load_RSet(
4224     const char *F, int L, const QType &t, const qset<TQ> &J,
4225     char vflag=0);
4226
4227 int save_XMap(
4228     const char *F, int L, const cgc_contract_id<MTI> &idc) const;
4229
4230 int load_XMap(
4231     const char *F, int L, const cgc_contract_id<MTI> &idc) const;
4232
4233 template <class TQ>
4234 int get_file_name(

```

```

4235     const char *F, int L, wstring &file,
4236     const QSet<TQ> &Q, const char *ext=NULL, char mflag=0) const;
4237
4238 template <class TQ, class TD>
4239 int get_file_name(
4240     const char *F, int L, wstring &file,
4241     const genRG_base<TQ,TD> &R, const char *ext=NULL, char mflag=0
4242 ) const;
4243
4244 int get_file_name(
4245     const char *F, int L, wstring &file,
4246     const cgc_contract_id<MTI> &idc, const char *ext=NULL, char mflag=0,
4247     QType *t=NULL
4248 ) const;
4249
4250 size_t rclog(const QType &t,
4251     const char *F, int L, const char *fmt, ...);
4252
4253 size_t rclog(const QType &t,
4254     const char *F, int L, char vflag, const char *fmt, ...);
4255
4256 size_t vrclog(const QType &t,
4257     const char *F, int L, const char *fmt, va_list args);
4258
4259 void Info(const char *F=0, int L=0) {
4260     if (F) wblog(F,L," * %s()\n",FCT);
4261     root.print("RCStore::root");
4262     fprintf(stdout,"\n\n");
4263 }
4264
4265 int is_new(const QType &t) {
4266     return (g_sym[t] ? 0 : 1);
4267 };
4268
4269 Wb::Path root;
4270
4271 protected:
4272 private:
4273
4274 int get_rcs_path(
4275     const char *F, int L, char *path, unsigned len, const QType &t,
4276     const char *sub1=NULL, const char *sub2=NULL, const char *file=NULL,
4277     char mflag=0
4278 ) const;
4279
4280 int setup_root_directory(
4281     const char *F, int L, const char *env, Wb::Path &root,
4282     char append=0
4283 );
4284 };
4285
4286 RCStore gStore;
4287
4288 Wb::IOstat gstatR,gstatM,gstatC,gstatX;
4289
4290 #define RC_MDIR 1
4291 #define RC_SYNC 2
4292 #define RC_CURR 4
4293
4294 #define RC_SAVE (RC_MDIR|RC_CURR)
4295 #define RC_SAVE_S (RC_MDIR|RC_SYNC)
4296
4297 #define RCL_V1 1
4298 #define RCL_V2 2
4299
4300 #define RCL_ALL (RCL_V1|RCL_V2)
4301
4302 int RCStore::setup_root_directory(
4303     const char *F, int L, const char *env, Wb::Path &root, char append
4304 ){
4305     int m=0, k=0, e=0, kmax;
4306     const char *sval;
4307
4308     if (!env || !env[0]) wblog(F,L,
4309         "ERR %s() no variable specified !?",FCT);
4310
4311     sval=getenv(env); if (!sval || !sval[0]) {
4312         if (!append) root.init();
4313         return m;
4314     }
4315
4316     if (!append)
4317         { m=root.init(F,L,sval,":",1024); }
4318     else { k=root.numel(); m=root.append(F,L,sval,":"); }
4319
4320     for (kmax=k+m; k<kmax; ++k) { char *dir=root[k];
4321

```

```

4322     if (!dir || !dir[0]) {
4323         if (!e) { PRINTF("\n"); }; e|=1;
4324         PRINTF("    ERR invalid %s entry (empty)\n",env);
4325     }
4326     if (Wb::fexist(dir,'d')) continue;
4327
4328     if (k+1<kmax) {
4329         if (!e) { PRINTF("\n"); }; e|=2;
4330         PRINTF("    ERR invalid %s entry (%s) ^1\n",env,dir);
4331     }
4332
4333     unsigned n=strlen(dir), i=n-1, i0=0; int q;
4334
4335     for (; i<n; --i) { if (dir[i]=='/') { i0=i; break; }}
4336     if (i0 && (
4337         !strncasecmp(dir+i0+1,"RCStore",7) ||
4338         !strncasecmp(dir+i0+1,"RCSync", 6))
4339     ){
4340         dir[i0]=0; q=Wb::fexist(dir,'d');
4341         dir[i0]='.'/;
4342         if (q) {
4343             mode_t p=(S_IRWXU | S_IRWXG | S_IROTH | S_IXOTH);
4344             if ((q=mkdir(dir,p))==0) {
4345                 PRINTF("    creating %-8s root directory (%s)\n",
4346                     env, Wb::repHome(dir).data);
4347             }
4348             else {
4349                 if (!e) { PRINTF("\n"); }; e|=4;
4350                 PRINTF("    ERR invalid %s entry (%s) ^2\n",env,dir);
4351             }
4352         }
4353     }
4354     else if (!Wb::fexist(dir,'d')) {
4355         if (!e) { PRINTF("\n"); }; e|=8;
4356         PRINTF("    ERR invalid %s entry (%s) ^3\n",env,dir);
4357     }
4358 }
4359 if (e) { PRINTF("\n");
4360     if (e&2) PRINTF("    ^1 all but last entry must exist\n");
4361     if (e&4) PRINTF("    ^2 failed to create directory\n");
4362     if (e&8) PRINTF("    ^3 last directory must also exist if not named %s*\n",
4363         strncmp(env,"RC_SYNC",8) ? "RCStore":"RCSync" );
4364     if (e>1) { PRINTF("\n"); }
4365 }
4366
4367 if (e) wblog(FL,"ERR invalid env %s directory (e=%d)",env);
4368
4369 return (e ? -m : m);
4370 };
4371
4372 namespace CG {
4373
4374 template <class TQ, class TD>
4375 double getSymmetryStates(const char *F, int L, const QType &q,
4376     const wbvector< wbSparrowTD > &Sp,
4377     const wbvector< wbSparrowTD > &Sz,
4378     wbvector< wbSparrowTD > &U,
4379     wbvector<unsigned> &dd,
4380     wbvector<genRG_base<TQ,TD> > &RR,
4381     wbvector<double> &c2eps,
4382     wbMatrix<unsigned> *iOM=NULL,
4383     char vflag=1
4384 );
4385
4386 template <class TQ>
4387 SPIDX_T findMaxWeight(
4388     const QType &q, const wbMatrix<double> &Z, qset<TQ> *J=NULL, wbperm *P=NULL
4389 );
4390
4391 template <class TQ, class TD>
4392 void get_SU2mat(TQ s2,
4393     wbvector<double> &sz,
4394     wbsparrow<TD> &Sp, wbsparrow<TD> &Sz,
4395     wbsparrow<TD> &S2, wbsparrow<TD> &E
4396 );
4397
4398 int isQSet (const mxArray *S, unsigned k=-1);
4399 int isCData(const mxArray *S, unsigned k=-1);
4400 int isCRef (const mxArray *S, unsigned k=-1);
4401
4402 int MemStat(const char *F=0, int L=0) {
4403     int n=0; const char *hstr="RCStore"; const char *xstr="/full";
4404
4405     wblog(F_L,"%N%s() %33R",FCT,"-");
4406     gRS.Info();
4407     gCS.Info();
4408     gXS.Info();

```

```

4409     Wb::MemStat (FL);
4410
4411     if (!gstatR.isEmpty()) gstatR.print("RData", (++n==1)? hstr:0, xstr);
4412     if (!gstatM.isEmpty()) gstatM.print("map3", (++n==1)? hstr:0, xstr);
4413     if (!gstatC.isEmpty()) gstatC.print("CData", (++n==1)? hstr:0, xstr);
4414     if (!gstatX.isEmpty()) gstatX.print("XData", (++n==1)? hstr:0, xstr);
4415
4416     if (n) { PRINTF("\n"); }
4417     else wblog(FL,"I/O no data read or written to RCStore yet");
4418
4419     return 0;
4420 };
4421
4422 class FileLock {
4423
4424     public:
4425
4426         FileLock() : fid(-1) {};
4427
4428         ~FileLock() {
4429             if (fid!=-1) { ReleaseLock(FL); }
4430         };
4431
4432         template<class TQ>
4433         FileLock(
4434             const QType &q_, const qset<TQ> &J1, const qset<TQ> &J2,
4435             const char *ext="mp3"
4436         ) { init(q,J1,J2,ext); }
4437
4438         template<class TQ>
4439         int init(
4440             const QType &q_, const qset<TQ> &J1, const qset<TQ> &J2,
4441             const char *ext="mp3"
4442         );
4443
4444         int GetLock(const char *F=NULL, int L=0, char flag='W') {
4445             return tryGetLock(F,L,flag); };
4446
4447         int ReleaseLock(const char *F=NULL, int L=0) {
4448             return tryGetLock(F,L,'u');
4449         };
4450
4451         int fid;
4452         QType q;
4453         wstring fname;
4454
4455     protected:
4456     private:
4457
4458         int tryGetLock(const char *F, int L, char flag);
4459 };
4460
4461 #ifdef QS_USING_OMP
4462     map <size_t, Wb::ompLockNest*> lock_map;
4463
4464     class Guard {
4465
4466     public:
4467
4468         Guard() : active(false) {};
4469
4470         template<class TQ>
4471         Guard(const char *F, int L, const QSet<TQ> &Q, const char *tag=NULL,
4472             const char *cond=NULL)
4473             : active(false) { acquire(F_L,Q,tag,cond); };
4474
4475         template<class TM>
4476         Guard(const char *F, int L, const cgc_contract_id<TM> &x,
4477             const char *cond=NULL)
4478             : active(false) { acquire(F_L,x,cond); }
4479
4480         template<class TQ>
4481         Guard(const char *F, int L, const QSet<TQ> &QA, const QSet<TQ> &QB)
4482             : active(false) { acquire(F_L,QA,QB); };
4483
4484         template<class TQ>
4485         Guard(const char *F, int L,
4486             const QSet<TQ> &QA, const QSet<TQ> &QB, const QSet<TQ> &QC)
4487             : active(false) { acquire(F_L,QA,QB,QC); };
4488
4489         ~Guard() {
4490             if (hid.len) { release(); hid.init(); }
4491             active=false;
4492         };
4493
4494         template<class TQ>

```

```

4496     int acquire(const char *F, int L,
4497                const QSet<TQ> &Q, const char *tag=NULL, const char *cond=NULL);
4498
4499     template<class TM>
4500     int acquire(const char *F, int L,
4501                const cgc_contract_id<TM> &x, const char *cond=NULL);
4502
4503     template<class TQ>
4504     void acquire(const char *F, int L, const QSet<TQ>* Q[], unsigned n);
4505
4506     template<class TQ>
4507     void acquire(const char *F, int L,
4508                const QSet<TQ> &QA, const QSet<TQ> &QB)
4509     { const QSet<TQ>* QQ[2]={ &QA, &QB }; acquire(F,L,QQ,2); };
4510
4511     template<class TQ>
4512     void acquire(const char *F, int L,
4513                const QSet<TQ> &QA, const QSet<TQ> &QB, const QSet<TQ> &QC)
4514     { const QSet<TQ>* QQ[3]={ &QA, &QB, &QC }; acquire(F,L,QQ,3); };
4515
4516     inline int acquire_set(const char *ss[], int ntry=-1) {
4517         if (!hid.len) return -1;
4518
4519         int got[hid.len];
4520         Wb::ompLockNest *lk[hid.len];
4521
4522         int nlks=acquire_set_iter(got,lk,ss,0);
4523         if (nlks<=0) { if (ntry==1 || !ntry) return nlks;
4524             nlks=acquire_set_wait(got,lk,ss,ntry);
4525         }
4526
4527         if (nlks<=0) {
4528             if (ntry<=0 || ntry>99) wblog(FL,
4529                "ERR %s() failed to qdquire locks (%d/%d)",FCT,nlks,hid.len);
4530             return nlks;
4531         }
4532
4533         for (unsigned i=0; i<hid.len; ++i) {
4534             lk[i]->set_istr(ss[i]);
4535             if (*got < got[i]) { *got=got[i]; }
4536         }
4537         active=true;
4538
4539         return *got;
4540     };
4541
4542     int acquire_set_iter(
4543         int *got, Wb::ompLockNest *lk[], const char *ss[], unsigned iter=0);
4544     int acquire_set_wait(
4545         int *got, Wb::ompLockNest *lk[], const char *ss[], int ntry=-1);
4546
4547     void release();
4548
4549     int level() { int l=-1;
4550         for (unsigned i=0; i<hid.len; ++i) {
4551             const Wb::ompLockNest *lp = Guard::find(hid[i]);
4552             if (lp && l<(lp->level)) { l=lp->level; }
4553         }
4554         return l;
4555     };
4556
4557     int used() { int l=0;
4558         for (unsigned i=0; i<hid.len; ++i) {
4559             const Wb::ompLockNest *lp = Guard::find(hid[i]);
4560             if (lp && l<(lp->level)) {
4561                 if (l< lp->level ) { l=lp->level; }
4562                 if (l< lp->nactive) { l=lp->nactive; }
4563                 if (l< lp->ntot ) { l=lp->ntot; }
4564             }
4565         }
4566         if (!l && active) { l=-1; }
4567         return l;
4568     };
4569
4570     void print(const char *F, int L);
4571
4572     wstring toStr() const;
4573
4574     static wstring toStr(size_t h, const char *istr=NULL);
4575     static int toStr(wstring &s, size_t h, const char *istr=NULL);
4576
4577     template<class TQ>
4578     static wstring Status(const QSet<TQ> &Q, const char *tag=NULL);
4579
4580     template<class TM>
4581     static wstring Status(const cgc_contract_id<TM> &x);
4582

```

```

4583     static wstring Status(size_t h1, const char *istr=NULL);
4584
4585     static Wb::ompLockNest* find(size_t h);
4586
4587     static int print_CG_locks();
4588
4589 protected:
4590 private:
4591
4592     wvector<size_t> hid;
4593
4594     bool active;
4595
4596     const char* sprintf_locks(
4597         wstring &sout, const int* got, const char *istr[],
4598         const char *indent=NULL);
4599
4600     int deal_with_cond(const char *F, int L, size_t h0, const char *cond);
4601
4602 };
4603
4604 #endif
4605
4606 };
4607
4608 template<class TD>
4609 int BuildKrylovH(
4610     const QType &q,
4611     wbararray<TD> &HK, unsigned nk,
4612     wSparrayTD &X,
4613     wvector<TD> &EK,
4614     const wSparrayTD &HL,
4615     const wSparrayTD &HR,
4616     const wvector< wSparrayTD > SL,
4617     const wvector< wSparrayTD > SR,
4618     double &xmin
4619 );
4620
4621 template<class TD>
4622 void GetHPsi(
4623     wSparrayTD &HX, const wSparrayTD &X,
4624     const wSparrayTD &HL,
4625     const wSparrayTD &HR,
4626     const wvector< wSparrayTD > SL,
4627     const wvector< wSparrayTD > SR
4628 );
4629
4630 #endif
4631

```

5.2 cpat.hh

```

1  #ifndef __WB_CPAT_HCC__
2  #define __WB_CPAT_HCC__
3
4  /* ----- *
5  * class CPAT: class to store contraction pattern in QSpace structure
6  * ----- *
7  * contraction pattern (default: twoSite mode)
8  *
9  * deals with MatLab cell array of the form { A, i1, i2 [,opA] }
10 * where A can be a single QSpace or a cell vector of these
11 * i[12] specifies index onto which to contract onto Psi[12]
12 * NB! either i1 or i2 is set, the other one must be empty, e.g. []
13 *
14 * single site mode: expects { A, ic, [,opA] } instead as
15 * contraction pattern where A is always contracted onto "Psi1"
16 *
17 * ----- *
18 * AW (C) Jun 2006
19 * ----- */
20
21 #define CP1 "{ A, ic, [,opA] }"
22 #define CP2 "{ A, i1, i2 [,opA] }"
23 #define _CP (twoSite ? "{ A, i1, i2 [,opA] }" : "{ A, ic, [,opA] }")
24
25 template <class TQ, class TD>
26 class CPAT_Set;
27
28 int isCPAT(const mxArray *C, char twoSite=-1);
29 int isCPattern(const mxArray *C);
30
31 unsigned NR_ICTR_CALL=0;

```



```

32
33 template <class TQ, class TD>
34 class CPAT {
35
36 public:
37
38     CPAT() : z(0), sameCtrIdx(0), nr_check_ctr(0), HPsi_counter(0) {};
39
40     CPAT(const CPAT<TQ,TD> &D) {
41         wblog(FL,"ERR do not copy CPAT object!");
42     };
43
44     void init(
45         TD oz, const mxArray**, unsigned n,
46         unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
47         const char twoSite, const char ldir
48     );
49
50     int getCtrIndex(const QSpace<TQ,TD> &PSI);
51
52     QSpace<TQ,TD>& HZTimes(
53         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &HZPsi,
54         const char ='N', char disp=1
55     );
56
57     QSpace<TQ,TD>& HZApplyDiag(
58         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &EPsi,
59         TD a, TD b, char iflag=0, char disp=1
60     );
61
62     void check_contraction(const QSpace<TQ,TD> &PSI, char disp=1);
63     void initCounter() {
64         HPsi_counter=0;
65     };
66
67     QSpace<TQ,TD>& HZTimes_tst(
68         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &HZPsi,
69         const char ='N', char disp=1);
70
71     void getQRange(
72         unsigned k, wbMatrix<TQ> &QQ, wbvector<unsigned> &SS,
73         QSpace<TQ,TD> *B=NULL
74     ) const;
75
76     mxArray* toMx() const;
77
78     void put(const char *vname, const char* ws="base") const {
79         mxPutAndDestroy(FL,toMx(),vname, ws);
80     };
81
82     void PermIC(const wbperm &P);
83
84     wbvector< wbvector< CPAT_Set<TQ,TD> > > CP;
85     TD z;
86
87     unsigned sameCtrIdx;
88     unsigned nr_check_ctr;
89     unsigned HPsi_counter;
90
91 protected:
92 private:
93 };
94
95 template <class TQ, class TD>
96 class CPAT_Set {
97
98 public:
99
100     CPAT_Set () : opA(0), ic(0) {};
101
102     CPAT_Set (const CPAT_Set<TQ,TD> &D) : opA(0), ic(0) {
103         wblog(FL,"WRN Do not copy CPAT_Set!");
104     };
105
106     int init(
107         const mxArray *a,
108         unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
109         const char twoSite, const char ldir
110     );
111
112     int getCtrIndex(const QSpace<TQ,TD> &PSI);
113     int getCtrIndex(const wbvector< QSpace<TQ,TD> >&, const char ='N');
114
115     int getCtrIndex(const unsigned i, const QSpace<TQ,TD>&, char ='N');
116
117     void contract(
118         const QSpace<TQ,TD> &PSI, wbvector< QSpace<TQ,TD> > &C,

```

```

119     char='N') const;
120
121 void contract(
122     const wbvector< QSpace<TQ,TD> > &PSI1, wbvector< QSpace<TQ,TD> > &C,
123     char='N') const;
124
125 void contract(
126     const unsigned k, const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &C,
127     char='N') const;
128
129 void contract_diag(
130     const QSpace<TQ,TD> &PSI, wbvector< QSpace<TQ,TD> > &C,
131     TD a, TD b, char Iflag) const;
132
133 void contract_diag(
134     const wbvector< QSpace<TQ,TD> > &PSI1, wbvector< QSpace<TQ,TD> > &C,
135     TD a, TD b, char Iflag) const;
136
137 void contract_diag(
138     const unsigned k, const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &C,
139     TD a, TD b, char Iflag) const;
140
141 mxArray* toMx() const;
142
143 wbvector< QSpace<TQ,TD> > A;
144 char opA;
145 unsigned ic;
146
147 wbperm P;
148
149 wbvector< QSpace<TQ,TD> > CQn, CQt;
150 wbvector< wbMatrix<unsigned> > CIn, CIt;
151
152 protected:
153 private:
154 };
155
156 template <class TQ, class TD>
157 void CPAT<TQ,TD>::init(
158     TD oz, const mxArray** aa, unsigned N,
159     unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
160     const char twoSite, const char ldir
161 ) {
162     unsigned i,j,e,k,p,m=0,n;
163     wbvector<unsigned> dd;
164     mxArray *a;
165
166     if (ic1>=r1 || ic2>=r2) wblog(FL,
167         "ERR Contraction index out of bounds (%d/%d; %d/%d (%d))",
168         ic1+1, r1, ic2+1, r2, twoSite
169     );
170
171     z=oz;
172
173     CP.initDef(N);
174
175     for (p=k=0; k<N; k++) {
176         if (!mxIsCell(aa[k]))
177             wblog(FL, "ERR Invalid (pair) pattern %s", _CP);
178
179         n=isCPAT(aa[k], twoSite);
180
181         if (!n) werror(FL,str);
182
183         CP[p].initDef(n);
184
185         if (n==1) {
186             e=CP[p][0].init(aa[k],ic1,r1,ic2,r2,twoSite,ldir);
187             if (e) werror(FL,str);
188             m=CP[p][0].A.len;
189         }
190         else {
191             for (i=0; i<n; i++) {
192                 a=mxGetCell(aa[k],i);
193                 e=CP[p][i].init(a,ic1,r1,ic2,r2,twoSite,ldir);
194
195                 if (e) {
196                     printf("\n%s:%d contraction pattern %d:%d\n", FL, k+1, i+1);
197                     printf("%s\n",str); ExitMsg("");
198                 }
199
200                 if (i>0) { if (m!=CP[p][i].A.len) wblog(FL,
201                     "ERR Pattern pair dimension mismatch in A (%d.%d: %d,%d)",
202                     k+1, i, CP[p][0].A.len, CP[p][i].A.len); }
203                 else m=CP[p][0].A.len;
204             }
205         }

```

```

206     dd.init(m);
207     for (i=0; i<n; i++) {
208         const wvector< QSpace<TQ,TD> > &Aki=CP[p][i].A;
209         for (j=0; j<m; j++) if (!Aki[j].isEmpty()) dd[j]++;
210     }
211
212     for (j=0; j<m; j++) if (dd[j]==n) { p++; break; }
213 }
214
215 if (p<N) {
216     if (p==0) wblog(FL,"WRN eigs: no relevant contraction pattern!");
217     CP.len=p;
218 }
219
220 NR_ICTR_CALL=0;
221 }
222
223 template <class TQ, class TD>
224 int CPAT_Set<TQ,TD>::init(
225     const mxArray *C,
226     unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
227     const char twoSite, const char ldir
228 ){
229     unsigned k,m,w, ra, ica, ic0, N=mxGetNumberOfElements(C);
230     const char isC=(typeid(TD)==typeid(wbcomplex) ? 'C' : 0);
231     int r=-1;
232
233     mxArray *a; str[0]=0;
234
235     if (ic1>=r1 || ic2>=r2) {
236         sprintf(str, "%s:%d ERR Invalid input (%d/%d; %d/%d (%d))",
237             FL, ic1+1, r1, ic2+1, r2, twoSite);
238         return 1;
239     }
240
241     m=isCPattern(C);
242
243     if ((char)m!=twoSite+1) wblog(FL,
244         "ERR invalid input (not a %d-site CPattern; %d,%d)", twoSite+1,m);
245
246     a=mxGetCell(C,0);
247
248     if (mxIsQSpace(0,0,a,r,isC)>0) {
249         A.initDef(1);
250         A[0].init(FL,a,'r');
251
252         if (!A[0].isConsistentR(2)) {
253             sprintf(str, "%s:%d rank-2 object required (arg #1; %s)",FL,_CP);
254             return 1;
255         }
256     }
257     else {
258         if (mxIsQSpaceVec(a,2,isC)) { sprintf(str,
259             "%s:%d rank-2 QSpace (vector) required (arg #1; %s)",FL,_CP);
260             return 1;
261         }
262         mxInitQSpaceVec(FL,a,A,'r');
263     }
264
265     if (twoSite) {
266         w = (mxIsEmpty(mxGetCell(C,1)) ? 0 : 1) +
267             (mxIsEmpty(mxGetCell(C,2)) ? 0 : 2);
268         k=w;
269
270         if (w<1 || w>2) {
271             sprintf(str, "%s:%d Either arg #2 or #3 in CPAT %s",FL,_CP);
272             return 1;
273         }
274     }
275     else {
276         k=1; w=(ldir>0 ? 1 : 2);
277
278         if (mxIsEmpty(mxGetCell(C,k))) {
279             sprintf(str, "%s:%d Invalid arg #%d in CPAT %s",FL,k,CP1);
280             return 1;
281         }
282     }
283
284     ra = (w==1 ? r1 : r2);
285     ica = (w==1 ? ic1 : ic2);
286
287     a=mxGetCell(C,k);
288
289     if (mxGetNumber(a,ic)) { sprintf(str+strlen(str),

```

```

293     "\n%s:%d Invalid arg #%d in CPAT %s", FL, k+1, _CP);
294     return 1;
295 }
296
297 if (!ic || ic>ra) { sprintf(str+strlen(str),
298     "\n%s contraction index out of bounds (%d/%d; %d,%d) in CPAT %s",
299     shortFL(FL), ic, ra, r1, r2, _CP); return 1;
300 }
301
302 ic0=ic;
303 ic--;
304
305 if (twoSite && ic==ica) { sprintf(str+strlen(str),
306     "\n%s Index %d overlaps with ctr index in CPAT %s",
307     shortFL(FL), ic0, _CP); return 1;
308 }
309
310 if (twoSite) {
311     if (ic>ica) ic--;
312     if (ldir>0 && w==2) ic+=(r1-1); else
313     if (ldir<0 && w==1) ic+=(r2-1);
314 }
315 else {
316     if (ic>ica) ic--; else
317     if (ic==ica) ic=ra-1;
318 }
319
320 opA=0; k=(twoSite ? 3 : 2);
321
322 if (k<N) {
323     a=mxGetCell(C,k);
324
325     if (mxGetNumber(a,opA)) { sprintf(str,
326         "%s:%d Invalid arg #%d in CPAT %s", FL, 3+twoSite, _CP);
327         return 1;
328     }
329     if (N>k+1) { sprintf(str,
330         "%s:%d Too many arguments for CPAT %s", FL, _CP);
331         return 1;
332     }
333 }
334
335 opA=toupper(opA); if (!opA || opA==' ') opA='N';
336
337 if (!Wb::strchr("NTC*",opA)) { sprintf(str,
338     "%s:%d ERR invalid opA=%c<%d> in CPAT %s", FL, opA, opA, _CP);
339     return 1;
340 }
341
342 if (opA!='N') {
343     char tflag= (opA=='T' || opA=='C'),
344             cflag= (opA=='C' || opA=='*');
345     wbperm P("2 1");
346
347     for (unsigned i=0; i<A.len; i++) {
348         if (A[i].isEmpty()) continue;
349         if (tflag) A[i].Permute(P);
350         if (cflag) A[i].Conj();
351     }
352 }
353
354 return 0;
355 }
356
357 template <class TQ, class TD>
358 void CPAT<TQ,TD>::PermIC(const wbperm &P) {
359     unsigned i,j, m=CP.len, n;
360     wbperm iP;
361
362     P.invert(iP);
363
364     for (i=0; i<m; i++) {
365         for (n=CP[i].len, j=0; j<n; j++)
366             CP[i][j].ic = iP[CP[i][j].ic];
367     }
368 }
369
370 template <class TQ, class TD>
371 int CPAT<TQ,TD>::getCtrIndex(const QSpace<TQ,TD> &PSI) {
372
373     unsigned i,j,n; int m=0;
374     const QSpace<TQ,TD> PSI2;
375
376     if ((++NR_ICTR_CALL)%20==0) wblog(FL,
377         "%NWRN calling %s already %d times (?)", FCT, NR_ICTR_CALL);
378
379     for (i=0; i<CP.len; i++) {

```

```

380     m += CP[i][0].getCtrIndex(PSI);
381
382     for (n=CP[i].len, j=1; j<n; j++) {
383         if (CP[i][0].A.len!=CP[i][j].A.len) wblog(FL,
384             "ERR Severe dimension mismatch (%d: %d,%d)", j,
385             CP[i][0].A.len, CP[i][j].A.len);
386
387         m += CP[i][j].getCtrIndex(CP[i][j-1].CQn, 'N');
388         m += CP[i][j].getCtrIndex(CP[i][j-1].CQt, 'T');
389     }
390 }
391
392 return m;
393 }
394
395 template <class TQ, class TD>
396 int CPAT_Set<TQ,TD>::getCtrIndex(const QSpace<TQ,TD> &PSI) {
397
398     unsigned i,m=0,n=A.len;
399
400     if (CIn.len!=n) { CIn.initDef(n); m++; }
401     if (CIt.len!=n) { CIt.initDef(n); m++; }
402     if (CQn.len!=n) { CQn.initDef(n); m++; }
403     if (CQt.len!=n) { CQt.initDef(n); m++; }
404
405     if (A.len==0) return (m!=0);
406
407     P.Index(PSI.rank());
408     P.mvFirstTo(ic);
409
410     for (i=0; i<A.len; i++) {
411         m += getCtrIndex(i,PSI, 'N');
412         m += getCtrIndex(i,PSI, 'T');
413     }
414
415     return (m!=0);
416 }
417
418 template <class TQ, class TD>
419 int CPAT_Set<TQ,TD>::getCtrIndex(
420     const wbvector< QSpace<TQ,TD> > &PSI1,
421     const char tflag
422 ){
423     unsigned i, m=0, n=A.len;
424
425     if (!Wb::strchr("NT",tflag)) wblog(FL,
426         "ERR Invalid flag %c<%d>",tflag,tflag);
427
428     if (tflag=='N') {
429         if (CIn.len!=n) { CIn.initDef(n); m++; }
430         if (CQn.len!=n) { CQn.initDef(n); m++; }
431     }
432     else {
433         if (CIt.len!=n) { CIt.initDef(n); m++; }
434         if (CQt.len!=n) { CQt.initDef(n); m++; }
435     }
436
437     if (n==0) return (m!=0);
438
439     if (n!=PSI1.len) wblog(FL,
440         "ERR Severe dimension mismatch (%d,%d)", n, PSI1.len);
441
442     for (i=1; i<n; i++) if (PSI1[i].QDIM!=PSI1[0].QDIM)
443         wblog(FL, "ERR Severe QDIM inconsistency (%d,%d).",
444             PSI1[i].QDIM, PSI1[0].QDIM);
445
446     P.Index(PSI1[0].rank());
447     P.mvFirstTo(ic);
448
449     for (i=0; i<n; i++)
450         m += getCtrIndex(i, PSI1[i], tflag);
451
452     return (m!=0);
453 }
454
455 template <class TQ, class TD>
456 int CPAT_Set<TQ,TD>::getCtrIndex(
457     const unsigned i,
458     const QSpace<TQ,TD> &PSI,
459     char tflag
460 ){
461     int e;
462     unsigned ica = tflag=='N' ? 2 : (tflag=='T' ? 1 : 0);
463
464     QSpace<TQ,TD> &CQi = (tflag=='N' ? CQn[i] : CQt[i]);
465     wbMatrix<unsigned> &Cii = (tflag=='N' ? CIn[i] : CIt[i]);
466

```

```

467   QSpace<TQ,TD> CQ0;
468   const wbMatrix<unsigned> CIO(CIi);
469
470   if (!lica) wblog(FL, "ERR Invalid flag %c<%d>",tflag,tflag);
471
472   CQ0.QDIM=CQi.QDIM; CQi.QDIM=PSI.QDIM;
473   CQ0.QIDX=CQi.QIDX;
474
475   e=A[i].contract_getIdxSet(ica, PSI, ic+1, CQi.QIDX, CIi);
476   if (e<0) wbererror(FL,str);
477
478   CQi.PermuteQ(P);
479
480   return (CQi.QDIM!=CQ0.QDIM || CQi.QIDX!=CQ0.QIDX || CIi!=CIO);
481 }
482
483 #ifdef WB_CLOCK
484   Wb::Clock wbc_hzt("HZTimes");
485 #endif
486
487 template <class TQ, class TD>
488 QSpace<TQ,TD>& CPAT<TQ,TD>::HZTimes(
489   const QSpace<TQ,TD> &PSI,
490   QSpace<TQ,TD> &HZPsi,
491   const char tflag,
492   char disp
493 ){
494   unsigned i,j,n;
495
496   #ifdef WB_CLOCK
497     wbc_hzt.resume();
498   #endif
499
500   wbvector< QSpace<TQ,TD> > X1,X2;
501
502   if (!Wb::strchr("NTC",tflag))
503     wblog(FL,"ERR Invalid flag %c<%d>", tflag, tflag);
504
505   check_contraction(PSI,disp);
506   HPsi_counter++;
507
508   HZPsi=PSI;
509   HZPsi *= (tflag=='C' ? -CONJ(z) : -z);
510
511   for (i=0; i<CP.len; i++) {
512
513     CP[i][0].contract(PSI, X1, tflag);
514
515     for (n=CP[i].len, j=1; j<n; j++) {
516       X1.save2(X2);
517       CP[i][j].contract(X2,X1,tflag);
518     }
519
520     for (j=0; j<X1.len; j++)
521       X1[j].Append2AndDestroy(FL,HZPsi);
522   }
523
524   HZPsi.makeUnique();
525
526   #ifdef WB_CLOCK
527     wbc_hzt.stop();
528   #endif
529
530   return HZPsi;
531 };
532
533 template <class TQ, class TD>
534 QSpace<TQ,TD>& CPAT<TQ,TD>::HZApplyDiag(
535   const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &EPsi,
536   TD a, TD b, char Iflag, char disp
537 ){
538   unsigned i,j,n;
539
540   wbvector< QSpace<TQ,TD> > X1,X2;
541
542   check_contraction(PSI,disp);
543
544   EPsi.init();
545
546   for (i=0; i<CP.len; i++) {
547
548     CP[i][0].contract_diag(PSI, X1, a,b,Iflag);
549
550     for (n=CP[i].len, j=1; j<n; j++) {
551       X1.save2(X2);
552       CP[i][j].contract_diag(X2,X1, a,b,Iflag);
553     }

```

```

554
555     for (j=0; j<Xl.len; j++) {
556         Xl[j].SkipEmptyData();
557         Xl[j].Append2AndDestroy(FL,Epsi);
558     }
559 }
560
561 Epsi.makeUnique();
562
563 if (PSI.QIDX!=Epsi.QIDX) {
564     wblog(FL,
565         "WRN QIDX different for PSI and Epsi !? (%d/%d)",
566         Epsi.QIDX.dim1,PSI.QIDX.dim1);
567     wblog(FL,"TST a=%g, b=%g, '%c'",a,b,Iflag);
568 #ifdef MATLAB_MEX_FILE
569     PSI.put("PSI"); Epsi.put("Epsi");
570 #endif
571 }
572
573 return Epsi;
574 };
575
576 template <class TQ, class TD>
577 void CPAT<TQ,TD>::check_contraction(
578     const QSpace<TQ,TD> &PSI, char disp
579 ){
580     if (!nr_check_ctr) getCtrIndex(PSI); else
581     if (sameCtrIdx<nr_check_ctr) {
582         unsigned i=getCtrIndex(PSI);
583         if (i==0) {
584             sameCtrIdx++;
585
586             if (sameCtrIdx==nr_check_ctr && disp>10)
587                 wblog(FL,"<i> using sameCtrIdx=%d", sameCtrIdx);
588         }
589         else {
590             if (sameCtrIdx && disp)
591                 wblog(FL,"WRN sameCtrIdx=%d reset to 0.", sameCtrIdx);
592
593             sameCtrIdx=0;
594         }
595     }
596     else ARG_CHECK=0;
597 };
598
599 template <class TQ, class TD>
600 inline void CPAT<TQ,TD>::getQRange(
601     unsigned k,
602     wbMatrix<TQ> &QQ,
603     wbvector<unsigned> &SS,
604     QSpace<TQ,TD> *B
605 ) const {
606
607     unsigned i,j,l,n,nA,m=CP.len; int e=0;
608     wbvector<unsigned> S,D;
609     wbMatrix<TQ> Q;
610     wbperm P;
611
612     for (i=0; i<m; i++) { n=CP[i].len;
613         for (j=0; j<n; j++) { nA=CP[i][j].A.len; ;
614             if (CP[i][j].ic!=k) continue;
615
616             for (l=0; l<nA; l++) {
617                 CP[i][j].A[l].getQDim(Q,S);
618                 QQ.cat(l,Q);
619                 SS.Append(S);
620             }
621         }
622     }
623
624     if (B) {
625         B->getQDim(k,Q,S);
626         QQ.cat(l,Q);
627         SS.Append(S);
628     }
629
630     QQ.groupRecs(P,D);
631     e=SS.set2Group(P,D,0,'i');
632
633     if (e) wblog(FL,
634         "ERR QSpace size inconsistency (dim=%d): %s (%d)",k+1,str,e);
635 }
636
637 template <class TQ, class TD>
638 inline void QSpace<TQ,TD>::ExpandQ(
639     CPAT<TQ,TD> &CP,
640     double rnm,
641     wbvector<unsigned> &xflag,

```

```

641 char disp
642 ){
643     unsigned i,j,k,l,n,nr,s,r=rank(FL), dmin=0;
644     const char EXISTS=1, OLDSYM=1<<1, XPAND=1<<2, XPAND2=1<<3, rank2=(r<3);
645     int e=0;
646
647     wbvector< wbMatrix<TQ> > QQ;
648     wbvector< wbvector<unsigned> > SS;
649     wbvector<unsigned> D(r),S(r),dc;
650     wbMatrix<unsigned> SA;
651     wbvector<char> mark;
652     wbMatrix<TQ> Q1,Q2,QK1,QK2;
653     QSpace<TQ,TD> AA;
654     wbindex I1,I2,d1,d2,I(r);
655     wbp perm p1,p2;
656     TD n1,n2;
657
658     QQ.initDef(r);
659     SS.initDef(r);
660
661     for (i=0; i<r; i++) {
662         CP.getQRange(i,QQ[i],SS[i], this);
663
664         S[i]=SS[i].len;
665
666         D[i]=SS[i].sum();
667         if (D[i]>1) {
668             if (dmin==0) dmin=D[i]; else dmin=MIN(dmin,D[i]); }
669     }
670
671     if (xflag.isEmpty()) {
672         if (!rank2) {
673             xflag.init(r);
674             for (i=0; i<r; i++) if (D[i]<=dmin) xflag[i]=1;
675         }
676     }
677     else if (xflag.len!=r) wblog(FL,
678 "ERR %s - invalid xflag (dim. mismatch %d/%d)",FCT,xflag.len,r);
679
680     if (disp>9)
681 wblog(FL,"TST D=[%s], expand=[%s] (%d)",
682 D.toString().data, xflag.toString().data, dmin);
683
684     s=S.prod(); I.set(0);
685     AA.init(s,r,QDIM); SA.init(s,r); mark.init(s);
686
687     for (l=S.len-1, i=0; i<s; i++) {
688         for (k=0; k<I.len; k++) {
689             AA.setQRec(i, k, QQ[k].rec(I[k]));
690             SA(i,k)=SS[k][I[k]];
691         }
692
693         I[l]++; k=1;
694         while(I[k]>=S[k] && k>0) { I[k]=0; ++I[--k]; }
695     }
696
697     QIDX.blockSum(QDIM,Q1); Q1.makeUnique();
698     AA.QIDX.blockSum(QDIM,Q2); Q2.groupRecs(p2,d2);
699
700     e=matchIndex(Q1,Q2,I1,I2);
701     if (e || I1.len!=Q1.dim1) wblog(FL,
702 "ERR Failed to locate all existing symmetry sectors\n"
703 "(e=%d; len=%d/%d)", e, I1.len, Q1.dim1);
704
705     d2.cumsum_(dc);
706     for (i=0; i<I2.len; i++) {
707         l=dc[I2[i]]; n=d2[I2[i]];
708         for (j=0; j<n; j++) mark[p2[l+j]] |= OLDSYM;
709     }
710
711     if (disp>9) sprintf(str,
712 "Psi having %d/%d symmetry sector(s)", Q1.dim1, Q2.dim1);
713
714     xflag.find(0,I); if (!I.isEmpty()) {
715         getQsub(I,QK1); QK1.groupRecs(p1,d1);
716         AA.getQsub(I,QK2); QK2.groupRecs(p2,d2);
717
718         e=matchIndex(QK1,QK2,I1,I2);
719         if (e || I1.len!=QK1.dim1) wblog(FL,
720 "ERR Failed to locate all existing symmetry sectors (%d/%d)",
721 I1.len, QK1.dim1);
722
723         d2.cumsum_(dc);
724         for (i=0; i<I2.len; i++) {
725             l=dc[I2[i]]; n=d2[I2[i]];
726             for (j=0; j<n; j++) mark[p2[l+j]] |= XPAND;
727         }

```



```

728     }
729     else {
730         for (i=0; i<mark.len; i++) mark[i] |= XPAND;
731     }
732
733     e=matchIndex(QIDX, AA.QIDX, I1, I2);
734     if (e || I1.len!=QIDX.dim1) wblog(FL,
735         "ERR Failed to locate all existing data blocks (%d/%d)",
736         I1.len, QIDX.dim1);
737
738     for (i=0; i<I2.len; i++) {
739         mark[I2[i]] |= EXISTS;
740         DATA[I1[i]]->save2(*(AA.DATA[I2[i]]));
741
742         if (!SA.recEqual(I2[i], AA.DATA[I2[i]]->SIZE.data)) {
743             SA.recPrint(i, "SA");
744             AA.DATA[I2[i]]->SIZE.print("SIZE");
745             wblog(FL, "ERR Severe size inconsistency (%d)", i+1);
746         }
747     }
748
749     if (rnrn<=0) {
750         for (i=0; i<mark.len; i++)
751             if (mark[i]>0 && !(mark[i] & OLDSYM)) mark[i]=-mark[i];
752     }
753
754     for (n=i=0; i<mark.len; i++) if (mark[i]>0) n++;
755
756     init(n,r,QDIM);
757     I1.init(n);
758
759     for (nr=l=i=0; i<mark.len; i++) {
760         if (mark[i]<=0) continue;
761
762         QIDX.recSetP(1, AA.QIDX.rec(i));
763
764         if (AA.DATA[i]->isEmpty()) {
765             S.init2ref(SA.dim2, SA.rec(i));
766             DATA[l]->init(S);
767             if (!(mark[i] & OLDSYM)) {
768                 DATA[l]->setRand('s');
769                 I1[l]=1; nr++;
770             }
771         }
772         else AA.DATA[i]->save2(*DATA[l]);
773
774         l++;
775     }
776
777     if (rnrn>0 && nr) {
778         double dbl,fac=0,r2=rnrn*rnrn*1E-3;
779         if (fabs(rnrn)>1) wblog(FL,"WRN rnrn = %g !?",rnrn);
780
781         for (n1=n2=0,i=0; i<I1.len; i++) {
782             if (I1[i]) n2+=(DATA[i]->norm2());
783             else {
784                 dbl=(DATA[i]->norm2()); fac+=dbl;
785                 if (dbl>r2) n1+=dbl;
786                 else {
787                     DATA[i]->setRand('s');
788                     I1[i]=2; n2+=(DATA[i]->norm2());
789                 }
790             }
791         }
792
793         if (n1!=0 && n2!=0) {
794             r2=rnrn*rnrn; fac/=(n1+r2);
795
796             if (disp>9) wblog(FL,
797                 "TST n1=%g, n2=%g, r2=%g -> fac=%g",n1,n2,r2,fac);
798
799             n1=(TD)sqrt(fac*n1);
800             n2=(TD)sqrt(fac*(r2/n2));
801
802             for (i=0; i<I1.len; i++)
803                 if (I1[i]) (*DATA[i])*=n2; else (*DATA[i])*=n1;
804         }
805         else wblog(FL,"WRN rand. norm = %.3g (%.3g) !?",n2,n1);
806     }
807
808     if (xflag.count(0)) {
809
810         unsigned itry=0,iTRY=0, ntry=16, nQ=1, m=CP.CP.len, mx=0, m0=QIDX.dim1;
811         QSpace<TQ,TD> AX;
812
813         Q1.init(); Q2.init();
814

```

```

815     for (iTRY=0; iTRY<ntry && nQ; iTRY++) {
816     for (itry=0; itry<ntry; itry++) {
817         CP.getCtrIndex(*this);
818
819         for (i=0; i<m; i++) {
820             j=CP.CP[i].len-1;
821             nQ=CP.CP[i][j].CQn.len;
822
823             for (l=0; l<nQ; l++)
824                 Q1.cat(1,
825                     CP.CP[i][j].CQn[l].QIDX
826                 );
827         }
828
829         Q1.makeUnique();
830         if (Q1==Q2) break; else Q1.save2(Q2);
831     }
832
833     if (itry==ntry) wblog(FL,
834         "ERR CP|psi> keeps generating new terms (%d).\n"
835         "Symmetries not preserved !?", ntry);
836     if (Q1.dim2!=QIDX.dim2) wblog(FL,
837         "ERR Severe QIDX dimension mismatch (%d/%d)", Q1.dim2, QIDX.dim2);
838
839     e=matchIndex(Q1, AA.QIDX, I1, I2);
840
841     if (e || I1.len!=Q1.dim1) wblog(FL,
842         "ERR Failed to locate all existing data blocks (%d/%d)",
843         I1.len, Q1.dim1);
844
845     for (nQ=i=0; i<I2.len; i++) if (mark[I2[i]]<=0) nQ++;
846     if (nQ) {
847         l=QIDX.dim1; if (l!=DATA.len) wblog(FL, "%d/%d ???", l, DATA.len);
848         QIDX.Resize(l+nQ, QIDX.dim2);
849         DATA.Resize(l+nQ);
850         for (i=0; i<nQ; i++) DATA[l+i] = new wbararray<TD>;
851
852         for (i=0; i<I2.len; i++) {
853             j=I2[i]; if (mark[j]>0) continue;
854             if (mark[j]<0) mark[j]=-mark[j]; else
855                 mark[j] |= XPAND2; mx++;
856
857             QIDX.recSetP(1, AA.QIDX.rec(j));
858
859             if (AA.DATA[j]->isEmpty()) {
860                 S.init2ref(SA.dim2, SA.rec(j));
861                 DATA[l]->init(S);
862             }
863             else {
864                 wblog(FL, "WRN excuse moi !?");
865                 AA.DATA[j]->save2(*DATA[l]);
866             }
867
868             l++;
869         }
870     }
871
872     if (disp>9) wblog(FL, "TST itry=%d, iTRY=%d (%d)", itry, iTRY, ntry);
873
874     if (iTRY==ntry) wblog(FL,
875         "ERR CP|psi> keeps generating new terms (%d).\n"
876         "Symmetries not preserved !?", ntry);
877
878     if (disp>9)
879         wblog(FL, "TST D=[%s], %d/%d completing to %d+%d blocks",
880             getDim().toStr().data, iTRY+1, itry+1, m0, mx);
881 }
882
883 if (disp>9)
884     wblog(FL, "TST %s -> %d", str, getDQtot());
885 }
886
887 template <class TQ, class TD>
888 QSpace<TQ, TD>& CPAT<TQ, TD>::HZTimes_tst(
889     const QSpace<TQ, TD> &PSI,
890     QSpace<TQ, TD> &HZPsi,
891     const char tflag,
892     char disp
893 ){
894     unsigned i, j, it;
895     static unsigned call=0;
896     wbpem P;
897
898     if (!(call++)) wblog(FL, "TST %N*** using HZTimes_tst ***%N");
899
900     QSpace<TQ, TD> X1, X2, XX;
901

```

```

902     if (tflag=='C' && ISREAL(TD)) tflag='T';
903
904     if (!Wb::strchr("NTC",tflag)) wblog(FL,
905     "ERR Invalid flag %c<%d>", tflag, tflag);
906
907     it=( tflag=='N' ? 2 : ( tflag=='T' ? 1 : -1 ));
908
909     HZPsi=PSI;
910     HZPsi *= (tflag=='C' ? -CONJ(z) : -z);
911
912     for (i=0; i<CP.len; i++) {
913
914         if (CP[i].len<=1) {
915             CPAT_Set<TQ,TD> &C1=CP[i][0];
916             for (j=0; j<C1.A.len; j++) {
917                 P.initFirstTo(C1.ic, PSI.rank());
918                 if (tflag!='C')
919                     C1.A[j].contract(it,PSI, C1.ic+1, X1, P);
920                 else C1.A[j].opA(tflag,XX).contract(2, PSI, C1.ic+1, X1, P);
921
922                 HZPsi+=X1;
923             }
924         }
925         else {
926             CPAT_Set<TQ,TD> &C1=CP[i][0], &C2=CP[i][1];
927             if (C1.A.len!=C2.A.len || CP[i].len>2) wblog(FL,"ERR ???");
928
929             for (j=0; j<C1.A.len; j++) {
930                 P.initFirstTo(C1.ic, PSI.rank());
931                 if (tflag!='C')
932                     C1.A[j].contract(it,PSI, C1.ic+1, X1, P);
933                 else C1.A[j].opA(tflag,XX).contract(2, PSI, C1.ic+1, X1, P);
934
935                 P.initFirstTo(C2.ic, PSI.rank());
936                 if (tflag!='C')
937                     C2.A[j].contract(it,X1 , C2.ic+1, X2, P);
938                 else C2.A[j].opA(tflag,XX).contract(2, X1 , C2.ic+1, X2, P);
939
940                 HZPsi+=X2;
941             }
942         }
943     }
944
945     return HZPsi;
946 };
947
948 template <class TQ, class TD>
949 inline void CPAT_Set<TQ,TD>::contract (
950     const QSpace<TQ,TD> &PSI,
951     wbvector< QSpace<TQ,TD> > &C,
952     char tflag
953 ) const {
954
955     unsigned i,n=A.len;
956     if (C.len!=n) C.initDef(n);
957
958     for (i=0; i<n; i++)
959         contract(i,PSI,C[i],tflag);
960 }
961
962 template <class TQ, class TD>
963 inline void CPAT_Set<TQ,TD>::contract_diag (
964     const QSpace<TQ,TD> &PSI,
965     wbvector< QSpace<TQ,TD> > &C,
966     TD a, TD b, char Iflag
967 ) const {
968
969     unsigned i,n=A.len;
970     if (C.len!=n) C.initDef(n);
971
972     for (i=0; i<n; i++)
973         contract_diag(i,PSI,C[i],a,b,Iflag);
974 }
975
976 template <class TQ, class TD>
977 inline void CPAT_Set<TQ,TD>::contract (
978     const wbvector< QSpace<TQ,TD> > &PSI1,
979     wbvector< QSpace<TQ,TD> > &C,
980     char tflag
981 ) const {
982
983     unsigned i,n=A.len;
984     if (C.len!=n) C.initDef(n);
985
986     for (i=0; i<n; i++)
987         contract(i,PSI1[i],C[i],tflag);
988 }

```

```

989
990 template <class TQ, class TD>
991 inline void CPAT_Set<TQ,TD>::contract_diag(
992     const wbvector< QSpace<TQ,TD> > &PSI1,
993     wbvector< QSpace<TQ,TD> > &C,
994     TD a, TD b, char Iflag
995 ) const {
996
997     unsigned i,n=A.len;
998     if (C.len!=n) C.initDef(n);
999
1000     for (i=0; i<n; i++)
1001         contract_diag(i,PSI1[i],C[i],a,b,Iflag);
1002 }
1003
1004 #ifdef WB_CLOCK
1005     Wb::Clock wbc_cpat_ctr("cpat::contract");
1006 #endif
1007
1008 template <class TQ, class TD>
1009 void CPAT_Set<TQ,TD>::contract(
1010     const unsigned k,
1011     const QSpace<TQ,TD> &PSI,
1012     QSpace<TQ,TD> &C,
1013     char tflag
1014 ) const {
1015
1016     unsigned i,n,ia,ib,iu,it;
1017     wbMatrix<unsigned> &Iabu = (tflag=='N' ? CIN[k] : CIT[k]);
1018     wbarray<TD> XX;
1019
1020 #ifdef WB_CLOCK
1021     wbc_cpat_ctr.resume();
1022 #endif
1023
1024     if (tflag=='C' && ISREAL(TD)) tflag='T';
1025
1026     if (!Wb::strchr("NTC",tflag)) wblog(FL,
1027         "ERR Invalid flag %c<%d>",tflag,tflag);
1028
1029     C.QDIM=PSI.QDIM;
1030     C.QIDX=(tflag=='N' ? CQn[k].QIDX : CQt[k].QIDX);
1031     C.setupDATA();
1032
1033     it=( tflag=='N' ? 2 : ( tflag=='T' ? 1 : -1 ));
1034
1035     for (n=Iabu.dim1, i=0; i<n; i++) {
1036         ia=Iabu(i,0); ib=Iabu(i,1); iu=Iabu(i,2);
1037
1038         if (tflag!='C') {
1039             A[k].DATA[ia]->
1040                 contract(it,*(PSI.DATA[ib]), ic+1, *(C.DATA[iu]), P); }
1041         else {
1042             A[k].DATA[ia]->opA(tflag,XX).
1043                 contract(2, *(PSI.DATA[ib]), ic+1, *(C.DATA[iu]), P);
1044         }
1045     }
1046
1047 #ifdef WB_CLOCK
1048     wbc_cpat_ctr.stop();
1049 #endif
1050 }
1051
1052 template <class TQ, class TD>
1053 void CPAT_Set<TQ,TD>::contract_diag(
1054     const unsigned k,
1055     const QSpace<TQ,TD> &PSI,
1056     QSpace<TQ,TD> &C,
1057     TD a, TD b, char Iflag
1058 ) const {
1059
1060     unsigned i,n,ia,ib,iu;
1061     wbMatrix<unsigned> &Iabu = CIN[k];
1062     wbarray<TD> XX;
1063
1064 #ifdef WB_CLOCK
1065     wbc_cpat_ctr.resume();
1066 #endif
1067
1068     C.QDIM=PSI.QDIM; C.QIDX=CQn[k].QIDX;
1069     C.setupDATA();
1070
1071     for (n=Iabu.dim1, i=0; i<n; i++) {
1072         ia=Iabu(i,0); ib=Iabu(i,1); iu=Iabu(i,2);
1073
1074         if (A[k].isDiagBlock(ia))
1075             PSI.DATA[ib]->contractDiag(

```

```

1076         ic+1,*A[k].DATA[ia], a,b, *C.DATA[iu], Iflag
1077     );
1078 }
1079
1080 #ifdef WB_CLOCK
1081     wbc_cpat_ctr.stop();
1082 #endif
1083 }
1084
1085 template <class TQ, class TD>
1086 mxArray* CPAT<TQ,TD>::toMx() const {
1087
1088     unsigned i,j,l,k, m=CP.len, n;
1089     mxArray *S, *C, *a;
1090
1091     C=mxCreateCellMatrix(m,1);
1092
1093     for (i=0; i<m; i++) {
1094         n=CP[i].len;
1095         S=mxCreateStructMatrix(1,n,0,NULL);
1096
1097         for (j=0; j<n; j++) {
1098             a=CP[i][j].toMx();
1099             k=mxGetNumberOfFields(a);
1100
1101             if (j==0)
1102                 for (l=0; l<k; l++) {
1103                     if (mxAddField(S,mxGetFieldNameByNumber(a,l))<0) wblog(FL,
1104                         "ERR Could not add field '%s'", mxGetFieldNameByNumber(a,l));
1105                 }
1106
1107                 for (l=0; l<k; l++) {
1108                     mxSetFieldByNumber(S, j, l,
1109                         mxDuplicateArray(mxGetFieldByNumber(a,0,l)));
1110                 }
1111
1112                 mxDestroyArray(a);
1113             }
1114             mxSetCell(C,i,S);
1115         }
1116
1117         S=mxCreateStructMatrix(1,1,0,NULL);
1118         mxAddField2Scalar(FL,S,"CP",C);
1119         mxAddField2Scalar(FL,S,"z",numtoMx(z));
1120
1121         return S;
1122     }
1123
1124 template <class TQ, class TD>
1125 mxArray* CPAT_Set<TQ,TD>::toMx() const {
1126
1127     mxArray *S, *a;
1128     unsigned i;
1129
1130     S=mxCreateStructMatrix(1,1,0,NULL);
1131
1132     mxAddField2Scalar(FL,S,"A", A.toMx() );
1133     mxAddField2Scalar(FL,S,"ic", numtoMx(ic+1));
1134     mxAddField2Scalar(FL,S,"opA", numtoMx(opA) );
1135     mxAddField2Scalar(FL,S,"P", (P+1).toMx() );
1136
1137     for (i=0; i<CQn.len; ++i) CQn[i].setupDATA();
1138     for (i=0; i<CQt.len; ++i) CQt[i].setupDATA();
1139
1140     mxAddField2Scalar(FL,S,"CQn", CQn.toMx());
1141     mxAddField2Scalar(FL,S,"CQt", CQt.toMx());
1142
1143     a=mxCreateCellMatrix(CIn.len,1);
1144     for (unsigned i=0; i<CIn.len; i++)
1145         mxSetCell(a, i, CIn[i].toMx());
1146     mxAddField2Scalar(FL,S,"CIn",a);
1147
1148     a=mxCreateCellMatrix(CIt.len,1);
1149     for (unsigned i=0; i<CIt.len; i++)
1150         mxSetCell(a, i, CIt[i].toMx());
1151     mxAddField2Scalar(FL,S,"CIt",a);
1152
1153     return S;
1154 }
1155
1156 int isCPAT(const mxArray *CC, char twoSite) {
1157     unsigned i,i0,k,n;
1158
1159     if (!CC || !mxIsCell(CC)) return 0;
1160     if (isCPattern(CC)) return 1;
1161
1162     n=mxGetNumberOfElements(CC);

```

```

1163     if (!n) return 0;
1164
1165     i0=isCPattern(mxGetCell(CC,0));
1166     if (!i0) return 0;
1167
1168     for (k=1; k<n; k++) {
1169         i=isCPattern(mxGetCell(CC,k));
1170         if (!i) return 0;
1171
1172         if (i!=i0) wblog(FL,
1173             "ERR CPAT mismatch twoSite=%d,%d ???", i0, i);
1174     }
1175
1176     if (twoSite>=0) if (1+twoSite!=(char)i0) wblog(FL,
1177         "ERR CPAT is not of type %d-site (%d)", 1+twoSite, i0);
1178
1179     return n;
1180 }
1181
1182 int isCPattern(const mxArray *C) {
1183     unsigned k,n,w;
1184     char twoSite, hasOp;
1185     mxArray *a;
1186
1187     if (!C || !mxIsCell(C)) {
1188         sprintf(str,"%s:%d invalid CP",FL);
1189         return 0;
1190     }
1191
1192     n=mxGetNumberOfElements(C);
1193     if (n<2 || n>4) {
1194         sprintf(str,"%s:%d invalid CP (%d entries)",FL,n);
1195         return 0;
1196     }
1197
1198     a=mxGetCell(C,0);
1199     if (!mxIsStruct(a) || mxIsCell(a)) {
1200         sprintf(str,"%s:%d invalid arg #1",FL);
1201         return 0;
1202     }
1203
1204     a=mxGetCell(C,n-1);
1205     hasOp=mxIsChar(a);
1206
1207     if (hasOp) if (mxGetNumberOfElements(a)!=1) {
1208         sprintf(str,"%s:%d arg #%d not a flag (char)",FL,n);
1209         return 0;
1210     }
1211
1212     k=n-hasOp;
1213     if (k==2) twoSite=0; else
1214     if (k==3) twoSite=1; else {
1215         sprintf(str,"%s:%d invalid CP (k=%d)",FL,k);
1216         return 0;
1217     }
1218
1219     if (twoSite) {
1220         w = (mxIsEmpty(mxGetCell(C,1)) ? 0 : 1) +
1221             (mxIsEmpty(mxGetCell(C,2)) ? 0 : 2);
1222         if (w<1 || w>2) {
1223             sprintf(str, "%s:%d either arg #2 or #3",FL);
1224             return 0;
1225         }
1226     }
1227     else w=1;
1228
1229     a=mxGetCell(C,w);
1230     if (!Mx::IsNumber(a)) {
1231         sprintf(str,"%s:%d arg #%d not a number",FL,w+1);
1232         return 0;
1233     }
1234
1235     return (1+twoSite);
1236 }
1237
1238 #endif
1239

```

5.3 gathered.hh

```

1 #ifndef __WB_GATHERED_HH__
2 #define __WB_GATHERED_HH__
3

```

```

4  /* ----- */
5  /* ----- */
6
7  namespace Wb {
8      const char* basename(const char *data, char c='/');
9
10     void print_backtrace(
11         const char *F=NULL, int L=0, const char *istr=NULL);
12
13     int GetNumThreads(const char *F, int L, int &n, const char *name);
14
15     template<class T>
16     int GetEnv (const char *F, int L, const char *name, T& val);
17
18     int EnvIsSet(const char *F, int L, const char *name);
19
20     wbsting repHome(const char *file);
21
22     inline int INT(const double a) {
23         int i=:round(a);
24         if (std::fabs(i-a)>1E-10) wblog(FL,"ERR %s(%g) !?",FCT,a);
25         return i;
26     }
27
28     const char* strstri (const char *s1, const char *s2);
29     const char* strstrw (const char *s1, const char *s2);
30     const char* strstrwi(const char *s1, const char *s2);
31     const char* strchrri(const char *s, char c);
32
33     void shift(char *s, unsigned n, int p, char erase=0);
34
35     void shift(char *s, unsigned n, int p, const char *srep);
36
37     bool isword(const char &c) { return (
38         (c>='0' && c<='9') ||
39         (c>='a' && c<='z') ||
40         (c>='A' && c<='Z') || c=='_'
41     ); };
42
43     template <class T>
44     double mod(double a, T b) {
45         if (b<=0) wblog(FL,"ERR %s() got b=%g",FCT,b);
46         a=:fmod(a,double(b)); if (a<0) { a+=b; }
47         return a;
48     };
49
50     template <class T, ENABLE_IF_isINT(T)>
51     int mod(int a, T b) {
52         if (b<=0) wblog(FL,"ERR %s() got b=%d",FCT,b);
53         a %= int(b); if (a<0) { a+=b; }
54         return a;
55     };
56
57     unsigned strlen(const char *s, unsigned n) {
58         for (unsigned i=0; i<n; ++i) if (!s[i]) return i;
59         return n;
60     };
61
62     template <class T>
63     bool isLower(const T* a, const T* b, const size_t n);
64
65     template <class T>
66     bool isEqual(const T* a, const T* b, const size_t n);
67
68     template <class T>
69     bool allEqual(const T* a, const size_t n, const T &x);
70
71     template <class T>
72     bool anyEqual(const T* a, const size_t n, const T &x);
73
74     template <class T>
75     bool anyUnqual(const T* a, const size_t n, const T &x);
76
77     template <class T>
78     T getscale(const T* d, size_t n);
79
80     template <class T> inline
81     void scale_eps(T &eps, const T* d, size_t n);
82
83 }
84
85 template <class T, class T2> inline
86 char GOT_EPS(const T &a, const T2 &eps) {
87     if (a==0 || (eps!=0 && ABS(a)<eps)) return 1;
88     return 0;
89 }
90

```

```

91 void initSigHandler(const char i='i');
92 void WBSigHandler(int sig);
93
94 char *gsh_F=NULL; int gsh_L=0;
95
96 class wbSigHandler {
97
98 public:
99     wbSigHandler(const char *F, int L) {
100         if (F && F[0]) {
101             const char *s=strrchr(F,'/'); if (!s) { s=F; }
102             gsh_F = new char[strlen(s)+1]; strcpy(gsh_F,s);
103             gsh_L=L;
104         }
105         else {
106             const char s0[]=__FILE__, *s=strrchr(s0,'/'); if (!s) { s=s0; }
107             gsh_F = new char[strlen(s)+1]; strcpy(gsh_F,s);
108             gsh_L=__LINE__;
109         }
110         #ifdef DBSTOP
111         #else
112             initSigHandler('i');
113         #endif
114     };
115
116     ~wbSigHandler() {
117         #ifdef DBSTOP
118         #else
119             initSigHandler('r');
120         #endif
121         gsh_L=0; if (gsh_F) { delete [] gsh_F; gsh_F=NULL; }
122     };
123
124     int check911(char tflag=0) { doflush();
125         if (tflag<=0) {
126             #ifndef DBSTOP
127                 WBSigHandler(911);
128             #endif
129             return 0;
130         }
131         else { return check911_(); }
132     };
133
134     static int check911_(const char *F=NULL, int L=0) {
135         int i=wbSigHandler::icount,
136             a=wbSigHandler::acount, n=wbSigHandler::NC0;
137
138         if (i>=n) { i=0; } else { i=n-i; if (i>9) { i=9; }}
139         if (a>=n) { a=0; } else { a=n-a; if (a>9) { a=9; }}
140         if (F) { wblog(F_L,"ERR %s() ",FCT); }
141         return (i + 10*a);
142     };
143
144     static unsigned NC0, icount, acount;
145
146 protected:
147 private:
148
149 };
150
151 unsigned wbSigHandler::NC0=3;
152
153 unsigned wbSigHandler::icount=wbSigHandler::NC0;
154 unsigned wbSigHandler::acount=wbSigHandler::NC0;
155
156 template <class TX>
157 class wbREF {
158
159 public:
160
161     wbREF (const TX &A, char flag) : orig(&A), ref(0) {
162         if (flag) ref=new TX(A); else ref=&A;
163     };
164
165     TX& Ref() { return *ref; }
166
167     ~wbREF() { if (ref!=orig) delete ref; }
168
169 protected:
170 private:
171     const TX *orig, *ref;
172 };
173
174 namespace Wb {
175
176 template <class T>
177 class num2Fmt {

```



```

178
179 public:
180     num2Fmt(const char *F, int L,
181             int m_=-2, int p_=-2, const char *t_="" ) : m(m_), p(p_) {
182
183         if (t_ && t_[0]) {
184             if (strlen(t_)>2) wblog(F_L,"ERR %s() invalid t='%s'",FCT,t_);
185             strcpy(t,t_);
186         }
187         else { memset(t,0,3); }
188
189         check_init();
190     };
191
192     int check_init();
193
194     explicit operator wstring() const;
195
196 private:
197     char m,p;
198     char t[3];
199
200     void get_fmt(wstring &s) const;
201 };
202
203 template<class T> inline
204 int num2int(const char *F, int L, const T &x);
205
206 template<class T> inline
207 unsigned checkInt(
208     const char *F, int L, const T* x, size_t n,
209     double eps=1E-14
210 );
211
212 char* strpad(char *s, char c, unsigned n, unsigned w=1) {
213     unsigned i,l=strlen(s);
214     for (i=0; i<w; ++i) s[l++]= ' ';
215     for (i=l; i<n; ++i) s[l++]=c;
216     return s;
217 };
218
219 unsigned strpad_(char *s, unsigned n, char c=' ', int z=1) {
220     if (int(n)>0) {
221         unsigned i=0; for (; i<n; ++i) { s[i]=c; }
222         if (z) s[i]=0;
223         return i;
224     }
225     return 0;
226 };
227
228 template <class T>
229 int charGetNumber(const char *F, int L, const char *s, T &x);
230
231 bool isBaseType(const std::type_info &type_id);
232 bool isComplexType(const std::type_info &type_id);
233
234 wstring TimeStamp(char type=0);
235
236 void inl(unsigned n) { for (unsigned i=0; i<n; i++) printf("\n"); }
237
238 void quietErrLog(const char *F, int L, const char *msg, char qflag) {
239     if (qflag) snprintf(str,256,"%s [%s:%d]",msg,Wb::basename(F),L);
240     else wblog(F,L,"ERR %s",msg);
241 }
242
243 template <class T1, class T2> inline
244 void safeConvert(const char *F, int L, const T1 &x1, T2 &x2);
245
246 template <class T1, class T2> inline
247 int checkUpdate(T1& a, const T2 &r) {
248     if (a!=T1(r)) { a=T1(r); return 1; } else return 0;
249 };
250
251 template <class T>
252 bool uniformRange(const T* d, const size_t n);
253
254 template <class T>
255 T maxRange(const T* d, const size_t n);
256
257 template <class T>
258 T minRange(const T* d, const size_t n);
259
260 template <class T>
261 T addRange2(const T* d, size_t n, const size_t stride=1);
262
263 template <class T>
264 T addRange(const T* d, size_t n, const size_t stride=1);

```

```

265
266     template <class T>
267     void addRange(
268         const T* a, const T* b, T* c, size_t n,
269         const size_t stride=1
270     );
271
272     template <class T>
273     void addRange(const T* a, T* c, size_t n, const size_t stride=1);
274
275     template <class T>
276     void minusRange(
277         const T* a, const T* b, T* c, const size_t n,
278         size_t stride=1
279     );
280
281     template <class T>
282     void diffRange(
283         const T* a, const T* b, T* c, size_t n,
284         size_t stride=1
285     );
286
287     template <class T>
288     void setRange2avg(T* a, size_t m);
289
290     template <class T>
291     size_t nnzRange(const T* a, size_t m);
292
293     template <class T, class T2> inline
294     void cpyStride(
295         T2* d,
296         const T* d0,
297         unsigned n,
298         const unsigned *idx, unsigned m,
299         unsigned D2=-1,
300         unsigned D0=-1,
301         char add_flag=0
302     );
303
304     template <class T, class T2> inline
305     void cpyStride(
306         T2* d,
307         const T* d0,
308         unsigned n,
309         unsigned m,
310         unsigned D2=-1,
311         unsigned D0=-1,
312         char add_flag=0
313     ){
314         return cpyStride(d,d0,n,(unsigned*)NULL,m,D2,D0,add_flag);
315     };
316
317     template <class T>
318     void cpyRange(T* a, const T* b, size_t n);
319
320     template <class T, class Tb>
321     void cpyRange(T* a, const Tb* b, size_t n, char check_type=1);
322
323     template <class T>
324     void cpyRangeR(T* a, const wbcomplex* z, size_t n);
325     template <class T>
326     void cpyRangeI(T* a, const wbcomplex* z, size_t n);
327     template <class T>
328     void cpyRangeA(T* a, const wbcomplex* z, size_t n);
329
330     template <class T>
331     T prodRange(const T* d, const size_t n);
332     template <class T>
333     T prodRange(const T* d, const size_t n, T x0);
334
335     template <class T>
336     T prodRange(const T* a, const T* b, size_t n, char conj=0);
337     template <class T>
338     T prodRange(const T* a, const T* b, size_t n, T x, char conj=0);
339
340     template <class T> inline
341     void TimesElRange(T* a, const T* b, size_t n, char conj=0);
342
343     template <class T> inline
344     void timesElRange(T* c, const T* a, const T* b, size_t n, char conj=0);
345
346     template <class T>
347     void timesElRange_add(
348         T* c, const T* a, const T* b, size_t n, T bfac=T(1), char conj=0);
349
350     template <class TD, class TX> inline
351     void timesRange(TD* d, TX x, const size_t n, size_t stride=1);

```

```

352
353 template <class T>
354 T rangeNorm2(const T* d, size_t n, size_t *k=NULL);
355
356 template <class T>
357 T rangeNormDiff2(const T* d1, const T* d2, size_t n,
358   const T &fac, size_t *k=NULL
359 );
360
361 template <class T>
362 T rangeNormDiff2(const T* d1, const T* d2, size_t n);
363
364 template <class T>
365 T rangeMaxDiff(const T* d1, const T* d2, size_t n, size_t *k=NULL);
366
367 template<class T>
368 T overlap(
369   const T *v1, const T *v2, size_t n,
370   size_t stride=1, char tnorm=0
371 );
372
373 template<class T>
374 T gs_project_range(
375   T *v1, const T *v2,
376   size_t n, size_t stride=1, char isnorm=0, char tnorm=0
377 );
378
379 template<class T>
380 void chopTiny_float(T *d, size_t n, T dref=-1) { return; };
381
382 template<class T>
383 void chopTiny_z(T *d, size_t n, double eps=1E-14) { return; };
384
385 template<class T>
386 double chopTiny_imag(T *d, size_t n, double eps=1E-14) { return 0; };
387
388 template <class T> inline
389 int recCompare(
390   const T* a, const T* b, const size_t n, char lex=1
391 ){
392   if (lex>0) {
393     for (size_t i=0; i<n; ++i) {
394       if (a[i]<b[i]) return -1; else
395       if (a[i]>b[i]) return +1;
396     }
397   }
398   else {
399     for (size_t i=n-1; i<n; --i) {
400       if (a[i]<b[i]) return -1; else
401       if (a[i]>b[i]) return +1;
402     }
403   }
404   return 0;
405 };
406
407 template <class T> inline
408 int recCompare(
409   const T* a, const T* b, const size_t n, char lex,
410   T eps, int *nwrn=NULL
411 ){
412   if (eps<=T(0)) return recCompare(a,b,n,lex);
413
414   int nwrn0=0, *nwrn=(nwrn_ ? nwrn_ : &nwrn0);
415   T x=0.2*eps;
416
417   if (lex>0) {
418     for (size_t i=0; i<n; ++i) {
419       if (a[i]<b[i]-eps) return -1; else
420       if (a[i]>b[i]+eps) return +1; else
421       if (!nwrn[0] && std::fabs(a[i]-b[i])>x) {
422         wblog(FL,"WRN %s() got |a-b|/eps=%3g (%d/%d)",
423           FCT, (a[i]-b[i])/eps,i+1,n); ++nwrn[0];
424       }
425     }
426   }
427   else {
428     for (size_t i=n-1; i<n; --i) {
429       if (a[i]<b[i]-eps) return -1; else
430       if (a[i]>b[i]+eps) return +1; else
431       if (!nwrn[0] && std::fabs(a[i]-b[i])>x) {
432         wblog(FL,"WRN %s() got |a-b|/eps=%3g (%d/%d)",
433           FCT, (a[i]-b[i])/eps,i+1,n); ++nwrn[0];
434       }
435     }
436   }
437   return 0;
438 };

```

```

439
440 template <class T>
441 int cmpRange(
442     const T *a, const T *b, const size_t n, char lex=+1
443 ){
444     if (lex>0) {
445         for (size_t i=0; i<n; ++i) { if (a[i]!=b[i]) {
446             return (a[i]<b[i] ? -1 : +1);
447         }}
448         return 0;
449     }
450     else {
451         if (!lex) wblog(FL,"WRN %s() got lex=%d",FCT,lex);
452         for (size_t i=n-1; i<n; --i) { if (a[i]!=b[i]) {
453             return (a[i]<b[i] ? -1 : +1);
454         }}
455         return 0;
456     }
457 };
458
459 template <class T> void cpyZRange(
460     const double *R, const double *I, T *Z, const size_t n);
461
462 template <class T> void splitZRange(
463     const T *Z, double *R, double *I, const size_t n);
464
465 template<class T>
466 void invertIndex(
467     const wbvector<T> &i1, unsigned N,
468     wbvector<T> &i2, char lflag=1
469 );
470
471 void setRand(wbvector<wbcomplex> &zz, double fac=1., double shift=0.);
472
473 template<class T>
474 void set2avg(T *dd, const size_t n);
475
476 template<class T>
477 void getDiff(const wbvector<T> &xx, wbvector<T> &dx);
478
479 template<class TX, class TY>
480 TY IntTrapez(const TX *xx, const TY *yy, const size_t n);
481
482 inline bool contains(const char* s, const char &x, const unsigned &m=16) {
483     if (s)
484         for (unsigned i=0; s[i] && i<m; i++) { if (s[i]==x) return 1; };
485     return 0;
486 };
487 };
488
489 };
490
491 template<class T>
492 void markSet(
493     wbvector< wbvector<T>* > &E0,
494     wbvector< wbvector<char> > &mark,
495     unsigned Nkmin,
496     int &Nkeep,
497     double Etrunc,
498     wbvector<T> &E,
499     double eps=0.,
500     double b=0.,
501     int dmax=-1,
502     const char *dir=0
503 );
504
505 template<class T>
506 void markSet(const char *F, int L,
507     const wbvector<T> &E,
508     wbvector<char> &mark,
509     unsigned Nkmin,
510     int &Nkeep,
511     double Etrunc=0,
512     double b=0.,
513     int dmax=-1,
514     const char *dir=0
515 );
516
517 #endif
518

```

5.4 memlib.hh

```
1 #ifndef __WB_MEM_LIB_HH__
```

```

2 #define __WB_MEM_LIB_HH__
3
4 // ----- //
5 // ----- //
6
7 template <class T1, class T2>
8 bool gotMemOverlap(
9     const T1* src, size_t len0,
10    const T2* dest, size_t len2=-1 // default -1 => len0
11 );
12
13 template <class T>
14 inline void MEM_COPY_BASE(T* data, size_t n, T const* const d=NULL) {
15
16 #ifdef MLB_VFLAG
17     wblog(FL,"TST base init");
18 #endif
19     if (n) {
20         if (!d)
21             { memset((void*)data,0,n*sizeof(T)); }
22         else { memcpy((void*)data,d,n*sizeof(T)); }
23     }
24 };
25
26 template <class T>
27 inline void MEM_COPY_BASE(
28     T* data, size_t n, size_t n1, T const* const d1, T const* const d2
29 ){
30
31 #ifdef MLB_VFLAG
32     wblog(FL,"TST base init %d->%d",n1,n);
33 #endif
34     if (n1) {
35         if (d1==NULL) wblog(FL,"ERR got (null) pointer"); else
36         if (n1>n) { wblog(FL,"WRN length out of bounds (%d/%d)",n1,n); n1=n; }
37         memcpy(data,d1,n1*sizeof(T));
38     }
39     if (n1<n) {
40         if (d2) memcpy(          data+n1, d2, (n-n1)*sizeof(T));
41         else    memset((void*)(data+n1), 0, (n-n1)*sizeof(T));
42     }
43 };
44
45 namespace Wb {
46
47 enum MTYPE {
48     MEM_DEF,
49     MEM_REF,
50     MEX_RETURN,
51     MEM_NUM_TYPES
52 };
53
54 const char* MTYPE_STR[MEM_NUM_TYPES] {
55     "MEM:default",
56     "MEM:ref",
57     "MEX:return"
58 };
59
60 template <class T>
61 class sptr {
62
63 public:
64
65     sptr() : mtype(MEM_DEF), len(0), nref(0), data(NULL) {};
66
67     ~sptr() {
68         #pragma omp critical (manage_SPTR)
69         { if (mtype || len || data || nref) wblog(FL,
70             "WRN %s() got %s",FCT,STR_(this));
71             init_def();
72         }
73     }
74
75     void init_def() {
76         mtype=MEM_DEF; len=nref=0; data=NULL;
77     }
78
79     T* malloc_base(size_t n, char init=1, MTYPE mt=MEM_DEF);
80     T* malloc_data(size_t n, const T *d0=NULL, MTYPE mt=MEM_DEF);
81
82     int rm_dref(char mflag=0);
83
84     unsigned add_dref();
85
86     T* save_dref(const char *F=0, int L=0);
87
88     T* init2ref(size_t n, const T* d0);

```

```

89
90 int isRef() const {
91     return (nref || mtype==Wb::MEM_REF);
92 };
93
94 explicit operator bool() const {
95     check_consistency(); return (len!=0); };
96 bool operator! () const {
97     check_consistency(); return (len==0); }
98
99 int check_consistency(char force=1) const {
100     int e=0;
101
102     #pragma omp critical (manage_SPTR)
103     { if (!len ^ !data) { e|=1; }
104       if (nref && !len) { e|=2; }
105     }
106
107     if (e && force) wblog(FL,
108         "ERR Wb::sptr() inconsistency (%p @ n=%d, r=%d)",data,len,nref);
109     return e;
110 };
111
112 wstring toStr() const;
113
114 MTYPE mtype;
115
116 size_t len;
117 unsigned nref;
118
119 T* data;
120
121 protected:
122 private:
123 };
124
125 };
126
127 template <class T, ENABLE_IF_isPOD(T) >
128 void MEM_COPY(T* data, size_t n, const T* d=NULL) {
129     MEM_COPY_BASE(data,n,d);
130 };
131
132 template <class T, ENABLE_IF_noPOD(T) >
133 void MEM_COPY(T* data, size_t n, const T* d=NULL) {
134
135     #ifdef MLB_VFLAG
136     wblog(FL,"TST class init");
137     #endif
138
139     if (d) { for (size_t i=0; i<n; ++i) data[i]=d[i]; }
140     else {
141         const T z=T();
142         for (size_t i=0; i<n; ++i) data[i]=z;
143     }
144 };
145
146 template <class T, ENABLE_IF_isPOD(T) >
147 void MEM_COPY(T* data, size_t n, size_t n1, const T* d1, const T* d2) {
148     MEM_COPY_BASE(data,n,n1,d1,d2);
149 };
150
151 template <class T, ENABLE_IF_noPOD(T) >
152 void MEM_COPY(T* data, size_t n, size_t n1, const T* d1, const T* d2) {
153     #ifdef MLB_VFLAG
154     wblog(FL,"TST class init %d->%d",n1,n);
155     #endif
156
157     size_t i=0;
158     if (n1) {
159         if (d1==NULL) wblog(FL,"ERR got (null) pointer"); else
160         if (n1>n) { wblog(FL,
161             "WRN length out of bounds (%d->%d)",n1,n); n1=n; }
162         for (; i<n1; ++i) data[i]=d1[i];
163     }
164     if (d2) { for (; i<n; ++i) { data[i]=d2[i-n1]; }}
165     else {
166         const T z=T();
167         for (; i<n; ++i) data[i]=z;
168     }
169 };
170
171 template <class T>
172 void MEM_COPY(T** data, size_t n, T* const* const d=NULL) {
173     #ifdef MLB_VFLAG
174     wblog(FL,"TST got pointer space");
175     #endif

```

```

176     MEM_COPY_BASE(data,n,d);
177 };
178
179 template <class T>
180 void MEM_COPY(T** data, size_t n, size_t n1,
181             T* const* const d1, T* const* const d2
182 ) {
183     #ifdef MLB_VFLAG
184         wblog(FL,"TST got pointer space");
185     #endif
186     MEM_COPY_BASE(data,n,n1,d1,d2);
187 };
188
189 template <class T> inline
190 void MEM_SET_BASE(T* data, size_t len, const T *a=NULL) {
191     #ifdef MLB_VFLAG
192         wblog(FL,"TST base memset");
193     #endif
194     if (!a)
195         { memset((void*)data,0,len*sizeof(T)); }
196     else { for (size_t i=0; i<len; ++i) data[i]=(*a); }
197 };
198
199 template <class T> inline
200 void MEM_SET_BASE(T** data, size_t len, T* const *a=NULL) {
201     #ifdef MLB_VFLAG
202         wblog(FL,"TST base memset (pointer space)");
203     #endif
204     if (!a)
205         { memset((void*)data,0,len*sizeof(T*)); }
206     else { for (size_t i=0; i<len; ++i) data[i]=(*a); }
207 };
208
209 template <class T>
210 class MEM_SET {
211 public:
212     MEM_SET(
213         T* data      __attribute__((unused)),
214         size_t len    __attribute__((unused)),
215         const T *a    __attribute__((unused)) = NULL
216     ) {
217     };
218 private:
219 };
220
221 template <class T>
222 class MEM_SET<T*> {
223 public:
224     MEM_SET(T** data, size_t len, T* const *a=NULL) {
225     #ifdef MLB_VFLAG
226         wblog(FL,"TST pointer memset");
227     #endif
228     MEM_SET_BASE(data,len,a);
229     }
230 private:
231 };
232
233 template <> inline MEM_SET<int>::MEM_SET(
234     int* data, size_t len, const int *a
235 ) { MEM_SET_BASE(data,len,a); }
236
237 template <> inline MEM_SET<unsigned>::MEM_SET(
238     unsigned* data, size_t len, const unsigned *a
239 ) { MEM_SET_BASE(data,len,a); }
240
241 template <> inline MEM_SET<unsigned long>::MEM_SET(
242     unsigned long* data, size_t len, const unsigned long *a
243 ) { MEM_SET_BASE(data,len,a); }
244
245 template <> inline MEM_SET<long>::MEM_SET(
246     long* data, size_t len, const long *a
247 ) { MEM_SET_BASE(data,len,a); }
248
249 template <> inline MEM_SET<char>::MEM_SET(
250     char* data, size_t len, const char *a
251 ) { MEM_SET_BASE(data,len,a); }
252
253 template <> inline MEM_SET<double>::MEM_SET(
254     double* data, size_t len, const double *a
255 ) { MEM_SET_BASE(data,len,a); }
256
257

```

```

263 ){ MEM_SET_BASE(data,len,a); }
264
265 template <> inline MEM_SET<float>::MEM_SET(
266     float* data, size_t len, const float *a
267 ){ MEM_SET_BASE(data,len,a); }
268
269 template <> inline MEM_SET<wbcomplex>::MEM_SET(
270     wbcomplex* data, size_t len, const wbcomplex *a
271 ){ MEM_SET_BASE(data,len,a); }
272
273 #endif
274

```

5.5 memtrack.hh

```

1  #ifndef __WB_MEM_TRACK_HH__
2  #define __WB_MEM_TRACK_HH__
3
4  // Wb,Jul27,12
5
6  namespace Wb {
7  // moved there from util.hh since used below // Wb,Sep23,16
8  void MemStat(const char *F=0, int L=0, char lflag=0);
9  };
10
11 #if !( defined __WBDEBUG__ || defined __WB_MEM_CHECK__ )
12
13 #define WB_NEW(p,n) Wb::NEW(p,n)
14 #define WB_NEW_1(p) Wb::NEW_1(p);
15
16 #define WB_NEW_2(p,x) { p = new x; \
17     if (!p) wblog(__FILE__,__LINE__,"ERR failed to allocate instance"); \
18 }
19
20 #define WB_DELETE(p) if (p) { delete [] p; p=NULL; }
21 #define WB_DELETE_1(p) if (p) { delete p; p=NULL; }
22
23 namespace Wb {
24
25     template<class T>
26     inline void NEW(T* &p, const size_t &n) {
27
28         if (n) {
29             { try { p = new T[n]; } catch (...) { p=NULL; } }
30
31             if (!p) { printf("\n");
32                 MemStat(FL);
33                 wblog(FL,"ERR out of memory (%ld * %d = %.1fG) !?",
34                     n, sizeof(T), n*sizeof(T)/double(1<<30)
35                 );
36             }
37         }
38         else { p=NULL; }
39     };
40
41     template<class T>
42     inline void NEW_1(T* &p) {
43         try { p = new T; } catch (...) { p=NULL; }
44
45         if (!p) { printf("\n");
46             MemStat(FL);
47             wblog(FL,"ERR failed to allocate instance !?");
48         }
49     };
50
51 };
52
53 #else
54
55 #define WB_NEW(p,n) Wb::NEW(__FILE__,__LINE__,p,n);
56 #define WB_NEW_1(p) Wb::NEW_1(__FILE__,__LINE__,p);
57
58 #define WB_NEW_2(p,x) { p = new x; \
59     if (!p) wblog(__FILE__,__LINE__,"ERR failed to allocate instance"); \
60 }
61
62 #define WB_DELETE(p) \
63     if (p) { Wb::gML.rm_ptr(__FILE__,__LINE__,p); delete [] p; p=NULL; }
64
65 #define WB_DELETE_1(p) \
66     if (p) { Wb::gML.rm_ptr(__FILE__,__LINE__,p); delete p; p=NULL; }
67
68 #include "memtrack.cc"

```



```

69
70 #endif
71
72 #endif
73

```

5.6 mexlib.hh

```

1  #ifndef __WB_MEXLIB_HH__
2  #define __WB_MEXLIB_HH__
3
4  /* ----- */
5  /* ----- */
6
7  void mexDisp(const mxArray *a, const char *vname=0);
8
9  void mexIssueWRN(const char *s);
10
11  wbststring sprint_info(const mxArray *a);
12
13  double wbtoc(
14      const char *F=NULL, int L=0,
15      const char *istr="", char restart=0
16  );
17
18  void wbtic() { wbtoc(0,0); }
19
20  template<class T>
21  int mxGetNumber(const mxArray *a, T &d, const char qflag=0);
22
23  double mxGetNumber(
24      const char *F, int L,
25      const mxArray *S, const char *vname, const char qflag=0
26  );
27
28  int getFlag(const mxArray *a) {
29      double q=0;
30      Mx::Array<double> A(a);
31      if (A.len>1) wblog(FL,
32          "ERR %s() invalid flag (got array, n=%d)",FCT,A.len);
33      A.ncopy_to(&q,1);
34      return int(q);
35  };
36
37  int mxGetString(const mxArray *a, char *s);
38  int mxGetString(const mxArray *a, wbststring &s);
39
40  wbststring mxTypeSize2Str(const mxArray *a);
41  wbststring mxSize2Str(const mxArray *a);
42
43  template<class T>
44  void mxGetVector(const char* F, int L, const mxArray *a, wbstvector<T> &v) {
45      size_t n=mxGetNumberOfElements(a);
46      v.init(n);
47      try {
48          Mx::Array<T>(a).copyTo(v.data);
49      }
50      catch (...) { wblog(FL,"ERR %s()",FCT); }
51  };
52
53  template<class T>
54  void mxGetVector( const char* F, int L,
55      const mxArray *S, const char *vname,
56      wbstvector<T> &v);
57
58  mxArray* matGetVariable(const char *F, int L,
59      const char *fname, const char *vname, char force=0
60  );
61  mxArray* matGetVariableInfo(const char *F, int L,
62      const char *fname, const char *vname, char force=0
63  );
64
65  double* mexSetCell2Matrix(mxArray *a, int fid, unsigned d1, unsigned d2);
66  void mxSetFieldToNumber(mxArray *S, unsigned i, unsigned fid,
67      wbstcomplex z, char dblcheck=1);
68
69  template<class T>
70  int getNumGlobal(const char *vname, T &x, const char *ws="global");
71  int isFlagGlobal(const char *vname);
72
73  template<class T>
74  mxArray* vecVec2Mx(
75      const wbstvector< wbstvector<T> > &V, const char tflag=0);

```

```

76
77     template<class T>
78     mxArray* vecVec2Mx(
79     const wbvector< wbvector<T> * > &V, const char tflag=0);
80
81     template<class T>
82     mxArray* vecMat2Mx(
83     const wbvector< wbMatrix<T> > &M, const char rawflag=0);
84
85     template<class T>
86     mxArray* cpyRange2Mx(const T* d, const wbvector<size_t> &S0);
87
88     int mxAddField2Scalar(
89     const char *F, int L,
90     mxArray* S, const char *vname, mxArray *a=NULL
91     );
92
93     void mxAppendStructToStruct(
94     const char *F, int L,
95     mxArray *&S0, mxArray *S
96     );
97
98     int mxAddField(
99     const char *F, int L,
100    mxArray* S, const char *vname
101    );
102
103    void mxReplaceField(
104    const char *F, int L,
105    mxArray* S, unsigned k, int fid, mxArray *a
106    );
107
108    int mxUpdateField(
109    const char *F, int L,
110    mxArray* S, const char *name, unsigned k, mxArray *a
111    );
112
113    void matPutVariable(const char *F, int L,
114    const char *file, const char *name, mxArray *a, unsigned keep=0
115    );
116
117    namespace Wb {
118
119        template <class TQ, class TD>
120        mxArray* mxCreateSparse(
121        const char *F, int L, unsigned d1, unsigned d2,
122        const wbMatrix<TQ> &IJ, const wbvector<TD> &D,
123        const wperm_t *p=NULL, int *info=NULL
124        );
125
126    }
127
128    template<class T>
129    mxArray* numtoMx(const T &x) {
130        mxArray *a=mxCreateDoubleMatrix(1,1,mxREAL);
131        mxGetDoubles(a)[0]=double(x); return a;
132    };
133
134    template<>
135    mxArray* numtoMx(const wbcomplex &z) {
136        mxArray *a;
137        if (z.i!=0.) {
138            a=mxCreateDoubleMatrix(1,1,mxCOMPLEX);
139            z.explicit_copy2(mxGetComplexDoubles(a)[0]);
140        }
141        else a=numtoMx(z.r);
142        return a;
143    };
144
145    template<>
146    mxArray* numtoMx(const char &c) {
147        if (isalpha(c)) {
148            char s[2]={c,0};
149            return mxCreateString(s);
150        }
151        else return numtoMx(double(c));
152    };
153
154    class mxStruct {
155    public:
156
157        mxStruct(const char *file, int line) : S(NULL) { init(file,line); };
158        ~mxStruct() {
159            if (S) {
160                wblog(FL,"WRN mxStruct() destroyed without further ado");
161                mxDestroyArray(S);
162            }

```

```

163     };
164
165     void init(const char *file, int line) {
166         F=file; L=line;
167         if (S) mxDestroyArray(S);
168         S=mxCreateStructMatrix(1,1,0,NULL);
169     };
170
171     void put(const char *name) {
172         if (S) { mxPutAndDestroy(F.data,L,S,name); S=NULL; } else
173             wblog(FL,"WRN mxStruct::put() not initialized yet - return");
174         F.init(); L=0;
175     };
176
177     void save2Struct(mxArray *S0, const char *name) {
178         if (S) {
179             mxAddField2Scalar(F.data, L, S0, name, S);
180             S=NULL;
181         } else
182             wblog(FL,"WRN mxStruct::save2Struct() not initialized yet");
183         F.init(); L=0;
184     };
185
186     void checkNewField(const char *f=NULL, int l=0);
187
188     mxStruct& operator()(const char *fn) { fld=fn; return *this; };
189
190     template<class T>
191     mxStruct& operator=(const T&x) {
192         checkNewField(FL);
193         mxAddField2Scalar(F.data, L, S, fld.data, numtoMx(x));
194         return *this;
195     };
196
197     mxStruct& operator=(mxArray *a) {
198         checkNewField(FL);
199         mxAddField2Scalar(F.data, L, S, fld.data, a);
200         return *this;
201     };
202
203     template<class T>
204     mxStruct& operator=(const wbvector<T>& x) {
205         checkNewField(FL);
206         mxAddField2Scalar(F.data, L, S, fld.data, x.toMx());
207         return *this;
208     };
209
210     template<class T>
211     mxStruct& operator=(const wbMatrix<T>& x) {
212         checkNewField(FL);
213         mxAddField2Scalar(F.data, L, S, fld.data, x.toMx());
214         return *this;
215     };
216
217     mxArray *S;
218     wbsting F, fld;
219     unsigned L;
220
221 protected:
222 private:
223
224 };
225
226 template<>
227 mxStruct& mxStruct::operator=(const wbperm &P) {
228     checkNewField(FL);
229     mxAddField2Scalar(F.data, L, S, fld.data, P.toMx());
230     return *this;
231 }
232
233 template<>
234 mxStruct& mxStruct::operator=(const wbindex &I) {
235     checkNewField(FL);
236     mxAddField2Scalar(F.data, L, S, fld.data, I.toMx());
237     return *this;
238 }
239
240 inline void mxStruct::checkNewField(const char *f, int l) {
241     int i; if (!f) { f=F.data; l=L; }
242
243     if (!S) wblog(f,l,"ERR mxStruct() S not set yet");
244     if (fld.isEmpty()) wblog(f,l,"ERR mxStruct() field not specified");
245
246     i=mxGetFieldNumber(S,fld.data);
247     if (i>=0) wblog(f,l,"ERR Field '%s' already exists!");
248 };
249

```

```

250 class MXPut {
251 public:
252
253     MXPut(const char *vn=0, const char *ws=0)
254     : S(0), vname(0), F(0), L(0) {
255         strncpy(wsp, ws && ws[0]? ws:"caller",wlen-1); wsp[wlen-1]=0;
256         init(0,0,vn);
257     };
258
259     MXPut(const char *file, int line, const char *vn="ans", const char *ws=0)
260     : S(0), vname(0), F(0), L(0) {
261         strncpy(wsp, ws && ws[0]? ws:"caller",wlen-1); wsp[wlen-1]=0;
262         init(file,line,vn);
263     };
264
265     ~MXPut() { put(); }
266
267     void init() {
268         if (F) { WB_DELETE(F); }; L=0;
269         if (vname) { WB_DELETE(vname); }
270         if (S) {
271             if (mxGetNumberOfFields(S)) { const char *v="I_check_this";
272                 wblog(F_L,"WRN %s() clearing non-empty S (see %s)",FCT,v);
273                 mxPutAndDestroy(F,L,S,v,"caller");
274             }
275             else { mxDestroyArray(S); }
276             S=NULL;
277         }
278     };
279
280     void init(const char *file, int line, const char *vn=NULL) {
281         if (S || F || vname) init();
282         if (file && file[0]) { L=line;
283             WB_NEW(F,strlen(file)+1); strcpy(F,file);
284         }
285
286         if (vn && vn[0])
287             { WB_NEW(vname,strlen(vn)+1); strcpy(vname,vn); }
288         else { WB_NEW(vname,4); strcpy(vname,"ans"); }
289
290         #pragma omp critical (using_MEX_API)
291         S=mxCreateStructMatrix(1,1,0,NULL);
292
293         if (L) addFL();
294     };
295
296     void put(const char *ws=0, const char *vn=0) {
297         if (S) {
298             if (!vname) wblog(F_L,
299                 "ERR %s() got empty variable name !?",FCT,vname);
300             const char s0[]="ans", *s=((vn && vn[0]) ? vn : vname);
301             if (!s || !s[0]) s=s0;
302
303             if (!ws || !ws[0]) {
304                 if (!wsp[0]) wblog(F_L,"ERR %s() workspace not set",FCT);
305                 ws=wsp;
306             }
307
308             mxPutAndDestroy(F,L,S,s,ws);
309             S=NULL;
310         }
311         init();
312     };
313
314     void save2(mxArray* &S0) { S0=S; S=NULL; init(); };
315
316     mxArray* toMx() { mxArray* S0=S; S=NULL; init(); return S0; };
317
318     void add2Struct(mxArray *S0) {
319         if (S) {
320             if (!vname) wblog(F_L,
321                 "ERR %s() got empty variable name !?",FCT,vname);
322             mxAddField2Scalar(F_L,S0,vname,S); S=NULL;
323         }
324         init();
325     };
326
327     MXPut& addFL() {
328         if (!S) wblog(F_L,"ERR MXPut::%s() S not initialized yet",FCT);
329         else {
330             if (!vname) wblog(F_L,
331                 "ERR %s() got empty variable name !?",FCT,vname);
332             if (mxGetFieldNumber(S,"FL")<0)
333                 mxAddField2Scalar(F_L,S,"FL",wbstring(shortFL(F_L)).toMx());
334             else wblog(F_L,"WRN %s() field FL already set",FCT);
335         }
336         return *this;

```

```

337     };
338
339     template<class T>
340     MXPut& add(const T&x, const char *fn) {
341         if (!S) wblog(F_L, "ERR MXPut::%s() S not initialized yet", FCT);
342         else mxAddField2Scalar(F_L, S, fn, x.toMx());
343         return *this;
344     };
345
346     template<class T>
347     MXPut& addP(const T* x, const char *fn) {
348         if (!S) wblog(F_L, "ERR MXPut::%s() S not initialized yet", FCT);
349         else wblog(F_L,
350             "ERR %s() not defined yet for pointer to %s",
351             FCT, TSTR(T));
352         return *this;
353     };
354
355     mxArray *S;
356     char wsp[16];
357     char *vname, *F;
358     unsigned L;
359
360     static const unsigned wlen;
361
362     protected:
363     private:
364 };
365
366     const unsigned MXPut::wlen=16;
367
368     template<>
369     MXPut& MXPut::addP(const mxArray* a, const char *fn) {
370         if (!S) wblog(F_L, "ERR MXPut::%s() S not set yet", FCT); else
371         if (!vname) wblog(F_L,
372             "ERR %s() got empty variable name !?", FCT, vname);
373         else mxAddField2Scalar(F_L, S, fn, (mxArray*)a);
374         return *this;
375 };
376
377 #ifndef NOMEX
378     template<>
379     MXPut& MXPut::addP(const char* s, const char *fn) {
380         return addP(mxCreateString(s), fn);
381     };
382 #endif
383
384     template<>
385     MXPut& MXPut::add(const double &x, const char *fn) {
386         return addP(numtoMx(x), fn);
387     };
388     template<>
389     MXPut& MXPut::add(const float &x, const char *fn) {
390         return addP(numtoMx(x), fn);
391     };
392     template<>
393     MXPut& MXPut::add(const long double &x, const char *fn) {
394         return addP(numtoMx(x), fn);
395     };
396     template<>
397     MXPut& MXPut::add(const wbcomplex &x, const char *fn) {
398         return addP(numtoMx(x), fn);
399     };
400     template<>
401     MXPut& MXPut::add(const char &x, const char *fn) {
402         return addP(numtoMx(x), fn);
403     };
404
405     template<>
406     MXPut& MXPut::add(char* const &s, const char *fn) {
407         return add(wbstring(s), fn);
408     };
409     template<>
410     MXPut& MXPut::add(const char* const &s, const char *fn) {
411         return add(wbstring(s), fn);
412     };
413
414     template<>
415     MXPut& MXPut::add(const unsigned &x, const char *fn) {
416         return addP(numtoMx(x), fn);
417     };
418     template<>
419     MXPut& MXPut::add(const int &x, const char *fn) {
420         return addP(numtoMx(x), fn);
421     };
422     template<>
423     MXPut& MXPut::add(const bool &x, const char *fn) {

```

```

424     return addP(numtoMx(x),fn);
425 };
426 template<>
427 MXPut& MXPut::add(const long &x, const char *fn) {
428     return addP(numtoMx(x),fn);
429 };
430 template<>
431 MXPut& MXPut::add(const unsigned long &x, const char *fn) {
432     return addP(numtoMx(x),fn);
433 };
434
435 #ifdef __WB_MPFR_HH__
436 template<>
437 MXPut& MXPut::add(const Wb::quad &x, const char *fn) {
438     return addP(numtoMx(x),fn);
439 };
440 #endif
441
442 class mxArray_buf {
443 public:
444     mxArray_buf(const char *n=NULL) {
445         BUF.max_load_factor(0.2);
446         if (n && n[0])
447             { name = new char[strlen(n)+1]; strcpy(name,n); }
448         else { name = new char[4]; strcpy(name,"MXB"); }
449     };
450
451     ~mxArray_buf() {
452         size_t l=BUF.size();
453         if (l) wblog(FL,"WRN %s() %s got %d entries still",PROG,name,l);
454         BUF.clear();
455         if (name) { delete [] name; name=NULL; }
456     };
457
458     void add(void *p, mxArray *a) {
459         if (p) {
460             #pragma omp critical (using_mxCArray_P2X)
461             { auto im=BUF.insert({p,a});
462               if (!im.second) wblog(FL,
463                 "ERR %s() %s already got entry for %p",PROG,name,p);
464             }
465         }
466         else wblog(FL,"WRN %s() got null pointer",FCT,p);
467     };
468
469     mxArray* find(const char *F, int L, void *p) {
470         mxArray *a=NULL;
471
472         #pragma omp critical (using_mxCArray_P2X)
473         { auto im=BUF.find(p);
474           if (im!=BUF.end()) { a=im->second; }
475         }
476         if (!a && F) wblog(F,L,
477           "WRN %s() %s missing entry for %p",PROG,name,p);
478         return a;
479     };
480
481     mxArray* Return(const char *F, int L, void *p) {
482         mxArray *a=NULL;
483
484         #pragma omp critical (using_mxCArray_P2X)
485         { auto im=BUF.find(p);
486           if (im!=BUF.end()) {
487               a=im->second; BUF.erase(im->first);
488           }
489         }
490         if (!a && F) wblog(F,L,
491           "WRN %s() %s missing entry for %p",PROG,name,p);
492         return a;
493     };
494
495     int erase(void *p) {
496         int e=0;
497         #pragma omp critical (using_mxCArray_P2X)
498         { e=BUF.erase(p); }
499         return e;
500     };
501
502     unordered_map<void*, mxArray*> BUF;
503
504     char* name;
505
506 protected:
507 private:
508
509
510 };

```

```

511
512     mxArray_buf P2X("P2X");
513
514 #endif
515

```

5.7 mpsortho.hh

```

1 #ifndef __WB_ORTHO_HH__
2 #define __WB_ORTHO_HH__
3
4 #define WB_STOL_SVD 1E-8
5
6 /* ----- */
7 /* ----- */
8 // change log
9 // * split off mpsortho.hh from mpsortho.cc
10 // * transferred former QSpace member functions
11 //   toBlockMatrix() and initFromBlockMatrix()
12 //   to class QBlock(), as it fits more naturally there.
13 // Wb,Aug05,15
14
15 template <class TQ, class TD>
16 mxArray* orthoQS(
17     const QSpace<TQ,TD> &PSI,
18     QSpace<TQ,TD> &A1,
19     QSpace<TQ,TD> &A2, unsigned K,
20     unsigned Nkmin, unsigned Nkeep,
21     double stol,
22     iTag tx,
23     const char *info
24 );
25
26 template <class TQ, class TD>
27 mxArray* getSVD(
28     const QSpace<TQ,TD> &PSI,
29     QSpace<TQ,TD> &U,
30     QSpace<TQ,TD> &S,
31     QSpace<TQ,TD> &VT, unsigned K,
32     unsigned Nkmin, unsigned Nkeep, double stol,
33     iTag tx,
34     const char *info
35 );
36
37 char mxGetLRDir(
38     const char *F, int L, const mxArray *argin, unsigned k
39 );
40
41 template <class TQ, class T1, class T2, class T3>
42 void twoSiteInit(
43     const QSpace<TQ,T1> &Psi1,
44     const QSpace<TQ,T2> &Psi2,
45     QSpace<TQ,T3> &PSI, unsigned &K,
46     wid_x_t ic1, wid_x_t ic2,
47     char twoSite, char ldir
48 );
49
50 template <class TQ, class T1, class T2, class T3>
51 void twoSiteFinal(
52     const QSpace<TQ,T1> &Psi1,
53     const QSpace<TQ,T2> &Psi2,
54     QSpace<TQ,T3> &A1,
55     QSpace<TQ,T3> &A2,
56     unsigned ic1, unsigned ic2, char twoSite, char ldir
57 );
58
59 template <class TQ, class TD, class TE=double>
60 class QBlock {
61 public:
62
63     QBlock() : qdim(1), sfac(1) {};
64
65     QBlock& init() {
66         IO.init(); Ik.init(); It.init(); idx.init(); sfac=qdim=1;
67         Q1.init(); S1.init(); D1.init(); A.init(); S.init();
68         Q2.init(); S2.init(); D2.init(); U.init(); V.init();
69         return *this;
70     };
71
72     QBlock& operator=(const QBlock &B) {
73         if (this!=&B) {
74             IO=B.IO; Ik=B.Ik; idx=B.idx; qdim=B.qdim; sfac=B.sfac;
75             Q1=B.Q1; S1=B.S1; D1=B.D1; A=B.A; S=B.S;

```

```

76         Q2=B.Q2; S2=B.S2; D2=B.D2; U=B.U; V=B.V;
77     }
78     return *this;
79 };
80
81 QBlock& init_bare_refA(const QBlock &B, unsigned w=0);
82
83 void toBlockMatrix(
84     wbararray<TD> &MM,
85     unsigned K,
86     const wbperm &P=wbperm()
87 );
88
89 void initFromBlockMatrix(
90     QSpace<TQ,TD> &B,
91     const wbararray<TD> &MM,
92     const IndexLabels &it,
93     char flag,
94     const wbperm *cgp=NULL
95 );
96
97 void updateBlockDim(unsigned dim2);
98
99 double get_s2t(const wbvector<char> &mark) {
100     double s2t=0; unsigned i=0;
101     if (mark.len!=S.len) wblog(FL,"ERR %s() "
102         "length mismatch (mark.len=%d/%d)",FCT,mark.len,S.len);
103     mark.find(Ik);
104     for (; i<mark.len; ++i) { if (!mark[i]) s2t+=S[i]*S[i]; }
105     if (qdim!=1) { s2t*=qdim; }
106     return s2t;
107 };
108
109 void print(
110     const char *F=NULL, int L=0, const IndexLabels *t=NULL
111 ) const {
112
113     wblog(F_L,"%s() QBlock content ===== %N",FCT);
114     A.print("sub(PSI)"); if (t) {
115         printf("> itags: \"%s\\n\",t->toStr().data); }
116         printf("      qdim=%g, sfac=%g\\n\\n",double(qdim),sfac);
117         printf("      Q =[ %s; %s ]\\n",STR(Q1),STR(Q2));
118         printf("      S =[ %s; %s ]\\n",STR(S1),STR(S2));
119         printf("      D1=[ %s ]\\n",STR(D1));
120         printf("      D2=[ %s ]\\n\\n",STR(D2));
121     };
122
123     mxArray* toMx() const {
124         const char *f[]={
125             "A","qdim","sfac", "Q1","S1","D1", "Q2","S2","D2",
126             "I0","Ik","idx","U","S","V"
127         };
128         unsigned i=0, n=1;
129         mxArray *q=mxCreateStructMatrix(n,1,15,f);
130
131         mxSetFieldByNumber(q,i, 0, A .toMx());
132         mxSetFieldByNumber(q,i, 1, numtoMx(double(qdim)));
133         mxSetFieldByNumber(q,i, 2, numtoMx(sfac));
134         mxSetFieldByNumber(q,i, 3, Q1.toMx());
135         mxSetFieldByNumber(q,i, 4, S1.toMx());
136         mxSetFieldByNumber(q,i, 5, D1.toMx());
137         mxSetFieldByNumber(q,i, 6, Q2.toMx());
138         mxSetFieldByNumber(q,i, 7, S2.toMx());
139         mxSetFieldByNumber(q,i, 8, D2.toMx());
140         mxSetFieldByNumber(q,i, 9, I0.toMx());
141         mxSetFieldByNumber(q,i,10, Ik.toMx());
142         mxSetFieldByNumber(q,i,11, idx.toMx());
143         mxSetFieldByNumber(q,i,12, U .toMx());
144         mxSetFieldByNumber(q,i,13, S .toMx());
145         mxSetFieldByNumber(q,i,14, V .toMx());
146
147         return q;
148     };
149
150     void put(const char *vname, const char *ws="caller") {
151         mxArray *a=toMx();
152         mxPutAndDestroy(FL,a,vname,ws);
153     };
154
155     QSpace<TQ,TD> A;
156     wbindex I0;
157     wbindex Ik;
158     wbindex It;
159
160     wbMatrix<TQ> Q1, Q2;
161     wbMatrix<widx_t> S1, S2;
162     wbMatrix<widx_t> D1, D2;

```



```

163
164     wbMatrix<int> idx;
165
166     wbararray<TD> U,V;
167     wbvector<TE> S;
168
169     double qdim;
170     double sfac;
171     protected:
172     private:
173 };
174
175 template <class TQ, class TD>
176 class SVDDData {
177     public:
178         SVDDData() {};
179
180         void blockSVD(const QSpace<TQ,TD> &PSI, const unsigned K);
181
182         unsigned dmrgTruncate(
183             unsigned Nkmin, unsigned Nkeep, double stol,
184             iTag tx,
185             wbMatrix<double> &SV,
186             QSpace<TQ,TD> &UQ,
187             QSpace<TQ,TD> *SQ,
188             QSpace<TQ,TD> &VQ,
189             mxArray *Sout=NULL, char vflag=0
190         );
191
192         mxArray* toMx() const;
193
194         void put(const char *vname, const char *ws="base") const {
195             put(0,0,vname,ws);
196         };
197
198         void put(const char *F, int L,
199             const char *vname, const char *ws="base"
200         ) const {
201             mxArray *a=toMx();
202             int i=mexPutVariable(ws,vname,a);
203             if (i) wblog(F,L,"ERR failed to write variable '%s' (%d)",vname,i);
204             if (F) wblog(F,L,"I/O putting variable '%s' to '%s'",vname,ws);
205             mxDestroyArray(a);
206         };
207
208         wbvector< QBlock<TQ,TD,double> > QB;
209
210     protected:
211     private:
212 };
213
214 #endif
215

```

5.8 mxlib.hh

```

1  #ifndef __WB_MXLIB_HH__
2  #define __WB_MXLIB_HH__
3
4  /* ----- */
5  // switching to matlab 2018a => got interleaved complex format
6  // => phasing out mxGetPr_17(FL,) and mxGetPi_17(FL,)
7
8  #ifdef MATLAB_MEX_FILE
9
10     mxDouble* mxGetPr_17(const char *F, int L, const mxArray *pm) {
11         wblog(F_L,"ERR mxGetPr() no longer supported in matlab>=2018");
12         return 0;
13     };
14
15     mxDouble* mxGetPi_17(const char *F, int L, const mxArray *pm) {
16         wblog(F_L,"ERR mxGetPi() no longer supported in matlab>=2018");
17         return 0;
18     };
19
20 #endif
21
22 #ifndef NOMEX
23 int isHelpIndicator(const char *s) {
24     if (s && s[0]!='-') {
25         if (!strcmp(s,"-?") || !strcmp(s,"-h") || !strcmp(s,"--help"))
26             return 1;
27     }
28 }
29

```

```

28     return 0;
29 };
30 #endif
31
32 int isHelpIndicator(const mxArray *a) {
33     char s[8];
34     if (!a || !mxGetString(a,s,8)) { return 0; }
35     return isHelpIndicator(s);
36 };
37
38 int checkHelpVersion(const mxArray *a) {
39     char s[16];
40     if (a && !mxGetString(a,s,16)) {
41         if (isHelpIndicator(s)) { usage(); return 1; }
42         if (!strcmp(s,"--version")) { print_version_info(); return 1; }
43     }
44     return 0;
45 };
46
47 inline mxArray* mx_get_field(mxArray *S, unsigned k, int i) {
48     if (S && i>=0 && int(k)>=0) {
49         mxArray *a=mxGetFieldByNumber(S,k,i);
50         if (a && mxGetNumberOfElements(a)) return a;
51     }
52     return NULL;
53 };
54
55 void mxPutArray(const char *file, int line,
56     mxArray *a, const char *vname="ans", const char* ws="base",
57     const char* dstr="");
58
59 void mxPutAndDestroy(const char *file, int line,
60     mxArray *a, const char *vname="ans", const char* ws="base");
61
62 namespace Mx {
63
64 int IsEqual(const mxArray *a, const char *s) {
65
66     int n=strlen(s)+1;
67     char sx[n+1]; sx[0]=0;
68
69     if (mxGetString(a,sx,n)) { return 0; }
70     else { return (strcmp(sx,s) ? 0 : 1); }
71 };
72
73 template <class T>
74 class Array {
75
76 public:
77
78     Array() : ax(0), data(0),
79         len(0), rank(0), trans(0), cmplx(0), mxref(0) { };
80
81     Array(unsigned ndim, size_t *dims, char t_=0, char c_=-1)
82         : ax(0), data(0), len(0), rank(ndim), trans(t_), cmplx(c_), mxref(0) {
83         if (cmplx<0) { cmplx=ISCOMPLX_(T); }
84         init(ndim,dims);
85     };
86
87     Array(size_t d1, size_t d2, char t_=0, char c_=-1)
88         : ax(0), data(0), len(0), rank(2), trans(t_), cmplx(c_), mxref(0) {
89         size_t dims[2]={d1,d2}; if (cmplx<0) { cmplx=ISCOMPLX_(T); }
90         init(2,dims);
91     };
92
93     Array(const wvector<size_t> &S, char t_=0, char c_=-1);
94
95     Array(const mxArray* a, char t_=0, char c_=-1)
96         : ax(0), data(0), len(0), rank(0), trans(t_), cmplx(c_), mxref(1) {
97         if (cmplx<0) { cmplx=ISCOMPLX_(T); }
98         init(a);
99     };
100
101     Array(const char *F, int L, const mxArray* a, char t_=0, char c_=-1)
102         : ax(0), data(0), len(0), rank(0), trans(t_), cmplx(c_), mxref(1)
103         { if (cmplx<0) { cmplx=ISCOMPLX_(T); }
104
105             try { init(a); }
106             catch (...) { wblog(F_L,"ERR %s() ",FCT); }
107
108             if (!ax || (!mxref && !data)) wblog(F_L,
109                 "ERR %s() invalid input\n%s %s",FCT,shortFL(FL),STR_(this));
110         };
111
112     wstring toStr(char vflag=0, unsigned l=64) const;
113     wstring toStrT(char vflag=0) const;
114

```

```

115 void init_xstr() {
116     sprintf(str, "Mx::Array<%s>", sTSTR(T));
117     xstr=str;
118 };
119
120 const char *xStr() const {
121     return xstr.c_str();
122 };
123
124 ~Array() {
125     if ((ax || data) && mxref<=0) wblog(FL,
126         "WRN %s() got ax=%p, data=%p !?", FCT, ax, data);
127 }
128
129 mxArray* Return(T*& dd) { dd=data; return Return(); };
130
131 mxArray* Return() {
132     mxArray *a=ax;
133     ax=NULL; data=NULL; len=0;
134     mxref=(mxref>0? 1:0);
135     return a;
136 };
137
138 Array& init(unsigned ndim, size_t *dims, char t_=-1, char c_=-1);
139 Array& init(const mxArray *a, char t_=-1, char c_=-1);
140
141 template <class Tb>
142 mxArray* copyFromTR(const Tb* b, char tcheck=0) {
143
144     if (!trans && typeid(T)==typeid(Tb) && data && b)
145         { memcpy(data,b,len*sizeof(T)); }
146     else { copy_from(b,tcheck); }
147
148     return Return();
149 };
150
151 template <class Tb>
152 int copy_from(const Tb* b, char tcheck=0);
153
154 template <class Tb>
155 size_t copyTo(Tb* b, char tcheck=0) const {
156     if (!trans && data && b) {
157         if (mxref<=8) wblog(FL, "WRN %s() ref=%d !?", FCT, mxref);
158         memcpy(b, data, len*sizeof(T));
159     }
160     else { copy_to(b, NULL, tcheck); }
161     return len;
162 };
163
164 size_t copy_to(T* b, const wperm *P=NULL, char tcheck=0) const;
165
166 size_t rcopy_to(T* b, const wperm *P=NULL, char tcheck=0) const;
167 size_t zcopy_to(T* b, const wperm *P=NULL, char tcheck=0) const;
168
169 template <class Tx>
170 size_t cpy_to_(T* b, const wperm *P, char tcheck, const Tx *ad_) const;
171
172 size_t ncopy_to(T* b, size_t n, char tcheck=0) {
173     size_t l_=len; if (len>n) len=n;
174     n=copy_to(b, NULL, tcheck); len=l_;
175     return n;
176 };
177
178 mxArray *ax;
179 T *data;
180 size_t len;
181 unsigned rank;
182
183 bool trans;
184 char cmplx;
185 char mxref;
186
187 std::string xstr;
188
189 protected:
190 private:
191
192 void issue_type_error(
193     const char *F, int L, unsigned ndim=0, const size_t *dims=NULL);
194
195 int init_dptra();
196
197 int safe_set_dptra(T *md) { data=md; return 0; }
198
199 template <class TM>
200 int safe_set_dptra(TM *md __attribute__((unused))) { return 1; }
201

```

```

202     void MX_INIT_DATA(unsigned ndim, const size_t *dims);
203 };
204
205 template <> template <>
206 int Mx::Array<wbcomplex>::safe_set_dptr(mxComplexDouble *md) {
207     data=(wbcomplex*)md; return 0;
208 };
209
210 template <> template <>
211 int Mx::Array<char>::safe_set_dptr(int8_T *md) {
212     unsigned m=sizeof(char), n=sizeof(int8_T);
213     if (m!=n) wblog(FL,"ERR %s() size mismatch (char: %d/%d)",FCT,m,n);
214     data=(char*)md; return 0;
215 };
216
217 template <> template <>
218 int Mx::Array<size_t>::safe_set_dptr(uint64_T *md) {
219     unsigned m=sizeof(size_t), n=sizeof(uint64_T);
220     if (m!=n) wblog(FL,"ERR %s() size mismatch (size_t: %d/%d)",FCT,m,n);
221     data=(size_t*)md; return 0;
222 };
223
224 template <> template <>
225 int Mx::Array<unsigned long long>::safe_set_dptr(uint64_T *md) {
226     unsigned m=sizeof(size_t), n=sizeof(uint64_T);
227     if (m!=n) wblog(FL,"ERR %s() size mismatch (size_t: %d/%d)",FCT,m,n);
228     data=(unsigned long long*)md; return 0;
229 };
230
231 template <> template <>
232 int Mx::Array<long long>::safe_set_dptr(int64_T *md) {
233     unsigned m=sizeof(long long), n=sizeof(int64_T);
234     if (m!=n) wblog(FL,"ERR %s() size mismatch (long: %d/%d)",FCT,m,n);
235     data=(long long*)md; return 0;
236 };
237
238 template <>
239 size_t Mx::Array<double>::copy_to(double * b, const wbperm *P, char tcheck)
240 const { return rcopy_to(b,P,tcheck); };
241
242 template <>
243 size_t Mx::Array<int>::copy_to(int * b, const wbperm *P, char tcheck)
244 const { return rcopy_to(b,P,tcheck); };
245
246 template <>
247 size_t Mx::Array<unsigned>::copy_to(unsigned* b, const wbperm *P, char tcheck)
248 const { return rcopy_to(b,P,tcheck); };
249
250 template <>
251 size_t Mx::Array<size_t>::copy_to(size_t * b, const wbperm *P, char tcheck)
252 const { return rcopy_to(b,P,tcheck); };
253
254 template <>
255 size_t Mx::Array<char>::copy_to(char* b, const wbperm *P, char tcheck)
256 const { return rcopy_to(b,P,tcheck); };
257
258 template <>
259 void Mx::Array<double>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
260     data=0;
261     #pragma omp critical (using_MEX_API)
262     { ax=mxCreateNumericArray(ndim,dims, mxDOUBLE_CLASS, mxREAL);
263       if (ax) { safe_set_dptr(mxGetDoubles(ax)); }
264     }; if (!data) issue_type_error(FL,ndim,dims);
265 };
266
267 template <>
268 void Mx::Array<wbcomplex>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
269     data=0;
270     #pragma omp critical (using_MEX_API)
271     { ax=mxCreateNumericArray(ndim,dims, mxDOUBLE_CLASS, mxCOMPLEX);
272       if (ax) { safe_set_dptr(mxGetComplexDoubles(ax)); }
273     }; if (!data) issue_type_error(FL,ndim,dims);
274 };
275
276 template <>
277 void Mx::Array<char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
278     data=0;
279     #pragma omp critical (using_MEX_API)
280     { ax=mxCreateNumericArray(ndim,dims, mxINT8_CLASS, mxREAL);
281       if (ax) { safe_set_dptr(mxGetInt8s(ax)); }
282     }; if (!data) issue_type_error(FL,ndim,dims);
283 };
284
285 template <>
286 void Mx::Array<signed char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
287     data=0;
288     #pragma omp critical (using_MEX_API)

```

```

289 { ax=mxCreateNumericArray(ndim,dims, mxINT8_CLASS, mxREAL);
290   if (ax) { safe_set_dptr(mxGetInt8s(ax)); }
291 }; if (!data) issue_type_error(FL,ndim,dims);
292 };
293
294 template <>
295 void Mx::Array<unsigned char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
296   data=0;
297   #pragma omp critical (using_MEX_API)
298   { ax=mxCreateNumericArray(ndim,dims, mxUINT8_CLASS, mxREAL);
299     if (ax) { safe_set_dptr(mxGetUint8s(ax)); }
300   }; if (!data) issue_type_error(FL,ndim,dims);
301 };
302
303 template <>
304 void Mx::Array<mxChar>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
305   data=0;
306   #pragma omp critical (using_MEX_API)
307   { ax=mxCreateCharArray(ndim,dims);
308     if (ax) { safe_set_dptr(mxGetChars(ax)); }
309   }; if (!data) issue_type_error(FL,ndim,dims);
310 };
311
312 template <>
313 void Mx::Array<int>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
314   data=0;
315   #pragma omp critical (using_MEX_API)
316   { ax=mxCreateNumericArray(ndim,dims, mxINT32_CLASS, mxREAL);
317     if (ax) { safe_set_dptr(mxGetInt32s(ax)); }
318   }; if (!data) issue_type_error(FL,ndim,dims);
319 };
320
321 template <>
322 void Mx::Array<unsigned>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
323   data=0;
324   #pragma omp critical (using_MEX_API)
325   { ax=mxCreateNumericArray(ndim,dims, mxUINT32_CLASS, mxREAL);
326     if (ax) { safe_set_dptr(mxGetUint32s(ax)); }
327   }; if (!data) issue_type_error(FL,ndim,dims);
328 };
329
330 template <>
331 void Mx::Array<long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
332   data=0;
333   #pragma omp critical (using_MEX_API)
334   { ax=mxCreateNumericArray(ndim,dims, mxINT64_CLASS, mxREAL);
335     if (ax) { safe_set_dptr(mxGetInt64s(ax)); }
336   }; if (!data) issue_type_error(FL,ndim,dims);
337 };
338
339 template <>
340 void Mx::Array<long long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
341   data=0;
342   #pragma omp critical (using_MEX_API)
343   { ax=mxCreateNumericArray(ndim,dims, mxINT64_CLASS, mxREAL);
344     if (ax) { safe_set_dptr(mxGetInt64s(ax)); }
345   }; if (!data) issue_type_error(FL,ndim,dims);
346 };
347
348 template <>
349 void Mx::Array<size_t>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
350   data=0;
351   #pragma omp critical (using_MEX_API)
352   { ax=mxCreateNumericArray(ndim,dims, mxUINT64_CLASS, mxREAL);
353     if (ax) { safe_set_dptr(mxGetUint64s(ax)); }
354   }; if (!data) issue_type_error(FL,ndim,dims);
355 };
356
357 template <>
358 void Mx::Array<unsigned long long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
359   data=0;
360   #pragma omp critical (using_MEX_API)
361   { ax=mxCreateNumericArray(ndim,dims, mxUINT64_CLASS, mxREAL);
362     if (ax) { safe_set_dptr(mxGetUint64s(ax)); }
363   }; if (!data) issue_type_error(FL,ndim,dims);
364 };
365
366 int IsNumArray(
367   const char *F, int L,
368   const mxArray* a, unsigned r=2, char cflag=0, char type='d'
369 );
370
371 bool IsDblMat(const char *F, int L,
372   const mxArray* a, char cflag=0, char dflag='d')
373 { return Mx::IsNumArray(F,L,a,2,cflag,dflag)>0; }
374
375 bool IsDblArr(const char *F, int L,

```

```

376     const mxArray* a, char cflag=0, char dflag='d')
377 { return Mx::IsNumArray(F,L,a,-1,cflag,dflag)>0; }
378
379 bool IsDblVector(const char *F, int L,
380     const mxArray *a, char cflag=0, char dflag='d')
381 { return Mx::IsNumArray(F,L,a,1,cflag,dflag)>0; }
382
383 bool IsDblScalar(const char *F, int L,
384     const mxArray *a, char cflag=0, char dflag='d') {
385     if (Mx::IsNumArray(F,L,a,1,cflag,dflag)<=0) return 0;
386     return mxGetNumberOfElements(a)==1;
387 }
388
389 bool IsNumber(const char *F, int L,
390     const mxArray *a, char cflag=0, char dflag='*') {
391     if (Mx::IsNumArray(F,L,a,1,cflag,dflag)<=0) return 0;
392     return (mxGetNumberOfElements(a)==1);
393 }
394
395 bool IsScalar(const mxArray *a) {
396     return (mxGetNumberOfDimensions(a)==2 && mxGetNumberOfElements(a)==1);
397 };
398
399 bool IsVector(const mxArray *a) {
400     const size_t *s=mxGetDimensions(a);
401     const int n=mxGetNumberOfDimensions(a);
402     return (n==2 && (s[0]==1 || s[1]==1));
403 };
404
405 bool IsIndex(const mxArray *a, int base=0);
406
407 template <class T>
408 bool IsIndex(const T *x, size_t n, int base=0) {
409     if (!WbUtil<T>().isPOD()) wblog(FL,
410         "ERR %s() invalid data type '%s'",FCT,TSTR(T));
411
412     if (WbUtil<T>().isInt()) {
413         for (size_t i=0; i<n; ++i) { if (x[i]<base) return 0; }
414     }
415     else {
416         for (size_t i=0; i<n; ++i) {
417             if (x[i]<base || x[i]!=round(x[i])) return 0; }
418     }
419     return 1;
420 };
421
422 };
423
424 namespace Wb {
425     int CallMatlab(
426         int nargout, mxArray *argout[], int nargin, mxArray *argin[],
427         const char *fctname);
428
429     double CallMatlab(
430         const char *F, int L, const char* cmd, const char *arg1=NULL);
431
432     double CallMatlab(const char* cmd, const char *arg1=NULL) {
433         return CallMatlab(0,0,cmd,arg1);
434     };
435 };
436
437 #ifndef MATLAB_MEX_FILE
438 #undef mexPutVariable
439 #define mexPutVariable mexPutVariable_wbx
440 int mexPutVariable_wbx(const char *ws, const char *vname, const mxArray *a){
441     wblog(FL,"WRN mexPutVariable(%s,%lX,%s) not available outside MatLab",
442         vname?vname:"",a,ws?ws:""); return 1;
443 };
444
445 #undef mexCallMATLAB
446 #define mexCallMATLAB mexCallMATLAB_wbx
447 int mexCallMATLAB_wbx(
448     int nargout, mxArray **argin, int nargin, mxArray **argout,
449     const char *cmd
450 ){ wblog(FL,"ERR mexCallMATLAB(%d,%lX,%d,%lX,%s) not available outside MatLab",
451     nargin,argin,nargout,argout,cmd?cmd:""); return 1;
452 };
453
454 #undef mexGetVariable
455 #define mexGetVariable mexGetVariable_wbx
456 int mexGetVariable_wbx(const char *ws, const char *vname) {
457     wblog(FL,"WRN mexGetVariable(%s,%s) not available outside MatLab",
458         vname?vname:"",ws?ws:""); return 1;
459 };
460
461 #undef mexGetVariablePtr

```

```

463 #define mexGetVariablePtr mexGetVariablePtr_wbx
464 mxArray* mexGetVariablePtr_wbx(const char *ws, const char *vname) {
465     wblog(FL,"WRN mexGetVariablePtr(%s,%s) not available outside MatLab",
466         vname?vname:"",ws?ws:""); return NULL;
467 };
468
469 #undef mexPrintf
470 #define mexPrintf mexPrintf_wbx
471 int mexPrintf_wbx(const char *fmt, ...){
472     wblog(FL,"WRN mexPrintf(%s,...) not available outside MatLab",
473         fmt?fmt:""); return 1;
474 };
475
476 #undef mexFunctionName
477 #define mexFunctionName mexFunctionName_wbx
478
479 const char* mex_function_name="(program)";
480
481 const char* mexFunctionName_wbx(const char *fmt, ...){
482     wblog(FL,"WRN mexFunctionNamef(%s,...) not available outside MatLab",
483         fmt?fmt:""); return mex_function_name;
484 };
485
486 const char* mexFunctionName_wbx(){
487     wblog(FL,"WRN mexFunctionNamef() not available outside MatLab");
488     return mex_function_name;
489 };
490
491 #endif
492
493 #endif
494

```

5.9 nomex.hh

```

1  /* ===== */
2  /* dummy version of bracketing out MEX library if
3   (i) it is not available and
4   (ii) also not required in the code per se!
5   in case any of the mex-routines is required
6   an error message issued and the program terminates.
7
8   NB! return types taken from Matlab R2013a documentation
9   Wb, Apr02,13 */
10 /* ===== */
11
12 typedef char mxArray;
13
14 // taken from Matlab R2013a documentation
15 enum mxComplexity {mxREAL=0, mxCOMPLEX};
16 // #define mxREAL 0
17 // #define mxCOMPLEX 1
18
19 #define mxDOUBLE_CLASS 10
20
21 #define MATFile FILE
22 #define mxChar short
23 #define mwSize size_t
24 #define mwIndex int
25
26 #define ERRMSG_NOMEX wblog(__FILE__, __LINE__, "ERR NOMEX (%s)", __FUNCTION__)
27
28 int wblog(const char* file, int line, const char *fmt, ...);
29
30 int mexCallMATLAB(
31     unsigned nout, mxArray** aout,
32     unsigned nin, mxArray** ain, const char* fstr)
33 { ERRMSG_NOMEX; return 0; };
34
35 int mxIsChar (const mxArray* a) { ERRMSG_NOMEX; return 0; };
36 int mxIsDouble (const mxArray* a) { ERRMSG_NOMEX; return 0; };
37 int mxIsComplex (const mxArray* a) { ERRMSG_NOMEX; return 0; };
38 int mxIsCell (const mxArray* a) { ERRMSG_NOMEX; return 0; };
39 int mxIsStruct (const mxArray* a) { ERRMSG_NOMEX; return 0; };
40 int mxIsSparse (const mxArray* a) { ERRMSG_NOMEX; return 0; };
41
42 int mxIsEmpty (const mxArray* a) { ERRMSG_NOMEX; return 0; };
43 size_t mxGetM (const mxArray* a) { ERRMSG_NOMEX; return 0; };
44 size_t mxGetN (const mxArray* a) { ERRMSG_NOMEX; return 0; };
45
46 int mxGetNzmax (const mxArray* a) { ERRMSG_NOMEX; return 0; };
47 int* mxGetIr (const mxArray* a) { ERRMSG_NOMEX; return (int*)0; };
48 int* mxGetJc (const mxArray* a) { ERRMSG_NOMEX; return (int*)0; };

```

```

49
50 int mxDestroyArray (mxArray* a) { ERRMSG_NOMEX; return 0; };
51
52 void mexErrMsgTxt(const char* estr) {
53     printf("\nmexErrMsgTxt:%s\nExit.\n\n", estr);
54     exit(66);
55 };
56
57 char* mxGetClassName(const mxArray* a) {
58     static char ostr[]="(none)";
59     ERRMSG_NOMEX; return ostr;
60 };
61
62 size_t mxGetNumberOfElements(const mxArray* a) {
63     ERRMSG_NOMEX; return 0;
64 };
65
66 int mxGetNumberOfFields(const mxArray* a) {
67     ERRMSG_NOMEX; return 0;
68 };
69
70 unsigned mxGetNumberOfDimensions(const mxArray* a) {
71     ERRMSG_NOMEX; return 0;
72 };
73
74 const size_t* mxGetDimensions(const mxArray* a) {
75     ERRMSG_NOMEX; return (const size_t*)NULL;
76 };
77
78 mxArray* mxGetFieldByNumber(const mxArray *a, unsigned i, unsigned j) {
79     ERRMSG_NOMEX; return (mxArray*)NULL;
80 };
81
82 const char* mxGetFieldNameByNumber(const mxArray *a, int fid) {
83     ERRMSG_NOMEX; return (const char*)NULL;
84 };
85
86 int mxGetFieldNumber(const mxArray *a, const char *fld) {
87     ERRMSG_NOMEX; return 0;
88 };
89
90 int mxSetFieldByNumber(mxArray* S, unsigned i, unsigned j, mxArray* a) {
91     ERRMSG_NOMEX; return 0;
92 };
93
94 mxArray* mxGetField(const mxArray *a, mwIndex idx, const char *fld) {
95     ERRMSG_NOMEX; return (mxArray*)NULL;
96 };
97
98 int mxAddField(const mxArray *a, const char *fld) {
99     ERRMSG_NOMEX; return 0;
100 };
101
102 int mxGetString(const mxArray* a, char *ostr, int len) {
103     ERRMSG_NOMEX; return 0;
104 };
105
106 mxArray* mxCreateString(const char *s) {
107     ERRMSG_NOMEX; return (mxArray*)NULL;
108 };
109
110 mxArray* mxCreateDoubleMatrix(size_t d1, size_t d2, char mxreal) {
111     ERRMSG_NOMEX; return (mxArray*)NULL;
112 };
113
114 mxArray* mxCreateNumericArray(int d, size_t *dims, int cflag, int rflag) {
115     ERRMSG_NOMEX; return (mxArray*)NULL;
116 };
117
118 mxArray* mxCreateSparse(size_t m, size_t n, size_t nzmax, mxComplexity cflag) {
119     ERRMSG_NOMEX; return (mxArray*)NULL;
120 };
121
122 mxArray* mxCreateCharArray(int d, size_t *dims) {
123     ERRMSG_NOMEX; return (mxArray*)NULL;
124 };
125
126 mxArray* mxCreateStructMatrix(size_t d1, size_t d2, int nf, const char** fnames) {
127     ERRMSG_NOMEX; return (mxArray*)NULL;
128 };
129
130 mxArray* mxCreateCellMatrix(size_t d1, size_t d2) {
131     ERRMSG_NOMEX; return (mxArray*)NULL;
132 };
133
134 mxArray* mxGetCell(const mxArray* S, unsigned idx) {
135     ERRMSG_NOMEX; return (mxArray*)NULL;

```



```

136 };
137
138 int mxSetCell(mxArray* S, unsigned idx, mxArray* b) {
139     ERRMSG_NOMEX; return 0;
140 };
141
142 int mexPutVariable(const char *ws, const char *vname, mxArray *a) {
143     ERRMSG_NOMEX; return 0;
144 };
145
146 int matPutVariable(MATFile *mfp, const char *vname, const mxArray *a) {
147     ERRMSG_NOMEX; return 0;
148 };
149
150 mxArray* matGetVariable(MATFile *mfp, const char *vname) {
151     ERRMSG_NOMEX; return (mxArray*)NULL;
152 };
153
154 mxArray* matGetVariableInfo(MATFile *mfp, const char *vname) {
155     ERRMSG_NOMEX; return (mxArray*)NULL;
156 };
157
158 int matDeleteVariable(MATFile *mfp, const char *name) {
159     ERRMSG_NOMEX; return 0;
160 };
161
162 MATFile* matOpen(const char *file, const char *mode) {
163     ERRMSG_NOMEX; return 0;
164 };
165
166 int matClose(MATFile *mfp) {
167     ERRMSG_NOMEX; return 0;
168 };
169

```

5.10 nrgdata.hh

```

1 #ifndef __WB_NRGDATA_HCC__
2 #define __WB_NRGDATA_HCC__
3
4 /* ----- //
5 // change log
6
7 Wb, Feb22, 19: changed internally K, T labels to K, D labels,
8 besides that T is also temperature, TK is Kondo tempearture
9 TD is a template data type name, etc.
10
11 // ----- */
12
13 // NB! make sure SkipZeroData() skips only numerical noise! using plain
14 // DEPS=DBL_EPSILON can lead to trace(eig(rho)) inconsistency, otherwise.
15 // with the main difference coming from considering EIG(rho) instead of rho!
16 // => keep 1E-6 buffer w.r.t. numerical noise for large data blocks.
17 // Wb, Mar31, 11
18 const double DEPS2=1E-6*DBL_EPSILON;
19
20 template <class TQ, class TD>
21 class NRGData;
22
23 #define NRG_FLEN 128
24 const char NRG_EMPTY_STR[]="(NRG) ";
25
26 unsigned NRG_N, NRG_ITER;
27 char NRG_FILE[NRG_FLEN];
28
29 unsigned STRICT_ITER0=1;
30
31 Wb::Clock fdmTime_0("FDM all");
32 Wb::Clock fdmTime_5("NRG data I/O");
33
34 wbvector<double> gES;
35
36 inline double gEScale(int iter) {
37     if (iter<0 || iter>=(int)gES.len) wblog(FL,
38         "ERR %s() index out of bounds (%d/%d)", FCT, iter, gES.len);
39     return gES[iter];
40 };
41
42 void wblog_ERR_updateOp(
43     const char *F, int L, const char *istr, int i, int nop, int iter) {
44
45     wblog(F_L,
46         "ERR failed to update operator (%s-op %d/%d @ iter=%d)",

```

```

47     istr && istr[0] ? istr : "(fdm)", i+1,nop,iter
48 );
49 }
50
51 template <class TQ, class TD, class TZ>
52 void WeightE0_dim1(QSpace<TQ,TZ> &X, const QSpace<TQ,TD> &H, double w);
53
54 double nrgGetE0(
55     const wbvector<double> &dE,
56     const wbvector<double> &ES,
57     wbvector<double> &E0
58 ){
59     if (E0.len!=dE.len) { E0.init(dE.len); }
60     if (!E0.len) { wblog(FL,"WRN %s() got empty E0 !?",FCT); return 0; }
61     if (dE.len!=ES.len) wblog(FL,
62         "ERR %s() length mismatch (%d/%d)",FCT,dE.len,ES.len);
63
64     unsigned k=dE.len-1;
65     double x=0, xl=dE[k]*ES[k]; E0[k]=0;
66
67     for (; k; --k) {
68         x=dE[k-1]*ES[k-1];
69         E0[k-1]=E0[k]-xl;
70         xl=x;
71     }
72
73     return x-E0[0];
74 };
75
76 template <class TQ, class TD>
77 int getSpecDimOps(
78     const wbvector<QSpace<TQ,TD> > &FF, wbvector<double> &fac
79 ){
80     unsigned i=0, r,m, n=FF.len, gotfac=0; widx_t nq;
81     fac.init2val(n,1);
82
83     for (; i<n; ++i) {
84         r=FF[i].rank(FL); if (r<=2) continue;
85         if (r>3) wblog(FL,"ERR %s() invalid rank-%g !?",FCT,r);
86
87         m=FF[i].getDim(2,&nq);
88         if (int(nq)<1) wblog(FL,"ERR %s() %d/%d !?",FCT,m,nq);
89         if (nq>1) { fac[i]=nq; ++gotfac; }
90     }
91     return gotfac;
92 };
93
94 class RIXS_para {
95
96 public:
97
98     RIXS_para()
99         : Eoffset(0.), sigma(0.), afac(-1.), wrxs(0),
100         E0(0.), E2(0.), dloc(0), iter(0) {};
101
102     void init() {
103         Einc=0; Eoffset=sigma=E0=E2=0; wrxs=0; dloc=iter=0;
104         afac=-1;
105     };
106
107     void setE0(int it, double e0, double e2) {
108         iter=it; E0=e0+Eoffset; E2=e2;
109     }
110
111     void check(const char *F=0, int L=0, char init=0);
112
113     Wb::String toStr() const;
114     mxArray* toMx() const;
115
116     wbcplx Einc;
117     double Eoffset;
118     double sigma;
119     double afac;
120     char wrxs;
121
122     double E0, E2;
123     unsigned dloc;
124     int iter;
125
126 private:
127 };
128
129 RIXS_para gRX;
130
131 void RIXS_para::check(const char *F, int L, char init) {
132
133     if (Einc.i<=0) {

```

```

134     if (!Einc.i && init) {
135         Einc.i=gES.last();
136     }
137     else wblog(F_L,
138         "ERR %s() got imag(Einc)=%g < 0 (RIXS) !?",FCT,Einc.i
139     );
140 }
141 else if (F && init) {
142     if (Einc.i<1E-2*Einc.r || Einc.i>1E2*Einc.r) wblog(FL,
143         "WRN %s() got Einc.i / Einc_r = %.3g / %.3g = %.3g",
144         PROG, Einc.i, Einc.r, Einc.i/Einc.r
145     );
146 }
147
148 if (sigma<0) wblog(F_L,
149     "ERR RIXS::%s() invalid sigma=%g<0",FCT,sigma);
150 if (int(dloc)<=1 || dloc>(1<10)) wblog(F_L,
151     "ERR RIXS::%s() invalid dloc=%d",FCT,dloc);
152
153 if (init) {
154     if (!sigma) {
155         if (Einc.r) { sigma = Einc.i/Einc.r; }
156         else wblog(FL,"ERR %s() got bot sigma and real(Einc) zero !?",FCT);
157     }
158     else if (sigma<1E-3 && F) {
159         wblog(FL,"WRN %s() got sigma = %g",PROG,sigma);
160     }
161 }
162
163 };
164
165 Wb::String RIXS_para::toStr() const {
166
167     Wb::String s(64);
168     s.pushf("Einc = %.3g",Einc.r);
169
170     if (Einc.r) {
171         double x=Einc.i/Einc.r;
172         if (x && fabs((x-sigma)/x)<1E-4)
173             s.pushf(" @ sigma=%.3g",sigma);
174         else s.pushf(" @ %.3gi, sigma=%g",x,sigma);
175     }
176     else { s.pushf(" + %.3gi, sigma=%g",Einc.i,sigma); }
177
178     if (wrxs ) { s.pushf(" mode=%d",wrxs); }
179     if (afac>=0) { s.pushf(" afac=%g",afac); }
180
181     return s;
182 };
183
184 mxArray* RIXS_para::toMx() const {
185     const char* fields[]={
186         "Einc","Eoffset","sigma","wrxs","dloc";
187
188     mxArray* a=mxCreateStructMatrix(1,1,5,fields);
189     mxSetFieldByNumber(a,0,0, numtoMx(Einc ));
190     mxSetFieldByNumber(a,0,1, numtoMx(Eoffset));
191     mxSetFieldByNumber(a,0,2, numtoMx(sigma));
192     mxSetFieldByNumber(a,0,3, numtoMx(wrxs ));
193     mxSetFieldByNumber(a,0,4, numtoMx(dloc ));
194
195     return a;
196 };
197
198 template <class TQ, class TD>
199 class NRGIndex {
200
201 public:
202
203     NRGIndex() : idx(-1), nrg(NULL) {};
204     NRGIndex(const NRGIndex &I) : idx(I.idx), nrg(I.nrg), tag(I.tag) {};
205
206     void init() { idx=-1; nrg=NULL; };
207     void init(int i, NRGData<TQ,TD> *d) { idx=i; nrg=d; }
208     void init(int i, const char *t) { idx=i; tag=t; }
209
210     mxArray* toMx() const;
211     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
212     void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
213
214     void put (const char *vname, const char *ws="base") const {
215         mxArray *S=toMx();
216         mxPutAndDestroy(FL,S,vname,ws);
217     };
218
219     int idx;
220     NRGData<TQ,TD> *nrg;

```

```

221     wstring tag;
222
223 private:
224 };
225
226 template <class TQ, class TD>
227 mxArray* NRGIndex<TQ,TD>::mxCreatStruct(unsigned m, unsigned n) const {
228     const char *fields[] = {"idx","nrg","tag"};
229     return mxCreatStructMatrix(m,n,3,fields);
230 };
231
232 template <class TQ, class TD>
233 void NRGIndex<TQ,TD>::add2MxStruct(
234     mxArray *S, unsigned i, char tst __attribute__((unused))
235 ) const {
236
237     mxSetFieldByNumber(S,i,0, numtoMx(idx));
238     mxSetFieldByNumber(S,i,1, numtoMx((unsigned long)(nrg)));
239     mxSetFieldByNumber(S,i,2, mxCreatString(tag.isEmpty() ? tag.data:""));
240 };
241
242 namespace Wb {
243
244     template <class TQ, class TD>
245     int updateOp(const char *F, int L,
246         const NRGData<TQ,TD> &NRG,
247         wvector< QSpace<TQ,TD> > &F12,
248         const QSpace<TQ,TD> &A1,
249         const QSpace<TQ,TD> &A2,
250         const wvector< QSpace<TQ,TD> > &FKK,
251         unsigned iter
252     );
253
254     template<class TT>
255     void initLocalOp(const char *F, int L,
256         const mxArray* a, TT &C,
257         int k=0, char force=0, char *name=NULL
258     );
259
260     template<class TQ, class TD>
261     inline void initLocalOp_aux(const char *F, int L,
262         const mxArray* a, wvector< QSpace<TQ,TD> > &C
263     ){ mXInitQSpaceVec(F,L,a,C); }
264
265     template<class TQ, class TD>
266     inline void initLocalOp_aux(const char *F, int L,
267         const mxArray* a, wMatrix< QSpace<TQ,TD> > &C
268     ){ mXInitQSpaceMat(F,L,a,C); }
269
270     template<class TQ, class TD>
271     inline void initLocalOp_aux(const char *F, int L,
272         const mxArray* a, wvector< wvector< QSpace<TQ,TD> > > &C
273     ){ mXInitQSpaceVecVec(F,L,a,C); }
274 };
275
276 template <class TQ, class TD>
277 class NRGData {
278 public:
279
280     NRGData(const char *s=NULL)
281         : name(s), calc(0), store(0), kloc(0), locMX(0), MX(NULL), aux(NULL) {};
282
283     NRGData(const NRGData &A)
284         : name(A.name), nrgIdx(A.nrgIdx), calc(0), store(0), kloc(A.kloc),
285         locMX(0), MX(NULL), aux(NULL)
286     {
287         wblog(FL,"ERR generating copy of NRGData !?");
288     };
289
290     ~NRGData() {
291         if (MX && !NAME.isEmpty() && !locMX) wblog(FL,
292             "WRN ** data return '%s' not implemented yet **\n(%lX; %d)",
293             NAME.data, MX, locMX);
294         init();
295     };
296
297     void initName(const char *n) { name=n; };
298
299     void init(const char *F, int L, const mxArray* a,
300         wvector< QSpace<TQ,TD> > &C0,
301         int k=0, char force=0, const char *vname=0
302     );
303
304     void init(const char *F, int L, const char *XX, int iter=-1);
305
306     void initXX(const char *F, int L, int m=4, int iter=-1) {
307         if (m==1 || m==4) { init(F,L,"KK",iter); } else

```

```

308         if (m==2 || m>4) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
309         if (m>2) {
310             init(F,L,"KD",iter);
311             init(F,L,"DK",iter);
312             init(F,L,"DD",iter);
313         }
314     };
315
316 void initX(const char *F, int L, int m=2, int iter=-1) {
317     if (m==2) { init(F,L,"D",iter); }
318     else if (m!=1) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
319     init(F,L,"K",iter);
320 };
321
322 void init(char flag=0) {
323     if (!flag) { name.init(); NAME.init(); }
324
325     nrgIdx.init(); calc=0; store=0;
326     if (aux) {
327         mxDestroyArray(aux);
328         aux=NULL;
329     }
330
331     if (MX && locMX) { mxDestroyArray(MX); }
332     MX=NULL; locMX=0;
333
334     KK.init(); DK.init(); K.clearQSpace();
335     KD.init(); DD.init(); D.clearQSpace();
336 };
337
338 void setupMX(
339     const char *F, int L, int N, char store_=1, char locMX_=1
340 ){
341     if (MX || locMX) wblog(F,L,
342         "ERR %s() already got MX=%p (%d,%d) !?",FCT,MX,locMX,store);
343     MX=mxCreateStructMatrix(1,N,0,NULL);
344     store=store_; locMX=locMX_;
345 };
346
347 void PUT(
348     const char *F, int L, const char *vn, const char *ws="caller"
349 ){
350     if (MX && locMX) {
351         if (!vn || !vn[0]) wblog(FL,"ERR %s() invalid usage",FCT);
352         int i=mexPutVariable(ws,vn,MX);
353         if (!i) wblog(FL,"I/O putting %s to %s",vn,ws);
354         else wblog(FL,"ERR %s() mexPutVariable() returned i=%d !?",FCT,i);
355     }
356     else wblog(FL,"WRN %s() failed having %p (%d)",FCT,MX,locMX);
357 };
358
359 void check_dloc(
360     const char *F, int L, const char *fun,
361     unsigned dloc, unsigned iter) const;
362
363 template<class TD_>
364 int checkOpConsistency(
365     const char *F, int L, const NRGData<TQ,TD_> &B) const;
366
367 bool isQSpaceVec(const char *nm, int iter=-1);
368
369 bool checkVec(const char *F, int L,
370     const char *XX, unsigned R, const char *istr="");
371
372 bool checkVecVec(const char *XX, unsigned r);
373
374 void forceCalc();
375 void getRelevantOps(wbindex &I) const;
376
377 wbvector< QSpace<TQ,TD> >& getOpsXX(const char *t) {
378     if (t && strlen(t)==2)
379         switch (t[0]) {
380             case 'K': return (t[1]=='K' ? KK : KD);
381             case 'D': return (t[1]=='K' ? DK : DD);
382         }
383     wblog(FL,"ERR invalid tag '%s'", t ? t : "(null)");
384     return KK;
385 };
386
387 const QSpace<TQ,TD>& getQSpace(const char *t, unsigned i=0) const;
388 QSpace<TQ,TD>& getQSpace(const char *t, unsigned i=0) {
389     return const_cast<QSpace<TQ,TD>&>(
390         static_cast<const NRGData<TQ,TD>*>(this)->getQSpace(t,i));
391 }
392
393 const QSpace<TQ,TD>& getQSpace(char t1, char t2, unsigned i=0) const {
394     char t[3]={t1,t2,0}; return getQSpace(t,i);

```

```

395     };
396     QSpace<TQ,TD>& getQSpace(char t1, char t2, unsigned i=0) {
397         char t[3]={t1,t2,0}; return getQSpace(t,i);
398     };
399
400     const QSpace<TQ,TD>& getQSpace(char t) const {
401         switch (t) {
402             case 'K': return K;
403             case 'D': return D;
404         }
405         wblog(FL,"ERR invalid tag '%c' (%d)",t,t);
406         return K;
407     };
408     QSpace<TQ,TD>& getQSpace(char t) {
409         return const_cast<QSpace<TQ,TD>&>(
410             static_cast<const NRGData<TQ,TD>*>(this)->getQSpace(t));
411     };
412
413     void info(const char *vstr=NULL) const;
414     void dispIdx(const char *F, int L) const;
415     void info(const char *F, int L, const char *istr=" * ") const;
416     void disp_otype(const char *F=0, int L=0) const;
417
418     const mxArray* getMxInfo(const char *n);
419
420     template <class TX>
421     void getMxInfo(const char *F, int L, const char *v, TX &X) {
422         const mxArray *a=getMxInfo(v);
423         if (a) {
424             X.init(F_L,a);
425         }
426         else wblog(F_L,
427             "ERR failed to read '%s' from %s_info.mat",v,wnrg());
428     };
429
430     mxArray* getMxData(
431         const char *nm, int iter=-1, char ionly=0);
432
433     const char* wnrg() {
434         if (NAME.data) return NAME.data;
435         return (name.data ? name.data : NRG_EMPTY_STR);
436     };
437
438     mxArray* toMx() const;
439
440     void put(const char *vname, const char* ws="base") {
441         mxPutAndDestroy(FL, toMx(), vname);
442     };
443
444     void put(const char *F, int L, const char *vname, const char* ws="base"
445     ){ wblog(F,L,"I/O putting '%s' to %s", vname, ws);
446         mxPutAndDestroy(FL, toMx(), vname);
447     };
448
449     template<class T>
450     int getPara(const char *name, T &x, int iter=-1, char i=0) {
451 wblog(FL,"ERR %s() check!",FCT);
452         mxArray *a=getMxData(iter);
453         return mxGetNumber(a,x);
454     }
455
456     bool opExists(char tflag=1);
457
458     void initOp(unsigned n=1, char calc_=1, char store_=0) {
459         KK.initDef(n); DK.initDef(n); nrgIdx.initDef(n);
460         KD.initDef(n); DD.initDef(n); calc=calc_; store=store_;
461     };
462
463     bool initOp(const char *F, int L,
464         const wbvector< QSpace<TQ,TD> > &C0,
465         char tflag=1,
466         NRGData<TQ,TD>* CR=NULL,
467         const char *istr="")
468     );
469
470     bool initOp(const char *F, int L,
471         const wbvector< wbvector< QSpace<TQ,TD> > > &CC,
472         NRGData<TQ,TD>* CR=NULL,
473         const char *istr="",
474         char tflag=1
475     );
476
477     void initSym(const char *tag);
478
479     void updatePara(
480         const char *F, int L,
481         const char *tag, mxArray *, int iter=-1

```

```

482     ) const;
483
484     void updatePara(const char *F, int L,
485         const char *tag, int iter=-1,
486         const char *tag2=NULL, char all=-1
487     ) const;
488
489     void updateOp(const char *F, int L,
490         const NRGData<TQ,TD> &A,
491         const NRGData<TQ,TD> &B, unsigned nop, int iter=-1
492     );
493
494     template<class TC>
495     void updateOpProd(
496         const char *F, int L, unsigned iter,
497         const NRGData<TQ,TD> &B,
498         const NRGData<TQ,TC> &C
499     );
500
501     void updateOp(const char *F, int L,
502         const NRGData<TQ,TD> &A, unsigned nop, int iter=-1
503     ) { updateOp(F,L,A,A,nop,iter); };
504
505     template <class TA>
506     void backPropagateOp(const char *F, int L,
507         const NRGData<TQ,TA> &A0,
508         const NRGData<TQ,TA> &A2
509     );
510
511     template<class TD_>
512     void initBG(
513         const char *F, int L,
514         const NRGData<TQ,TD_> &B,
515         const NRGData<TQ,TD_> &H1, const NRGData<TQ,TD_> &H2,
516         int m=3
517     );
518
519     void unsetAllRefs();
520     unsigned skipOpZeros(double eps=DEPS, char bflag='b');
521
522     void saveOp(const char*, int, int iter=-1, unsigned m=4) const;
523
524     void times(TD fac, const char *flag);
525
526     void setupIO(const char*, int,
527         const char*, const mxArray*, const char *istr=0);
528
529     void updateInfo(
530         const char *F, int L,
531         const char*, mxArray *, char keep=-1
532     ) const;
533
534     void updateInfop(
535         const char *F, int L,
536         const char *vtag, mxArray *, char keep=-1
537     ) const;
538
539     const char* getFileName(unsigned k, char lenient=0) const {
540         if (!lenient && k>=NRG_N) wblog(FL,
541             "ERR %s - index out of bounds (%d/%d)", __FUNCTION__, k, NRG_N);
542         return getFileName_aux( k);
543     };
544
545     wbsting name;
546
547     wbvector< NRGIndex<TQ,TD> > nrgIdx;
548
549     char calc, store;
550     int kloc;
551     char locMX;
552
553     wbvector< wbvector< QSpace<TQ,TD> > > CI;
554
555     wbvector< QSpace<TQ,TD> > KK, KD, DK, DD;
556
557     QSpace<TQ,TD> K, D;
558
559     wbvector<char> issym;
560
561     mxArray* MX;
562     wbsting NAME;
563
564     mxArray *aux;
565
566 private:
567
568     const char* getFileNameI() const {

```

```

569         return getFileName_aux(-1);
570     };
571
572     const char* getFileName_aux(int k) const {
573         int i;
574         if (k>=0)
575             i=snprintf(NRG_FILE, NRG_FLEN, "%s_02d.mat", NAME.data, k);
576         else i=snprintf(NRG_FILE, NRG_FLEN, "%s_info.mat", NAME.data);
577         if (i>=NRG_FLEN) wblog(FL, "ERR File name exceeds length (%d/127)", i);
578         return NRG_FILE;
579     }
580 };
581
582 template<class TQ, class TD>
583 void NRGData<TQ, TD>::setupIO(
584     const char *F, int L, const char *s, const mxArray *a,
585     const char *istr
586 ){
587     if (!s || !s[0]) {
588         if (!a || (!mxIsCell(a) && !mxIsStruct(a))) wblog(F, L,
589             "ERR invalid input NRG data (%s)", a ? mxGetClassName(a): "NULL");
590         NAME.init(); NRG_N=0; NRG_ITER=0;
591         MX=(mxArray*)a;
592     }
593     else {
594         Wb::matFile f; unsigned n=strlen(s);
595         Wb::Clock_resume sw(&fdmTime_5);
596
597         if (!n || n+16>NRG_FLEN) wblog(FL,
598             "ERR invalid file/variable name '%s' (%d/%d)", s, n, NRG_FLEN);
599
600         NRG_N=0; NRG_ITER=0; MX=NULL;
601         NAME=s;
602
603 #ifdef MATLAB_MEX_FILE
604         mxArray *a=mexGetVariable("caller", s);
605         if (a && (mxIsStruct(a) || mxIsCell(a))) {
606             if (istr) wblog(F, L, "<i> I/O NRG%s structure variable: %s", istr, s);
607
608             MX=a;
609             return;
610         }
611 #endif
612
613         if (!f.open(F, L, getFileName(0, 'l'), "r"))
614             wblog(F, L, "ERR cannot open files of type %s_##.mat", s);
615         else f.close();
616
617         if (istr) {
618             if (strlen(s)<40) {
619                 wblog(F, L, "<i> NRG%s is %s_##.mat", istr, s);
620             }
621             else { char s[256];
622                 if (getcwd(s, 256)==NULL) wblog(FL, "ERR failed read cwd");
623                 wblog(F, L, "<i> using NRG%s data ...", istr);
624                 printf("\n  pwd: %s\n  I/O: %s_##.mat\n\n",
625                     Wb::repHome(s).data, Wb::repHome(s).data);
626             }
627         }
628     }
629 };
630
631 template<class TQ, class TD>
632 void NRGData<TQ, TD>::init(const char *F, int L, const char *XX, int iter) {
633
634     char s[64]; unsigned n=strlen(XX), e=0;
635     mxArray *a;
636
637     if (!n || n>2) wblog(FL, "ERR invalid operator tag '%s'", XX);
638     if (iter<0) iter=NRG_ITER;
639
640     strncpy(s, name.data, 60); s[60]=0; strcat(s, XX);
641     a=getMxData(s, iter);
642     if (!a) wblog(FL, "ERR reading NRGData '%s' (iter=%d) !?", s, iter);
643
644     if (n==1) {
645         if (!strcmp(XX, "K")) K.init(F, L, a); else
646         if (!strcmp(XX, "D")) D.init(F, L, a); else ++e;
647     }
648     else {
649         if (!strcmp(XX, "KK")) mXInitQSpaceVec(F, L, a, KK); else
650         if (!strcmp(XX, "KD")) mXInitQSpaceVec(F, L, a, KD); else
651         if (!strcmp(XX, "DK")) mXInitQSpaceVec(F, L, a, DK); else
652         if (!strcmp(XX, "DD")) mXInitQSpaceVec(F, L, a, DD); else ++e;
653     }
654     if (e) wblog(FL, "ERR invalid operator tag '%s'", XX);
655 };

```



```

656
657 template<class TQ, class TD>
658 void NRGData<TQ,TD>::init(
659     const char *F, int L, const mxArray* a,
660     wbvector< QSpace<TQ,TD> > &C0,
661     int k,
662     char force,
663     const char *vname0
664 ){
665     char nstr[32]; nstr[0]=0;
666
667     Wb::initLocalOp(F,L,a,C0,k,force,nstr);
668
669     if (vname0 && vname0[0]) { name=vname0; }
670     else if (nstr[0]) { name=nstr; }
671
672 }
673
674 template<class TT>
675 void Wb::initLocalOp(const char *F, int L,
676     const mxArray* a,
677     TT &C,
678     int k,
679     char force,
680     char *name
681 ){
682     size_t n=12; char istr[n];
683     if (k)
684         snprintf(istr,n,"arg #%d",k);
685     else strcpy(istr,"operator");
686
687     if (!a || mxIsEmpty(a)) {
688         if (force) wblog(F,L,"ERR invalid %s%s",istr,a ? " (empty)":"" );
689         return;
690     }
691
692     if (mxIsChar(a)) {
693         const unsigned n=32; wbstring aux;
694
695         if (name==NULL) { aux.init(n+1); name=aux.data; }
696         name[0]=0;
697
698         if (mxGetString(a,name,n) || name[0]==0) wblog(F,L,
699             "ERR invalid string of %s%s",istr,name[0] ? "":" (empty)");
700
701         a=mexGetVariablePtr("caller",name);
702
703         if (!a) wblog(F,L,
704             "ERR Can not read QSpace '%s' (%s) from workspace",name,istr);
705     }
706
707     Wb::initLocalOp_aux(F,L,a,C);
708
709     if (force && C.isEmpty()) wblog(F,L,
710         "ERR invalid QSpace (%s is empty)",istr);
711 };
712
713 template<class TQ, class TD>
714 void NRGData<TQ,TD>::forceCalc() {
715
716     wbvector< NRGIndex<TQ,TD> > &I=nrgIdx;
717     if (I.len==0) return;
718
719     for (unsigned i=0; i<I.len; ++i) {
720         if (I[i].idx<0) continue;
721         if (I[i].nrg==NULL) I[i].idx=-1;
722         if (!I[i].tag.isEmpty()) {
723             I[i].init(); I[i].idx=-1;
724         }
725     }
726
727     calc=store=1;
728 }
729
730 template<class TQ, class TD>
731 void NRGData<TQ,TD>::updateOp(
732     const char *F, int L,
733     const NRGData<TQ,TD> &A, const NRGData<TQ,TD> &B,
734     unsigned nop,
735     int iter
736 ){
737     unsigned i=0,j,nrefs=0, m=nrgIdx.len;
738     const char* tags[4] = { "KK","KD","DK","DD" };
739     int e=0,k;
740
741     wbvector< QSpace<TQ,TD> > X, X2;
742

```

```

743     if (iter<0) iter=NRG_ITER;
744     if (nrgIdx.len==0) return;
745
746     for (; i<m; ++i) { if (nrgIdx[i].idx>=0) ++nrefs; }
747
748     unsetAllRefs();
749
750     if (nrefs<nrgIdx.len) {
751         if (nop==4) {
752             Wb::updateOp(F,L,*this, DD, A.D, B.D, KK, iter);
753             Wb::updateOp(F,L,*this, DK, A.D, B.K, KK, iter);
754             Wb::updateOp(F,L,*this, KD, A.K, B.D, KK, iter);
755             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
756         }
757         else if (nop==3) {
758             Wb::updateOp(F,L,*this, DD, A.D, B.D, KK, iter);
759             Wb::updateOp(F,L,*this, DK, A.D, B.K, KK, iter);
760             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
761
762             if (KD.len!=DK.len) KD.initDef(DK.len);
763             for (i=0; i<DK.len; ++i) DK[i].transp(KD[i]);
764         }
765         else if (nop==1) {
766             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
767         }
768         else wblog(FL,"ERR %s() invalid nop (%d)",FCT,nop);
769     }
770
771     if (e) wblog(FL,
772         "ERR %s() operator %s[%d] updates to empty QSpace!",
773         FCT,name.data,iter
774     );
775
776     if (nrefs) {
777         for (i=0; i<nop; ++i) {
778
779             wvector< QSpace<TQ,TD> > &C = getOpsXX(tags[i]);
780             if (C.len!=m) C.initDef(m);
781
782             X.init(); X2.init();
783
784             for (j=0; j<m; ++j) {
785                 k=nrgIdx[j].idx; if (k<0) continue;
786
787                 if (nrgIdx[j].nrg) {
788                     wvector< QSpace<TQ,TD> > &R=nrgIdx[j].nrg->getOpsXX(tags[i]);
789
790                     if (k>=(int)R.len) wblog(FL,
791                         "ERR index out of bounds (%d/%d)",k,R.len);
792
793                     C[j].init2ref(R[k]);
794                 }
795                 else if (nrgIdx[j].tag.isEmpty()) {
796                     if (!X.len) {
797                         sprintf(str,"%s%s", name.data, tags[i]);
798                         mxInitQSpaceVecR23(F,L,getMxData(str,iter),X);
799                     }
800                     if (k>=(int)X.len) wblog(FL,
801                         "ERR index out of bounds (%d/%d; %d)",k,X.len,nrgIdx.len);
802
803                     X[k].save2(C[j]);
804                 }
805                 else {
806                     if (!X2.len) {
807                         sprintf(str,"%s%s", nrgIdx[j].tag.data, tags[i]);
808                         mxInitQSpaceVecR23(F,L, getMxData(str,iter),X2);
809                     }
810                     if (k>=(int)X2.len) wblog(FL,
811                         "ERR index out of bounds (%d/%d)",k,X2.len);
812
813                     X2[k].save2(C[j]);
814                 }
815             }
816         }
817     };
818
819     template <class TQ, class TS>
820     template<class TD>
821     void NRGData<TQ,TS>::updateOpProd(
822         const char *F, int L, unsigned iter,
823         const NRGData<TQ,TS> &B,
824         const NRGData<TQ,TD> &C
825     ){
826         unsigned i=0,r,n;
827
828         QSpace<TQ,TS> Xd;
829

```

```

830 char ics[13] = "2 2* 23 23*", *ic=0, *ic_=0;
831 for (; ics[i]; ++i) { if (ics[i]!=' ') ics[i]=0; }
832
833 n=B.checkOpConsistency(0,1,C);
834 if (int(n)<0) wblog(FL,
835     "ERR got dim/itag inconsistency between B and C !?");
836 if (!n || B.KK.len!=n || B.DD.len!=n) wblog(FL,
837     "ERR %s() got empty operators (%d/%d/%d) !?",FCT,B.KK.len,B.DD.len,n);
838 KK.init(n); KD.init(n); DK.init(n); DD.init(n);
839
840 for (i=0; i<n; ++i) {
841     r=B.KK[i].rank(FL); if (!r) r=B.DD[i].rank(FL);
842     if (r==2) { ic=ics+0; ic_=ics+3; } else
843     if (r==3) { ic=ics+6; ic_=ics+9; } else
844     wblog(FL, "ERR %s() invalid rank r=%d (i=%d)",FCT,r,i);
845
846     B.KK[i].contract(F,L,ic,C.KK[i],ic_,KK[i]);
847     B.KD[i].contract(F,L,ic,C.KD[i],ic_,Xd); KK[i]+=Xd;
848
849     B.KK[i].contract(F,L,ic,C.DK[i],ic_,KD[i]);
850     B.KD[i].contract(F,L,ic,C.DD[i],ic_,Xd); KD[i]+=Xd;
851
852     B.DK[i].contract(F,L,ic,C.KK[i],ic_,DK[i]);
853     B.DD[i].contract(F,L,ic,C.KD[i],ic_,Xd); DK[i]+=Xd;
854
855     B.DK[i].contract(F,L,ic,C.DK[i],ic_,DD[i]);
856     B.DD[i].contract(F,L,ic,C.DD[i],ic_,Xd); DD[i]+=Xd;
857
858     if (KK[i].itags.len>2) wblog(FL,
859         "ERR %s() KK[%d] '%s' !?",FCT,i+1,IT2STR(KK[i]));
860     if (DD[i].itags.len>2) wblog(FL,
861         "ERR %s() DD[%d] '%s' !?",FCT,i+1,IT2STR(DD[i]));
862 }
863 };
864
865 template <class TQ, class TS, class TD>
866 void applyBG(const char *F, int L,
867     QSpace<TQ,TS> &B,
868     const QSpace<TQ,TD> &H0,
869     const QSpace<TQ,TD> &H2
870 ){
871     unsigned i,j,k,m,n;
872
873     wvector<unsigned> I1,I2;
874     wvector<double> E1,E2;
875     windex ilh,i2h;
876
877     warray<TD> Omega;
878     TD *e1,*e2;
879
880     wcomplex z, Einc = gRX.Einc + gRX.E0, dEinc = Einc - gRX.E2;
881
882     gRX.check();
883
884     double e12, sgm=gRX.sigma/2.;
885
886     if (B.isEmpty()) { return; }
887     matchH12(FL,B,H0,H2,gRX.iter, ilh,E1,I1, i2h,E2,I2);
888
889     for (k=0; k<ilh.len; ++k) {
890         warray<TS> &Bk = *B.DATA[k];
891         TS *xk=Bk.data;
892
893         i=ilh[k]; m=I1[i+1]-I1[i];
894         j=i2h[k]; n=I2[j+1]-I2[j];
895
896         if (!Bk.hasSameSize(m,n)) { Bk.info(FL,"B",k);
897             MXPut(FL,"i").add(Bk,"B").add(m,"m").add(n,"n").add(k+1,"k");
898             wblog(FL, "ERR %s() dimension mismatch (%d) !?",FCT,k+1);
899         }
900
901         e1=E1.data+I1[i]; e2=E2.data+I2[j];
902
903         for (j=0; j<n; ++j) {
904             for (i=0; i<m; ++i) { e12 = e2[j] - e1[i];
905                 z.r = dEinc.r - e12;
906                 z.i =-Einc.i;
907
908                 if (!gRX.wrxs) {
909                     z/=sqrt(abs(z.i));
910                 }
911                 else {
912                     wblog(FL, "ERR %s() deprecated using BHeisenberg!!",FCT);
913
914                     z.i -= sgm*(gRX.E0+gRX.E2);
915                     if (z.i>=0) wblog(FL, "ERR %s() got z=%g%+gi",FCT,z.r,z.i);
916

```

```

917         z/=sqrt(abs(z.i));
918     }
919
920     xk[i+m*j]/=z;
921 }
922 }
923 };
924
925 template <class TQ, class TS>
926 template <class TD>
927 void NRGData<TQ,TS>::initBG(
928     const char *F, int L,
929     const NRGData<TQ,TD> &B,
930     const NRGData<TQ,TD> &H0,
931     const NRGData<TQ,TD> &H2,
932     int m
933 ){
934     unsigned i;
935
936     KD.initT_(B.KD); for (i=0; i<KD.len; ++i) applyBG(F_L,KD[i], H0.K, H2.D);
937     DK.initT_(B.DK); for (i=0; i<DK.len; ++i) applyBG(F_L,DK[i], H0.D, H2.K);
938     DD.initT_(B.DD); for (i=0; i<DD.len; ++i) applyBG(F_L,DD[i], H0.D, H2.D);
939
940     if (m==3) return; else
941     if (m!=4) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
942
943     KK.initT_(B.KK); for (i=0; i<KK.len; ++i) applyBG(F_L,KK[i], H0.K, H2.K);
944 };
945
946 template <class TQ, class TD, class TZ>
947 void WeightE0_dim1(QSpace<TQ,TZ> &X, const QSpace<TQ,TD> &H, double a) {
948
949     unsigned i,j,k; widx_t m1,m2,d,D;
950     double efac,w=1;
951
952     wbMatrix<TQ> Qh,Qx;
953     wbindx ih,ix;
954
955     if (X.isEmpty()) { return; }
956     if ((i=X.rank())<2 || i>3 || H.rank()!=2) wblog(FL,
957         "ERR %s() got rank-%d data (%d)",FCT,i,H.rank());
958
959     if (a<0) { w=0; a=-a; } else
960     if (!a) wblog(FL,"ERR %s() invalid w=%g",FCT,a);
961
962     X.getQsub(0,Qx);
963     H.getQsub(0,Qh);
964
965     matchIndex(Qh,Qx,ih,ix,1,&m1,&m2);
966     if (m1 || ix.len!=Qx.dim1) wblog(FL,"ERR %s() "
967         "need unique match (%d/%d; %d/%d)",FCT,m1,m2,ix.len,Qx.dim1);
968     if (ix.len!=X.DATA.len) wblog(FL,"ERR %s() "
969         "severe QSpace inconsistency (%d/%d)!",FCT,ix.len,X.DATA.len);
970
971     for (k=0; k<ix.len; ++k) {
972         wbarray<TZ> &xk=X.DATA[ix[k]];
973         const wbarray<TD> &ek=H.DATA[ih[k]];
974         xk.getMatSize(d,D,1);
975
976         if (!ek.isVector()) wblog(FL,
977             "ERR %s() non-diagonal H (%d: %s)",FCT,k+1,SSTR(ek));
978         if (d!=ek.dim1()) wblog(FL,"ERR %s() "
979             "size mismatch (%d: d=%ld/%ld x %ld)",FCT,k+1,d,ek.dim1(),D);
980
981         for (i=0; i<d; ++i) {
982             efac=exp(-a*ek[i]*ek[i]); if (!w) { efac=1-efac; }
983             efac=sqrt(efac);
984             for (j=0; j<D; ++j) { xk[i+d*j]*=efac; }
985         }
986     }
987 };
988
989 template <class TQ, class TS, class TD>
990 void matchH12(
991     const char *F, int L,
992     QSpace<TQ,TS> &X,
993     const QSpace<TQ,TD> &H1, const QSpace<TQ,TD> &H2, unsigned iter,
994     wbindx &i1h, wvector<double> &E1, wvector<unsigned> &I1,
995     wbindx &i2h, wvector<double> &E2, wvector<unsigned> &I2,
996     char redCGR=0
997 ){
998     unsigned i,j,e=0; widx_t m1,m2;
999     wbindx i1,i2;
1000     wbperm P;
1001
1002     wvector<unsigned> D1,D2;
1003     wbMatrix<TQ> Q1h,Q2h,Q1,Q2;

```

```

1004
1005     double Escale = gEScale(iter);
1006
1007     if (X.isEmpty()) {
1008         wblog(F_L, "ERR %N empty matrix elements !?");
1009     }
1010
1011     if (X.rank(F_L)==3 || !X.CGR.isEmpty()) { X.isConsistent(F_L);
1012         if (X.CGR.isEmpty()) {
1013             if (!X.allAbelian()) { wblog(F_L,
1014                 "ERR %s() expecting CG data for rank-3 data (%s)",
1015                 FCT, X.qtype.toStr().data);
1016             }
1017         }
1018         else {
1019             for (i=0; i<X.DATA.len; ++i) {
1020                 wbararray<TS> &D=(*X.DATA[i]);
1021
1022                 if (D.SIZE.len<2 || D.SIZE.len>3) wblog(F_L,
1023                     "ERR invalid rank operator (%s)", SSTR(D));
1024                 if (D.SIZE.len==3) {
1025                     if (D.SIZE[2]!=1) wblog(F_L,
1026                         "ERR expecting rank-3 to have dim3=1 (%s)", SSTR(D));
1027                     else if (redCGR) { D.SIZE.len=2; }
1028                 }
1029
1030                 if (redCGR) {
1031                     for (j=0; j<X.CGR.dim2; ++j) {
1032                         if (!X.CGR(i,j).isScalar()) wblog(F_L,
1033                             "ERR %s() got non-scalar cgreg (%d,%d): %s",
1034                             FCT, i+1, j+1, X.CGR(i,j).toStr().data
1035                             );
1036                     }
1037                 }
1038             }
1039         }
1040
1041         getEData(H1,E1,D1); E1*=Escale; D1.cumsum_(I1,'x');
1042         getEData(H2,E2,D2); E2*=Escale; D2.cumsum_(I2,'x');
1043
1044         H1.getQsub(0,Q1h); X.getQsub(0,Q1);
1045         H2.getQsub(0,Q2h); X.getQsub(1,Q2);
1046
1047         i=matchIndex(Q1h,Q1,i1h,i1,1,&m1,&m2); if (m1 || i1.len!=Q1.dim1) ++e;
1048         i=matchIndex(Q2h,Q2,i2h,i2,1,&m1,&m2); if (m1 || i2.len!=Q2.dim1) ++e;
1049
1050         if (e) wblog(F_L,
1051             "ERR %s() need unique and exact match (%d)!", FCT, e);
1052         if (i1.len!=X.DATA.len) wblog(F_L,
1053             "ERR %s() severe QSpace inconsistency (%d %d)!",
1054             FCT, i1.len, X.DATA.len
1055         );
1056
1057         i1.toPerm(P); P.Invert(); i1h.Select(P);
1058         i2.toPerm(P); P.Invert(); i2h.Select(P);
1059     };
1060
1061     template<class TQ, class TD>
1062     void NRGData<TQ,TD>::unsetAllRefs() {
1063
1064         unsigned i,j,m=0;
1065         const char* tags[4] = { "KK", "KD", "DK", "DD" };
1066
1067         for (i=0; i<4; ++i) {
1068             wvector< QSpace<TQ,TD> > &C = getOpsXX(tags[i]);
1069
1070             for (j=0; j<C.len; ++j) {
1071                 if (nrgIdx[j].idx<0 || !C[j].isref) continue;
1072                 C[j].init(); ++m;
1073             }
1074         }
1075     }
1076 }
1077
1078     template<class TQ, class TD>
1079     unsigned NRGData<TQ,TD>::skipOpZeros(double eps, char bflag) {
1080
1081         wvector< QSpace<TQ,TD> > *XX[] = { &KK, &KD, &DK, &DD };
1082         unsigned i,j,n,r=0;
1083
1084         for (i=0; i<4; ++i) {
1085             n=XX[i]->len;
1086             for (j=0; j<n; ++j)
1087                 r += (*XX[i])[j].SkipZeroData(eps,bflag);
1088         }
1089
1090         return r;

```

```

1091 }
1092
1093 template<class TQ, class TD>
1094 void NRGData<TQ,TD>::times(TD fac, const char *flag) {
1095
1096     unsigned i,j,n=(flag ? strlen(flag) : 0);
1097
1098     if (n==2) {
1099         if (!strcmp(flag,"XX")) {
1100             const char* tags[4] = { "KK","KD","DK","DD" };
1101             for (i=0; i<4; ++i) {
1102                 wbvector< QSpace<TQ,TD> > &X=getOpsXX(tags[i]);
1103                 for (j=0; j<X.len; ++j) X[j]*=fac;
1104             }
1105         }
1106         else {
1107             wbvector< QSpace<TQ,TD> > &X=getOpsXX(flag);
1108             for (j=0; j<X.len; ++j) X[j]*=fac;
1109         }
1110     }
1111     else if (n==1) {
1112         if (flag[0]=='X') {
1113             getQSpace('K')*=fac;
1114             getQSpace('D')*=fac;
1115         }
1116         else {
1117             getQSpace(flag[0])*=fac;
1118         }
1119     }
1120     else wblog(FL, "ERR invalid flag '%s'", flag ? flag : "(null)");
1121 };
1122
1123 template<class TQ, class TD>
1124 void NRGData<TQ,TD>::saveOp(
1125     const char *F, int L, int iter, unsigned m) const {
1126
1127     unsigned i=0, n=nrgIdx.len, nrefs=0;
1128     const char* tags[4]={"KK","KD","DK","DD"};
1129
1130     if (!m || m==2 || m>4) wblog(FL,
1131         "ERR value out of bounds (m=%d/%d)",m,4);
1132     if (iter<0) iter=NRG_ITER;
1133
1134     for (; i<n; ++i) { if (nrgIdx[i].idx>=0) { ++nrefs; }}
1135
1136     if (store && (!n || nrefs<n)) { i=0;
1137         if (m==3) { i=1; m=4; }
1138         #if 0
1139         if (WBLOG_IO) { sprintf(str,
1140             "%s() iter=%2d: store=%d, nrefs=%d/%ld, i=%d/%d ",
1141             FCT, iter, store, nrefs,nrgIdx.len, i,m);
1142             wblog(FL, MX? "MX: %s":"MAT %s",str);
1143         }
1144         #endif
1145         for (; i<m; ++i) { updatePara(F,L,tags[i],iter); }
1146     }
1147 };
1148
1149 template<class TQ, class TD>
1150 void NRGData<TQ,TD>::updatePara(
1151     const char *F, int L, const char *tag, mxArray *a, int iter) const {
1152
1153     sprintf(str,"%s%s", name.data, tag);
1154     if (!str[0]) wblog(F,L,"ERR invalid name (empty)");
1155
1156     if (iter<0) iter=NRG_ITER;
1157     if (MX) {
1158         if (!NAME.isEmpty())
1159             mxUpdateField(F,L, MX,str,iter,a);
1160     }
1161     else {
1162         Wb::Clock_resume sw(&fdmTime_5);
1163         matPutVariable(F,L,getFileName(iter),str,a);
1164     }
1165 };
1166
1167 template<class TQ, class TD>
1168 void NRGData<TQ,TD>::updatePara(const char *F, int L,
1169     const char *tag,
1170     int iter,
1171     const char *tag2,
1172     char all
1173 ) const {
1174
1175     unsigned n=strlen(tag), e=0;
1176     mxArray *a=0;
1177

```

```

1178     if (n==1) {
1179         if (tag[0]=='K') { a=K.toMx(); } else
1180         if (tag[0]=='D') { a=D.toMx(); } else e|=1;
1181     }
1182     else if (n==2) {
1183         const wvector< QSpace<TQ,TD> > *XX=0;
1184         if (!strcmp(tag,"KK")) { XX=&KK; } else
1185         if (!strcmp(tag,"KD")) { XX=&KD; } else
1186         if (!strcmp(tag,"DK")) { XX=&DK; } else
1187         if (!strcmp(tag,"DD")) { XX=&DD; } else e|=2;
1188
1189         if (!e) {
1190             if (all>0 || (all<0 && store>1)) { a=XX->toMx(); }
1191             else {
1192                 windex I; getRelevantOps(I);
1193                 a=QSpaceVec2Mx(*XX,I);
1194             }
1195         }
1196     }
1197     else { e|=4; }
1198     if (e) wblog(FL,"ERR invalid tag '%s'",tag);
1199
1200     sprintf(str,"%s%s", name.data, tag2 && tag2[0] ? tag2 : tag);
1201     if (!str[0] || !name.isName()) wblog(F,L,"ERR invalid name '%s'",str);
1202
1203     if (iter<0) iter=NRG_ITER;
1204     if (MX) {
1205         mxUpdateField(F,L, MX,str,iter,a);
1206     }
1207     else {
1208         Wb::Clock_resume sw(&fdmTime_5);
1209         matPutVariable(F,L,getFileName(iter),str,a);
1210     }
1211 };
1212
1213 template<class TQ, class TD>
1214 inline void NRGData<TQ,TD>::getRelevantOps(windex &I) const {
1215     unsigned i,k;
1216     I.init(nrgIdx.len);
1217
1218     for (k=i=0; i<nrgIdx.len; ++i) {
1219         if (nrgIdx[i].idx<0) I[k++]=i;
1220     }
1221
1222     I.len=k;
1223 }
1224
1225
1226 template<class TQ, class TD>
1227 const QSpace<TQ,TD>& NRGData<TQ,TD>::getQSpace(
1228     const char *t, unsigned i
1229 ) const {
1230
1231     int d=-1;
1232
1233     if (!t || !index("KD",t[0]) || !index("KD",t[1]) || t[2])
1234         wblog(FL,"ERR invalid tag '%s'", t ? t : "(null)");
1235
1236     if (t[0]=='K') {
1237         if (t[1]=='K')
1238             { if (i<KK.len) return KK[i]; else d=KK.len; }
1239         else { if (i<KD.len) return KD[i]; else d=KD.len; }
1240     }
1241     else {
1242         if (t[1]=='K')
1243             { if (i<DK.len) return DK[i]; else d=DK.len; }
1244         else { if (i<DD.len) return DD[i]; else d=DD.len; }
1245     }
1246
1247     wblog(FL,"ERR index out of bounds (%s: %d/%d)",t,i,d);
1248     return K;
1249 };
1250
1251 template<class TQ, class TD>
1252 void NRGData<TQ,TD>::updateInfo(const char *F, int L,
1253     const char *vname, mxArray *a, char keep
1254 ) const {
1255
1256     if (!a) wblog(F,L,
1257         "WRN update info '%s' with NULL mxArray", vname);
1258
1259     if (MX) {
1260         mexPutVariable("caller", vname, a);
1261         if (!keep) mxDestroyArray(a);
1262     }
1263     else {
1264         Wb::Clock_resume sw(&fdmTime_5);

```

```

1265     matPutVariable(F,L,getFileNamel(),vname,a,keep);
1266 }
1267 }
1268
1269 template<class TQ, class TD>
1270 void NRGData<TQ,TD>::updateInfop(const char *F, int L,
1271     const char *vtag, mxArray *a, char keep
1272 ) const {
1273
1274     if (!name.data || !name.data[0]) wblog(F,L,
1275         "WRN update info '%s' of unnamed operator", vtag);
1276
1277     sprintf(str,"%s%s",
1278         name.data ? name.data : "",
1279         vtag ? vtag : "");
1280 };
1281
1282     updateInfo(F,L,str,a,keep);
1283 }
1284
1285 template<class TQ, class TD>
1286 bool NRGData<TQ,TD>::opExists(char tflag) {
1287
1288     if (tflag) {
1289         unsigned i, k=(NRG_N>2 ? NRG_N-2 : 0);
1290         const char* tag[3] = { "KK","KD","DK" };
1291
1292         for (i=0; i<3; ++i) {
1293             sprintf(str,"%s%s",name.data,tag[i]);
1294             if (!isQSpaceVec(str,k)) return 0;
1295         }
1296     }
1297     else {
1298         sprintf(str,"%sKK",name.data);
1299         return isQSpaceVec(str,0);
1300     }
1301
1302     return 1;
1303 }
1304
1305 template<class TQ, class TD>
1306 bool NRGData<TQ,TD>::initOp(const char *F, int L,
1307     const wvector< QSpace<TQ,TD> > &C0,
1308     char tflag,
1309     NRGData<TQ,TD>* CR,
1310     const char *istr
1311 ){
1312     unsigned i,j,m,iter,e,nrefs=0;
1313     const char* tag[3] = { "KK","KD","DK" };
1314     unsigned ntag=sizeof(tag)/sizeof(char*);
1315     wvector< QSpace<TQ,TD> > NRG;
1316     mxArray *a; int r=-1;
1317
1318     nrgIdx.initDef(C0.len); kloc=0;
1319
1320     if (C0.len==0) {
1321         init(1); return 0;
1322     }
1323
1324     if (istr && istr[0]) sprintf(str,"%s ",istr); else str[0]=0;
1325
1326     if (!name.isName(99)) wblog(F,L,
1327         "ERR invalid name for %soperator (%d, '%s')",
1328         str, nrgIdx.len, name.data
1329     );
1330
1331     for (i=0; i<C0.len; ++i) {
1332         r=-1;
1333         if (!C0[i].isConsistent(r) || r<2 || r>4) wblog(F,L,
1334             "ERR invalid operator (got rank-%d QSpace) !?\n"
1335             "%s (%s)",r,shortFL(FL),str);
1336         if (C0[i].QDIM!=C0[0].QDIM) wblog(FL,
1337             "ERR invalid operator (QDIM=%d, %d) !?\n%s:%d %s",
1338             C0[i].QDIM, C0[0].QDIM,FL,str);
1339     }
1340
1341     KK.initDef(C0);
1342
1343     for (i=0; i<C0.len; ++i) {
1344         if (nrgIdx[i].idx>=0) continue;
1345         for (j=i+1; j<C0.len; ++j)
1346             if (C0[j]==C0[i]) {
1347                 nrgIdx[j].init(i,this);
1348                 ++nrefs;
1349             }
1350     }
1351

```



```

1352     if (CR) {
1353         m=CR->KK.len;
1354         for (i=0; i<C0.len; ++i) {
1355             if (nrgIdx[i].idx>=0) continue;
1356             for (j=0; j<m; ++j) if (C0[i]==CR->KK[j]) {
1357                 nrgIdx[i].init(j,CR);
1358                 ++nrefs;
1359                 break;
1360             }
1361         }
1362         sprintf(str,"%s%s",CR->name.data,"KK_");
1363         a=getMxData(str,0);
1364         if (a && mxIsQSpaceVecR23(FL,a)) {
1365             mxInitQSpaceVecR23(F,L,a,NRG); m=NRG.len;
1366             for (i=0; i<C0.len; ++i) {
1367                 if (nrgIdx[i].idx>=0) continue;
1368                 for (j=0; j<m; ++j) if (C0[i]==NRG[j]) {
1369                     nrgIdx[i].init(j,CR->name.data);
1370                     ++nrefs;
1371                     break;
1372                 }
1373             }
1374         }
1375     }
1376     if (nrefs==nrgIdx.len) {
1377         return nrefs;
1378     }
1379 }
1380
1381 e=0;
1382
1383 if (tflag) {
1384     iter=(NRG_N>2 ? NRG_N-2 : 0);
1385     for (i=0; i<ntag; ++i) {
1386         sprintf(str,"%s%s",name.data,tag[i]);
1387         if (!isQSpaceVec(str,iter)) { ++e; break; }
1388     }
1389 }
1390
1391 iter=0;
1392
1393 sprintf(str,"%s%s",name.data,"KK");
1394 if (!isQSpaceVec(str,iter)) ++e;
1395
1396 sprintf(str,"%s%s",name.data,"KK_");
1397 a=getMxData(str,iter); if (!a || !mxIsQSpaceVecR23(FL,a)) ++e;
1398
1399 if (e) {
1400     calc=store=1;
1401     return nrefs;
1402 }
1403
1404 mxInitQSpaceVecR23(F,L,a,NRG);
1405
1406 for (i=0; i<C0.len; ++i) {
1407     if (nrgIdx[i].idx>=0 && nrgIdx[i].tag.isEmpty()) continue;
1408     for (j=0; j<NRG.len; ++j) if (C0[i]==NRG[j]) {
1409         nrgIdx[i].tag.init();
1410         nrgIdx[i].idx=j; ++nrefs;
1411         break;
1412     }
1413 }
1414
1415 calc= (nrefs==C0.len ? 0 : 1);
1416 store=(nrefs==C0.len ? 0 : 1);
1417
1418 return nrefs;
1419 };
1420
1421 template<class TQ, class TD>
1422 bool NRGData<TQ,TD>::initOp(const char *F, int L,
1423     const wbvector< wbvector< QSpace<TQ,TD> > > &CC,
1424     NRGData<TQ,TD>* CR,
1425     const char *istr,
1426     char tflag
1427 ){
1428     unsigned i,j,l,m,iter,q=0,e, nrefs=0;
1429     const char* tag[3]={ "KK","KD","DK" };
1430     unsigned ntag=sizeof(tag)/sizeof(char*);
1431     const mxArray *a; int r=-1;
1432
1433     wbvector< wbvector< QSpace<TQ,TD> > > NRG;
1434
1435     nrgIdx.initDef(CC.len);
1436
1437     if (CC.len==0) {
1438         init(1); return 0;

```

```

1439     }
1440
1441     if (istr && istr[0])
1442         sprintf(str,"%s ",istr); else str[0]=0;
1443
1444     if (!name.isName(99)) wblog(F,L,
1445         "ERR invalid name for %soperator (%d,'%s')",
1446         str, nrgIdx.len, name.data
1447     );
1448
1449     kloc=-1;
1450
1451     for (i=0; i<CC.len; ++i) {
1452         for (l=j=0; j<CC[i].len; ++j) {
1453             const QSpace<TQ,TD> &x = CC[i][j]; if (x.isEmpty()) continue;
1454             r=-1; l=j; if (!q) q=x.QDIM;
1455
1456             if (!x.isConsistent(r) || r<2 || r>3) wblog(F,L,
1457                 "ERR invalid operator (rank-%d QSpace) !?\n"
1458                 "--> %s %s",r, shortFL(FL), str);
1459             if (x.QDIM!=q) wblog(FL,
1460                 "ERR invalid operator (QDIM=%d, %d) !?\n--> %s %s",
1461                 x.QDIM, q, shortFL(FL), str
1462             );
1463         } kloc=MAX(kloc,(int)l); }
1464
1465     if (kloc<0) wblog(FL,"ERR all empty operators !?");
1466     kloc=MAX(0,kloc-1);
1467
1468     KK.initDef(CC.len);
1469     CI.initDef(CC.len);
1470     for (i=0; i<CC.len; ++i) {
1471         if (!CC[i].len) wblog(FL,"ERR empty operator set !?");
1472         CI[i].initDef(CC[i]);
1473         KK[i]=CC[i][0];
1474     }
1475
1476     for (i=0; i<CC.len; ++i) { if (nrgIdx[i].idx>=0) continue;
1477         for (j=i+1; j<CC.len; ++j)
1478             if (CC[i].isEqual(CC[j])) { ++nrefs;
1479                 nrgIdx[j].init(i,this);
1480             }
1481     }
1482
1483     if (CR && CR->CI.len) { m=CR->CI.len;
1484         for (i=0; i<CC.len; ++i) {
1485             if (nrgIdx[i].idx>=0) continue;
1486
1487             for (j=0; j<m; ++j)
1488                 if (CC[i].isEqual(CR->CI[j])) {
1489                     nrgIdx[i].init(j,CR);
1490                     ++nrefs; break;
1491                 }
1492         }
1493
1494         sprintf(str,"%s%s",CR->name.data,"CI_");
1495         a=getMxInfo(str);
1496         if (a) { if (mxIsQSpaceVecVec(a,2)) {
1497             mxInitQSpaceVecVec(F,L,a,NRG); m=NRG.len;
1498             for (i=0; i<CC.len; ++i) {
1499                 if (nrgIdx[i].idx>=0) continue;
1500                 for (j=0; j<m; ++j) if (CC[i]==NRG[j]) {
1501                     nrgIdx[i].init(j,CR->name.data);
1502                     ++nrefs; break;
1503                 }
1504             } else wblog(FL,
1505                 "WRN %s::CI_ not a QSpace cell vector !?", CR->name.data);
1506         }
1507
1508         if (nrefs==nrgIdx.len) { return nrefs; }
1509     }
1510
1511     e=0;
1512
1513     if (tflag && NRG_N>2) {
1514         iter=NRG_N-2;
1515         for (i=0; i<ntag; ++i) {
1516             sprintf(str,"%s%s",name.data,tag[i]);
1517             if (!isQSpaceVec(str,iter)) { ++e; break; }
1518         }
1519     }
1520
1521     iter=0;
1522     sprintf(str,"%s%s",name.data,"KK");
1523     if (!isQSpaceVec(str,iter)) ++e;
1524
1525     sprintf(str,"%s%s",name.data,"CI_");

```

```

1526     a=getMxData(str,iter); if (!a || !mxIsQSpaceVec(FL,a)) ++e;
1527
1528     if (e) { calc=store=1;
1529         wblog(FL," * calculate all operators %s (%d)",
1530             name.data, nrgIdx.len); return nrefs;
1531     }
1532
1533     mxInitQSpaceVecVec(F,L,a,NRG); m=NRG.len;
1534
1535     for (i=0; i<CC.len; ++i) {
1536         if (nrgIdx[i].idx>=0 && nrgIdx[i].tag.isEmpty()) continue;
1537         for (j=0; j<m; ++j) if (CC[i].isEqual(NRG[j])) {
1538             nrgIdx[i].tag.init();
1539             nrgIdx[i].idx=j; ++nrefs;
1540             break;
1541         }
1542     }
1543
1544     calc = (nrefs==CC.len) ? 0 : 1;
1545     store = (nrefs==CC.len) ? 0 : 1;
1546
1547     return nrefs;
1548 };
1549
1550 template<class TQ, class TD>
1551 void NRGData<TQ,TD>::initSym(const char *tag) {
1552
1553     if (strlen(tag)==2) {
1554         const wvector< QSpace<TQ,TD> > &K=getOpsXX(tag);
1555         unsigned i,n=K.len;
1556         issym.init(n);
1557
1558         for (i=0; i<n; ++i) {
1559             if (K[i].isHConj()) issym[i]=+1; else
1560             if (K[i].isAHerm()) issym[i]=-1;
1561         }
1562     }
1563     else if (strlen(tag)==1) {
1564         const QSpace<TQ,TD> &K=getQSpace(tag);
1565         issym.init(1);
1566         if (K.isHConj()) issym[0]=+1; else
1567         if (K.isAHerm()) issym[0]=-1;
1568     }
1569     else wblog(FL,"ERR invalid tag '%s'", tag ? tag : "(null)");
1570 }
1571
1572 template<class TQ, class TD>
1573 void NRGData<TQ,TD>::check_dloc(
1574     const char *F, int L, const char *fun, unsigned dloc, unsigned iter) const {
1575
1576     widx_t d;
1577     K.getDim(2,&d);
1578
1579     if (iter+1>=NRG_N) {
1580         if (d) wblog(F,L,
1581             "ERR %s() got dloc=%li/%d at k=%d/%d !?",fun,d,dloc,iter,NRG_N);
1582         return;
1583     }
1584
1585     if (d!=dloc && (iter || !D.isEmpty())) {
1586         char s[64]; sprintf(s,64,
1587             "got inconsistent dloc=%li/%d (k=%d/%d)",d,dloc,iter,NRG_N);
1588
1589         if (D.isEmpty())
1590             wblog(F,L,"WRN %s() %s",fun,s);
1591         else wblog(F,L,"ERR %s() %s",fun,s);
1592     }
1593 };
1594
1595 template<class TQ, class TD>
1596 template<class TD_>
1597 int NRGData<TQ,TD>::checkOpConsistency(
1598     const char *F, int L, const NRGData<TQ,TD_> &B) const {
1599
1600     unsigned e=0, n = (KK.len ? KK.len : B.KK.len);
1601     if (!n) n = (DD.len ? DD.len : B.DD.len);
1602     if (!n) { return n; }
1603
1604     if (KK.len && B.KK.len) {
1605         if (KK.len!=B.KK.len) { ++e;
1606             if (L) wblog(F,L,"WRN ... KK.len=%d/%d",KK.len,B.KK.len); }
1607         if (KD.len!=B.KD.len || KD.len!=n) { ++e;
1608             if (L) wblog(F,L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
1609         if (DK.len!=B.DK.len || DK.len!=n) { ++e;
1610             if (L) wblog(F,L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
1611         if (DD.len!=B.DD.len || DD.len!=n) { ++e;
1612             if (L) wblog(F,L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }

```

```

1613     }
1614     else if (KK.len) {
1615         if (KD.len!=n) { ++e;
1616             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
1617         if (DK.len!=n) { ++e;
1618             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
1619         if (DD.len!=n) { ++e;
1620             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
1621     }
1622     else {
1623         if (B.KD.len!=n) { ++e;
1624             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
1625         if (B.DK.len!=n) { ++e;
1626             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
1627         if (B.DD.len!=n) { ++e;
1628             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
1629     }
1630     if (e) {
1631         if (F) wblog(FL,"ERR %s() got size consistency",FCT);
1632         return -e;
1633     }
1634     return n;
1635 };
1636
1637 template<class TQ, class TD>
1638 bool NRGData<TQ,TD>::isQSpaceVec(const char *nm, int iter) {
1639
1640     if (iter<0) iter=NRG_ITER;
1641     if (iter>=(int)NRG_N) wblog(FL,
1642         "ERR Index out of bounds (%d/%d)", iter, NRG_N);
1643
1644     const mxArray *a=getMxData(nm,iter,'i');
1645     if (a && mxIsQSpaceVecR23(FL,a)) return 1;
1646
1647     return 0;
1648 }
1649
1650
1651 template<class TQ, class TD>
1652 bool NRGData<TQ,TD>::checkVec(
1653     const char *F, int L,
1654     const char *XX, unsigned R, const char *istr
1655 ){
1656     unsigned l,n=128;
1657     char nm[n]; int fid;
1658
1659     char cflag=WbUtil<TD>::isComplex();
1660
1661     l=snprintf(nm,n,"%s%s", name.data, XX);
1662     if (l>=n) wblog(FL,"ERR strlen out of bounds (%d/%d)",l,n);
1663
1664     if (MX) {
1665         const size_t *s=mxGetDimensions(MX);
1666         const int n=mxGetNumberOfDimensions(MX);
1667
1668         fid=mxGetFieldNumber(MX, nm);
1669         if (fid<0) {
1670             if (istr) {
1671                 wblog(F,L,"WRN invalid %s", istr);
1672                 wblog(FL, "ERR field '%s' could not be found",nm);
1673             }
1674             return 1;
1675         }
1676
1677         if (n!=2 || (s[0]!=1 && s[1]!=1)) {
1678             if (istr) {
1679                 wblog(F,L,"WRN invalid %s (%s)",istr,nm);
1680                 wblog(FL, "ERR NRG data must be row structure vector (%d,%d)",
1681                     (int)s[0],(int)s[1]);
1682             }
1683             return 1;
1684         }
1685
1686         if (!NRG_N) NRG_N=unsigned(s[0]*s[1]); else
1687         if (NRG_N!=unsigned(s[0]*s[1])) {
1688             if (istr) {
1689                 wblog(F,L,"WRN invalid %s (%s)",istr,nm);
1690                 wblog(FL, "ERR size mismatch (%d/%d)", NRG_N, int(s[0]*s[1]));
1691             }
1692             return 1;
1693         }
1694     }
1695
1696     return !mxIsQSpaceVec(MX,fid,R,cflag);
1697 }
1698
1699 else {
1700     int i=mxIsQSpaceVec(F_L,NAME.data,nm,R,0,0,cflag,NRG_N);

```

```

1700         if (i<0) {
1701             wblog(F,L,"ERR invalid %s (%s)%N%N%s%N",istr? istr:"!?",nm,str);
1702             return 1;
1703         }
1704
1705         if (!NRG_N) NRG_N=i; else
1706         if (i!=(int)NRG_N) {
1707             wblog(F,L,"WRN invalid %s (%s)", istr?istr:"!?", nm);
1708             wblog(FL,"ERR size mismatch (%d,%d)", i, NRG_N);
1709             return 1;
1710         }
1711     }
1712
1713     return 0;
1714 };
1715
1716 template<class TQ, class TD>
1717 bool NRGData<TQ,TD>::checkVecVec(const char *XX, unsigned R) {
1718
1719     unsigned l,n=128;
1720     char nm[n]; int fid;
1721
1722     l=snprintf(nm,n,"%s%s",name.data,XX);
1723     if (l>n) wblog(FL,"ERR strlen out of bounds (%d/%d)",l,n);
1724
1725     if (MX) {
1726         fid=mxGetFieldNumber(MX,nm);
1727         if (fid<0) {
1728             sprintf(str, "%s:%d field '%s' could not be found.",
1729                 FL,nm); return 1;
1730         }
1731
1732         if (!NRG_N) {
1733             const size_t *s=mxGetDimensions(MX);
1734             const int n=mxGetNumberOfDimensions(MX);
1735             if (n!=2 || (s[0]!=1 && s[1]!=1)) { sprintf(str,
1736                 "%s:%d NRG data must be row structure vector (%d,%d)",
1737                 FL, (int)s[0], (int)s[1]); return 1;
1738             }
1739             NRG_N=unsigned(s[0]*s[1]);
1740         }
1741         return !mxIsQSpaceVEC(MX, fid, R);
1742     }
1743     else {
1744
1745         int i=mxIsQSpaceVEC(FL,NAME.data, nm, R);
1746         if (i<0) return 1;
1747         if (!NRG_N) NRG_N=i;
1748         if (i!=(int)NRG_N) {
1749             sprintf(str,"%s:%d size mismatch (%d,%d)",
1750                 FL, i, NRG_N); return 1;
1751         }
1752     }
1753
1754     return 0;
1755 }
1756
1757 template<class TQ, class TD>
1758 mxArray* NRGData<TQ,TD>::getMxData(const char *vs, int iter, char ionly) {
1759     if (iter<0) iter=NRG_ITER;
1760     if (iter>=(int)NRG_N) wblog(FL,
1761         "ERR %s() index out of bound (%d/%d)",FCT,iter,NRG_N);
1762
1763     if (MX) {
1764         int fid=mxGetFieldNumber(MX, vs);
1765         if (fid<0) return NULL;
1766         return mxGetFieldByNumber(MX,iter,fid);
1767     }
1768     else {
1769         Wb::matFile F(FL,getFileName(iter),"r");
1770         Wb::Clock_resume sw(&fdmTime_5);
1771
1772         if (aux) mxDestroyArray(aux);
1773         if (ionly)
1774             aux = matGetVariableInfo(F.mfp, vs);
1775         else aux = matGetVariable(F.mfp, vs);
1776
1777         F.close(); return aux;
1778     }
1779 };
1780
1781 template<class TQ, class TD>
1782 const mxArray* NRGData<TQ,TD>::getMxInfo(const char *v) {
1783
1784     if (!v || !v[0]) wblog(FL,
1785         "ERR %s() invalid usage (got %s)",FCT,v? "empty":"null");
1786

```

```

1787     if (aux) { mxDestroyArray(aux); aux=NULL; }
1788
1789     if (MX) {
1790         return mexGetVariablePtr("caller",v);
1791     }
1792     else {
1793         Wb::matFile F(FL, getFileNameI(), "r");
1794
1795         Wb::Clock_resume sw(&fdmTime_5);
1796         aux=matGetVariable(F.mfp, v);
1797
1798         return aux;
1799     }
1800 };
1801
1802 template<class TQ, class TD>
1803 mxArray* NRGData<TQ,TD>::toMx() const {
1804
1805     MXPut S(0,0);
1806     S.add(name,"name")
1807       .add(calc,"calc").add(store,"store")
1808       .add(kloc,"kloc").add(issym,"issym")
1809       .addP(MXPut(0,0)
1810         .add(CI,"CI").add(nrgIdx,"nrgIdx").add(NAME,"NAME")
1811         .add(Wb::num2Str(MX,"0x%lX"),"MX").add(locMX,"locMX")
1812         .add(NRG_ITER,"ITER")
1813         .add(NRG_N,"N").toMx(),"Idx"
1814       );
1815
1816     if (KK.len || KD.len || DK.len || DD.len) {
1817         S.add(KK,"KK").add(KD,"KD").add(DK,"DK").add(DD,"DD"); }
1818
1819     if (!K.isEmpty() || !D.isEmpty()) {
1820         S.add(K,"K").add(D,"D"); }
1821
1822     return S.toMx();
1823 };
1824
1825 template<class TQ, class TD>
1826 void NRGData<TQ,TD>::info(const char *vstr) const {
1827
1828     unsigned i,nl=0;
1829     char sx[nrgIdx.len+1], s_[128];
1830
1831     for (i=0; i<nrgIdx.len; ++i) {
1832         if ( nrgIdx[i].idx<0) sx[i]='C';
1833         if (!nrgIdx[i].nrg) sx[i]='R';
1834         else if ( nrgIdx[i].nrg==this)
1835             sx[i]='1';
1836         else sx[i]='2';
1837     }; sx[i]=0;
1838
1839     snprintf(s_,128,"%d/%d; calc=%d (%s; %d); store=%d",
1840     NRG_ITER+1, NRG_N, calc, sx, nrgIdx.len, store);
1841
1842     if (MX) {
1843         printf("\n%s Structure '%s' (0x%lX), %s\n",
1844         vstr && vstr[0] ? vstr : (name.data ? name.data : "ans"),
1845         NAME.data ? NAME.data : "",MX,s_);
1846     }
1847     else {
1848         printf("\n%s File structure '%s', %s\n",
1849         vstr && vstr[0] ? vstr : (name.data ? name.data : "ans"),
1850         NAME.data ? NAME.data : "",s_);
1851     }
1852
1853     if (!KK.isEmpty()) {
1854         if (!nl) { printf("\n"); ++nl; }
1855         for (i=0; i<KK.len; ++i) {
1856             sprintf(s_,"KK[%d]", i);
1857             KK[i].info(s_);
1858         }
1859     }
1860
1861     if (!KD.isEmpty()) {
1862         if (!nl) { printf("\n"); ++nl; }
1863         for (i=0; i<KD.len; ++i) {
1864             sprintf(s_,"KD[%d]", i);
1865             KD[i].info(s_);
1866         }
1867     }
1868     if (!DK.isEmpty()) {
1869         if (!nl) { printf("\n"); ++nl; }
1870         for (i=0; i<DK.len; ++i) {
1871             sprintf(s_,"DK[%d]", i);
1872             DK[i].info(s_);
1873         }
1874     }

```

```

1874     }
1875     if (!DD.isEmpty()) {
1876         if (!nl) { printf("\n"); ++nl; }
1877         for (i=0; i<DD.len; ++i) {
1878             sprintf(s_, "DD[%d]", i);
1879             DD[i].info(s_);
1880         }
1881     }
1882
1883     nl=0;
1884
1885     if (!K.isEmpty()) { if (!nl) { printf("\n"); ++nl; }; K.info("K"); }
1886     if (!D.isEmpty()) { if (!nl) { printf("\n"); ++nl; }; D.info("D"); }
1887
1888     if (nl) printf("\n");
1889 };
1890
1891 template <class TQ, class TD>
1892 void NRGData<TQ,TD>::dispIdx(const char *F, int L) const {
1893
1894     char s[32];
1895     wblog(F,L,"%NNRGIndex %s (calc=%d, store=%d)%N", name.data, calc, store);
1896
1897     for (unsigned i=0; i<nrgIdx.len; ++i) {
1898         if (!nrgIdx[i].tag.isEmpty())
1899             strncpy(s, nrgIdx[i].tag.data, 32);
1900         else if (nrgIdx[i].nrg)
1901             strcpy(s, nrgIdx[i].nrg==this ? "(this)" : "(that)");
1902         else if (nrgIdx[i].idx>=0)
1903             strcpy(s, "(file)");
1904         else strcpy(s, "(calc)");
1905
1906         printf("%6d: %4d  %s:%lX\n", i, nrgIdx[i].idx, s, nrgIdx[i].nrg);
1907     }
1908
1909     if (nrgIdx.len) printf("\n");
1910 }
1911
1912 template <class TQ, class TD>
1913 void NRGData<TQ,TD>::info(const char *F, int L, const char *istr) const {
1914
1915     unsigned i, nt1=0, nt2=0, nx1=0, nx2=0, nc=0;
1916     wstring mark(nrgIdx.len+1);
1917     wstring S(128);
1918
1919     for (i=0; i<nrgIdx.len; ++i) {
1920
1921         if (nrgIdx[i].nrg && nrgIdx[i].nrg!=this) { ++nx1; mark[i]='x'; } else
1922         if (!nrgIdx[i].tag.isEmpty()) { ++nx2; mark[i]='X'; } else
1923         if (nrgIdx[i].nrg==this) { ++nt1; mark[i]='r'; } else
1924         if (nrgIdx[i].idx>=0) { ++nt2; mark[i]='R'; }
1925         else { ++nc; mark[i]='*'; }
1926     }
1927
1928     if (nc) {
1929         S.pushf(FL, " %s%d/%d op%s",
1930             calc ? "calc" : "", nc, KK.len, KK.len!=1 ? "s":""");
1931     }
1932     else {
1933         S.pushf(FL, " %d op%s%s", KK.len, KK.len!=1 ? "s":""",
1934             calc ? " , calc!?" : "", all from store or referenced");
1935     }
1936
1937     if (store) S.push(FL, " and store");
1938     else if (nc || calc) {
1939         S.push(FL, " (not stored)");
1940     }
1941
1942     if (kloc          ) S.pushf(FL, " , nloc=%d", kloc+1);
1943
1944     if (strcmp(istr,name.data)) {
1945         wblog(F,L, " * %s (%3s): [%s] %s",
1946             name.data, istr, mark.data, S.data[0] ? S.data+2 : "");
1947     }
1948     else {
1949         wblog(F,L, " * %s [%s] %s",
1950             name.data, mark.data, S.data[0] ? S.data+2 : "");
1951     }
1952 };
1953
1954 template <class TQ, class TD>
1955 void NRGData<TQ,TD>::disp_otype(const char *F, int L) const {
1956
1957     wstring s(128); unsigned i=0;
1958
1959     if (name.data) s.cpy(FL,name.data);
1960

```

```

1961     if (KK.len) {
1962         s.pushf(FL,"%sKK: ",i?"; ":"");
1963         for (i=0; i<KK.len; ++i) s.pushf(FL,"%d",KK[i].otype);
1964     }
1965     if (DK.len) {
1966         s.pushf(FL,"%sTK: ",i?"; ":"");
1967         for (i=0; i<DK.len; ++i) s.pushf(FL,"%d",DK[i].otype);
1968     }
1969     if (KD.len) {
1970         s.pushf(FL,"%sKT: ",i?"; ":"");
1971         for (i=0; i<KD.len; ++i) s.pushf(FL,"%d",KD[i].otype);
1972     }
1973     if (DD.len) {
1974         s.pushf(FL,"%sTT: ",i?"; ":"");
1975         for (i=0; i<DD.len; ++i) s.pushf(FL,"%d",DD[i].otype);
1976     }
1977
1978     wblog(F_L," * NRGData of type %s() %s ",FCT,s);
1979 };
1980
1981 template <class TQ, class TD, class TZ>
1982 void backPropagateOps(const char *F, int L,
1983     wbvector< QSpace<TQ,TZ> > &BK,
1984     const wbvector< QSpace<TQ,TZ> > &BX,
1985     const QSpace<TQ,TD> &A0,
1986     const QSpace<TQ,TD> &A2
1987 ){
1988     if (!BX.len) return;
1989     if (!BK.len) BK.init(BX.len); else
1990     if (BK.len!=BX.len) wblog(F_L,
1991         "ERR %s() length mismatch (%d/%d)",FCT,BK.len,BX.len);
1992
1993     unsigned i=0,r;
1994     QSpace<TQ,TZ> Q1,Q2;
1995
1996     for (; i<BX.len; ++i) { r=BK[i].rank(F_L);
1997         if (!r && !i) {
1998             for (; i<BX.len; ++i) { if (BX[i].rank(F_L)) break; }
1999             if (i==BX.len) return;
2000         }
2001         if (r<2 || r>3) wblog(F_L,
2002             "ERR %s() invalid rank r=%d (%d/%d) !?",FCT,r,i+1,BX.len);
2003
2004         A2.contract(F_L,"2*",BX[i],2,Q1);
2005         A0.contract(F_L,"23",Q1,"32",Q2);
2006         BK[i]+=Q2;
2007     }
2008 };
2009
2010 template <class TQ, class TZ>
2011 template <class TD>
2012 void NRGData<TQ,TZ>::backPropagateOp(const char *F, int L,
2013     const NRGData<TQ,TD> &A0,
2014     const NRGData<TQ,TD> &A2
2015 ){
2016     unsigned i=0, e=0;
2017     wbvector< QSpace<TQ,TZ> > BK;
2018
2019     A2.check_dloc(F_LF,gRX.dloc,gRX.iter);
2020
2021     backPropagateOps(F,L,BK, KK, A0.K, A2.K);
2022     backPropagateOps(F,L,BK, KD, A0.K, A2.D);
2023     backPropagateOps(F,L,BK, DK, A0.D, A2.K);
2024     backPropagateOps(F,L,BK, DD, A0.D, A2.D);
2025
2026     for (; i<BK.len; ++i) { if (BK[i].isEmpty()) { ++e; }
2027         BK[i]*=(1./gRX.dloc);
2028         BK[i].SkipZeroData(DEPS2,'b');
2029     }
2030     if (e && gRX.iter>kloc) {
2031         MXPut(FL,"Ie").add(*this,"B02").add(gRX.iter,"iter").add(BK,"BK")
2032         .add(A0,"A0").add(A2,"A2");
2033         wblog(FL,"ERR %s() got %d/%d empty BK at iter=%d/%d !?",
2034             FCT,e,BK.len,gRX.iter,kloc);
2035     }
2036
2037     BK.save2(KK);
2038 };
2039
2040 template <class TQ, class TD> inline
2041 void updateOp_L(const char *F, int L,
2042     const QSpace<TQ,TD> &Xin,
2043     const QSpace<TQ,TD> &A1,
2044     const QSpace<TQ,TD> &A2,
2045     QSpace<TQ,TD> &Xout
2046 ){
2047     QSpace<TQ,TD> Xk; unsigned r=-1;

```



```

2048
2049     if (Xin.isEmpty()) { wblog(F_L,
2050         "ERR %s() got empty L-operator",FCT); return; }
2051     if (Xin.isOperator(&r)<=0) { Xin.info("L-op");
2052         wblog(F_L,"ERR %s() got invalid L-operator (%d)",FCT,r);
2053     }
2054
2055     Xin.contract(F,L,2,A2,1,Xk);
2056     A1.contract(F,L,"1,3;*",Xk, (r!=3 ? "1,3":"1,4"), Xout);
2057
2058     if (r==3) Xout.Permute("1,3,2");
2059
2060     Xout.otYPE=Xin.otYPE;
2061     Xout.SkipZeroData();
2062 };
2063
2064 template <class TQ, class TD> inline
2065 void updateOp_s(const char *F, int L,
2066     const QSpace<TQ,TD> &x,
2067     const QSpace<TQ,TD> &A1,
2068     const QSpace<TQ,TD> &A2,
2069     QSpace<TQ,TD> &Xout
2070 ){
2071     QSpace<TQ,TD> Xk; unsigned r=-1;
2072
2073     if (x.isEmpty()) { wblog(FL,
2074         "WRN %s() got empty L-operator",FCT); return; }
2075     if (x.isOperator(&r)<=0) { x.info("local-op");
2076         wblog(FL,"ERR %s() got invalid L-operator (%d)",FCT,r);
2077     }
2078
2079     A2.contract(F,L,3,x,2, Xk);
2080     A1.contract(F,L,"1,3;*", Xk,"1,3", Xout);
2081
2082     Xout.otYPE=x.otYPE;
2083     Xout.SkipZeroData();
2084 };
2085
2086 template <class TQ, class TD>
2087 inline void updateOp_Ls(
2088     const QSpace<TQ,TD> &Xin,
2089     const QSpace<TQ,TD> &A1,
2090     const QSpace<TQ,TD> &A2,
2091     QSpace<TQ,TD> &Xout
2092 ){
2093     QSpace<TQ,TD> Xk; unsigned r=-1;
2094
2095     if (Xin.isEmpty()) { wblog(FL,
2096         "WRN %s() got empty L-operator",FCT); return; }
2097     if (Xin.isOperator(&r,'x')<=0 || r!=4) { Xin.info("Ls-op");
2098         wblog(FL,"ERR %s() got invalid Ls-operator (%d)",FCT,r);
2099     }
2100
2101     Xin.contract("3,4",A2,"1,3", Xk);
2102     A1.contract("1,3;*",Xk,"1,2",Xout);
2103
2104     Xout.otYPE=Xin.otYPE;
2105     Xout.SkipZeroData();
2106 };
2107
2108 template <class TQ, class TD>
2109 inline void updateOp_sL(
2110     const QSpace<TQ,TD> &Xin,
2111     const QSpace<TQ,TD> &A1,
2112     const QSpace<TQ,TD> &A2,
2113     QSpace<TQ,TD> &Xout
2114 ){
2115     QSpace<TQ,TD> Xk; unsigned r=-1;
2116
2117     if (Xin.isEmpty()) { wblog(FL,
2118         "WRN %s() got empty L-operator",FCT); return; }
2119     if (Xin.isOperator(&r)<=0 || r!=4) { Xin.info("sL-op");
2120         wblog(FL,"ERR %s() got invalid sL-operator (%d)",FCT,r);
2121     }
2122
2123     Xin.contract("4,3",A2,"1,3", Xk);
2124     A1.contract("1,3;*",Xk,"2,1",Xout);
2125
2126     Xout.otYPE=Xin.otYPE;
2127     Xout.SkipZeroData();
2128 };
2129
2130 template <class TQ, class TD> inline
2131 void updateOp_loc(const char *F, int L,
2132     const NRGData<TQ,TD> &NRG, unsigned i,
2133     const QSpace<TQ,TD> &XL,
2134     const QSpace<TQ,TD> &x,

```

```

2135     const QSpace<TQ,TD> &A1,
2136     const QSpace<TQ,TD> &A2,
2137     QSpace<TQ,TD> &Xout,
2138     unsigned iter
2139 ){
2140     QSpace<TQ,TD> Ax,XA;
2141     unsigned r1=-1, r2=-1;
2142     bool o1=0, o2=0;
2143
2144     if (!x.isEmpty()) {
2145         if (x.isOperator(&r2)<=0) { x.info("local_op"); wblog(FL,
2146             "ERR %s() got invalid local operator (%d)",FCT,r2); }
2147         if (XL.otype!=x.otype && !XL.isEmpty()) wblog(FL,
2148             "WRN %s() got different operator types (%s; %s)",
2149             FCT, QS_STR[XL.otype],QS_STR[x.otype]);
2150         o2=(int(r2)==3);
2151
2152         wblog(FL, " * contracting %s[%d] onto A[%d].%s",
2153             NRG.name.data,i, iter, A2.itags.toStrk(str,8,2,"s"));
2154
2155         A2.contract(F,L,3,x,"-op",Ax);
2156     }
2157     else Ax.init2ref(A2);
2158
2159     if (iter || !XL.isEmpty()) {
2160         if (XL.isOperator(&r1)<=0) { x.info("L-op"); wblog(FL,
2161             "ERR %s() got invalid L-operator (%d)",FCT,r1); }
2162         o1=(int(r1)==3);
2163
2164         if (iter<1) wblog(FL,"TST contracting onto A[%d].%s",
2165             iter, A2.itags.toStrk(str,8,0,"L"));
2166
2167         if (o1 && o2) { o1=o2=0;
2168             XL.contract(F,L,"23",Ax,"14",XA); }
2169         else {
2170             XL.contract(F,L,"-op",Ax,1,XA);
2171         }
2172     }
2173     else XA.init2ref(Ax);
2174
2175     if (o1)
2176         A1.contract(F,L,"13*",XA,"14", Xout,"132");
2177     else A1.contract(F,L,"13*",XA,"13", Xout);
2178
2179     if (o1) Xout.otype=XL.otype; else
2180     if (o2) Xout.otype=x .otype;
2181
2182     Xout.SkipZeroData();
2183 };
2184
2185 template <class TQ, class TD>
2186 int Wb::updateOp(const char *F, int L,
2187     const NRGData<TQ,TD> &NRG,
2188     wbvector< QSpace<TQ,TD> > &F12,
2189     const QSpace<TQ,TD> &A1,
2190     const QSpace<TQ,TD> &A2,
2191     const wbvector< QSpace<TQ,TD> > &FKK,
2192     unsigned iter
2193 ){
2194     unsigned i, j=0, m=0, r=-1, rx=-1, nop=FKK.len;
2195     int e=0, i1=0;
2196
2197     const wbvector< wbvector<QSpace<TQ,TD> > > &CI=NRG.CI;
2198     const wbvector< NRGIndex<TQ,TD> > &nrgIdx=NRG.nrgIdx;
2199
2200     if (A1.isEmpty() || A2.isEmpty()) { F12.initDef(nop); return 0; }
2201
2202     if (nrgIdx.len!=nop) wblog(FL,
2203         "ERR %s() severe size mismatch (nrgIdx: %d,%d)",FCT,nrgIdx.len,nop);
2204
2205     if (iter>0) {
2206         for (i=0; i<nop; ++i) { rx=r;
2207             if (nrgIdx[i].idx>=0) continue;
2208             if (! (i1=FKK[i].isConsistent(FL))
2209                 || int(r=FKK[i].isOperator(&rx))<=0) break;
2210         }
2211     }
2212     else if (CI.isEmpty()) {
2213         for (i=0; i<nop; ++i) { rx=r;
2214             if (! (i1=FKK[i].isConsistent(FL)) ||
2215                 int(r=FKK[i].isOperator(&rx,'x'))<=0
2216             ){ wblog(FL,
2217                 "WRN FKK[%d] -> (%d; %d/%d/%d) ",i+1,i1,r,rx,FKK[i].rank());
2218                 break;
2219             }
2220         }
2221     }

```

```

2222     else {
2223         if (nop!=CI.len) wblog(F,L,
2224             "ERR size inconsistency (%d/%d)",nop,CI.len);
2225
2226         for (i=0; i<nop; ++i) { m=CI[i].len;
2227             for (j=0; j<m; ++j) { if (!CI[i][j].isEmpty()) { rx=r;
2228                 if (!(i1=CI[i][j].isConsistent(F,L)) ||
2229                     int(r=CI[i][j].isOperator(&rx)<=0) break;
2230             }}}
2231     }
2232
2233     if (i<nop || j<m) wblog(F,L,
2234         "ERR invalid rank-%d operator (iter=%d, op=%d/%d)",r,iter,i+1,nop);
2235     if (A1.rank(F,L)!=3 || A2.rank(F,L)!=3 || A1.QDIM!=A2.QDIM) wblog(F,L,
2236         "ERR %s() severe rank inconsistency (%s; %s)",
2237         FCT, A1.sizeStrQ().data, A2.sizeStrQ().data);
2238
2239     if (F12.len!=nop)
2240         F12.initDef(nop);
2241
2242     if ((int)iter>NRG.kloc) {
2243         for (i=0; i<nop; ++i) {
2244             if (nrgIdx[i].idx>=0) continue;
2245             updateOp_L(F,L, FKK[i], A1, A2, F12[i] );
2246             wbtot().runningLarge(F,L);
2247
2248             if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
2249         }
2250     }
2251     else if (!CI.isEmpty()) {
2252         QSpace<TQ,TD> E;
2253
2254         for (i=0; i<nop; ++i) { if (nrgIdx[i].idx>=0) continue;
2255             try { updateOp_loc(F,L,NRG,i,
2256                 FKK[i], iter+2>CI[i].len ? E : CI[i][iter+1],
2257                 A1, A2, F12[i],iter);
2258             }
2259             catch (...) { wblog_ERR_updateOp(F,L,"loc",i,nop,iter); }
2260
2261             if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
2262         }
2263     }
2264     else {
2265         char Lflag = A1.getDim(1)>1 || A2.getDim(1)>1;
2266         unsigned l=0, tlen=16; char tag[tlen];
2267
2268         wbsting mark(nop,'*');
2269
2270         static time_t last_call_iter0=0;
2271         time_t tnow=time(0);
2272
2273         for (i=0; i<nop; ++i) {
2274             if (nrgIdx[i].idx>=0) continue;
2275             r=FKK[i].rank(F,L);
2276
2277             if (r==2 || (r==3 && FKK[i].otype==QS_OPERATOR)) {
2278                 if (Lflag &&
2279                     FKK[i].hasQOverlap(1,A1,1,'<')>0 &&
2280                     FKK[i].hasQOverlap(2,A1,1,'<')>0
2281                 ){
2282                     try { updateOp_L(F,L,FKK[i],A1,A2, F12[i]); }
2283                     catch (...) { wblog_ERR_updateOp(F,L,"L",i,nop,iter); }
2284                     mark[i]='L';
2285                 }
2286                 else
2287                 if (FKK[i].hasQOverlap(1,A1,3,'<')>0 &&
2288                     FKK[i].hasQOverlap(2,A1,3,'<')>0
2289                 ){
2290                     try { updateOp_s(F,L,FKK[i], A1, A2, F12[i]); }
2291                     catch (...) { wblog_ERR_updateOp(F,L,"s",i,nop,iter); }
2292                     mark[i]='s';
2293                 }
2294                 else
2295                 if (FKK[i].hasQOverlap(1,A1,2,'<')>0 &&
2296                     FKK[i].hasQOverlap(2,A1,2,'<')>0
2297                 ){
2298                     mark[i]='R';
2299                 }
2300                 else wblog(F,L,
2301                     "ERR %s() failed to deal input operator with A%d",FCT,iter);
2302             }
2303         }
2304         else if (r==4) {
2305             wvector<int> m1,m2;
2306             int mm[] = {
2307                 FKK[i].hasQOverlap(1,A1,1,'<')>0,
2308                 FKK[i].hasQOverlap(2,A2,3,'<')>0,

```

```

2309         FKK[i].hasQOverlap(3,A2,1,'<')>0,
2310         FKK[i].hasQOverlap(4,A2,3,'<')>0,
2311
2312         FKK[i].hasQOverlap(1,A1,3,'<')>0,
2313         FKK[i].hasQOverlap(2,A2,1,'<')>0,
2314         FKK[i].hasQOverlap(3,A2,3,'<')>0,
2315         FKK[i].hasQOverlap(4,A2,1,'<')>0
2316     };
2317     m1.init(4,mm); m2.init(4,mm+4);
2318
2319     if (m1.allUnequal(0)) {
2320         try { updateOp_Ls(FKK[i], A1, A2, F12[i]); }
2321         catch (...) { wblog_ERR_updateOp(FL,"Ls",i,nop,iter); }
2322         mark[i]='D';
2323     }
2324     else
2325     if (m2.allUnequal(0)) {
2326         try { updateOp_sL(FKK[i], A1, A2, F12[i]); }
2327         catch (...) { wblog_ERR_updateOp(FL,"sL",i,nop,iter); }
2328         mark[i]='d';
2329     }
2330     else {
2331         if (STRICT_ITER0) { mark[i]='?';
2332             if (mark.count('?')==1 && last_call_iter0+5<tnow){
2333                 wblog(FL,
2334                     "WRN %NFailed to fully associate rank-4 tensor product\n"
2335                     "operator with first site. [ %s; %s ]\n"
2336                     "A1: [ %s ]; A2: [ %s ]%N%N"
2337                     "NB! Hint: A0 may not be complete due to truncation "
2338                     "in NRGWilsonQS already at iter=0 !?%N",
2339                     m1.toStr().data, m2.toStr().data,
2340                     A1.getDim().toStrf("%d", "x").data,
2341                     A2.getDim().toStrf("%d", "x").data);
2342             }}
2343
2344         try { updateOp_Ls(FKK[i], A1, A2, F12[i]); }
2345         catch (...) { wblog_ERR_updateOp(FL,"Ls",i,nop,iter); }
2346     }
2347 }
2348 else wblog(FL,
2349 "ERR %s() invalid rank-%d operator (iter=%d)",FCT,r,iter);
2350
2351 if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
2352 }
2353
2354 l=snprintf(tag,tlen,"FDM %2d/%d",iter,NRG_N); {
2355     if (NRG.name.data && NRG.name.data[0] && l<tlen)
2356         l+=snprintf(tag+l,tlen-l," %s:",NRG.name.data);
2357     if (l>=tlen) wblog(FL,
2358         "ERR %s() string out of bounds (%d/%d)",FCT,l,tlen);
2359 }
2360
2361 if (!nop)
2362     wblog(FL,"%s got empty spectral ops",tag);
2363 else {
2364     unsigned il=0, m=mark.count('*');
2365     for (i=0; i<nop; ++i) { if (mark[i]!='*') { il=i; break; } }
2366     for (; i<nop; ++i) { if (mark[i]!='*') {
2367         if (mark[i]!=mark[il]) break;
2368     } }
2369
2370     if (m==nop) { wblog(FL,
2371         "%s no spectral ops to contract (all referenced)",
2372         tag,mark.data);
2373     }
2374     else if (i==nop && !m) {
2375         char ostr[32];
2376         if (nop>1)
2377             sprintf(ostr,"all %d spectral ops",nop);
2378         else sprintf(ostr,"spectral operator");
2379
2380         if (mark[0]!='R') wblog(FL,
2381             "%s contracting %s onto %c",tag,ostr,mark[0]);
2382         else wblog(FL,
2383             "%s %s already assumed in %c",tag,ostr,mark[0]
2384         );
2385     }
2386     else if (i==nop && m) {
2387         if (mark[il]!='R') wblog(FL,"%s contracting %d/%d "
2388             "spectral ops onto [%s]",tag,nop-m,nop,mark.data);
2389         else wblog(FL,"%s all %d/%d spectral "
2390             "ops already assumed in %s",tag,nop-m,nop,mark.data);
2391     }
2392     else {
2393         wblog(FL,"%s contracting "
2394             "%d/%d spectral ops onto [%s]",tag,nop-m,nop,mark.data);
2395     }

```

```

2396     }
2397
2398     last_call_iter0=tnow;
2399 }
2400
2401 if (iter==0) {
2402     for (i=0; i<F12.len; ++i) {
2403         if (F12[i].itags.len>2 && !F12[i].itags[2]) {
2404             F12[i].itags[2].init("op");
2405         }
2406     }
2407 }
2408
2409 wbttop().runningLarge(FL);
2410 return e;
2411 };
2412
2413 template <class TQ, class TD>
2414 unsigned applyZ0(const QSpace<TQ,TD> &Z0, QSpace<TQ,TD> &CK){
2415     unsigned r=CK.rank(FL);
2416     const QS_TYPES otype=CK.otype;
2417
2418     if (r==2 || r==3) {
2419         if (r==3 && CK.otype!=QS_OPERATOR) wblog(FL,
2420             "ERR %s() expecting otype==%s for rank-%d op (%s)",
2421             FCT,QS_STR[QS_OPERATOR],r,QS_STR[CK.otype]);
2422         if (Z0.rank(FL)!=2) wblog(FL,
2423             "ERR %s() Z0 of rank-2 required for rank-%d op (%d)",
2424             FCT,r,Z0.rank(FL));
2425     };
2426
2427     if (Z0.hasQOverlap(2, CK,1,'>')<=0) {
2428         wblog(FL,"ERR %s() Z0 does not match operator space",FCT);
2429         return 1;
2430     }
2431     Z0.contract(2,CK,1,CK);
2432     CK.otype=otype;
2433 }
2434
2435 else if (r==4) {
2436     if (Z0.rank(FL)!=4) wblog(FL,
2437         "ERR tensor Z0 required for tensor operator (rank=%d/%d)",
2438         Z0.rank(FL), r);
2439 };
2440
2441 if (Z0.hasQOverlap(3, CK,1,'>')<=0 ||
2442     Z0.hasQOverlap(4, CK,2,'>')<=0) {
2443     wblog(FL,"ERR %s() Z0 does not match operator space",FCT);
2444     return 2;
2445 }
2446 Z0.contract("3 4",CK,"1 2",CK);
2447 CK.otype=otype;
2448 }
2449 else wblog(FL,"ERR %s() invalid rank=%d.",FCT,r);
2450
2451 return 0;
2452 };
2453
2454 template <class TQ, class TD>
2455 unsigned getDLoc(
2456     NRGData<TQ,TD> &A, WbSigHandler *sig=NULL, unsigned *Dk=NULL,
2457     char fflag=0
2458 ){
2459     unsigned iter, N=NRG_N; widx_t d0=0, d=0;
2460     char cgflag=-1;
2461
2462     if (Dk) { (*Dk)=0; }; printf("\r");
2463
2464     for (iter=0; iter<N; ++iter) { if (sig) sig->check911();
2465         printf(" %s: %2d/%d loading AK ... \r",FCT,iter,N);
2466         fflush(0);
2467
2468         A.init(FL,"K",iter);
2469
2470         if (iter==0) cgflag=A.K.gotCGS(FL);
2471
2472         if (A.K.isEmpty()) {
2473             if (iter==N-1) continue; else wblog(FL,
2474                 "ERR %s() empty AK at intermediate iteration %d/%d",FCT,iter,N);
2475         }
2476         else {
2477             A.K.isConsistentR(3,FL);
2478             if (cgflag<=1 && !A.K.QIDX.isUnique()) wblog(FL,
2479                 "ERR invalid AK at iteration %d/%d",iter,N);
2480         }
2481
2482         #ifndef WB_SKIP_ASSERT

```

```

2483         if (fflag) { QSpace<TQ,TD> E;
2484         A.K.contract("1,3;*",A.K,"1,3",E);
2485         if (!E.isIdentityMatrix(1E-12)) {
2486             MXPut (FL).add(A,"A").add(E,"E");
2487             wblog(FL,"ERR NRG[%d].AK not in LRs index order",iter,N);
2488         }
2489     }
2490 #endif
2491
2492     A.K.getDim(2,&d);
2493
2494     if (iter>2 || d0) {
2495         if (d!=d0) {
2496             MXPut (FL,"a").add(A.K,"K").add(d,"d").add(d0,"d0")
2497                 .add(iter,"iter").add(N,"N");
2498             wblog(FL,"ERR %s() inconsistent local dimension "
2499                 "(d=%d/%d @ k=%d/%d)",FCT,d,d0,iter,N
2500                 );
2501         }
2502     }
2503     else if (iter==2 || iter+2==N) { d0=d; }
2504     else {
2505         A.init(FL,"D",iter);
2506         if (!A.D.isEmpty()) { d0=d; }
2507     }
2508
2509     if (Dk) {
2510         d=A.K.getDim(1);
2511         if (*Dk<d) { *Dk=d; continue; }
2512     }
2513     if (!fflag && iter>2) break;
2514 }
2515
2516 if (!d0) wblog(FL,"ERR %s() "
2517     "failed to get local dimension (d=%d; %d/%d)",FCT,d0,iter,N);
2518 return d0;
2519 };
2520
2521 #endif
2522

```

5.11 QSpace.hh

```

1 #ifndef __WB_QSPACE_COL_MAJOR_HH__
2 #define __WB_QSPACE_COL_MAJOR_HH__
3
4 /* ----- //
5 QSpace - set of objects (possibly complex matrices)
6 organized by generalized index set storeed as row vectors (recs)
7 in index matrix. // Wb,Aug08,05 Apr07,06
8
9 Introduction of non-abelian QSpace through variable 'qtype'.
10 see also clebsch.hh // Wb,Sep25,09
11
12 NB! QSpace currently only implemented on linux system
13 (matlab+gcc)
14
15 Trying to get it run on a macbook // Wb,Jan18,19
16 Problems encountered:
17 * OSX, by default, has case-*IN*sensitive files system (APFS)
18 encountered troubel with wbcmat.hh + wbCMat.hh
19 * default "gcc" compiler actually is with Xcode
20 which is much more picky concerning header files!
21
22 // ----- */
23
24 #ifdef WB_CLOCK
25 #define WBC_QSPACE_IO
26 #endif
27
28 enum QS_TYPES {
29     QS_NONE, QS_OPERATOR, QS_AMATRIX,
30     QS_NUM_TYPES
31 };
32
33 const char* QS_STR[QS_NUM_TYPES] = {
34     "", "operator", "A-matrix",
35 };
36
37 template <class TQ, class TD>
38 class QSpace {
39
40 public:

```

```

41
42 QSpace(unsigned n=0)
43 : QDIM(n), otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.) {};
44
45 QSpace(unsigned m, unsigned n, unsigned nq)
46 : QSpace(nq) { init(m,n,nq); };
47
48 QSpace(const QSpace &B)
49 : QIDX(B.QIDX), CGR(B.CGR), qtype(B.qtype), QDIM(B.QDIM),
50 otype(QS_NONE), itags(B.itags), mt(Wb::MEM_DEF),
51 isref(0), ctime(B.ctime)
52 {
53     if (this!=&B) {
54         unsigned i=0; setupDATA();
55         for (; i<B.DATA.len; ++i) { DATA[i]->init(*(B.DATA[i])); }
56     }
57 };
58
59 template<class TQ_, class TD_>
60 QSpace(const QSpace<TQ_,TD_> &B,
61 char xflag = WbUtil<TD_>::isComplex() ? 1 : 0
62 ): QIDX(B.QIDX), CGR(B.CGR), qtype(B.qtype), QDIM(B.QDIM),
63 otype(QS_NONE), itags(B.itags), mt(Wb::MEM_DEF),
64 isref(0), ctime(B.ctime)
65 {
66     if (!xflag) wblog(FL,"ERR %s() verify type cast %s -> %s",
67 FCT,TSTR(TD_),TSTR(TD_));
68     setupDATA();
69     for (unsigned i=0; i<B.DATA.len; ++i) {
70         DATA[i]->initT(*(B.DATA[i]));
71     }
72 };
73
74 QSpace(const char *F, int L, const mxArray *a, char ref=0)
75 : otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.)
76 { init(F,L,a,ref); };
77
78 QSpace(const mxArray *a, char ref=0)
79 : otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.)
80 { init(FL,a,ref); };
81
82 ~QSpace() { clearQSpace(); };
83
84 QSpace& clearQSpace() {
85     if (QIDX.data || DATA.data) { mt=Wb::MEM_DEF; }
86     QIDX.init(); qtype.init(); itags.init();
87     clearDATA(); CGR.init(); isref=0; ctime=0.;
88     return *this;
89 };
90
91 void clearDATA() {
92     if (DATA.len) {
93         if (!isref && !DATA.isref) {
94             for (unsigned i=0; i<DATA.len; ++i) WB_DELETE_1(DATA[i]);
95         }
96         DATA.init();
97     }
98 };
99
100 unsigned rank(const char *F=NULL, int L=0) const {
101     if (!QDIM) {
102         if (QIDX.dim2) wblog(F_L,"ERR QSpace inconsistency "
103 " (QIDX: %dx%d/%d)",QIDX.dim1, QIDX.dim2, QDIM);
104         return 0;
105     }
106
107     if (QIDX.dim2%QDIM) wblog(F_L,"ERR QSpace inconsistency "
108 " (QIDX: %dx%d/%d)",QIDX.dim1, QIDX.dim2, QDIM);
109
110     return (QIDX.dim2/QDIM);
111 };
112
113 unsigned len() const {
114     if (QIDX.dim1!=DATA.len) wblog(FL,
115 "ERR qspace inconsistency (QIDX.dim1=%d, len=%d).",
116 QIDX.dim1, DATA.len);
117     return QIDX.dim1;
118 };
119
120 unsigned nsym() const {
121     if (CGR.dim2 && qtype.len!=CGR.dim2) wblog(FL,
122 "ERR qspace inconsistency (nsym=%d '%s'; %s !?)",
123 qtype.len, STR(qtype), SSTR(CGR));
124     return (qtype.len ? qtype.len : QDIM);
125 };
126
127

```

```

128 TQ* Qref(unsigned r, unsigned k) { unsigned l=k*QDIM;
129     if (!QDIM || r>=QIDX.dim1 || (l+QDIM)>QIDX.dim2) wblog(FL,
130         "ERR %s() index out of bounds (%d/%d; %d*%d/%d)",
131         FCT,r,QIDX.dim1,k,QDIM,QIDX.dim2
132     );
133     return (QIDX.data + r*QIDX.dim2 + l);
134 };
135
136 const TQ* Qref(unsigned r, unsigned k) const {
137     unsigned l=k*QDIM;
138     if (!QDIM || r>=QIDX.dim1 || (l+QDIM)>QIDX.dim2) wblog(FL,
139         "ERR %s() index out of bounds (%d/%d; %d*%d/%d)",
140         FCT,r,QIDX.dim1,k,QDIM,QIDX.dim2
141     );
142     return (QIDX.data + r*QIDX.dim2 + l);
143 };
144
145 void init() { clearQSpace(); };
146
147 void init(unsigned m, unsigned n, unsigned nq) {
148     clearQSpace();
149     QDIM=nq; QIDX.init(m,nq*n);
150     setupDATA();
151 };
152
153 void init(const wbMatrix<TQ> &Q, unsigned qdim, const QVec *qt=NULL) {
154     clearQSpace();
155     QIDX=Q; QDIM=qdim; if (qt) qtype=(*qt); else qtype.init();
156     setupDATA(FL);
157 };
158
159 QSpace& init(const QSpace &B) {
160     initQT_safe(B);
161     for (unsigned i=0; i<B.DATA.len; ++i) {
162         DATA[i]->init(*(B.DATA[i]));
163     }
164     return *this;
165 };
166
167 template<class TB>
168 QSpace& initT(const QSpace<TQ,TB> &B) {
169     initQT_safe(B);
170     for (unsigned i=0; i<B.DATA.len; ++i) {
171         DATA[i]->initT(*(B.DATA[i]));
172     }; return *this;
173 };
174
175 template<class TR>
176 QSpace& getReal(QSpace<TQ,TR> &R) const {
177     unsigned i=0; R.initQT_safe(*this);
178     for (; i<DATA.len; ++i) { DATA[i]->getReal(*(R.DATA[i])); }
179     return R;
180 };
181
182 QSpace<TQ,double> getReal() const {
183     QSpace<TQ,double> R; unsigned i=0; R.initQT_safe(*this);
184     for (; i<DATA.len; ++i) { DATA[i]->getReal(*(R.DATA[i])); }
185     return R;
186 };
187
188 template<class TI>
189 QSpace& getImag(QSpace<TQ,TI> &I) const {
190     unsigned i=0; I.initQT_safe(*this);
191     for (; i<DATA.len; ++i) { DATA[i]->getImag(*(I.DATA[i])); }
192     return I;
193 };
194
195 QSpace<TQ,double> getImag() const {
196     QSpace<TQ,double> I; unsigned i=0; I.initQT_safe(*this);
197     for (; i<DATA.len; ++i) { DATA[i]->getImag(*(I.DATA[i])); }
198     return I;
199 };
200
201 void init(const char *file, int line, const mxArray *S, char ref=0) {
202     unsigned n = (S ? mxGetNumberOfElements(S) : 0);
203     if (n!=1) {
204         if (!n) wblog(FL,"ERR %s() got empty array",FCT);
205         else wblog(FL,"ERR %s() got QSpace array (%d entries)",FCT,n);
206     }
207     init(file,line,S,ref,0);
208 };
209
210 void init(const char *file, int line,
211     const mxArray *S, char ref, unsigned k);
212
213 QSpace& setupDATA(const char *file=NULL, int line=0){
214

```



```

215         if (!QIDX.dim1) { clearDATA(); return *this; }
216
217         if (QIDX.dim2 && (!QDIM || QIDX.dim2%QDIM)) wblog(__FL__,
218             "ERR invalid QIDX (%dx%d; %d)", QIDX.dim1, QIDX.dim2, QDIM);
219         clearDATA();
220
221         if (isref>1) wblog(FL, "ERR %s() got isref=%d", FCT, isref);
222
223         DATA.init(QIDX.dim1);
224
225         if (!isref) {
226             if (mt) { if (mt==Wb::MEX_RETURN) {
227                 if (WBLOG_MMEX) wblog(FL,
228                     "TST QSpace::init() to %s", Wb::MTYPE_STR[mt]); }
229                 else wblog(FL,
230                     "ERR %s() unexpected mtype=%s", FCT, Wb::MTYPE_STR[mt]);
231             }
232
233             for (unsigned i=0; i<DATA.len; ++i) {
234                 WB_NEW_1(DATA[i]);
235                 DATA[i]->mtype=mt;
236             }
237         }
238
239         return *this;
240     };
241
242     int gotCGS(const char *F=NULL, int L=0) const;
243
244     int gotCGX(const char *F=NULL, int L=0) const;
245
246     QSpace& Enlarge(unsigned m) {
247         if (!isConsistent(FL) || !QIDX.dim1) wblog(FL,
248             "ERR %s() can't enlarge empty QSpace (%d)", FCT, QIDX.dim1);
249         if (!m) return *this;
250
251         unsigned i, n0=QIDX.dim1, n=n0+m;
252         gotCGS(FL);
253
254         QIDX.Resize(n, QIDX.dim2);
255         DATA.Resize(n);
256         for (i=n0; i<n; ++i) { WB_NEW_1(DATA[i]); }
257
258         CGR.Resize(n, CGR.dim2);
259         return *this;
260     };
261
262     QSpace& Trim(unsigned n) {
263         if (!isConsistent(FL, 0) || n>QIDX.dim1) wblog(FL,
264             "ERR %s() can't trim to LARGER QSpace (%d/%d)", FCT, n, QIDX.dim1);
265         if (n==QIDX.dim1) { return *this; }
266
267         unsigned i, n2=QIDX.dim1;
268         gotCGS(FL);
269
270         QIDX.Resize(n, QIDX.dim2);
271
272         for (i=n; i<n2; ++i) { WB_DELETE_1(DATA[i]); }
273         DATA.Resize(n);
274
275         CGR.Resize(n, CGR.dim2);
276
277         return *this;
278     };
279
280     QSpace& initScalar(const TD &x) {
281         clearQSpace(); DATA.init(1);
282         QIDX.init(1, 0);
283
284         WB_NEW_2(DATA[0], wbararray<TD>(1, 1));
285         DATA[0]->data[0]=x;
286
287         return *this;
288     };
289
290     void init2ref(const mxArray *S) {
291         init(FL, S, 'r', 0);
292     };
293
294     void init2ref(const QSpace &);
295
296     void initOpZ_WET(
297         const char *F, int L, const QVec &qvec,
298         const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2,
299         const wbMatrix<TQ> &Q, const wbararray<TD> &D3,
300         const double eps1=1E-10,
301         const double eps2=1E-13

```

```

302     );
303
304     void reduceMatEl(const char *F, int L,
305         const wbMatrix<TQ> &Q1, const wbvector<TD> &dd,
306         const wbMatrix<TQ> &Q2, const wbvector<TD> &dc,
307         wbvector<unsigned> &I1, wbvector<unsigned> &I2,
308         wbvector<TD> &dr
309     ) const;
310
311     void unRef() {
312
313         if (!isref && !qtype.isref && !itags.isref &&
314             !QIDX.isref && !DATA.isref && !CGR.isref
315         ) return;
316
317         wblog(FL, "WRN %s() [%d%d%d%d%d]", FCT,
318             isref, QIDX.isref, DATA.isref, CGR.isref, qtype.isref, itags.isref);
319
320         QIDX.unRef(); qtype.unRef(); itags.unRef();
321
322         if (DATA.isref) DATA.unRef(); else
323             for (unsigned i=0; i<DATA.len; ++i) DATA[i]->unRef();
324
325         if (CGR.isref) CGR.unRef();
326     };
327
328     void initIdentity(const QSpace &A, char dflag=0);
329     void initIdentity(const wbMatrix<TQ> &Q);
330
331     QSpace& setupCGR(const char *F=NULL, int L=0);
332
333     QSpace& initIdentityCGS(const char *F=NULL, int L=0);
334     QSpace& initIdentityCGS(const char *F, int L, unsigned k, unsigned r=0);
335
336     template<class TD2>
337     QSpace& initIdentityCGS(const char *F, int L, const QSpace<TQ,TD2>& H);
338
339     template<class TA>
340     QSpace<TQ,TD>& initIdentityCG(
341         const wbvector< QSpace<TQ,TA> > &A, const wbindex &ia,
342         char zflag=0
343     );
344
345     template<class TA, class TB>
346     QSpace<TQ,TD>& initIdentityCG(
347         const wbvector< QSpace<TQ,TA> > &A, const wbindex &ia,
348         const wbvector< QSpace<TQ,TB> > &B, const wbindex &ib,
349         char vflag='v'
350     );
351
352     template<class TA, class TB>
353     QSpace<TQ,TD>& initIdentityCG(
354         const wbvector< QSpace<TQ,TA> > &A,
355         const wbvector< QSpace<TQ,TB> > &B,
356         const char *pstr=NULL
357     ){ wbindex i;
358        initIdentityCG(A,i,B,i);
359        if (pstr && pstr[0]) Permute(wbperm(pstr));
360        return *this;
361    }
362
363     QSpace& initDiagonal(
364         const wbMatrix<TQ> &Q,
365         const wbvector<unsigned> &S,
366         const wbvector<TD> &D,
367         char rcflag
368     );
369
370     QSpace& init2DiffOp(
371         const QSpace &A, unsigned ia,
372         const QSpace &B, unsigned ib
373     );
374
375     template<class TB>
376     QSpace& initQT_safe(const QSpace<TQ,TB> &B) {
377
378         clearQSpace();
379         QIDX=B.QIDX; QDIM=B.QDIM; qtype=B.qtype; otype=B.otype;
380         setupDATA(); itags=B.itags; CGR=B.CGR;
381
382         if (CGR.dim2) {
383             if (qtype.len!=CGR.dim2) wblog(FL,
384                 "ERR %s() got CGR size mismatch (%dx%d < %dx%d)",
385                 FCT,CGR.dim1,CGR.dim2,QIDX.dim1,qtype.len
386             );
387         }
388         else if (QIDX.data) {

```

```

389         if (DATA.len && !qtype.allAbelian()) wblog(FL,
390             "ERR %s() missing CGR data for %s (%dx%d <> %dx%d)",
391             FCT,STR(qtype),CGR.dim1,CGR.dim2,QIDX.dim1,qtype.len
392         );
393     }
394     return *this;
395 };
396
397 template <class TA>
398 QSpace& initQ(const QSpace<TQ,TA> &A, char full=1){
399     if (isref) wblog(FL,"WRN %s() got QSpace ref!",FCT);
400     qtype=A.qtype; QDIM=A.QDIM; if (full) {
401         otype=A.otype; itags=A.itags; }
402     return *this;
403 };
404
405 template <class TA, class TB>
406 QSpace& initQ(const char *F, int L,
407     const QSpace<TQ,TA> &A, char full=1, const QSpace<TQ,TB> *B=NULL
408 ){
409     if (B) A.checkQ(F,L,*B);
410     return initQ(A,full);
411 };
412
413 void initQ(unsigned n, unsigned r, const QSpace &A){
414     clearQSpace();
415     QDIM=A.QDIM; qtype=A.qtype; QIDX.init(n,r*QDIM);
416     setupDATA(FL);
417     if (!qtype.allAbelian()) setupCGR(FL);
418 };
419
420 void initQ(const wbMatrix<TQ> &Qk, const QSpace &A){
421     clearQSpace();
422     QDIM=A.QDIM; qtype=A.qtype; QIDX=Qk;
423     setupDATA(FL);
424     if (A.CGR.data) setupCGR(FL);
425 };
426
427 void init_itags(const char *F, int L, const char *s,
428     unsigned r=-1, char lflag=-1);
429
430 QSpace& SetTag(unsigned k, const iTag &b) {
431     if ((--k)>=itags.len) wblog(FL,
432         "ERR %s() index out of bounds (%d/%d)",FCT,k+1,itags.len);
433     itags.data[k].tSet(b);
434     return *this;
435 };
436
437 QSpace& SetTags(const iTag &b) {
438     unsigned k=rank(FL);
439     if (b.t && k>2) wblog(FL,
440         "ERR %s() got '%s' for rank-%d QSpace !?",FCT,STR(b),k);
441     for (k=0; k<itags.len; ++k) { itags.data[k].tSet(b); }
442
443     return *this;
444 };
445
446 QSpace& SetConjTags() {
447     for (unsigned k=0; k<itags.len; ++k) { itags.data[k].Conj(); }
448     return *this;
449 };
450
451 QSpace& SetConjTags(unsigned k1, unsigned k2) {
452     if (!k1 || !k2) wblog(FL,
453         "ERR %s(%d) expects 1-based index",FCT,k1,k2);
454     itags.SetConj(0,0,k1-1);
455     itags.SetConj(0,0,k2-1); return *this;
456 };
457
458 QSpace& SetConjTag(unsigned k) {
459     if (!k) wblog(FL,"ERR %s(%d) expects 1-based index",FCT,k);
460     itags.SetConj(0,0,k-1); return *this;
461 };
462
463 QSpace& SetConjTag(const char* F, int L, unsigned k) {
464     if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
465     itags.SetConj(F,L,k-1); return *this;
466 };
467
468 QSpace& SetFlag(const char *F, int L, unsigned k, unsigned l) {
469     if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
470     itags.SetFlag(F_L,k-1,l); return *this;
471 };
472
473 QSpace& UnsetFlag(const char *F, int L, unsigned k, unsigned l) {
474     if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
475     itags.UnsetFlag(F_L,k-1,l); return *this;

```

```

476     };
477
478     QSpace& UnsetFlags() { itags.UnsetFlags(); return *this; };
479
480     QSpace& RmItags() { itags.RmItags(); return *this; };
481
482     template<class TB>
483     unsigned matchITags(
484         const char *F, int L, const QSpace<TQ,TB> &B,
485         ctrIdx &ia, ctrIdx &ib) const;
486
487     unsigned RemoveZLabels(
488         const char *F, int L, wbMatrix<TQ> &QQ, wbMatrix<TQ> *Z=NULL) const;
489
490     void RemoveZLabels(const char *F, int L, wbMatrix<TQ> *Z){
491         if (QDIM==0 && qtype.len) { QDIM=qtype.Qlenz(); }
492         QDIM=RemoveZLabels(F,L,QIDX,Z);
493     };
494
495     void ExpandDiagonal(unsigned i1=1, unsigned i2=2);
496
497     size_t getDataSize(wbMatrix<size_t> &S) const;
498     size_t getDataSize(wbvector<size_t> &S) const;
499     size_t getDataSize() const;
500     size_t getCGSSize() const;
501     wbstring totSize2Str() const;
502
503     wbstring itags2Str() const {
504         return itags.toStr().data;
505     };
506
507     unsigned map2Vec (const char *F, int L, wbvector<TD> &V,
508         const wbvector<unsigned> &S=wbvector<unsigned>());
509
510     unsigned map2Vec(TD *v, char Iflag=0);
511
512     void ExpandQ(
513         CPAT<TQ,TD> &CP, double nrm, wbvector<unsigned> &xflag, char disp=0);
514
515     void ResetRec(unsigned k) {
516         isConsistent(FL); if (k>=QIDX.dim1) wblog(FL,
517             "ERR %s() index out of bounds (%d/%d)",FCT,k,QIDX.dim1);
518
519         QIDX.setRec(k,0); DATA[k]->init();
520         for (unsigned j=0; j<CGR.dim2; ++j) CGR(k,j).init();
521     };
522
523     void setData(const TD c) {
524         for (unsigned i=0; i<DATA.len; ++i)
525             DATA[i]->set(c);
526     };
527
528     void setRand(double nrm=0);
529
530     void markUnequalZero(const TD m=1, const TD eps=0) {
531         for (unsigned i=0; i<DATA.len; ++i)
532             DATA[i]->markUnequalZero(m,eps);
533     };
534
535     void PrependSingletons(unsigned R);
536
537     void swap(QSpace &B) {
538         if (isref || QIDX.isref || DATA.isref || CGR.isref) wblog(FL,
539             "ERR must not save referenced data (isref=%d,%d,%d)",
540             isref, QIDX.isref,DATA.isref);
541         if (B.isref || B.QIDX.isref || B.DATA.isref || B.CGR.isref) wblog(FL,
542             "ERR must not save referenced data (isref=%d,%d,%d)",
543             B.isref, B.QIDX.isref, B.DATA.isref);
544
545         DATA .swap(B.DATA );
546         QIDX .swap(B.QIDX ); SWAP(QDIM, B.QDIM );
547         CGR .swap(B.CGR ); SWAP(otype,B.otype);
548         qtype.swap(B.qtype);
549         itags.swap(B.itags); SWAP(isref,B.isref); SWAP(mt,B.mt);
550     };
551
552     QSpace& save2(QSpace &B) {
553         if (isref || QIDX.isref || DATA.isref || CGR.isref) wblog(FL,
554             "ERR must not save referenced data (isref=%d,%d,%d)",
555             isref, QIDX.isref,DATA.isref);
556         for (unsigned i=0; i<DATA.len; ++i) {
557             if (DATA[i]->isRef()) wblog(FL,"%NERR using save2 with "
558                 "referenced data\n(data[%d] has isref=%d)",
559                 i,DATA[i]->isRef());
560         };
561     }
562     B.clearQSpace();

```

```

563
564     DATA .save2(B.DATA );
565     QIDX .save2(B.QIDX ); B.QDIM=QDIM;
566     CGR .save2(B.CGR );
567
568     qtype.save2(B.qtype);
569     itags.save2(B.itags);
570     B.otype=otype; otype=QS_NONE;
571
572     B.isref=isref; isref=0;
573     B.ctime=ctime;
574     B.mt=mt;
575
576     return B;
577 };
578
579 unsigned Reduce2AbelianOp(const char *F=NULL, int L=0);
580
581 char allAbelian() const {
582     return qtype.allAbelian();
583 };
584
585 char isNonAbelian() const {
586     return qtype.isNonAbelian();
587 };
588
589 bool permitsOM() const {
590     if (CGR.isEmpty() || qtype.allAbelian()) return 0;
591     return qtype.permitsOM(rank(FL));
592 };
593
594 bool hasOM() const {
595     int q=QIDX.isSorted(), om=permitsOM();
596     if (!q) wblog(FL,"ERR %s() got non-sorted QIDX (q=%d)",FCT,q);
597     if (!(q&2) && !om) wblog(FL,"ERR %s() %s @ r=%d "
598         "got non-unique QIDX (q=%d/%d)",FCT,STR(qtype),rank(),q,om);
599     return (q&2 ? 0 : 1);
600 };
601
602 bool allAdditive() const {
603     return qtype.allAdditive();
604 };
605
606 bool isEmpty() const {
607     if (QIDX.dim1!=DATA.len && (QIDX.dim1 || DATA.len>1)) {
608         wblog(FL,"ERR QSpace inconsistency (%d,%d)",
609             QIDX.dim1, DATA.len);
610     }
611     return (QIDX.data==NULL);
612 };
613
614 bool isConsistent(
615     int &r, const char *F=NULL, int L=0, char level=2) const;
616
617 bool isConsistent(
618     const char *F=NULL, int L=0, char level=2
619 ) const { int r=-1; return isConsistent(r,F,L,level); };
620
621 bool isConsistentR(
622     int r, const char *F=NULL, int L=0, char level=2
623 ) const { return isConsistent(r,F,L,level); };
624
625 bool isScalar() const {
626     unsigned r=rank(FL); if (r || DATA.len!=1) return 0;
627     else {
628         const wvector<size_t> &S = DATA[0]->SIZE;
629         if (QIDX.dim1!=1 || S[0]!=1 || S[1]!=1) wblog(FL,
630             "ERR QSpace::%s() inconsistency (%d; S=%dx%d)",
631             FCT,QIDX.dim1,S[0],S[1]);
632     }
633     return 1;
634 };
635
636 bool isComplex() const {
637     unsigned i=0, n=DATA.len; isConsistent(FL);
638     for (; i<n; ++i) { if (DATA[i]->isComplex()) return 1; }
639     return 0;
640 };
641
642 bool isDiagBlock(unsigned k) const;
643
644 bool isBlockDiagMatrix(const char *F, int L, char dflag=0) const;
645 bool isBlockDiagMatrix(char dflag=0) const {
646     return isBlockDiagMatrix(NULL,0,dflag); }
647
648 bool isDiagMatrix(const double eps=1e-14) const;
649 bool isIdentityMatrix(const TD eps=1e-14) const;

```

```

650     char hasIdentityCGS(const TD eps=1e-14) const;
651
652     bool isR3Op() const {
653         return (rank(FL)==3 && otype==QS_OPERATOR);
654     };
655
656     int isOperator(unsigned *dr, char xflag=0) const;
657
658     bool isOperator() const { return (isOperator(NULL,0)>0 ? 1 : 0); };
659
660     bool isAtensor(unsigned r=-1) const { return itags.isAtensor(r); }
661
662     int checkAbelianOp() const;
663     bool isAbelianOp() const { return (checkAbelianOp())>0; };
664
665     bool isQSym(const char *F=0, int L=0, char dflag=0) const;
666
667     bool isHConj(
668         const char *F=0, int L=0, double eps=1e-14, char vflag=0
669     ) const;
670
671     bool isAHerm(
672         const char *F=0, int L=0, double eps=1e-14, char vflag=0
673     ) const;
674
675     int isSingleton(unsigned k) const;
676
677     template <class T2>
678     bool hasSameQ(const QSpace<TQ,T2> &B) const;
679
680     bool hasSameQ(
681         unsigned r1, unsigned k1,
682         unsigned r2, unsigned k2
683     ) const {
684         if (memcmp( Qref(r1,k1), Qref(r2,k2), QDIM*sizeof(TQ) )) return 0;
685         if (DATA[r1]->dim0(k1) != DATA[r2]->dim0(k2)) {
686             wblog(FL,
687                 "WRN %s()=1, yet DATA size mismatch (%s <> %s; %d,%d)",
688                 FCT, DATA[r1]->sizeStr().data,
689                 DATA[r2]->sizeStr().data, k1+1, k2+1);
690             return 0;
691         }
692         return 1;
693     };
694
695     template <class T2>
696     bool hasSameRank(const QSpace<TQ,T2> &B) const {
697         return (QIDX.dim2==B.QIDX.dim2 && QDIM==B.QDIM);
698     };
699
700     char checkQ(const char *F=NULL, int L=0) const;
701
702     template <class TB>
703     void checkQ(
704         const char *F, int L, const QSpace<TQ,TB> &B,
705         char aflag=0
706     ) const;
707
708     char checkQ_CGR(const char *F, int L, unsigned k=-1) const;
709
710     int check_CGR_cgw(const char *F=0, int L=0) const;
711
712     void checkScalarCGS(const char *F=0, int L=0) const;
713
714     template <class TB>
715     bool sameType(
716         const QSpace<TQ,TB> &B, int r=-1, const char *istr=NULL
717     ) const;
718
719     QDir& getQDir(QDir &qdir) const;
720     bool sameQDir(const char *F, int L, const QSpace &B) const;
721
722     bool hasQOverlap(const QSpace &A) const;
723
724     template <class TB>
725     int hasQOverlap(
726         unsigned ica, const QSpace<TQ,TB> &B, unsigned icb,
727         const char type=0
728     ) const;
729
730     int getQOverlapU(
731         const char *F, int L, const QSpace &B,
732         wbMatrix<TQ> &QQ, wbMatrix<int> &IQ,
733         char oonly=0
734     ) const;
735
736     int getQOverlap(

```

```

737     const char *F, int L, const QSpace &B,
738     wbMatrix<TQ> &QQ, wbMatrix<int> &IQ,
739     wbvector<widx_t> &DQ
740 ) const;
741
742 bool findDimQ(const TQ* q, unsigned &d) const;
743 bool findDimQ(const TQ* q, unsigned k, unsigned &d) const;
744
745 bool operator==(const QSpace &B) const {
746     if (this!=&B) {
747         if (! (QIDX==B.QIDX)) return 0;
748         if (! (qtype==B.qtype)) return 0;
749         if (! (CGR==B.CGR)) return 0;
750         if (! DATA.deepEqualP(B.DATA)) return 0;
751     }
752     return 1;
753 };
754
755 bool operator!=(const QSpace &B) const { return !((*this)==B); }
756
757 template<class T>
758 void operator*= (const T fac);
759
760 template<class T>
761 QSpace& times(const T fac, QSpace &B) const {
762     B=*this; B*=fac; return B;
763 };
764
765 QSpace& operator= (const QSpace &A) { return init(A); };
766
767 void operator+= (const QSpace &Q);
768 void operator-= (const QSpace &Q);
769
770 QSpace& Cat(const char *F, int L,
771     const QSpace<TQ,TD> &A, const QSpace<TQ,TD> &B,
772     TD afac=1, TD bfac=1, char uflag=1
773 );
774
775 QSpace& plus(
776     const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const;
777
778 QSpace& minus(
779     const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const;
780
781 QSpace& Plus (const QSpace &A, TD bfac=1);
782 QSpace& Minus(const QSpace &A, TD bfac=1);
783
784 QSpace& plus_plain(const QSpace &B, QSpace &C, TD bfac=1) const;
785
786 QSpace& NormCG(char skipzeros=1);
787
788 TD norm2() const;
789
790 TD norm() const { return sqrt(norm2()); }
791
792 TD normDiff2(const QSpace &B) const;
793 TD scalarProd(const QSpace &B) const;
794
795 void checkNorm(
796     const char *F, int L, double nrm, char vflag=1, double eps=1E-12
797 ) const;
798
799 QSpace& TimesEl(const QSpace &B, char conj=0) {
800     if (permitsOM() && (hasOM() || B.hasOM()))
801         { return TimesEl_OM(B,conj); }
802     else { return TimesEl_plain(B,conj); }
803 };
804
805 QSpace& TimesEl_plain(const QSpace &B, char conj=0);
806 QSpace& TimesEl_OM(const QSpace &B, char conj=0);
807
808 TD sumData() const;
809
810 template <class TM>
811 wbMatrix<TM>& diag(wbMatrix<TM> &xd) const;
812
813 QSpace& TensorProd(const QSpace &B, QSpace &C,
814     const char aflag='N', const char bflag='N') const;
815
816 template <class TC, ENABLE_IF_COMPLEX(TC) >
817 QSpace<TQ,TC>& TensorProd(const QSpace &B, QSpace<TQ,TC> &C,
818     const char aflag='N', const char bflag='N'
819 ) const { QSpace<TQ,TD> C_;
820     this->TensorProd(B,C_,aflag,bflag);
821     return C_.initT(C_);
822 };
823

```

```

824     template <class TC, ENABLE_IF_COMPLEX(TC) >
825     QSpace<TQ,TC>& TensorProd(const QSpace<TQ,TC> &B, QSpace<TQ,TC> &C,
826         const char aflag='N', const char bflag='N'
827     ) const { QSpace<TQ,TC> A; A.initT(*this);
828         return A.TensorProd(B,C,aflag,bflag);
829     };
830
831     void TensorProdUnity(const wbMatrix<TQ> &Q, QSpace &C) const;
832
833     void EigenSymmetric(wbMatrix<double> &Etot) const {
834         QSpace Ak,Ad;
835         QSpace<TQ,double> Ek,Et;
836         wbMatrix<unsigned> DB; int Nkeep=-1;
837
838         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
839     };
840
841     void EigenSymmetric(
842         QSpace &Ak, QSpace<TQ,double> &Ek, wbMatrix<double> &Etot
843     ) const {
844         QSpace Ad;
845         QSpace<TQ,double> Et;
846         wbMatrix<unsigned> DB; int Nkeep=-1;
847
848         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
849     };
850
851     void EigenSymmetric(QSpace &Ak, QSpace<TQ,double> &Ek) const {
852         QSpace Ad;
853         QSpace<TQ,double> Et;
854         wbMatrix<unsigned> DB; int Nkeep=-1;
855         wbMatrix<double> Etot;
856
857         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
858     };
859
860     void EigenSymmetric(
861         QSpace &Ak, QSpace &Ad,
862         QSpace<TQ,double> &Ek, QSpace<TQ,double> &Et, wbMatrix<double> &Etot,
863         wbMatrix<unsigned> &DD, int &Nkeep,
864         double Etrunc=0, double *E0=NULL,
865         const wbperm &P=wbperm(), double eps=0., double b=0., int dmax=-1,
866         const char *sort=NULL
867     ) const;
868
869     void GroupIndizes(unsigned K);
870
871     void Append(unsigned k);
872     void Append(const wbvector<TQ> &, const wbararray<TD> &D);
873
874     QSpace& Append(const char *F, int L, const QSpace &A, char uflag=1);
875
876     int Append2AndDestroy(
877         const char *F, int L, QSpace &A, char unique=0);
878
879     template <class TA, ENABLE_IF_COMPLEX(TA) >
880     int Append2AndDestroy(
881         const char *F, int L, QSpace<TQ,TA> &A, char unique=0
882     ){
883         QSpace<TQ,TA> X; X.initT(*this);
884         return X.Append2AndDestroy(F,L,A,unique);
885     };
886
887     void TransferSpace(const char *F, int L,
888         QSpace &B, unsigned k,
889         const QSpace<TQ,double> &W, double eps
890     );
891
892     void TransferSpace(const char *F, int L,
893         QSpace &B, unsigned k
894     );
895
896     unsigned makeUnique(char omFlag=-1);
897     unsigned makeUnique_plain(char omFlag=-1);
898     unsigned makeUnique_ortho(char vflag=0);
899
900     bool isZero(const double eps=0., char flag=0) const;
901     unsigned SkipZeroData(
902         double eps=0., char bflag=0, char cgflag=1,
903         char all=0
904     );
905
906     unsigned skipZeroOffDiag(double eps=0., char bflag=0);
907
908     unsigned SkipEmptyData(const char *F=0, int L=0);
909
910     bool gotZeroData(unsigned i, const double eps=0.) const;

```



```

911
912 void Set(unsigned k, const wbvector<TQ> &Q, const wbararray<TD> &D);
913 void SetQ(unsigned i, unsigned k, const TQ *d);
914
915 QSpace& Select(const wbindex &I, char data_only=0);
916
917 template <class TM>
918 QSpace& SelectMarked(const wbvector<TM> &mark){
919     if (mark.len!=QIDX.dim1) wblog(FL,
920         "ERR %s() size inconsistency (%d/%d)",FCT,mark.le,QIDX.dim1);
921     wbindex I; mark.find(I);
922     return Select(I);
923 };
924
925 void Sort(const QSpace &A);
926 void Sort() {
927     wbpem P; QIDX.SortRecs(P); DATA.Select(P);
928     if (qtype.allAbelian()) {
929         if (!CGR.isEmpty()) wblog(FL,"ERR %s() "
930             "invalid all-abelian CGR (%dx%d)",FCT,CGR.dim1,CGR.dim2);
931     }
932     else CGR.recPermute(P);
933 };
934
935 unsigned getOM(wbMatrix<unsigned> &OM,
936     wbvector<unsigned> *oml=NULL, wbMatrix<TQ> *Q=NULL) const;
937
938 unsigned cgsDimScalar(unsigned k) const;
939 unsigned cgsDim(unsigned k, unsigned r) const;
940 unsigned getDim(unsigned k, widx_t *D=NULL) const;
941
942 unsigned getDIM(unsigned k) const {
943     widx_t D=0; getDim(k,&D);
944     return D;
945 };
946
947 void getDim(wbvector<widx_t> &D, wbvector<widx_t> *DD=NULL) const;
948
949 wbvector<widx_t> getDim() const {
950     wbvector<widx_t> D; getDim(D); return D;
951 }
952
953 void getQDim(unsigned k,
954     wbMatrix<TQ> &Q,
955     wbvector<widx_t> &S,
956     wbMatrix<widx_t> *SC=NULL
957 ) const;
958
959 void getQDim(wbMatrix<TQ> &Q,
960     wbvector<widx_t> &S,
961     wbMatrix<widx_t> *SC=NULL
962 ) const;
963
964 void getDRange(TD &dmin, TD &dmax) const;
965
966 void getDistQtot(wbMatrix<TQ> &Qtot, wbvector<double> &w) const;
967 widx_t getDQtot() {
968     wbMatrix<TQ> Q; QIDX.blockSum(QDIM,Q); Q.makeUnique();
969     return Q.dim1;
970 };
971
972 wbMatrix<TQ> & getQsub(const wbindex &I, wbMatrix<TQ> &QI) const;
973 wbMatrix<TQ> & getQsub(
974     const unsigned k, wbMatrix<TQ> &Qk, char iflag=0) const;
975
976 void getQsub(const wbindex &I, wbMatrix<TQ> &QI, wbMatrix<TQ> &Q2) const;
977
978 wbMatrix<TQ> getQsub(const unsigned k) const {
979     wbMatrix<TQ> Qk; getQsub(k,Qk); return Qk;
980 };
981
982 wbMatrix<TQ> & getQtot(wbMatrix<TQ> &Qtot, char uflag=0) const;
983 wbMatrix<TQ> & getQsum(const wbindex &J, wbMatrix<TQ> &Q) const;
984 wbMatrix<TQ> & getQsum(const wbMatrix<TQ> &Q0, wbMatrix<TQ> &Q) const;
985
986 wbMatrix<TQ> & getQfinal(const wbindex &J, wbMatrix<TQ> &Q) const;
987
988 QSpace& getSub(const wbindex &I, QSpace &A, char ref=0) const;
989
990 QSpace& getSubInit(const wbindex &I, QSpace &A) const;
991
992 void saveSub2(QSpace &A, const wbindex &I);
993
994 void toFull(
995     wbararray<TD> &A,
996     wbvector< wbMatrix<TQ> > &Q,
997     wbvector< wbvector<unsigned> > &S

```

```

998     ) const;
999
1000     void toFull12(
1001         wbararray<TD> &A,
1002         wbMatrix<TQ> &Q,
1003         wbMatrix<wid_x_t> &S
1004     ) const;
1005
1006     void ExtendQ(const QSpace &B);
1007
1008     void takeMainQS(char disp=1);
1009
1010     void setQ(unsigned k, const QSpace &B, unsigned k0);
1011     void setQRec(unsigned r, unsigned j, const TQ *q0, const TQ *q2=NULL);
1012
1013     bool hasEqualQ(unsigned i1, unsigned i2) const;
1014
1015     double maxDiff(const QSpace &B) const;
1016
1017     void PermuteQ(const wbperm &P);
1018     void permuteQ(const wbperm &P, wbMatrix<TQ>&) const;
1019
1020     QSpace& Permute(const wbperm &P, char iflag=0);
1021     QSpace& permute(const wbperm &P, QSpace &B, char iflag=0) const;
1022
1023     void permute (const char *pstr, QSpace &B, unsigned offset=1
1024     ) const {
1025         permute (wbperm(pstr,offset),B);
1026     };
1027
1028     void Permute(const char* pstr, unsigned offset=1) {
1029         Permute (wbperm(pstr,offset));
1030     };
1031
1032     void PermuteFirstTo (unsigned k) {
1033         unsigned r=rank(FL);
1034         if (k>r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
1035         wbperm P2(r); P2.mvFirstTo(k); Permute(P2);
1036     };
1037     void permuteFirstTo (unsigned k, QSpace &C) {
1038         if (this==&C) { PermuteFirstTo(k); return; }
1039         unsigned r=rank(FL);
1040         if (k>r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
1041         wbperm P2(r); P2.mvFirstTo(k); permute(P2,C);
1042     };
1043
1044     void PermuteLastTo (unsigned k) {
1045         unsigned r=rank(FL);
1046         if (k>r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
1047         wbperm P2(r); P2.mvLastTo(k); Permute(P2);
1048     };
1049     void permuteLastTo (unsigned k, QSpace &C) {
1050         if (this==&C) { PermuteLastTo(k); return; }
1051         unsigned r=rank(FL);
1052         if (k>r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
1053         wbperm P2(r); P2.mvLastTo(k); permute(P2,C);
1054     };
1055
1056     void FlipQ();
1057
1058     QSpace& trace(
1059         const char *F, int L, ctrIdx &i1, ctrIdx &i2, QSpace &C) const;
1060     TD trace() const;
1061
1062     void contractMat(
1063         unsigned ia, const QSpace &B, unsigned ib,
1064         QSpace &C
1065     ) const;
1066
1067     template <class TB, class TC>
1068     int contract_getIdxSet(const char *F, int L,
1069         const ctrIdx &ica, const QSpace<TQ,TB> &B, const ctrIdx &icb,
1070         wbindx &Ia, wbindx &Ib, wbvector<wid_x_t> &Dc,
1071         QSpace<TQ,TC> &C
1072     ) const;
1073
1074     template<class TB, class TC>
1075     int contract(
1076         const char *F, int L, const ctrIdx &ica,
1077         const QSpace<TQ,TB> &B, const ctrIdx &icb, QSpace<TQ,TC> &C,
1078         const wbperm &P=wbperm(), char cgflag=-1,
1079         char cg_preview=0
1080     ) const;
1081
1082     template<class TB, class TC>
1083     int contract(const char *F, int L,
1084         unsigned ia, const QSpace<TQ,TB> &B, unsigned ib,

```

```

1085     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char cgflag=-1
1086 ) const {
1087
1088     unsigned ra=rank(FL), rb=B.rank(FL);
1089     if (!ia || (ia>ra && (QIDX.dim1 || QIDX.dim2))) wblog(FL,
1090         "ERR %s() index out of bounds (ia=%d/%d; %s)",
1091         FCT,ia,ra,SSTR(QIDX));
1092     if (!ib || (ib>rb && (B.QIDX.dim1 || B.QIDX.dim2))) wblog(FL,
1093         "ERR %s() index out of bounds (ib=%d/%d; %s)",
1094         FCT,ib,rb,SSTR(B.QIDX)
1095     );
1096
1097     ctrIdx ica(1,&(--ia)), icb(1,&(--ib));
1098     return contract(F,L,ica,B,icb,C,P,cgflag);
1099 };
1100
1101 template<class TB, class TC>
1102 int contract(const char *F, int L,
1103     unsigned ia,
1104     const QSpace<TQ,TB> &B, const char *sb,
1105     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char cgflag=-1
1106 ) const {
1107
1108     ctrIdx ica(1,&(--ia)), icb;
1109
1110     if (isEmpty() || B.isEmpty()) { C.init(); return 1; }
1111     if (ia>itags.len) {
1112         wblog(FL,"ERR %s() index out of bounds (ia=%d/%d; %s)",
1113             FCT,ia+1,itags.len,STR(itags));
1114     }
1115
1116     if (sb && isdigit(sb[0]))
1117         { icb.init(F_L,sb); }
1118     else { icb.initOp(FL,itags[ia],B,sb); }
1119
1120     return contract(F,L,ica,B,icb,C,P,cgflag);
1121 };
1122
1123 template<class TB, class TC>
1124 int contract(const char *F, int L,
1125     const char *sa,
1126     const QSpace<TQ,TB> &B, unsigned ib,
1127     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char cgflag=-1
1128 ) const {
1129
1130     ctrIdx ica, icb(1,&(--ib));
1131
1132     if (isEmpty() || B.isEmpty()) { C.init(); return 1; }
1133
1134     if (ib>B.itags.len) wblog(FL,
1135         "ERR %s() index out of bounds (ib=%d/%d; %s)",
1136         FCT,ib+1,B.itags.len,STR(B.itags));
1137
1138     if (sa && isdigit(sa[0]))
1139         { ica.init(F_L,sa); }
1140     else { ica.initOp(FL,B.itags[ib],*this,sa); }
1141
1142     return contract(F,L,ica,B,icb,C,P,cgflag);
1143 };
1144
1145 template<class TB, class TC> inline
1146 int contract(const char *F, int L,
1147     const char *sa, const QSpace<TQ,TB> &B, const char *sb,
1148     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char cgflag=-1
1149 ) const {
1150
1151     ctrIdx ica(F_L,sa), icb(F_L,sb);
1152     return contract(F,L,ica,B,icb,C,P,cgflag);
1153 };
1154
1155 template<class TB, class TC>
1156 int contract(
1157     const char *sa, const QSpace<TQ,TB> &B, const char *sb,
1158     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char cgflag=-1
1159 ) const { return contract(FL,sa,B,sb,C,P,cgflag); };
1160
1161 template<class TB, class TC>
1162 int contract(
1163     const ctrIdx &ica, const QSpace<TQ,TB> &B, const ctrIdx &icb,
1164     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char cgflag=-1
1165 ) const { return contract(FL,ica,B,icb,C,P,cgflag); };
1166
1167 template<class TB, class TC>
1168 int contract(
1169     unsigned ia, const QSpace<TQ,TB> &B, unsigned ib,
1170     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char cgflag=-1
1171 ) const { return contract(FL,ia,B,ib,C,P,cgflag); };

```

```

1172
1173     mxArray* toMx(bool setcls=0) const;
1174
1175     mxArray* save2Mx(char vflag=0);
1176
1177     mxArray* QIDX_toMx() const;
1178     mxArray* INFO_toMx() const;
1179     mxArray* DATA_toMx() const;
1180     mxArray* DATA_save2Mx(char vflag=0);
1181
1182     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
1183     void add2MxStruct(mxArray *S, unsigned i, bool setcls=0, char chk=0) const;
1184
1185     void save2MxStruct(mxArray *S, unsigned i, char tst=0, char vflag=0);
1186
1187     mxArray* mxCreateCell(unsigned m, unsigned n) const {
1188         wblog(FL,"ERR %s()",FCT); return 0; };
1189     void add2MxCell(mxArray *S, unsigned i, char tst=0) const {
1190         wblog(FL,"ERR %s()",FCT); };
1191
1192     void put (
1193         const char *F, int L,
1194         const char *vname, const char *ws="caller"
1195     ) const {
1196         wblog(F,L,"I/O putting '%s' to %s", vname, ws);
1197         put(vname,ws);
1198     };
1199
1200     void put (const char *vname, const char *ws="caller") const {
1201         mxArray *S=toMx();
1202         mxPutAndDestroy(0,0,S,vname,ws);
1203     };
1204
1205     wbsting sizeStrQ(const char *F=NULL, int L=0) const;
1206
1207     wbsting sizeStr(char vflag=0) const;
1208     wbsting qStr() const { return qtype.toStr(); };
1209
1210     wbsting qrec2Str(unsigned k) const {
1211         return QIDX.rec2Str(k,"", " ", QDIM," ");
1212     };
1213
1214     void disp_cgs(const char *F, int L, const char *istr) const;
1215
1216     void info(const char *vname=NULL,
1217         unsigned nlt=0, unsigned nlb=0,
1218         unsigned nind=2
1219     ) const;
1220
1221     void print(const char *vname, char vflag=1) const;
1222
1223     void initOType(const char *F, int L, const mxArray* a);
1224
1225     wbsting otype2Str(const char *fmt="%s") const;
1226
1227     QSpace& ConjOpScalar() {
1228         unsigned r=rank(FL);
1229         if (r!=3) {
1230             if (r!=2) wblog(FL,
1231                 "WRN %s() got rank-%d QSpace !?",FCT,r);
1232             return *this;
1233         }
1234         for (unsigned n=CGR.numel(), i=0; i<n; ++i) {
1235             CGR[i].HConjOpScalar();
1236         }
1237
1238         if (itags.len!=3) wblog(FL,
1239             "ERR %s() got itags.len=%d/3 !?",FCT,itags.len);
1240         itags[2].Conj();
1241
1242         return *this;
1243     };
1244
1245     QSpace& SortDegQ(const char *F=0, int L=0) {
1246         for (unsigned n=CGR.numel(), i=0; i<n; ++i) {
1247             CGR.data[i].SortDegQ(F_L); }
1248         return *this;
1249     };
1250
1251     QSpace& Conj() {
1252         unsigned i,n; SetConjTags();
1253         if (WbUtil<TD>::isComplex()) {
1254             for (n=DATA.len, i=0; i<n; ++i) DATA[i]->Conj(); }
1255         for (n=CGR.numel(), i=0; i<n; ++i) { CGR[i].Conj(); }
1256         return SortDegQ(FL);
1257     };
1258

```

```

1259     QSpace& dConj() {
1260         if (WbUtil<TD>::isComplex()) {
1261             for (unsigned n=DATA.len, i=0; i<n; ++i) DATA[i]->Conj(); }
1262         return *this;
1263     };
1264
1265     QSpace& HConj() {
1266         QSpace B; this->save2(B);
1267         return B.hconj(*this);
1268     };
1269
1270     QSpace& hconj(QSpace &B) const {
1271         if (!isEmpty()) {
1272             if (!QDIM || QIDX.dim2%QDIM) wblog(FL,
1273                 "ERR %s() QSpace inconsistency (%dx%d/%d)",
1274                 FCT, QIDX.dim1, QIDX.dim2, QDIM);
1275             unsigned r=QIDX.dim2/QDIM;
1276             if (r%2 && (r!=3 || otype!=QS_OPERATOR)) wblog(FL,"ERR %s() "
1277                 "requires even-rank object or operator (%dx%d/%d; %s)",
1278                 FCT, QIDX.dim1, QIDX.dim2, QDIM, QS_STR[otype]);
1279             transp(B); B.Conj(); B.otype=otype;
1280             return B;
1281         }
1282         return B.clearQSpace();
1283     };
1284
1285     QSpace& transp(QSpace &B) const {
1286         if (!isEmpty()) { unsigned r=rank(FL);
1287             if (r%2==0) { wbpem P2;
1288                 P2.initTranspose(r); permute(P2,B);
1289             }
1290             else if (r==3 && otype==QS_OPERATOR) {
1291                 permute("2,1,3", B);
1292             }
1293             else wblog(FL,"ERR %s got rank-%d object",FCT,r);
1294         }
1295         else B.clearQSpace();
1296         return B;
1297     };
1298
1299     const QSpace& opA(char tflag, QSpace& X) const {
1300         if (!strchr("NTC",tflag))
1301             wblog(FL,"ERR %s() invalid flag %c<%d>",FCT,tflag,tflag);
1302
1303         if (tflag=='N' || isEmpty()) return (*this);
1304         else {
1305             wbpem P2("2 1"); unsigned r=rank(FL);
1306
1307             if (r!=2) wblog(FL,
1308                 "ERR %s() only applicable to rank-2 objects (%d)",FCT,r);
1309
1310             permute(P2,X); if (tflag=='C') X.Conj();
1311             return X;
1312         }
1313     };
1314
1315     };
1316
1317     WbMatrix<TQ> QIDX;
1318
1319     Wbvector<Wbarray<TD>*> DATA;
1320     WbMatrix< CRef<TQ> > CGR;
1321
1322     QVec qtype;
1323
1324     unsigned QDIM;
1325
1326     QS_TYPES otype;
1327     IndexLabels itags;
1328
1329     Wb::MTYPE mt;
1330
1331     char isref;
1332
1333     double ctime;
1334
1335 protected:
1336 private:
1337
1338     bool isSym_aux(
1339         const char *F, int L, const char *fct,
1340         RTD eps=1E-14, char symflag='s', char vflag=0
1341     ) const;
1342
1343     void recSetQ(unsigned k, unsigned i);
1344
1345 };

```

```

1346
1347 template <class TQ, class TD>
1348 template <class T> inline
1349 void QSpace<TQ,TD>::operator*= (const T fac) {
1350     if (fac==T(1)) return;
1351     for (unsigned i=0; i<DATA.len; ++i)
1352         (*DATA[i]) *= fac;
1353 };
1354
1355 template <class TQ, class TD> inline
1356 void QSpace<TQ,TD>::operator+= (const QSpace &B) {
1357     QSpace<TQ,TD> X;
1358     this->plus(B,X); X.save2(*this);
1359 };
1360
1361 template <class TQ, class TD> inline
1362 void QSpace<TQ,TD>::operator-= (const QSpace &B) {
1363     QSpace<TQ,TD> X;
1364     this->plus(B,X,-1); X.save2(*this);
1365 };
1366
1367 template <class TQ, class TD> inline
1368 QSpace<TQ,TD>& QSpace<TQ,TD>::Plus(const QSpace &B, TD bfac) {
1369     QSpace<TQ,TD> X;
1370     this->plus(B,X,bfac); return X.save2(*this);
1371 };
1372
1373 template <class TQ, class TD> inline
1374 QSpace<TQ,TD>& QSpace<TQ,TD>::Minus(const QSpace &B, TD bfac) {
1375     QSpace<TQ,TD> X;
1376     this->plus(B,X,-bfac); return X.save2(*this);
1377 };
1378
1379 template <class TQ, class TD> inline
1380 QSpace<TQ,TD>& QSpace<TQ,TD>::minus(
1381     const QSpace &B, QSpace &C, TD bfac, char vflag
1382 ) const {
1383     plus(B, C,-bfac,vflag); return C;
1384 }
1385
1386 template <class TQ, class TD>
1387 QSpace<TQ,TD>& QSpace<TQ,TD>::plus(
1388     const QSpace &B, QSpace &C, TD bfac, char vflag
1389 ) const {
1390
1391     if (B.isEmpty() || !bfac) { C=(*this); } else
1392     if (this->isEmpty()) { C=B; if (bfac!=1.) { C*=bfac; }} else
1393     if (gotCGS(FL)<2) { plus_plain(B,C,bfac); }
1394     else {
1395         C=B; if (bfac!=1.) { C*=bfac; }
1396         C.Append(FL,*this, vflag<1 ? 'q':vflag);
1397     }
1398
1399     return C;
1400 };
1401
1402 #ifdef WB_CLOCK
1403 Wb::Clock wbc_qs_scpod("QS::scalarProd");
1404 #endif
1405
1406 template <class TQ, class TD>
1407 TD QSpace<TQ,TD>::scalarProd(const QSpace<TQ,TD> &B) const {
1408
1409     unsigned i; int i1,i2;
1410     wbMatrix<int> IQ;
1411     wbMatrix<TQ> QQ;
1412     TD x=0;
1413
1414     if (this==&B) return norm2();
1415
1416 #ifdef WB_CLOCK
1417     Wb::Clock_resume sw(&wbc_qs_scpod);
1418 #endif
1419
1420     if (isEmpty() || B.isEmpty()) {
1421         wblog(FL,"WRN Overlap with empty object (%d,%d) !?",
1422             isEmpty(), B.isEmpty()); return x;
1423     }
1424
1425     getQOverlapU(FL,B,QQ,IQ,1);
1426
1427     for (i=0; i<IQ.dim1; ++i) {
1428         i1=IQ(i,0); i2=IQ(i,1);
1429         if (i1<0 || i2<0) wblog(FL, "ERR i1=%d, i2=%d !?", i1, i2);
1430
1431         x+=(DATA[i1]->scalarProd(*B.DATA[i2]));
1432     }

```

```

1433
1434     return x;
1435 };
1436
1437 template <class TQ, class TD>
1438 QSpace<TQ,TD>& QSpace<TQ,TD>::TensorProd(
1439     const QSpace<TQ,TD> &B, QSpace<TQ,TD> &C,
1440     const char aflag, const char bflag
1441 ) const {
1442
1443     if (isEmpty() || B.isEmpty()) { C.clearQSpace(); return C; }
1444
1445     if (gotCGS(FL)>0 || B.gotCGS(FL)>0) wblog(FL,
1446         "ERR %s() not implemented for non-abelian symmetries (got %s)",
1447         FCT,STR(qtype));
1448
1449     if (aflag!='N' || bflag!='N') {
1450         QSpace<TQ,TD> AX, BX;
1451         const QSpace<TQ,TD> *Ap=this, *Bp=&B;
1452
1453         if (!strchr("NTC",aflag) || rank(FL)!=2) wblog(FL,
1454             "ERR invalid aflag=%c<%d> (rank-%d)",aflag,aflag,rank(FL));
1455         if (!strchr("NTC",bflag) || B.rank(FL)!=2) wblog(FL,
1456             "ERR invalid bflag=%c<%d> (rank-%d)",bflag,bflag,B.rank(FL));
1457
1458         if (aflag!='N') {
1459             permute("2,1",AX); if (aflag=='C') AX.Conj();
1460             Ap=&AX;
1461         }
1462         if (bflag!='N') {
1463             B.permute("2,1",BX); if (bflag=='C') BX.Conj();
1464             Bp=&BX;
1465         }
1466
1467         return Ap->TensorProd(*Bp,C,'N','N');
1468     }
1469
1470     unsigned i,ia,ib,l,r;
1471     const unsigned n1=QIDX.dim1, n2=B.QIDX.dim1, R=rank(FL), s=QDIM*sizeof(TQ);
1472     const TQ *qa, *qb; TQ *q;
1473
1474     if (R!=B.rank(FL) || QDIM!=B.QDIM) wblog(FL,
1475         "ERR %s() size mismatch (%d,%d)",FCT,R,B.rank(FL));
1476
1477     C.clearQSpace();
1478     C.QIDX.init(n1*n2, 2*QIDX.dim2); C.QDIM=QDIM; C.setupDATA();
1479
1480     C.initQ(FL,*this,'f',&B);
1481
1482     if (itags.len || B.itags.len) { C.itags.init(2*R);
1483         if (!B.itags.len) {
1484             if (itags.len!=R) wblog(FL,
1485                 "ERR %s() invalid itag %s",FCT, STR(itags));
1486             for (i=0; i<R; ++i) {
1487                 C.itags[2*i]=C.itags[2*i+1]=itags[i];
1488             }
1489         }
1490         else if (!itags.len) {
1491             if (B.itags.len!=R) wblog(FL,
1492                 "ERR %s() invalid itag %s",FCT, STR(B.itags));
1493             for (i=0; i<R; ++i) {
1494                 C.itags[2*i]=C.itags[2*i+1]=B.itags[i];
1495             }
1496         }
1497         else {
1498             if (itags.len!=B.itags.len || itags.len!=R || !sameQDir(FL,B))
1499                 wblog(FL,"ERR %s() itag/qdir mismatch (%s <> %s)",FCT,
1500                     STR(itags),STR(B.itags));
1501             for (i=0; i<R; ++i) {
1502                 C.itags[2*i] = itags[i];
1503                 C.itags[2*i+1]=B.itags[i];
1504             }
1505         }
1506     }
1507
1508     for (i=ib=0; ib<n2; ++ib) {
1509         qb=B.QIDX.rec(ib);
1510
1511         for (ia=0; ia<n1; ++ia, ++i) {
1512             qa=QIDX.rec(ia); q=C.QIDX.rec(i);
1513
1514             for (l=r=0; r<R; ++r, l+=QDIM) {
1515                 memcpy(q,qa+l,s); q+=QDIM;
1516                 memcpy(q,qb+l,s); q+=QDIM;
1517             }
1518             DATA[ia]->tensorProd(*B.DATA[ib], *C.DATA[i]);
1519         }

```

```

1520     }
1521
1522     return C;
1523 };
1524
1525 template <class TQ, class TD> inline
1526 void QSpace<TQ,TD>::GroupIndizes(unsigned K) {
1527     unsigned i; int r=-1;
1528
1529     if (!isConsistent(r)) werror(FL,str);
1530
1531     if (!K || unsigned(r)%K) wblog(FL,
1532     "ERR %s() cannot block QSpace (%d/%d)",FCT,r,K);
1533
1534     QDIM=K*QDIM;
1535
1536     for (i=0; i<DATA.len; ++i)
1537         DATA[i]->GroupIndizes(K);
1538 }
1539
1540 template <class TQ, class TD>
1541 size_t QSpace<TQ,TD>::getDataSize(wbMatrix<size_t> &S) const {
1542
1543     size_t s=0; unsigned i,m,n=DATA.len;
1544
1545     if (n!=QIDX.dim1) wblog(FL,
1546     "ERR size mismatch (%d/%d)",QIDX.dim1,DATA.len);
1547
1548     if (isEmpty()) { S.init(); return s; }
1549
1550     m=DATA[0]->SIZE.len; S.init(n,m);
1551
1552     for (i=0; i<n; ++i) {
1553         const wbvector<size_t> &si=DATA[i]->SIZE;
1554         if (si.len!=m) wblog(FL,"ERR size mismatch (%d/%d)",si.len,m);
1555         S.recSetP(i,si.data);
1556         s+=si.prod(0);
1557     }
1558     return s;
1559 };
1560
1561 template <class TQ, class TD> inline
1562 size_t QSpace<TQ,TD>::getDataSize(wbvector<size_t> &S) const {
1563     size_t s=0; S.init(DATA.len);
1564     for (unsigned i=0; i<DATA.len; ++i) { s+=(S[i]=DATA[i]->SIZE.prod()); }
1565     return s;
1566 };
1567
1568 template <class TQ, class TD> inline
1569 size_t QSpace<TQ,TD>::getDataSize() const {
1570     size_t s=0;
1571     for (unsigned i=0; i<DATA.len; ++i) s+=DATA[i]->SIZE.prod();
1572     return s;
1573 };
1574
1575 template <class TQ, class TD> inline
1576 size_t QSpace<TQ,TD>::getCGSSize() const {
1577     size_t s=0;
1578     if (gotCGS(FL)>0) {
1579         unsigned i,j;
1580         for (i=0; i<CGR.dim1; ++i) {
1581             for (j=0; j<CGR.dim2; ++j) s+=CGR(i,j).numel(); }
1582     }
1583     return s;
1584 };
1585
1586 template <class TQ, class TD> inline
1587 wstring QSpace<TQ,TD>::totSize2Str() const {
1588
1589     unsigned long ss[3]={
1590         sizeof(TD)*getDataSize(),
1591         sizeof(TQ)*(QIDX.dim1*QIDX.dim2),
1592         sizeof(double)*getCGSSize()
1593     };
1594     char s_[32];
1595     memsize2Str(ss[0]+ss[1]+ss[2],s_);
1596     return s_;
1597 };
1598
1599 template <class TQ, class TD> inline
1600 bool QSpace<TQ,TD>::isZero(const double eps, char flag) const {
1601
1602     for (unsigned i=0; i<DATA.len; ++i)
1603         if (!(DATA[i]->isZero(eps,flag))) return 0;
1604
1605     return 1;
1606 }

```



```

1607
1608 template <class TQ, class TD> inline
1609 void QSpace<TQ,TD>::Set(
1610     unsigned k, const wbvector<TQ> &Q, const wbararray<TD> &D
1611 ){
1612
1613     QIDX.recSet(k,Q);
1614
1615     if (k>=DATA.len)
1616         wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,k,DATA.len);
1617
1618     DATA[k]->init(D);
1619 }
1620
1621 template <class TQ, class TD> inline
1622 void QSpace<TQ,TD>::SetQ(unsigned i, unsigned k, const TQ *d){
1623
1624     if (i>=QIDX.dim1 || k>=rank(FL)) wblog(FL,
1625         "ERR index out of bounds (%d/%d; %d/%d)",i,QIDX.dim1,k,rank(FL));
1626
1627     memcpy(QIDX.data+QIDX.dim2*i+QDIM*k, d, QDIM*sizeof(TQ));
1628 }
1629
1630 template <class TQ, class TD> inline
1631 void QSpace<TQ,TD>::Sort(const QSpace<TQ,TD> &R) {
1632     int e;
1633     wbindex P,Ir; wbpem Pr;
1634
1635     if (QIDX.dim1==0) return;
1636
1637     e=matchIndex(QIDX,R.QIDX, P, Ir);
1638
1639     if (e || P.len!=QIDX.dim1) wblog(FL,
1640         "ERR QSpace R not complete to be used as sorting reference "
1641         "(%d/%d)",P.len, QIDX.dim1);
1642
1643     Ir.Sort(Pr); P.Select(Pr);
1644
1645     QIDX.recPermute(P); if (!CGR.isEmpty())
1646         CGR .recPermute(P);
1647     DATA.Select(P);
1648 };
1649
1650 template <class TQ, class TD> inline
1651 unsigned QSpace<TQ,TD>::getOM(
1652     wbMatrix<unsigned> &OM,
1653     wbvector<unsigned> *om1, wbMatrix<TQ> *Q_
1654 ) const {
1655
1656     unsigned om=0, m=(qtype.len ? qtype.len : QDIM);
1657     wbpem P; wbvector<wid_x_t> D;
1658     wbMatrix<TQ> Q(QIDX);
1659
1660     Q.groupRecs(P,D);
1661     OM.init2val(Q.dim1,m,1); if (Q_) Q.save2(*Q_);
1662
1663     if (!CGR.dim1) {
1664         if (Q.dim1!=QIDX.dim1) wblog(FL,"ERR %s() "
1665             "got non-compressed QSpace !? (%d/%d)",FCT,Q.dim1,QIDX.dim1);
1666         if (om1) om1->init2val(Q.dim1,1);
1667         return om;
1668     }
1669
1670     unsigned i,j,s,l,id,d;
1671
1672     if (CGR.dim2!=m) wblog(FL,"ERR %s() "
1673         "CGR/qtype inconsistency (%dx%d/%d)",FCT,CGR.dim1,CGR.dim2,m);
1674
1675     for (l=id=0; id<D.len; ++id, l+=d) { d=D[id]; i=P[l];
1676         for (s=0; s<m; ++s) { OM(id,s)=CGR(i,s).getOM(FL); }
1677
1678         for (j=1; j<d; ++j) { i=P[l+j]; ++om;
1679             for (s=0; s<m; ++s) { if (OM(id,s)!=CGR(i,s).getOM(FL)) {
1680                 wblog(FL,"ERR %s() got OM inconsistency (%d/%d)\n%s",
1681                     FCT, CGR(i,s).getOM(FL), OM(id,s), STR(CGR(i,s)));
1682             }}}
1683     }
1684
1685     wbvector<unsigned> x;
1686     wbMatrix<unsigned> OX(QIDX.dim1,OM.dim2);
1687
1688     OM.recProd(x); l=x.sum()-x.len;
1689
1690     if (int(l)<0 || l<om) wblog(FL,"ERR %s() got om=%d/%d",FCT,l,om);
1691     if (l!=om) wblog(FL,"TST %s() om=%d/%d (l=%d)",FCT,l,om,x.len);
1692
1693     om=l+x.len;

```

```

1694
1695     for (l=id=0; id<D.len; ++id, l+=d) {
1696         for (d=D[id], j=0; j<d; ++j) OX.recSetP(P[l+j],OM.rec(id)); }
1697     OX.save2(OM);
1698
1699     if (oml) OM.recProd(*oml);
1700     return om;
1701 };
1702
1703 template <class TQ, class TD> inline
1704 void QSpace<TQ,TD>::getDim(
1705     wvector<widxt_t> &D, wvector<widxt_t> *DD
1706 ) const {
1707
1708     unsigned k; int r=-1;
1709
1710     if (!isConsistent(r)) {
1711         wblog(FL,"ERR %s() %s",FCT,str);
1712     }
1713     if (r<0) wblog(FL,"ERR invalid rank r=%d !?",r);
1714
1715     if (otype==QS_OPERATOR) r=rank(FL);
1716
1717     D.init(r);
1718     if (DD) { DD->init(r);
1719         for (k=0; (int)k<r; ++k) D[k]=getDim(k,DD->data+k); }
1720     else { for (k=0; (int)k<r; ++k) D[k]=getDim(k); }
1721 };
1722
1723 template <class TQ, class TD> inline
1724 unsigned QSpace<TQ,TD>::getDim(unsigned k, widxt_t *D_) const {
1725
1726     widxt_t i,s, l=0, Dm=0, Ds=0;
1727     wvector<widxt_t> dd;
1728     wbMatrix<TQ> Qk;
1729     wperm pp;
1730
1731     char cgflag=gotCGS(FL);
1732
1733     if (QIDX.dim1==0) { if (D_) *D_=0; return 0; }
1734
1735     getQsub(k,Qk).SkipTiny_float().groupRecs(pp,dd);
1736
1737     if (cgflag==1 && qtype.permitsOM(>1) { cgflag=2; }
1738     if (cgflag>0) {
1739         unsigned j,ip,e=0,dc,r=rank(FL);
1740
1741         for (l=i=0; i<dd.len; ++i, l+=dd[i-1]) { ip=pp[l];
1742             const warray<TD> &a=*DATA[ip];
1743             if (a.SIZE.len!=r) wblog(FL,
1744                 "ERR %s() rank mismatch (%s; %d)",FCT,a.sizeStr().data,r);
1745             s=a.SIZE[k]; Dm+=s;
1746
1747             for (dc=1,j=0; j<CGR.dim2; ++j) {
1748                 dc*=CGR(ip,j).Size(k);
1749             }
1750
1751             if (cgflag<=1) {
1752                 unsigned d0=qtype.QDim(QIDX.ref(ip,QDIM*k)); if (d0!=dc)
1753                     wblog(FL,"ERR %s() cgsDim inconsistency! (%d/%d)",FCT,dc,d0);
1754             }
1755             Ds+=(s*dc);
1756         }
1757
1758         if (e) wblog(FL,
1759             "WRN %s() CGR data not yet initialized !? (%d)",FCT,e);
1760     }
1761     else {
1762         for (l=i=0; i<dd.len; ++i, l+=dd[i-1]) {
1763             Dm+=(s=DATA[pp[l]]->SIZE[k]);
1764             if (qtype.len) Ds += s * qtype.QDim(Qk.rec(i));
1765         }
1766     }
1767
1768     if (D_) { D_[0] = (Ds ? Ds : Dm); }
1769
1770     return Dm;
1771 };
1772
1773 template <class TQ, class TD> inline
1774 unsigned QSpace<TQ,TD>::cgsDim(unsigned i, unsigned k) const {
1775
1776     if (i>=QIDX.dim1) wblog(FL,
1777         "ERR %s() index out of bounds (%d/%d)",FCT,i,QIDX.dim1);
1778
1779     unsigned d=1, R=rank(FL);
1780     if (k>=R) wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,k,R);

```

```

1781     if (!CGR.dim2) return d;
1782
1783     if (CGR.dim1!=QIDX.dim1) wblog(FL,
1784         "ERR %s() size mismatch (%dx%d <> %dx%d)",
1785         FCT,QIDX.dim1,QIDX.dim2,CGR.dim1,CGR.dim2);
1786
1787     for (unsigned j=0; j<CGR.dim2; ++j) { d*=CGR(i,j).Size(k); }
1788
1789     return d;
1790 };
1791
1792 template <class TQ, class TD> inline
1793 unsigned QSpace<TQ,TD>::cgsDimScalar(unsigned i) const {
1794
1795     if (i>=QIDX.dim1) wblog(FL,
1796         "ERR %s() index out of bounds (%d/%d)",FCT,i,QIDX.dim1);
1797     unsigned d=1, r=rank(FL);
1798
1799     if (r!=2) wblog(FL,"ERR %s() invalid operator rank=%d",FCT,r);
1800     if (CGR.dim2) {
1801         if (CGR.dim1!=QIDX.dim1) wblog(FL,
1802             "ERR %s() size mismatch (%dx%d <> %dx%d)",
1803             FCT,QIDX.dim1,qtype.len,CGR.dim1,CGR.dim2);
1804     };
1805     for (unsigned j=0; j<CGR.dim2; ++j) {
1806         if (CGR(i,j).cgb) { d*=CGR(i,j).cgb->dim(); }
1807     }
1808 }
1809
1810 if (!d) wblog(FL,"ERR %s() got dim(cgd)=%d !?",FCT,d);
1811
1812 return d;
1813 };
1814
1815 template <class TQ, class TD>
1816 void getQDimGen(wbvector< const QSpace<TQ,TD>* > &A,
1817     wbMatrix<TQ> &Q, wbvector<widx_t> &S,
1818     wbindex I,
1819     wbMatrix<widx_t> *SC_=NULL
1820 ){
1821     if (!A.len) wblog(FL,"ERR got empty input space");
1822
1823     unsigned i,j,k,l,ic,d, d2=0, m,nq,p,il,i2, n=0; widx_t *ip;
1824     unsigned r=-1, rx, QDIM=A[0]->QDIM;
1825     const QS_TYPES &otype=A[0]->otype;
1826     const QVec &qtype=A[0]->qtype;
1827     wbMatrix<widx_t> S0,SC,II;
1828     wbvector<widx_t> D;
1829     wbpem P;
1830
1831     char cgflag=(SC_!=0);
1832
1833     nq=A[0]->CGR.dim2;
1834
1835     for (k=0; k<A.len; ++k) { const QSpace<TQ,TD> &Ak=(*A[k]);
1836         if (!A[k]) wblog(FL,"ERR %s() got null object",FCT);
1837         n+=Ak.QIDX.dim1;
1838
1839         if (I.len) { r=rx=Ak.rank(FL); }
1840         else { rx=r;
1841             if (int(r=Ak.isOperator(&rx,'x'))<=0) wblog(FL,
1842                 "ERR %s() invalid rank=%d operator\n(%d: %d/%d, '%s')",
1843                 FCT,Ak.itags.len,k+1,rx,r,A[0]->otype2Str().data);
1844             if (Ak.otype!=otype) wblog(FL,
1845                 "ERR %s() object type inconsistency (%s ; %s)",
1846                 FCT, Ak.otype2Str().data, A[0]->otype2Str().data);
1847         };
1848     }
1849
1850     if (!k) d2=r*QDIM;
1851
1852     if (Ak.qtype!=qtype) wblog(FL,
1853         "ERR %s() Q-type inconsistency (%s ; %s)",
1854         FCT, Ak.qStr().data, STR(qtype));
1855     if (Ak.QDIM!=QDIM || Ak.QIDX.dim2<d2) wblog(FL,
1856         "ERR %s() rank inconsistency (%d: %d/%d %d/%d)",
1857         FCT, k+1, Ak.QDIM, QDIM, Ak.QIDX.dim2, d2);
1858     if (Ak.QIDX.dim1!=Ak.DATA.len)
1859         wblog(FL,"ERR %s() size mismatch (%dx%d <> %d)",
1860             FCT, Ak.QIDX.dim1, Ak.QIDX.dim2, Ak.DATA.len);
1861 }
1862
1863
1864 if (Ak.gotCGS(FL)<=0) {
1865     if (k==0 && cgflag) cgflag=0;
1866     else if (cgflag) {
1867         wblog(FL,"ERR %s() CGR inconsistency (%d)",FCT,k+1);

```

```

1868     }
1869 }
1870 if (cgflag && k && Ak.CGR.dim2!=nq) wblog(FL,
1871 "ERR %s() CGR inconsistency (%d: %d/%d)",FCT,k+1,Ak.CGR.dim2,nq);
1872 }
1873
1874 if (!r) wblog(FL,"ERR got empty QSpaces !?");
1875
1876 if (!I.len) I.Index(r);
1877 else if (I.anyGE(r)) wblog(FL,
1878 "ERR %s() index out of bounds (%s; %d)",FCT,STR((I+1)),r);
1879
1880 S0.init(n,I.len);
1881 II.init(n,I.len*2);
1882 Q .init(n,I.len*QDIM);
1883
1884 wbvector<unsigned> rm;
1885 if (cgflag) {
1886     SC.init(n,nq*I.len).set(1);
1887     rm.init2val(nq,rx);
1888     for (j=0; j<nq; ++j) {
1889         if (qtype[j].permitsOM(rx)) ++rm[j];
1890     }
1891 }
1892
1893 for (l=k=0; k<A.len; ++k) {
1894     const QSpace<TQ,TD> &Ak=(*A[k]);
1895     const wbMatrix<TQ> &qq=Ak.QIDX; m=Ak.DATA.len;
1896
1897     for (i=0; i<m; ++i, ++l) {
1898         const wbvector<widx_t> &s=Ak.DATA[i]->SIZE;
1899         if (!Ak.DATA[i]->hasRank(rx))
1900             wblog(FL,"ERR %s() rank mismatch (data[%d]: %s (r=%d)",
1901                 FCT,i+1,SSTR_(Ak.DATA[i]), rx);
1902
1903         Q .recSetB(l,I,QDIM, qq.rec(i));
1904         S0.recSetB(l,I,l, s.data);
1905
1906         if (cgflag) {
1907             for (j=0; j<nq; ++j) {
1908                 wbvector<widx_t> s; Ak.CGR(i,j).getSize(s);
1909                 if (!s.len || s.allEqual(1)) continue;
1910                 if (s.len<rx || s.len>rm[j]) wblog(FL,
1911                     "ERR %s() rank mismatch (cgb[%d,%d]: [%s] (r=%d)",
1912                     FCT,i+1,j+1, Ak.CGR(i,j).sizeStr().data, r
1913                 );
1914                 for (ic=j, il=0; il<I.len; ++il, ic+=nq) {
1915                     SC(l,ic)=s[I.data[il]];
1916                 }
1917             }
1918         }
1919
1920         ip=II.rec(l);
1921         for (p=j=0; j<II.dim2; j+=2, ++p) { ip[j]=k; ip[j+1]=I[p]; }
1922     }
1923 }
1924
1925 Q .Reshape(n*I.len,QDIM);
1926 S0.Reshape(n*I.len,1);
1927 II.Reshape(n*I.len,2);
1928
1929 if (cgflag)
1930     SC.Reshape(n*I.len,nq);
1931
1932 Q.groupRecs(P,D); S.init(D.len);
1933 if (cgflag) SC_>init(D.len,nq);
1934
1935 for (l=i=0; i<D.len; ++i) { d=D[i]; il=P[l++];
1936     S[i]=S0(il,0);
1937     if (cgflag) SC_>recSetP(i,SC.ref(il));
1938     for (j=1; j<d; ++j, ++l) { i2=P[l];
1939         if (S0.recCompare(il,i2)) {
1940             sprintf(str,
1941                 "%s() data{} block size inconsistency (SC=%d)\n"
1942                 "A(%ld) @%ld <> A(%ld) @%ld: Q=[%s] D = %s / %s !?",FCT,cgflag,
1943                 II(il,0)+1, II(il,1)+1,
1944                 II(i2,0)+1, II(i2,1)+1, Q.rec2Str(i).data,
1945                 S0.rec2Str(il).data,
1946                 S0.rec2Str(i2).data
1947             );
1948             if (II(il,0)==II(i2,0)) wblog(FL,"ERR %s\n"
1949                 "(specify explicit leg for non-hermitian objects?)",str);
1950             else wblog(FL,"ERR %s",str);
1951         }
1952         if (cgflag && SC.recCompare(il,i2)) wblog(FL,
1953             "ERR %s() size inconsistency\nCGS: %d:%d <> %d:%d",
1954             FCT, II(il,0)+1, II(il,1)+1, II(i2,0)+1, II(i2,1)+1

```

```

1955     };
1956     }
1957 }
1958 };
1959
1960 template <class TQ, class TD> inline
1961 void getQDimGen(const wbvector< QSpace<TQ,TD> > &A,
1962     wbMatrix<TQ> &Q, wbvector<widxt_t> &S,
1963     const wbindex &I,
1964     wbMatrix<widxt_t> *SC=NULL
1965 ){
1966     wbvector< const QSpace<TQ,TD>* > Ap(A.len);
1967     for (unsigned i=0; i<A.len; ++i) Ap[i]=&A[i];
1968     getQDimGen(Ap,Q,S,I,SC);
1969 };
1970
1971 template <class TQ, class TD> inline
1972 void QSpace<TQ,TD>::getQDim(wbMatrix<TQ> &Q,
1973     wbvector<widxt_t> &S,
1974     wbMatrix<widxt_t>*SC
1975 ) const {
1976     wbindex I;
1977     wbvector< const QSpace* > A(1); A[0]=this;
1978
1979     getQDimGen(A,Q,S,I,SC);
1980 };
1981
1982
1983 template <class TQ, class TD> inline
1984 void QSpace<TQ,TD>::getQDim(unsigned k,
1985     wbMatrix<TQ> &Q,
1986     wbvector<widxt_t> &SD,
1987     wbMatrix<widxt_t> *SC
1988 ) const {
1989     unsigned i,j,q, r=rank(FL),n=DATA.len;
1990     wbvector<widxt_t> D;
1991     wbindex Ig;
1992     wbpem P;
1993
1994     if (k>=r) wblog(FL,"ERR %s() dim out of bounds (%d/%d)",FCT,k,r);
1995     if (QIDX.dim1!=n) wblog(FL,
1996         "ERR %s() QSpace inconsistency (%d/%d)",FCT,QIDX.dim1,n);
1997
1998     getQsub(k,Q).groupRecs(P,D,-1,1,&Ig);
1999     SD.init(D.len).set(-1);
2000
2001     for (i=0; i<n; ++i) {
2002         const wbvector<widxt_t> &si=DATA[i]->SIZE;
2003         widxt_t &s=SD.el(Ig[i]);
2004         if (si.len>r || k>=si.len) wblog(FL,
2005             "ERR %s() index out of bounds (%d/%d,%d)",FCT,k+1,si.len,r);
2006         if (int(s)>=0) {
2007             if (s!=si.data[k]) wblog(FL,
2008                 "ERR %s() size mismatch DATA(%d): %s [S(%d)=%d]",
2009                 FCT,i+1,DATA[i]->sizeStr().data, k+1, s
2010             );
2011         }
2012         else s=si.data[k];
2013     }
2014 }
2015
2016 if (SC==NULL) return;
2017 if (gotCGS(FL)<=0) { SC->init(); return; }
2018
2019 SC->init(D.len,CGR.dim2).set(-1);
2020
2021 for (i=0; i<n; ++i) {
2022     for (j=0; j<CGR.dim2; ++j) {
2023         widxt_t &c=SC->el(Ig[i],j);
2024         q=CGR(i,j).qdim(k,&qtype[j]);
2025
2026         if (int(c)>=0) {
2027             if (c!=q) wblog(FL,
2028                 "ERR %s() size mismatch CGR(%d,%d): %s [S(%d)=%d]",
2029                 FCT,i+1,j+1, CGR(i,j).sizeStr().data, k+1, q
2030             );
2031             else c=q;
2032         }
2033     }
2034 };
2035
2036 template <class TQ, class TD> inline
2037 int QSpace<TQ,TD>::isSingleton(unsigned k) const {
2038     unsigned i;
2039
2040     if (k>=rank(FL)) wblog(FL,
2041         "ERR %s() index out of bounds (%d,%d)",FCT,k,rank(FL));

```

```

2042
2043     for (i=0; i<DATA.len; ++i)
2044     if (DATA[i]->SIZE[k]!=1) break;
2045
2046     return (i==DATA.len);
2047 }
2048
2049 template <class TQ, class TD> inline
2050 void QSpace<TQ,TD>::getDRange(TD &dmin, TD &dmax) const {
2051     unsigned i,n=DATA.len;
2052
2053     if (!isConsistent()) { info("this"); werror(FL,str); }
2054
2055     for (i=0; i<n; ++i) if (DATA[i] && !DATA[i]->isEmpty()) {
2056         dmin=dmax=DATA[i]->data[0];
2057         break;
2058     }
2059
2060     for (i=0; i<n; ++i) if (DATA[i] && !DATA[i]->isEmpty()) {
2061         dmin=MIN(dmin, DATA[i]->min());
2062         dmax=MAX(dmax, DATA[i]->max());
2063     }
2064 }
2065
2066 template <class TQ, class TD> inline
2067 QSpace<TQ,TD>& QSpace<TQ,TD>::getSub(
2068     const windex &I, QSpace<TQ,TD> &A, char ref
2069 ) const {
2070
2071     if ((void*)this==(void*)&A) {
2072         QSpace<TQ,TD> X; A.save2(X);
2073         if (ref) wblog(FL,"WRN %s() got ref on cyclic reference",FCT);
2074         X.getSub(I,A);
2075         return A;
2076     }
2077
2078     unsigned i,j;
2079     char gotcg=gotCGS(FL);
2080
2081     A.init();
2082     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype; A.itags=itags;
2083
2084     A.isref=(ref ? 1 : 0);
2085
2086     A.setupDATA();
2087
2088     if (gotcg>0) {
2089         if (CGR.dim1!=DATA.len) wblog(FL,
2090             "ERR %s() size mismatch (CGR: %d/%dx%d)",
2091             FCT,DATA.len,CGR.dim1,CGR.dim2);
2092         A.setupCGR();
2093     }
2094
2095     for (i=0; i<I.len; ++i) {
2096         if (I[i]>=DATA.len) wblog(FL,
2097             "ERR %s() index out of bounds (%d,%d)",FCT,I[i],DATA.len);
2098
2099         if (A.isref)
2100             A.DATA[i]=DATA[I[i]];
2101         else A.DATA[i]->init(*DATA[I[i]]);
2102
2103         if (gotcg>0) {
2104             for (j=0; j<CGR.dim2; ++j) {
2105                 A.CGR(i,j)=CGR(I[i],j);
2106             }
2107         }
2108     }
2109
2110     return A;
2111 };
2112
2113 template <class TQ, class TD> inline
2114 QSpace<TQ,TD>& QSpace<TQ,TD>::getSubInit(
2115     const windex &I,
2116     QSpace<TQ,TD> &A
2117 ) const {
2118
2119     if (this==&A) {
2120         QSpace<TQ,TD> X; A.save2(X);
2121         return X.getSubInit(I,A);
2122     }
2123
2124     unsigned i,j;
2125     char cgflag=gotCGS(FL);
2126
2127     A.init();
2128     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype;

```

```

2129     A.setupDATA();
2130
2131     if (cgflag) { A.setupCGR();
2132
2133         for (i=0; i<I.len; ++i) {
2134             if (I[i]>=QIDX.dim1) wblog(FL,
2135                 "ERR %s() index out of bounds (%d,%d)",FCT,I[i]+1,QIDX.dim1);
2136             for (j=0; j<CGR.dim2; ++j) A.CGR(i,j).init(CGR(I[i],j));
2137         }
2138     }
2139
2140     return A;
2141 };
2142
2143 template <class TQ, class TD> inline
2144 void QSpace<TQ,TD>::saveSub2(QSpace<TQ,TD> &A, const windex &I) {
2145
2146     if (this==&A) wblog(FL,
2147         "ERR %s() got overlapping input and output space!",FCT);
2148     if (isref) wblog(FL,"ERR %s() called for ISREF!",FCT);
2149
2150     if (!I.len) { A.init(); return; }
2151
2152     unsigned i,j;
2153     char cgflag=gotCGS(FL);
2154
2155     A.init();
2156     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype;
2157     A.setupDATA();
2158     A.setupCGR();
2159
2160     for (i=0; i<I.len; ++i) {
2161         if (I[i]>=DATA.len) wblog(FL,
2162             "ERR %s() index out of bounds (%d,%d)",FCT,I[i],DATA.len);
2163
2164         DATA[I[i]]->save2(*A.DATA[i]);
2165         WB_DELETE_1(DATA[I[i]]);
2166
2167         if (cgflag) {
2168             for (j=0; j<CGR.dim2; ++j) {
2169                 CGR(I[i],j).save2(A.CGR(i,j)); }
2170         }
2171     }
2172
2173     windex I2; I.invert(QIDX.dim1,I2);
2174
2175     QIDX.Set2Recs(I2);
2176     DATA.Select(I2); if (cgflag) { CGR.Set2Recs(I2); }
2177 };
2178
2179 template <class TQ, class TD>
2180 void QSpace<TQ,TD>::toFull(
2181     warray<TD> &A,
2182     wvector< wMatrix<TQ> > &QB,
2183     wvector< wvector<unsigned> > &SB
2184 ) const {
2185
2186     if (gotCGS(FL)>0 || gotCGX()) wblog(FL,
2187         "ERR %s() not implemented yet for %d %d",FCT,
2188         gotCGS(FL), gotCGX(FL)
2189     );
2190
2191     widx_t i,j,k,r,M=QIDX.dim1; int ir=-1;
2192     wvector<widx_t> N,SA,D;
2193     warray<unsigned> mark;
2194     wMatrix<widx_t> G;
2195     windex I,J;
2196     wperm P,iP;
2197
2198     if (!isConsistent(ir)) wblog(FL,"%s",str);
2199     if (isEmpty()) {
2200         A.init(); QB.init(); SB.init();
2201         return;
2202     }
2203
2204     r=(unsigned)ir;
2205
2206     QB.initDef(r); SB.initDef(r); D.init(r);
2207     I.init(r); P.init(r); iP.init(r); G.init(M,r); N.init(r);
2208
2209     for (k=0; k<r; ++k) {
2210         getQsub(k,QB[k]).groupRecs(P,D); P.invert(iP);
2211
2212         I.BlockIndex(D).Permute(iP);
2213         G.setCol(k,I.data);
2214
2215         N[k]=D.len; SB[k].init(N[k]);

```

```

2216     }
2217
2218     mark.init(N+1);
2219
2220     for (i=0; i<M; ++i) {
2221         G.getRec(i,I);
2222
2223         if (mark(I++) wblog(FL,
2224             "ERR %s() block (%s) not unique (%d)",
2225             FCT, (I+1).toStrf(" ", " ").data, mark(I)
2226         );
2227
2228         J.wbvector<widx_t>::operator=(N);
2229
2230         for (k=0; k<r; ++k) {
2231             wbvector<unsigned> &SBk=SB[k];
2232
2233             J[k]=j=I[k]; if (k) J[k-1]=N[k-1];
2234             if (!mark(J)) {
2235                 SBk[j]=DATA[i]->SIZE[k];
2236             }
2237             else if (SBk[j]!=DATA[i]->SIZE[k]) wblog(FL,
2238                 "ERR %s - block size mismatch (%d: %d,%d)",
2239                 FCT,i+1, SBk[j], DATA[i]->SIZE[k]
2240             );
2241         }
2242     }
2243
2244     SA.init(r); for (k=0; k<r; ++k) SA[k]=SB[k].sum();
2245     A.init(SA);
2246
2247     for (i=0; i<DATA.len; ++i) {
2248         G.getRec(i,I);
2249         A.setBlock(SB, I, *DATA[i]);
2250     }
2251 }
2252
2253 template <class TQ, class TD>
2254 void QSpace<TQ,TD>::toFull2(
2255     wbararray<TD> &A,
2256     wbMatrix<TQ> &QB,
2257     wbMatrix<widx_t> &SB
2258 ) const {
2259
2260     if (gotCGS(FL)>0 || gotCGX()) wblog(FL,
2261         "ERR %s() not implemented yet for %d %d",FCT,
2262         gotCGS(FL), gotCGX(FL)
2263     );
2264
2265     unsigned i,j,k,r,K,N,M=QIDX.dim1; int r_=-1;
2266     wbMatrix<TQ> Q1,Q2;
2267     wbvector< wbvector<widx_t> > S2;
2268     wbvector<widx_t> s,D,SA;
2269     wbMatrix<unsigned> mark; wbMatrix<widx_t> IJ;
2270     wbindex I,I1,I2;
2271     wbararray<TD> a;
2272     wbperm P,iP;
2273
2274     if (!isConsistent(r_)) wblog(FL,"%s",str);
2275     if (isEmpty()) {
2276         A.init(); QB.init(); SB.init();
2277         return;
2278     }
2279
2280     if (r_==3 && getDim(2)==1) { --r_; }
2281
2282     if (r_%2) wblog(FL,"ERR %s even-rank object required (r=%d)", FCT,r_);
2283     r=unsigned(r_);
2284     K=r/2;
2285
2286     I.Index(0,K-1); getQsub(I,Q1);
2287     I.Index(K,r-1); getQsub(I,Q2);
2288
2289     QB.Cat(1,Q1,Q2);
2290
2291     QB.groupRecs(P,D); P.invert(iP);
2292
2293     I.BlockIndex(D); I.Select(iP);
2294
2295     I1.init(M, I.data);
2296     I2.init(M, I.data+M);
2297
2298     N=D.len;
2299     mark.init(N,N+1);
2300     SB.init(N,K);
2301     IJ.init(M,2);
2302

```



```

2303     for (k=0; k<M; ++k) {
2304         i=IJ(k,0)=I1[k]; j=IJ(k,1)=I2[k];
2305
2306         if (mark(i,j)++) wblog(FL,
2307             "WRN %s - block (%d,%d) not unique (%d)", FCT, i, j, mark(i,j));
2308
2309         s.init2ref(K, DATA[k]->SIZE.data);
2310         if (mark(i,N)++) {
2311             if (!SB.recEqual(i,s.data)) wblog(FL,
2312                 "ERR %s - block size mismatch (%d: %s != %s)", FCT, k+1,
2313                 SB.rec2Str(i, "", "x").data, s.toStrD().data);
2314         }
2315         else SB.recSet(i,s);
2316
2317         s.init2ref(K, DATA[k]->SIZE.data+K);
2318         if (mark(j,N)++) {
2319             if (!SB.recEqual(j,s.data)) wblog(FL,
2320                 "ERR %s - block size mismatch (%d: %s != %s)", FCT, k+1,
2321                 SB.rec2Str(j, "", "x").data, s.toStrD().data);
2322         }
2323         else SB.recSet(j,s);
2324     }
2325
2326     S2.init(2); SB.recProd(S2[0]); S2[1]=S2[0];
2327     SA.init(2); SA[0]=SA[1]=S2[0].sum();
2328
2329     A.init(SA);
2330
2331     for (i=0; i<DATA.len; ++i) {
2332         I.init2ref(2,IJ.rec(i));
2333
2334         a.init2ref(*DATA[i]);
2335         a.GroupIndizes(K);
2336
2337         A.setBlock(S2,I,a);
2338     }
2339 }
2340
2341 template <class TQ, class TD>
2342 void QSpace<TQ,TD>::ExtendQ(const QSpace<TQ,TD> &B) {
2343     unsigned i,k,m,n=B.QIDX.dim1;
2344     wbvector<char> mark;
2345     wbindex Ia,Ib;
2346
2347     if (B.isEmpty()) return;
2348     if (isEmpty()) {
2349         (*this)=B;
2350         for (i=0; i<DATA.len; ++i) DATA[i]->set(TD(0));
2351         return;
2352     }
2353
2354     if (QIDX.dim2!=B.QIDX.dim2) wblog(FL,
2355         "ERR %s - size mismatch (%d/%d)", FCT, QIDX.dim2, B.QIDX.dim2);
2356
2357     matchIndex(QIDX,B.QIDX,Ia,Ib);
2358
2359     mark.init(n);
2360     for (i=0; i<Ib.len; ++i) mark[Ib[i]]++;
2361
2362     m=mark.count(0);
2363     if (m==0) return;
2364
2365     k=QIDX.dim1;
2366     Append(m);
2367
2368     for (i=0; i<n; ++i) {
2369         if (mark[i]) continue;
2370         QIDX.recSetP(k,B.QIDX.rec(i));
2371         DATA[k]->init(B.DATA[i]->SIZE);
2372         ++k;
2373     }
2374 }
2375 };
2376
2377 template <class TQ, class TD>
2378 void QSpace<TQ,TD>::takeMainQS(char disp) {
2379     unsigned i,j,n,i0,imax=0;
2380     wbvector<widx_t> d, dc, I;
2381     wbperm P;
2382
2383     wbMatrix<TQ> QT;
2384     wbvector<TD> Z;
2385     TD z, zmax=0, ztot=0, eps=1E-3;
2386
2387     getQtot(QT);
2388     QT.groupRecs(P,d);

```

```

2390
2391     d.cumsum_(dc); Z.init(d.len);
2392
2393     for (i=0; i<d.len; ++i) {
2394         for (z=0, n=d[i], i0=dc[i], j=0; j<n; ++j)
2395             z+=DATA[P[i0+j]]->norm2();
2396
2397         Z[i]=z;
2398
2399         if (z>zmax) { zmax=z; imax=i; }
2400         ztot+=z;
2401     }
2402
2403     if (disp) { char s_[128];
2404         z=fabs((ztot-zmax)/ztot);
2405         snprintf(s_,128,"%s skipping weight %s of total %s (%s%s)\n"
2406             "    Q=[%s]", (ABS(z)>eps ? "WRN" : " * "),
2407             NSTR(ztot-zmax), NSTR(ztot), NSTR(z*100.),
2408             ABS(z)>0.1 ? "% !?" : "%", QT.rec2Str(imax).data
2409         );
2410
2411         wblog(FI,s_);
2412     }
2413
2414     I.init(d[imax], P.data+dc[imax]);
2415
2416     QIDX.Set2Recs(I);
2417     DATA.Select(I);
2418 }
2419
2420 template <class TQ, class TD> inline
2421 wbMatrix<TQ>& QSpace<TQ,TD>::getQsub(
2422     const unsigned k, wbMatrix<TQ> &Qk, char iflag
2423 ) const {
2424
2425     unsigned i, r=rank(FI), s=QDIM*sizeof(TQ), offset=k*QDIM;
2426
2427     if (iflag) {
2428         wbindex I; I.Index_ex(r,k); getQsub(I,Qk);
2429         return Qk;
2430     }
2431
2432     if (k>r) wblog(FI,
2433         "ERR %s() index out of bounds (%dx%d/%d; %d)",
2434         FCT, QIDX.dim1, QIDX.dim2, QDIM, k);
2435
2436     Qk.init(QIDX.dim1, QDIM);
2437
2438     for (i=0; i<QIDX.dim1; ++i)
2439         memcpy(Qk.rec(i), QIDX.rec(i)+offset, s);
2440
2441     return Qk;
2442 };
2443
2444 template <class TQ, class TD> inline
2445 wbMatrix<TQ>& QSpace<TQ,TD>::getQsub(
2446     const wbindex &I,
2447     wbMatrix<TQ> &QI
2448 ) const {
2449     unsigned i,k, r=rank(FI);
2450     TQ *q;
2451
2452     if (&QI==&QIDX) wblog(FI,
2453         "ERR %s() QIN and QOUT space are the same!",FCT);
2454     for (i=0; i<I.len; ++i) if (I[i]>r) wblog(FI,
2455         "ERR %s() index out of bounds (%d/%d)",FCT,I[i],r);
2456
2457     QI.init(QIDX.dim1, I.len*QDIM);
2458
2459     for (i=0; i<QIDX.dim1; ++i) {
2460         const TQ *q0=QIDX.rec(i); q=QI.rec(i);
2461
2462         for (k=0; k<I.len; ++k, q+=QDIM) {
2463             Wb::cpyRange(q, q0+I[k]*QDIM, QDIM);
2464         }
2465     }
2466
2467     return QI;
2468 }
2469
2470 template <class TQ, class TD> inline
2471 void QSpace<TQ,TD>::getQsub(
2472     const wbindex &I,
2473     wbMatrix<TQ> &QI,
2474     wbMatrix<TQ> &Q2
2475 ) const {
2476

```

```

2477     unsigned i, k, r=rank(FL), s=QDIM*sizeof(TQ);
2478     wbindex I2;
2479     TQ *q1,*q2; const TQ *q0;
2480
2481     I.invert(r,I2);
2482
2483     if (&QI==&QIDX || &Q2==&QIDX)
2484         wblog(FL,"ERR getQsub requires distinct QOUT space!");
2485
2486     QI.init(QIDX.dim1, I .len*QDIM);
2487     Q2.init(QIDX.dim1, I2.len*QDIM);
2488
2489     for (i=0; i<QIDX.dim1; ++i) {
2490         q0=QIDX.rec(i); q1=QI.rec(i); q2=Q2.rec(i);
2491
2492         for (k=0; k<I .len; ++k) memcpy(q1+k*QDIM, q0+I [k]*QDIM, s);
2493         for (k=0; k<I2.len; ++k) memcpy(q2+k*QDIM, q0+I2[k]*QDIM, s);
2494     }
2495 }
2496
2497 template <class TQ, class TD> inline
2498 wbMatrix<TQ>& QSpace<TQ,TD>::getQtot(
2499     wbMatrix<TQ> &Qtot,
2500     char uflag
2501 ) const {
2502
2503     unsigned i,k,l,r, r0=rank(FL);
2504     TQ *q; const TQ *q0;
2505
2506     if (&Qtot==&QIDX)
2507         wblog(FL,"ERR getQtot requires distinct QOUT space!");
2508
2509     Qtot.init(QIDX.dim1, QDIM);
2510
2511     for (i=0; i<QIDX.dim1; ++i) {
2512         q0=QIDX.rec(i); q=Qtot.rec(i);
2513         for (l=r=0; r<r0; ++r)
2514             for (k=0; k<QDIM; ++k, ++l) q[k]=q0[l];
2515     }
2516
2517     if (uflag) Qtot.makeUnique();
2518
2519     return Qtot;
2520 }
2521
2522 template <class TQ, class TD> inline
2523 void QSpace<TQ,TD>::getDistQtot(
2524     wbMatrix<TQ> &Q, wbvector<double> &w
2525 ) const {
2526     unsigned i,j,l,d;
2527     wbperm P; wbvector<widx_t> D;
2528     double n2;
2529
2530     QIDX.blockSum(QDIM,Q); Q.groupRecs(P,D);
2531     w.init(D.len);
2532
2533     for (l=i=0; i<D.len; ++i, l+=d) { d=D[i];
2534         for (n2=0, j=0; j<d; ++j) n2 += (DATA[P[l+j]]->norm2());
2535         w[i]=sqrt((double)n2);
2536     }
2537 }
2538
2539 template <class TQ, class TD> inline
2540 wbMatrix<TQ>& QSpace<TQ,TD>::getQsum(
2541     const wbindex &J, wbMatrix<TQ> &Q
2542 ) const {
2543
2544     unsigned i,j, r=rank(FL);
2545     const TQ *q0; TQ *q;
2546
2547     if (J.isEmpty()) { Q.init(); return Q; }
2548
2549     for (i=0; i<J.len; ++i) if (J[i]>=r) wblog(FL,
2550         "ERR block index out of range (%d/%d)",J[i],r);
2551
2552     Q.init(QIDX.dim1,QDIM);
2553
2554     if (allAdditive() || J.len==1) {
2555         for (i=0; i<QIDX.dim1; ++i) {
2556             q=Q.rec(i); q0=QIDX.rec(i);
2557             for (j=0; j<J.len; ++j) Wb::addRange(q0+J[j]*QDIM, q, QDIM);
2558         }
2559     }
2560     else if (allAbelian()) {
2561         if (qtype.len!=QDIM) wblog(FL,
2562             "ERR %s() got inconsistent qtype (len=%d/%d; %s)",
2563             FCT,qtype.len,QDIM,qStr().data

```

```

2564     );
2565
2566     for (i=0; i<QIDX.dim1; ++i) { q=Q.rec(i); q0=QIDX.rec(i);
2567         for (j=0; j<QDIM; ++j) {
2568             q[j]=get_qtot_abelian(qtype[j],q0+j,J.len,J.data,QDIM);
2569         }
2570     }
2571 }
2572 else wblog(FI,
2573 "ERR %s() got non-abelian symmetries (%s)",FCT,qStr().data);
2574
2575 return Q;
2576 };
2577
2578 template <class TQ, class TD> inline
2579 wbMatrix<TQ>& QSpace<TQ,TD>::getQsum(
2580     const wbMatrix<TQ> &Q0, wbMatrix<TQ> &Q
2581 ) const {
2582
2583     if (!Q0) { Q.init(); return Q; }
2584     if (Q0.dim2==QDIM) { Q=Q0; return Q; }
2585
2586     if (!QDIM || (Q0.dim2 % QDIM)) wblog(FI,
2587         "ERR size mismatch Q0 (%dx%d /%d)",Q0.dim1,Q0.dim2,QDIM);
2588
2589     unsigned i,j, m=Q0.dim2/QDIM;
2590     const TQ *q0; TQ *q;
2591
2592     Q.init(Q0.dim1,QDIM);
2593
2594     if (allAdditive()) {
2595         for (i=0; i<Q0.dim1; ++i) { q=Q.rec(i); q0=Q0.rec(i);
2596             for (j=0; j<m; ++j) Wb::addRange(q0+j*QDIM, q, QDIM);
2597         }
2598     }
2599     else if (allAbelian()) {
2600         if (qtype.len!=QDIM) wblog(FI,
2601             "ERR %s() got inconsistent qtype (len=%d/%d; %s)",
2602             FCT,qtype.len,QDIM,qStr().data
2603         );
2604
2605         for (i=0; i<Q0.dim1; ++i) { q=Q.rec(i); q0=Q0.rec(i);
2606             for (j=0; j<QDIM; ++j) {
2607                 q[j]=get_qtot_abelian(qtype[j],q0+j,m,NULL,QDIM);
2608             }
2609         }
2610     }
2611     else wblog(FI,"ERR %s() "
2612         "got non-abelian symmetries (%s, m=%d)",FCT,qStr().data,m);
2613
2614     return Q;
2615 };
2616
2617 template <class TQ, class TD> inline
2618 void QSpace<TQ,TD>::FlipQ() {
2619     wbperm P(rank(FI)); P.Flip();
2620     PermuteQ(P);
2621 }
2622
2623 template <class TQ, class TD> inline
2624 void QSpace<TQ,TD>::PermuteQ(const wbperm &P) {
2625
2626     unsigned i,k,r=rank(FI), s=QDIM*sizeof(TQ);
2627     wbMatrix<TQ> Q0(QIDX);
2628     TQ *q0, *qp;
2629
2630     if (!QDIM) wblog(FI,"WRN QDIM=0!");
2631     if (P.isEmpty()) return;
2632     if (isEmpty() || QIDX.dim2==0) return;
2633
2634     if (!P.isValidPerm(r)) wblog(FI,
2635         "ERR %s() invalid permutation [%s] (%d)",FCT,STR(P),r);
2636
2637     for (i=0; i<QIDX.dim1; ++i) {
2638         q0=Q0.rec(i); qp=QIDX.rec(i);
2639         for (k=0; k<P.len; ++k) memcpy(qp+k*QDIM, q0+P[k]*QDIM, s);
2640     }
2641 }
2642
2643 template <class TQ, class TD> inline
2644 void QSpace<TQ,TD>::permuteQ(const wbperm &P, wbMatrix<TQ> &Q) const {
2645
2646     if (&Q==&QIDX) {
2647         wblog(FI,"ERR You shall not permuteQ onto itself");
2648     }
2649     else {
2650         unsigned i, k, r=rank(FI), s=QDIM*sizeof(TQ);

```

```

2651     const TQ *q0; TQ *qp;
2652
2653     if (P.isEmpty()) { Q=QIDX; return; }
2654
2655     if (!P.isValidPerm(r))
2656         wblog(FL,"ERR invalid permutation [%s].", P.toStrD().data);
2657
2658     Q.init(QIDX.dim1, QIDX.dim2);
2659
2660     for (i=0; i<QIDX.dim1; ++i) {
2661         q0=QIDX.rec(i); qp=Q.rec(i);
2662         for (k=0; k<P.len; ++k) memcpy(qp+k*QDIM, q0+P[k]*QDIM, s);
2663     }
2664 }
2665 }
2666
2667 template <class TQ, class TD>
2668 QSpace<TQ,TD>& QSpace<TQ,TD>::Permute(
2669     const wbperm &P_, char iflag
2670 ){
2671     unsigned i,n,r=rank(FL);
2672     if ((!r && isEmpty()) || !P_.len || P_.isIdentityPerm(FL)) {
2673         return *this;
2674     }
2675     if (P_.len>r) wblog(FL,
2676         "ERR %s() size mismatch (%d/%d)",FCT,P_.len,r);
2677
2678     wbperm P(P_,iflag,r);
2679     PermuteQ(P);
2680
2681     for (i=0; i<DATA.len; ++i) DATA[i]->Permute(P);
2682     if (itags.len) itags.Select(P);
2683
2684     for (n=CGR.numel(), i=0; i<n; ++i) {
2685         CGR[i].Permute(P);
2686     }
2687
2688     return *this;
2689 };
2690
2691 template <class TQ, class TD> inline
2692 QSpace<TQ,TD>& QSpace<TQ,TD>::permute(
2693     const wbperm &P_, QSpace<TQ,TD> &B, char iflag
2694 ) const {
2695
2696     unsigned r=rank(FL);
2697     if (P_.isEmpty() || P_.isIdentityPerm(FL,r)) {
2698         B.init(*this); return B;
2699     }
2700
2701     unsigned i,n;
2702     wbperm P(P_,iflag); B.init();
2703
2704     permuteQ(P,B.QIDX);
2705     B.QDIM=QDIM; B.qtype=qtype; B.itags=itags;
2706
2707     if (otype && P.isOpTranspose()) {
2708         B.otype=otype;
2709     }
2710
2711     if (B.itags.len) B.itags.Select(P);
2712
2713     B.setupDATA();
2714     for (n=DATA.len, i=0; i<n; ++i) {
2715         DATA[i]->permute(*B.DATA[i],P);
2716     }
2717
2718     if (CGR.isEmpty()) { B.CGR.init(); }
2719     else {
2720         B.setupCGR(); n=CGR.numel();
2721         if (n!=B.CGR.numel()) wblog(FL,"ERR %s() got CGR size mismatch "
2722             "(%s <> %s)",FCT,CGR.sizeStr().data,B.CGR.sizeStr().data);
2723         for (i=0; i<n; ++i) {
2724             CGR[i].permute(P,B.CGR[i]);
2725         }
2726     }
2727
2728     return B;
2729 };
2730
2731 template <class TQ, class TD> inline
2732 void QSpace<TQ,TD>::setQ(unsigned k,
2733     const QSpace<TQ,TD> &B, unsigned k0
2734 ) {
2735
2736     unsigned i, s=QDIM*sizeof(TQ), dd0=B.QIDX.dim2, dd2=QIDX.dim2;
2737     TQ *d0 = B.QIDX.data+k0*QDIM, *d2 = QIDX.data+k*QDIM;

```

```

2738
2739     if (QIDX.dim1!=B.QIDX.dim1 || QDIM!=B.QDIM) wblog(FL,
2740         "ERR %s() incompatible objects (%d,%d; %d,%d)",
2741         FCT, QIDX.dim1, B.QIDX.dim1, QDIM, B.QDIM);
2742
2743     if (rank(FL)<=k || B.rank(FL)<=k0) wblog(FL,
2744         "ERR %s() index out of bounds (%d/%d; %d/%d)",
2745         FCT, k, rank(FL), k0, B.rank(FL));
2746
2747     for (i=0; i<QIDX.dim1; ++i, d0+=dd0, d2+=dd2)
2748         memcpy(d2,d0,s);
2749 }
2750
2751 template <class TQ, class TD> inline
2752 void QSpace<TQ,TD>::setQRec(
2753     unsigned r, unsigned j, const TQ *q0, const TQ *q2
2754 ){
2755     if (r==QIDX.dim1 || QDIM*(j+1)>QIDX.dim2) wblog(FL,
2756         "ERR %s() index out of bounds (%d,%d; %d,%d)",
2757         FCT, r, j, QIDX.dim1, QDIM);
2758
2759     TQ *q = QIDX.rec(r)+j*QDIM;
2760     if (q2==NULL)
2761         for (unsigned i=0; i<QDIM; ++i) q[i]=q0[i];
2762     else for (unsigned i=0; i<QDIM; ++i) q[i]=q0[i]+q2[i];
2763 }
2764
2765 template <class TQ, class TD> inline
2766 bool QSpace<TQ,TD>::hasEqualQ(unsigned i1, unsigned i2) const {
2767
2768     unsigned i, s=QDIM*sizeof(TQ), D=QIDX.dim2, r=rank(FL);
2769     TQ *d1 = QIDX.data+i1*QDIM, *d2 = QIDX.data+i2*QDIM;
2770
2771     if (i1>=r || i2>=r) wblog(FL,
2772         "ERR %s() index out of bounds (%d,%d; %d)",FCT,i1,i2,r);
2773
2774     for (i=0; i<QIDX.dim1; ++i, d1+=D, d2+=D)
2775         if (memcmp(d1,d2,s)) return 0;
2776
2777     return 1;
2778 };
2779
2780 template <class TQ, class TD>
2781 bool QSpace<TQ,TD>::hasQOverlap(const QSpace<TQ,TD> &A) const {
2782
2783     wbindex Ia, Ib;
2784     if (qtype!=A.qtype) {
2785         wblog(FL,"WRN got objects with different qtype");
2786         return 0;
2787     }
2788
2789     matchIndex(QIDX, A.QIDX, Ia, Ib);
2790     return (Ia.len!=0);
2791 };
2792
2793 template <class TQ, class TD>
2794 template <class TB>
2795 int QSpace<TQ,TD>::hasQOverlap(
2796     unsigned ica, const QSpace<TQ,TB> &B, unsigned icb,
2797     const char type
2798 ) const {
2799
2800     unsigned ra=rank(FL), rb=B.rank(FL); int q=0;
2801     wbMatrix<TQ> QA,QB;
2802     wbindex Ia,Ib,Ja,Jb;
2803
2804     if (!ica || !icb) wblog(FL,
2805         "ERR %s() index must be 1-based (%d,%d)",FCT,ica,icb);
2806     --ica; --icb;
2807
2808     if (QDIM!=B.QDIM) wblog(FL,
2809         "ERR %s() incompatible objects\nQDIM=%d,%d",
2810         FCT, QDIM, B.QDIM);
2811     if (ica>=ra || icb>=rb) wblog(FL,
2812         "ERR %s() index out of bounds\n%d/%d, %d/%d (QDIM=%d,%d)",
2813         FCT, ica,ra, icb,rb, QDIM, B.QDIM);
2814
2815     getQsub(ica,QA).makeUnique(Ja);
2816     B.getQsub(icb,QB).makeUnique(Jb); matchIndex(QA,QB,Ia,Ib);
2817
2818     if (Ia.len==QA.dim1) { q|=2; }
2819     if (Ib.len==QB.dim1) { q|=4; }
2820
2821     if (type=='<') { if (!(q&2)) { q=0; }} else
2822     if (type=='>') { if (!(q&4)) { q=0; }} else
2823     if (type=='=' ) { if ( q!=6) { q=0; }} else {
2824         if (type) wblog(FL,"WRN %s() invalid type=%c<%d>",FCT,type,type);

```

```

2825     if (!Ia.len) { q=0; }
2826 }
2827
2828 if (q && type) {
2829     for (unsigned i=0; i<Ia.len; ++i) {
2830         if (!DATA[Ja[Ia[i]]->hasSameSize(ica, *B.DATA[Jb[Ib[i]]],icb)) {
2831             q=-q; break;
2832         }
2833     }
2834 }
2835
2836 return q;
2837 };
2838
2839 template <class TQ, class TD>
2840 mxArray* QSpace<TQ,TD>::QIDX_toMx() const {
2841     unsigned k, r;
2842     mxArray *a;
2843
2844     if (QIDX.isEmpty())
2845         return mxCreateCellMatrix(0,0);
2846
2847     if (!QDIM || QIDX.dim2>QDIM) wblog(FL,
2848         "ERR %s() invalid QSpace (QIDX: %dx%d; %d)",
2849         FCT,QIDX.dim1,QIDX.dim2,QDIM
2850     );
2851
2852     r=QIDX.dim2/QDIM; a=mxCreateCellMatrix(1,r);
2853     if (!a) wblog(FL,"ERR could not allocate cell array (%d)", r);
2854
2855     for (k=0; k<r; ++k)
2856         mxSetCell(a, k, getQsub(k).toMx());
2857
2858     return a;
2859 }
2860
2861 template <class TQ, class TD>
2862 mxArray* QSpace<TQ,TD>::INFO_toMx() const {
2863     if ((qtype.isEmpty() && CGR.isEmpty() &&
2864         itags.isEmpty() && otype==QS_NONE)
2865     ){ return mxCreateCellMatrix(0,0); }
2866
2867     const char *fields[] = { "qtype","otype","itags","ctime","cgr" };
2868
2869     mxArray *S=mxCreateStructMatrix(1,1,5,fields);
2870
2871     mxSetFieldByNumber(S,0,0,qStr().toMx());
2872     mxSetFieldByNumber(S,0,1,otype2Str().toMx());
2873     mxSetFieldByNumber(S,0,2,itags.toMx());
2874     mxSetFieldByNumber(S,0,3,numtoMx(ctime));
2875
2876     mxSetFieldByNumber(S,0,4,CGR.toMx());
2877
2878     return S;
2879 };
2880
2881 template <class TQ, class TD>
2882 mxArray* QSpace<TQ,TD>::DATA_toMx() const {
2883     mxArray *a=mxCreateCellMatrix(DATA.len, DATA.len ? 1 : 0);
2884     if (!a) wblog(FL,"ERR failed to allocate cell array (%d)",DATA.len);
2885     if (DATA.len) {
2886         unsigned i, n=DATA.len; mxArray* x[n];
2887         int np=MAX( QSP_NUM_THREADS, OMP_NUM_THREADS );
2888         if (np>int(n)) { np=(n ? n : 1); }
2889         for (i=0; i<n; ++i) { x[i]=0; }
2890
2891         Wb::LogException ex;
2892
2893         #pragma omp parallel for num_threads(np)
2894         for (i=0; i<n; ++i) { if (!ex) {
2895             try { x[i]=DATA[i]->toMx(); }
2896             catch (Wb::LogException &e_) { ex+=e_; }
2897             catch (...) { ++ex; }
2898         } }
2899         ex.report(FLF);
2900
2901         for (i=0; i<n; ++i) { if (x[i]) mxSetCell(a,i,x[i]); }
2902     }
2903
2904     return a;
2905 };
2906
2907 template <class TQ, class TD>
2908 mxArray* QSpace<TQ,TD>::DATA_save2Mx(char vflag) {

```

```

2912
2913     mxArray *a=mxCreateCellMatrix(DATA.len, DATA.len ? 1 : 0);
2914     if (!a) wblog(FL,"ERR failed to allocate cell array (%d)",DATA.len);
2915
2916     if (vflag) { size_t sd=getDataSize();
2917         if (sd<(1<<30)) vflag=0; else wblog(FL,
2918             "TST %s() %d entries @ %.3gG ...",FCT,DATA.len,sd/double(1<<30));
2919     }
2920
2921     for (unsigned k=0; k<DATA.len; ++k) {
2922         mxSetCell(a, k, DATA[k]->toMx());
2923         if (vflag && k%10==0) printf(" %6d/%1d ... \r",k+1,DATA.len);
2924         DATA[k]->init();
2925     }
2926     if (vflag) printf("\r%60s\r","");
2927
2928     return a;
2929 };
2930
2931 template <class TQ, class TD>
2932 mxArray* QSpace<TQ,TD>::toMx(bool setcls) const {
2933
2934     mxArray *S=mxCreateStruct(1,1);
2935     this->add2MxStruct(S,0,setcls);
2936     return S;
2937 };
2938
2939 template <class TQ, class TD>
2940 mxArray* QSpace<TQ,TD>::save2Mx(char vflag) {
2941
2942     mxArray *S=mxCreateStruct(1,1);
2943     this->save2MxStruct(S,0,0,vflag);
2944     return S;
2945 };
2946
2947 #ifdef WBC_QSPACE_IO
2948     Wb::Clock wbc_qs_toMx("QS::toMx");
2949 #endif
2950
2951 template <class TQ, class TD>
2952 mxArray* QSpace<TQ,TD>::mxCreateStruct(unsigned m, unsigned n) const {
2953
2954     const char *fields[] = { "Q","data","info" };
2955     return mxCreateStructMatrix(m,n,3,fields);
2956 }
2957
2958 template <class TQ, class TD>
2959 void QSpace<TQ,TD>::add2MxStruct(
2960     mxArray *S, unsigned i,
2961     bool setcls,
2962     char chk
2963 ) const {
2964
2965     mxArray *a;
2966
2967     #ifdef WBC_QSPACE_IO
2968         Wb::Clock_resume sw(&wbc_qs_toMx);
2969     #endif
2970
2971     if (chk) { unsigned s=0; int q=0;
2972         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s ||
2973             (q=mxGetFieldNumber(S,"qtype")<0) wblog(FL,
2974                 "ERR %s() must follow mxCreateStruct()\n%lx, %d/%d, %d",
2975                 FCT,S,i+1,s,q);
2976     }
2977
2978     a=QIDX_toMx(); mxSetFieldByNumber(S,i,0,a);
2979     a=DATA_toMx(); mxSetFieldByNumber(S,i,1,a);
2980     a=INFO_toMx(); mxSetFieldByNumber(S,i,2,a);
2981
2982     if (setcls && Wb::my_caller_tid==omp_get_thread_num()) {
2983         mxSetClassName(S,"QSpace");
2984     }
2985 };
2986
2987 template <class TQ, class TD>
2988 void QSpace<TQ,TD>::save2MxStruct(
2989     mxArray *S, unsigned i, char tst, char vflag
2990 ){
2991     mxArray *a;
2992
2993     #ifdef WBC_QSPACE_IO
2994         Wb::Clock_resume sw(&wbc_qs_toMx);
2995     #endif
2996
2997     if (tst) {
2998         unsigned s=0; int q=0;

```



```

2999     if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s
3000         || (q=mxGetFieldNumber(S,"qtype"))<0) wblog(FL,
3001     "ERR %s() must follow mxCreateStruct()\n%lx, %d/%d, %d",
3002     FCT,S,i+1,s,q);
3003 }
3004
3005 a=QIDX_toMx();          mxSetFieldByNumber(S,i,0,a);
3006 a=DATA_save2Mx(vflag);  mxSetFieldByNumber(S,i,1,a);
3007 a=INFO_toMx();          mxSetFieldByNumber(S,i,2,a);
3008
3009     init();
3010 };
3011
3012 #ifdef WBC_QSPACE_IO
3013     Wb::Clock wbc_qs_mxInit("QS:mxInit");
3014 #endif
3015
3016 template <class TQ, class TD>
3017 void QSpace<TQ,TD>::init(
3018     const char *F, int L, const mxArray *S,
3019     char ref,
3020     unsigned k
3021 ){
3022     unsigned i,j,l,m,n,r,dim1=0,dim2=0; unsigned rk=-1;
3023     mxArray *aq,*ad,*ai;
3024     wbvector< wbMatrix<TQ> > MQ;
3025     wbvector< wbMatrix<TQ>* > mq;
3026     const char cflag = (typeid(TD)==typeid(wbcomplex) ? 'C' : 0);
3027     char refD=0, refC=1, isa=0;
3028
3029     #ifdef WBC_QSPACE_IO
3030         Wb::Clock_resume sw(&wbc_qs_mxInit);
3031     #endif
3032
3033     clearQSpace();
3034
3035     if (!S || mxIsEmpty(S) || mxIsEmptyQSpace(S,k)) return;
3036
3037     aq=mxGetField(S,k,"Q");
3038     ad=mxGetField(S,k,"data");
3039     ai=mxGetField(S,k,"info");
3040
3041     if (mxIsQSpace(F,L,S,rk,cflag,k)<=0 || !aq || !ad) wblog(F,L,
3042     "ERR invalid QSpace (%d,%d)",aq=0,ad=0);
3043
3044     if (ref) {
3045         if (ref=='r') { refD      = ref; } else
3046             wblog(FL,"ERR %s() got invalid ref=%c<%d>",FCT,ref,ref);
3047     }
3048
3049
3050     m=mxGetM(aq); r=mxGetN(aq);
3051
3052     if ((m!=1 && r!=1 && m && r) || mxGetNumberOfDimensions(aq)>2) {
3053         wblog(F,L,"ERR %s() invalid QSpace\n"
3054     "Q must be blocked into row cell vector (%dx%d; %d)",
3055     FCT,m,r,mxGetNumberOfDimensions(aq));
3056     }
3057
3058     r*=m; MQ.init(r);
3059
3060     for (i=0; i<r; ++i) {
3061         MQ[i].init(mxGetCell(aq,i));
3062
3063         if (i==0) {
3064             dim1=MQ[i].dim1; dim2=MQ[i].dim2;
3065         }
3066         else if (MQ[i].dim1!=dim1 || MQ[i].dim2!=dim2) { wblog(F,L,
3067     "ERR %s() invalid QSpace\nsize mismatch in Q data (%dx%d; %dx%d)",
3068     FCT, MQ[i].dim1, MQ[i].dim2, dim1, dim2);
3069         }
3070     }
3071
3072     mq.init(MQ.len); for (i=0; i<MQ.len; ++i) mq[i] = &MQ[i];
3073
3074     QIDX.CAT(2, (const wbMatrix<TQ>**) mq.data, mq.len);
3075     QDIM=dim2;
3076
3077     setupDATA();
3078
3079     if (ai && !mxIsEmpty(ai)) {
3080         if (mxIsStruct(ai)) {
3081             mxArray *a; QVec qv; unsigned dq=0;
3082
3083             a=mxGetField(ai,0,"qtype");
3084             if (a) qtype.init(F,L,a); else qtype.init();
3085

```

```

3086     isa=qtype.allAbelian();
3087     if (isa==1 && qtype.len) qtype.ReduceU1();
3088
3089     initOType(F_L,mxGetField(ai,0,"otype"));
3090
3091     itags.init(F_L,mxGetField(ai,0,"itags"));
3092
3093     if (itags.len && itags.len!=r) wblog(FL,
3094         "ERR %s() invalid number of itags (%s; %d/%d)",
3095         FCT,IT2STR__,itags.len,r);
3096
3097     a=mxGetField(ai,0,"ctime");
3098     if (a && !mxIsEmpty(a)) { mxGetNumber(a,ctime); }
3099
3100     a=mxGetField(ai,0,"cgr");
3101     if (a && !mxIsEmpty(a)) {
3102
3103         QSet<TQ> Q;
3104         char xflag=(isa ? 0 : 1);
3105         wvector<unsigned> qdc; qtype.Qpos(qdc);
3106
3107         setupCGR();
3108         m=mxGetM(a); n=mxGetN(a);
3109
3110         if (m==1 && !QIDX.dim1) {
3111             if (!mxIsScalarQSpace(S) || m>1 || r) wblog(FL,
3112                 "ERR %s() invalid scalar QSpace",FCT);
3113             QIDX.init(1,0); QDIM=qtype.Qlen(); r=2;
3114             setupDATA(); setupCGR();
3115         }
3116         else
3117         if (!QDIM || QIDX.dim2%QDIM || QIDX.dim2/QDIM!=r) wblog(FL,
3118             "ERR %s() empty QDIM while info.cgr is specified (%d/%d)",
3119             FCT,m,n);
3120         else
3121         if (mxGetNumberOfDimensions(ad)>2 ||
3122             m!=QIDX.dim1 || m!=CGR.dim1 ||
3123             n!=qtype.len || n!=CGR.dim2 || qtype.Qlen()!=QDIM)
3124             wblog(FL,"ERR %s() invalid dimensions for cell array "
3125                 "info.cgr\n%s) cgr: %dx%d / %dx%d; QIDX: %dx%d/%d",
3126                 FCT, qStr().data, m,n, CGR.dim1, CGR.dim2,
3127                 QIDX.dim1, QIDX.dim2, QDIM
3128             );
3129
3130         if (!mxIsCell(a)&& !mxIsStruct(a)) wblog(FL,
3131             "ERR %s() cell array expected for info.cgr field (%s)",
3132             FCT,mxGetClassName(a)
3133         );
3134
3135         for (l=j=0; j<n; ++j)
3136         for ( i=0; i<m; ++i, ++l) {
3137             Q.init(qtype[j], QIDX.rec(i)+qdc[j], r, QDIM, itags);
3138             CGR(i,j).init(F_L,a,l,refC,&Q,xflag);
3139         }
3140     }
3141
3142     if (qtype.len) { dq=qtype.Qlen();
3143         if (dq!=QDIM && (QIDX.dim2 || CGR.dim2)) wblog(FL,"ERR "
3144             "incompatible number of symmetry labels (%d/%d; %s; %s)",
3145             QDIM, dq, QIDX.sizeStr().data, CGR.sizeStr().data
3146         );
3147     }
3148 }
3149 else wblog(F,L,"ERR invalid qtype (structure required)");
3150
3151 if (isa) {
3152     if (!CGR.isEmpty()) {
3153         for (i=0; i<CGR.dim1; ++i)
3154         for (j=0; j<CGR.dim2; ++j) { if (CGR(i,j).cgb) {
3155             const cdata__ &c=(CGR(i,j).cgb->cgd);
3156             if (!c.isScalar()) wblog(FL,
3157                 "ERR %s() invalid abelian CGC space (%s)",FCT,SSTR(c));
3158             if (c.D.data && c.D.data[0]!=1) wblog(FL,
3159                 "ERR %s() invalid abelian CGC coefficient (%.4g)",
3160                 FCT,double(c.D.data[0])
3161             );
3162         }}
3163         checkQ_CGR(FL); CGR.init();
3164     }
3165 }
3166 }
3167 }
3168 else qtype.init();
3169
3170 m=mxGetM(ad); n=mxGetN(ad);
3171
3172 if ((m>1 && n>1) || mxGetNumberOfDimensions(ad)>2) wblog(F,L,

```

```

3173     "ERR QSpace() cell VECTOR expected for data (%d,%d)",m,n);
3174
3175     m*=n;
3176
3177     if (DATA.len!=m) {
3178         if (m==1 && DATA.len==0 && QIDX.isEmpty() && CGR.isEmpty()) {
3179             QIDX.init(1,0); setupDATA();
3180             if (r<2) r=2;
3181         }
3182         else wblog(F,L,
3183             "ERR QSpace() dimension mismatch (QIDX: %dx%d, data: %d)",
3184             QIDX.dim1, QIDX.dim2, m
3185         );
3186     }
3187
3188     for (i=0; i<m; ++i) {
3189         DATA[i]->init(mxGetCell(ad,i), refD);
3190         DATA[i]->addSingletons(r);
3191     }
3192
3193 };
3194
3195 template <class TQ, class TD> inline
3196 void QSpace<TQ,TD>::init2ref(const QSpace<TQ,TD> &A) {
3197
3198     clearQSpace();
3199
3200     QIDX.init2ref(A.QIDX); QDIM=A.QDIM;
3201     DATA.init2ref(A.DATA);
3202     qtype.init2ref(A.qtype); otype=A.otype;
3203     itags.init2ref(A.itags);
3204
3205     if (!A.CGR.isEmpty()) CGR.init2ref(A.CGR);
3206
3207     isref=1;
3208 };
3209
3210 template <class TQ, class TD>
3211 QSpace<TQ,TD>& QSpace<TQ,TD>::NormCG(char skipzeros) {
3212
3213     if (CGR.isEmpty()) return *this;
3214
3215     wvector<unsigned> mark;
3216     unsigned i,j; double cfac;
3217
3218     gotCGS(FL);
3219     unRef();
3220
3221     for (i=0; i<CGR.dim1; ++i) {
3222         for (cfac=1., j=0; j<CGR.dim2; ++j) {
3223             cfac *= CGR(i,j).NormSignR();
3224         }
3225         if (cfac!=1.) {
3226             if (cfac!=0) { (*DATA[i])*=TD(cfac); }
3227             else if (skipzeros) {
3228                 if (!mark.len) { mark.init(CGR.dim1); }
3229                 mark[i]=1;
3230             }
3231         }
3232     }
3233
3234     if (mark.len) {
3235         windex I;
3236         Select(mark.findzeros(I));
3237     }
3238
3239     if ((i=qtype.allAbelian())) {
3240         checkQ_CGR(FL); CGR.init();
3241         if (i==1) qtype.ReduceU1();
3242     }
3243
3244     return *this;
3245 };
3246
3247 template <class TQ, class TD> inline
3248 void QSpace<TQ,TD>::setRand(double nrm) {
3249
3250     unsigned i;
3251     for (i=0; i<DATA.len; ++i) DATA[i]->setRand();
3252
3253     if (nrm>0) {
3254         double fac=nrm/sqrt(double(norm2()));
3255         for (i=0; i<DATA.len; ++i) (*DATA[i]) *= fac;
3256     }
3257 }
3258
3259 template <class TQ, class TD>

```

```

3260 void QSpace<TQ,TD>::info(const char *vname,
3261     unsigned nlt, unsigned nlb,
3262     unsigned nind
3263 ) const {
3264
3265     wbvector<widx_t> D,DX;
3266     char cgflag=gotCGS();
3267     unsigned r=(QDIM!=0 ? QIDX.dim2/QDIM : 0);
3268     size_t l=0, n=64; char s[n], sp_[nind+1];
3269
3270     if (QDIM && QIDX.dim2 && QIDX.dim2%QDIM) wblog(FL,
3271         "ERR %s() %d/%d = ?",FCT, QIDX.dim2, QDIM);
3272     if (nind) { memset(sp_, ' ', nind); }; sp_[nind]=0;
3273
3274     l=snprintf(s,n,"QSpace<%s,%s> %s",
3275         sTSTR(TQ), sTSTR(TD), qtype.len ? qStr().data : "");
3276     if (l<n && itags.len) {
3277         l+=snprintf(s+l,n-l," '%s'",STR(itags));
3278     }
3279     if (l>n) wblog(FL,"WRN %s() string out of bounds (%d/%d)",FCT,l,n);
3280
3281     for (l=0; l<nlt; ++l) { printf("\n"); }
3282     if (vname && vname[0]) { printf("%s%-4s",sp_,vname); }
3283     else if (sp_[0]) { printf("%s",sp_); }
3284     printf(" %-34s {%3ldx%d}%d",s,QIDX.dim1,r,QDIM);
3285
3286     getDim(D,&DX);
3287
3288     printf(" %10s",D.toStrD().data);
3289     if (cgflag>0) printf(" |%12s", DX.toStrD().data);
3290     printf("%s\n", isref? " *REF*":"" );
3291
3292     for (l=0; l<nlb; ++l) { printf("\n"); }
3293 };
3294
3295 template <class TQ, class TD>
3296 void QSpace<TQ,TD>::print(const char *vname, char vflag) const {
3297
3298     unsigned i=0,j,s, n=QIDX.dim1, m=(n<12? n:4);
3299     char cgflag=gotCGS(FL), fmt[]="%2g";
3300
3301     if (typeid(TQ)!=typeid(double)) fmt[2]='d';
3302
3303     if (vflag & 'D') { vflag |=3; }
3304     if (vflag&1) { printf("\n"); }
3305
3306     info(vname,0,0,0); printf("\n");
3307
3308     for (; i<n; ++i) {
3309         if (i>=m) {
3310             if (n>2*m) { i=n-m; printf(" ... \n"); }
3311             m=n;
3312         }
3313
3314         printf("%3d. { %s } %8s ", i+1,
3315             i<QIDX.dim1? QIDX.rec2Str(i,fmt, " ",QDIM,";").data : "!",? ",
3316             i<DATA.len ? DATA[i]->sizeStr().data : "!",? ");
3317
3318         if (i>=DATA.len) { printf(" !?\n"); continue; }
3319         if (vflag & 64) {
3320             if (!(DATA[i]->prntdata(vname))) printf("\n");
3321         }
3322         else {
3323             if (cgflag>0) { printf("|");
3324                 for (j=0; j<CGR.dim2; ++j) { printf("%8s",SSTR(CGR(i,j))); }
3325                 printf(" |");
3326             }
3327
3328             s=DATA[i]->numel();
3329
3330             if (s==0) { printf(" []\n"); } else
3331             if (s==1) { printf(" %s", Wb::num2Str(DATA[i]->data[0]).data); }
3332             else {
3333                 s *= sizeof(TD);
3334                 if (s<(1<10)) printf(" %5d B", s); else
3335                 if (s<(1<20)) printf(" %5.2f kB", s/double(1<10));
3336                 else printf(" %5.2f MB", s/double(1<20));
3337             }
3338             printf("%s\n",DATA[i]->isRef() ? " ref*":"" );
3339         }
3340     }
3341
3342     if (vflag&2) { printf("\n"); }
3343 };
3344
3345 #endif
3346

```

5.12 QSpace_aux.hh

```

1  #ifndef __WB_QSpace_AUX_HH__
2  #define __WB_QSpace_AUX_HH__
3
4  template <class TQ, class TD> class QSpace;
5  template <class TQ, class TD> class CPAT;
6
7  // ===== //
8  // ensure orthogonal CGC spaces through decomposition/orthogonalization
9  // w.r.t. existing CGC coefficient spaces; required for:
10 // * QSpace::CGC contraction in the presence of outer multiplicity
11 // => see contractCG_ortho(), followed by contractDATA_ortho().
12 // * make CGC spaces of given QSpace unique
13 // => see makeUnique_ortho()
14 // Wb, Jul10,14
15 // ===== //
16
17 template <class TQ>
18 class OrthoCGS {
19
20 public:
21
22     OrthoCGS() : cgb(NULL), conj(0), rtype(CGR_DEFAULT) {
23     };
24
25     OrthoCGS& init() {
26         Q.init(); cgb=NULL; conj=0; rtype=CGR_DEFAULT;
27         cgp.init(); cgQ.init(); cgR.init();
28         return *this;
29     };
30
31     OrthoCGS& init(const CRef<TQ> &R) {
32         Q.init(R); cgb=R.cgb; cgp=R.cgp; conj=R.conj; rtype=R.rtype;
33         cgQ.init(); cgR.init();
34         return *this;
35     };
36
37     template <class TA, class TB>
38     int contractCG_ortho(
39         const char *F, int L,
40         const QSpace<TQ,TA> &A, C_UVEC &Ia, const ctrIdx &ica,
41         const QSpace<TQ,TB> &B, C_UVEC &Ib, const ctrIdx &icb,
42         unsigned isym,
43         char cg_preview
44     );
45
46     template <class TD>
47     unsigned getCGS_ortho(
48         const char *F, int L,
49         QSpace<TQ,TD> &A, C_UVEC &Ia, unsigned isym
50     );
51
52     unsigned getOM() const { return getOM(wbvector<unsigned>()); }
53
54     QSet<TQ> Q;
55     const CDATA_TQ *cgb;
56
57     wbperm cgp;
58     char conj;
59
60     cgrType rtype;
61
62     wbarray<double> cgQ;
63     wbarray<double> cgR;
64
65 protected:
66 private:
67 };
68
69 template <class TQ, class TA, class TB, class TC>
70 int contractDATA_ortho(
71     const char *F, int L,
72     const QSpace<TQ,TA> &A, C_UVEC &Ia, const ctrIdx &ica,
73     const QSpace<TQ,TB> &B, C_UVEC &Ib, const ctrIdx &icb,
74     const wbvector< OrthoCGS<TQ> > &Mc,
75     QSpace<TQ,TC> &C, unsigned ic
76 );
77
78 template <class TQ, class TA, class TB, class TC>
79 void contractDATA_plain(
80     const char *F, int L,
81     const QSpace<TQ,TA> &A, C_UVEC &Ia, const ctrIdx &ica,
82     const QSpace<TQ,TB> &B, C_UVEC &Ib, const ctrIdx &icb,
83     QSpace<TQ,TC> &C, unsigned ic
84 );
85

```

```

86 template <class TQ,class TD>
87 unsigned combineDATA_ortho(
88     const char *F, int L,
89     const QSpace<TQ,TD> &A, C_UVEC &Ia,
90     const wbvector< OrthoCGS<TQ> > &MC, QSpace<TQ,TD> &C, unsigned ic
91 );
92
93 template<class TQ, class TD>
94 void mxInitQSpaceVec(
95     const char *F, int L, const mxArray* C,
96     wbvector< QSpace<TQ,TD> > &Fk,
97     const char ref=0,
98     const unsigned *rmin=NULL,
99     const unsigned *rmax=NULL
100 );
101
102 template<class TQ, class TD>
103 void mxInitQSpaceVecVec(
104     const char *F, int L, const mxArray* C,
105     wbvector< wbvector< QSpace<TQ,TD> > > &Fk, const char ref=0
106 );
107
108 template<class TQ, class TD>
109 void mxCInitQSpaceVec(
110     const char *F, int L, const mxArray* C,
111     wbvector< QSpace<TQ,TD> > &Fk, const char ref=0
112 );
113
114 template<class TQ, class TD>
115 mxArray* QSpaceVec2Mx(
116     const wbvector< QSpace<TQ,TD> > &Fk
117 );
118
119 template<class TQ, class TD>
120 mxArray* QSpaceVec2Mx(
121     const wbvector< QSpace<TQ,TD> > &F, wbindex &I
122 );
123
124 template<class TQ, class TD>
125 mxArray* QSpaceVecVec2Mx(
126     const wbvector< wbvector< QSpace<TQ,TD> > > &F,
127     const char ref=0
128 );
129
130 template<class TQ, class TD>
131 mxArray* QSpaceVec2Mxc(
132     const wbvector< QSpace<TQ,TD> > &F,
133     const char ref=0
134 );
135
136 template<class TQ, class TD>
137 void putQSpaceVec(
138     const wbvector< QSpace<TQ,TD> > &Fk,
139     const char *vname, const char *ws="caller"
140 ){
141     mxArray *a=Fk.toMx();
142     mxPutAndDestroy(FL,a,vname);
143 };
144
145 template <class TQ, class TD>
146 void getDiffOp(
147     const QSpace<TQ,TD> &A, unsigned ia,
148     const QSpace<TQ,TD> &B, unsigned ib, const QSpace<TQ,TD> &C
149 );
150
151 template<class TQ, class TD>
152 void getQall(const wbvector< QSpace<TQ,TD> > &F, wbMatrix<TQ> &QA);
153
154 int mxIsQSpace(
155     const char *F, int L,
156     const mxArray *a, unsigned &rank,
157     char cflag=0,
158     unsigned k=-1,
159     const unsigned *rmin=NULL,
160     const unsigned *rmax=NULL,
161     const char *istr=NULL
162 );
163
164 inline int mxIsQSpace(
165     const char *F, int L, const mxArray *a, char cflag=0,
166     const char *istr=NULL
167 ){
168     unsigned r=-1;
169     return mxIsQSpace(F,L,a,r,cflag,-1,NULL,NULL,istr);
170 };
171
172 inline int mxIsQSpace(

```

```

173     const mxArray *a, char cflag=0, const char *istr=NULL
174 ){
175     unsigned r=-1;
176     return mxIsQSpace(NULL,0,a,r,cflag,-1,NULL,NULL,istr);
177 };
178
179 bool mxIsQSpaceArr(
180     const char *F, int L,
181     const mxArray *S,
182     unsigned rank=-1,
183     int arrdim=2,
184     const unsigned *rmin=NULL,
185     const unsigned *rmax=NULL,
186     char cflag=0
187 );
188
189 bool mxIsQSpaceVecVec(const mxArray *C, unsigned r=-1, char cflag=0);
190
191 inline bool mxIsQSpaceVec(
192     const char *F, int L,
193     const mxArray *S,
194     unsigned rank=-1,
195     const unsigned *rmin=NULL,
196     const unsigned *rmax=NULL,
197     char cflag=0
198 ){ return mxIsQSpaceArr(F,L,S,rank,1,rmin,rmax,cflag); };
199
200 inline bool mxIsQSpaceVecR23(
201     const char *F, int L,
202     const mxArray *S,
203     char cflag=0
204 ){
205     unsigned rank=-1, rmin=2, rmax=3;
206     return mxIsQSpaceArr(F,L,S,rank,1,&rmin,&rmax,cflag);
207 };
208
209 inline bool mxIsQSpaceMat(
210     const char *F, int L,
211     const mxArray *S,
212     unsigned rank=-1,
213     const unsigned *rmin=NULL,
214     const unsigned *rmax=NULL,
215     char cflag=0
216 ){ return mxIsQSpaceArr(F,L,S,rank,2,rmin,rmax,cflag); };
217
218 bool mxIsEmptyQSpace(const mxArray *a, unsigned k=0);
219
220 bool mxIsScalarQSpace(const mxArray *a, unsigned k=0);
221
222 inline bool mxIsQSpaceScalar(
223     const char *F, int L,
224     const mxArray *S,
225     unsigned rank=-1,
226     const unsigned *rmin=NULL,
227     const unsigned *rmax=NULL,
228     char cflag=0
229 ){ return mxIsQSpaceArr(F,L,S,rank,0,rmin,rmax,cflag); };
230
231 bool mxsIsQSpaceVec(const mxArray *S, int fid, unsigned r=-1, char cflag=0);
232 bool mxsIsQSpaceVEC(const mxArray *S, int fid, unsigned r=-1, char cflag=0);
233
234 int mxIsQSpaceVec(
235     const char *F, int L,
236     const char *fname, const char *vname,
237     unsigned rank=-1,
238     const unsigned *rmin=NULL,
239     const unsigned *rmax=NULL,
240     char cflag=0,
241     unsigned N=-1
242 );
243
244 int mxIsQSpaceVEC(
245     const char *F, int L,
246     const char *fname, const char *vname,
247     unsigned rank=-1,
248     const unsigned *rmin=NULL,
249     const unsigned *rmax=NULL,
250     char cflag=0
251 );
252
253 bool mxIsQSpaceVecOrEmpty(const mxArray *S, int rank, char cflag=0) {
254     if (mxIsEmpty(S)) return 1;
255     return mxIsQSpaceVec(F,L,S,rank,NULL,NULL,cflag);
256 };
257
258 inline int mxIsQSpaceOrEmpty(
259     const mxArray *a, unsigned rank=-1, char cflag=0, unsigned k=-1

```

```

260 ){ if (mxIsEmpty(a)) return 1;
261     else return mxIsQSpace(NULL,0,a,rank,cflag,k);
262 };
263
264 inline int mxIsQSpaceOrEmpty(const char *F, int L,
265     const mxArray *a, unsigned rank=-1, char cflag=0, unsigned k=-1
266 ){ if (mxIsEmpty(a)) return 1;
267     else return mxIsQSpace(F,L,a,rank,cflag,k);
268 };
269
270 QVec qsGetSym(
271     const char *F, int L, const mxArray *a, unsigned k=-1);
272
273 char qsGotCGS(
274     const char *F, int L, const mxArray *a, unsigned k=-1
275 ){
276     QVec qtype=qsGetSym(F,L,a,k);
277     return qtype.permitsOM();
278 };
279
280 #endif
281

```

5.13 spectral.hh

```

1 #ifndef __WB_SPECTRAL_HCC__
2 #define __WB_SPECTRAL_HCC__
3
4 // default smoothing: norm-preserving log-gauss broadening
5 #define LOG_GAUSS_BRD 2
6
7 namespace Wb {
8
9 template <class TS>
10 void Fourier(
11     const wbvector<double> &om,
12     const wbMatrix<TS> &A,
13     const wbvector<double> &tt,
14     wbMatrix<wbcomplex> &az,
15     char domflag=0,
16     double alpha=0.,
17     char vflag=1
18 );
19
20 template <class TS>
21 void dispSumRule(
22     const wbvector<double> &om,
23     const wbMatrix<TS> &A
24 );
25
26 };
27
28 void KKimag(
29     const wbMatrix<double> &Rp,
30     const wbMatrix<double> &Rn,
31     const wbvector<double> &OM,
32     wbMatrix<double> &Ip,
33     wbMatrix<double> &In
34 );
35
36 template <class TS>
37 class Spectral {
38 public:
39
40     Spectral()
41         : sigma(0.6), sigma2(0.5), alpha(0.25), Emin(1E-8), Emax(10.),
42         eps(1E-6), nlog(128), l0(0), fac(1.), Delta(0.) {};
43
44     void init() {
45         Ap.init(); An.init(); Ap_buf.init(); An_buf.init();
46         Ar.init(); A0.init(); mspec.init();
47         istr.init();
48     };
49
50     void init(const char *istr, unsigned Nops,
51         double delta=0., double T_=0, unsigned Niter=0
52     );
53
54     void initBuf() {
55         if (Ap.dim1!=An.dim1) wblog(FL,
56             "ERR severe dimension mismatch (%d,%d).", Ap.dim1, An.dim1);
57
58         Ap_buf.init(Ap.dim1, Ap.dim2);
59

```



```

59     An_buf.init(An.dim1, An.dim2);
60 };
61
62 void calcSpectralMoments(const char *F, int L, unsigned n) {
63     if (n) {
64         if (n>4) wblog(F,L,"WRN %s with n=%d !?",FCT,n);
65         else wblog(F,L,"<i> calculate %d spectral moment(s)",n);
66         mspec.init(Ap.dim1,n+1);
67     }
68 };
69
70 void Add(
71     const wbararray<double> &Om, const wbararray<TS> &CC, unsigned s,
72     char dblock=0,
73     char mflag=0
74 );
75
76 void Add2buf(const wbararray<double> &Om, const wbararray<TS> &CC, unsigned s);
77
78 void crossAddBuf(const char minor=1, char vflag=1);
79
80 void saveIter(const char *F, int L, unsigned iter);
81
82 void getOmega(wbvector<double> &om, unsigned &linfit) const;
83 void getOmega(wbvector<double> &om) const {
84     unsigned linfit=0; getOmega(om,linfit);
85 };
86
87 void applyIROPfac(const char *F, int L, const wbvector<double> &fac);
88
89 void getRawData(
90     wbvector<double> &om, wbMatrix<TS> &A,
91     unsigned linfit=0, const char *isbuf=""
92 ) const;
93
94 void getPARTIAL(
95     wbararray<TS> &A1, wbararray<TS> &A2) const;
96
97 void pairRawData();
98
99 void detailedBalance(
100     const double T, wbvector<double> &om, wbMatrix<TS> &aa) const;
101
102 void dispSumRule();
103 void dispSumRule(const wbvector<double> &om, const wbMatrix<TS> &A);
104
105 void getSmoothSpec(
106     wbvector<double> &om, wbMatrix<TS> &A, char vflag=1);
107
108 void getSmoothSpec(
109     wbMatrix<TS> &A, const wbvector<double> &om, char vflag=1);
110
111 void KKreal(
112     wbMatrix<wbcomplex> &GZ,
113     wbvector<double> &OM,
114     wbvector<double> &DOM
115 ) const;
116
117 wbMatrix<TS> Ap, An;
118 wbMatrix<TS> Ap_buf, An_buf;
119 wbvector<TS> Ar;
120 wbMatrix<TS> A0;
121 wbMatrix<TS> mspec;
122
123 wbararray<TS> AP, AN;
124
125 wstring istr;
126 double Temp;
127
128 double sigma, sigma2, alpha, Emin, Emax, eps;
129 unsigned nlog;
130
131 protected:
132 private:
133
134     double l0, fac, Delta;
135
136     double OM2IDX(const double &om) const {
137         return ((log(fabs(om))-l0)*fac);
138     }
139
140     inline unsigned om2idx(double om) const {
141         if (Delta==0) { if (om==0.) return 0; }
142         else {
143             if (om> Delta) om-=Delta; else
144             if (om<=-Delta) om+=Delta; else return 0;
145         }

```

```

146     int k = (int)floor(OM2IDX(om));
147     if (k<1) k=1; else
148     if (k>=(int)Ap.dim2) { k=Ap.dim2-1; }
149     return (unsigned)k;
150 }
151
152 inline double idx2om(const unsigned &k) const {
153     return exp( (double(k)+0.5)/fac + 10 );
154 }
155
156 int crossAddBuf_aux(
157     TS *a, const TS *b, const unsigned M,
158     const char minor=1, char vflag=1
159 );
160
161 TS foldLogGaussian(
162     const double om, const TS *aa, const unsigned n
163 );
164
165 TS foldGaussian(
166     const double om, const double b,
167     const double *oo, const TS *aa, const unsigned n
168 );
169
170 };
171
172 template <class TS>
173 class RSpecData {
174     public:
175     RSpecData() { };
176
177     RSpecData& Append(const double *op, const TS *cp, unsigned n) {
178         om.Append(n,op);
179         cc.Append(n,cp); return *this;
180     };
181
182     mxArray* mxCreateStruct(unsigned m, unsigned n) const {
183         const char *flds[]={"om","cc"};
184         return mxCreateStructMatrix(m,n,2,flds);
185     };
186
187     void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const {
188         mxSetFieldByNumber(S,i,0,om.toMx(tflag));
189         mxSetFieldByNumber(S,i,1,cc.toMx(tflag));
190     };
191
192     wvector<double> om;
193     wvector<TS> cc;
194
195     protected:
196     private:
197
198 };
199
200 template <class TS>
201 class TDSpectral {
202     public:
203
204     TDSpectral() : raw(0), l0(0), fac(1.), check_avg_om(0) {};
205
206     void init(const char *F, int L,
207         const char *istr0, const wvector<double> &t0,
208         unsigned Nnrg, unsigned Nop,
209         double Emin=1E-8, double Emax=10., unsigned nl=512,
210         char vflag=1
211     );
212
213     void checkAvgOm() {
214         check_avg_om=1;
215         AW.init(2,AP.dim(2),AP.dim(3));
216     };
217
218     wbararray<TS> getAvgOm();
219
220     void Add(
221         const wbararray<double> &Om, const wbararray<TS> &CC,
222         unsigned iter, unsigned s
223     );
224
225     void addPartialFourier();
226
227     void getOmega(wvector<double> &om, unsigned &linfit) const;
228     void getOmega(wvector<double> &om) const {
229         unsigned linfit=0; getOmega(om,linfit);
230     }
231
232     void applyIROPfac(

```

```

233     const char *F, int L, const wbvector<double> &fac, char rixs=0);
234
235 void getSpecData(
236     wbvector<double> &om, wbararray<TS> &A,
237     unsigned linfit=0
238 ) const;
239
240 void getSpecData(
241     wbvector<double> &om, wbMatrix<TS> &A,
242     unsigned linfit=0
243 );
244
245 void mergeNRGData();
246
247 void getSmoothSpec_t(double sigma, double eps,
248     char bswitch = LOG_GAUSS_BRD, char vflag=1);
249
250 void Fourier(
251     wbvector<double> &om, wbMatrix<wbcomplex> &ST,
252     double alpha, double Lambda, char vflag=1
253 );
254
255 void Fourier(
256     wbvector<double> &om, wbMatrix<TS> &A,
257     const wbvector<double> &tt,
258     wbMatrix<wbcomplex> &at,
259     double alpha=0., char vflag=1
260 );
261
262 void dispSumRule();
263
264 wbararray<TS> AP,AN;
265 wbMatrix<TS> Ap,An;
266 wbMatrix<wbcomplex> At;
267 wbvector<double> tt;
268 wstring istr;
269
270 mxArray *toMx() const;
271 int init(
272     const mxArray *S,
273     double &emin, double &emax, unsigned &nlog
274 );
275
276 unsigned raw;
277 wbvector< RSpecData<TS> > RAW;
278
279 protected:
280 private:
281
282     double l0,fac;
283     unsigned N;
284
285     char check_avg_om;
286     wbararray<TS> AW;
287
288     double OM2IDX(const double &om) const {
289         return ((log(fabs(om))-l0)*fac);
290     }
291
292     inline unsigned om2idx(const double &om) const {
293         if (om==0.) return 0;
294
295         int k = (int)floor(OM2IDX(om));
296         if (k<1) k=1; else if (k>=(int)N) k=N-1;
297
298         return (unsigned)k;
299     }
300
301     inline double idx2om(const unsigned &k) const {
302         return exp( ((double)k+0.5)/fac + l0 );
303     }
304
305     void initLG(
306         wbMatrix<double> &wlg, unsigned n, double sigma,
307         char bswitch = LOG_GAUSS_BRD, char vflag=1
308     );
309
310     void foldLogGaussian(
311         const wbMatrix<double> &wlg,
312         TS *aa, TS a0, unsigned i0
313     );
314 };
315
316 template <class TS>
317 void Spectral<TS>::init (
318     const char *istr0,
319     unsigned Nops,

```

```

320     double d,
321     double T,
322     unsigned Niter
323 ){
324     unsigned Nom;
325     double l2;
326
327     if (Emin<=0. || Emax<=0. || Emin>=0.1*Emax || nlog==0) wblog(FL,
328         "ERR %s() invalid range E = [%g .. %g @ %d]%", myname, Emin, Emax, nlog,
329         (Emin>0 && Emax>0 && Emin<Emax) ? " (require Emin<Emax/10)":"");
330     if (d<0) wblog(FL, "ERR invalid Delta (%g neg!)", d);
331
332     if (eps<Emin) wblog(FL, "NB! eps=%g < min(|om|)=%g", eps, Emin);
333     if (alpha!=0.25) {
334         if (alpha==0.5) wblog(FL,
335             "WRN %s() using symmetric log-Gaussian broadening", FCT);
336         else wblog(FL, "WRN %s() using "
337             "log-Gaussian with shift alpha=%g !?", FCT, alpha);
338     }
339
340     Delta=d;
341
342     l0 =log(Emin);
343     l2 =log(Emax);
344     fac=log10(M_E)*double(nlog);
345     Nom=(unsigned)((l2-l0)*fac)+1;
346
347     Ap.init(Nops,Nom);
348     An.init(Nops,Nom); Ar.init(Nops);
349
350     if (int(Niter)>0) {
351         AP.init(Nom,Nops,Niter);
352         AN.init(Nom,Nops,Niter);
353     }
354
355     istr=istr0; Temp=T;
356 };
357
358 template <class TS>
359 void Spectral<TS>::Add(
360     const wbararray<double> &Om,
361     const wbararray<TS> &CC,
362     unsigned s,
363     char dblock,
364     char mflag
365 ){
366     unsigned i,k, n=Om.SIZE.prod();
367     double x, emin=0.1*idx2om(0);
368     TS q, r=0;
369
370     double bfac=(!mflag && Temp>0 ? 10/Temp : 0);
371
372     double zfac=(Temp>0 ? 1/Temp : 0);
373
374     if (!Om.hasSameSize(CC)) {
375         Om.info("Om"); CC.info("CC");
376         wblog(FL, "ERR size mismatch between Om and CC!");
377     }
378     if (dblock) {
379         if (Om.SIZE.len!=2 || Om.SIZE[0]!=Om.SIZE[1]) wblog(FL,
380             "ERR %s() quadratic block expected (%s)",
381             FCT, Om.sizeStr().data);
382     }
383
384     if (s>=Ap.dim1 || s>=Ar.len) wblog(FL,
385         "ERR index out of bounds (%d/%d;%d)", s, Ap.dim1, Ar.len);
386
387     for (i=0; i<n; ++i) { x=Om.data[i]; k=om2idx(x);
388         if (x<0. || (x<=emin && s%2))
389             An(s,k)+=CC.data[i];
390         else Ap(s,k)+=CC.data[i];
391
392         if (x!=0) {
393             q=CC.data[i]/x;
394             if (bfac>0) { x=bfac*fabs(x); if (x<10) {
395                 double w=exp(-x*x);
396                 q*=(1-w);
397             }}
398             r+=q;
399         }
400         else if (mflag && !(s%2)) {
401             r+=zfac*CC.data[i];
402         }
403     }
404
405     Ar[s]+=r;
406

```

```

407     if (mspec.dim2) {
408         double dbl; unsigned j;
409
410         if (mspec.dim1!=Ap.dim1) {
411             wblog(FL,
412                 "WRN mspec must be initialized AFTER ASpec!\n"
413                 "resize (%d->%d x %d)", mspec.dim1, Ap.dim1, mspec.dim2
414             );
415             mspec.Resize(Ap.dim1,mspec.dim2);
416         }
417
418         for (i=0; i<n; i++)
419             for (dbl=CC.data[i], j=0; j<mspec.dim2; j++) {
420                 if (j) dbl*=Om.data[i];
421                 mspec(s,j)+=dbl;
422             }
423     }
424 };
425
426 template <class TS>
427 void Spectral<TS>::Add2buf(
428     const wbararray<double> &Om, const wbararray<TS> &CC,
429     unsigned s
430 ){
431     unsigned i,k,n=Om.SIZE.prod();
432
433     if (!Om.hasSameSize(CC)) {
434         Om.info("Om"); CC.info("CC");
435         wblog(FL,"ERR Size mismatch between Om and CC!");
436     }
437
438     if (s>Ap_buf.dim1)
439         wblog(FL, "ERR Index out of bounds (%d,%d)",s, Ap_buf.dim1);
440
441     for (i=0; i<n; i++) {
442         k=om2idx(Om.data[i]);
443         if (Om.data[i]<0.)
444             An_buf(s,k)+=CC.data[i];
445         else Ap_buf(s,k)+=CC.data[i];
446     }
447 };
448
449 template <class TS>
450 void Spectral<TS>::crossAddBuf(
451     const char minor,
452     char vflag
453 ){
454     unsigned s, Nom=Ap.dim2;
455     int i;
456
457     if (!Ap.hasSameSize(Ap_buf) || !An.hasSameSize(An_buf) ||
458         !Ap.hasSameSize(An)
459     ){
460         Ap.info("Ap"); Ap_buf.info("Ap_buf");
461         An.info("An"); An_buf.info("An_buf");
462         wblog(FL,"ERR Spectral - severe size mismatch.");
463     }
464
465     for (s=0; s<Ap.dim1; s++) { i=0;
466
467         i+=crossAddBuf_aux(Ap.rec(s), Ap_buf.rec(s), Nom,minor,vflag);
468         i+=crossAddBuf_aux(An.rec(s), An_buf.rec(s), Nom,minor,vflag);
469
470         if (i>1) {
471             wblog(FL,"WRN empty buffer (%d/%d) !?",s,Ap.dim1);
472             continue;
473         }
474     }
475 };
476 };
477
478 template <class TS>
479 int Spectral<TS>::crossAddBuf_aux(
480     TS *a,
481     const TS *b,
482     const unsigned Nom,
483     const char minor,
484     char vflag
485 ){
486     unsigned i, i1, i2, amin, amax, bmin, bmax;
487     double t, tfac;
488
489     if (minor<1 || minor>2) wblog(FL,
490         "ERR invalid minor flag %d", minor);
491
492     for (i=0; i<Nom; ++i) { if (a[i]!=0.) break; }; amin=i;
493     for (i=Nom-1; i>=amin; --i) { if (a[i]!=0.) break; }; amax=i;

```

```

494     for (i=0; i<Nom; ++i) { if (b[i]!=0.) break; }; bmin=i;
495     for (i=Nom-1; i>=bmin; --i) { if (b[i]!=0.) break; }; bmax=i;
496
497     if (bmin>bmax) {
498         return vflag;
499     }
500
501     if (amin>amax) {
502         for (i=bmin; i<=bmax; i++) a[i]=b[i];
503         return 0;
504     }
505
506     i1=MAX(amin,bmin); i2=MIN(amax,bmax);
507
508     if (i1==i2) { a[i1]=0.5*(a[i1]+b[i2]); return 0; }
509
510     tfac = 1. / double(i2-i1);
511
512     for (i=0; i<Nom; ++i) {
513         if (i<i1 || i>i2) a[i]+=b[i];
514         else {
515             t = tfac * double(i-i1);
516             if (minor==1)
517                 a[i] = (1.-t)*a[i] + t*b[i];
518             else a[i] = t*a[i] + (1.-t)*b[i];
519         }
520     }
521
522     return 0;
523 };
524
525 template <class TS>
526 void Spectral<TS>::saveIter(const char *F, int L, unsigned iter){
527
528     unsigned n=Ap.numel();
529
530     if (AP.SIZE.len!=3 || AP.SIZE!=AN.SIZE || n!=AP.SIZE[0]*AP.SIZE[1])
531         wblog(F_L,
532             "ERR %s() invalid initialization of AP/AN [%s; %s; %dx%d]",
533             FCT,AP.sizeStr().data, AN.sizeStr().data, Ap.dim1, Ap.dim2
534         );
535     if (iter>=AP.SIZE[2]) wblog(F_L,
536         "ERR %s() iter out of bounds AP/AN [%s; %d/%d]",
537         FCT,AP.sizeStr().data, iter+1, AP.SIZE[2]
538     );
539
540     memcpy(AP.data+iter*n, Ap.data, n*sizeof(Ap.data[0]));
541     memcpy(AN.data+iter*n, An.data, n*sizeof(An.data[0]));
542 };
543
544 template <class TS>
545 void Spectral<TS>::getOmega(wbvector<double> &om, unsigned &linfit) const {
546
547     unsigned i, Nom=Ap.dim2;
548     double dbl;
549
550     if (linfit) linfit=unsigned(fac);
551     if (linfit>Nom) wblog(FL,"ERR Nom<linfit (%d,%d) !?",Nom,linfit);
552
553     om.init(2*Nom);
554
555     dbl = idx2om(linfit) / (linfit+0.5);
556
557     for (i=0; i<linfit; ++i) om[Nom-1-i] = -(om[Nom+i] = dbl*(i+0.5));
558     for ( ; i<Nom; ++i) om[Nom-1-i] = -(om[Nom+i] = idx2om(i) );
559 }
560
561 template <class TS>
562 void Spectral<TS>::applyIROPfac(
563     const char *F, int L, const wbvector<double> &fac) {
564
565     unsigned i1,i2,s,m, n=fac.len;
566     double x;
567
568     if (2*n!=Ap.dim1 || 2*n!=An.dim1 || 2*n!=Ar.len) wblog(FL,
569         "ERR %s() size mismatch (%d,%d,%d/2*%d)",
570         FCT,Ap.dim1,An.dim1,Ar.len,n);
571
572     for (s=0; s<n; ++s) { if (fac[s]!=1) {
573         if (fac[s]<1 || fac[s]!=round(fac[s])) wblog(FL,
574             "ERR %s() got fac[%d]=%g !?",FCT,s+1,fac[s]);
575         wblog(FL, " * applying IROP factor 1/%g to op(%d)",fac[s],s+1);
576
577         x=1/fac[s]; i1=2*s; i2=2*s+1;
578         Ap.scaleRow(i1,x); An.scaleRow(i1,x); Ar[i1]*=x;
579         Ap.scaleRow(i2,x); An.scaleRow(i2,x); Ar[i2]*=x;
580     }}

```

```

581
582     if (!Ap_buf.isEmpty() || !An_buf.isEmpty()) {
583         if (!Ap_buf.hasSameSize(Ap) || !An_buf.hasSameSize(Ap)) wblog(FL,
584             "ERR %s() severe size mismatch %dx%d <> %dx%d <> %dx%d !?", FCT,
585             Ap_buf.dim1, Ap_buf.dim2, An_buf.dim1, An_buf.dim2, Ap.dim1, Ap.dim2
586         );
587         for (s=0; s<n; ++s) { if (fac[s]>1) { x=1/fac[s]; i1=2*s; i2=2*s+1;
588             Ap_buf.scaleRow(i1,x); An_buf.scaleRow(i1,x);
589             Ap_buf.scaleRow(i2,x); An_buf.scaleRow(i2,x);
590         } }
591     }
592
593     if (!AP.isEmpty() || !AN.isEmpty()) {
594         if (AP.SIZE.len!=3 || !AP.hasSameSize(AN) || AP.SIZE[1]!=2*n)
595             wblog(FL,
596                 "ERR %s() size mismatch (%s; %s; %dx%d) !?", FCT,
597                 AP.sizeStr().data, AN.sizeStr().data, Ap.dim1, Ap.dim2
598             );
599
600         TS *ap=AP.data, *an=AN.data;
601         unsigned n2=2*AP.SIZE[0]; m=n*AP.SIZE[2];
602
603         for (i1=0; i1<m; ++i1, ap+=n2, an+=n2) { s=i1%n;
604             if (fac[s]>1) { x=1/fac[s];
605                 Wb::timesRange(ap, x, n2);
606                 Wb::timesRange(an, x, n2);
607             }
608         }
609     }
610
611     if (!A0.isEmpty() || !mspec.isEmpty()) { wblog(FL,
612         "WRN IROPfac yet not applied to A0 and mspec (%d,%d)",
613         A0.isEmpty(), mspec.isEmpty());
614     wblog(FL, " * A0: %dx%d", A0.dim1, A0.dim2);
615     wblog(FL, " * sp: %dx%d", mspec.dim1, mspec.dim2);
616 }
617 };
618
619 template <class TS>
620 void Spectral<TS>::getRawData(
621     wvector<double> &om, wMatrix<TS> &A, unsigned linfit,
622     const char *isbuf
623 ) const {
624     unsigned i,s,Nom=Ap.dim2;
625
626     getOmega(om, linfit);
627
628     A.init(Ap.dim1, 2*Nom);
629
630     if (A.dim2!=om.len || !Ap.hasSameSize(An)) wblog(FL,
631         "ERR severe dimension mismatch (%d,%d).", A.dim2, om.len);
632
633     if (isbuf && isbuf[0]) {
634         if (strcmp(isbuf,"buf"))
635             wblog(FL, "ERR invalid flag '%s'", isbuf);
636
637         if (!Ap.hasSameSize(Ap_buf)) {
638             if (Ap_buf.isEmpty())
639                 wblog(FL, "ERR buffer not set !?"); else
640                 wblog(FL, "ERR severe size mismatch !?");
641         }
642
643         for (s=0; s<Ap.dim1; s++) { A(s,Nom)=NAN;
644             for (i=0; i<Nom; ++i) {
645                 A(s,Nom-i-1) = An_buf(s,i);
646                 A(s,Nom+i) = Ap_buf(s,i);
647             }
648         }
649     }
650     else {
651         for (s=0; s<Ap.dim1; s++) { A(s,Nom)=NAN;
652             for (i=0; i<Nom; ++i) {
653                 A(s,Nom-i-1) = An(s,i);
654                 A(s,Nom+i) = Ap(s,i);
655             }
656         }
657     }
658
659     if (linfit) {
660         for (s=0; s<A.dim1; s++) {
661             Wb::set2avg(A.rec(s)+Nom, linfit);
662             Wb::set2avg(A.rec(s)+Nom-linfit, linfit);
663         }
664     }
665 };
666
667 template <class TS>

```

```

668 void Spectral<TS>::pairRawData() {
669     unsigned i,j,k, m=Ap.dim1, n=Ap.dim2;
670
671     if (!Ap.hasSameSize(An)) wblog(FL, "ERR severe size mismatch!");
672     if (Ap.dim1%2) wblog(FL,
673 "ERR Ap has odd number of rows !? (%d)", Ap.dim1);
674
675     for (i=0; i<m; i+=2)
676         for (k=i/2, j=0; j<n; j++) {
677             Ap(k,j)=Ap(i,j)+Ap(i+1,j);
678             An(k,j)=An(i,j)+An(i+1,j);
679         }
680
681     Ap.Resize(m/2,n);
682     An.Resize(m/2,n);
683
684     if (mspec.dim1==0) return;
685
686     m=mspec.dim1; n=mspec.dim2;
687
688     if (m%2) wblog(FL,
689 "ERR mspec has odd number of rows !? (%d)", m);
690
691     for (i=0; i<m; i+=2)
692         for (k=i/2, j=0; j<n; j++) mspec(k,j)=mspec(i,j)+mspec(i+1,j);
693
694     mspec.Resize(m/2,n);
695 }
696
697 template <class TS>
698 void Spectral<TS>::getPARTIAL(
699     wbararray<TS> &A1, wbararray<TS> &A2
700 ) const {
701
702     if (AP.isEmpty() && AN.isEmpty()) {
703         A1.init(); A2.init(); return;
704     }
705
706     if (AP.SIZE.len!=3 || AP.SIZE!=AN.SIZE || AP.SIZE[1]%2) wblog(FL,
707 "ERR %s() invalid initialization of AP/AN [%s; %s; %dx%d]",
708 FCT,AP.sizeStr().data, AN.sizeStr().data, Ap.dim1, Ap.dim2
709 );
710
711     int i,j, m=AP.SIZE[1]/2, n=AP.SIZE[0], K=AP.SIZE[2], n1=n-1, n2=2*n*m;
712     const TS *ap=AP.data, *an=AN.data;
713     TS *a1, *a2;
714
715     A1.init(2*n,m,K); a1=A1.data;
716     A2.init(2*n,m,K); a2=A2.data;
717
718     for (K*=m, j=0; j<K; ++j) {
719         if (j>=m) {
720             for (i=0; i<n; ++i) { a1[i]=an[n1-i] - an[n1-i-n2]; }; a1+=n; an+=n;
721             for (i=0; i<n; ++i) { a1[i]=ap[ i] - ap[ i-n2]; }; a1+=n; ap+=n;
722             for (i=0; i<n; ++i) { a2[i]=an[n1-i] - an[n1-i-n2]; }; a2+=n; an+=n;
723             for (i=0; i<n; ++i) { a2[i]=ap[ i] - ap[ i-n2]; }; a2+=n; ap+=n;
724         }
725         else {
726             for (i=0; i<n; ++i) { a1[i]=an[n1-i]; }; a1+=n; an+=n;
727             for (i=0; i<n; ++i) { a1[i]=ap[ i]; }; a1+=n; ap+=n;
728             for (i=0; i<n; ++i) { a2[i]=an[n1-i]; }; a2+=n; an+=n;
729             for (i=0; i<n; ++i) { a2[i]=ap[ i]; }; a2+=n; ap+=n;
730         }
731     }
732 };
733
734 template <class TS>
735 void Spectral<TS>::detailedBalance(
736     const double T,
737     wbvector<double> &om, wbMatrix<TS> &aa
738 ) const {
739
740     unsigned i,j,k1,k2,m,n;
741     wbvector<double> f1,f2;
742     wbMatrix<TS> ar;
743
744     if (T<0) wblog(FL,"ERR T=%g !?", T);
745
746     getRawData(om,ar);
747
748     n=Ap.dim2; f1.init(2*n);
749
750     if (T==0) {
751         for (i=0; i<n; i++) f1[i]=1;
752     }
753     else {
754         double beta=1./T;

```



```

755     for (i=0; i<om.len; i++) f1[i]=1./(1.+exp(-beta*om[i]));
756 }
757
758 f1.flip(f2);
759
760 m=Ap.dim1; n=ar.dim2;
761 aa.init(2*m,n);
762
763 for (i=0; i<m; i++) {
764     k1=2*i; k2=k1+1;
765
766     for (j=0; j<n; j++) {
767         aa(k1,j)=ar(i,j)*f2[j];
768         aa(k2,j)=ar(i,j)*f1[j];
769     }
770 }
771 }
772
773 template <class TS>
774 void Spectral<TS>::dispSumRule() {
775
776     wbvector<TS> sp=Ap.recSum(), sn=An.recSum();
777     unsigned i; double w=0;
778
779     wblog(FL, "\r%60s\n"
780           "      sum %sp_raw   : %s\n"
781           "      + %sn_raw    : %s ", "",
782           istr.data, sp.toStrf("%8.5f").data,
783           istr.data, sn.toStrf("%8.5f").data
784     );
785     sp+=sn;
786
787     if (!A0.isEmpty()) {
788         if (A0.dim1!=sp.len) wblog(FL,
789             "ERR size mismatch (%d/%dx%d)", sp.len, A0.dim1, A0.dim2);
790         sp+=A0.recSum();
791     }
792
793     if (sp.len) { w=REAL(sp[0]); }
794     for (i=0; i<sp.len; ++i) { if (ABS(sp[i]-w)>1E-2) break; }
795
796     if (i==sp.len && fabs(w-round(w))<1E-4) { sp-=w;
797         wblog(FL, "\r%60s\r      \e[34m\\", "");
798         printf("total%-7g: %s", -w, sp.toStrf("%8.2g").data);
799         printf("\e[0m\n");
800     }
801     else {
802         printf("      total      : ");
803         for (i=0; i<sp.len; ++i) { w=round(REAL(sp[i]));
804             if (ABS(sp[i]-w)<1E-5) {
805                 if (w!=0)
806                     printf("%2g%+5.0E ", w, REAL(sp[i])-w);
807                 else printf("%8.1E ", REAL(sp[i]));
808             }
809             else { printf("%8.5f ", REAL(sp[i])); }
810         }; printf("\n");
811     }
812 };
813
814 template <class TS>
815 void Spectral<TS>::dispSumRule(
816     const wbvector<double> &om, const wbMatrix<TS> &A
817 ) {
818     wbvector<TS> s(A.dim1);
819     unsigned i; double m=0;
820
821     if (om.len!=A.dim2) wblog(FL,
822         "ERR Dimension mismatch (%d,%d)", om.len, A.dim2);
823
824     for (unsigned i=0; i<A.dim1; i++)
825         s[i] = Wb::IntTrapez(om.data, A.rec(i), om.len);
826
827     wblog(FL, "\r%60s\r      "
828           "int AA(om)   : %s", "", s.toStrf("%8.5f").data);
829
830     if (!A0.isEmpty()) {
831         if (A0.dim1!=s.len) wblog(FL,
832             "ERR size mismatch (%d/%dx%d)", s.len, A0.dim1, A0.dim2);
833         s+=A0.recSum();
834     }
835
836     if (s.len) {
837         for (m=round(REAL(s[0])), i=0; i<s.len; ++i)
838             if (ABS(s[i]-m)>1E-2) return;
839     }; s-=m;
840
841     wblog(FL, "\r%60s\r      * \e[34m"

```

```

842     "total%+3g: %s\e[0m", "", -m, s.toStrf("%8.2g").data);
843 };
844
845 template <class TS>
846 void Spectral<TS>::getSmoothSpec(
847     wbvector<double> &om, wbMatrix<TS> &A, char vflag
848 ){
849     unsigned linfit=0;
850
851     getOmega(om, linfit);
852     getSmoothSpec(A, om, vflag);
853 };
854
855 template <class TS>
856 void Spectral<TS>::getSmoothSpec(
857     wbMatrix<TS> &A, const wbvector<double> &omega,
858     char vflag
859 ){
860     unsigned i, s, Nom=Ap.dim2;
861     wbvector<double> OM(Nom), I;
862     double x, dbl, om, aom;
863     TS w1=0., w2=0.;
864
865     if (fac<=0 || sigma<=0 || sigma2<=0 || eps<=0 || Nom<4)
866         wblog(FI, "ERR %s()\n"
867             "fac=%3g, sigma=%3g, sigma2=%3g, eps=%3g, Nom=%d !?",
868             FCT, fac, sigma, sigma2, eps, Nom
869         );
870
871     for (i=0; i<OM.len; i++) OM[i]=idx2om(i);
872
873     if (vflag) {
874         double w1=omega.min(), w2=omega.max(), w0=omega.aMin();
875         if (w1==w2)
876             sprintf(str, "%3g .. %3g", w0, w2);
877         else sprintf(str, "%3g .. %3g .. %3g", w1, w0, w2);
878
879         wblog(FI, " * "
880             "%s() %g bins/dec (len=%d)\n"
881             "sigma=%g [eps=%3g @ %3g]\n"
882             "|omega| range: %s",
883             myname, fac/log10(M_E), Nom, sigma, eps, sigma2, str);
884         if (fabs((w0-idx2om(0))/w0)>0.01 ||
885             fabs((w2-idx2om(Nom-1))/w2)>0.01) wblog(FI,
886             " |emin...emax|= %3g .. %3g (bins)", idx2om(0), idx2om(Nom-1));
887         if (vflag && (Ap.dim1<8 || vflag>1)) { dispSumRule(); }
888     }
889
890     A.init(Ap.dim1, omega.len);
891
892     for (s=0; s<Ap.dim1; ++s)
893         for (i=0; i<omega.len; ++i) { om=omega[i]; aom=fabs(om);
894
895             if (aom>eps) x=1.;
896             else {
897                 if (aom==0.) x=0.;
898                 else {
899                     dbl = (log(aom)-log(eps))/sigma;
900                     x = exp(-dbl+dbl);
901                     if (x<1E-10) x=0.;
902                 }
903             }
904
905             if (x!=0.)
906                 w1 = foldLogGaussian(aom, om>0 ? Ap.rec(s) : An.rec(s), Nom);
907
908             if (x!=1.) {
909                 double b=MAX(sigma2*eps, om*exp(-sigma*sigma/4));
910
911                 w2 = foldGaussian( om, b, OM.data, Ap.rec(s), Nom)
912                     + foldGaussian(-om, b, OM.data, An.rec(s), Nom);
913             }
914
915             A(s,i) = x*w1 + (1.-x)*w2;
916         }
917
918     if (vflag && (Ap.dim1<8 || vflag>1)) dispSumRule(omega, A);
919 };
920
921 template <class TS>
922 TS Spectral<TS>::foldLogGaussian(
923     const double om, const TS *aa, const unsigned n
924 ){
925     double k, dbl, efac;
926     TS xa=0.;
927
928     if (sigma<=0. || fac<=0.) wblog(FI,

```

```

929     "ERR sigma=%g, fac=%g !?", sigma, fac);
930
931     k=OM2IDX(om)-0.5;
932     efac=1./(fac*sigma);
933
934     double dx=alpha*sigma;
935
936     for (unsigned i=0; i<n; ++i) {
937         dbl=efac*(k-double(i))-dx;
938         xa += aa[i]*exp(-dbl*dbl);
939     }
940     xa /= (sigma*sqrt(M_PI)*om);
941
942     if (alpha!=0.25) {
943         xa *= exp(+sigma*sigma*(alpha-0.25));
944     }
945
946     return xa;
947 };
948
949 template <class TS>
950 TS Spectral<TS>::foldGaussian(
951     const double om, const double b,
952     const double *oo, const TS *aa, const unsigned n
953 ){
954     double dbl, efac;
955     TS xa=0.;
956
957     if (b<=0.) wblog(FL, "ERR b=%g !?", b);
958
959     efac = 1./b;
960
961     for (unsigned i=0; i<n; i++) {
962         dbl=efac*(om-oo[i]);
963         xa += aa[i]*exp(-dbl*dbl);
964     }
965
966     xa *= 1. / (b*sqrt(M_PI));
967
968     return xa;
969 };
970
971 template <class TS>
972 void Spectral<TS>::KKreal(
973     wbMatrix<wbcomplex> &GZ, wbvector<double> &OM, wbvector<double> &DOM
974 ) const {
975
976     unsigned i,j,s, d=Ap.dim1, Nom;
977     wbMatrix<double> II, RR;
978     double doi, doj;
979
980     getRawData(OM, II); Nom=OM.len;
981     Wb::getDiff(OM,DOM);
982
983     #if 0
984     wblog(FL,"TST Get smooth data ...");
985
986     getSmoothSpec(OM, II, 1E-6, 0.3, 0); Nom=OM.len;
987     Wb::getDiff(OM,DOM);
988
989     for (s=0; s<d; s++)
990     for (i=0; i<Nom; i++) II(s,i) *= DOM[i];
991     #endif
992
993     RR.init(d,Nom);
994
995     for (i=0; i<Nom; ++i)
996     for (j=i+1; j<Nom; ++j) {
997
998         doi=DOM[i]/(OM[j]-OM[i]);
999         doj=DOM[j]/(OM[i]-OM[j]);
1000
1001         for (s=0; s<d; s++) {
1002             RR(s,i) += II(s,j)*doi;
1003             RR(s,j) += II(s,i)*doj;
1004         }
1005     }
1006
1007     #if 0
1008     wbMatrix<double> Ip2, In2;
1009     wblog(FL,"TST backtransform real->imag");
1010
1011     RR*=(1/M_PI); RR.put("RR","base",'r');
1012
1013     KKimag(Rp,Rn,OM,Ip2,In2);
1014
1015     Ip2.put("Ip","base",'r');

```

```

1016     In2.put("In", "base", 'r');
1017
1018     wblog(FL, "ERR");
1019 #endif
1020
1021     II *= M_PI;
1022
1023     GZ.set(RR, II);
1024 }
1025
1026 void KKimag(
1027     const wbMatrix<double> &RR,
1028     const wbvector<double> &OM,
1029     wbMatrix<double> &II
1030 ){
1031     unsigned i, j, s, d=RR.dim1, Nom=RR.dim2;
1032     double doi, doj;
1033     wbvector<double> DOM;
1034
1035     Wb::getDiff(OM, DOM); II.init(d, Nom);
1036
1037     if (OM.len!=Nom) wblog(FL,
1038         "ERR severe dimension mismatch (%d,%d) !?", OM.len, Nom);
1039
1040     for (i=0; i<Nom; ++i)
1041     for (j=i+1; j<Nom; ++j) {
1042         doi=DOM[i]/(OM[i]-OM[j]); doj=DOM[j]/(OM[j]-OM[i]);
1043         for (s=0; s<d; s++) {
1044             II(s,i) += RR(s,j)*doi;
1045             II(s,j) += RR(s,i)*doj;
1046         }
1047     }
1048
1049     II *= (1./M_PI);
1050 }
1051
1052 template <class TS>
1053 void TDSpectral<TS>::init(const char *F, int L, const char *istr0,
1054     const wbvector<double> &t0,
1055     unsigned Nnrg, unsigned Nop,
1056     double Emin, double Emax, unsigned nl, char vflag
1057 ){
1058     double l2;
1059
1060     if (Emin<=0. || Emax<=0. || Emin>=0.1*Emax || nl==0) wblog(F, L,
1061         "ERR invalid parameter set [%g %g %d]s", Emin, Emax, nl,
1062         (Emin>0 && Emax>0 && Emin<Emax)? " (require Emin<Emax/10)": "");
1063
1064     l0 = log(Emin);
1065     l2 = log(Emax);
1066
1067     fac= log10(M_E)*double(nl);
1068
1069     N = (unsigned)((l2-l0)*fac)+1;
1070
1071     AP.init(N, Nop, Nnrg);
1072     AN.init(N, Nop, Nnrg); if (check_avg_om) {
1073     AW.init(2, Nop, Nnrg); }
1074
1075     if (vflag) {
1076         wblog(F, L, "%3s TDSpectral: %.2g .. %.2g (%d/dec; 2x%d)",
1077             istr0 ? istr0 : "<i>", Emin, Emax, nl, N);
1078     }
1079
1080     istr=istr0; tt=t0;
1081 };
1082
1083 template <class TS>
1084 wbarray<TS> TDSpectral<TS>::getAvgOm() {
1085
1086     if (AW.rank()!=3 || AW.dim(1)!=2) wblog(FL,
1087         "ERR invalid AW data structure [%s] !?", AW.sizeStr().data);
1088
1089     unsigned i=0, n=AW.numel();
1090     wbarray<TS> aw(AW); if (aw.isEmpty()) return aw;
1091     TS *a=AW.data, *b=aw.data;
1092
1093     for (; i<n; i+=2) { b[i] = ((a[i+1]!=0.) ? a[i]/a[i+1] : a[i]); }
1094
1095     return aw;
1096 };
1097
1098 template <class TS>
1099 void TDSpectral<TS>::Add(
1100     const wbarray<double> &Om,
1101     const wbarray<TS> &CC, unsigned iter, unsigned s
1102 ){

```

```

1103     static int iter_last=-1;
1104     unsigned i,k,n=Om.numel();
1105     TS *ap, *an;
1106
1107     if (!Om.hasSameSize(CC)) {
1108         Om.info("Om"); CC.info("CC");
1109         wblog(FL,"ERR size mismatch between Om and CC!");
1110     }
1111
1112     if (s>=AP.dim(2) || iter>=AP.dim(3)) wblog(FL,
1113         "ERR index out of bounds (op=%d/%d; k=%d/%d)",
1114         s+1,AP.dim(2),iter+1,AP.dim(3));
1115
1116     ap=&AP(0,s,iter);
1117     an=&AN(0,s,iter);
1118
1119     for (i=0; i<n; ++i) {
1120         k=om2idx(Om.data[i]);
1121         if (Om.data[i]<0.)
1122             an[k]+=CC.data[i];
1123         else ap[k]+=CC.data[i];
1124     }
1125
1126     if (raw) {
1127         if (iter_last!=(int)iter) {
1128             wblog(FL,"%NTST iter=%d",iter);
1129             if (RAW.len) addPartialFourier();
1130             RAW.initDef(AP.dim(2));
1131             wblog(FL,"TST done");
1132         }
1133
1134         RAW[s].Append(Om.data, CC.data, n);
1135
1136         iter_last=iter;
1137     }
1138
1139     if (check_avg_om) {
1140         if (AW.rank()!=3 || AW.dim(1)!=2 ||
1141             AW.dim(2)!=AP.dim(2) || AW.dim(3)!=AP.dim(3)) wblog(FL,
1142             "ERR size mismatch for AW [%s] ? [%s]",
1143             AW.sizeStr().data, AP.sizeStr().data
1144         );
1145         AW(0,s,iter)+=CC.sumTimesEl(Om);
1146         AW(1,s,iter)+=CC.sum();
1147     }
1148 };
1149
1150 template <class TS> inline
1151 void TDSpectral<TS>::addPartialFourier() {
1152
1153     if (!RAW.len) return;
1154
1155     unsigned s=0, m=RAW.len;
1156     wbMatrix<wbcomplex> at;
1157     wbMatrix<TS> aa;
1158
1159     if (m!=AP.dim(2)) wblog(FL,
1160         "ERR severe size mismatch (%d/%d)",m,AP.dim(2));
1161
1162     if (At.isEmpty()) At.init(AP.dim(2),tt.len);
1163
1164     for (s=0; s<m; ++s) {
1165         const RSpecData<TS> &R=RAW[s];
1166         if (R.om.len!=R.cc.len) wblog(FL,
1167             "ERR RAW[%d].len=%d/%d !?",s,R.om.len,R.cc.len);
1168
1169         aa.init2ref(1,R.cc.len,R.cc.data);
1170
1171         Wb::Fourier(R.om,aa,tt,at);
1172         At.recAddP(s,at.data);
1173     }
1174
1175     RAW.init();
1176 };
1177
1178 template <class TS>
1179 void TDSpectral<TS>::getOmega(
1180     wbvector<double> &om, unsigned &linfit
1181 ) const {
1182
1183     unsigned i;
1184     double dbl;
1185
1186     if (linfit) linfit=unsigned(fac);
1187     if (linfit>N) wblog(FL,"ERR linfit>N (%d,%d) !?", N, linfit);
1188
1189     om.init(2*N);

```

```

1190
1191     dbl = idx2om(linfit) / (linfit+0.5);
1192
1193     for (i=0; i<linfit; i++) om[N-1-i] = -(om[N+i] = dbl*(i+0.5));
1194     for (    ; i<N;      i++) om[N-1-i] = -(om[N+i] = idx2om(i) );
1195 }
1196
1197 template <class TS>
1198 void TDSpectral<TS>::applyIROPfac(
1199     const char *F, int L, const wbvector<double> &fac, char rixs) {
1200
1201     unsigned i,k=0, n=fac.len;
1202     double x;
1203
1204     for (i=0; i<n; ++i) { if (fac[i]!=1) {
1205         if (fac[i]<1 || fac[i]!=round(fac[i])) wblog(F,
1206             "ERR %s() got fac[%d]=%g !?",FCT,i+1,fac[i]);
1207         else {
1208             wblog(F, " * applying IROP factor 1/%g%s to op(%d)",
1209                 fac[i], rixs? "^2":"",i+1);
1210             ++k;
1211         }
1212     } }
1213     if (!k) return;
1214
1215     if (!Ap.isEmpty() || !An.isEmpty()) {
1216         if (n!=Ap.dim1 || n!=An.dim1) wblog(F,
1217             "ERR %s() size mismatch (%d,%d/2*%d)",FCT,Ap.dim1,An.dim1,n);
1218
1219         for (i=0; i<n; ++i) {
1220             if (fac[i]!=1) { x=1/fac[i]; if (rixs) { x*=x; }
1221                 Ap.scaleRow(i,x);
1222                 An.scaleRow(i,x);
1223             }
1224         }
1225     }
1226
1227     if (!AP.isEmpty() || !AN.isEmpty()) {
1228         if (AP.SIZE.len!=3 || !AP.hasSameSize(AN) || AP.SIZE[1]!=n)
1229             wblog(F, "ERR %s() size mismatch (%s; %s /%d) !?",
1230                 FCT, SSTR(AP), SSTR(AN), n);
1231
1232         TS *ap=AP.data, *an=AN.data;
1233         unsigned n0=AP.SIZE[0], m=n*AP.SIZE[2];
1234
1235         for (i=0; i<m; ++i, ap+=n0, an+=n0) { k=i%n;
1236             if (fac[k]>1) { x=1/fac[k]; if (rixs) { x*=x; }
1237                 Wb::timesRange(ap, x, n0);
1238                 Wb::timesRange(an, x, n0);
1239             }
1240         }
1241     }
1242 };
1243
1244 template <class TS>
1245 void TDSpectral<TS>::getSpecData(
1246     wbvector<double> &om, wbarray<TS> &A,
1247     unsigned linfit
1248 ) const {
1249     unsigned i,r,s, d1=AP.dim(2), d2=AP.dim(3);
1250
1251     getOmega(om,linfit);
1252     A.init(2*N,d1,d2);
1253
1254     if (om.len!=2*N || !AP.hasSameSize(AN)) wblog(F,
1255         "ERR severe dimension mismatch (%d,%d).",2*N,om.len);
1256
1257     for (r=0; r<d1; r++)
1258     for (s=0; s<d2; s++) {
1259         for (i=0; i<N; i++) {
1260             A(N-i-1,r,s) = AN(i,r,s);
1261             A(N+i    ,r,s) = AP(i,r,s);
1262         }
1263     }
1264
1265     if (linfit) {
1266         for (r=0; r<d1; r++)
1267         for (s=0; s<d2; s++) {
1268             Wb::set2avg(A.ref(r,s)+N,      linfit);
1269             Wb::set2avg(A.ref(r,s)+N-linfit, linfit);
1270         }
1271     }
1272 };
1273
1274 template <class TS>
1275 void TDSpectral<TS>::getSpecData(
1276     wbvector<double> &om, wbMatrix<TS> &A,

```

```

1277     unsigned linfit
1278 }{
1279
1280     unsigned i,r;
1281
1282     getOmega(om,linfit);
1283
1284     if (Ap.isEmpty()) mergeNRGData();
1285     A.init(Ap.dim1,2*N);
1286
1287     if (A.dim2!=om.len || !Ap.hasSameSize(An)) wblog(FL,
1288         "ERR Severe dimension mismatch (%d,%d).",A.dim2,om.len);
1289
1290     for (r=0; r<A.dim1; r++)
1291     for (i=0; i<N; i++) {
1292         A(r,N-i-1) = An(r,i);
1293         A(r,N+i) = Ap(r,i);
1294     }
1295
1296     if (linfit) {
1297         for (r=0; r<A.dim1; r++) {
1298             Wb::set2avg(A.rec(r)+N, linfit);
1299             Wb::set2avg(A.rec(r)+N-linfit, linfit);
1300         }
1301     }
1302 }
1303
1304 template <class TS>
1305 void TDSpectral<TS>::Fourier(
1306     wbvector<double> &om,
1307     wbMatrix<wbcomplex> &ST,
1308     double alpha, double Lambda, char vflag
1309 ){
1310     unsigned k,it,ik,is,io,l=0,no, nt=tt.len, ns=AP.dim(2), nk=AP.dim(3);
1311
1312     wbararray<wbcomplex> TKS(nt,nk,ns), KST;
1313     wbMatrix<double> aTK(nt,nk);
1314     wbararray<TS> AR;
1315     double ek,oi,*at; TS kso,*ap;
1316     wbcomplex z,*st;
1317
1318     if (vflag) wblog(FL,
1319         "<i> %s (damping alpha=%g, Lambda=%g)",FCT,alpha,Lambda);
1320
1321     getSpecData(om,AR);
1322     ap=AR.data; no=om.len;
1323
1324     if (alpha==0) aTK.set(1); else
1325     for (ik=0; ik<nk; ++ik) {
1326         ek=pow(Lambda, -double(ik)/2.) * 0.5*(Lambda+1);
1327         for (it=0; it<nt; it++)
1328             aTK(it,ik)=exp(-fabs(alpha*ek*tt[it]));
1329     }
1330
1331     for (ik=0; ik<nk; ik++)
1332     for (is=0; is<ns; is++)
1333     for (io=0; io<no; io++, l++) if (ap[l]!=0.) { kso=ap[l]; oi=om[io];
1334         for (it=0; it<nt; it++)
1335             TKS(it,ik,is)+=kso*expi(oi*tt[it]);
1336     }
1337
1338     TKS.permute(KST,"2 3 1");
1339     ST.init(ns,nt);
1340
1341     for (is=0; is<ns; ++is)
1342     for (it=0; it<nt; ++it) {
1343         st=KST.ref(is,it); z=0;
1344         if (alpha==0)
1345             for (k=0; k<nk; ++k) z+=st[k];
1346         else for (at=aTK.rec(it), k=0; k<nk; ++k) z+=at[k]*st[k];
1347         ST(is,it)=z;
1348     }
1349
1350 };
1351
1352 template <class TS>
1353 void TDSpectral<TS>::Fourier(
1354     wbvector<double> &om, wbMatrix<TS> &A,
1355     const wbvector<double> &tt,
1356     wbMatrix<wbcomplex> &at,
1357     double alpha, char vflag
1358 ){
1359     if (Ap.isEmpty()) { mergeNRGData(); dispSumRule(); }
1360     getSpecData(om,A);
1361
1362     Wb::Fourier(om,A,tt,at,0,alpha,vflag);
1363 };

```

```

1364
1365 template <class TS>
1366 void Wb::Fourier(
1367     const wbvector<double> &om, const wbMatrix<TS> &A,
1368     const wbvector<double> &tt,
1369     wbMatrix<wbcomplex> &az,
1370     char domflag,
1371     double alpha,
1372     char vflag
1373 ){
1374     unsigned i,j,l,m,n,b1,b2, nom, nt=tt.len, D=512, stop=0;
1375     wbMatrix<wbcomplex> ez;
1376     wbcomplex z;
1377     double f,ot;
1378
1379     wbvector< wbMatrix<wbcomplex> > AZ;
1380     wbvector<double> dom(om.len);
1381
1382     m=A.dim1; nom=om.len;
1383
1384     if (nom!=A.dim2) wblog(FL,
1385         "ERR %s - size mismatch (%d/%d)",nom,A.dim2);
1386     if (!tt.len || !m || !nom) {
1387         wblog(FL,"WRN Empty data [%d %d %d] - return",tt.len,m,nom);
1388         return;
1389     }
1390     if (nom<2) wblog(FL,"ERR %s - omega too short (%d/2)",nom);
1391
1392     n=om.len-1;
1393     for (i=1; i<n; i++) dom[i]=0.5*(om[i+1]-om[i-1]);
1394     dom[0]=om[1]-om[0]; dom[n]=om[n]-om[n-1];
1395
1396     for (i=0; i<dom.len; i++) if (dom[i]<=0) { wblog(FL,
1397         "WRN omega not strictly ascending !?"); break; }
1398
1399     for (i=0; i<tt.len; i++) if (tt[i]<0) { wblog(FL,
1400         "WRN negative tt values !?"); break; }
1401
1402     if (domflag) {
1403         wbMatrix<double> B(A);
1404
1405         for (i=0; i<B.dim1; ++i)
1406             for (j=0; j<B.dim2; ++j) B(i,j)*=dom[j];
1407
1408         Wb::Fourier(om,B,tt,az,0,alpha,vflag);
1409         return;
1410     }
1411
1412     if (vflag) wblog(FL," * %s() spectral data "
1413         "(alpha=%4g, %s)",FCT,alpha, domflag? "func":"disc");
1414
1415     AZ.initDef(nt<D ? 1 : nt/D);
1416
1417     for (l=b1=0; !stop; l++,b1+=D) {
1418         b2=b1+D-1; if (b2+D>=nt) { b2=nt-1; stop=1; }
1419         n=b2-b1+1;
1420
1421         if (n*nom*sizeof(double)/(1<<20)>900) {
1422             wblog(FL,"WRN trying to allocate %.6g MB !?",
1423                 n*nom*sizeof(double)/double(1<<20));
1424         }
1425
1426         ez.init(nom,n);
1427         for (i=0; i<n; i++)
1428             for (j=0; j<n; j++) { ot=om[i]*tt[b1+j];
1429                 ez(i,j).init_expi(ot);
1430
1431                 if (alpha>0) {
1432                     f=:exp(-:fabs(alpha*dom[i]*tt[b1+j]));
1433                     ez(i,j)*=f;
1434                 }
1435             }
1436         Wb::MatProd(A,ez,AZ[l]);
1437     }
1438
1439     az.CAT(2,AZ);
1440 };
1441
1442 template <class TS>
1443 void TDSpectral<TS>::mergeNRGData() {
1444
1445     unsigned i,k, d1=AP.dim(1), d2=AP.dim(2), d3=AP.dim(3);
1446     unsigned s=d1*d2, S=s*d3;
1447     TS *Dp,*Dn,*dp,*dn;
1448
1449     if (!AP.hasSameSize(AN)) wblog(FL, "ERR severe size mismatch!");
1450

```



```

1451   Ap.init(d2,d1); dp=Ap.data; Dp=AP.data;
1452   An.init(d2,d1); dn=An.data; Dn=AN.data;
1453
1454   for (i=0; i<S; i++) { k=i%s;
1455       dp[k]+=Dp[i];
1456       dn[k]+=Dn[i];
1457   }
1458 };
1459
1460 template <class TS>
1461 void TDSpectral<TS>::dispSumRule() {
1462
1463     unsigned i=0; double m=0;
1464     wbvector<TS> sp,sn;
1465     wbararray<TS> x;
1466
1467     AP.sum("1,3",x); sp.init(x.numel(),x.data);
1468     AN.sum("1,3",x); sn.init(x.numel(),x.data);
1469
1470     wblog(FL," * om>0: %s\nom<0: %s",
1471           sp.toStrf("%8.5f").data, sn.toStrf("%8.5f").data);
1472
1473     sp+=sn;
1474
1475     if (sp.len) {
1476         m=ROUND(REAL(sp[0]));
1477         for (i=0; i<sp.len; i++) if (ABS(sp[i]-m)>1E-2) break;
1478     }
1479
1480     if (i==sp.len) { sp-=TS(m); wblog(FL,
1481 " * \e[34mtotal %s\e[0m (%g)", sp.toStrf("%8.2g").data, m); }
1482     else wblog(FL,
1483 " * \e[34mtotal %s\e[0m",      sp.toStrf("%8.5f").data);
1484 };
1485
1486 template <class TS>
1487 void Wb::dispSumRule(
1488     const wbvector<double> &om,
1489     const wbMatrix<TS> &A
1490 ){
1491     wbvector<double> s(A.dim1);
1492     unsigned i; double m=0;
1493
1494     if (om.len!=A.dim2) wblog(FL,
1495 "ERR Dimension mismatch (%d,%d)", om.len, A.dim2);
1496
1497     for (unsigned i=0; i<A.dim1; i++)
1498         s[i] = Wb::IntTrapez(om.data, A.rec(i), om.len);
1499
1500     wblog(FL,"SUM %s", s.toStrf("%8.5f").data);
1501
1502     if (s.len) {
1503         for (m=ROUND(s[0]), i=1; i<s.len; i++)
1504             if (ABS(s[i]-m)>1E-2) return;
1505     }
1506
1507     s-=m; wblog(FL,
1508 "\e[34m * total-%g      = %s\e[0m", m, s.toStrf("%8.2g").data);
1509 }
1510
1511 template <class TS>
1512 void TDSpectral<TS>::getSmoothSpec_t(
1513     double sigma, double eps, char bswitch, char vflag
1514 ) {
1515     unsigned i,s;
1516     wbMatrix<double> Ap0(Ap), An0(An), wlg;
1517
1518     if (fac<=0 || sigma<=0 || N<4) wblog(FL,
1519 "ERR invalid paramters (fac=%g, sigma=%g, N=%d)",fac,sigma,N);
1520     if (fac*sigma<1) wblog(FL,
1521 "WRN sigma < discrete log spacing (%.3g / %.3g)",sigma,1/fac);
1522     if (vflag) wblog(FL,
1523 "<i> %s() %dx%d bins (sigma=%g)",myname,AP.dim(2),N,sigma);
1524
1525     if (Ap.isEmpty()) { mergeNRGData(); };
1526     if (vflag) { dispSumRule(); }
1527
1528     Ap.set(0); An.set(0);
1529     initLG(wlg,N,sigma,bswitch,vflag);
1530
1531     for (s=0; s<Ap.dim1; ++s)
1532         for (i=0; i<Ap.dim2; ++i) {
1533             foldLogGaussian(wlg, Ap.rec(s), Ap0(s,i), i);
1534             foldLogGaussian(wlg, An.rec(s), An0(s,i), i);
1535         }
1536
1537     if (eps) {

```

```

1538     if (vflag>1) {
1539         wblog(FL," * %s() eps=%.3g/%.3g",myname,eps,exp(10)); }
1540
1541     unsigned j,l, i0=0, i1=0, i2=0;
1542     unsigned nE=(1<12);
1543
1544     double eps0=0.05*eps, eps1=1.5*eps, x1=5., xfac=x1/double(nE);
1545     double x=1./sqrt(M_PI), dbl, lfac=1/xfac;
1546
1547     wbvector<double> om(Ap.dim2), dom(om.len);
1548     wbvector<double> gauss_lin(2*nE+1), erf_om(om.len);
1549
1550     if (om.len<10) wblog(FL,"ERR %s() om.len = %d !?",FCT,om.len);
1551     for (i=0; i<om.len; ++i) {
1552         om[i]=idx2om(i);
1553
1554         if (!i0 && i && om[i]>eps0) { i0=i; }
1555
1556         if (!i1 && i && (om[i]-om[i-1])>eps1) { i1=i; }
1557
1558         if (i>1) {
1559             dom[i-1] = 0.5*( om[i] - om[i-2] ); }
1560     }
1561     dom[i-1] = 2*dom[i-2];
1562     dom[0] = 0.5*( om[0] + om[1] );
1563
1564     if (i1) { if (vflag) {
1565         sprintf(str, "(%.1g <) |om| < %.1g (%.0f%%)",om[i0],om[i1],
1566             100*( (i1-i0)/double(om.len) ));
1567         wblog(FL,"--> Gaussian broadening for %s",str);
1568     } }
1569     else {
1570         i1=om.len-1; sprintf(str, "(%.3g; eps=%.3g)",om[i1],eps);
1571         wblog(FL,"WRN Gaussian broadening for all omega %s",str);
1572     }
1573
1574     i2=i1+10; if (i2>om.len) { i2=om.len-1; }
1575
1576     for (i=0; i<erf_om.len; ++i) {
1577         erf_om[i] = (0.5*( std::erf( om[i]/eps ) + 1 ));
1578     }
1579
1580     for (i=0; i<gauss_lin.len; ++i) {
1581         dbl = i*xfac - x1;
1582         gauss_lin[i] = x*exp(-dbl*dbl);
1583     }
1584     dom*=(1/eps);
1585
1586     Ap0=Ap; An0=An;
1587     for (s=0; s<Ap.dim1; ++s)
1588         for (i=0; i<i1; ++i) { Ap(s,i) = An(s,i) = 0; }
1589
1590     for (s=0; s<Ap.dim1; ++s) {
1591         for (x=0., i=0; i<i0; ++i) { x += (Ap0(s,i) + An0(s,i)); }
1592         for (i=0; i<i2; ++i) {
1593             Ap(s,i) += ( dbl = x*(erf_om[i] - (i? erf_om[i-1] : 0.5)) );
1594             An(s,i) += dbl;
1595         }
1596     }
1597
1598     for (s=0; s<Ap.dim1; ++s)
1599         for (i=i0; i<i1; ++i) {
1600             for (j=0; j<i2; ++j) {
1601                 dbl = (om[j]-om[i])/eps;
1602                 if (fabs(dbl)<x1 ) { l = (dbl + x1) * lfac + 0.5;
1603                     if (l<gauss_lin.len) {
1604                         Ap(s,j) += Ap0(s,i) * (dbl = gauss_lin[l] * dom[j]);
1605                         An(s,j) += An0(s,i) * dbl;
1606                     } }
1607
1608                 dbl = (om[j]+om[i])/eps;
1609                 if (fabs(dbl)<x1 ) { l = (dbl + x1) * lfac + 0.5;
1610                     if (l<gauss_lin.len) {
1611                         An(s,j) += Ap0(s,i) * (dbl = gauss_lin[l] * dom[j]);
1612                         Ap(s,j) += An0(s,i) * dbl;
1613                     } }
1614             }
1615         }
1616     }
1617
1618 };
1619
1620 template <class TS>
1621 void TDSpectral<TS>::initLG(
1622     wbMatrix<double> &wlg,
1623     unsigned n, double sigma, char bswitch, char vflag
1624 ){

```

```

1625     unsigned i,j;
1626     double dbl,efac,s=0;
1627
1628     if (sigma<=0. || fac<=0.) wblog(FL,"ERR sigma=%g, fac=%g",sigma,fac);
1629
1630     efac=1./(fac*sigma); wlg.init(2,n);
1631
1632     switch (bswitch) {
1633     case 0:
1634         strcpy(str,"symmetric in log. space");
1635         s=0; break;
1636     case 1:
1637         strcpy(str,"symmetric log-gauss");
1638         s=sigma/2.;
1639         break;
1640     case 2:
1641         strcpy(str,"norm preserving log-gauss");
1642         s=sigma/4.;
1643         break;
1644     default:
1645         wblog(FL,"ERR invalid bswitch=%d",bswitch);
1646     }
1647     if (vflag) wblog(FL,"--> %s() using %s",FCT,str);
1648
1649     for (i=0; i<n; i++) {
1650         dbl=efac*double(i)+s; wlg(0,i)=exp(-dbl*dbl);
1651         dbl=efac*double(i)-s; wlg(1,i)=exp(-dbl*dbl);
1652     }
1653
1654     wlg*=(1./wlg.sum());
1655
1656     for (i=0; i<2; ++i)
1657         for (j=0; j<n; ++j) if (fabs(wlg(i,j))<1E-20) wlg(i,j)=0;
1658 };
1659
1660 template <class TS> inline
1661 void TDSpectral<TS>::foldLogGaussian(
1662     const wbMatrix<double> &wlg,
1663     TS *aa, TS a0, unsigned i0
1664 ){
1665     unsigned i, n=wlg.dim2;
1666     const double *wp=wlg.rec(0), *wn=wlg.rec(1);
1667     double wi, nrm=0.;
1668
1669     for (i=0; i<n; ++i) {
1670         nrm += (i0>i ? wp[i0-i] : wn[i-i0]); }
1671     nrm=a0/nrm;
1672
1673     for (i=0; i<n; ++i) {
1674         wi=( i0>i ? wp[i0-i] : wn[i-i0] );
1675         if (wi) aa[i]+=nrm*wi;
1676     }
1677 };
1678
1679 template <class TS>
1680 mxArray* TDSpectral<TS>::toMx() const {
1681     mxArray *S=mxCreateStructMatrix(1,1,0,NULL);
1682
1683     mxAddField2Scalar(FL,S,"name",istr.toMx());
1684     mxAddField2Scalar(FL,S,"AP", AP.toMx());
1685     mxAddField2Scalar(FL,S,"AN", AN.toMx());
1686     mxAddField2Scalar(FL,S,"fac", numtoMx(fac));
1687     mxAddField2Scalar(FL,S,"l0", numtoMx(l0 ));
1688
1689     if (raw) {
1690         mxAddField2Scalar(FL,S,"RAW",RAW.toMx()); }
1691
1692     return S;
1693 };
1694
1695 template <class TS>
1696 int TDSpectral<TS>::init(const mxArray *S,
1697     double &emin, double &emax, unsigned &nlog
1698 ){
1699     if (!mxIsStruct(S) || !Mx::IsScalar(S) ||
1700         mxGetFieldNumber(S,"name")<0 ||
1701         mxGetFieldNumber(S,"AP" )<0 ||
1702         mxGetFieldNumber(S,"AN" )<0 ||
1703         mxGetFieldNumber(S,"fac" )<0 ||
1704         mxGetFieldNumber(S,"l0" )<0
1705     ) return 1;
1706
1707     istr.init(mxGetField(S,0,"name" ));
1708     AP .init(mxGetField(S,0,"AP"));
1709     AN .init(mxGetField(S,0,"AN"));
1710     mxGetNumber(mxGetField(S,0,"fac"),fac);
1711     mxGetNumber(mxGetField(S,0,"l0"), l0 );

```

```

1712
1713     if (AP.SIZE!=AN.SIZE || AP.isEmpty() || AN.isEmpty()) return 2;
1714     if (fac<=0) return 3;
1715
1716     N=AP.dim(1);
1717
1718     emin=exp(10);
1719     emax=idx2om(N);
1720     nlog=(unsigned)(fac/log10(M_E) + 0.5);
1721
1722     wblog(FL,
1723         "<i> TDSpectral '%s' from RAW data: %dx%d bins [x%d]",
1724         istr.data, AP.dim(1), AP.dim(2), AP.dim(3));
1725
1726     return 0;
1727 };
1728
1729 #endif

```

5.14 util.hh

```

1 #ifndef __WB_UTIL_HH__
2 #define __WB_UTIL_HH__
3
4 /* ----- */
5 namespace Wb {
6 /* ----- */
7
8     int Rational(
9         double &x,          // rational number (returns approximation of any)
10         long &P, long &Q,    // rational representation P/Q (output)
11         double *rerr=NULL,  // relative error
12         unsigned *nmax=NULL, // max number of iterations actually used
13         wbvector<double>* aa=NULL, // continued fraction sequence
14         unsigned niter=6,    // (max) number of iterations
15         long pqmax=-1,       // max enumerator/denominator (<0: no limit)
16         double eps=1E-6,    // threshold for acceptance within continued fraction
17         double eps2=1E-12,   // overall threshold for acceptance
18         char vflag=0
19     );
20
21     template<class T>
22     double FixRational(
23         const char *F, int L, T *d, unsigned n, char rflag=0,
24         unsigned niter=6,
25         long pqmax=-1,
26         double eps=1E-6, double eps2=1E-12,
27         double *rz=NULL, double *ra=NULL, char vflag=0
28     );
29
30     template<class T>
31     double SkipZeros(T *d, unsigned n, double eps=1E-14){ return 0; };
32
33     void ResSummary(const char *F, int L, const char *istr=NULL);
34
35 #ifndef __WB_MEM_TRACK_HH__
36     void MemStat(const char *F=0, int L=0, char lflag=0);
37 #else
38     void MemStat(const char *F, int L, char lflag);
39 #endif
40
41 #ifndef __APPLE__
42     long getCpuInfo(const char *tag, const char *f="/proc/cpuinfo");
43     size_t getProcSize(const char *tag, const char *f="/proc/meminfo");
44 #endif
45     size_t getProcSize(const char *tag, const pid_t &p);
46     size_t getProcSize(const char *tag, const char *f);
47
48     wbstring hostname(unsigned len=16);
49     wbstring hostid(char pflag=0);
50
51     wbstring size2Str(double n, char p=4);
52     const char* filename(const char *s);
53
54     size_t getFileSize(const char* f);
55     double getFileInfo(const char* f, char w='m');
56
57     bool fexist(const char *f, char type='*');
58     bool isFile(const char *f);
59     bool isDir(const char *f);
60
61     int mkdir(
62         const char *F, int L, const char *path,

```

```

63     char priv=0
64 ) {
65     mode_t p=(S_IRWXU
66             | S_IXGRP
67             | S_IXOTH
68     );
69     if (!priv) p |= (
70             S_IRGRP
71             | S_IROTH
72     );
73
74     int e;
75
76     #pragma omp critical (manage_WB_FILE_IO)
77     { e::mkdir(path,p); }
78
79     if (e && F) wblog(F_L,
80         "ERR %s() failed to create directory (e=%d)\n%s",FCT,e,path);
81     return e;
82 };
83
84 template <class TI>
85 TI sub2ind(const TI *s, const TI *I, unsigned n);
86
87 template <class T1, class T2> inline
88 void ind2sub(T1 k, const T1 *s, T2 *I, unsigned n);
89
90 int atoi(const char *s, int &iout, unsigned n, char white=0);
91
92 int isUIntString(const char *s, char white=1);
93
94 template <class T>
95 size_t findfirst_sorted(const char *F, int L,
96     const T* d0, const T* dd, size_t m, size_t N, size_t M,
97     char lex=1
98 );
99
100 template <class T>
101 size_t findlast_sorted(const char *F, int L,
102     const T* d0, const T* dd, size_t m, size_t N, size_t M,
103     char lex=1
104 );
105
106 template <class T0, class T> inline
107 size_t findfirst_sorted(const char *F, int L,
108     const T0* d0, const T* dd, size_t m, size_t N, size_t M, char lex=1
109 ) {
110     T d2[m];
111     for (unsigned i=0; i<m; ++i) d2[i]=T(d0[i]);
112     return findfirst_sorted(F,L,d2,dd,m,N,M,lex);
113 };
114
115 template <class T0, class T> inline
116 size_t findlast_sorted(const char *F, int L,
117     const T0* d0, const T* dd, size_t m, size_t N, size_t M, char lex=1
118 ) {
119     T d2[m];
120     for (unsigned i=0; i<m; ++i) d2[i]=T(d0[i]);
121     return findlast_sorted(F,L,d2,dd,m,N,M,lex);
122 };
123
124 template <class T>
125 wstring bits(const T &x, char compact=1);
126
127 class IOstat {
128 public:
129
130     IOstat()
131         : nread(0), nwrite(0), sread(0), swrite(0), aux1(0), aux2(0) {};
132
133     IOstat& init() {
134         nread=nwrite=0; sread=swrite=aux1=aux2=0;
135         return *this;
136     }
137
138     void gotread (size_t s=0) { nread +=1; sread +=s; }
139     void gotwrite(size_t s=0) { nwrite+=1; swrite+=s; }
140
141     bool isEmpty() {
142         return (!nread && !nwrite && !sread && !swrite && !aux1 && !aux2);
143     };
144
145     void print(const char *istr,
146         const char *hstr=0, const char *xstr1=0, const char *xstr2=0);
147
148     unsigned nread, nwrite;
149     size_t sread, swrite, aux1, aux2;

```

```

150
151     protected:
152     private:
153 };
154
155 template <class T, class T2>
156 class iterator {
157     public:
158
159     iterator(T* p_) : i(0), p(p_) {
160         if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
161         n=p->numel();
162     };
163
164     iterator& operator++() { ++i; return *this; }
165     bool end() { return i>=n; }
166
167     size_t iter () const { return i; }
168     size_t numel() const { return n; }
169
170     T2& operator[](size_t k) {
171         if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
172         return (*p)[k];
173     }
174     T2& at() {
175         if (i>=n) wblog(FL,
176             "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
177         return (*p)[i];
178     };
179
180     size_t i, n;
181     T* p;
182
183     protected:
184     private:
185 };
186
187 template <class T, class T2>
188 class iterator<T, T2*> {
189     public:
190
191     iterator(T* p_) : i(0), p(p_) {
192         if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
193         n=p->numel();
194     };
195
196     iterator& operator++() { ++i; return *this; }
197     bool end() { return i>=n; }
198
199     size_t iter () const { return i; }
200     size_t numel() const { return n; }
201
202     T2* operator[](size_t k) {
203         if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
204         return (*p)[k];
205     }
206     T2* at() {
207         if (i>=n) wblog(FL,
208             "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
209         return (*p)[i];
210     };
211
212     size_t i, n;
213     T* p;
214
215     protected:
216     private:
217 };
218
219 template <class T, class T2>
220 class citerator {
221     public:
222
223     citerator(const T* p_) : i(0), p(p_) {
224         if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
225         n=p->numel();
226     };
227
228     citerator& operator++() { ++i; return *this; }
229     bool end() { return i>=n; }
230
231     size_t iter () const { return i; }
232     size_t numel() const { return n; }
233
234     const T2& operator[](size_t k) {
235         if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
236         return (*p)[k];

```

```

237     }
238     const T2& at() {
239         if (i>=n) wblog(FL,
240             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,n);
241         return (*p)[i];
242     };
243
244     size_t i,n;
245     const T* p;
246
247     protected:
248     private:
249 };
250
251 template <class T, class T2>
252 class citerator<T,T2*> {
253     public:
254
255     citerator(const T* p_) : i(0), p(p_) {
256         if (!p_) wblog(FL,"ERR %s() got p=NULL !?",FCT);
257         n=p->numel();
258     };
259
260     citerator& operator++() { ++i; return *this; }
261     bool end() { return i>=n; }
262
263     size_t iter () const { return i; }
264     size_t numel() const { return n; }
265
266     const T2* operator[](size_t k) {
267         if (!p) wblog(FL,"ERR %s() object not yet initialized !?",FCT);
268         return (*p)[k];
269     }
270     const T2* at() {
271         if (i>=n) wblog(FL,
272             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,n);
273         return (*p)[i];
274     };
275
276     size_t i,n;
277     const T* p;
278
279     protected:
280     private:
281 };
282
283 class rand {
284
285     public:
286
287     rand() {};
288
289     double get_dfac() { return dfac; }
290
291     template <class T>
292     T& rand_(T &x);
293
294     template <class T>
295     T& rand_(T &x, double fac);
296
297     template <class T>
298     T& rands(T &x);
299
300     template <class T>
301     T& randb(T &x);
302
303     template <class T>
304     void scale_rand(T *x, size_t n, double fac) {
305         for (size_t i=0; i<n; ++i) { x[i]*=fac; }
306     }
307
308     protected:
309     private:
310
311     static double dfac;
312 };
313
314 double rand::dfac = (1.-1./double(long(1)<<52)) / double(RAND_MAX);
315
316 template<> inline
317 double& rand::rand_(double &x) { return (x=dfac*::rand()); };
318
319 template<> inline
320 int& rand::rand_(int &x) { return (x=(100*dfac)*::rand()); };
321
322 template<> inline
323 unsigned& rand::rand_(unsigned&x) { return (x=(100*dfac)*::rand()); };

```

```

324
325 template<> inline
326 long& rand::rand_(long &x) { return (x=(100*dfac)*::rand()); };
327
328 template<> inline
329 size_t& rand::rand_(size_t &x) { return (x=(100*dfac)*::rand()); };
330
331 template<> inline
332 int& rand::rand_(int&x, double fac) {
333     return (x=(fac*dfac)*::rand());
334 };
335
336 template<> inline
337 unsigned& rand::rand_(unsigned &x, double fac) {
338     return (x=(fac*dfac)*::rand());
339 };
340
341 template<> inline
342 long& rand::rand_(long &x, double fac) {
343     return (x=(fac*dfac)*::rand());
344 };
345
346 template<> inline
347 size_t& rand::rand_(size_t &x, double fac) {
348     return (x=(fac*dfac)*::rand());
349 };
350
351 template<> inline
352 double& rand::randb(double &x) { return (x=(2*dfac)*::rand()-1.); }
353
354 template<> inline
355 int& rand::randb(int &x) { return (x=(199*dfac)*::rand()-99); }
356
357 template<> inline
358 double& rand::rands(double &x) {
359     x=(2*dfac*::rand()-1);
360     if (fabs(x)>=1) wblog(FL,"ERR %s() got x=%.4g (1%+.4g) !?",FCT,x,x-1);
361     return (x=atanh(x));
362 };
363
364 template<> inline
365 int& rand::rands(int &x) {
366     double d=dfac*::rand();
367     if (fabs(d)>=1) wblog(FL,"ERR %s() got d=%.4g (1%+.4g) !?",FCT,d,d-1);
368     return (x=(100*atanh(d)+0.5));
369 };
370
371 template<> inline
372 void rand::scale_rand(int *x, size_t n, double fac) {
373     fac/=double(99);
374     for (size_t i=0; i<n; ++i) { x[i]=((double(x[i]))*fac); }
375 };
376
377 };
378
379 #ifdef __APPLE__
380
381 namespace Mac {
382     double getMemSize(const char *tag);
383     size_t getProcMemSize(
384         pid_t p=0,
385         char res=0
386     );
387 };
388
389 #endif
390
391 void wb_srand();
392
393 template <class T>
394 class WbUtil {
395 public:
396
397     static void adjust_tnorm(char &tnorm);
398
399     static bool hasConj();
400     static bool hasNoConj();
401
402     static constexpr bool isPOD() {
403         return std::is_trivially_copyable<T>::value;
404     };
405
406     static bool isFloat();
407     static bool isInt();
408
409     static constexpr char isComplex();
410     static char isReal() {

```



```

411         char q=isComplex(); if (q>=0) q=( q ? 0 : 1);
412         return q;
413     }
414
415     T eps();
416
417     protected:
418     private:
419
420 };
421
422 template <class T> inline
423 void WbUtil<T>::adjust_tnorm(char &tnorm){ tnorm=0; };
424
425 template <> inline
426 void WbUtil<wbcomplex>::adjust_tnorm(char &tnorm __attribute__((unused))){};
427
428 template <class T> inline bool WbUtil<T>::hasConj(){ return 0; };
429 template <> inline bool WbUtil<wbcomplex>::hasConj(){ return 1; };
430
431 template <class T> inline bool WbUtil<T>::hasNoConj(){ return 1; };
432 template <> inline bool WbUtil<wbcomplex>::hasNoConj(){ return 0; };
433
434 template <class T> inline
435 constexpr char ISCOMPLX_(T){ return 0; };
436
437 template <> inline
438 constexpr char ISCOMPLX_(wbcomplex){ return 1; };
439
440 template <> inline
441 constexpr char ISCOMPLX_(void){ return -1; };
442
443 #ifdef MATLAB_MEX_FILE
444 template <> inline
445 constexpr char ISCOMPLX_(mxComplexDouble){ return 2; };
446 #endif
447
448 #ifdef _COMPLEX_H
449 template <> inline
450 constexpr char ISCOMPLX_(complex){ return 3; };
451 #endif
452
453 template <class T> inline
454 bool WbUtil<T>::isFloat(){ return 0; };
455
456 template <> inline bool WbUtil<double>::isFloat(){ return 1; };
457 template <> inline bool WbUtil<long double>::isFloat(){ return 1; };
458 template <> inline bool WbUtil<float>::isFloat(){ return 1; };
459 template <> inline bool WbUtil<wbcomplex>::isFloat(){ return 1; };
460
461 template <class T> inline
462 bool WbUtil<T>::isInt(){ return 0; };
463
464 template <> inline bool WbUtil<int>::isInt(){ return 1; };
465 template <> inline bool WbUtil<char>::isInt(){ return 1; };
466 template <> inline bool WbUtil<long>::isInt(){ return 1; };
467
468 template <> inline bool WbUtil<unsigned>::isInt(){ return 1; };
469 template <> inline bool WbUtil<unsigned char>::isInt(){ return 1; };
470 template <> inline bool WbUtil<unsigned long>::isInt(){ return 1; };
471
472 template <> inline double WbUtil<double>::eps(){ return DBL_EPSILON; };
473 template <> inline float WbUtil<float>::eps(){ return FLT_EPSILON; };
474
475 template <> inline
476 long double WbUtil<long double>::eps(){ return LDBL_EPSILON; };
477
478 #include <sys/time.h>
479 #include <sys/resource.h>
480
481 class cpu_time {
482     public:
483
484     cpu_time() { tcpu=time('c'); tsys=time('s'); };
485
486     void init() { tcpu=time('c'); tsys=time('s'); };
487
488     cpu_time since() {
489         cpu_time dt; dt.tcpu=tcpu; dt.tsys=tsys;
490         return dt;
491     };
492
493     static double time(char flag) {
494         struct rusage result;
495         getrusage(RUSAGE_SELF,&result);
496         if (flag=='c') {

```

```

498         return (
499             double(result.ru_utime.tv_sec)
500             + 1e-6 * result.ru_utime.tv_usec
501         );
502     } else {
503         return (
504             double(result.ru_stime.tv_sec)
505             + 1e-6 * result.ru_stime.tv_usec
506         );
507     }
508 };
509
510 wbststring toStr(char flag='c');
511 double tcpu, tsys;
512
513 protected:
514 private:
515 };
516
517 cpu_time gCPUTime;
518
519 class wbsys {
520 public:
521     wbststring MemTot2Str() const;
522     wbststring MemFree2Str() const;
523     wbststring SwapTot2Str() const;
524     wbststring SwapFree2Str() const;
525
526     size_t getMemTot() const;
527     size_t getMemFree() const;
528     size_t getSwapTot() const;
529     size_t getSwapFree() const;
530
531     char checkSwapSpace(const char *F=NULL, int L=0) const;
532
533     int getNumCores() const;
534
535 protected:
536 private:
537 };
538
539 class wbttop {
540 public:
541     wbttop() {
542         pid=getpid(); ppid=getppid();
543         systemem=wbsys().getMemTot();
544     };
545
546     wbststring VmSize2Str() const;
547     size_t getVmSize() const { return Wb::getProcSize("VmSize",pid); };
548
549     wbststring VmPeak2Str() const;
550     size_t getVmPeak() const { return Wb::getProcSize("VmPeak",pid); };
551
552     int runningLarge(const char *F=NULL, int L=0,
553                     double th0=0.80,
554                     double fac=1.2
555                     );
556
557     void MemStat(const char *F=0, int L=0, char lflag=0) const;
558
559     pid_t pid, ppid;
560     size_t systemem;
561
562 protected:
563 private:
564 };
565
566 #endif
567
568 #endif
569

```

5.15 wbararray.hh

```

1 #ifndef __WB_ARRAY_COL_MAJOR_HH__
2 #define __WB_ARRAY_COL_MAJOR_HH__
3
4 // #define CHECK_ELEMENT_RANGE
5

```

```

6 //=====//
7 // wbararray - N-D Array class //
8 // //
9 // generic N-dimensional class that allows permutations //
10 // and reshaping; index order is column-major. //
11 // //
12 // NOTES //
13 // //
14 // Wb,Sep14,09 ; Wb,Aug11,05 //
15 //=====//
16
17 template <class T>
18 class wbararray {
19
20 public:
21
22     wbararray()
23         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { };
24
25     wbararray(size_t d1)
26         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(d1); };
27
28     wbararray(size_t d1, size_t d2)
29         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(d1,d2); };
30
31     wbararray(size_t d1, size_t d2, size_t d3)
32         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(d1,d2,d3); };
33
34     wbararray(size_t d1, size_t d2, size_t d3, size_t d4)
35         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(d1,d2,d3,d4); };
36
37     wbararray(char *sidx)
38         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(Str2Idx(sidx)); };
39
40     wbararray(const wbvector<unsigned> &S)
41         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(S); };
42
43     wbararray(const wbvector<size_t> &S)
44         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(S); };
45
46     wbararray(const wbararray &a, const char ref=0)
47         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(a,ref); };
48
49     template<class T2>
50     wbararray(const wbararray<T2> &a)
51         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) {
52         NEW(a.SIZE); init_T(a.data);
53     };
54
55     wbararray(const mxArray* a, char ref=0, char vec=0)
56         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { init(a,ref,vec); };
57
58     wbararray(const char *F, int L, const mxArray* a, char ref=1, char vec=0)
59         : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) {
60         init(F,L,a,ref,vec);
61     };
62
63     wbararray(const wbMatrix<T> &a, char ref=0)
64         : wbararray() { init(a,ref); };
65
66     virtual ~wbararray() {
67         DELETE_DATA();
68     };
69
70     wbararray& initX() {
71         DELETE_DATA('!'); SIZE.init();
72         return *this;
73     };
74
75     wbararray& init(size_t d1=0) {
76         if (d1)
77             { NEW(wbvector<size_t>(1,&d1)); }
78         else { DELETE_DATA(); SIZE.init(); }
79         return *this;
80     };
81
82     wbararray& init(size_t d1, size_t d2) {
83         size_t s[2]= {d1,d2}; NEW(wbvector<size_t>(2,s));
84         return *this;
85     };
86
87     wbararray& init(size_t d1, size_t d2, size_t d3) {
88         size_t s[3]= {d1,d2,d3}; NEW(wbvector<size_t>(3,s));
89         return *this;
90     };
91
92     wbararray& init(size_t d1, size_t d2, size_t d3, size_t d4) {

```

```

93     size_t s[4]= {d1,d2,d3,d4}; NEW(wbvector<size_t>(4,s));
94     return *this;
95 };
96
97 wbarray& initp(size_t d1, const T *d) {
98     NEW(wbvector<size_t>(1,&d1),d);
99     return *this;
100 };
101
102 wbarray& initp(size_t d1, size_t d2, const T *d) {
103     size_t s[]= {d1,d2};
104     NEW(wbvector<size_t>(2,s),d);
105     return *this;
106 };
107
108 wbarray& initp(size_t d1, size_t d2, size_t d3, const T *d) {
109     size_t s[]= {d1,d2,d3};
110     NEW(wbvector<size_t>(3,s),d);
111     return *this;
112 };
113
114 wbarray& init2ref(const wbarray &a) {
115     return INIT2REF(a.data,&a.SIZE);
116 };
117
118 wbarray& init(const wbarray &a, char ref=0) {
119     if (this!=&a) {
120         if (!ref) { NEW(a.SIZE, a.data); }
121         else { DELETE_DATA(); SIZE=a.SIZE;
122               a.check_consistency(FL);
123               if (a.sptr) {
124                   (sptr=a.sptr)->add_dref();
125                   data=a.data;
126               }
127         }
128     }
129     return *this;
130 };
131
132 wbarray& init(const wbMatrix<T> &a, char ref=0) {
133     if (ref) {
134         size_t s[]={ a.dim2, a.dim1 }; SIZE.init(2,s);
135         INIT2REF(a.data);
136     }
137     else {
138         size_t i,j, l=0; init(a.dim1,a.dim2);
139         for (i=0; i<a.dim1; ++i)
140             for (j=0; j<a.dim2; ++j, ++l) { data[i+a.dim1*j]=a.data[l]; }
141     }
142     return *this;
143 };
144
145 wbindex& ind2sub(size_t k, wbindex &I) const {
146     const size_t *s=SIZE.data, n=SIZE.len;
147     size_t i,r, e=0, *idx;
148
149     I.init(n); idx=I.data;
150     for (i=0; i<n; ++i) {
151         if (!s[i]) wblog(FL,
152             "ERR wbarray::%s() got size %s",FCT,sizeStr().data);
153
154         r=k/s[i]; idx[i]=k-r*s[i]; k=r;
155         if (idx[i]>=s[i]) e++;
156     }
157
158     if (e || k) wblog(FL,
159         "ERR wbarray::%s() out of bounds (%d => [%s; %s])",
160         FCT, STR(I+1), sizeStr().data
161     );
162
163     return I;
164 };
165
166 void adjustMMat(
167     const char* F, int L,
168     char flag, const T fac, wbarray &A
169 ) const {
170
171     if (!isMatrix()) wblog(F,L,
172         "ERR %d - rank-2 object required (%d)",FCT,SIZE.len);
173     if (!strchr("NTC",flag)) wblog(F,L,
174         "ERR %d - invalid flag %c<%d>",FCT,flag,flag);
175
176     size_t s=numel();
177
178     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
179

```

```

180         if (fac!=T(1) || flag!='N') { size_t i;
181             if (flag!='N') permute(A,"2 1"); else A=(*this);
182             if (flag=='C') for (i=0; i<s; i++) A[i]=CONJ(A[i]);
183             if (fac!=T(1)) for (i=0; i<s; i++) A[i]*=fac;
184         }
185     } else { A.init2ref(*this); }
186 }
187
188 void adjustDMat(
189     const char* F, int L,
190     char flag, const T fac, wbararray &A
191 ) const {
192
193     if (SIZE.len!=1) wblog(F,L,
194         "ERR %d - 1D representation required (%d)",FCT,SIZE.len);
195     if (!strchr("NTC",flag)) wblog(F,L,
196         "ERR %d - invalid flag %c<%d>",FCT,flag,flag);
197
198     size_t n=SIZE[0];
199
200     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
201
202     if (fac!=T(1) || flag=='C') { size_t i; A=(*this);
203         if (flag=='C') for (i=0; i<n; i++) A[i]=CONJ(A[i]);
204         if (fac!=T(1)) for (i=0; i<n; i++) A[i]*=fac;
205     }
206     else { A.init2ref(*this); }
207 }
208
209 void adjustVec(
210     const char* F, int L, char &flag, wbararray &A, char pos
211 ) const { A=(*this); A.adjustVec(F,L,flag,pos); };
212
213 void adjustVec(const char* F, int L, char &flag, char pos) {
214     if (SIZE.len!=1) wblog(F,L,
215         "ERR %s() vector expected (%d)",FCT,SIZE.len);
216     if (!strchr("NTC",flag)) wblog(F,L,
217         "ERR %s() invalid flag %c<%d>",FCT,flag,flag);
218
219     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
220     if (flag=='C') {
221         for (size_t s=numel(), i=0; i<s; ++i)
222             data[i]=CONJ(data[i]);
223     }
224     flag='N';
225
226     SIZE.Resize(2);
227     if (pos==1) { SIZE[1]=SIZE[0]; SIZE[0]=1; } else
228     if (pos==2) { SIZE[1]=1; } else
229     wblog(F,L,"ERR %s() invalid pos=%d",FCT,pos);
230 };
231
232 void adjustCMat(const char* F, int L,
233     T cfac, size_t s1, size_t s2, char cforce
234 ) {
235     if (cfac!=T(0)) {
236         if (data==NULL) {
237             if (cforce) wblog(F,L,
238                 "WRN C = A*B + c*[] with c=%s !?", Wb::num2Str(cfac).data);
239         }
240         else if (!isMatrix() || SIZE[0]!=s1 || SIZE[1]!=s2) {
241             wblog(F,L,"ERR %s() dimension mismatch: C=(%s) =? (%d,%d).",
242                 FCT, sizeStr().data, s1, s2); return;
243         }
244         else {
245             if (cfac!=T(1)) {
246                 size_t i, s=SIZE.prod(0);
247                 for (i=0; i<s; i++) data[i]*=cfac;
248             }
249             return;
250         }
251     }
252     init();
253 };
254
255 void set(const wbararray &a, const T fac=1) {
256     if (fac==T(0)) { NEW(a.SIZE); }
257     else {
258         NEW(a.SIZE, a.data);
259         if (fac!=T(1)) { size_t i=0, n=numel();
260             if (fac==T(-1))
261                 for (i<n; ++i) data[i]=-data[i];
262             else for (i<n; ++i) data[i]*=fac;
263         }
264     }
265 };
266

```

```

267 void setBlock(
268     const wvector< WBINDEX > &D,
269     const windex &IB,
270     const warray &A
271 );
272
273 void addBlock(
274     const WBINDEX &I,
275     const warray &A, const char dflag=0
276 );
277
278 warray& addBlock(
279     size_t i, size_t j,
280     size_t n, size_t m,
281     warray &a
282 ) const;
283
284 warray& getBlock(
285     size_t i, size_t j,
286     size_t n, size_t m,
287     warray &a
288 ) const;
289
290 warray& BlockDiag(const wvector< warray > &D);
291
292 warray& blockTrace(
293     size_t D,
294     warray &a, size_t D2=-1
295 ) const;
296
297 void copyStride(T* dd, size_t stride) const;
298 void addStride(T* dd, size_t stride, const T afac=1) const;
299
300 template<class T0>
301 warray& init_T(const T0 *dd) {
302     if (SIZE.len) {
303         for (size_t n=SIZE.prod(), i=0; i<n; ++i) { data[i]=T(dd[i]); }
304         return *this;
305     }
306
307     template<class T0>
308     warray& init_T(const char *F, int L, const T0 *dd) {
309         for (size_t n=SIZE.prod(), i=0; i<n; ++i) { data[i]=T(dd[i]);
310             if (T0(data[i])!=dd[i]) wblog(F_L,
311                 "ERR type conversion changes value (%g->%g) !?",
312                 double(dd[i]), double(data[i])
313             );
314         }
315         return *this;
316     };
317
318     warray& initT(const warray<T> &a) { return init(a); };
319
320     template<class T0>
321     warray& initT(const warray<T0> &a) {
322         NEW(a.SIZE); init_T(a.data); return *this;
323     };
324
325     template <class T0>
326     warray& initT(const char *F, int L, const warray<T0> &a) {
327         NEW(a.SIZE); init_T(F_L,a.data); return *this;
328     };
329
330     warray& init(char *s, T* d=NULL) {
331         init(Str2Idx(s), d); return *this;
332     };
333
334     warray& init(const WBINDEX &S, T* d=NULL) {
335         NEW(S,d); return *this;
336     };
337
338     template <class IT>
339     warray& init(const wvector<IT> &S_, T* d=NULL) {
340         WBINDEX S(S_.len);
341         for (size_t i=0; i<S.len; ++i) { S.data[i]=size_t(S_.data[i]); }
342         NEW(S,d); return *this;
343     };
344
345     warray& init(const char *F, int L,
346         const mxArray *a, char ref=1, char vec=0);
347
348     warray& init(const char *F, int L,
349         const mxArray *a, wperm &P0);
350
351     warray& init(const mxArray *a, char ref=0, char vec=0) {
352         return init(FL,a,ref,vec); }
353     warray& init(const mxArray *a, wperm &P0) {

```

```

354     return init(FL,a,P0); }
355
356 wbararray& init2ref(const T *x, const WBINDEX &S) {
357     SIZE=S; return INIT2REF(x);
358 };
359
360 wbararray& init2ref(size_t d1, const T *x) {
361     SIZE.init(1,&d1);
362     return INIT2REF(x);
363 };
364 wbararray& init2ref(size_t d1, size_t d2, const T *x) {
365     size_t s[] = {d1,d2}; SIZE.init(2,s);
366     return INIT2REF(x);
367 };
368 wbararray& init2ref(size_t d1, size_t d2, size_t d3, const T *x) {
369     size_t s[] = {d1,d2,d3}; SIZE.init(3,s);
370     return INIT2REF(x);
371 };
372
373 int isRef() const { return (sptr && sptr->isRef()); };
374
375 wbararray& Instantiate(const char *F=NULL, int L=0) {
376     if (isRef() && data) {
377         if (!sptr) wblog(F_L,"ERR %s() got null sptr !?",FCT);
378         T *d0=data; NEW_DATA(SIZE.prod(0),d0);
379     }
380     else if (F) wblog(F,L,
381         "WRN %s() no need to instantiate %s",SSTR_(this));
382
383     return *this;
384 };
385
386 wbararray& unRef() { return Instantiate(); };
387
388 void init2Vec(const T *d0, size_t len, char ref=0) {
389     SIZE.init(1,&len);
390     if (data!=d0) { NEW_DATA(len,d0,ref); }
391 };
392
393 void init2ref(const wbararray &A, size_t s1, size_t s2) {
394     if (s1*s2!=A.SIZE.prod(0)) wblog(FL,
395         "ERR %s() cannot reshape array %s into %dx%d",
396         SSTR_(this), s1, s2
397     );
398
399     SIZE.init(2); SIZE[0]=s1; SIZE[1]=s2;
400     INIT2REF(A.data);
401 };
402
403 void initTst();
404
405 void init2Vec(const wbvector<T> &a) {
406     WBINDEX S(1); S[0]=a.len;
407     NEW(S, a.data);
408 };
409
410 wbararray& initIdentity(size_t d, char dflag=0, T dval=1);
411 wbararray& initIdentity(const WBINDEX &S, char dflag=0);
412 wbararray& initIdentityB(
413     size_t d1, size_t d2, size_t k=0);
414 wbararray& initIdentityB3(
415     size_t d1, size_t d2, size_t D, size_t i0, T one=1);
416
417 wbararray& Expand2Projector(const T eps=0);
418 wbararray& ExpandDiagonal(unsigned i1, unsigned i2);
419 wbararray& ExpandDiagonal();
420
421 void Diag2Vec();
422 wbararray& Reduce2Diag(T eps=0);
423
424 wbararray& initDiag(unsigned n, const T *d){
425     init(n,n);
426     for (size_t i=0; i<n; ++i) data[i+i*n]=d[i];
427     return *this;
428 };
429
430 wbararray& initDiag(const wbvector<T> &d){
431     return initDiag(d.len, d.data); };
432
433 void initVec(size_t d, const T val=0) {
434     init(d); if (val) set(val);
435 };
436
437 void resize(const wbvector<size_t> &S, wbararray &B) const;
438
439 void Resize(const wbvector<size_t> &S, const T* d0=NULL) {
440     if (d0!=NULL) { NEW(S,d0); }

```

```

441     else if (S!=SIZE) {
442         wbararray<T> X; this->save2(X);
443         X.resize(S,*this);
444     };
445 };
446
447 void Resize(size_t s1, size_t s2) {
448     size_t s[2]={s1,s2};
449     if (SIZE.len!=2) wblog(FL,"ERR rank-2 required (%d)",SIZE.len);
450
451     wbararray X; this->save2(X);
452     X.resize(wbvector<size_t>(2,s), *this);
453 };
454
455 wbararray& Enlarge(
456     size_t dim,
457     size_t D,
458     size_t istart=0
459 );
460
461 wbararray& save2(wbararray&);
462 wbararray& save2(wbMatrix<T>&);
463
464 wbararray& Append2(
465     const char *F, int L, wbararray &B, unsigned dim);
466 wbararray& Append2(wbararray &B, unsigned dim){
467     return Append2(0,0,B,dim);
468 }
469
470 void Append2(
471     const char *F, int L,
472     wbararray &B, unsigned dim,
473     wbararray<double> &W, double eps
474 );
475 void Append2(wbararray &B, unsigned dim, wbararray &W, double eps){
476     return Append2(0,0,B,dim,W,eps);
477 };
478
479 void rand(const UVEC &d) { init(d); setRand(); };
480 void setRand(const char pnflag=0);
481
482 wbararray& Reset(const T *dref) {
483     size_t n=SIZE.prod();
484     if (dref) { memcpy(data,dref,sizeof(T)*n); }
485     else { MEM_SET<T>(data,n); }
486     return *this;
487 };
488
489 wbararray& set(const T& c) {
490     size_t n=SIZE.prod();
491     if (c!=T(0)) { for (size_t i=0; i<n; i++) data[i]=c; }
492     else { MEM_SET<T>(data,n); }
493     return *this;
494 };
495
496 wbararray& operator= (const wbararray &a) { return init(a); };
497
498 template <class T0>
499 wbararray<T>& operator= (const wbararray<T0> &a) {
500     if ((void*)this!=(void*)&a) {
501         NEW(a.SIZE);
502         for (size_t n=a.numel(), i=0; i<n; ++i) data[i]=T(a.data[i]);
503     }
504     return *this;
505 };
506
507 void SetRef(const widx_t *I, size_t len, const T* d0);
508 void SetRef(size_t k, const T* d0) { SetRef(&k,1,d0); };
509
510 const T* ref(const widx_t *I, size_t n) const {
511     if (!n) return NULL;
512     return data+serial_index(I,n);
513 };
514 T* ref(const widx_t *I, size_t n) {
515     if (!n) return NULL;
516     return data+serial_index(I,n);
517 };
518
519 T* col(size_t k) {
520     if (SIZE.len!=2 || k>=SIZE[1]) wblog(FL,
521         "ERR %s() index out of bounds (%s; %d)",FCT,sizeStr().data,k);
522     return (data+k*SIZE[0]);
523 };
524 const T* col(size_t k) const {
525     if (SIZE.len!=2 || k>=SIZE[1]) wblog(FL,
526         "ERR %s() index out of bounds (%s; %d)",FCT,sizeStr().data,k);
527     return (data+k*SIZE[0]);

```



```

528     };
529
530     wbvector<T>& getCol(size_t k, wbvector<T> &v) const {
531         return v.init(SIZE[0],col(k));
532     };
533
534     T* ref(const WBINDEX &I) { return ref(I.data,I.len); };
535     const T* ref(const WBINDEX &I) const {
536         return ref(I.data,I.len);
537     };
538
539     const T* ref(widx_t i) const { return ref(&i,1); }
540     T* ref(widx_t i) { return ref(&i,1); }
541
542     const T* ref(widx_t i1, widx_t i2) const {
543         widx_t I[]={i1,i2}; return ref(I,2);
544     };
545     T* ref(widx_t i1, widx_t i2) {
546         widx_t I[]={i1,i2}; return ref(I,2);
547     };
548
549     const T* ref(widx_t i1, widx_t i2, widx_t i3) const {
550         widx_t I[]={i1,i2,i3}; return ref(I,3);
551     };
552     T* ref(widx_t i1, widx_t i2, widx_t i3) {
553         widx_t I[]={i1,i2,i3}; return ref(I,3);
554     };
555
556     const T& operator[] (size_t i) const { return data[i]; }
557     T& operator[] (size_t i) { return data[i]; }
558
559     const T& operator() (size_t i) const {
560         if (SIZE.len!=1) wblog(FL,
561             "ERR %s requires vector type (%s)",FCT,sizeStr().data);
562         return data[i];
563     };
564     T& operator() (size_t i) {
565         if (SIZE.len!=1) wblog(FL,
566             "ERR %s requires vector type (%s)",FCT,sizeStr().data);
567         return data[i];
568     };
569
570     const T& operator() (size_t i, size_t j) const {
571         if (SIZE.len!=2) wblog(FL,
572             "ERR %s(i,j) requires matrix type (%s)",FCT,sizeStr().data);
573         return data[ i + j*SIZE[0] ];
574     };
575     T& operator() (size_t i, size_t j) {
576         if (SIZE.len!=2) wblog(FL,
577             "ERR %s(i,j) requires matrix type (%s)",FCT,sizeStr().data);
578         return data[ i + j*SIZE[0] ];
579     };
580
581     const T& operator() (size_t i, size_t j, size_t k) const {
582         if (SIZE.len!=3) wblog(FL,
583             "ERR %s(i,j,k) requires 3D-object (%s)",FCT,sizeStr().data);
584         return data[ i + SIZE[0] * (j + SIZE[1]*k) ];
585     };
586     T& operator() (size_t i, size_t j, size_t k) {
587         if (SIZE.len!=3) wblog(FL,
588             "ERR %s(i,j,k) requires 3D-object (%s)",FCT,sizeStr().data);
589         return data[ i + SIZE[0] * (j + SIZE[1]*k) ];
590     };
591
592     const T& operator() (const WBINDEX &I) const {
593         if (I.len!=SIZE.len) wblog(FL,
594             "ERR %s() invalid index [%s]",FCT,STR(I));
595         return data[serial_index(I.data,I.len)];
596     };
597     T& operator() (const WBINDEX &I) {
598         if (I.len!=SIZE.len) wblog(FL,
599             "ERR invalid index [%s] having %s",STR(I),sizeStr().data);
600         return data[serial_index(I.data,I.len)];
601     };
602
603     const T& element(const WBINDEX &I) const {
604         if (I.len!=SIZE.len) wblog(FL,"ERR %s() invalid index [%s] "
605             "having %s",FCT,STR(I),sizeStr().data);
606         for (size_t i=0; i<I.len; i++) if (I.data[i]>=SIZE.data[i])
607             wblog(FL,"ERR %s() index out of bounds (%s; %s)",
608                 FCT,STR(I),sizeStr().data);
609         return data[serial_index(I.data,I.len)];
610     };
611     T& element(const WBINDEX &I) {
612         if (I.len!=SIZE.len) wblog(FL,"ERR %s() invalid index [%s] "
613             "having %s",FCT,STR(I),sizeStr().data);
614         for (size_t i=0; i<I.len; i++) if (I.data[i]>=SIZE.data[i])

```

```

615         wblog(FL,"ERR %s() index out of bounds (%s; %s)",
616             FCT,STR(I),sizeStr().data);
617     return data[serial_index(I.data,I.len)];
618 };
619
620 size_t numel() const {
621     if (SIZE.len) {
622         const size_t *s=SIZE.data;
623         size_t n=s[0], r=SIZE.len, i=1; for (; i<r; i++) n*=s[i];
624         return n;
625     }
626     return 0;
627 };
628
629 wbststring sizeStr() const {
630     if (SIZE.len) return SIZE.toStrf("","x");
631     else return wbststring("{}");
632 };
633
634 void squeeze();
635
636 void skipSingletons();
637 void skipSingleton(const char *F, int L, unsigned n=1);
638
639 void addSingletons(unsigned r, const char *F=NULL, int L=0);
640 void prependSingletons(unsigned r);
641
642 size_t size(unsigned i) const { return SIZE[i]; };
643 size_t dim(unsigned i) const {
644     if (i==0 || i>SIZE.len) wblog(FL,
645         "ERR index out of bounds (%d/%d)", i-1, SIZE.len);
646     return SIZE[i-1];
647 };
648
649 size_t dim0(unsigned i) const {
650     if (i>=SIZE.len) wblog(FL,
651         "ERR index out of bounds (%d/%d)", i, SIZE.len);
652     return SIZE[i];
653 };
654
655 size_t dim1() const {
656     size_t D=0;
657     if (SIZE.len==1) { D=SIZE[0]; } else
658     if (SIZE.len) {
659         unsigned z=0, l=-1, n=SIZE.len; const size_t *s=SIZE.data;
660         for (unsigned i=0; i<n; ++i) {
661             if (s[i]>1) {
662                 if (l>i) { l=i; } else
663                     wblog(FL,"ERR %s() invalid vector %s",FCT,SSTR_(this));
664             }
665             else if (!s[i]) { ++z; }
666         }
667
668         if (!z) { D=s[l<n ? l : 0]; } else
669         if (n>2 || l<n) {
670             wblog(FL,"ERR %s() invalid vector %s",FCT,SSTR_(this));
671         }
672     }
673     return D;
674 };
675
676 size_t dim2() const {
677     if (!SIZE.len) return 0;
678     if (SIZE.len!=2) wblog(FL,
679         "ERR %s() got rank-%d array (%s) !?",FCT,SIZE.len,SSTR_(this));
680     if (SIZE[0]!=SIZE[1]) wblog(FL,
681         "ERR %s() got non-square matrix (%s) !?",FCT,SSTR_(this));
682     return SIZE[0];
683 };
684
685 void getMatSize(
686     const char *F, int L, size_t &dim1, size_t &dim2) const {
687
688     size_t i, r=SIZE.len, r2=r/2;
689     const size_t *const &s=SIZE.data;
690
691     if (r%2) wblog(F,L,
692         "ERR %s() even rank object required (%s)",FCT,sizeStr().data);
693     if (r) {
694         dim1=s[0]; dim2=s[r2];
695         for (i=1; i<r2; i++) { dim1*=s[i]; dim2*=s[i+r2]; }
696     } else { dim1=dim2=0; }
697 };
698
699 void getMatSize(size_t &dim1, size_t &dim2, unsigned m) const {
700     unsigned i=0, r=SIZE.len;
701

```

```

702     const size_t *const &s=SIZE.data;
703
704     if (m>r) { m=r; }
705     if (m) { dim1=s[0]; for (++i; i<m; ++i) { dim1*=s[i]; }}
706     else { dim1=(r?1:0); }
707
708     if (i<r) { dim2=s[i]; for (++i; i<r; ++i) { dim2*=s[i]; }}
709     else { dim2=(r?1:0); }
710 };
711
712 unsigned rank() const { return SIZE.len; };
713
714 bool isRank(unsigned r) const {
715     if (SIZE.len!=r) {
716         if (SIZE.len<r) return 0;
717         for (unsigned i=r; i<SIZE.len; i++) if (SIZE[i]!=1) return 0;
718     }
719     return 1;
720 };
721
722 bool isOpS(size_t *n=NULL) const;
723 bool isOpS(unsigned &n) const { bool q;
724     size_t N; q=isOpS(&N); n=N;
725     if (size_t(n)!=N) wblog(FL,
726         "ERR %s() unsigned out of bounds (%d/%ld)",FCT,n,N);
727     return q;
728 };
729
730 wbarry& SkipTiny_float(const T eps=1E-14) { return *this; };
731 double SkipTiny_imag (double eps=1E-14) { return 0; };
732
733 double SkipTiny(const double eps_=1E-14) {
734     T zero=T(0), x2=zero, eps=T(eps_);
735     if (eps!=zero) { size_t i=0, n=numel(); T a;
736         for (; i<n; ++i) { if (data[i]!=zero) {
737             a=ABS(data[i]); if (a<eps) { x2+=NORM2(a); data[i]=zero; }
738         }}
739     }
740     return std::sqrt(double(x2));
741 };
742
743 bool isMatrix() const { return SIZE.len==2; }
744 bool isSMatrix(unsigned d=-1) const {
745     if (SIZE.len!=2 || SIZE[0]!=SIZE[1]) return 0;
746     if (int(d)>=0 && d!=SIZE[0]) return 0;
747     return 1;
748 };
749
750 bool isSquare() const {
751     if (SIZE.len%2) return 0;
752     else {
753         for (unsigned i=0, n=SIZE.len/2; i<n; ++i) {
754             if (SIZE[i]!=SIZE[n+i]) return 0; }
755     }
756     return 1;
757 };
758
759 char isEmpty() const { return (data==NULL); };
760 bool isComplex() const;
761 bool isZero(double eps=0., char flag=0) const;
762
763 bool isFinite() {
764     for (size_t n=numel(), i=0; i<n; ++i) {
765         if (!isfinite(data[i])) return 0; }
766     return 1;
767 };
768
769 explicit operator bool() const { return !isEmpty(); }
770 bool operator! () const { return isEmpty(); }
771
772 wbarry& Symmetrize(
773     const char *F, int L,
774     double *delta=NULL, char cflag=1, char tflag=0, char fflag=0
775 );
776
777 wbarry& swapRows(size_t i1, size_t i2);
778 wbarry& swapCols(size_t j1, size_t j2);
779
780 wbarry& setCol(
781     size_t k, const T* d, const char *F=NULL, int L=0);
782 wbarry& setCol(
783     size_t k, size_t k0, const char *F=NULL, int L=0);
784 wbarry& setRow(
785     size_t k, const T* d, const char *F=NULL, int L=0);
786
787 wbvector<T>& colNorm2(wbvector<T> &a) const;
788

```

```

789     T colNorm2(size_t k) const;
790
791     unsigned QRdecomp(
792         const char *F, int L, wbararray<T> &Q, wbararray<T> &R,
793         T eps=1E-15);
794
795     int Householder(
796         const char *F, int L,
797         size_t k, T *u,
798         WBPPerm *P=NULL,
799         T eps=1E-15);
800
801     unsigned eigTriDiag(
802         const char *F, int L, wbvector<T> &E,
803         wbararray<T> *U=NULL, T eps=1E-15) const;
804
805     wbararray& ColProject(
806         size_t k1, size_t k2, char nflag=0, char tnorm=0);
807
808     wbararray& ColPermute(const wbpPerm &P, char iflag=0);
809
810     bool isOrthogonalCol(size_t k, char tnorm=0) const;
811
812     wbararray& SignCol(size_t k, const T *d0, char tnorm=0);
813     wbararray& FlipSignCol(size_t k);
814     wbararray& SignConventionCol(
815         size_t k=-1,
816         double eps=1E-12
817     );
818
819     T NormalizeCol(size_t k, char tnorm=0, char qflag=0);
820     wbararray& NormalizeCols(
821         const char *F, int L,
822         double *amin=NULL, double *amax=NULL, char tnorm=0
823     );
824
825     wbararray& OrthoNormalizeCols(
826         const char *F=NULL, int L=0, char tnorm=0,
827         char qxflag=0, double eps=1E-14, unsigned np=1
828     );
829
830     wbararray& balanceOp(
831         const char *F, int L, T &xref, double &xscale
832     );
833
834     bool isProptoId(T &x, T eps=1E-14) const;
835
836     bool isDiagMatrix(double eps=1E-14) const {
837         return isDiag_aux(eps,"isDiag"); }
838
839     bool isIdentityMatrix(double eps=1E-14) const { T one=1;
840         return isProptoId(one,eps);
841     }
842
843     bool isDiagMatrix(double *eps) const {
844         return isDiag_aux(eps,"isDiag"); }
845
846     bool isIdentityMatrix(double *eps) const {
847         return isDiag_aux(eps,"isIdty"); }
848
849     size_t nnz(const T eps=0, WBINDEX *I=NULL) const;
850
851     double sparsity(const T eps=0) const {
852         size_t n=numel();
853         return (n ? nnz(eps)/double(n) : 1);
854     };
855
856     bool isHConj(
857         const wbararray &B, double eps=1E-12, double* xref=NULL
858     ) const { return isSym_aux(FLF, B, eps, xref, 's'); };
859
860     bool isHConj(
861         double eps=1E-12, double* xref=NULL
862     ) const { return isSym_aux(FLF,*this,eps,xref,'s'); };
863
864     bool isAHerm(
865         const wbararray &B, double eps=1E-12, double* xref=NULL
866     ) const { return isSym_aux(FLF, B, eps, xref, 'a'); };
867
868     bool isAHerm(
869         double eps=1E-12, double* xref=NULL
870     ) const { return isSym_aux(FLF,*this,eps,xref,'a'); };
871
872     char hasGroupSize(size_t s1, size_t s2) const;
873
874     double aMax(size_t *i=NULL) const;
875     double maxDiff (const wbararray &B) const;

```

```

876
877     T froNorm2(const wbararray &B) const;
878     T norm2() const;
879     T norm() const { return Sqrt(norm2()); };
880
881     T normCol2(size_t k) const;
882
883     T scalarProd(const wbararray &B) const;
884     T sum() const;
885
886     wbararray& sum(const WBINDEX &I, wbararray&) const;
887     wbararray& sum(const char *sidx, wbararray &A) const {
888         sum(Str2Idx(sidx,1), A);
889         return A;
890     };
891
892     char fitsSize(const wbararray &, char strict=0) const;
893
894     size_t getSize(unsigned i, char lflag=1) const {
895         size_t s=0;
896         if (i<SIZE.len) { s=SIZE[i]; } else
897         if (lflag && SIZE.len) {
898             for (i=0; i<SIZE.len; ++i) { if (!SIZE[i]) { break; }}
899             if (i==SIZE.len) { s=1; }
900         }
901         return s;
902     };
903
904     bool hasSameSize(const size_t *s, unsigned n) const;
905
906     bool hasSameSize(size_t d1, size_t d2) const {
907         size_t s[2]={d1,d2}; return hasSameSize(s,2);
908     };
909
910     bool hasSameSize1(const size_t *s, unsigned n) const;
911
912     template<class TB>
913     bool hasSameSize(const wbararray<TB> &, char lflag=1) const;
914
915     template<class TB>
916     bool hasSameSize(
917         unsigned ia, const wbararray<TB> &B, unsigned ib, char lflag=1
918     ) const {
919         size_t sa=getSize(ia,lflag), sb=B.getSize(ib,lflag);
920         return (sa==sb);
921     };
922
923     char sameUptoFac(
924         const wbararray &B, T *fac=NULL, double eps=1E-12
925     ) const;
926
927     bool hasRank(unsigned r) const {
928         if (SIZE.len<r) return 0;
929         if (SIZE.len>r) {
930             for (unsigned i=r; i<SIZE.len; ++i) {
931                 if (SIZE[i]!=1) return 0; }
932         }
933         return 1;
934     };
935
936     bool operator==(const wbararray &B) const {
937         if (this==&B) return 1;
938         if (SIZE!=B.SIZE) return 0;
939         if (data==B.data) return 1;
940         return (!memcmp(data, B.data, numel()*sizeof(T)));
941     };
942
943     bool operator!=(const wbararray &B) const {
944         return !(*this==B);
945     };
946
947     T normDiff2(const wbararray &B) const;
948     T normDiff (const wbararray &B) const { return Sqrt(normDiff2(B)); };
949     T normDiff2(const wvector<T> &B) const;
950     T normDiff (const wvector<T> &B) const { return Sqrt(normDiff2(B)); };
951
952     void TimesEl(const wbararray &, char conj=0);
953     wbararray& timesEl(
954         const wbararray &B, wbararray &C, T bfac=1, T cfac=0, char conj=0) const;
955
956     T sumTimesEl(const wbararray &, char conj=0) const;
957     T weightedAvg(const wbararray &) const;
958
959     void TensorProd(const wbararray &B,
960         const char aflag='N', const char bflag='N', const char kflag=0
961     ){
962         wbararray X(*this);

```

```

963     X.tensorProd(B,*this,aflag,bflag,kflag);
964 };
965
966 void tensorProd(
967     const wbararray &, wbararray &,
968     const char aflag='N', const char bflag='N', const char kflag=0
969 ) const;
970
971 void kron(
972     const wbararray &B, wbararray &X, char aflag='N', char bflag='N'
973 ) const { tensorProd(B,X,aflag,bflag,'k'); };
974
975 void Kron(const wbararray &B, char aflag='N', char bflag='N') {
976     wbararray X(*this);
977     X.tensorProd(B,*this,aflag,bflag,'k');
978 };
979
980 wbararray& Cat(unsigned dim, const wbvector< wbararray* > &ap);
981 wbararray& Cat(unsigned dim, const wbvector< wbararray > &ap);
982
983 wbararray& BlockCat(
984     const wbararray< wbararray > &aa,
985     WBINDEX *D1=NULL, WBINDEX *D2=NULL
986 ){
987     wbararray< const wbararray* > ap(aa.SIZE);
988     for (size_t n=aa.numel(), i=0; i<n; ++i) { ap[i]=&aa[i]; }
989     return BlockCat(ap,D1,D2);
990 };
991
992 wbararray& BlockCat(
993     const wbararray< const wbararray* > &ap,
994     WBINDEX *D1=NULL, WBINDEX *D2=NULL
995 );
996
997 void operator+= (T x) {
998     if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
999     for (size_t s=numel(), i=0; i<s; ++i) { data[i]+=x; }
1000 };
1001
1002 void operator-= (T x) {
1003     if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
1004     for (size_t s=numel(), i=0; i<s; ++i) { data[i]-=x; }
1005 };
1006
1007 void operator*=(const T a) {
1008     if (a!=T(1)) { size_t i=0, s=numel();
1009         if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
1010         if (a==T(-1)) for (; i<s; ++i) { data[i] = -data[i]; } else
1011             if (a==T( 0)) MEM_SET<T>(data,s);
1012         else { for (; i<s; ++i) { data[i] *= a; } }
1013     }
1014 };
1015
1016 void operator+= (const wbararray &B) { Plus(B); };
1017 void operator-= (const wbararray &B) { Plus(B,-1); };
1018
1019 wbararray& plus(
1020     const wbararray &B, wbararray &C, T bfac=1, char iflag=0) const;
1021
1022 wbararray& Plus(const wbararray &B, T bfac=1, char iflag=0);
1023
1024 bool equal (const wbararray&, const T& eps) const;
1025 bool unequal(const wbararray&, const T& eps) const;
1026 bool equal (const wbararray&, const double& eps, double& maxdiff) const;
1027 bool unequal(const wbararray&, const double& eps, double& maxdiff) const;
1028
1029 bool allEqual(const T &x) {
1030     for (size_t n=numel(), i=0; i<n; ++i) { if (data[i]!=x) return 0; }
1031     return 1;
1032 };
1033 bool allLE(const T &x) {
1034     for (size_t n=numel(), i=0; i<n; ++i) { if (data[i]>x) return 0; }
1035     return 1;
1036 };
1037
1038 char isVector() const {
1039     if (SIZE.len==1) return 1;
1040     if (SIZE.len!=2 || (SIZE[0]!=1 && SIZE[1]!=1)) return 0;
1041     if (SIZE[0]==1 && SIZE[1]==1) return -1;
1042     else return 2;
1043 };
1044
1045 bool isScalar() const { return SIZE.allEqual(1); };
1046
1047 bool requiresDataPerm(const wbarperm &P) const;
1048
1049 void markUnequalZero(const T m=1, const T eps=0) {

```

```

1050     size_t i=0, n=numel(); T z(0);
1051     if (eps==z)
1052         { for (; i<n; ++i) data[i] = (data[i]!=T(0) ? m:z); }
1053     else { for (; i<n; ++i) data[i] = (ABS(data[i])>eps ? m:z); }
1054 };
1055
1056 void toMatrixRef(
1057     wbarry &A,
1058     const UVEC &I,
1059     int pos,
1060     wbpperm &P
1061 ) const;
1062
1063 bool toMatrixRef(const char *F, int L,
1064     wbarry &A,
1065     const UVEC &I,
1066     int pos,
1067     char &tflag
1068 ) const;
1069
1070 void toMatrixRef(
1071     wbarry &A, unsigned K, const wbpperm &P0=wbpperm()
1072 ) const;
1073
1074 void toMatrixRef(
1075     wbarry &A, unsigned i, int pos, wbpperm &P
1076 ) const { return toMatrixRef( A, UVEC(1,i), pos, P); };
1077
1078 void toMatrixRef(
1079     wbarry &A, const char* sidx, int pos, wbpperm &P
1080 ) const { return toMatrixRef( A, Str2Idx(sidx,1), pos, P); };
1081
1082 wbarry& Reshape(const wbvector<size_t> &);
1083
1084 wbarry& Reshape(size_t s1, size_t s2) {
1085     WBINDEX S(2); S.data[0]=s1; S.data[1]=s2;
1086     return Reshape(S);
1087 };
1088 wbarry& Reshape(size_t s1, size_t s2, size_t s3) {
1089     WBINDEX S(3); S.data[0]=s1; S.data[1]=s2; S.data[2]=s3;
1090     return Reshape(S);
1091 };
1092
1093 wbarry<T>& Reshapel(const size_t *s, unsigned n) {
1094     if (!hasSameSize1(s,n)) wblog(FL,
1095         "ERR %s() size mismatch (%s <> %s)",FCT,
1096         SSTR_(this), wbvector<size_t>(n,s).toStrD().data);
1097     SIZE.init(n,s); return *this;
1098 };
1099 wbarry& Reshapel(size_t s1, size_t s2) {
1100     size_t s[2]={s1,s2}; return Reshapel(s,2);
1101 };
1102 wbarry& Reshapel(size_t s1, size_t s2, size_t s3) {
1103     size_t s[3]={s1,s2,s3}; return Reshapel(s,3);
1104 };
1105
1106 void GroupIndizes(size_t K);
1107
1108 wbarry& groupIndizes_DREF(
1109     wbarry &A, unsigned k1, unsigned k2
1110 ) const {
1111     size_t i, s1=1, s2=1;
1112
1113     if (k1+k2!=SIZE.len) wblog(FL,
1114         "ERR invalid index group %d+%d=%d !?",k1,k2,SIZE.len);
1115
1116     k2=SIZE.len;
1117
1118     for (i=0; i<k1; ++i) { s1*=SIZE[i]; }
1119     for ( ; i<k2; ++i) { s2*=SIZE[i]; }
1120
1121     A.SIZE.init(2); A.SIZE[0]=s1; A.SIZE[1]=s2;
1122
1123     if (sptr!=A.sptr) {
1124         A.INIT2REF(data);
1125     }
1126
1127     return A;
1128 };
1129
1130 void GroupIndizes(unsigned k1, unsigned k2) {
1131     GroupIndizes_DREF(*this,k1,k2);
1132 };
1133
1134 void groupIndizes_P(
1135     const WBINDEX&, int, wbpperm&,
1136     size_t* =NULL, size_t* =NULL

```

```

1137     ) const;
1138
1139     wbararray& transpose(const char *F, int L, wbararray &A) const;
1140     wbararray& Transpose(const char *F=0, int L=0) {
1141         wbararray X; transpose(F,L,X);
1142         return X.save2(*this);
1143     };
1144
1145     wbararray& flipLR(const char *F, int L, wbararray &A) const;
1146     wbararray& FlipLR(const char *F=0, int L=0);
1147
1148     wbararray& flipUD(const char *F, int L, wbararray &A) const;
1149     wbararray& FlipUD(const char *F=0, int L=0);
1150
1151     wbararray& MatPermute(const wbpperm &P, char iflag=0);
1152
1153     wbararray& Permute(const char* s, char iflag=0);
1154     wbararray& Permute(const WBPPerm&, char iflag=0);
1155     wbararray& permute(wbararray&, const char* s, char iflag=0) const;
1156     wbararray& permute(wbararray&, const WBPPerm&, char iflag=0) const;
1157
1158     wbararray& RPermute(C_UVEC &P, char iflag=0);
1159
1160     wbararray& select0(
1161         const WBPPerm &, unsigned dim, wbararray&) const;
1162     wbararray& Select0(const UVEC &I, unsigned dim) {
1163         wbararray A(*this); A.select0(I,dim,*this);
1164         return *this;
1165     };
1166
1167     wbararray& select0(
1168         size_t i1, size_t i2, unsigned dim, wbararray &Q
1169     ) const {
1170         if (dim>=SIZE.len) wblog(FI,
1171             "ERR dim out of bounds (%d/%d)",dim+1,SIZE.len);
1172         if (i1>=SIZE[dim] || i2>=SIZE[dim]) wblog(FI,
1173             "ERR index out of bounds (%d,%d/%d)",i1,i2,SIZE[dim]);
1174
1175         if (i2<i1) {
1176             Q.init(); Q.SIZE=SIZE; Q.SIZE[dim]=0;
1177             return Q;
1178         }
1179
1180         wbindex I; I.Index(i1,i2);
1181         return select0(I,dim,Q);
1182     };
1183
1184     wbararray& selectSqueeze(
1185         wbararray &A, unsigned p,
1186         unsigned dim) const;
1187
1188     T min() const;
1189     T max() const;
1190
1191     T aMin(char zflag=0, size_t *k=NULL) const;
1192
1193     void info(const char* = "") const;
1194     void info(const char *F, int L,
1195         const char* = "", unsigned k=-1, unsigned nlt=0, unsigned nlb=0) const;
1196
1197     void print(
1198         const char *F, int L,
1199         const char *istr="", const char *fmt=""
1200     ) const;
1201
1202     void print(const char *istr="", const char *fmt="") const {
1203         print(0,0,istr,fmt);
1204     };
1205
1206     int printdata(const char *istr="ans", const char *fmt="") const;
1207     void print_ref(const char *bstr=" ***", const char *nl="\n") const;
1208
1209     wbststring toStr() const {
1210         wbststring s;
1211         unsigned l=0;
1212         if (numel()==1 && SIZE.len<=2) {
1213             s.init(32); l=snprintf(s.data,s.len,"[ %s ]",
1214                 STR(wbvector<T>(numel(),data,'r')));
1215         }
1216         else {
1217             s.init(SIZE.len ? 5 + 10*(numel()+SIZE.len) : 3);
1218             if (SIZE.len) {
1219                 l=snprintf(s.data,s.len,"[%s] (%s)",
1220                     STR(wbvector<T>(numel(),data,'r')), sizeStr().data
1221                 );
1222             }
1223             else { strcat(s.data,"[]"); l=2; }

```



```

1224     }
1225
1226     if (l>=s.len) wblog(FL,
1227         "ERR %s() string out of bounds (%d/%d)",FCT,l,s.len);
1228     return s;
1229 };
1230
1231 void contractDiag(
1232     unsigned ic, const wbararray &B,
1233     T a, T b, wbararray &C, char Iflag=0, char bflag='N'
1234 ) const;
1235
1236 template<class TB, class TC>
1237 wbararray<TC>& contractMat(
1238     unsigned, const wbararray<TB>&, wbararray<TC>&, unsigned=1) const;
1239
1240 wbararray& ContractMat(unsigned i1, const wbararray &B, unsigned i2=1){
1241     wbararray A; this->save2(A);
1242     return A.contractMat(i1,B,*this,i2);
1243 };
1244
1245 template<class TB, class TC>
1246 wbararray<TC>& comm(
1247     const wbararray<TB>&, wbararray<TC>&, char aflag='N', char bflag='N'
1248 ) const;
1249
1250 template<class TB, class TC>
1251 wbararray<TC>& acomm(
1252     const wbararray<TB>&, wbararray<TC>&, char aflag='N', char bflag='N'
1253 ) const;
1254
1255 template<class TB, class TC>
1256 wbararray<TC>& contract(const char *F, int L,
1257     const ctrIdx&, const wbararray<TB>&,
1258     const ctrIdx&, wbararray<TC>&,
1259     const wbperm &pfinal=wbperm(), const T& afac=1.
1260 ) const;
1261
1262 template<class TB>
1263 wbararray<T>& Contract(
1264     char*, const wbararray<TB>&, char*,
1265     const wbperm &pfinal=wbperm()
1266 );
1267
1268 template<class TB>
1269 wbararray<T>& Contract(
1270     const wbvector<unsigned>&, const wbararray<TB>&,
1271     const wbvector<unsigned>&,
1272     const wbperm &pfinal=wbperm()
1273 );
1274
1275 template<class TB, class TC>
1276 wbararray<TC>& contract(const char *F, const int L,
1277     const char*, const wbararray<TB>&, const char*, wbararray<TC>&,
1278     const wbperm &pfinal=wbperm()
1279 ) const;
1280
1281 template<class TB, class TC>
1282 wbararray<TC>& contract(
1283     char *i1, const wbararray<TB> &B, char *i2, wbararray<TC> &Q,
1284     const wbperm &pfinal=wbperm()) const
1285 {
1286     return contract(FL,i1,B,i2,Q,pfinal);
1287 };
1288
1289 template<class TB, class TC>
1290 wbararray<TC>& contract(
1291     const wbvector<unsigned> &i1, const wbararray<TB> &B0,
1292     const wbvector<unsigned> &i2,
1293     wbararray<TC> &Q0, const wbperm &P) const
1294 {
1295     return contract(FL,i1,B0,i2,Q0,P);
1296 };
1297
1298 template<class TB, class TC>
1299 wbararray<TC>& contract(
1300     unsigned i1, const wbararray<TB>&B, unsigned i2, wbararray<TC>&C,
1301     const wbperm &pfinal=wbperm()) const
1302 {
1303     wbvector<unsigned> I1, I2;
1304     if (!i1 || !i2) wblog(FL,"ERR Index into contract() is 1-based!");
1305     i1--; i2--; I1.init(1,&i1); I2.init(1,&i2);
1306
1307     return contract(I1,B,I2,C, pfinal);
1308 };
1309
1310 wbararray operator+(const wbararray &B) const {

```

```

1311     wbararray C;
1312     return plus(B,C);
1313 };
1314
1315 wbararray operator-(const wbararray &B) const {
1316     wbararray C;
1317     return plus(B,C,-1);
1318 };
1319
1320 wbararray operator*(const wbararray &B) const {
1321     wbararray C;
1322     wbvector<unsigned> ica(1), icb(1); ica[0]=1; icb[0]=0;
1323     return contract(FL,ica,B,icb,C);
1324 };
1325
1326 T trace() const;
1327 wbararray& trace(unsigned i1, unsigned i2, wbararray &C) const;
1328 wbararray& trace(const ctrIdx &I1, const ctrIdx &I2, wbararray<T> &C) const;
1329
1330 mxArray* toMx() const { return toMx_Struct(); }
1331 mxArray* toMx_Struct() const;
1332 mxArray* toMx_base() const;
1333
1334 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
1335 void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const;
1336
1337 void put(const char *vname="ans", const char* ws="base") const {
1338     mxArray *a=toMx();
1339     mxPutAndDestroy(FL,a,vname,ws);
1340 };
1341
1342 void put(const char *F, int L,
1343     const char *vname="ans", const char* ws="base"
1344 ) const {
1345     mxArray *a=toMx();
1346     wblog(F,L,"I/O putting array '%s' to %s.",vname,ws);
1347     mxPutAndDestroy(F,L,a,vname,ws);
1348 };
1349
1350 void getReal(wbararray<double> &R) const;
1351 void getImag(wbararray<double> &I) const;
1352 void Conj();
1353
1354 void set(const wbararray<double> &R, const wbararray<double> &I);
1355
1356 const wbararray& opA(char tflag, wbararray& aux) {
1357     if (!isMatrix()) wblog(FL,
1358         "ERR Cannot use opA() with rank-%d object.", SIZE.len);
1359
1360     if (tflag=='N') { return *this; } else
1361     if (tflag=='T' || tflag=='C') {
1362         permute(aux, wbperm("2 1")); if (tflag=='C') aux.Conj();
1363         return aux;
1364     }
1365     wblog(FL, "ERR Invalid flag %c<%d>", tflag, tflag);
1366     return *this;
1367 };
1368
1369 bool isSym_aux(
1370     const char *file, int line, const char *fct,
1371     const wbararray &B, double eps, double* xref,
1372     const char symflag='s',
1373     const char lflag=1
1374 ) const;
1375
1376 wbvector<size_t> SIZE;
1377
1378 Wb::MTYPE mtype;
1379
1380 Wb::sptr<T> *sptr;
1381
1382 T *data;
1383
1384 protected:
1385
1386 bool isDiag_aux(const T eps, const char* task) const;
1387 bool isDiag_aux(
1388     double *eps,
1389     const char* task) const;
1390
1391 size_t serial_index(const widx_t *I, size_t n) const;
1392
1393 private:
1394
1395 void print_rec (UVEC&, size_t, const char*, const char*) const;
1396
1397 void make2D() {

```

```

1398         if (SIZE.len<2) {
1399             SIZE.Resize(2); if (SIZE[0]) { SIZE[1]=1; }
1400         }
1401         else if (SIZE.len>2) wblog(FL,
1402             "ERR make2D() - invalid usage (got rank-%d)",SIZE.len);
1403     };
1404
1405     void MemErrMsg(const char* file, int line, size_t s) {
1406         wblog(file,line, "ERR Out of memory? (%dx%d)", s, sizeof(T));
1407     };
1408
1409     int check_consistency(const char *F=0, int L=0) const {
1410         int e=0;
1411
1412         if (sptr) {
1413             if (!data || sptr->data!=data ) { e|=1; }
1414             if (sptr->check_consistency(0)) { e|=2; }
1415             if (!SIZE.len || SIZE.min()==0) { e|=4; }
1416
1417             if (e && F) wblog(F,L,
1418                 "ERR Wb::sptr = %s\n%p / %p (r=%ld, e=%d)",
1419                 STR_(sptr), sptr, data, SIZE.len, e
1420             );
1421
1422             if (mtype!=sptr->mtype) {
1423                 if (mtype || sptr->mtype!=Wb::MEM_REF) wblog(F,L,
1424                     "WRN Wb::sptr mtype = %s / %s !?",
1425                     Wb::MTYPE_STR[mtype], Wb::MTYPE_STR[sptr->mtype]);
1426             }
1427         }
1428         else {
1429             if (SIZE.len && SIZE.min()) { e|=8; }
1430             if (data) { e|=16; }
1431
1432             if (e && F) wblog(F,L,
1433                 "ERR Wb::sptr inconsistency: %p / %p",sptr,data
1434             );
1435         }
1436         return e;
1437     };
1438
1439     void DELETE_DATA(char mflag=0) {
1440
1441         if (sptr) { check_consistency(FL);
1442             if (sptr->rm_dref(mflag)) {
1443                 WB_DELETE_1(sptr);
1444             }
1445             else { sptr=NULL; }
1446             data=NULL;
1447
1448             mtype=Wb::MEM_DEF;
1449         }
1450         else if (data) wblog(FL,"ERR %s() %p / %p !?",FCT,data,sptr);
1451     };
1452
1453     wbarrray& NEW_DATA(size_t n, const T* d0=NULL, char ref=0, char init=1) {
1454         if (!n) {
1455             DELETE_DATA(); return *this;
1456         }
1457
1458         if (!ref) {
1459             if (!sptr) WB_NEW_1(sptr);
1460
1461             if (mtype && WBLOG_MMEX) { wblog(FL,
1462                 " | %s ... %p -> %s", Wb::MTYPE_STR[mtype],
1463                 this, Wb::size2Str(n*sizeof(T)).data);
1464             }
1465
1466             if (d0 || init)
1467                 { data=sptr->malloc_data(n,d0,mtype); }
1468             else { data=sptr->malloc_base(n, 0,mtype); }
1469         }
1470         else {
1471             RENEW_SPTR();
1472             data=(sptr->init2ref(n,d0));
1473         }
1474         return *this;
1475     };
1476
1477     void RENEW_SPTR() {
1478         if (sptr) { DELETE_DATA(); }
1479         WB_NEW_1(sptr);
1480     };
1481
1482     wbarrray& INIT2REF(const T *d0, const wvector<size_t> *S=NULL) {
1483         if (S) { SIZE=(*S); }
1484         size_t n=SIZE.prod(0);

```

```

1485
1486     if (d0) { if (!sptr || !isRef() || d0!=data) {
1487         RENEW_SPTR();
1488         data=(sptr->init2ref(n,d0));
1489     }}
1490     else if (n) wblog(FL,
1491         "ERR data/size inconsistency: %p / %p => %s",data,d0,SSTR_(this));
1492
1493     return *this;
1494 };
1495
1496 wbarrray& NEW(const wbvector<size_t> &S, const T* d0=NULL) {
1497     SIZE=S; NEW_DATA(SIZE.prod(0),d0);
1498     return *this;
1499 };
1500
1501 wbarrray& NEW(const wbvector<size_t> &S, size_t n, const T* d0=NULL) {
1502     SIZE.init(n,S); NEW_DATA(SIZE.prod(0),d0);
1503     return *this;
1504 };
1505
1506 T& dummy() const {
1507     static T d; memset(&d,0,sizeof(T));
1508     return d;
1509 };
1510
1511 };
1512
1513 template<class T> inline
1514 void householder(const T *u, T *x, size_t n, const T& eps);
1515
1516 template <class T>
1517 class wbarrRef {
1518 public:
1519     wbarrRef() : dc(0.), fac(1.), tflag(0), D(NULL) {};
1520
1521     wbarrRef(const wbarrRef<T> &r) {
1522         memcpy(this, &r, sizeof(r));
1523     };
1524
1525     wbarrRef<T>& operator=(const wbarrRef<T> &r) {
1526         memcpy(this, &r, sizeof(r));
1527         return *this;
1528     };
1529
1530     void print(const char *vname="ans") {
1531         printf("\nwbarrRef %s\n", vname);
1532         printf("... wbarrray : 0x%lX (%s%s)\n", D,
1533             D && D->SIZE.len==1 ? "vec of length " : "",
1534             D ? D->sizeStr().data : "");
1535         printf("... dc      : %g\n", dc);
1536         printf("... fac      : %g\n", fac);
1537         printf("... tflag     : %d\n\n", tflag);
1538     };
1539
1540     double dc;
1541     double fac;
1542     bool tflag;
1543
1544     wbarrray<T> *D;
1545
1546 protected:
1547 private:
1548 };
1549
1550 namespace NR {
1551
1552 template<class T> inline
1553 T pythag(T a, T b) {
1554     if (a<0) a=-a;
1555     if (b<0) b=-b;
1556     if (a>b)
1557         { T x=b/a; return a*SQRT(T(1)+x*x); } else if (b==0) return 0;
1558     else { T x=a/b; return b*SQRT(T(1)+x*x); }
1559 };
1560
1561 template<class T> inline
1562 T sign(const T &a, const T &b) {
1563     return (b>=0 ? (a>=0 ? a : -a) : (a>=0 ? -a : a));
1564 };
1565
1566 };
1567
1568 template<class T>

```

```

1572 wbararray<T> TestContract(
1573     const wbararray<T> &A, C_UVEC i1,
1574     const wbararray<T> &B, C_UVEC i2, const wbperm &pfinal=wbperm());
1575
1576 template<class T>
1577 void cell2mat(
1578     const wbMatrix< wbarrRef<T> > &C,
1579     wbararray<T> &M, WBINDEX &D1, WBINDEX &D2
1580 );
1581
1582 template<class T> inline
1583 size_t wbararray<T>::serial_index(const widx_t *I, size_t len) const {
1584
1585     size_t i,k, r=SIZE.len, l=r-1, nz=r-len;
1586
1587     if (len==0) return -1;
1588     if (!r || len>r || (len && data==NULL)) wblog(FL,
1589         "ERR %s[%s] having %s array",FCT,
1590         STR(WBINDEX(len,I)+1), sizeStr().data);
1591
1592 #ifdef CHECK_ELEMENT_RANGE
1593     for (i=0; i<len; i++) if (I[i]>=SIZE[i+nz]) {
1594         wblog(FL,"ERR %s[%s] index out of bounds (%s)",FCT,
1595             STR(WBINDEX(len,I)+1), sizeStr().data);
1596     }
1597 #endif
1598
1599     for (k=I[l-nz], i=l-1; i<r; i--) {
1600         k = k*SIZE[i] + ((i>=nz) ? I[i-nz] : 0);
1601     }
1602
1603 #ifdef CHECK_ELEMENT_RANGE
1604     if (k>=numel()) wblog(FL,
1605         "ERR %s() index out of bounds (%d; %s)",FCT,k,sizeStr().data);
1606 #endif
1607
1608     return k;
1609 };
1610
1611 template<class T>
1612 mxArray* wbararray<T>::toMx_Struct() const {
1613
1614     size_t i,dim1,dim2,r=SIZE.len, s=numel();
1615     WBINDEX I(r);
1616     T A;
1617
1618     if (r==2) { dim1=SIZE[0]; dim2=SIZE[1]; } else
1619     if (r==1) { dim1=SIZE[0]; dim2=(dim1 ? 1 : 0); } else
1620     if (r==0) { dim1=dim2=0; }
1621     else wblog(FL,"ERR %s() not implemented yet for rank>2 (%d)",SIZE.len);
1622
1623     mxArray *S=A.mxCreateStruct(dim1,dim2);
1624
1625     for (i=0; i<s; ++i) data[i].add2MxStruct(S,i);
1626
1627     return S;
1628 };
1629
1630 template<class T>
1631 mxArray* wbararray<T>::mxCreateStruct(unsigned m, unsigned n) const {
1632     return mxCreateCellMatrix(m,n);
1633 }
1634
1635 template<class T>
1636 void wbararray<T>::add2MxStruct(mxArray *S, unsigned i, char tflag) const {
1637
1638     if (tflag) { size_t s=0;
1639         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s) wblog(FL,
1640             "ERR %s() must follow mxCreateStruct()\n%lx, %d/%d",FCT,S,i+1,s);
1641         }
1642
1643     mxSetCell(S,i,toMx());
1644 };
1645
1646 template <class T> inline
1647 mxArray* wbararray<T>::toMx_base() const {
1648
1649     mxArray *a=NULL;
1650
1651     if (sptr && sptr->mtype) { check_consistency(FL);
1652         if (sptr->mtype==Wb::MEX_RETURN) {
1653             size_t l=P2X.BUF.size();
1654             a=P2X.Return(0,0,data);
1655
1656             if (a) {
1657                 size_t s1=numel(), s2=mxGetNumberOfElements(a);
1658                 if (s1==s2) {

```

```

1659         if (WBLOG_MMEX) wblog(FL,
1660             "2Mx %s (PSX @ %d)", STR_(sptr), 1);
1661         mxSetDimensions(a, SIZE.data, SIZE.len);
1662     }
1663     else wblog(FL,
1664         "TST %s() numel=%ld/%ld %p/%p (PSX @ %ld)",
1665         FCT, s1, s2, mxGetData(a), data, 1
1666     );
1667 }
1668 else wblog(FL, "WRN sptr=%s not in P2X(%d)", STR_(sptr), 1);
1669 }
1670 else {
1671     if (sptr->mtype!=Wb::MEM_REF)
1672         wblog(FL, "TST %s() got %s", FCT, Wb::MTYPE_STR[sptr->mtype]);
1673 }
1674 }
1675
1676 if (!a) { a=cpyRange2Mx(data, SIZE); }
1677
1678 return a;
1679 };
1680
1681 template<> mxArray* wbararray<unsigned>::toMx() const { return toMx_base(); }
1682 template<> mxArray* wbararray<int>::toMx() const { return toMx_base(); }
1683 template<> mxArray* wbararray<char>::toMx() const { return toMx_base(); }
1684 template<> mxArray* wbararray<double>::toMx() const { return toMx_base(); }
1685 template<> mxArray* wbararray<wbcomplex>::toMx() const { return toMx_base(); }
1686
1687 template<class T>
1688 void wbararray<T>::resize(const wbvector<size_t> &S, wbararray<T> &B) const {
1689
1690     if (SIZE.len!=S.len) wblog(FL,
1691         "ERR length mismatch (%d/%d)", SIZE.len, S.len);
1692
1693     if (S==SIZE) { B=(*this); return; }
1694     else { B.NEW(S); }
1695
1696     size_t i, j, k, r=S.len, l=r-1, *s1=SIZE.data, *s2=B.SIZE.data;
1697     WBINDEX I(r); T *b=B.data;
1698
1699     size_t s[r];
1700     for (i=0; i<r; i++) s[i]=MIN(S[i], SIZE[i]);
1701
1702     while (I[l]<s[l]) {
1703         for (i=j=I[l], k=l-1; k<r; k--) {
1704             i=i*s1[k]+I[k];
1705             j=j*s2[k]+I[k];
1706         }
1707
1708         b[j]=data[i];
1709
1710         k=0; I[0]++;
1711         while(I[k]>=s[k] && k<l) { I[k]=0; ++I[++k]; }
1712     }
1713 };
1714
1715 template<class T>
1716 wbararray<T>& wbararray<T>::Enlarge(
1717     size_t dim,
1718     size_t D,
1719     size_t istart
1720 ){
1721     WBINDEX S(SIZE);
1722     wbararray<T> X;
1723     wbIndex I(S);
1724
1725     if (!dim || dim>S.len) wblog(FL,
1726         "ERR %s() dimension out of bounds (%d/%d)", FCT, dim, S.len);
1727     if (D<S[--dim]) wblog(FL,
1728         "ERR %s() envalid new size (%d: %d/%d)", FCT, dim+1, S[dim], D);
1729     if (istart+S[dim]>D) wblog(FL,
1730         "ERR %s() index out of bounds (%d+%d/%d)", FCT, istart+1, S[dim], D);
1731
1732     S[dim]=D; X.init(S);
1733     if (istart) {
1734         for (size_t i=0; ++I; ++i) {
1735             I.data[dim]+=istart; X(I)=data[i];
1736             I.data[dim]-=istart;
1737         }
1738     } else { for (size_t i=0; ++I; ++i) X(I)=data[i]; }
1739
1740     X.save2(*this);
1741     return *this;
1742 };
1743
1744 template<class T>
1745 wbararray<T>& wbararray<T>::Append2(

```

```

1746     const char *F, int L,
1747     wbararray<T> &B,
1748     unsigned dim
1749 ){
1750     size_t i=0, D=0, ib=0;
1751
1752     if (B.isEmpty()) { save2(B); return B; }
1753
1754     if (SIZE.len!=B.SIZE.len) wblog(F_L,
1755     "ERR %s() incompatible objects (%s <> %s)",
1756     FCT,sizeStr().data,B.sizeStr().data);
1757     if (dim>=SIZE.len) wblog(F_L,
1758     "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
1759
1760     for (i=0; i<SIZE.len; ++i) {
1761         if (i!=dim) { if (SIZE[i]!=B.SIZE[i]) wblog(F_L,
1762         "ERR %s() size mismatch (%s <> %s @ %d/%d; %d)",
1763         FCT,sizeStr().data,B.sizeStr().data,i+1,SIZE.len,dim+1);
1764         }
1765         else { D=SIZE[i]+B.SIZE[i]; ib=B.SIZE[i]; }
1766     }
1767
1768     WBINDEX S(SIZE); S[dim]=D;
1769     wbIndex Ia(SIZE), Ib(B.SIZE);
1770     size_t &ia=Ia.data[dim];
1771     wbararray<T> X(S);
1772
1773     for (i=0; ++Ib; ++i) X(Ib)=B.data[i];
1774     for (i=0; ++Ia; ++i) {
1775         ia+=ib; X(Ia)=data[i];
1776         ia-=ib;
1777     }
1778
1779     X.save2(B); init();
1780     return B;
1781 };
1782
1783 template<class T>
1784 void wbararray<T>::Append2(
1785     const char *F, int L,
1786     wbararray<T> &B, unsigned dim,
1787     wbararray<double> &W, double eps
1788 ){
1789     size_t i,n, D=0, ib=0;
1790     const double *w=W.data;
1791
1792     if (this->isEmpty()) return;
1793     if (dim>=SIZE.len) wblog(F_L,
1794     "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
1795
1796     if (B.isEmpty()) {
1797         WBINDEX S(SIZE); S[dim]=0;
1798         B.init(S);
1799     }
1800
1801     if (SIZE.len!=B.SIZE.len) wblog(F_L,
1802     "ERR %s() incompatible objects (%s <> %s)",
1803     FCT,sizeStr().data,B.sizeStr().data);
1804     for (i=0; i<SIZE.len; ++i) { if (i!=dim) {
1805         if (SIZE[i]!=B.SIZE[i]) wblog(F_L,
1806         "ERR %s() size mismatch (%s <> %s; %d; %d/d)",
1807         FCT,sizeStr().data,B.sizeStr().data,dim+1,i+1,SIZE.len);
1808     } }
1809
1810     wbindex J(SIZE[dim]);
1811     size_t m=0, *j=J.data; n=W.numel();
1812
1813     if (!W.isVector() || n!=SIZE[dim]) wblog(F_L,
1814     "ERR %s() invalid weights W (%s <> %s @ %d)",
1815     FCT, W.sizeStr().data, sizeStr().data, dim+1);
1816     for (i=0; i<n; ++i) { if (w[i]<eps) { j[m++]=i; } }
1817
1818     if (!m) return;
1819     if (m==n) { Append2(F_L,B,dim); return; }
1820
1821     J.len=m;
1822     D=m+B.SIZE[dim]; ib=B.SIZE[dim];
1823
1824     WBINDEX S(SIZE); S[dim]=m;
1825     wbIndex Ix(S), Ib(B.SIZE); S[dim]=D;
1826     wbindex Ia(S.len),J2;
1827     wbararray<T> X(S);
1828
1829     size_t &ia=Ia.data[dim], &ix=Ix.data[dim];
1830
1831     for (i=0; ++Ib; ++i) { X(Ib)=B.data[i]; }
1832     for (i=0; ++Ix; ++i) { Ia.setp(Ix.data); ia=j[ia];

```

```

1833         ix+=ib; X(Ix)=(*this)(Ia);
1834         ix-=ib;
1835     }
1836
1837     X.save2(B);
1838     J.invert(SIZE[dim],J2); Select0(J2,dim);
1839 };
1840
1841 template<class T>
1842 wbararray<T>& wbararray<T>::init(
1843     const char *F, int L, const mxArray *a, char ref, char vec) {
1844
1845     if (!a) { init(); return *this; }
1846
1847     unsigned r=mxGetNumberOfDimensions(a);
1848     const size_t *sz=mxGetDimensions(a);
1849
1850     if (!mxIsNumChar(a)) wblog(F,L,
1851         "ERR got rank-%d array %s",r,mxGetClassName(a));
1852
1853     wvector<size_t> S(r,sz); init();
1854
1855     if (vec && r==2) {
1856         if (S[1]==1) { S.len=1; } else
1857             if (S[0]==1) { S[0]=S[1]; S.len=1; }
1858     }
1859
1860     Mx::Array<T> A(a);
1861
1862     if (ref && A.data) {
1863         INIT2REF((T*)A.data,&S);
1864     }
1865     else {
1866         init(S); A.copy_to(data);
1867         if (ref && WBLOG_TCAST) wblog(FL,
1868             "WRN ignoring ref (%s)",A.toStrT().data);
1869     }
1870
1871     return *this;
1872 };
1873
1874 template<class T>
1875 wbararray<T>& wbararray<T>::init(
1876     const char *F, int L, const mxArray *a, wperm &P
1877 ){
1878     if (!a || mxIsEmpty(a)) { init(); return *this; }
1879     if (P.isIdentityPerm()) { return init(F,L,a); }
1880
1881     unsigned i, r=mxGetNumberOfDimensions(a);
1882     const size_t *sz=mxGetDimensions(a);
1883     size_t len=1;
1884
1885     if (!mxIsNumeric(a)) wblog(F,L,
1886         "ERR got rank-%d array %s",r,mxGetClassName(a));
1887     if (!r || !sz || P.len!=r) wblog(FL,
1888         "ERR %s() got %d/%d @ sz=%p !?",FCT,P.len,r,sz);
1889
1890     SIZE.init(r);
1891     for (i=0; i<r; ++i) { SIZE[i]=sz[P[i]]; len*=sz[i]; }
1892
1893     NEW_DATA(len,NULL,0,0);
1894
1895     Mx::Array<T>(a).copyTo(data,P);
1896
1897     return *this;
1898 }
1899
1900 template<class T> inline
1901 void wbararray<T>::initTst() {
1902
1903     size_t i,j,k, r=SIZE.len, l=r-1, s=numel(), x;
1904     WBINDEX I(r);
1905
1906     for (i=0; i<s; i++) {
1907         for (j=I[l], k=l-1; k<r; k--) j = j*SIZE[k] +I[k];
1908         for (x=I[0]+1, k=1; k<r; k++) x = x*10 + (I[k]+1);
1909
1910         data[j]=(T)x;
1911
1912         k=0; I[0]++;
1913         while(I[k]>=SIZE[k] && k<l) { I[k]=0; ++I[++k]; }
1914     }
1915 }
1916
1917 template<class T> inline
1918 wbararray<T>& wbararray<T>::initIdentity(size_t d, char dflag, T dval) {
1919

```



```

1920     if (dflag) { init(d); set(T(1)); }
1921     else {
1922         init(d,d);
1923         for (size_t i=0; i<d; ++i) data[i+d*i]=dval;
1924     }
1925
1926     return *this;
1927 };
1928
1929 template<class T> inline
1930 wbararray<T>& wbararray<T>::initIdentity(
1931     const WBINDEX &S, char dflag
1932 ){
1933     size_t i,d,m=S.len/2;
1934     size_t const* const s=S.data;
1935
1936     if (S.len%2) wblog(FL,
1937         "ERR %s() requires even rank object (%s)",FCT,sizeStr().data);
1938     for (i=0; i<m; i++) if (s[i]!=s[i+m]) wblog(FL,
1939         "ERR %s() requires symmetric object (%s)",FCT,sizeStr().data);
1940     if (m>1 && dflag) wblog(FL,
1941         "ERR %s() rank-2 object required with dflag (%s)",FCT,sizeStr().data);
1942
1943     if (!m) { init(); return *this; }
1944     if (dflag) { init(s[0]).set(T(1)); return *this; }
1945     if (m==1 && s[0]==1) { init(S).set(T(1)); return *this; }
1946
1947     init(S); for (d=s[0], i=1; i<m; i++) d*=s[i];
1948     for (i=0; i<d; i++) data[i+d*i]=1;
1949
1950     return *this;
1951 };
1952
1953 template<class T> inline
1954 wbararray<T>& wbararray<T>::initIdentityB(
1955     size_t d1, size_t d2, size_t k
1956 ){
1957     init(d1,d2);
1958
1959     if (int(k)>=0) {
1960         if (k<d2) {
1961             size_t j=k, dx=d1+1; T *x=data+j*d1;
1962             for (; j<d2 && (j-k)<d1; ++j, x+=dx) { x[0]=1; }
1963         }
1964         else wblog(FL,"WRN %s() "
1965             "got all-zero block (%dx%d w/ k=%d)",FCT,d1,d2,k);
1966     }
1967     else { k=-k;
1968         if (k<d1) {
1969             size_t i=k, dx=d1+1; T *x=data+i;
1970             for (; i<d1 && (i-k)<d2; ++i, x+=dx) { x[0]=1; }
1971         }
1972         else wblog(FL,"WRN %s() "
1973             "got all-zero block (%dx%d w/ k=%d)",FCT,d1,d2,-k);
1974     }
1975
1976     return *this;
1977 };
1978
1979 template<class T> inline
1980 wbararray<T>& wbararray<T>::initIdentityB3(
1981     size_t d1, size_t d2, size_t D,
1982     size_t i0,
1983     T one
1984 ){
1985     size_t d12=d1*d2, i=0;
1986
1987     if (i0+d12>D) wblog(FL,
1988         "ERR %s() index out of bounds\n(%d*%d) x %d starting at %d",
1989         FCT,d1,d2,D,i0+1);
1990
1991     init(d1,d2,D); i0*=d12;
1992     for (; i<d12; i++) data[i0+i*d12]=one;
1993
1994     return *this;
1995 };
1996
1997 template<class T> inline
1998 void wbararray<T>::Diag2Vec() {
1999     size_t i,m;
2000     WBINDEX S(1);
2001
2002     if (!isSMatrix()) wblog(FL,
2003         "ERR Calling %s for rank-%d object (%s).",
2004         FCT, SIZE.len, sizeStr().data);
2005
2006     m=S[0]=SIZE[0];

```

```

2007     for (i=1; i<m; i++) data[i]=data[i+i*m];
2008
2009     Resize(S);
2010 };
2011
2012 template<class T> inline
2013 wbararray<T>& wbararray<T>::Reduce2Diag(T eps) {
2014
2015     if (!SIZE.len) { return *this; }
2016     else if (!isSMatrix()) wblog(FL,
2017         "ERR %s() symmetric matrix required\n"
2018         "(got rank-%d object; %s)",FCT,rank(),sizeStr().data
2019     );
2020
2021     unsigned i, j=0, n=SIZE[0];
2022     wbararray<T> X(n,1); T *d0=data;
2023
2024     X.init(n,1);
2025     for (; j<n; ++j, d0+=n) {
2026         X.data[j]=d0[j];
2027         for (i=0; i<n; ++i) { if (i!=j) {
2028             if (ABS(d0[i])>eps) wblog(FL,
2029                 "ERR %s() got off-diagonal matrix element (%g/%g)",
2030                 FCT,double(d0[i]),double(eps)
2031             );
2032         }}
2033     }
2034
2035     return X.save2(*this);
2036 };
2037
2038 template<class T>
2039 void wbararray<T>::contractDiag(
2040     unsigned ic, const wbararray<T> &B,
2041     T a, T b, wbararray<T> &C, char lflag, char bflag
2042 ) const {
2043
2044     if (B.SIZE.len!=2) wblog(FL,"ERR invalid rank-%d",SIZE.len);
2045     if (ic<1 || ic>SIZE.len) wblog(FL,
2046         "ERR invalid contraction index (ic=%d)",ic);
2047     if (!strchr("NTCc",bflag)) wblog(FL,
2048         "ERR invalid flag %c<%d>",bflag,bflag);
2049
2050     size_t i,k, s=numel(), r=SIZE.len, l=r-1;
2051     size_t n=B.SIZE.min(), m=B.SIZE[0];
2052     WBINDEX I_(r);
2053     wvector<T> h_(n);
2054     wbararray<T> X;
2055
2056     size_t *I=I_.data, *S=SIZE.data;
2057     T *h=h_.data;
2058
2059     ic--;
2060     if (n!=SIZE[ic]) wblog(FL,"ERR dimension mismatch (%d/%d)",n,SIZE[ic]);
2061     if (n==0) return;
2062
2063     for (i=0; i<n; i++) h[i]=B.data[i+i*m];
2064     if (bflag=='C' || bflag=='c') h_.Conj();
2065
2066     X=*this; if (a!=1) X*=a;
2067
2068     for (i=0; i<s; i++) {
2069         if (lflag)
2070             X.data[i]/=(b-h[I[ic]]+1E-33);
2071         else X.data[i]*=(b-h[I[ic]]);
2072
2073         k=0; I[0]++;
2074         while(I[k]>=S[k] && k<1) { I[k]=0; ++I[++k]; }
2075     }
2076
2077     if (C.isEmpty()) X.save2(C); else {
2078         if (C.SIZE!=SIZE) wblog(FL,"ERR size mismatch in C: %s <> %s",
2079             sizeStr().data, C.sizeStr().data);
2080         C+=X;
2081     }
2082 };
2083
2084 template<class T>
2085 wbararray<T>& wbararray<T>::ExpandDiagonal() {
2086
2087     size_t i,n=numel();
2088     wbararray<T> X;
2089
2090     if (!isVector())
2091         wblog(FL,"ERR %s() got %s array",FCT,SSTR_(this));
2092
2093     save2(X); init(n,n);

```

```

2094     for (i=0; i<n; ++i) data[i+n*i]=X.data[i];
2095
2096     return *this;
2097 };
2098
2099 template<class T>
2100 wbararray<T>& wbararray<T>::ExpandDiagonal(unsigned i1, unsigned i2) {
2101
2102     if (i1>=SIZE.len || i2>=SIZE.len || i1==i2) wblog(FL,
2103 "ERR %s() invalid indices (%d %d; %d)",FCT,i1+1,i2+1,SIZE.len);
2104     if (SIZE[i1]!=1 && SIZE[i2]!=1) wblog(FL,
2105 "ERR %s() array should be diagonal in (%s; %d %d)",
2106     FCT, sizeStr().data, i1+1, i2+1);
2107     if (SIZE[i1]==1 && SIZE[i2]==1) { return *this; }
2108
2109     if (SIZE[i1]!=1) SWAP(i1,i2);
2110
2111     size_t i,j,k,*s0,*s, r=SIZE.len, l=r-1;
2112     WBINDEX S=SIZE;
2113     wbararray<T> X;
2114     windex I(r);
2115
2116     save2(X); s0=X.SIZE.data;
2117     S[i1]=S[i2]; init(S); s=SIZE.data;
2118
2119     for (i=0; I[l]<s0[l]; i++) {
2120         for (j=0,k=l; k<r; k--) {
2121             if (j) { j*=s[k]; }
2122             j += (k!=i1 ? I[k] : I[i2]);
2123         }
2124         data[j] = X.data[i];
2125
2126         k=0; I[0]++;
2127         while(I[k]>=s0[k] && k<l) { I[k]=0; ++I[++k]; }
2128     }
2129     return *this;
2130 };
2131
2132 template<class T>
2133 wbararray<T>& wbararray<T>::Expand2Projector(T eps) {
2134
2135     size_t i,j,m,n=numel();
2136     WBINDEX S(2);
2137     wbvector<char> mark(n);
2138
2139     if (n==0) { if (SIZE.len>1) SIZE[0]=0; return *this; }
2140     if (!isVector()) wblog(FL,
2141 "ERR %s() array must be 1-dim vector (got %s)",
2142     FCT, sizeStr().data
2143 );
2144
2145     for (m=i=0; i<n; i++) if (ABS(data[i])>eps) { mark[i]=1; m++; }
2146
2147     S[0]=n; S[1]=m; init(S);
2148
2149     for (i=j=0; i<n; i++)
2150         if (mark[i]) { data[i+(j++)*n]=1; }
2151
2152     return *this;
2153 };
2154
2155 template<class T> inline
2156 T wbararray<T>::froNorm2(const wbararray<T> &B) const {
2157     T x=0;
2158
2159     if (!hasSameSize(B)) wblog(FL,"ERR %s() size mismatch (%s/%s)",
2160     FCT, sizeStr().data, B.sizeStr().data);
2161
2162     for (size_t n=numel(), i=0; i<n; i++) x+=CONJ(data[i])*B.data[i];
2163     return x;
2164 };
2165
2166 template<class T> inline
2167 T wbararray<T>::norm2() const {
2168     T d=0;
2169     for (size_t n=numel(), i=0; i<n; i++) d+=data[i]*data[i];
2170     return d;
2171 };
2172
2173 template<> inline
2174 wcomplex wbararray<wcomplex>::norm2() const {
2175     double d=0;
2176
2177     for (size_t n=numel(), i=0; i<n; i++)
2178         d+=data[i].abs2();
2179
2180     return d;

```

```

2181 }
2182
2183 template<class T> inline
2184 T wbararray<T>::normCol2(size_t k) const {
2185
2186     if (!isMatrix()) wblog(FL,
2187         "ERR %s() matrix expected (%s)", FCT, sizeStr().data);
2188     if (k>=SIZE[1]) wblog(FL,
2189         "ERR %s() index out of bounds (%d/%d)", FCT, k, SIZE[1]);
2190
2191     size_t i=0, dim1=SIZE[0];
2192     const T *d=data+k*dim1; T d2=0;
2193
2194     for (i=0; i<dim1; i++) d2+=d[i]*d[i];
2195     return d2;
2196 };
2197
2198 template<> inline
2199 wbcomplex wbararray<wbcomplex>::normCol2(size_t k) const {
2200
2201     if (!isMatrix()) wblog(FL,
2202         "ERR %s() matrix expected (%s)", FCT, sizeStr().data);
2203     if (k>=SIZE[1]) wblog(FL,
2204         "ERR %s() index out of bounds (%d/%d)", FCT, k, SIZE[1]);
2205
2206     size_t i=0, dim1=SIZE[0];
2207     const wbcomplex *d=data+k*dim1; double d2=0;
2208
2209     for (i=0; i<dim1; i++) d2+=d[i].abs2();
2210     return d2;
2211 };
2212
2213 template<class T> inline
2214 T wbararray<T>::scalarProd(const wbararray<T> &B) const {
2215     T x=0;
2216
2217     if (!hasSameSize(B)) wblog(FL,
2218         "ERR %s() size mismatch: %s <> %s", FCT,
2219         sizeStr().data, B.sizeStr().data);
2220
2221     for (size_t n=numel(), i=0; i<n; i++)
2222         x+=data[i]*B.data[i];
2223
2224     return x;
2225 }
2226
2227 template<> inline
2228 wbcomplex wbararray<wbcomplex>::scalarProd(
2229     const wbararray<wbcomplex> &B
2230 ) const {
2231
2232     wbcomplex x=0;
2233
2234     if (!hasSameSize(B)) wblog(FL,
2235         "ERR %s() size mismatch: %s <> %s",
2236         FCT, sizeStr().data, B.sizeStr().data);
2237
2238     for (size_t n=numel(), i=0; i<n; i++)
2239         x+=data[i]*B.data[i].conj();
2240
2241     return x;
2242 }
2243
2244 template<class T> inline
2245 T wbararray<T>::sum() const {
2246     T x=0;
2247
2248     for (size_t n=numel(), i=0; i<n; i++)
2249         x+=data[i];
2250
2251     return x;
2252 }
2253
2254 template<class T> inline
2255 wbararray<T> & wbararray<T>::sum(
2256     const WBINDEX &K,
2257     wbararray<T> &A
2258 ) const {
2259
2260     size_t i,j,k,q, r=SIZE.len, l=r-1, s=numel();
2261     WBINDEX S=SIZE;
2262     wvector<char> mark(r);
2263     char *m=mark.data;
2264     windex I;
2265
2266     for (k=i=0; i<K.len; i++) {
2267         if (K[i]<SIZE.len) { mark[K[i]]++; k++;

```

```

2268         if (mark[K[i]]>1) wblog(FL,
2269             "ERR %s() index not unique (%s; %d)",
2270             FCT,STR(K+1),SIZE.len);
2271     }
2272     else wblog(FL,
2273         "ERR %s() index out of bounds (%s; %d/%d)",
2274         FCT, STR(K+1), K[i],SIZE.len);
2275 }
2276
2277 for (k=i=0; i<mark.len; i++) if (!mark[i]) S[k++]=SIZE[i];
2278 if (k) S.len=k;
2279 else { A.init(1,1); A[0]=Wb::addRange(data,s); return A; }
2280
2281 A.init(S); I.init(SIZE.len);
2282 for (q=1; q<r; q--) if (!m[q]) break;
2283
2284 for (i=0; i<s; i++) {
2285     for (j=I[q], k=q-1; k<r; k--) if (!m[k]) j=j*SIZE[k]+I[k];
2286     A.data[j]+=data[i];
2287
2288     k=0; I[0]++;
2289     while(I[k]>=SIZE[k] && k<l) { I[k]=0; ++I[++k]; }
2290 }
2291
2292 return A;
2293 }
2294 }
2295
2296 template<class T> inline
2297 T wbararray<T>::trace() const {
2298     T x=0;
2299
2300     if (SIZE.len%2) wblog(FL,
2301         "ERR %s() got object with odd rank %d",FCT,SIZE.len);
2302     if (!isSquare()) wblog(FL,
2303         "WRN %s() of non-square array %s",FCT,SSTR_(this));
2304
2305     if (SIZE.len) {
2306         size_t n=SIZE[0], m;
2307
2308         if (SIZE.len==2) { m=SIZE[1]; }
2309         else {
2310             unsigned i=1, r2=SIZE.len/2; m=SIZE[r2];
2311             for (; i<r2; ++i) {
2312                 n*=SIZE[i];
2313                 m*=SIZE[r2+i];
2314             }
2315         }
2316
2317         if (m>n) { m=n; }
2318         for (size_t i=0; i<m; ++i) { x+=data[i+i*n]; }
2319     }
2320
2321     return x;
2322 };
2323
2324 template<class T> inline
2325 T wbararray<T>::normDiff2(const wbararray<T> &B) const {
2326     if (!hasSameSize(B)) wblog(FL,
2327         "ERR size mismatch (%s; %s)",sizeStr().data,SSTR(B));
2328     return Wb::rangeNormDiff2(data, B.data, numel());
2329 };
2330
2331 template<class T> inline
2332 T wbararray<T>::normDiff2(const wbvector<T> &b) const {
2333     if (!isVector() || numel()!=b.len) wblog(FL,
2334         "ERR size mismatch (%s; %d)",sizeStr().data,b.len);
2335     return Wb::rangeNormDiff2(data, b.data, numel());
2336 };
2337
2338 #endif
2339

```

5.16 wbararray_blas.hh

```

1 #ifndef __WB_ARRAY_BLAS_HH__
2 #define __WB_ARRAY_BLAS_HH__
3
4 // -----
5 // also link wblas.hh for wrapper routines to Fortran library
6 // extern "C" { ... }
7 // Wb, Sep16, 09
8 // -----

```

```

9
10 namespace Wb {
11
12 template<class TA, class TB, class TC>
13 wbararray<TC>& MatProd(
14     const wbararray<TA> &A, const wbararray<TB> &B,
15     wbararray<TC> &C,
16     char aflag='N', char bflag='N',
17     TA afac=1., TC cfac=0., char cforce=0
18 );
19
20 template<class T>
21 void VMatProd(
22     const wbararray<T> &A0, const wbararray<T> &B0, wbararray<T> &C,
23     char aflag='N', char bflag='N', const T afac=1., const T cfac=0.,
24     const char cforce=1
25 );
26
27 template<class T>
28 void MatVProd(
29     const wbararray<T> &A, const wbvector<T> &v, wbararray<T> &x,
30     char aflag='N', char bflag='N', const T afac=1., const T cfac=0.,
31     const char cforce=1
32 );
33
34 template<class T> T VMatVprod(
35     const wbararray<T> &v1, const wbararray<T> &M, const wbararray<T> &v2);
36
37 template<class T> T VMatVprod(
38     const wbvector<T> &v1, const wbararray<T> &M, const wbvector<T> &v2);
39
40 template<class TA, class TB, class TC>
41 void DMatProd(
42     const wbararray<TA> &A0, const wbararray<TB> &B0, wbararray<TC> &C,
43     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
44     const char cforce=1
45 );
46
47 template<class TA, class TB, class TC>
48 void DMatProd(
49     const wbvector<TA> &A0, const wbararray<TB> &B0, wbararray<TC> &C,
50     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
51     const char cforce=1
52 ){
53     wbararray<TA> A; A.init2Vec(A0.data,A0.len,'r');
54     DMatProd(A,B0,C,aflag,bflag,afac,cfac,cforce);
55 };
56
57 template<class TA, class TB, class TC>
58 void DMatProd(
59     const wbararray<TA> &A0, const wbvector<TB> &B0, wbararray<TC> &C,
60     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
61     const char cforce=1
62 ){
63     wbararray<TB> B; B.init2Vec(B0.data,B0.len,'r');
64     DMatProd(A0,B,C,aflag,bflag,afac,cfac,cforce);
65 };
66
67 };
68
69 template<class T>
70 wbararray<T>& wbInverse(
71     const char *F, int L,
72     const wbararray<T>&M, wbararray<T> &Minv
73 ){
74     wblog(FL,"ERR %s() not yet defined for T=%s",FCT,TSTR(T));
75     return Minv;
76 }
77
78 template<class T>
79 wbararray<T>& wbInverse(const char *F, int L, wbararray<T>&M);
80
81 void wbEigenS(
82     const wbararray<double>&M,
83     wbararray<double>&U, wbvector<double> &E
84 );
85
86 void wbEigenS(
87     const wbararray<wbcomplex>&M,
88     wbararray<wbcomplex>&U, wbvector<double> &E
89 );
90
91 void wbEigen_CS(
92     wbararray<wbcomplex> &M,
93     wbararray<wbcomplex>&U, wbvector<wbcomplex> &E,
94     char wjob='R', char issym='H', char tnorm=1, char qlflag=0
95 );

```

```

96
97     template<class T>
98     void wbEigen_CS_regen(
99         const wbarry<T> &M0, wbarry<T> &V, wvector<T> &E,
100         char wjob
101     );
102
103 #define SVD_BUG_SAFEGUARD
104
105     template<class T>
106     void wbSVD(
107         const wbarry<T> &A,
108         wbarry<T> &U, wvector<double> &S, wbarry<T> &Vc,
109         const WBINDEX &I=WBINDEX(),
110         char bare=0
111     );
112
113     template <class TQ>
114     void wbEigenS (
115         const wbsparray<double>&M,
116         wbsparray<double>&U, wvector<double> &E
117     ){
118         wbarry<double> Mf,Uf; M.toFull(Mf);
119         wbEigenS(Mf,Uf,E); U.init(FL,Uf,1E-14);
120     };
121
122 template<class T>
123 class opFlags {
124     public:
125
126     opFlags() : p(0), type(0) {};
127     opFlags(const opFlags &f) : p(f.p), type(0) {};
128
129     opFlags(const char *F, int L, char f)
130         : p(0), type(0) { init(F,L,f); };
131
132     opFlags(const opFlags &f1, const opFlags &f2)
133         : p(0), type(0) { init(f1,f2); };
134
135     opFlags<T>& operator= (char f) { init(f); return *this; }
136     opFlags<T>& operator= (const opFlags<T> &x) { p=x.p; return *this; }
137
138     opFlags(char f) : p(0), type(0) { init(FL,f); };
139     operator char() { return p; }
140
141     opFlags<T>& init(const char *F, int L, char f);
142     opFlags<T>& init(char f) { return init(0,0,f); };
143
144     opFlags<T>& init(const opFlags<T> &f1, const opFlags<T> &f2){
145         p=(f1.p ^ f2.p); return *this; };
146
147     char trans() const { return (p & TRANS_PAT); }
148     char conj () const { return (p & CONJ_PAT); }
149     char idle () const { return (!(p & TRANS_PAT) && !(p & CONJ_PAT)); }
150
151     void applyOrRef(
152         const char *F, int L,
153         const wbarry<T> &A0, wbarry<T> &A, const T fac=1
154     ) const;
155
156     template <class T2>
157     T2& applyOrRef(const char *F, int L, const T2 &A0, const T fac=1) const;
158
159     template <class T2>
160     const T2& applyConjOrRef(const char *F, int L, const T2 &A0, const T fac=1) const;
161
162     template <class T2>
163     char apply(const char *F, int L, T2 &A, const T fac=1) const;
164
165     char toChar() const {
166         char c, tflag=(p & TRANS_PAT), cflag=(p & CONJ_PAT);
167         if (tflag)
168             c=(cflag ? 'C' : 'T');
169         else c=(cflag ? 'c' : 'N');
170         return c;
171     }
172
173     mxArray *toMx() const { return numtoMx(this->toChar()); };
174
175     void info(const char *F, int L, const char *istr="") {
176         wblog(F,L,"<i> opFlags<%s> %s: %d -> [%d %d]",
177             TSTR(T), istr && istr[0] ? istr : "",
178             p, trans(), conj());
179     };
180
181     char p;
182

```

```

183 private:
184
185     T type;
186
187     static char TRANS_PAT;
188     static char CONJ_PAT;
189 };
190
191 template <class T> char opFlags<T>::TRANS_PAT = 1;
192 template <class T> char opFlags<T>::CONJ_PAT = 2;
193
194 template <class T> inline
195 opFlags<T>& opFlags<T>::init(
196     const char *file, int line, char f
197 ){
198     p=0;
199
200     if (f>16) {
201         if (!strchr("ENCTc",f)) wblog(__FILE__, "ERR invalid flag %c<%d>",f,f);
202         if (strchr("CT",f)) p |= TRANS_PAT;
203         if (f=='E') type=(T)'E';
204
205         if (typeid(T)==typeid(wbcomplex))
206             if (strchr("Cc",f)) p |= CONJ_PAT;
207     }
208     else {
209         p=f;
210         if (p & CONJ_PAT && typeid(T)!=typeid(wbcomplex))
211             p &= ~CONJ_PAT;
212     }
213
214     return *this;
215 };
216
217 template <class T> inline
218 void opFlags<T>::applyOrRef(
219     const char *F, int L,
220     const wbararray<T> &A0, wbararray<T> &A, const T fac
221 ) const {
222
223     unsigned r=A0.SIZE.len;
224     char p2=p;
225
226     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
227         wblog(F,L,"ERR invalid flags (%d)",p);
228
229     if (!r) { A.init(); return; }
230     if (r>2 && r%2) wblog(F,L,"ERR invalid rank-%d object (odd)",r);
231
232     if (r==1) p2 &= ~TRANS_PAT;
233
234     if (p2 || fac!=T(1)) {
235         if (p2 & TRANS_PAT) {
236             wbperm P; P.initTranspose(r);
237             A0.permute(A,P);
238         } else A=A0;
239
240         if (p2 & CONJ_PAT) A.Conj();
241
242         if (fac!=T(1)) A*=fac;
243     }
244     else A.init2ref(A0);
245 };
246
247 template <class T>
248 template <class T2> inline
249 T2& opFlags<T>::applyOrRef(
250     const char *F, int L, const T2 &A, const T fac
251 ) const {
252
253     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
254         wblog(F,L,"ERR invalid flags (%d)",p);
255
256     if (!p && fac==1) return A;
257
258     T2 *a = new T2;
259     if (p & TRANS_PAT) { A.transpose(F_L,*a); } else { (*a)=A; }
260     if (p & CONJ_PAT) a->Conj();
261     if (fac!=1) (*a)*=fac;
262
263     return (*a);
264 };
265
266 template <class T>
267 template <class T2> inline
268 const T2& opFlags<T>::applyConjOrRef(
269     const char *F, int L, const T2 &A, const T fac

```



```

270 ) const {
271
272     char c=(p & CONJ_PAT);
273     if (p && (p & ~TRANS_PAT & ~CONJ_PAT))
274         wblog(F_L,"ERR invalid flags (%d)",p);
275
276     if (!c && fac==1) return A;
277     else {
278         T2 *a = new T2(A);
279         if (c) a->Conj();
280         if (fac!=1) (*a)*=fac;
281         return (*a);
282     }
283 };
284
285 template <class T>
286 template <class T2> inline
287 char opFlags<T>::apply(
288     const char *F, int L, T2 &A, const T fac
289 ) const {
290
291     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
292         wblog(F_L,"ERR invalid flags (%d)",p);
293
294     if (p || fac!=1) {
295         if (p & TRANS_PAT) A.Transpose(F_L);
296         if (p & CONJ_PAT) A.Conj();
297         if (fac!=1) A*=fac;
298         return 1;
299     }
300     return 0;
301 };
302
303 #endif
304

```

5.17 wbbitsset.hh

```

1 #ifndef __WB_BITSET_HH__
2 #define __WB_BITSET_HH__
3
4 // ----- //
5 namespace Wb {
6 // ----- //
7
8 // arbitrary length bitset
9 class Bitset;
10
11 template <class T> class bitset;
12
13 template <class T>
14 void SetBit(T &x, unsigned i) {
15     unsigned nbits=8*sizeof(T);
16     if (i>=nbits) wblog(FL,
17         "ERR %s() bit index out of range (%d/%d)",FCT,i,nbits);
18     T r=1; r<=i; x|=r;
19 };
20
21 template <class T> inline
22 T setbit(const T &x0, unsigned i) { T x=x0; SetBit(x,i); return x; };
23
24 template <class T>
25 void UnsetBit(T &x, unsigned i) {
26     unsigned nbits=8*sizeof(T);
27     if (i>=nbits) wblog(FL,
28         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
29     T r=1; r=~(r<i)); x&=r;
30 };
31
32 template <class T> inline
33 T unsetbit(const T &x0, unsigned i) { T x=x0; UnsetBit(x,i); return x; };
34
35 template <class T>
36 bool issetbit(const T &x, unsigned i) {
37     unsigned nbits=8*sizeof(T);
38     if (i>=nbits) wblog(FL,
39         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
40     T r=1; r<=i;
41     return (x&r ? 1 : 0);
42 };
43
44 template <class T>
45 wstring bitstr(const T &x);

```

```

46
47 class Bitset: public wvector<char> {
48
49 public:
50
51     Bitset(unsigned n=0 ) : nbits(0) { NEW_BITSET(n); };
52     Bitset(const Bitset &b)
53     : wvector<char>(), nbits(0) { NEW_BITSET(b.nbits, b.data); };
54
55     Bitset& init(unsigned n=0) { return NEW_BITSET(n); };
56     Bitset& init(unsigned n, const char* b) { return NEW_BITSET(n,b); };
57
58     Bitset& setall(bool t) {
59         memset(data, t ? 255:0, len);
60         if (t && len) { (data[len-1])>=(8*len-nbits); }
61         return *this;
62     };
63
64     Bitset& set(unsigned i) {
65         if (i>=nbits) wblog(FL,
66             "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
67         unsigned k=i/8; data[k] |= (1<<(i-8*k));
68         return *this;
69     };
70
71     Bitset& unset(unsigned i) {
72         if (i>=nbits) wblog(FL,
73             "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
74         unsigned k=i/8; data[k] &= ~(1<<(i-8*k));
75         return *this;
76     };
77
78     bool isset(unsigned i) {
79         if (i>=nbits) wblog(FL,
80             "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
81         unsigned k=i/8;
82         return (data[k] & (1<<(i-8*k)) ? 1 : 0);
83     };
84
85     unsigned nnz() const {
86         unsigned i,n=0;
87         for (i=0; i<len; ++i) {
88             const char &b=data[i];
89             for (char r=1; r; r<=1) { if (b&r) ++n; }
90         }
91         return n;
92     };
93
94     bool operator==(const Bitset &b) const {
95         if (len!=b.len || nbits!=b.nbits) return 0;
96         if (data!=b.data) {
97             for (unsigned i=0; i<len; i++)
98                 if (data[i]!=b.data[i]) return 0;
99         }
100         return 1;
101     };
102
103     bool operator!=(const Bitset &b) const { return !((*this)==b); };
104
105     wstring toStr() const;
106
107     mxArray* toMx() const;
108     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
109     void add2MxStruct(mxArray *S, unsigned i) const;
110
111     void put(const char *vname, const char *ws="base"
112 ) const { put(0,0,vname,ws); };
113
114     void put(const char *file, int line,
115         const char *vname, const char *ws="base"
116 ) const;
117
118     unsigned nbits;
119
120 protected:
121
122     Bitset& NEW_BITSET(unsigned n, const char* d=NULL) {
123         nbits=n; if (n) RENEW(((n-1)/8)+1,d);
124         return *this;
125     }
126
127 };
128
129 template <class T>
130 class bitset {
131
132 public:

```

```

133
134     bitset() : data(0) {};
135     bitset(const bitset<T> &b) : data(0) { data=b.data; };
136
137     bitset& setall(bool t) {
138         memset(&data, t ? 255:0, sizeof(T));
139         return *this;
140     };
141
142     bitset& set(unsigned i) {
143         T r=1; r<=i;
144         if (r) { data |= r; } else wblog(FL,
145             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
146         return *this;
147     };
148
149     bitset& unset(unsigned i) {
150         T r=1; r<=i;
151         if (r) { data &=(~r); } else wblog(FL,
152             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
153         return *this;
154     };
155
156     bool isset(unsigned i) {
157         T r=1; r<=i;
158         if (!r) wblog(FL,
159             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
160         return (data&r ? 1 : 0);
161     };
162
163     unsigned nnz() const {
164         unsigned n=0;
165         for (T r=1; r; r<=1) { if (data&r) ++n; }
166         return n;
167     };
168
169     bool operator==(const bitset &b) const { return (data==b.data); };
170     bool operator!=(const bitset &b) const { return (data!=b.data); };
171
172     wbsting toStr() const;
173
174     mxArray* toMx() const;
175     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
176     void add2MxStruct(mxArray *S, unsigned i) const;
177
178     void put(const char *vname, const char *ws="base"
179 ) const { put(0,0,vname,ws); };
180
181     void put(const char *file, int line,
182         const char *vname, const char *ws="base"
183 ) const;
184
185     T data;
186
187     protected:
188
189 };
190
191 };
192
193 #endif
194

```

5.18 wbbblas.hh

```

1 #ifndef __WB_BLAS_HH__
2 #define __WB_BLAS_HH__
3
4 // ----- //
5 // NB! matlab's mwblas/mwlapack have their own header files
6 // specifically, starting with matlab-R2009, the LDA type
7 // (e.g. in DGEMM) is no longer int but ptrdiff_t
8 // for compatibility reasons with 64bit linux (*@&$)!
9 //
10 // trying to avoid matlabs blas.h and lapack.h as it defines
11 // other troublesome stuff, eg. second as wrapper for fortran _second,
12 // but the latter is defined/used with maps!)*&#
13
14 // #define pINT int // for matlab <= R2008
15 // #define pINT ptrdiff_t // for matlab >= R2009
16 // NB! all int, also info variable, replaced by pINT (!)
17
18 extern "C" {

```

```
19
20 #define dgemm dgemm_
21 void dgemm (
22
23     const char    &,
24     const char    &,
25     const pINT    &,
26     const pINT    &,
27     const pINT    &,
28     const double  &,
29     double [],
30     const pINT    &,
31     double [],
32     const pINT    &,
33     const double  &,
34     double [],
35     const pINT    &
36 );
37
38 #define zgemm zgemm_
39 void zgemm (
40
41     const char    &,
42     const char    &,
43     const pINT    &,
44     const pINT    &,
45     const pINT    &,
46     const wbcomplex &,
47     wbcomplex [],
48     const pINT    &,
49     wbcomplex [],
50     const pINT    &,
51     const wbcomplex &,
52     wbcomplex [],
53     const pINT    &
54 );
55
56 #define ilaenv ilaenv_
57
58 int ilaenv(
59     const pINT &,
60     const char *,
61     const char *,
62     const pINT &,
63     const pINT &,
64     const pINT &,
65     const pINT &
66 );
67
68 #define dgetri dgetri_
69 void dgetri(
70     const pINT    &,
71     double [],
72     const pINT    &,
73     pINT [],
74     double [],
75     const pINT    &,
76     pINT    &
77 );
78
79 #define dgetrf dgetrf_
80 void dgetrf(
81     const pINT    &,
82     const pINT    &,
83     double [],
84     const pINT    &,
85     pINT [],
86     pINT    &
87 );
88
89 #define dsyevd dsyevd_
90
91 void dsyevd(
92     const char    &,
93     const char    &,
94     const pINT    &,
95     double [],
96     const pINT    &,
97     double [],
98     double [],
99     const pINT    &,
100     pINT    [],
101     const pINT    &,
102     pINT    &
103 );
104
105 #define zheev zheev_
```

```

106
107 void zheev (
108     const char &,
109     const char &,
110     const pINT &,
111     wbbcomplex[],
112     const pINT &,
113     double [],
114     wbbcomplex[], const pINT &,
115     double [],
116     pINT &
117 );
118
119 #define zheevd zheevd_
120
121 void zheevd(
122     const char &,
123     const char &,
124     const pINT &,
125     wbbcomplex[],
126     const pINT &,
127     double [],
128     wbbcomplex[], const pINT &,
129     double [], const pINT &,
130     pINT [], const pINT &,
131     pINT &
132 );
133
134 #define dgeev dgeev_
135 void dgeev (
136
137     const char &,
138     const char &,
139     const pINT &,
140     double [],
141     const pINT &,
142     double [],
143     double [],
144     double [],
145     const pINT &,
146     double [],
147     const pINT &,
148     double [],
149     const pINT &,
150     pINT &
151 );
152
153 #define zgeev zgeev_
154 void zgeev (
155
156     const char &,
157     const char &,
158     const pINT &,
159     wbbcomplex [],
160     const pINT &,
161     wbbcomplex [],
162     wbbcomplex [],
163     const pINT &,
164     wbbcomplex [],
165     const pINT &,
166     wbbcomplex [],
167     const pINT &,
168     double [],
169     pINT &
170 );
171
172 #define dgesvd dgesvd_
173
174 void dgesvd (
175     const char &,
176     const char &,
177     const pINT &,
178     const pINT &,
179     double [],
180     const pINT &,
181     double [],
182     double [],
183     const pINT &,
184     double [],
185     const pINT &,
186
187     double [],
188     const pINT &,
189     pINT &
190 );
191
192 #define dgesdd dgesdd_

```

```

193
194 void dgesdd (
195     const char &,
196     const pINT &,
197     const pINT &,
198     double [],
199     const pINT &,
200     double [],
201     double [],
202     const pINT &,
203     double [],
204     const pINT &,
205
206     double [],
207     const pINT &,
208     pINT [],
209     pINT &
210 );
211
212 #define zgesvd zgesvd_
213
214 void zgesvd (
215     const char &,
216     const char &,
217     const pINT &,
218     const pINT &,
219     wcomplex [],
220     const pINT &,
221     double [],
222     wcomplex [],
223     const pINT &,
224     wcomplex [],
225     const pINT &,
226     wcomplex [],
227     const pINT &,
228     wcomplex [],
229     pINT &
230 );
231
232 #define zgesdd zgesdd_
233
234 void zgesdd (
235     const char &,
236     const pINT &,
237     const pINT &,
238     wcomplex [],
239     const pINT &,
240     double [],
241     wcomplex [],
242     const pINT &,
243     wcomplex [],
244     const pINT &,
245     wcomplex [],
246     const pINT &,
247     double [],
248     pINT [],
249     pINT &
250 );
251
252 }
253
254 class STAT_DGEMM {
255 public:
256     STAT_DGEMM() { nn.init(32); ss.init(32); }
257     ~STAT_DGEMM() {
258         unsigned i; double fac=nn.max(); fac=1./(fac!=0 ? fac:1);
259         if (nn.max()<1) return;
260
261         wblog(FL,"%NDGEMM statistics");
262         printf("\n idx_n  avg.dim      count      cost (^2.5)\n\n");
263         for (i=0; i<nn.len; i++) if (nn[i])
264             printf("  %3d  %8.2f %12d  %9g\n", i,
265                 pow(double(ss[i])/nn[i], 1/3.), nn[i],
266                 pow(double(ss[i])/nn[i], 2.5/3) * nn[i]*fac);
267         printf("\n");
268     };
269
270     void account(unsigned n0) {
271         unsigned i,n=n0;
272         for (i=0; i<nn.len; i++) { n+=1; if (n==0) break; }
273         if (i>nn.len) i=nn.len-1;
274         nn[i]++; ss[i]+=n0;
275     };
276
277     wvector<unsigned> nn;
278     wvector<unsigned long> ss;
279

```

```

280 private:
281
282 };
283
284 #ifdef WB_CLOCK
285
286     STAT_DGEMM stat_dgemm;
287
288     Wb::Clock wbc_matprod("Wb::MatProd");
289
290     Wb::Clock wbc_dgemm("dgemm");
291
292     Wb::Clock wbc_zgemm("zgemm");
293     Wb::Clock wbc_zgeev("zgeev");
294     Wb::Clock wbc_zgeNN("zgeNN");
295     Wb::Clock wbc_zgeNX("zgeNX");
296
297     Wb::Clock wbc_dgesvd("dgesvd");
298     Wb::Clock wbc_zgesvd("zgesvd");
299
300 #endif
301
302 #endif
303

```

5.19 wbclock.hh

```

1 #ifndef __WB_CLOCK_HH__
2 #define __WB_CLOCK_HH__
3
4 #include <set>
5
6 namespace Wb {
7
8     class Clock;
9
10     void pause(const char *F, int L, double tsec);
11     void pause(double tsec) { pause(0,0,tsec); }
12     wstring sec2Str(double t);
13
14 };
15
16 set<Wb::Clock*> gwb_Clocks;
17
18 #define CHRONO_NOW std::chrono::high_resolution_clock::now
19
20 #define CHRONO_CLOCK std::chrono::time_point<std::chrono::high_resolution_clock>
21 #define CHRONO_DIFF2SEC(A) (1E-9 * \
22 double(std::chrono::duration_cast<std::chrono::nanoseconds>(A).count()))
23
24 namespace Wb {
25
26     class Clock {
27
28     public:
29
30         Clock(const char *s=0)
31             : ttot(0), ctot(0), tneg(0), tref(), cref(0), nz(0), ncall(0)
32             { init(s); gwb_Clocks.insert(this); };
33
34         ~Clock() {
35             if (ncall) stop();
36             if (WBLOG_CLK) info();
37             name.init(); gwb_Clocks.erase(this);
38         };
39
40         void Init() { info(); name.init(); }
41
42         void init(const char *s=NULL) { char s_[32]; reset();
43             #ifndef myname_TAG
44             if (s) { snprintf(s_, 32, "%.16s", myname_TAG, s); name=s_; }
45             else { name = myname_TAG; }
46             #elif defined(myname)
47             if (s) { snprintf(s_, 32, "%.16s:%s", myname, s); name=s_; }
48             else { name = myname; }
49             #else
50             name = s? s:"";
51             #endif
52         };
53
54         void reset() { ttot=ctot=tneg=0; cref=0; nz=ncall=0; }
55
56         void start() {

```

```

57         cref=clock(); ctot=ttot=tneg=0; nz=0; ncall=1;
58         tref=CHRONO_NOW();
59     };
60
61     void resume() {
62         if (!ncall) { start(); } else
63         if ((ncall%2)==0) {
64             cref=clock(); ++ncall;
65             tref=CHRONO_NOW();
66         }
67     };
68
69     int stop(const char *F=NULL, int L=0) {
70         if (ncall%2) { double tw,tc; ++ncall;
71             ttot+=(tw=get_dt_wall());
72             ctot+=(tc=double(clock()-cref)/CLOCKS_PER_SEC);
73
74             if (tw<=0 || tc<=0) { ++nz; if (tw) tneg-=tw; }
75             return 1;
76         }
77         else {
78             if (!ncall) wblog(F_L,
79                 "WRN clock '%s' not yet started",name.data? name.data:"");
80             return 0;
81         }
82     };
83
84     double gettime(char cflag=0) const;
85
86     mxArray* toMx (const char *F=NULL, int L=0);
87     mxArray* toMxS(const char *F=NULL, int L=0);
88
89     void info(const char *istr=NULL) const;
90
91     double ttot, ctot, tneg;
92
93     CHRONO_CLOCK tref;
94     clock_t cref;
95
96     size_t nz, ncall;
97
98     wbststring name;
99
100 protected:
101 private:
102
103     double get_dt_wall() const {
104         return ( CHRONO_DIFF2SEC( CHRONO_NOW() - tref) );
105     };
106
107     double getTickFreq() const;
108 };
109
110 class Clock_resume {
111 public:
112
113     Clock_resume(Clock *t_) : t(t_) { t->resume(); };
114
115     ~Clock_resume() { if (t) t->stop(); };
116
117     int stop() {
118         int r=0; if (t) { r=t->stop(); t=NULL; }
119         return r;
120     };
121
122     int Switch(Clock *t_) {
123         int r=0; if (t) r=t->stop(); t=t_; t->resume();
124         return r;
125     };
126
127     Clock *t;
128
129 protected:
130 private:
131 };
132
133 };
134
135 unsigned showAllClocks() {
136     set<Wb::Clock*>::iterator i=gwb_Clocks.begin();
137     for (; i!=gwb_Clocks.end(); ++i) { (*i)->info(); }
138     return gwb_Clocks.size();
139 };
140
141 unsigned initAllClocks() {
142     set<Wb::Clock*>::iterator i=gwb_Clocks.begin();
143     for (; i!=gwb_Clocks.end(); ++i) { (*i)->info(); (*i)->reset(); }

```



```

144     return gwb_Clocks.size();
145 };
146
147 namespace Wb {
148
149 class UseClock {
150 public:
151
152     UseClock(Wb::Clock *c) : C(c) { if (C) C->resume(); };
153     ~UseClock() { if (C) C->stop(); };
154
155     void done() { if (C) { C->stop(); C=0; }; }
156     void stop() { if (C) C->stop(); };
157     void resume() { if (C) C->resume(); };
158
159 protected:
160 private:
161
162     Wb::Clock *C;
163 };
164
165 };
166
167 double Wb::Clock::gettime(char cflag) const {
168
169     double dt=0;
170
171     if (cflag) {
172         dt=ctot; if (ncall%2) {
173             dt+=double(clock()-cref)/CLOCKS_PER_SEC; }
174     }
175     else {
176         dt=ttot; if (ncall%2) {
177             dt+=get_dt_wall(); }
178     }
179
180     return dt;
181 };
182
183 void Wb::Clock::info(const char *istr) const {
184
185     static int first_call=1;
186
187     if (!ncall) return;
188
189     char s[16];
190     double tc=gettime('c'), tw=gettime();
191
192     if (nz) {
193         double p0=1-nz/(double)(ncall/2);
194         snprintf(s,16,"%5.1g%%", 100*p0);
195     } else s[0]=0;
196
197     if (first_call) { first_call=0;
198         printf("\n %-30s Count   CPU-time   Wall-time       / call\n",
199             "Wb::Clocks");
200     }
201
202     printf("%c %-25s %10ld %10s %10s %9.3g %s\n",
203         (ncall%2) ? '*' : ' ', istr? istr:name.data, ncall/2,
204         sec2Str(tc).data, sec2Str(tw).data,
205         tw/(ncall/2), s
206     );
207 };
208
209 double Wb::Clock::getTickFreq() const {
210
211     unsigned i,j,m=512,n=32;
212     clock_t c1,c2,nc=0;
213
214     CHRONO_CLOCK t1,t2;
215     double dt=0;
216
217     for (i=j=0; i<n; ++i) {
218         t1=CHRONO_NOW(); c1=clock(); while (c1==clock()) { if (++j>m) break; };
219         t2=CHRONO_NOW(); c2=clock();
220
221         if (i) {
222             nc+=(c2-c1);
223             dt+=CHRONO_DIFF2SEC( t2-t1 );
224         }
225     }
226
227     return (dt/nc);
228 };
229
230 namespace Wb {

```

```

231     class Profile;
232
233     class line_stat__ {
234     public:
235         line_stat__() : dt(0), aux(0), ncall(0) {};
236
237         void add(double dt_, double dx) { dt+=dt_; aux+=dx; ++ncall; };
238
239         double dt, aux;
240         size_t ncall;
241
242     protected:
243     private:
244     };
245
246     void save_and_clear_Profiling();
247 };
248
249     map< std::string, map<int, Wb::line_stat__ > > gwb_Profs;
250
251 namespace Wb {
252 class Profile {
253     public:
254
255     Profile(const char *F, int L, const char *istr=0) {
256         unsigned i=0, l=0, n=32; char fl[32];
257         if (!F || !F[0] || !L) wblog(FL,"ERR %s() invalid empty F !?",FCT);
258
259         for (; F[l]; ++l) { if (F[l]=='/') { l=i+1; }}
260         for (i=0; F[l] && i<n; ++l, ++i) {
261             if (isalpha(F[l])) { fl[i]=F[l]; } else
262             if ((F[l]=='.' || F[l]=='_') && i) { fl[i]='_'; } else { fl[i]='X'; }
263         }; l=i;
264
265         if (l<n) { l+=snprintf(fl+l,n-l,"_%03d",L); }
266         if (l<n && istr && istr[0]) { l+=snprintf(fl+l,n-l,"%s",istr); }
267         if (l>=n) { fl[n-1]=0; wblog(FL,
268             "ERR %s() string out of bounds (%d/%d)\n%s",FCT,l,n,fl);
269         }
270         std::string fl_=fl;
271
272         #pragma omp critical (gwb_Profs__)
273         { auto im=gwb_Profs.find(fl_);
274             if (im==gwb_Profs.end()) {
275                 gwb_Profs[fl_]=map<int, Wb::line_stat__ >();
276                 if ((im=gwb_Profs.find(fl_))==gwb_Profs.end())
277                     { wblog(FL,"ERR %s() map entry failed !?",FCT); }
278             }
279             M=&(im->second);
280         }
281         tic();
282     };
283
284     void tic() { tref=CHRONO_NOW(); };
285     void toc(int L, double x=0) {
286         double dt=CHRONO_DIFF2SEC( CHRONO_NOW() - tref );
287
288         #pragma omp critical (gwb_ProfL__)
289         { auto im=M->find(L);
290             if (im==M->end()) { (*M)[L]=Wb::line_stat__();
291                 if ((im=M->find(L))==M->end())
292                     { wblog(FL,"ERR %s() map entry failed !?",FCT); }
293             }
294             im->second.add(dt,x);
295         }
296     };
297
298     CHRONO_CLOCK tref;
299
300     protected:
301     private:
302
303     map<int, Wb::line_stat__> *M;
304 };
305
306 };
307
308 #endif
309

```

5.20 wbcomplex.hh

```

1 #ifndef __WB_COMPLEX_HCC__

```

```

2 #define __WB_COMPLEX_HCC__
3
4 class wbcomplex;
5
6 /* ----- */
7 /* ----- */
8 // complex number class
9 // Wb,Dec20,01
10
11 class wbcomplex {
12
13 public:
14
15     wbcomplex() : r(0.), i(0.) {};
16
17     wbcomplex(const wbcomplex &z) = default;
18
19     wbcomplex(double r0, double i0=0.) : r(r0), i(i0) {};
20
21     ~wbcomplex() = default;
22
23     wbcomplex& operator= (const wbcomplex &z) = default;
24
25     wbcomplex& operator= (const double &dbl) {
26         r=dbl; i=0.; return *this;
27     };
28
29     explicit operator bool() const { return (r!=0 || i!=0); };
30     bool operator! () const { return (r==0 && i==0); };
31
32     operator double() const {
33         if (i!=0) wblog(FL,
34             "ERR got double(%.4g%+.4gi) type conversion !?",r,i);
35         return r;
36     };
37
38     wbcomplex& init(double r0, double i0) { r=r0; i=i0; return *this; };
39     wbcomplex& set(double r0, double i0) { r=r0; i=i0; return *this; };
40
41     wbcomplex& setRand(double fac=2., double shift=-1.);
42
43     wbcomplex& init_expi(double ph) {
44         r=std::cos(ph); i=std::sin(ph);
45         return *this;
46     };
47
48     mxComplexDouble& explicit_copy2(mxComplexDouble &zm) const {
49         zm.real=r; zm.imag=i;
50         return zm;
51     };
52
53     bool isinf() const { return (std::isinf(r) || std::isinf(i)); };
54     bool isnan() const { return (std::isnan(r) || std::isnan(i)); };
55
56     bool operator== (const wbcomplex &z) const {
57         return (float(r)==float(z.r) && float(i)==float(z.i));
58     };
59     bool operator!= (const wbcomplex &z) const {
60         return (float(r)!=float(z.r) || float(i)!=float(z.i));
61     };
62
63     bool operator== (const double &dbl) const {
64         return (i==0. && float(r)==float(dbl)); };
65     bool operator!= (const double &dbl) const {
66         return (i!=0. || float(r)!=float(dbl)); };
67
68     bool operator> (const wbcomplex &z) const {
69         return (r>z.r || (r==z.r && i>z.i) ? 1 : 0); };
70
71     bool operator>= (const wbcomplex &z) const {
72         return (r>z.r || (r==z.r && i>=z.i) ? 1 : 0); };
73
74     bool operator< (const wbcomplex &z) const {
75         return (r<z.r || (r==z.r && i<z.i) ? 1 : 0); };
76
77     bool operator<= (const wbcomplex &z) const {
78         return (r<z.r || (r==z.r && i<=z.i) ? 1 : 0); };
79
80     wbcomplex& operator+= (const wbcomplex &z) {
81         r+=z.r; i+=z.i; return *this; };
82     wbcomplex& operator-= (const wbcomplex &z) {
83         r-=z.r; i-=z.i; return *this; };
84     wbcomplex& operator*= (const wbcomplex &z) {
85         double r_;
86         r_ = r*z.r - i*z.i;
87         i = r*z.i + i*z.r; r=r_; return *this;
88     };

```

```

89     wbcomplex& operator/= (const wbcomplex &z);
90
91     wbcomplex& operator*= (double dbl) { r*=dbl; i*=dbl; return *this; }
92     wbcomplex& operator+= (double dbl) { r+=dbl; return *this; }
93     wbcomplex& operator-= (double dbl) { r-=dbl; return *this; }
94
95     wbcomplex operator+ (const wbcomplex &z) const {
96         return wbcomplex(r+z.r, i+z.i); }
97     wbcomplex operator- (const wbcomplex &z) const {
98         return wbcomplex(r-z.r, i-z.i); }
99     wbcomplex operator* (const wbcomplex &z) const {
100         return wbcomplex( r*z.r-i*z.i, i*z.r+r*z.i); }
101     wbcomplex operator/ (const wbcomplex &z) const;
102
103     wbcomplex operator+ (const double &dbl) const {
104         return wbcomplex(r+dbl,i); }
105     wbcomplex operator- (const double &dbl) const {
106         return wbcomplex(r-dbl,i); }
107     wbcomplex operator* (const double &dbl) const {
108         return wbcomplex(r*dbl, i*dbl); }
109     wbcomplex operator/ (const double &dbl) const {
110         if (dbl==0.) wblog(FL,"ERR DIV/0! (%g+%gi)/%g",r,i,dbl);
111         return wbcomplex(r/dbl, i/dbl);
112     };
113
114     wbcomplex operator/= (double dbl) {
115         if (dbl==0.) wblog(FL,"ERR DIV/0! (%g+%gi)/%g",r,i,dbl);
116         r/=dbl; i/=dbl; return *this;
117     };
118
119     wbcomplex times(const wbcomplex &) const;
120
121     wbcomplex ConjProd(const wbcomplex &z) const {
122         return wbcomplex(r*z.r+i*z.i, -i*z.r+r*z.i);
123     };
124
125     void Conj() { i=-i; };
126     wbcomplex conj() const { return wbcomplex(r,-i); }
127
128     double abs2() const { return double(r*r+i*i); }
129     double abs() const;
130
131     wbcomplex sqrt() const;
132
133     wbcomplex sign() const {
134         wbcomplex z;
135         double a=abs(); if (a) { z.init(r/a, i/a); }
136         return z;
137     };
138
139     double arg() const;
140     double arg_degree() { return (180./M_PI)*arg(); }
141
142     wbcomplex round() const {
143         return wbcomplex( ::round(r), ::round(i) );
144     };
145
146     wbsting toStr(const char *fmt="%.4g") const;
147     void put (const char *vname="ans", const char *ws="base") const;
148
149     double r,i;
150
151 protected:
152
153 private:
154 };
155
156 inline wbcomplex wbcomplex::operator/ (const wbcomplex &b) const {
157     double x, den;
158
159     if (fabs(b.r)>=fabs(b.i)) {
160         if (b.r==0) { wblog(FL,"WRN DIV/0"); return wbcomplex(Inf,Inf); }
161         x=b.i/b.r;
162         den = b.r + x*b.i;
163         return wbcomplex((r+x*i)/den, (i-x*r)/den);
164     }
165     else {
166         x=b.r/b.i;
167         den = b.i + x*b.r;
168
169         return wbcomplex((x*r+i)/den, (x*i-r)/den);
170     }
171 };
172
173 inline wbcomplex& wbcomplex::operator/= (const wbcomplex &z) {
174
175     double r_, dbl=1./z.abs2();

```

```

176     if (z==0.) wblog(FL,"ERR DIV/0! (%g+i%g)/(%g+i%g)",r,i,z.r,z.i);
177
178     r_ = (r * z.r + i * z.i) * dbl;
179     i = (i * z.r - r * z.i) * dbl; r = r_; return *this;
180 };
181
182 inline wbcomplex wbcomplex::times(const wbcomplex &z) const {
183     double a,b; a=r*z.r; b=i*z.i;
184     wbcomplex zz(a-b, (r+i)*(z.r+z.i)-a-b);
185     return zz;
186 };
187
188 inline wbcomplex& wbcomplex::setRand(double fac, double shift) {
189     static char first_call=1;
190
191     if (first_call) { wb_srand(); first_call=0; }
192     fac/=(double)RAND_MAX;
193
194     if (shift==0.)
195         { r=fac*rand(); i=fac*rand(); }
196     else { r=fac*rand()+shift; i=fac*rand()+shift; }
197
198     return *this;
199 };
200
201 inline double wbcomplex::abs() const {
202     double a=0.;
203     double x=fabs(r), y=fabs(i);
204
205     if (!x) { a=y; } else
206     if (!y) { a=x; } else if (x>y)
207         { a=y/x; a = x*std::sqrt(1.+a*a); }
208     else { a=x/y; a = y*std::sqrt(1.+a*a); }
209
210     return a;
211 };
212
213 inline wbcomplex wbcomplex::sqrt() const {
214     wbcomplex z;
215     if (!r && !i) { return z; }
216
217     double ar=fabs(r), ai=fabs(i), w, x;
218     if (ar>=ai)
219         { x=ai/ar; w=std::sqrt(ar)*std::sqrt(0.5*(1.+std::sqrt(1+x*x))); }
220     else { x=ar/ai; w=std::sqrt(ai)*std::sqrt(0.5*(x +std::sqrt(1+x*x))); }
221
222     if (r>=0.) { z.init( w, i/(2.*w) ); } else
223     if (i>=0.) { z.init( i/(2.*w), w ); } else { z.init(-i/(2.*w),-w ); }
224
225     return z;
226 };
227
228 inline double wbcomplex::arg() const {
229     double phi=0.;
230
231     if (!i) { if (r<0) { phi+=M_PI; }; } else
232     if (!r) { phi=( i>0? +0.5 : -0.5 )*M_PI; }
233     else {
234         phi=atan(i/r);
235         if (r<0) { phi+=(i>0 ? M_PI : -M_PI); }
236     }
237     return phi;
238 };
239
240 void wbcomplex::put(const char *vname, const char *ws) const {
241
242     mxArray *a=NULL;
243     if (i)
244         { a=Mx::Array<wbcomplex>(1,1).copyFromTR(this); }
245     else { a=Mx::Array<double>(1,1).copyFromTR(&r ); }
246
247     int e=mexPutVariable(ws,vname,a);
248     if (e==1) wblog(FL,
249         "ERR failed to put variable %s to workspace '%s'",vname,ws);
250
251     mxDestroyArray(a);
252 };
253
254 inline wbcomplex operator+(const wbcomplex &z) { return z; }
255 inline wbcomplex operator-(const wbcomplex &z) {
256     return wbcomplex(-z.r, -z.i);
257 };
258 inline wbcomplex operator*(double d, const wbcomplex &z) {
259     return wbcomplex(z.r*d, z.i*d);
260 };
261 inline wbcomplex operator+(double d, const wbcomplex &z) {
262     return wbcomplex(d+z.r, +z.i);

```

```

263 };
264 inline wbcomplex operator-(double d, const wbcomplex &z) {
265     return wbcomplex(d-z.r, -z.i);
266 };
267 inline wbcomplex operator/(double d, const wbcomplex &z) {
268     return wbcomplex(d,0)/z;
269 };
270 inline bool isnan(const wbcomplex &z) { return z.isnan(); };
271
272 template <class T> inline T real(const T &d) { return d; };
273 template <class T> inline T imag(const T &d) { return 0; };
274 template <class T> inline T conj(const T &d) { return d; };
275
276 template <class T> inline T CONJ(const T &d) { return d; };
277 template <class T> inline T ABS (const T &a) { return (a<0 ? -a : a); };
278 template <class T> inline T ABS2(const T &d) { return d*d; };
279 template <class T> inline T NORM (const T &a){ return (a<0 ? -a : a); };
280 template <class T> inline T NORM2(const T &d){ return d*d; };
281
282 inline double real(const wbcomplex &z) { return z.r; };
283 inline double imag(const wbcomplex &z) { return z.i; };
284 inline wbcomplex conj(const wbcomplex &z) {
285     return wbcomplex(z.r,-z.i);
286 };
287
288 inline double ROUND(const double &d) { return ::round(d); }
289 inline wbcomplex ROUND(const wbcomplex &d) { return d.round(); }
290
291 #ifdef __WB_MPFR_HH__
292 template <unsigned P> inline
293 Wb::mpfr__<P> ROUND(const Wb::mpfr__<P> &q) { return q.round(); };
294 #endif
295
296 template <class T> inline
297 T SQRT(const T &d) { return std::sqrt(d); }
298
299 template <> inline
300 wbcomplex SQRT(const wbcomplex &d) { return d.sqrt(); }
301
302 #ifdef __WB_MPFR_HH__
303 template <unsigned P> inline
304 Wb::mpfr__<P> SQRT(const Wb::mpfr__<P> &q) { return q.sqrt(); };
305 #endif
306
307 template <> inline
308 double ABS(const double &a) { return std::fabs(a); };
309 template <> inline
310 double NORM(const double &a) { return std::fabs(a); };
311
312 template <> inline
313 wbcomplex CONJ(const wbcomplex &a) { return a.conj(); };
314
315 inline double ABS (const wbcomplex &a) { return a.abs(); };
316 inline double ABS2 (const wbcomplex &a) { return a.abs2(); };
317 inline double NORM (const wbcomplex &a) { return a.abs(); };
318 inline double NORM2(const wbcomplex &a) { return a.abs2(); };
319
320 inline bool ISINF(const double &a) { return std::isinf(a); }
321 inline bool ISINF(const wbcomplex &a) {
322     return (std::isinf(a.r) || std::isinf(a.i)); }
323
324 inline bool ISNAN(const double &a) { return std::isnan(a); }
325 inline bool ISNAN(const wbcomplex &a) {
326     return (std::isnan(a.r) || std::isnan(a.i)); }
327
328 inline double REAL(const double &a) { return a; }
329 inline double REAL(const wbcomplex &a) { return a.r; }
330
331 inline double IMAG(const double &a __attribute__((unused))) { return 0; }
332 inline double IMAG(const wbcomplex &a) { return a.i; }
333
334 #ifdef __WB_MPFR_HH__
335
336 template <unsigned P> inline
337 Wb::mpfr__<P> ABS(const Wb::mpfr__<P> &a) { return a.abs(); };
338 template <unsigned P> inline
339 Wb::mpfr__<P> NORM(const Wb::mpfr__<P> &a) { return a.abs(); };
340
341 template <unsigned P> inline
342 Wb::mpfr__<P> ABS2(const Wb::mpfr__<P> &a) { return a.abs2(); };
343 template <unsigned P> inline
344 Wb::mpfr__<P> NORM2(const Wb::mpfr__<P> &a) { return a.abs2(); };
345
346 #endif
347
348 template <class T> inline double ABSDIFF_F(
349     const T &d1, const T &d2

```

```

350 ){ return double(fabs(float(d1)-float(d2))); };
351
352 template <> inline double ABSDIFF_F(
353     const wbcomplex &d1, const wbcomplex &d2
354 ){
355     float dr=float(d1.r)-float(d2.r), di=float(d1.i)-float(d2.i);
356     return std::sqrt(double(dr*dr+di*di));
357 };
358
359 inline wbcomplex exp(const wbcomplex &z) {
360     double r=exp(z.r);
361     return wbcomplex(r*cos(z.i), r*sin(z.i));
362 };
363
364 inline wbcomplex expi(const double &x) {
365     return wbcomplex(cos(x), sin(x));
366 };
367
368 inline wbsting toStr(const wbcomplex &z, const char* fmt="%.4g");
369 inline wbsting toStr(const double &d, const char* fmt="%.4g");
370
371 #endif
372

```

5.21 wbeigs.hh

```

1 #ifndef __WB_EIGS_HH__
2 #define __WB_EIGS_HH__
3
4 // ----- //
5 // ----- //
6
7 #define AUPDFUN dsaupd
8 #define EUPDFUN dseupd
9 #define AUPDSTR "DSAUPD"
10 #define EUPDSTR "DSEUPD"
11
12 extern "C" {
13
14 #define dsaupd dsaupd_
15 void dsaupd (
16
17     const int    &,
18     const char   &,
19     const int    &,
20     const char[2] ,
21     const int    &,
22     const double &,
23     double[],
24     const int    &,
25     double[],
26     const int    &,
27     const int[11] ,
28     const int[11] ,
29     double[],
30     double[],
31     const int    &,
32
33     const int    &
34 );
35
36 #define dseupd dseupd_
37 void dseupd (
38
39     const int    &,
40     const char   &,
41     const int[]  ,
42     const double[],
43     const double[],
44     const int    &,
45     const double &,
46
47     const char   &,
48     const int    &,
49     const char[2] ,
50     const int    &,
51     const double &,
52     double[],
53     const int    &,
54     double[],
55     const int    &,
56     const int[11] ,
57     const int[11] ,

```

```

58         double[],
59         double[],
60     const int    &,
61     const int    &
62 );
63
64 }
65
66 char* info2str(int info) {
67     char s[64];
68     switch (info) {
69         case -1: strcpy(s,"N<=0"); break;
70         case -2: strcpy(s,"NEV<=0"); break;
71         case -3: strcpy(s,"NCV<=NEV || NCV>N"); break;
72         case -4: strcpy(s,"MAXIT<0"); break;
73         case -5: strcpy(s,"invalid WHICH"); break;
74         case -6: strcpy(s,"invalid BMAT"); break;
75         case -7: strcpy(s,
76             "length of private work array WORKL not sufficient"); break;
77         case -8: strcpy(s,"error return from trid. eigenvalue "
78             "calculation (from lapack::dsteqr)"); break;
79         case -9: strcpy(s,"starting vector is zero"); break;
80         case -10: strcpy(s,"invalid MODE=IPARAM[6]"); break;
81         case -11: strcpy(s,"IPARAM[6]==1 && BMAT=='G' are incompatible"); break;
82         case -12: strcpy(s,"invalid IPARAM[0] (must be equal to 0 or 1)"); break;
83         case -13: strcpy(s,"NEV && WHICH=='BE' are incompatible."); break;
84         case -9999: strcpy(s,"could not build an Arnoldi factorization."); break;
85         case -15: strcpy(s,"HOWMNY must be one of 'A' or 'S' if RVEC==true"); break;
86         case -16: strcpy(s,"HOWMNY = 'S' not yet implemented"); break;
87         case -17: strcpy(s,"data inconsistency between DSAUPD and DSEUPD"); break;
88         default: sprintf(s,"unknown info=%d",info);
89     }
90
91     return s;
92 }
93
94 #endif
95

```

5.22 wbfht.hh

```

1
2 #ifndef __WB_FFTW_C__
3 #define __WB_FFTW_C__
4
5 #include <rfftw.h>
6
7 namespace Wb {
8
9 int FFT2D (
10     wbMatrix<double> &X2,
11     wbMatrix<double> &F2,
12     char dir
13 ){
14
15     int info=-1, thesame=(X2==F2);
16     wbvector<int> dim(2);
17
18     if (dir=='F') {
19         if (X2.isEmpty()) wblog(FL,
20             "ERR %s - invalid dim [%d,%d]",FCT,X2.dim1,X2.dim2);
21
22         dim[0] = X2.dim1;
23         dim[1] = X2.dim2;
24
25         if (!thesame) F2=X2;
26
27         F2.Resize(dim[0],dim[1]+2);
28
29         info=FFT_ND(dim,F2[0],dir);
30     }
31     else
32     if (dir=='B') {
33         if (F2.isEmpty()) wblog(FL,
34             "ERR %s - invalid dim [%d,%d]",FCT,F2.dim1,F2.dim2);
35
36         dim[0] = F2.dim1;
37         dim[1] = F2.dim2-2;
38
39         if (!thesame) X2 = F2;
40
41         info = FFT_ND (dim, X2[0], dir);
42

```



```

43         X2.Resize(dim[0], dim[1]);
44     }
45     else
46         wblog(FL, "ERR %s - invalid dir=%c<%d>", FCT, dir, dir);
47
48     return info;
49 }
50
51 int FFT_ND (
52     wbvector<int> &dim,
53     double *zz,
54     char dir
55 ){
56
57     unsigned i; int s=dim.prod(), info=-1;
58
59     if (dim.len==0 || s<=0) wblog(FL,
60         "ERR %s - invalid dimensions [%s]", FCT, dim.toStr().data);
61
62     if (dim.last()%2) wblog(FL,
63         "ERR %s - invalid dimensions [%s]\n"
64         "leading/last dimension must be even!", FCT, dim.toStr().data);
65
66     if (dir=='F') {
67
68         rfftwnd_plan plan;
69
70         plan = rfftwnd_create_plan (
71             dim.len, dim.data, FFTW_FORWARD, FFTW_ESTIMATE | FFTW_IN_PLACE);
72
73         rfftwnd_one_real_to_complex (plan, zz, NULL);
74
75         rfftwnd_destroy_plan(plan);
76         info=0;
77     }
78
79     else if (dir=='B') {
80
81         double fact; int ntot, nlast=dim.last();
82         rfftwnd_plan plan;
83
84         plan = rfftwnd_create_plan (
85             dim.len, dim.data, FFTW_BACKWARD, FFTW_ESTIMATE | FFTW_IN_PLACE);
86
87         rfftwnd_one_complex_to_real (plan, (fftw_complex*)zz, NULL);
88
89         for (ntot=1, i=0; i<dim.len; i++) ntot *= dim[i];
90         fact = 1./ntot;
91         ntot = (ntot / nlast) * (nlast+2);
92         for (i=0; i<ntot; i++) zz[i] *= fact;
93
94         rfftwnd_destroy_plan(plan);
95         info = 0;
96     }
97     else wblog(FL, "ERR %s - invalid dir=%c<%d>", dir, dir);
98
99     return info;
100 }
101
102 }
103
104 #endif
105

```

5.23 wbindex.hh

```

1  #ifndef __WB_INDEX_HH__
2  #define __WB_INDEX_HH__
3
4  // ----- //
5  // index class
6  // Wb, Apr19, 06
7  // ----- //
8
9  // inherit class wbvector<unsigned>
10
11 class wbindex : public wbvector<widx_t> {
12
13 public:
14
15     wbindex(widx_t n=0, char iflag=0) {
16         if (iflag) Index(n); else init(n);
17     };
18

```

```

18
19     wbindx(const ctrIdx& ic) { init(ic); };
20
21     explicit wbindx(widx_t n, const widx_t* d)
22     : wbvector<widx_t>(n,d) {};
23
24     explicit wbindx(const char* s, unsigned offset=0) {
25         int i=Str2Idx(s,*this,offset);
26         if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
27     };
28
29     wbindx& init(widx_t n=0, const widx_t *d=NULL) {
30         wbvector<widx_t>::init(n,d);
31         return *this;
32     };
33
34     wbindx& init(const ctrIdx &ic);
35
36     wbindx& init(
37         const char *F, int L, const mxArray *a, widx_t offset=0);
38
39     widx_t init(
40         const char *F, int L, const mxArray *a,
41         const char *istr, char check_type=1
42     ){ return wbvector<widx_t>::init(F,L,a,istr,check_type); };
43
44     wbindx& init(const mxArray *a, widx_t offset=0) {
45         return init(0,0,a,offset);
46     };
47
48     wbindx& initGroup(
49         const wbvector<widx_t> &P, wbvector<widx_t> &D
50     ){
51         widx_t i,d, k=0, ig=0, m=D.sum();
52         if (m!=P.len) wblog(FL,
53             "ERR %s() invalid group index (%d/%d)",FCT,m,P.len);
54         init(P.len+D.len);
55
56         for (; ig<D.len; ++ig) {
57             data[k+ig]=d=D[ig]; m=ig+1;
58             for (i=0; i<d; ++i, ++k) data[k+m]=P[k];
59         }
60
61         return *this;
62     };
63
64     wbindx& initGroup0(
65         const wbvector<widx_t> &P, wbvector<widx_t> &D
66     ){
67         widx_t l=0, i=0; init(D.len);
68         for (; i<D.len; ++i) {
69             if (l>=P.len) wblog(FL,"ERR %s() index out of bounds",FCT,l,P.len);
70             data[i]=P[l]; l+=D[i];
71         }
72         return *this;
73     };
74
75     void initStr(const char* s, widx_t offset=0) {
76         int i=Str2Idx(s,*this,offset);
77         if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
78     };
79
80     bool isIndex(widx_t n=0) {
81         if (n) {
82             if (swidx_t(n)>0) {
83                 for (widx_t i=0; i<len; ++i) {
84                     if (data[i]!=(n+i)) return 0; }
85             }
86             else {
87                 n=(1-n-len); if (swidx_t(n)<0) return 0;
88                 for (widx_t i=0; i<len; ++i) {
89                     if (data[i]!=(n+i)) return 0; }
90             }
91         }
92         else {
93             for (widx_t i=0; i<len; ++i) { if (data[i]!=i) return 0; }
94         }
95         return 1;
96     };
97
98     wbindx& Index(widx_t n) {
99         init(n);
100         if (n) { for (widx_t i=0; i<n; ++i) data[i]=i; }
101         return *this;
102     };
103
104     wbindx& Index_ex(widx_t n, widx_t ix) {

```

```

105     widx_t i,k;
106
107     if (ix>=n) wblog(FL,
108     "WRN index to be excluded of no relevance (%d/%d)",ix,n);
109
110     init(ix<n ? n-1 : n); if (n==0) return *this;
111
112     for (k=i=0; i<n; i++) if (i!=ix) data[k++]=i;
113
114     return *this;
115 };
116
117 wbindex& Index(widx_t i1, widx_t i2) {
118     if (i1>i2) { init(); return *this; }
119     init(i2-i1+1);
120     for (widx_t i=0; i<len; ++i) data[i]=i+i1;
121
122     return *this;
123 };
124
125 wbindex& Index(widx_t i1, widx_t i2, int p) {
126     if (i1>i2) { init(); return *this; }
127
128     widx_t n=i2-i1+1; p=(-p)%int(n);
129     if (!p || n==1) { return Index(i1,i2); }
130     if (p<0) p+=n;
131
132     init(n); {
133         widx_t i, ix=i1+p, *d=data;
134         for (i=ix; i<=i2; ++i, ++d) { (*d)=i; }
135         for (i=i1; i< ix; ++i, ++d) { (*d)=i; }
136     }
137     return *this;
138 };
139
140 wbindex& BlockIndex(const wbvector<widx_t> &D, char sub=0) {
141     widx_t i,j,d,l; init(D.sum());
142
143     if (sub==0) {
144         for (l=i=0; i<D.len; ++i)
145             for (d=D[i], j=0; j<d; ++j,++l) data[l]=i;
146     }
147     else {
148         for (l=i=0; i<D.len; ++i)
149             for (d=D[i], j=0; j<d; ++j,++l) data[l]=j;
150     }
151
152     return *this;
153 };
154
155 wbindex& BlockIndex(widx_t n, widx_t D) {
156     widx_t i,j,l; init(n*D);
157
158     for (l=i=0; i<n; ++i)
159         for ( j=0; j<D; ++j,++l) data[l]=i;
160
161     return *this;
162 };
163
164 wbindex& BlockIndexI(widx_t n, widx_t D) {
165     widx_t i,j,l; init(n*D);
166
167     for (l=j=0; j<D; ++j)
168         for ( i=0; i<n; ++i,++l) data[l]=i;
169
170     return *this;
171 };
172
173 wbindex& IndexTranspose(widx_t m, widx_t n) {
174     widx_t i,j,l;
175     init(n*m); if (!n || !m) return *this;
176
177     for (l=j=0; j<m; ++j) {
178         data[l++]=j;
179         for (i=1; i<n; ++i,++l) data[l]=data[l-1]+m;
180     }
181
182     return *this;
183 };
184
185 wbindex& operator= (const char* s) {
186     int i=Str2Idx(s,*this,0);
187     if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
188     return *this;
189 };
190
191 int extend2Perm(widx_t N, wbpperm &P, char toend=0) const;

```

```

192
193     void toPerm(wbperm &P) const {
194         P.initT(len, data);
195
196         if (!P.isValidPerm()) { print("this");
197             wblog(FL, "WRN No valid perm."); }
198     };
199
200     wbindex& invert(widx_t N, wbindex &I2, char uflag=1) const;
201
202     wbindex& Invert(widx_t N, char uflag=1) {
203         wbindex X; save2(X);
204         return X.invert(N,*this,uflag);
205     };
206
207     wbindex flipIdx(widx_t ndim) const;
208     char isUnique(widx_t imax, widx_t offset=0) const;
209
210     mxArray* toMx(const char tflag=0) const {
211         return toMx_offset(1,tflag);
212     };
213
214     protected:
215     private:
216
217 };
218
219 wbperm::wbperm (const wbindex &P) {
220     NEW(P.len,P.data);
221     if (!isValidPerm()) dispInvalidPerm(FL);
222 };
223
224 class wbIndex : public wbvector<widx_t> {
225
226     public:
227
228     wbIndex() : wbvector<widx_t>() {};
229     wbIndex(const wbvector<widx_t> &S) { init(S); };
230
231     template<class TI>
232     wbIndex(widx_t n, const TI *S) { init(n,S); };
233
234     wbIndex(const wbIndex &I) : wbvector<widx_t>() {
235         if (!I.isEmpty()) {
236             wbvector<widx_t>::init(I.len); SIZE=I.SIZE;
237             if (len) --data[0];
238         }
239     };
240
241     wbIndex& init() {
242         wbvector<widx_t>::init(); SIZE.init();
243         return *this;
244     };
245
246     wbIndex& init(const wbvector<widx_t> &S) {
247         const widx_t* const &s=S.data;
248         for (widx_t k=0; k<S.len; k++) if (!s[k]) {
249             wbvector<widx_t>::init(); SIZE.init();
250             return *this;
251         }
252
253         wbvector<widx_t>::init(S.len); SIZE=S;
254         if (len) --data[0];
255         return *this;
256     };
257
258     wbIndex& reset() {
259         if (len!=SIZE.len) wblog(FL,
260             "ERR %s() severe size mismatch (%d/%d)",FCT,len,SIZE.len);
261         if (len) {
262             data[0]=-1;
263             for (widx_t i=1; i<len; ++i) data[i]=0;
264         }
265         return *this;
266     };
267
268     template<class TI>
269     wbIndex& init(unsigned n, const TI *S) {
270         for (unsigned k=0; k<n; ++k) if (!S[k]) {
271             wbvector<widx_t>::init(); SIZE.init();
272             return *this;
273         }
274
275         wbvector<widx_t>::init(n); SIZE.initT(n,S);
276         if (len) --data[0];
277         return *this;
278     };

```

```

279
280 wbindex& set(wbvector<widx_t> &I) {
281     widx_t k=0, *const &i=I.data, *const &s=SIZE.data;
282     if (I.len!=len) wblog(FL,
283         "ERR wbindex::%s() size mismatch (%d/%d)",FCT,I.len,len);
284     for (; k<len; k++) if (i[k]>=s[k]) {
285         wblog(FL,"ERR wbindex::%s() index out of bounds (%d: %d/%d)",
286             FCT,k+1,i[k],s[k]);
287         data[k]=i[k];
288     }
289     return *this;
290 };
291
292 bool isValid() const {
293     if (len!=SIZE.len) return 0;
294     for (widx_t i=0; i<len; ++i) if (data[i]>=SIZE.data[i]) return 0;
295     return 1;
296 };
297
298 void checkValid(const char *F, int L) const {
299     if (!isValid()) wblog(F_L,
300         "ERR %s() invalid hyperindex\n%s",FCT,toStr().data);
301 };
302
303 bool operator++() {
304     widx_t k=0, l=len-1, *const &s=SIZE.data;
305
306     if (len!=SIZE.len) wblog(FL,
307         "ERR wbindex::%s() got corrupted index (%d/%d)",
308         FCT, len, SIZE.len
309     );
310     if (!len) return 0;
311
312     k=0; data[0]++;
313     while (data[k]==s[k] && k<l) { data[k]=0; ++data[++k]; }
314
315     if (data[k]>=s[k]) {
316         if (k!=l) wblog(FL,
317             "WRN wbindex::%s() premature stop (%d/%d: %s)",
318             FCT, k+1,len, toStr().data);
319         return 0;
320     }
321     return 1;
322 };
323
324 bool operator--() {
325     widx_t k=0, l=len-1, *const &s=SIZE.data;
326
327     if (len!=SIZE.len) wblog(FL,
328         "ERR wbindex::%s() got corrupted index (%d/%d)",
329         FCT, len, SIZE.len
330     );
331     if (!len) return 0;
332
333     k=0; data[0]--;
334     while (s[widx_t](data[k])==s[k]-1 && k<l) { data[k]=s[k]-1; --data[++k]; }
335
336     if (data[k]>=s[k]) {
337         if (k!=l) wblog(FL,
338             "WRN wbindex::%s() premature stop (%d/%d: %s)",
339             FCT, k+1,len, toStr().data);
340         return 0;
341     }
342     return 1;
343 };
344
345 bool isEmpty() const {
346     if (len!=SIZE.len) wblog(FL,"ERR %d/%d",len,SIZE.len);
347     if (!len) return 1;
348     else {
349         const widx_t *s=SIZE.data;
350         for (widx_t i=0; i<len; i++) if (!s[i]) return 1;
351     }
352     return 0;
353 };
354
355 bool gotmore() const {
356     if (len!=SIZE.len) wblog(FL,"ERR %d/%d",len,SIZE.len);
357     for (widx_t i=0; i<len; ++i) {
358         if (data[i]+1<SIZE.data[i]) return 1; else
359         if (data[i]>=SIZE.data[i]) wblog(FL,
360             "WRN %s() index out of bounds (%s)",FCT,toStr().data
361         );
362     }
363     return 0;
364 };
365

```

```

366 bool isdiag() const {
367     if (len%2) wblog(FL,
368         "ERR %s() applies to even-rank objects only (%s)",
369         FCT,toStrf("","x").data
370     );
371     for (widx_t r=len/2, i=0; i<r; i++) {
372         if (data[i]!=data[i+r]) return 0;
373     }
374     return 1;
375 };
376
377 widx_t numel() const { return SIZE.prod(0); };
378
379 widx_t serial(
380     const wbvector<widx_t> *S2=NULL
381 ){
382     widx_t is, k=len-1, *const &s=(S2 ? S2->data : SIZE.data);
383     if (!len) wblog(FL,"ERR wbIndex::%s() got empty index",FCT);
384     for (is=data[k--]; k<len; --k) { is = is*s[k]+data[k]; }
385     return is;
386 };
387
388 wbsting sizeStr() const { return SIZE.toStrf("","x"); };
389 wbsting toStr() const {
390     char s[128]; snprintf(s,128,"[%s] %s",
391         wbvector<widx_t>::toStr().data,
392         SIZE.toStrf("","x").data);
393     return s;
394 };
395
396 mxArray* toMx() const {
397     const char *fields[]={"S","idx"};
398     mxArray *S=mxCreateStructMatrix(1,1,2,fields);
399
400     mxSetFieldByNumber(S,0,0, SIZE.toMx());
401     mxSetFieldByNumber(S,0,1, ((*this)+1).toMx());
402     return S;
403 };
404
405 wbvector<widx_t> SIZE;
406
407 protected:
408 private:
409
410 };
411
412 template <class T=widx_t>
413 class groupIndex {
414
415 public:
416
417     groupIndex() {};
418
419     groupIndex(
420         const wbvector<T> &P_, const wbvector<T> &D_,
421         const wperm_t **p=NULL
422     ){ P=P_; D=D_; setup(); if (p) (*p)=P.data; };
423
424     groupIndex(const wbvector<T> &DI, T ng=-1) { init(DI,ng); }
425
426     void init() { P.init(); D.init(); IO.init(); };
427
428     groupIndex& init(
429         const wbvector<T> &P_, const wbvector<T> &D_
430     ){ P=P_; D=D_; setup(); return *this; };
431
432     groupIndex& initX(
433         wbvector<wperm_t> &P_, wbvector<T> &D_,
434         const wperm_t **p=NULL
435     ){
436         P_.save2(P); D_.save2(D); setup(); if (p) (*p)=P.data;
437         return *this;
438     };
439
440     groupIndex& init(const wbvector<T> &DI, T ng=-1) {
441         if (DI.isEmpty()) { init(); return *this; }
442
443         if (long(ng)<0) {
444             T n=0,l=0;
445             for (; l<DI.len; ++n, l+=(1+DI[l]));
446             if (l!=DI.len) wblog(FL,
447                 "ERR %s() invalid group index (%d/%d)",FCT,l,DI.len);
448             ng=n;
449         }
450         P.init(DI.len-ng); D.init(ng);
451
452         T i,d,k=0,l=0; T *dp=P.data;

```

```

453     while (l<DI.len) { d=DI.data[l++];
454         for (i=0; i<d; ++i, ++k, ++l) { dp[k]=DI.data[l]; }
455     }
456     if (k!=P.len || l!=DI.len) wblog(FL,
457         "ERR %s() %d/%d %d/%d",FCT,k,P.len,l,DI.len);
458     setup(); return *this;
459 };
460
461 groupIndex& init(T ng) {
462     D.init(ng); IO.init(ng+1); P.init();
463     return *this;
464 };
465
466 groupIndex& init_g(T ig, T nz) {
467     if (ig>=D.len || ig+1>=IO.len) wblog(FL,"ERR %s() "
468         "index out of bounds (%d/%d,%d)",FCT,ig,D.len,IO.len-1);
469     if (ig==0) IO[ig]=0;
470     IO[ig+1]=nz; D[ig]=nz-IO[ig];
471     return *this;
472 };
473
474 groupIndex& setD(T i, T nz) {
475     if (i>=D.len) wblog(FL,"ERR %s() "
476         "index out of bounds (%d/%d)",FCT,i,D.len);
477     D[i]=nz; return *this;
478 };
479
480 T init_ic() {
481     if (IO.len!=D.len+1) wblog(FL,"ERR %s() "
482         "IO not yet initialized !? (%d/%d+1)",FCT,IO.len,D.len+1);
483     T i=0; IO[0]=0;
484     for (; i<D.len; ++i) { IO[i+1]=IO[i]+D[i]; }
485     return IO[i];
486 };
487
488 groupIndex& setup(char xflag=0) {
489     T n=D.cumsum_(IO,'N'); if (n!=P.len) wblog(FL,
490         "ERR %s() severe size mismatch (%d/%d)",FCT,n,P.len);
491     if (xflag) { Ig.init(P.len);
492         for (T l=0, j=0, d=0, i=0; i<D.len; ++i, l+=d) {
493             for (d=D[i], j=0; j<d; ++j) Ig[P[l+j]]=i;
494         }
495     }
496     return *this;
497 };
498
499 T dim(T ig) {
500     if (ig>=D.len) wblog(FL,
501         "ERR %s() index out of bounds (%d/%d)",FCT,ig,D.len);
502     return D.data[ig];
503 };
504
505 T numGroups() const { return D.len; };
506
507 void getGroup(T ig, T* &p0, T &d) const {
508     if (ig>=D.len || IO.len!=D.len+1) wblog(FL,
509         "ERR %s() index out of bounds (%d/%d/%d)",FCT,ig,IO.len-1,D.len);
510     p0=P.data+IO.data[ig];
511     d=D.data[ig];
512 };
513
514 T getGroup(T ig, T &j, T &j2) const {
515     if (long(ig)==-1) return 0;
516     if (ig>=D.len || IO.len!=D.len+1) wblog(FL,
517         "ERR %s() index out of bounds (%d/%d/%d)",FCT,ig,IO.len-1,D.len);
518     j=IO.data[ig];
519     j2=IO.data[ig+1]; return ig+1;
520 };
521
522 T getGroup(T ig) const {
523     if (long(ig)==-1) return 0;
524     if (ig>=IO.len) wblog(FL,"ERR %s() "
525         "index out of bounds (%d/%d/%d)",FCT,ig,IO.len-1,D.len);
526     return IO.data[ig];
527 };
528
529 template <class T2>
530 void getRecs0(const wbMatrix<T2> &, wbMatrix<T2> &, T m) const;
531
532 T groupIter(T &ig, T &j, T &j2) const {
533     if (long(ig)>=0) { ++ig; } else { ig=0;
534         if (IO.len!=D.len+1) wblog(FL,"ERR %s() "
535             "index out of bounds (%d/%d/%d)",FCT,ig,IO.len-1,D.len);
536     }
537     if (ig<D.len) {
538         j=IO.data[ig];
539         j2=IO.data[ig+1];

```

```

540     }
541     else {
542         ig=-1;
543     }
544
545     return ig+1;
546 };
547
548 mxArray* toMx() const {
549     const char *fields[]={"P","D","IO","pp"};
550     mxArray *S=mxCreateStructMatrix(1,1,4,fields);
551
552     mxSetFieldByNumber(S,0,0,(P+1).toMx());
553     mxSetFieldByNumber(S,0,1,D.toMx());
554     mxSetFieldByNumber(S,0,2,(IO+1).toMx());
555
556     wbvector<T> p;
557     mxArray *c=mxCreateCellMatrix(1,D.len);
558
559     for (T i=0; i<D.len; ++i) {
560         p.init(D[i],P.data+IO[i]);
561         mxSetCell(c,i,p.toMx());
562     }
563     mxSetFieldByNumber(S,0,3,c);
564
565     return S;
566 };
567
568 wbvector<T> D;
569
570 wbperm P;
571
572 wbvector<T> IO;
573
574 wbvector<T> Ig;
575
576 protected:
577 private:
578 };
579
580 #define IDT unsigned long
581 #define ITAG_LEN sizeof(IDT)
582
583 #define CC_ITAG '*'
584
585 #define IS_ITAGS_SEP(c) ((c)==' ' || (c)=='\n' || (c)=='\t')
586 #define IS_CHAR_ITAGS(c) ((c)>32 && (c)<127)
587 #define IS_CHAR_ITAG(c) (((c)>32 && (c)<127) && !((c)==CC_ITAG || IS_ITAGS_SEP(c)))
588
589 #define IT2STR(a) (a).itags.toStr().data
590 #define IT2STR__ itags.toStr().data
591
592 class iTag {
593
594 public:
595
596     iTag() : t(0) {};
597     iTag(const char *F, int L, const char *s) { init(F,L,s); };
598     iTag(const char *s) { init(FL,s); };
599     iTag(const iTag &x) : t(x.t) {};
600
601     iTag& init(const char* F, int L, const char *s, unsigned n=-99);
602
603     iTag& init(const char *s=NULL) { return init(FL,s); };
604
605     iTag& init_qdir(char c) { t=0;
606         if (c=='-' || CC_ITAG) Conj();
607         else if (c!='+') wblog(FL,"ERR %s() invalid c=%c<%d>",FCT,c,c);
608         return *this;
609     };
610
611     template<class TQ, class TD>
612     iTag& init(const char *F, int L,
613         const wbvector< QSpace<TQ,TD> > &A, wbindx ia);
614
615 #ifndef NOMEX
616     iTag& init(const char* F, int L, const mxArray *a);
617 #endif
618
619     iTag& operator=(unsigned n) { t=n; return *this; }
620     iTag& operator=(const char *s) { return init(FL,s); }
621
622     bool operator==(const iTag &x) const { return sameAs(x); };
623     bool operator!=(const iTag &x) const { return !sameAs(x); };
624     bool operator! () const { return !t; };
625
626     char sameAs(const iTag &x, char lflag=0) const;

```



```

627
628 bool isConj(const iTag &x, char full=1) const {
629     IDT c128=128;
630     return (full ? (t^c128)==x.t : (t&c128) ^ (x.t & c128) );
631 };
632
633 bool isConjx(const iTag &x, char lflag) const;
634 int isValid() const;
635
636 bool isEmpty() const {
637     return (t & ~(IDT)(128) ? 0 : 1);
638 };
639
640 explicit operator bool() const { return (t!=0); };
641
642 iTag& tSet(const iTag &x) {
643     if (isConj() ^ x.isConj()) { t=x.t; Conj(); } else { t=x.t; }
644     return *this;
645 };
646
647 iTag& Set_(const iTag &x, char keep=0) {
648     if (isEmpty() && !keep) { t=x.t; } else
649     if (isConj() ^ x.isConj()) { Conj(); }
650     return *this;
651 };
652
653 iTag& Conj() {
654     ((char&t) ^= char(128);
655     return *this;
656 };
657
658 iTag& setConj() {
659     ((char&t) |= char(128);
660     return *this;
661 };
662
663 bool isConj() const {
664     return (((char&t)<0);
665 };
666
667 iTag& deConj() {
668     char &c=((char&t)); if (c<0) c+=char(128);
669     return *this;
670 };
671
672 iTag conj() const { iTag x(*this); return x.Conj(); }
673 iTag deconj() const { iTag x(*this); return x.deConj(); }
674
675 iTag& SetFlag(unsigned k) {
676     if (k>=ITAG_LEN) wblog(FL,
677     "ERR %s() index out of bounds (%d/%d)",FCT,k,ITAG_LEN);
678     ((char*)(&t))[k] ^= char(128);
679     return *this;
680 };
681
682 iTag& UnsetFlag(unsigned k) {
683     if (k>=ITAG_LEN) wblog(FL,
684     "ERR %s() index out of bounds (%d/%d)",FCT,k,ITAG_LEN);
685     ((char*)(&t))[k] &= ~char(128);
686     return *this;
687 };
688
689 iTag& UnsetFlags() {
690     unsigned k=1, n=ITAG_LEN;
691     char *q = (char*)(&t), c128=char(128);
692     for (; k<n; ++k) { if (q[k]<0) {
693         if (k>1) wblog(FL,
694         "WRN %s() got iTag flag at %d/%d",FCT,k,n);
695         q[k]+=c128;
696     }}
697     return *this;
698 };
699
700 iTag& RmItag() {
701     char *s = (char*)(&t), c128=char(128);
702     unsigned k=0, n=ITAG_LEN; for (; k<n; ++k) { s[k]&=c128; }
703     return *this;
704 };
705
706 iTag& AppendChar(char q, const char *qs=NULL);
707 iTag& PrependChar(char q, const char *qs=NULL);
708 int CheckFirstChar(char q, const char *qs=NULL);
709
710 iTag& SetK() {
711     if (CheckFirstChar('K',"AEX")>0) return *this;
712     return AppendChar('K',"KD~");
713 };

```

```

714     iTag& SetD() {
715         if (CheckFirstChar('D',"AEX")>0) return *this;
716         return AppendChar('D',"KD~");
717     };
718
719     iTag& SetM() { return AppendChar('~',"KD~"); };
720     iTag& AddTilde() { return AppendChar('~',"KD~"); };
721
722     unsigned to_str(char *s, unsigned len, char cflag=1) const;
723
724     wbsting toStr(char cflag=1) const {
725         wbsting s(ITAG_LEN+4);
726         to_str(s.data,s.len,cflag);
727         return s;
728     };
729
730     mxArray* toMx() const { return toStr().toMx(); };
731
732     IDT t;
733
734 protected:
735 private:
736
737 };
738
739 class IndexLabels : public wbvector<iTag> {
740
741 public:
742
743     IndexLabels(unsigned n=0) : wbvector<iTag>(n) {};
744     IndexLabels(const char *F, int L, const char *s) { init(F,L,s); };
745     IndexLabels(const char *s) { init(FL,s); };
746
747     unsigned init(const char* F, int L, const char *s);
748     unsigned init(const char *s) { return init(FL,s); };
749
750     IndexLabels& init(unsigned n=0) {
751         wbvector<iTag>::init(n);
752         return *this;
753     };
754
755     IndexLabels& init(const IndexLabels &b) {
756         wbvector<iTag>::init(b); return *this;
757     };
758     IndexLabels& operator=(const IndexLabels &b) {
759         wbvector<iTag>::init(b); return *this;
760     };
761
762     IndexLabels& MatchSet(const IndexLabels &b) {
763         if (len!=b.len) { wblog(FL,
764             "ERR %s() itag length mismatch '%s' | '%s' (%d/%d)",
765             FCT,STR_(this),STR(b),len,b.len);
766         }
767         for (unsigned i=0; i<len; ++i) {
768             if (!data[i].isEmpty()) { if (data[i]!=b.data[i]) {
769                 wblog(FL,"ERR %s() itag mismatch '%s' | '%s'",
770                     FCT,STR_(this),STR(b),len,b.len);
771             }}
772             else if (data[i].isConj() ^ b.data[i].isConj()) {
773                 wblog(FL,"ERR %s() itag conj mismatch '%s' | '%s'",
774                     FCT,STR_(this),STR(b),len,b.len);
775             }
776         }
777         return init(b);
778     };
779
780 #ifdef LOAD_CGC_QSPACE
781     unsigned init_qdir(const char *s);
782 #endif
783
784 #ifndef NOMEX
785     unsigned init(const char* F, int L, const mxArray *a);
786 #endif
787
788     IndexLabels& init3() {
789         wbvector<iTag>::init(3); data[2].Conj();
790         return *this;
791     };
792
793     IndexLabels& init(size_t l1, const iTag* d1, size_t l2, const iTag* d2) {
794         wbvector<iTag>::init(l1,d1,l2,d2);
795         return *this;
796     };
797
798     unsigned Init(const char* F, int L, const char *s,
799         unsigned k=-1,
800         char lflag=-1);

```

```

801
802 IndexLabels& initOp(iTag t, const IndexLabels &b, const char *ot="op");
803
804 IndexLabels& SetConj(const char* F, int L, unsigned k) {
805     if (len) {
806         if (k>=len) wblog(F_L,
807             "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
808         data[k].Conj();
809     }; return *this;
810 };
811
812 IndexLabels& SetFlag(const char* F, int L, unsigned k, unsigned l) {
813     if (len) {
814         if (k>=len) wblog(F_L,
815             "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
816         data[k].SetFlag(l);
817     }; return *this;
818 };
819
820 IndexLabels& UnsetFlag(const char* F, int L, unsigned k, unsigned l) {
821     if (len) {
822         if (k>=len) wblog(F_L,
823             "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
824         data[k].UnsetFlag(l);
825     }; return *this;
826 };
827
828 IndexLabels& UnsetFlags() {
829     for (unsigned k=0; k<len; ++k) data[k].UnsetFlags();
830     return *this;
831 };
832
833 IndexLabels& RmItags() {
834     for (unsigned k=0; k<len; ++k) data[k].RmItag();
835     return *this;
836 };
837
838 bool operator==(const IndexLabels &b) const {
839     if (len!=b.len) return 0;
840     for (unsigned i=0; i<len; ++i) {
841         if (data[i].sameAs(b.data[i], 'l')<=0) return 0;
842     }
843     return 1;
844 };
845 bool operator!=(const IndexLabels &b) const {
846     return !(*this==b);
847 };
848
849 int findRegEx(const char *r) const;
850
851 unsigned match(
852     const IndexLabels &b, ctrIdx &ia, ctrIdx &ib,
853     const char tflag=0) const;
854
855 bool sameConj(const IndexLabels &b) const {
856     if (len!=b.len) return 0;
857     for (unsigned i=0; i<len; ++i) {
858         if (data[i].isConj(b.data[i],0)) return 0;
859     }
860     return 1;
861 };
862
863 char same(unsigned i, unsigned j) const {
864     if (i>=len || j>=len) wblog(F_L,
865         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i,j,len);
866     return data[i].sameAs(data[j]);
867 };
868
869 bool isOp(unsigned r=-1) const {
870     if (int(r)>=0) {
871         if (r<2 || r>3) wblog(F_L, "ERR %s() invalid r=%d !?",FCT,r);
872         if (len!=r) return 0;
873     }
874     else if (len<2 || len>3) return 0;
875     if (!data[0].isConj(data[1]) || !data[1].isConj()) return 0;
876     return 1;
877 };
878
879 bool isOpX(unsigned r=-1) const {
880     if (len<2 || len%2 || (int(r)>=0 && len!=2*r)) return 0;
881     for (unsigned n=len/2, i=0; i<n; ++i) {
882         if (!data[i].isConj(data[i+n]) || !data[i+n].isConj()) return 0;
883     }
884     return 1;
885 };
886
887 bool isAtensor(unsigned r=-1) const {

```

```

888     if (int(r)>=0) {
889         if (r<3) wblog(FL,"ERR %s() r=%d !?",FCT,r);
890         if (len!=r) return 0;
891     }
892     else if (len<3) { return 0; }
893
894     if (data[2].isConj()) return 0;
895     if (data[0].isConj() && data[1].isConj()) return 0;
896     return 1;
897 };
898
899 bool got_qdir(const char *qdir, char fflag=0) const {
900     unsigned i=0, n=(qdir? strlen(qdir) : 0);
901     if (n<2) wblog(FL,
902         "ERR %s() invalid qdir=%s !?",FCT,qdir?qdir:"(null)");
903     if (len<n || (fflag && len!=n)) return 0;
904
905     for (; i<n; ++i) {
906         if (qdir[i]=='+' ) { if ( data[i].isConj()) return 0; } else
907         if (qdir[i]=='-' ) { if (!data[i].isConj()) return 0; }
908         else wblog(FL,"ERR %s() invalid qdir='%s' !?",FCT,qdir);
909     }
910     return 1;
911 };
912
913 unsigned SkipLast(unsigned n=1) {
914     if (n<len) { len-=n; }
915     else {
916         if (n>len) wblog(FL,
917             "ERR %s() range out of bounds (%d/%d) !?",FCT,n,len);
918         init();
919     }
920     return len;
921 };
922
923 wstring toStr(char vflag=0) const;
924
925 const char* toStrk(
926     char *sout, unsigned l, unsigned k,
927     const char *st="(itag?)", char cflag=0
928 ) const {
929     if (k>=len || data[k].isEmpty())
930         { snprintf(sout,l,"%s",st?st:""); }
931     else { snprintf(sout,l,"%s",data[k].toStr(cflag).data); }
932     return sout;
933 };
934
935 mxArray* toMx() const {
936     if (len) {
937         mxArray* a=mxCreateCellMatrix(1,len);
938         for (unsigned i=0; i<len; ++i) {
939             mxSetCell(a,i,data[i].toMx());
940         }
941         return a;
942     }
943     return mxCreateCellMatrix(0,0);
944 };
945
946 int got_op_labels(const char *F=NULL, int L=0) const {
947     int q=0, x=0;
948     unsigned i=0, l=len/2;
949     for (; i<l; ++i) {
950         if (!x=data[i].isValid()) wblog(F_L,
951             "ERR %s() got invalid itags (%s)",FCT,toStr().data);
952         if (data[i].isConj() || data[i].conj()!=data[i+1]) return 0;
953         if (q>0) { if (q!=x) q=-1; } else
954         if (!q) { q=x; }
955     }
956     return q;
957 };
958
959 unsigned gotLabels() const {
960     unsigned m=0;
961     for (int q, k=0; k<int(len); ++k) {
962         q=data[k].isValid();
963         if (q>1) { ++m; } else if (!q) return 0;
964     }
965     return m;
966 };
967
968 protected:
969 private:
970
971     unsigned checkUnique(const char *F=NULL, int L=0) const {
972         if (len) {
973             unsigned i,j;
974             for (i=0; i<len; ++i)

```

```

975         for (j=i+1; j<len; ++j) if (data[i].t==data[j].t) {
976             if (F) wblog(F,L,
977                 "WRN got non-unique itags! (%s)",toStr().data);
978             return 0;
979         }
980     }
981     return len;
982 };
983
984 };
985
986 #define CTR_IS_NUM(c)      (((c)>='1' && (c)<='9') || ((c)>='a' && (c)<='z'))
987
988 #define CTR_IS_SEP_ANY(c)  (isspace(c) || (c)==',' || (c)=='\n')
989 #define CTR_IS_CONJ(c)    ((c)=='*')
990 #define CTR_IS_OTHER(c)   (CTR_IS_SEP_ANY(c) || CTR_IS_CONJ(c))
991
992 #define CTR_IS_VALID(c)    (CTR_IS_NUM(c) || CTR_IS_OTHER(c))
993
994 class ctrIdx : public wbvector<unsigned> {
995 public:
996     ctrIdx(unsigned l=0, unsigned *d=NULL, char c=0)
997         : wbvector<unsigned>(l,d), conj(c) {};
998
999     ctrIdx(const char *F, int L, const char *s, char c=0)
1000         : conj(c) { init(F,L,s); };
1001
1002     ctrIdx(const char *F, int L, const mxArray *a)
1003         : conj(0) { init(F,L,a); };
1004
1005     ctrIdx(const ctrIdx &I)
1006         : wbvector<unsigned>(I), conj(I.conj), newtags(I.newtags) {};
1007
1008     ctrIdx(const wbvector<unsigned> &I, char c=0)
1009         : wbvector<unsigned>(I), conj(c) {};
1010
1011     ctrIdx& init(unsigned l=0, unsigned *d=NULL, char c=0) {
1012         wbvector<unsigned>::init(l,d); conj=c;
1013         return *this;
1014     };
1015
1016     template <class T>
1017     ctrIdx& initT(unsigned l=0, T* d=NULL, char c=0) {
1018         wbvector<unsigned>::initT(l,d); conj=c;
1019         return *this;
1020     };
1021
1022     ctrIdx& init(const wbvector<unsigned> &I, char c=0) {
1023         wbvector<unsigned>::init(I); conj=c;
1024         return *this;
1025     };
1026
1027     ctrIdx& init1(unsigned k, char c=0) {
1028         wbvector<unsigned>::init(1);
1029         data[0]=k; conj=c;
1030         return *this;
1031     };
1032
1033     ctrIdx& init2(unsigned k1, unsigned k2, char c=0) {
1034         wbvector<unsigned>::init(2);
1035         if (k1==k2) wblog(FL,"ERR %s() got k=(%d,%d) !?",FCT,k1,k2);
1036         data[0]=k1; data[1]=k2; conj=c;
1037         return *this;
1038     };
1039
1040     template<class TQ, class TD>
1041     ctrIdx& initOp(const char *F, int L,
1042         iTag t1, const QSpace<TQ,TD> &B, const char *sopt);
1043
1044     int init(const char *F, int L, const char *s, unsigned offset=1);
1045
1046     ctrIdx& init(const char *F, int L, const mxArray *a);
1047     ctrIdx& init(const mxArray *a) { return init(FL,a); }
1048
1049     ctrIdx& Index(unsigned l, char c=0) {
1050         wbvector<unsigned>::init(l); conj=c;
1051         for (unsigned i=0; i<l; ++i) data[i]=i;
1052         return *this;
1053     };
1054
1055     int checkUniqueS(const char *F=0, int L=0, const unsigned r=-1) const;
1056
1057     ctrIdx& Sort(wbperm &P, unsigned r=-1) {
1058         if (len) {
1059             if (!isSorted())

```

```

1062         { wbvector<unsigned>::Sort(P); }
1063     else { P.init(); }
1064     if (int(r)>=0) checkUniqueS(FL,r);
1065 }
1066 else { P.init(); }
1067 return *this;
1068 };
1069
1070 ctrIdx& sort(ctrIdx &I, wbperm &P, unsigned r=-1) const {
1071     if (len) {
1072         if (!isSorted())
1073             { wbvector<unsigned>::sort(I,P); }
1074         else { I.init(*this); P.init(); }
1075         if (int(r)>=0) I.checkUniqueS(FL,r);
1076     }
1077     else { I.init(); P.init(); }
1078     return I;
1079 };
1080
1081 ctrIdx& Sort_(ctrIdx &J, unsigned r=-1) {
1082     if (len!=J.len) wblog(FL,
1083         "ERR %s() length mismatch (%s/%s) !?",FCT,STR_(this),STR(J));
1084     if (len) {
1085         if (!isSorted()) {
1086             wbperm P; wbvector<unsigned>::Sort(P);
1087             J.wbvector<unsigned>::Permute(P);
1088         }
1089         if (int(r)>=0) { checkUniqueS(FL,r); }
1090     }
1091     return *this;
1092 };
1093
1094 ctrIdx& invert(unsigned r, ctrIdx &I) const;
1095 ctrIdx& Invert(unsigned r) {
1096     ctrIdx I; save2(I);
1097     return I.invert(r,*this);
1098 };
1099
1100 ctrIdx& Conj() {
1101     conj=(conj ? 0 : 1);
1102     return *this;
1103 };
1104
1105 int extend2Perm(widx_t N, wbperm &P, char toend=0) const;
1106
1107 wbperm& extend2Perm( unsigned ra, unsigned ma,
1108     const ctrIdx &icb, unsigned rb, unsigned mb,
1109     const wbperm &pab, char conj, unsigned mc,
1110     wbperm &Pbc,
1111     ctrIdx &kb,
1112     ctrIdx &kcb,
1113     char Bflag=1
1114 ) const;
1115
1116 ctrIdx& save2(ctrIdx &J) {
1117     wbvector<unsigned>::save2(J);
1118     J.conj=conj; return J;
1119 };
1120
1121 mxArray* toMx() const { return toStr().toMx(); };
1122
1123 wbstring toStr(char xflag=0) const;
1124
1125 char conj;
1126
1127 IndexLabels newtags;
1128
1129 protected:
1130 private:
1131
1132 };
1133
1134 bool isCtrIdx(const mxArray *a) {
1135     unsigned i=0, r=60;
1136
1137     if (Mx::IsDblVector(0,0,a)) { wbvector<double> x(FL,a);
1138         if (mxGetM(a)!=1 || x.len>r) return 0;
1139
1140         for (; i<x.len; ++i) {
1141             if (x[i]<0 || double(int(x[i]))!=x[i] || x[i]>r) return 0;
1142         }
1143         if (!x.wbvector<double>::isUnique()) return 0;
1144         return 1;
1145     }
1146     else if (mxIsChar(a)) { ctrIdx idx; wbstring s(a);
1147         if (idx.init(0,0,s.data)<=0) return 0;

```

```

1149         for (; i<idx.len; ++i) { if (idx[i]>r) return 0; }
1150         if (!idx.wbvector<unsigned>::isUnique()) return 0;
1151         return 1;
1152     }
1153     return 0;
1154 };
1155
1156 template <class T=SPIDX_T>
1157 class sparseIndex2D {
1158 public:
1159     sparseIndex2D() : rmaj(0) {};
1160
1161     sparseIndex2D& Setup(char use_rmaj=0);
1162
1163     T cidx(T j) const;
1164     T ridx(T ic) const;
1165
1166     wbsting sizeStr() const {
1167         if (S.len) return S.toStrf("", "x");
1168         else return wbsting("[]");
1169     };
1170
1171     char rmaj;
1172     wbMatrix<T> IJ;
1173     wbvector<T> S;
1174
1175     wbperm P;
1176
1177     wbvector<T> cIdx;
1178
1179 protected:
1180 private:
1181 };
1182 #endif
1183
1184
1185
1186
1187

```

5.24 wbio.h

```

1 #ifndef __WB_WBIO_H__
2 #define __WB_WBIO_H__
3
4 /* ----- */
5 // tags: tmpfile, tempfile, tempmat, TMPDIR // Wb,Jun06,19
6
7 namespace Wb {
8
9 wbsting tmpfile(const char *F, int L,
10     const char *t=NULL, const char *x=NULL, const char *f=NULL);
11
12 class tmpFile {
13 public:
14     tmpFile() : fid(NULL), ncall(-1) { };
15
16     tmpFile(const char *F, int L,
17         const char *f=NULL, const char *x=NULL, const char *mode=NULL)
18         : fid(NULL), ncall(-1) { init(F,L,f,x,mode); };
19
20     ~tmpFile() { if (fid) {
21         wblog(PFL,
22             "I/O closing tmpfile '%s'",fout.data? fout.data:"(null!)");
23         fprintf(fid, "\n%-18s %5s %s close %s\n\n",
24             shortFL(PFL), "", Wb::TimeStamp('T').data, Wb::TimeStamp('D').data);
25         fclose(fid); fid=NULL;
26     } };
27
28     void init(const char *F, int L,
29         const char *f=NULL, const char *x=NULL, const char *mode=NULL);
30
31     int blogf(const char *F, int L, const char *fmt, ...);
32
33     int sepline(const char *w="-", unsigned len=-1);
34
35     void flush() { if (fid) fflush(fid); };
36
37     FILE *fid;
38     int ncall;
39     wbsting fout;
40
41 protected:

```

```

42 private:
43 };
44
45 };
46
47 template<class T>
48 int Str2Idx(
49     const char *F, int L,
50     const char* s, wbvector<T> &idx, T offset=0, char cmpct=0
51 );
52
53 template<class T>
54 int Str2Idx(
55     const char* s, wbvector<T> &idx, T offset=0, char cmpct=0
56 );
57
58 template<class T> int Str2Idx(
59     const char* s, wbvector<T> &idx, unsigned offset=0, char cmpct=0
60 ){ return Str2Idx(s,idx,T(offset),cmpct); };
61
62 wbvector<unsigned> Str2Idx(
63     const char *F, int L, const char* s, unsigned offset=0);
64 wbvector<unsigned> Str2Idx(const char* s, unsigned offset=0);
65
66 template<class T>
67 int Str2Idx(
68     const char* F, int L,
69     wbvector<T> &I, const mxArray *a, T offset=0, char cmpct=0
70 );
71
72 template<class T>
73 int Str2Idx(
74     wbvector<T> &I, const mxArray *a, T offset=0, char cmpct=0
75 ){ return Str2Idx(F,I,a,offset,cmpct); };
76
77 void WbPrintMatrixC (
78     const wbMatrix<wbcomplex> &M, const char *istr="", int space=10);
79
80 #endif

```

5.25 wblib.h

```

1 #ifndef __WB_WB_LIB_MEX_HCC__
2 #define __WB_WB_LIB_MEX_HCC__
3
4 /* ----- //
5     library for mex programs
6     A. Weichselbaum (C) Jan 2006
7 // ----- */
8
9 // comment out for testing purposes when compiling for mac // Wb,May21,20
10 // see also undef for mac below
11 #define QS_USING_OMP
12
13 #if __linux__
14     #ifndef QS_USING_OMP
15         #warning compiling without QS_USING_OMP
16     #endif
17 #elif __unix__
18     #warning got unix
19 #elif __APPLE__
20     #undef QS_USING_OMP
21 #else
22     #error GOT UNSUPPORTED OPERATING SYSTEM
23 #endif
24
25 #include <assert.h>
26 #include <execinfo.h>
27
28 #include <time.h>
29 #include <signal.h>
30 #include <unistd.h>
31 #include <fcntl.h>
32 #include <regex.h>
33
34 #include <stdio.h>
35 #include <stdlib.h>
36 #include <chrono>
37
38 #include <exception>
39
40 #include <cmath>
41

```



```

42 #include <ctype.h>
43 #include <climits>
44 #include <stdarg.h>
45 #include <string.h>
46
47 #include <string>
48 #include <iostream>
49 #include <typeinfo>
50
51 #include <sys/ipc.h>
52 #include <sys/msg.h>
53 #include <sys/types.h>
54 #include <sys/stat.h>
55
56 #include <map>
57 #include <queue>
58 #include <unordered_map>
59 #include <algorithm>
60 #include <vector>
61
62 #ifdef QS_USING_OMP
63 #include <omp.h>
64 #else
65     int omp_get_max_threads() { return 1; }
66     int omp_get_num_threads() { return 1; }
67     int omp_get_thread_num() { return 0; }
68     int omp_in_parallel() { return 0; }
69 #endif
70
71 #if __APPLE__
72     #ifndef expl0
73         inline double expl0(double x) { return pow(double(10),x); }
74     #endif
75     #ifndef sighandler_t
76         #define sighandler_t sig_t
77     #endif
78
79     #include <mach/mach_host.h>
80     #include <mach/task.h>
81
82     #include <sys/sysctl.h>
83     #undef setbit
84     #undef isset
85
86 #endif
87
88     #define PP_STR__(a) #a
89     #define PP_STRFY(a) PP_STR__(a)
90
91 unsigned ARG_CHECK=1;
92
93 #define STRLEN 1023
94 char str[STRLEN+1];
95
96 template <class T> class wbvector;
97 template <class T> class wbMatrix;
98 template <class T> class wbarray;
99 template <class TD> class wbsparray;
100 template <class TQ, class TD> class QSpace;
101 template <class T> class sparseIndex2D;
102
103 class wbcomplex;
104 class wbstring;
105 class bitset;
106 class wbperm;
107 class wbindex;
108
109 class IndexLabels;
110 class ctrIdx;
111
112 using namespace std;
113
114 #define ENABLE_IF_COMPLEX(T__) typename \
115     std::enable_if< WbUtil<T__>::isComplex(), T__ >::type* = nullptr
116
117 #define ENABLE_IF_isPOD(T__) typename \
118     std::enable_if< WbUtil<T__>::isPOD(), T__ >::type* = nullptr
119
120 #define ENABLE_IF_noPOD(T__) typename \
121     std::enable_if< !WbUtil<T__>::isPOD(), T__ >::type* = nullptr
122
123 #define ENABLE_IF_isINT(T__) typename \
124     std::enable_if<std::is_integral<T__>::value>* = nullptr
125
126 #if 1
127     #define widx_t size_t
128     #define swidx_t long

```

```

129 #define wperm_t size_t
130 #define swperm_t long
131 #else
132 #define widx_t unsigned
133 #define swidx_t int
134 #define wperm_t unsigned
135 #define swperm_t int
136 #endif
137
138 #if 1
139 #define SPIDX_T size_t
140 #define sSPIDX_T long
141 #else
142 #define SPIDX_T unsigned
143 #define sSPIDX_T int
144 #endif
145
146 #define WBINDEX wbvector<widx_t>
147 #define WBPERM wbvector<wperm_t>
148 #define WBIDXMAT wbMatrix<widx_t>
149
150 #define UVEC wbvector<unsigned>
151 #define C_UVEC const wbvector<unsigned>
152 #define DMAT wbMatrix<double>
153 #define C_DMAT const wbMatrix<double>
154 #define UMAT wbMatrix<unsigned>
155 #define C_UMAT const wbMatrix<unsigned>
156 #define IMAT wbMatrix<int>
157 #define C_IMAT const wbMatrix<int>
158 #define C_TMAT const wbMatrix<T>
159 #define C_MEX const mxArray
160 #define C_UINT const unsigned
161 #define C_CHAR const char
162
163 #define DMAT0 wbMatrix<double>()
164
165 #define __FL__ (file ? file : __FILE__), (line ? line : __LINE__)
166 #define F_L (F ? F : __FILE__), (F ? L : __LINE__)
167 #define F_L_F (F ? F : __FILE__), (F ? L : __LINE__), __FUNCTION__
168 #define F_L_F (F ? F : __FILE__), (F ? L : __LINE__), (fct ? fct : __FUNCTION__)
169
170 #define FSTR __FILE__ ": "
171 #define FLINE __FILE__, __LINE__
172 #define FL __FILE__, __LINE__
173 #define FLF __FILE__, __LINE__, __FUNCTION__
174 #define FCT __FUNCTION__
175 #define FCL __FUNCTION__, __LINE__
176
177 std::string shortPF(const char *F, int L, unsigned lmax=20);
178
179 std::string getName(const std::type_info &type_id, char vflag=1);
180
181 #define PFL shortPF(__FILE__, __LINE__).data(), __LINE__
182 #define PF_L shortPF(F?F:__FILE__,F?L:__LINE__).data(), (F?L:__LINE__)
183
184 #ifndef TIME
185 #define TIME Wb::TimeStamp('T').data
186 #endif
187
188 #ifndef shortFLT
189 #define shortFLT shortFL(__FILE__, __LINE__), Wb::TimeStamp('T').data
190 #endif
191
192 #ifndef MYTID
193 #ifdef QS_USING_OMP
194 #define MYTID Wb::getMyOMP_ID().c_str()
195 #else
196 #define MYTID 0
197 #endif
198 #endif
199
200 #ifndef Inf
201 #define Inf FP_INFINITE
202 #endif
203
204 #if !defined(MAX)
205 template <class T> inline
206 const T& MAX(const T&a, const T&b) { return ((b)>(a) ? (b) : (a)); }
207 #endif
208
209 template <class T> inline
210 const T& MAX3(const T&a, const T&b, const T&c) {
211     if ((b)>(a))
212         { return ((c)>(b) ? (c) : (b)); }
213     else { return ((c)>(a) ? (c) : (a)); }
214 }
215

```

```

216 #if !defined(MIN)
217     template <class T> inline
218     const T& MIN(const T&a, const T&b) { return ((b)<(a) ? (b) : (a)); };
219 #endif
220
221     template <class T> inline
222     const T& MIN3(const T&a, const T&b, const T&c) {
223         if ((b)<(a))
224             { return ((c)<(b) ? (c) : (b)); }
225         else { return ((c)<(a) ? (c) : (a)); }
226     };
227
228 #if !defined(SWAP)
229     template <class T>
230     inline void SWAP(T &a, T &b) { T x=a; a=b; b=x; }
231 #endif
232
233 #if !defined(NSTR)
234 #define NSTR(a) Wb::num2Str(a).data
235 #endif
236
237 #if !defined(RSTR)
238 #define RSTR(a,n) wbsttring().initrep(a,n).data
239 #endif
240
241 #if !defined(SEC2STR)
242 #define SEC2STR(a) Wb::sec2Str(a).data
243 #endif
244
245 #if !defined(TSTR)
246 #define TSTR(a) getName(typeid(a),1).c_str()
247 #endif
248
249 #if !defined(STSTR)
250 #define STSTR(a) getName(typeid(a),0).c_str()
251 #endif
252
253 #if !defined(SSTR)
254 #define SSTR(a) (a).sizeStr().data
255 #endif
256
257 #if !defined(SSTR_)
258 #define SSTR_(a) (a)->sizeStr().data
259 #endif
260
261 #if !defined(QSTR)
262 #define QSTR(a) (a).qStr().data
263 #endif
264
265 #if !defined(QSTR_)
266 #define QSTR_(a) (a)->qStr().data
267 #endif
268
269 #if !defined(STR)
270 #define STR(a) (a).toStr().data
271 #endif
272
273 #if !defined(STR_)
274 #define STR_(a) (a)->toStr().data
275 #endif
276
277 #if !defined(STR2)
278 #define STR2(a,b) (a).toStr(b).data
279 #endif
280
281 #if !defined(STR2_)
282 #define STR2_(a,b) (a)->toStr(b).data
283 #endif
284
285 #if !defined(cSTR)
286 #define cSTR(a) Wb::char2Str(a).data
287 #endif
288
289 #if !defined(I2STR)
290 #define I2STR(a) Wb::int2Str(a).data
291 #endif
292
293 #if !defined(BITS)
294 #define BITS(a) Wb::bits(a).data
295 #endif
296
297 #if !defined(POW)
298
299     template <class T>
300     inline T POW(const T &a, const unsigned &n) {
301         T x = n ? a : 1;
302         for (unsigned i=1; i<n; i++) x*=n;

```

```

303     return x;
304 }
305
306 #endif
307 #if !defined(SGN)
308
309     template <class T>
310     inline int SGN(const T &a) {
311         return ((T(0)<a)-(a<T(0)));
312     }
313
314 #endif
315 #if !defined(NUMCMP)
316
317     template <class T>
318     inline char NUMCMP(const T &a, const T &b) {
319         if (a<b) return -1; else
320         if (a>b) return +1;
321         return 0;
322     }
323
324     template <class Ta, class Tb>
325     inline char NUMCMP(const Ta &a, const Tb &b) {
326         if (a<b) return -1; else
327         if (a>b) return +1;
328         return 0;
329     }
330
331 #endif
332
333 #define ISCOMPLX_(a) WbUtil<a>::isComplex()
334 #define ISREAL(a) WbUtil<a>::isReal()
335
336 #ifdef MAC
337     #include "Include/nomex.hh"
338     #include <Accelerate/Accelerate.h>
339 #elif defined(NOMEX)
340     #include "nomex.hh"
341 #else
342
343     #include <mex.h>
344     #include <mat.h>
345
346     #define QS_VERSION 3.2
347     #define QS_VERSION_SUB 0
348
349     #ifdef MEX_MLVER
350         #define MEX_MLVER_STR PP_STRFY(MEX_MLVER)
351     #else
352         #define MEX_MLVER_STR "(unknown)"
353     #endif
354
355     #ifdef HOST_NAME
356         #define HOST_NAME_STR PP_STRFY(HOST_NAME)
357     #else
358         #define HOST_NAME_STR "(host)"
359     #endif
360
361 #define mxIsNumChar(a) (mxIsNumeric(a) || mxIsChar(a) || mxIsLogical(a))
362
363 #endif
364
365 #if defined(MAIN) || defined(DBSTOP)
366     #undef printf
367 #endif
368
369 #ifndef PROG
370 #ifdef MATLAB_MEX_FILE
371     #define PROG mexFunctionName()
372 #else
373     #define PROG "(?prog?)"
374 #endif
375 #endif
376
377 namespace Wb {
378     class myname__ {
379     public:
380         myname__() { unsigned i=0, k=0;
381             #ifdef PROG
382                 snprintf(data,60,"%s",PROG);
383             #elif defined MATLAB_MEX_FILE
384                 snprintf(data,60,"%s",mexFunctionName());
385             #else
386                 snprintf(data,60,"(?myname?)");
387             #endif
388
389             for (i=0; data[i]; ++i) { if (data[i]=='/') k=i+1; }

```

```

390         if (k && data[k]) {
391             for (i=0; data[k]; ++k, ++i) { data[i]=data[k]; }
392             data[i]=0;
393         }
394
395         #ifdef PROG_TAG
396             snprintf(data+60,4, "%.3s", PROG_TAG);
397         #else
398             snprintf(data+60,4, "%.3s", data);
399         #endif
400     };
401
402     char data[64];
403
404     private:
405 };
406
407 myname__ myName;
408
409 #define myname Wb::myName.data
410 #define mytag Wb::myName.data+60
411
412 int get_WB_VERBOSE(const char *F=0, int L=0);
413 int got_DBSTOP(const char *F=0, int L=0);
414 int got_DESKTOP();
415 int is_DEPLOYED(const char *F=0, int L=0);
416
417 int envVRB= 0;
418 int envDKT=-1;
419
420 int envDBG= 0;
421
422 int my_caller_tid=0;
423
424 };
425
426 void dbstop(const char* file, int line);
427 void ExitMsg(const char* s="", char xflag=0);
428 void doflush();
429
430 #ifdef LOAD_CGC_QSPACE
431 #ifndef QS_SKIP_MPFR
432     #define QS_USING_MPFR
433 #endif
434 #endif
435
436 #ifdef __WBDEBUG__
437 #define __WB_MEM_CHECK__
438 #undef WB_SKIP_ASSERT
439 #elif defined __WB_MEM_CHECK__
440 #define __WBDEBUG__
441 #endif
442
443 namespace Wb {
444 class ARGV {
445     public:
446         ARGV(va_list *v1=NULL) : args(v1) {};
447         ~ARGV() { if (args) { va_end(*args); args=NULL; }};
448     private:
449         va_list *args;
450 };
451 };
452 };
453
454 namespace wblog {
455     const int SLEN=256;
456 };
457
458 #ifdef MATLAB_MEX_FILE
459 #define PRINTF(...) { \
460     if (Wb::my_caller_tid==omp_get_thread_num()) \
461         { printf(__VA_ARGS__); } \
462     else { fprintf(stdout, __VA_ARGS__); }}
463 #else
464 #define PRINTF printf
465 #endif
466 #include "wblibx.h"
467 #include "wblog.h"
468
469 #ifdef QS_USING_OMP
470 #include "wbomp.hh"
471 #endif
472
473
474
475
476

```

```
477 #include "memtrack.hh"
478
479 #include "util.hh"
480 #include "memlib.hh"
481 #include "mxlib.hh"
482
483 #ifdef QS_USING_MPFR
484 #include <gmp.h>
485 #include <mpfr.h>
486 #include "wbmpfr.hh"
487 #endif
488
489 #include "wbcomplex.hh"
490 #include "gathered.hh"
491 #include "wbsort.hh"
492
493 #include "wbvector.hh"
494 #include "wbstring.hh"
495 #include "wbbitset.hh"
496 #include "wbio.h"
497
498 #include "wbperm.hh"
499 #include "wbindex.hh"
500
501 #include "wbclock.hh"
502 #include "wbMatrix.hh"
503
504 #include "mexlib.hh"
505
506 #include "wbMatIO.hh"
507
508 #include "wbarray.hh"
509 #include "wbsparray.hh"
510
511 #include "wbutil.hh"
512
513 #include "wbbblas.hh"
514 #include "wbMatrix_blas.hh"
515 #include "wbarray_blas.hh"
516
517 #include "wbopts.hh"
518
519 #ifdef DBG_GCX_LOCKS
520     Wb::tmpFile LKF (FL, "-lks");
521 #endif
522
523 #ifdef DBG_QSX_BUF
524     Wb::tmpFile BFF (FL, "-buf");
525 #endif
526
527 #ifdef LOAD_CGC_QSPACE
528 #include "clebsch.hh"
529 #include "QSpace_aux.hh"
530 #include "QSpace.hh"
531 #include "mpsortho.hh"
532 #endif
533
534 #include "wblog.c"
535
536 #ifdef QS_USING_MPFR
537 #include "wbmpfr.cc"
538 #endif
539
540 #ifdef QS_USING_OMP
541 #include "wbomp.cc"
542 #endif
543
544 #include "wbsort.cc"
545 #include "wbstring.cc"
546 #include "wbbitset.cc"
547 #include "memlib.cc"
548 #include "wbcomplex.cc"
549 #include "gathered.cc"
550 #include "util.cc"
551
552 #ifdef LOAD_CGC_QSPACE
553 #include "QSpace_aux.cc"
554 #include "QSpace.cc"
555 #endif
556
557 #include "wbvector.cc"
558 #include "wbperm.cc"
559 #include "wbindex.cc"
560 #include "wbMatrix.cc"
561 #include "wbarray.cc"
562 #include "wbsparray.cc"
563
```

```

564 #include "wblock.cc"
565 #include "mexlib.cc"
566 #include "matlib.cc"
567 #include "wbMatrix_blas.cc"
568 #include "wbarray_blas.cc"
569
570 #include "wbio.c"
571 #include "wbmsg.hh"
572
573 #ifdef LOAD_CGC_QSPACE
574 #include "clebsch.cc"
575 #include "clebsch_aux.cc"
576 #include "clebsch_io.cc"
577 #include "clebsch_old.cc"
578 #include "mpsorth.cc"
579 #endif
580
581 namespace Wb {
582
583     class gpara__ {
584     public:
585         gpara__() : x(0) {
586             #ifdef __WBDEBUG__
587                 wblog(FL,"TST %s (%s)",FCT,myname);
588             #endif
589             wb_srand();
590
591             #ifdef __WB_MPFR_HH__
592                 gmp_randinit_default(Wb::wb_rstate);
593             #endif
594
595             envDKT=got_DESKTOP();
596             my_caller_tid=omp_get_thread_num();
597
598             init();
599         };
600
601         void init() {
602             memset(str,0,STRLEN+1);
603             envVRB=get_WB_VERBOSE();
604             envDBG=got_DBSTOP();
605
606             #ifdef LOAD_CGC_QSPACE
607             #ifdef QS_USING_OMP
608                 Wb::GetNumThreads(FL,OMP_NUM_THREADS,"OMP_NUM_THREADS");
609                 Wb::GetNumThreads(FL,QSP_NUM_THREADS,"QSP_NUM_THREADS");
610
611                 sp_num_threads=MAX(OMP_NUM_THREADS,QSP_NUM_THREADS);
612             #endif
613             #endif
614
615             my_caller_tid=omp_get_thread_num();
616         };
617
618         char x;
619
620     private:
621     };
622
623     gpara__ gpara;
624
625     class Cleanup {
626     public:
627         Cleanup() : i(0) {
628             gpara.init();
629         };
630
631         ~Cleanup() {
632             wblog::myIO.clear();
633             if (gwb_Profs.size()) { Wb::save_and_clear_Profiling(); }
634         };
635
636         int i;
637     };
638 };
639
640 };
641
642 #ifdef MAIN
643 #ifdef MATLAB_MEX_FILE
644     #warning got MAIN with MATLAB_MEX_FILE
645     #pragma message "\n\n" \
646         "    Don't define MAIN with MEX files, this kills the whole matlab seesion!\n" \
647         "    See ExitMsg() in wblib.h (!)\n" \
648     ""
649 #endif
650

```

```

651 void ExitMsg(const char* s, char xflag) {
652     if (s && s[0])
653         { printf("\n%s !? %s\n",shortFL(FL),s); }
654     else { printf("\n"); }
655
656     if (Wb::envDBG) {
657         dbstop(FL); if (xflag) { exit(-1); }
658     }
659     else {
660
661         if (xflag) {
662             exit(-1);
663         }
664         else {
665             mexErrMsgIdAndTxt("Wb:MEX:wblog",s);
666         }
667     }
668 }
669
670 void doflush() { fflush(0); };
671
672 #else
673
674 #ifdef DBSTOP
675 void ExitMsg(const char* s, char xflag) {
676     if (s && s[0]) printf("%s >%s<\n",shortFL(FL),s ? s:"");
677     dbstop(FL);
678     printf("\n%s dbstop() and on we go ...\n", shortFL(FL));
679     mexErrMsgIdAndTxt("Wb:ERR:mex",s);
680 }
681 #else
682 void ExitMsg(const char* s, char xflag __attribute__((unused))) {
683     printf("\n");
684     if (Wb::is_DEPLOYED()) {
685         Wb::print_backtrace(FL,"from deployed code"); printf("\n"); }
686
687     mexErrMsgIdAndTxt("Wb:ERR:mex",s);
688     fprintf(stdout,"%s TST %s()\n",shortFL(FL),FCT);
689 }
690 #endif
691
692 void doflush() {
693     fflush(0);
694
695     #if 0
696
697     if (!omp_in_parallel()) {
698         mexEvalString("pause(0);");
699     }
700     #endif
701
702 };
703
704 #endif
705
706 #endif
707

```

5.26 wblibx.h

```

1 #ifndef __WB_WBLIBX_HH__
2 #define __WB_WBLIBX_HH__
3
4 /* ----- */
5 /* ----- */
6
7 void usage(const char *file=NULL, int line=0, const char* estr=NULL);
8 void print_version_info(const char *F=NULL, int L=0);
9
10 int isHelpIndicator(const mxArray *a);
11 int checkHelpVersion(const mxArray *a);
12
13 void wberror(const char *file, int line, const char* istr);
14
15 void dbstop(const char* file, int line);
16
17 char* memsize2Str(double x, char *s, unsigned l=-1);
18
19 template <class T>
20 class is_pointer_ { public: char q() { return 0; }; };
21
22 template <class T>
23 class is_pointer_<T*> { public: char q() { return 1; }; };

```



```

24
25 template <class T>
26 class is_pointer_<T*> { public: char q() { return 2; }; };
27
28 template <class T>
29 class is_pointer_<T***> { public: char q() { return 3; }; };
30
31 template <class T>
32 inline char ispointer(const T& x) { return is_pointer_<T>().q(); };
33
34 #endif
35

```

5.27 wblog.h

```

1 #ifndef __WB_LOG_H__
2 #define __WB_LOG_H__
3
4 const char* shortFL(const char *F, int L=-1, char *s=NULL, unsigned n=-1);
5
6 // e.g. set in matlab via
7 // » setenv('WB_VERBOSE',num2str(1«10 + 1«21));
8 // » setenvb WB_VERBOSE 10 21
9 // read in wblib.h -> envVRB=gathered.cc -> get_WB_VERBOSE()
10 // default 0xF1; // 0xF1 = 241: bit 0 and 4..7
11
12 #define WBL_MSEC__ (1U«10)
13 #define WBL_TCAST__ (1U«12)
14
15 #define WBL_CLK__ (1U«20)
16 #define WBL_CTR__ (1U«21)
17 #define WBL_MMEX__ (1U«22)
18
19 #define WBL_IO__ (1U«23)
20 #define WBL_RESIZE__ (1U«24)
21
22 #define WBLOG_MSEC (Wb::envVRB & WBL_MSEC__ )
23 #define WBLOG_TCAST (Wb::envVRB & WBL_TCAST__ )
24 #define WBLOG_CLK (Wb::envVRB & WBL_CLK__ )
25 #define WBLOG_CTR (Wb::envVRB & WBL_CTR__ )
26 #define WBLOG_MMEX (Wb::envVRB & WBL_MMEX__ )
27 #define WBLOG_IO (Wb::envVRB & WBL_IO__ )
28 #define WBLOG_RESIZE (Wb::envVRB & WBL_RESIZE__ )
29
30 enum WBLOG_TYPE { OK, WRN, ERR, NUM_TYPE };
31
32 namespace Wb {
33
34 struct LogException : public exception {
35 public:
36     LogException (WBLOG_TYPE l=OK, const char *s=NULL);
37
38     LogException (const LogException &e)
39     : type(e.type), nrefs(e.nrefs), count(e.count), ith(e.ith), nth(e.nth) {
40         if (nrefs) { ++(*nrefs); ++count;
41         }
42         strncpy(istr,e.istr,64); istr[61]=istr[62]='.'; istr[63]=0;
43     };
44
45     ~LogException() noexcept {
46         if (nrefs) {
47             if (--(*nrefs)<1) {
48                 delete nrefs; nrefs=NULL;
49             }
50             else {
51             }
52         }
53     };
54
55     LogException& operator=(const LogException &e) {
56         type=e.type; count=e.count; ith=e.ith; nth=e.nth;
57         nrefs=e.nrefs; if (nrefs) { ++(*nrefs);
58         }
59         strncpy(istr,e.istr,64); istr[61]=istr[62]='.'; istr[63]=0;
60         return *this;
61     };
62
63     LogException& operator++() {
64         ++count;
65
66         int n=omp_get_num_threads();
67         if (nth<n) { nth=n; ith=omp_get_thread_num(); }
68
69         return *this;
70     };
71

```

```

69     };
70
71     LogException& operator+=(const LogException &e) {
72
73         if (!type)
74             { unsigned n=count; (*this)=e; count+=n; }
75         else { ++count; }
76
77         int n=omp_get_num_threads();
78         if (nth<n) { nth=n; ith=omp_get_thread_num(); }
79
80         return *this;
81     };
82
83     bool operator!() const { return count==0; }
84     explicit operator bool() const { return count!=0; }
85
86     LogException& init() {
87         if (nrefs) { --(*nrefs);
88             if (*nrefs<1) { delete nrefs; nrefs=NULL; }
89         }
90         type=OK; count=ith=nth=0; istr[0]=0;
91         return *this;
92     };
93
94     const char* what() const throw () { return istr; };
95
96     void report(const char *F=0, int L=0, const char *func=0);
97
98     WBLOG_TYPE type;
99     int *nrefs;
100     char istr[64];
101     unsigned count;
102     int ith, nth;
103
104 private:
105 };
106
107 };
108
109 namespace wblog {
110
111 class SBUF {
112 public:
113     SBUF(unsigned l0=wblog::SLEN) : l(0), slen(0), sbuf(NULL) {
114         init(l0);
115     };
116
117     ~SBUF() { if (sbuf) {
118         #pragma omp critical (__ensure_sequential_wblog__)
119         { flush(stderr,1); }
120         delete [] sbuf;
121     } };
122
123     SBUF& init(unsigned l0=0) { l=0;
124         if (sbuf) { delete [] sbuf; sbuf=NULL; }
125         if ((slen=l0)) {
126             sbuf = new char[slen];
127             if (!sbuf) { sprintf(str,"ERR %s() "
128                 "failed to allocate sbuf (l=%d)",__FUNCTION__,slen);
129                 ExitMsg(str);
130             }
131             memset(sbuf,0,slen);
132         }
133         return *this;
134     };
135
136     int cat(const char *s);
137     int catf(const char *fmt, ...);
138
139     void error_bounds(const char *F, int L, const char *fct, const char *fmt);
140     void increase_size(const char *F, int L, unsigned n);
141
142     void flush(FILE *fid=stdout, char fflag=0);
143     void print(const char *F=0, int L=0, const char *istr=0);
144
145     unsigned l,slen;
146     char *sbuf;
147
148 private:
149 };
150
151 class stdio_buf {
152 public:
153     stdio_buf(SBUF &S, FILE *f_, int t_) : s(NULL), fid(0), tid(0), ref(0) {
154         if (f_!=stdin && f_!=stdout) {
155             fprintf(stderr,"\nERR %s() invalid fid=%p\n's'\n\n",

```

```

156         FCT,f_,S.sbuf?S.sbuf:"");
157     }
158     if (S.sbuf) {
159         s=S.sbuf; fid=f_; tid=t_; ref=1;
160         S.sbuf=0; S.slen=S.l=0;
161     }
162 };
163
164 stdio_buf(const stdio_buf &b) : s(NULL), fid(0), ref(0) {
165     if (!b.ref) fprintf(stderr,"\\nERR %s() got b.ref=%d\\n\\n",FCT,b.ref);
166     s=b.s; fid=b.fid; tid=b.tid;
167 };
168
169 void print_stdout(FILE *f=NULL) const {
170     if (Wb::envVRB &256) {
171         unsigned i=0, m=0; for (; s[i]; ++i) {
172             if (s[i]!='\\n') { if (++m>2) break; } else m=0;
173         }
174         if (m>2) { i-=2; char *x = (char*)s;
175             if (tid<10) x[i]=tid + '0'; else
176             if (tid<36) x[i]=tid-10+'a'; else
177             if (tid<62) x[i]=tid-36+'A'; else { x[i]='*'; }; x[i+1]=' ';
178         }
179     }
180     if (f) { fprintf(f,"%s",s); }
181     else {
182         if (Wb::envDKT)
183             { mexPrintf("%s",s); }
184         else { PRINTF("%s",s); }
185     }
186 };
187
188 ~stdio_buf() { if (s && !ref) {
189     if (Wb::my_caller_tid!=omp_get_thread_num()) {
190         fprintf(stderr,"WRN %s() got tid=%d/%d !?\\nWRN '%s'\\n",
191             FCT,omp_get_thread_num(),Wb::my_caller_tid,s);
192     }
193     else {
194         print_stdout();
195         delete [] s;
196         fflush(fid);
197     }
198     }
199 };
200 private:
201     const char *s;
202     FILE *fid;
203     int tid;
204     char ref;
205 };
206
207 std::deque<stdio_buf> myIO;
208
209 };
210
211 void wbSetLogLevel(unsigned l);
212
213 int wblogs(
214     wblog::SBUF &S, const char* file, int line,
215     const char *fmt, va_list args,
216     char Hflag=0
217 );
218
219 int wblogs(
220     wblog::SBUF &S, const char* file, int line, const char *fmt, ...
221 ){
222     va_list args; Wb::ARGV wd(&args);
223     va_start(args,fmt);
224     return wblogs(S,file,line,fmt,args);
225 };
226
227 int wblogf(FILE *fid,
228     const char* file, int line, const char *fmt, va_list args);
229
230 int wblogf(FILE *fid,
231     const char* file, int line, const char *fmt, ...) {
232
233     va_list args; Wb::ARGV wd(&args);
234     va_start(args,fmt);
235
236     int l=wblogf(fid,file,line,fmt,args);
237
238     return l;
239 };
240
241 int wblogl(const char* file, int line, const char *fmt, ...);
242

```

```

243 #define wblog(...) wblogf(stdout, __VA_ARGS__)
244
245 int wb_printf(const char *fmt, ...) {
246     unsigned l=0, n=256;
247     wblog::SBUF S(n); {
248         va_list args; Wb::ARGV wd(&args);
249         va_start(args,fmt);
250         l=vsprintf(S.sbuf,n,fmt,args);
251     }
252     if (l>=n) fprintf(stderr,
253         "\nERR string out of bounds (%d/%d)\nERR %s\n\n",l,n,S.sbuf);
254
255     #pragma omp critical (__ensure_sequential_wblog__)
256     { S.flush(); }
257
258     return l;
259 };
260
261 unsigned wblog_checktag(const char *fmt, const char *t, char *tag);
262
263 char wblog_findtoken(const char *istr, const char *tok, int maxoffset=-1);
264
265 void banner(
266     unsigned n, const char *s,
267     const char *istr=NULL, const char *fstr=NULL
268 );
269
270
271 const char* shortFL(const char *F, int L, char *s, unsigned n) {
272
273     static char fstr[512], ic=0;
274     unsigned i=0, j=0, l;
275
276     if (s==NULL) {
277         #pragma omp critical (using_shortFL)
278         { s=fstr+32*ic; if ((++ic)>15) ic=0; }
279     }
280
281     if (!F) { s[0]=0; return s; }
282     if (n>32) { n=32; }
283
284     if (F[0]=='/') {
285         for (; F[i]; ++i) if (F[i]=='/') j=i+1;
286     }
287
288     if (L>0)
289         l=snprintf(s,32,"%0.24s:%d",F+j,L);
290     else l=snprintf(s,32,"%0.24s:", F+j );
291
292     if (l>n && n>8) {
293         i=4; s[i]='\n'; j=l-n;
294         do { ++i; s[i]=s[i+j]; } while (s[i]);
295     }
296
297     return s;
298 };
299
300 const char* tmpmatFL(
301     const char *F, int L, const char *vn, char *s, unsigned n) {
302
303     if (!s || int(n)<8) wblog(FL,"ERR %s() invalid input (%p, %d)",FCT,s,n);
304     unsigned l=snprintf(s,n,"./tmp");
305
306     if (l<n && F && F[0]) { unsigned i=0, k=0;
307         for (; F[i]; ++i) { if (F[i]=='/') { k=i+1; } }
308         i=l; l+=snprintf(s+l,n-l,"%s",F+k);
309         for (; s[i] && i<n; ++i) { if (s[i]=='.') { s[i]=0; l=i; break; } }
310     }
311
312     if (l<n) { l+=snprintf(s+l,n-l,"%d",L); }
313 #ifdef QS_USING_OMP
314     if (l<n) {
315         int j=omp_get_thread_num();
316         if (j!=Wb::my_caller_tid) { l+=snprintf(s+l,n-l,"%d",j); }
317     }
318 #endif
319
320     if (l<n && vn && vn[0]) { l+=snprintf(s+l,n-l,"%s",vn); }
321     if (l<n) { l+=snprintf(s+l,n-l,".mat"); }
322
323     if (l>=n) wblog(F_L,"WRN %s() "
324         "string out of bounds (%d/%d)\n%s",FCT,l,n,s?s:"(null)");
325     return s;
326 };
327
328 std::string shortPF(const char *F, int L, unsigned lmax) {
329

```

```

330     const char *P = mytag; if (!P || !P[0]) { P=myname; }
331
332     unsigned i=0, j=0;
333     if (!F || !F[0]) { return (P?P:""); }
334
335     for (; F[i]; ++i) { if (F[i]=='/') j=i+1; }; F+=j;
336
337     if (!P || !P[0]) { return F; }
338
339     for (i=j=0; F[i]; ++i) { if (F[i]=='.' || F[i]==' ') j=i+1; }; if (!j) j=i;
340
341 #ifdef PROG
342     if (j && !strcmp(F,myname,j)) { return F; }
343 #endif
344
345     unsigned l1=strlen(F), l2=strlen(P);
346
347     unsigned ltot=l1+l2+1, n=lmax+4;
348     char s[n];
349
350     if (int(lmax)<8) wblog(FL,"ERR %s() invalid lmax=%d",FCT,lmax);
351
352     lmax-=1; if (L>0) { lmax-=log10(double(L)); }
353
354     if (ltot<lmax) { snprintf(s,n,"%s:%s",F,P); }
355     else if (l2<4) { char fmt[16];
356         unsigned nlx=MIN(l1,l2-j), n1=lmax-l2-nlx-2;
357         snprintf(fmt,16,"%%.%ds'%%.%.%ds:%%s", nl,nlx);
358         snprintf(s,n,fmt,F,F+j,P);
359     }
360     else if (l1<12) { char fmt[16];
361         unsigned n2=(lmax-l1-1);
362         snprintf(fmt,16,"%%.%ds:%%.%.%ds", n2);
363         snprintf(s,n,fmt,F,P);
364     }
365     else { char fmt[32];
366         unsigned nlx=MIN(l1,l2-j), n2=3, n1=lmax-nlx-n2-2;
367         if (j && n1>j-1) { n1=j-1; n2=lmax-n1-nlx-2; }
368         snprintf(fmt,32,"%%.%ds'%%.%.%ds:%%.%.%ds", n1,nlx,n2);
369         snprintf(s,n,fmt,F,F+j,P);
370     }
371
372     return s;
373 };
374
375 #endif
376

```

5.28 wbMatIO.hh

```

1 #ifndef __WB_MatIO_HCC__
2 #define __WB_MatIO_HCC__
3
4 // IO handling of mat files (MatLab binaries)
5
6 namespace Wb {
7
8     class matFile {
9     public:
10
11         matFile() : mfp(NULL) {};
12
13         matFile(const char *F, int L, const char *fname, const char *rw
14             ) : mfp(NULL) { open(F,L,fname,rw,1); };
15
16         matFile(const matFile &X) : mfp(NULL) {
17             wblog(FL,"ERR do not copy matFile object! (0x%lX)",&X);
18         };
19
20         ~matFile() {
21             init();
22         };
23
24         void close() { init(); };
25         void init() {
26             if (mfp) if (matClose(mfp)) {
27                 wblog(F.data,L,"WRN Could not close file %s%s /%s !?",
28                     strlen(fname.data)>20 ? "\n" : "", fname.data, rw.data);
29             }
30             mfp=NULL; F.init(); L=0; fname.init(); rw.init();
31         };
32
33         int Open(const char *F, int L, const char *FName, const char *RW

```

```

34     ){ return open(F,L,FName,RW,1); };
35
36 int open(
37     const char *file, int line, const char *FName, const char *RW,
38     char xflag=0
39 ){
40     if (mfp) init();
41     F=file; L=line; fname=FName;
42     if (!RW || !RW[0]) wblog(FL,
43         "ERR %s() got empty read/write flag",FCT);
44
45     if (!strcmp(RW,"w")) {
46         rw="w7.3";
47     }
48     else if (!strcmp(RW,"w:std")) {
49         rw="wz";
50     }
51     else { rw=RW; }
52
53     mfp=matOpen(fname.data, rw.data);
54     if (!mfp) {
55         if (xflag) { unsigned n=strlen(fname.data);
56             if (n<50)
57                 wblog(F.data,L,"ERR failed to %s access%s%s"
58                     "\n(hint: non-existing directory?)",
59                     rw.data, n<30?" ":" file\n",fname.data);
60             else {
61                 char *x,cwd[256]; cwd[0]=0; x=getcwd(cwd,256);
62                 wblog(F.data,L,"ERR failed to access (%s) ... %N%N"
63                     "   mat: %s !?%N"
64                     "   cwd: %s%N"
65                     " (hint: non-existing directory or no rw permission)%N",
66                     rw.data, fname.data, x ? cwd : "(pwd)");
67             }
68         }
69         init(); return 0;
70     }
71     return 1;
72 };
73
74 matFile& put(
75     const char *file, int line,
76     const char *vname, const mxArray* a
77 ){
78     int i=matPutVariable(mfp,vname,a);
79     if (i) {
80         wblog(file ? file : F.data, line ? line : L,
81             "ERR matPutVariable() returned %d \n(%s,%s; 0x%lX) !?\n"
82             "hint: possibly out of disc space", i,
83             fname.data ? fname.data : "(undefined)",
84             vname ? vname : "(empty)", mfp
85         );
86     }
87     return *this;
88 };
89 matFile& put(const char *vname, const mxArray* a){
90     return put(0,0,vname,a);
91 };
92
93 int existsVar(const char* vn) const {
94     mxArray *a=matGetVariableInfo(mfp,vn);
95     int r=(a!=NULL);
96     mxDestroyArray(a);
97     return r;
98 };
99
100 int deleteVar(const char* vn) const {
101     int i=matDeleteVariable(mfp,vn);
102     return i;
103 };
104
105 int deleteVar(const char *f, int L, const char* vn) const {
106     int i=matDeleteVariable(mfp,vn);
107     if (i) wblog(f,L,
108         "ERR Failed to delete variable '%s'\n"
109         "from file '%s' (0x%lX; %d/%d ?)",
110         vn, fname.data, mfp, existsVar(vn), i);
111     return i;
112 };
113
114 MATFile *mfp;
115
116 wbsting F, fname, rw;
117 int L;
118
119 protected:
120 private:

```

```

121 };
122
123 }
124
125 #endif
126

```

5.29 wbMatrix.hh

```

1  #ifndef __WB_MATRIX_ROW_MAJOR_HH__
2  #define __WB_MATRIX_ROW_MAJOR_HH__
3
4  #ifdef WB_SPARSE_CLOCK
5  Wb::Clock wbMat_group3("gr3::sortRecs");
6  Wb::Clock wbMat_group2("gr2::groupRecs");
7  Wb::Clock wbMat_group1("gr1::groupRecs");
8  #endif
9
10 /* ----- *
11  * wbMatrix - row-major matrix class
12  * AWb C Jan 2006
13  * ----- */
14
15 template <class T>
16 class wbMatrix {
17
18 public:
19
20     wbMatrix(size_t r=0, size_t c=0)
21         : data(NULL), dim1(r), dim2(c), isdiag(0), isref(0) {
22         if (r || c) NEW_DATA();
23     };
24
25     wbMatrix(size_t r, size_t c, T *d, const char ref=0)
26         : data(NULL), dim1(r), dim2(c), isdiag(0), isref(0) {
27         if (ref) { data=d; isref=1; }
28         else if (r || c) NEW_DATA(d);
29     };
30
31     wbMatrix(const wbMatrix &M)
32         : data(NULL), dim1(M.dim1), dim2(M.dim2), isdiag(M.isdiag), isref(0) {
33         NEW_DATA(M.data);
34     };
35
36     template <class T2>
37     wbMatrix(const wbMatrix<T2> &M)
38         : data(NULL), dim1(0), dim2(0), isdiag(0), isref(0) {
39         if (M.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,M.isdiag);
40         initT(M);
41     };
42
43     wbMatrix(const mxArray *a, char tflag=0, char ref=0)
44         : data(NULL), dim1(0), dim2(0), isdiag(0), isref(0) {
45         init(FL,a,tflag,ref);
46     };
47
48     wbMatrix(const char *F, int L, const mxArray *a)
49         : data(NULL), dim1(0), dim2(0), isdiag(0), isref(0) {
50         init(F,L,a);
51     };
52
53     virtual ~wbMatrix() { if (data && !isref) {
54         WB_DELETE(data);
55     } };
56
57     void copyStride(T* dd, size_t stride) const {
58         T *d0=data;
59         for (size_t i=0; i<dim1; ++i, d0+=dim2, dd+=stride)
60             MEM_COPY<T>(dd, dim2, d0);
61     };
62
63     wbMatrix& operator= (const wbMatrix &B) {
64         if (this!=&B) {
65             RENEW(B.dim1, B.dim2, B.data);
66             isdiag=B.isdiag;
67         }
68         return *this;
69     };
70
71     template<class TB>
72     wbMatrix& operator= (const wbMatrix<TB> &B) {
73         init(B.dim1, B.dim2);
74         for (size_t i=0, n=B.numel(); i<n; ++i) { data[i]=T(B.data[i]); }

```

```

75     isdiag=B.isdiag;
76     return *this;
77 };
78
79 explicit operator bool() const {
80     if (data)
81         { if ( dim1 && dim2) { return true; }}
82     else { if (!dim1 || !dim2) { return false; }}
83
84     wblog(FL,"ERR %s() dim=%gx%g 0x%p",FCT,dim1,dim2,data);
85     return 0;
86 };
87 bool operator! () const { return (*this ? false : true); }
88
89 bool operator==(const T &x) const {
90     for (size_t n=dim1*dim2, i=0; i<n; ++i)
91         { if (data[i]!=x) return 0; }
92     return 1;
93 };
94
95 bool operator==(const wbMatrix &B) const {
96     if (this!=&B) {
97         if (dim1!=B.dim1 || dim2!=B.dim2) return 0;
98         if (data!=B.data) {
99             for (size_t n=dim1*dim2, i=0; i<n; ++i)
100                 if (data[i]!=B.data[i]) return 0;
101         }
102     }
103     return 1;
104 };
105
106 bool operator!=(const wbMatrix &B) const {
107     return !(*this==B);
108 };
109
110 bool deepEqualP(const wbMatrix &B) const;
111
112 bool anyEqual(const T& x) const {
113     for (size_t n=dim1*dim2, i=0; i<n; ++i) {
114         if (data[i]==x) return 1; }
115     return 0;
116 };
117
118 bool allEqual(const T& x) const {
119     for (size_t n=dim1*dim2, i=0; i<n; ++i) {
120         if (data[i]!=x) return 0; }
121     return 1;
122 };
123
124 bool anyUnequal(const T& x) const { return !allEqual(x); };
125
126 bool anyLT(const T& x) const {
127     for (size_t n=dim1*dim2, i=0; i<n; ++i) {
128         if (data[i]<x) return 1; }
129     return 0;
130 };
131
132 bool anyGT(const T& x) const {
133     for (size_t n=dim1*dim2, i=0; i<n; ++i) {
134         if (data[i]>x) return 1; }
135     return 0;
136 };
137
138 bool col_anyGT(size_t j, const T& x) const {
139     if (dim1) {
140         if (j>=dim2) wblog(FL,
141             "ERR %s() index out of bounds (%d/%d)",FCT,j,dim2);
142         for (size_t i=0; i<dim1; ++i, j+=dim2) {
143             if (data[j]>x) return 1;
144         }
145     }
146     return 0;
147 };
148
149 wbMatrix& operator+= (const wbMatrix &B) {
150     size_t i,s=dim1*dim2;
151
152     if (isdiag) if (!B.isdiag) isdiag=0;
153
154     if (dim1!=B.dim1 || dim2!=B.dim2) wblog(FL,
155         "ERR Dimension mismatch (%dx%d + %dx%d).",dim1,dim2,B.dim1,B.dim2);
156
157     for (i=0; i<s; ++i) data[i]+=B.data[i];
158     return *this;
159 };
160
161 wbMatrix operator+ (const wbMatrix &B) const {

```



```

162     wbMatrix Mout(*this); Mout+=B;
163     return Mout;
164 };
165
166 wbMatrix operator+ (const T &x) const {
167     wbMatrix Mout(*this); Mout+=x;
168     return Mout;
169 };
170
171 void operator-= (const wbMatrix &B) {
172     size_t i,s=dim1*dim2;
173
174     if (isdiag) if (!B.isdiag) isdiag=0;
175
176     if (dim1!=B.dim1 || dim2!=B.dim2) wblog(FL,
177     "ERR Dimension mismatch (%dx%d + %dx%d).",dim1,dim2,B.dim1,B.dim2);
178
179     for (i=0; i<s; ++i) data[i]-=B.data[i];
180 };
181
182 wbMatrix operator- (const wbMatrix &B) const {
183     wbMatrix Mout(*this); Mout-=B;
184     return Mout;
185 };
186
187 void operator*= (const T c) {
188     size_t i, s=dim1*dim2;
189
190     if (c==T(+1)) { return; } else
191     if (c==T( 0)) { set((T)0); } else
192     if (c==T(-1)) { for (i=0; i<s; ++i) data[i]=-data[i]; }
193     else { for (i=0; i<s; ++i) data[i]*=c; }
194 };
195
196 wbMatrix& operator+= (const T c) { if (c) {
197     for (size_t s=dim1*dim2, i=0; i<s; ++i) data[i]+=c; }
198     return *this;
199 };
200
201 void operator-= (const T c) { if (c) {
202     for (size_t s=dim1*dim2, i=0; i<s; ++i) data[i]-=c; }
203 };
204
205 wbMatrix operator* (const T x) const {
206     wbMatrix X(*this); if (x!=1) X*=x;
207     return X;
208 };
209
210 wbMatrix& add (const wbMatrix &B, const T& c=1, const char zflag=0);
211 wbMatrix& minus(const wbMatrix &B, const T& c=1, const char zflag=0);
212 wbMatrix& times(T fac, wbMatrix &X) const {
213     X*=this; X*=fac; return X;
214 };
215
216 T prod() const {
217     size_t i,n=dim1*dim2; T x=0;
218
219     if (n==0) {
220         wblog(FL,"WRN %s() of null object !?",FCT);
221         return x;
222     }
223
224     for (x=data[0], i=1; i<n; ++i)
225         if (x<data[i]) x*=data[i];
226
227     return x;
228 };
229
230 T min() const {
231     size_t i,n=dim1*dim2; T x=0;
232
233     if (n==0) { wblog(FL,
234     "WRN %s() of null object !?",FCT); return x; }
235     for (x=data[0], i=1; i<n; ++i) { if (x>data[i]) x=data[i]; }
236
237     return x;
238 };
239
240 T max() const {
241     size_t i,n=dim1*dim2; T x=0;
242
243     if (n==0) { wblog(FL,
244     "WRN %s() of null object !?",FCT); return x; }
245     for (x=data[0], i=1; i<n; ++i) { if (x<data[i]) x=data[i]; }
246
247     return x;
248 };

```

```

249
250 T amax() const {
251     T m=0, x;
252     size_t n=dim1*dim2;
253
254     if (n==0) { wblog(FL,
255         "WRN %s() of null object !?",FCT); return m; }
256     for (size_t i=0; i<n; ++i) { x=ABS(data[i]); if (m<x) m=x; }
257
258     return m;
259 };
260
261 T sum() const {
262     T x=0;
263     size_t i, n=dim1*dim2;
264     for (i=0; i<n; ++i) x+=data[i];
265     return x;
266 };
267
268 T cumsum0prodRec(wbvector<T> &cs, char xflag=0) const {
269     wbvector<T> x;
270     return recProd(x).cumsum_(cs,xflag);
271 };
272
273 T cumsum0prodRec(
274     wbvector<T> &cs, unsigned i1, unsigned i2, char xflag=0
275 ) const {
276
277     wbvector<T> x(dim1);
278     if (i2>=dim2) wblog(FL,
279         "ERR %s() index out of bounds (%d/%d)",FCT,i2+1,dim2);
280
281     if (i1<=i2) {
282         unsigned i=0, j, n=i2-i1+1; const T* d=data+i1;
283         for (; i<dim1; ++i, d+=dim2) {
284             x[i]=d[0];
285             for (j=1; j<n; ++j) { x[i]*=d[j]; }
286         }
287     }
288
289     return x.cumsum_(cs,xflag);
290 };
291
292 T norm() const { return Sqrt(norm2()); }
293
294 T norm2() const {
295     size_t i, n=dim1*dim2; T x=0;
296     for (i=0; i<n; ++i) x+=NORM2(data[i]);
297     return x;
298 };
299
300 double normReal() const;
301 double normImag() const;
302
303 size_t maxRec(char lex=1, size_t *m=NULL) const;
304 size_t maxRec_float(char lex=1, size_t *m=NULL, T xref=-1) const {
305     return maxRec(lex,m);
306 };
307
308 T recMax(size_t r, size_t *k=NULL) const;
309 T colMax(size_t c, size_t *k=NULL) const;
310
311 T colMax(size_t c, unsigned &k) const { T x;
312     size_t K; x=colMax(c,&K); k=K;
313     if (size_t(k)!=K) wblog(FL,
314         "ERR %s() unsigned out of bounds (%d/%ld)",FCT,k,K);
315     return x;
316 };
317
318 wbMatrix& NormalizeCol(size_t k);
319
320 double recMaxA(size_t r, size_t *k=NULL) const;
321
322 size_t skipNanRecs(wbindex &I);
323
324 wbMatrix& init(size_t r=0, size_t c=0, const T* d=NULL) {
325     if (isref) {
326         isref=0; dim1=dim2=0; data=0;
327     }
328     RENEW(r,c,d); return *this;
329 };
330
331 wbMatrix& init(
332     size_t r, size_t c, const T *d0, size_t stride
333 ) {
334     unsigned i=0, j=0; T* d;
335

```

```

336         if (int(stride)<0) wblog(FL,
337             "ERR %s() got stride=%ld !?",FCT,stride);
338         if (!d0) wblog(FL,"ERR %s() got null space !?",FCT);
339
340         RENEW(r,c,NULL,0); d=data;
341         for (; i<dim1; ++i, d+=dim2, d0+=stride) {
342             for (j=0; j<dim2; ++j) d[j]=d0[j];
343         }
344         return *this;
345     };
346
347     wbMatrix& init2val(size_t r, size_t c, const T& d) {
348         RENEW(r,c,NULL,0); set(d);
349         return *this;
350     };
351
352     wbMatrix& initDef(size_t r=0, size_t c=0, T* d=NULL) {
353         RENEW(r,c,NULL,0);
354         if (d) for (size_t n=r*c, i=0; i<n; ++i) data[i]=d[i];
355         return *this;
356     };
357
358     template<class T2>
359     wbMatrix<T>& initT(size_t r, size_t c, T2* d) {
360         RENEW(r,c,0,0);
361         for (size_t n=r*c, i=0; i<n; ++i) data[i]=(T)d[i];
362         return *this;
363     };
364
365     template<class T2>
366     wbMatrix<T>& initT(const wbMatrix<T2>& B) {
367         if (B.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,B.isdiag);
368         RENEW(B.dim1,B.dim2,0,0);
369         for (size_t n=dim1*dim2, i=0; i<n; ++i) data[i]=(T)B.data[i];
370         return *this;
371     };
372
373     template<class T2>
374     wbMatrix<T>& initT(const char *F, int L, const wbMatrix<T2>& B) {
375         if (B.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,B.isdiag);
376         RENEW(B.dim1,B.dim2,0,0);
377         for (size_t n=dim1*dim2, i=0; i<n; ++i) { data[i]=(T)B.data[i];
378             if (T2(data[i])!=B.data[i]) wblog(F,L,
379                 "ERR type conversion changes value (%g,%g)",
380                 double(B.data[i]), double(T2(data[i])))
381             );
382         }
383         return *this;
384     };
385
386     template<class TA, class TB>
387     wbMatrix<T>& init(const char *F, int L,
388         const wbMatrix<TA>& B, const wbvector<TB>& V, char dim=2
389     );
390
391     template<class T2>
392     wbMatrix<T>& initDim(const wbMatrix<T2>& B) {
393         RENEW(B.dim1, B.dim2);
394         return *this;
395     };
396
397     wbMatrix& init(const wbMatrix& B) {
398         RENEW(B.dim1, B.dim2, B.data); isdiag=B.isdiag;
399         return *this;
400     };
401
402     wbMatrix& init(
403         const char *F, int L, const mxArray *a,
404         char tflag=0, char ref=0, char tcheck=1);
405
406     void init(const mxArray *a) { init(FL,a); };
407
408     void init0(const mxArray *a) {
409         init(FL,a,1,0,1);
410     };
411
412     void initTST() {
413         size_t i,j,k=0;
414         for (i=0; i<dim1; ++i)
415             for (j=0; j<dim2; ++j) data[k++]=10*(i+1)+(j+1);
416     };
417
418     wbMatrix& init2ref(size_t r, size_t c, const T* d);
419     wbMatrix& init2ref(const wbvector<T>& v, const char tflag=0);
420     wbMatrix& init2ref(const wbMatrix& M);
421
422     wbMatrix& unRef();

```

```

423     wbMatrix& save2(wbMatrix &A, char ref=0);
424
425     void ref_check(const char *F, int L, const char *fct) {
426         if (isref) wblog(F,L,
427             "ERR %s() got reference (isref=%d)",fct,isref);
428     };
429
430     wbMatrix& ResizeXRecs(size_t r);
431     wbMatrix& Resize(size_t r, size_t c, const T* dx=NULL);
432     wbMatrix& resize(size_t m, size_t n, wbMatrix &M) const;
433
434     wbMatrix& Resize2Mult(const WBINDEX &M);
435
436     wbMatrix& transpose(wbMatrix &M) const;
437     wbMatrix& Transpose() {
438         wbMatrix X; save2(X); return X.transpose(*this);
439     };
440
441     wbMatrix& Reshape(size_t r, size_t c);
442
443     template <class T2>
444     wbMatrix<T>& AddCols(size_t n, const T2* d0=NULL) {
445         Resize(dim1,dim2+n);
446         if (d0) Wb::cpyStride(data+dim2-n, d0, n, dim1, dim2);
447         return *this;
448     };
449
450     template <class T2>
451     wbMatrix<T>& AddRows(size_t n, const T2* d0=NULL) {
452         Resize(dim1+n,dim2);
453         if (d0) MEM_CPY<T>(data+(dim1-n)*dim2, n*dim2, d0);
454         return *this;
455     };
456
457     bool isEmpty() const { return data==NULL; };
458     bool isNormal() const;
459
460     bool isSquare() const { return (dim1==dim2); };
461     bool isSquare(size_t d) const { return (dim1==dim2 && d==dim1); };
462
463     bool isScalar() const { return (dim1==1 && dim2==1); };
464     bool isVector() const { return (dim1==1 || dim2==1); };
465     bool isDiag() const;
466     bool isIdentity(const double &eps, double &maxdiff) const;
467
468     bool isHConj(
469         const wbMatrix &B, double eps=1E-12, double *xref=NULL
470     ) const { return isSym_aux(B, eps, xref, 's'); };
471
472     bool isHConj(
473         double eps=1E-12, double *xref=NULL
474     ) const { return isSym_aux(*this,eps,xref,'s'); };
475
476     bool isAHerm(
477         const wbMatrix &B, double eps=1E-12, double *xref=NULL
478     ) const { return isSym_aux(B, eps, xref, 'a'); };
479
480     bool isAHerm(
481         double eps=1E-12, double *xref=NULL
482     ) const { return isSym_aux(*this,eps,xref,'a'); };
483
484     bool isComplex() const;
485
486     bool isUnique() const;
487
488     bool isUniqueSorted(char dir=0, char lex=1) const;
489
490     int isSorted(
491         char dir=0,
492         char lex=1,
493         size_t m=-1
494     ) const;
495
496     bool quickCheckSorted(
497         char dir,
498         char lex=1,
499         const char *F=NULL, int L=0,
500         size_t n=2
501     ) const;
502
503     bool thisIsInt() const;
504
505     int getDiff(
506         const wbMatrix &B, wbindex &Ia, wbindex *Ib=NULL) const;
507
508     int getDiffSorted(
509         const wbMatrix &B, wbindex &Ia, wbindex *Ib=NULL) const;

```

```

510
511     size_t length() const { return (dim1>dim2 ? dim1 : dim2); };
512     size_t totSize() const { return (dim1*dim2); };
513     size_t numel() const { return dim1*dim2; };
514
515     void flipSign() {
516         size_t i,s=dim1*dim2;
517         for (i=0; i<s; ++i) data[i]=-data[i];
518     };
519
520     wbMatrix& Symmetrize() {
521         size_t i,j,r,s;
522         if (dim1!=dim2) wblog(FL,
523             "ERR Symmetrize() called with %dx%d matrix !?",dim1,dim2);
524
525         for (j=0; j<dim2; ++j)
526             for (i=j+1; i<dim1; ++i) {
527                 r=i*dim2+j; s=j*dim2+i;
528                 data[r]=data[s]=0.5*(data[r]+data[s]);
529             }
530
531         return *this;
532     };
533
534     void set(const T &x) {
535         for (size_t s=dim1*dim2, i=0; i<s; ++i) { data[i]=x; }
536         isdiag=0;
537     };
538
539     wbMatrix& setRand(double fac=1., double shift=0.);
540     void setDiagRand(double fac=1., double shift=0.);
541
542     double maxRelDiff(const wbMatrix &M) const;
543
544     template <class T2>
545     double normDiff(const wbMatrix<T2> &M) const;
546
547     T normDiff2(const wbMatrix &M, size_t *k=NULL) const;
548     T normDiff (const wbMatrix &M, size_t *k=NULL) const {
549         return SQRT(normDiff2(M,k));
550     };
551
552     bool hasSameSize(const wbMatrix B) const {
553         return (dim1==B.dim1 && dim2==B.dim2);
554     };
555
556     WBINDEX& toIndex(WBINDEX &I, const WBINDEX *S=NULL) const {
557         wblog(FL,"ERR %s() not defined for type '%s'",FCT,
558             TSTR(T)); return I;
559     };
560
561     WBIDXMAT& toIndex2D(
562         const WBINDEX &S, const wbindeX &ic, WBIDXMAT &IJ,
563         char pos=1
564     ) const {
565         wblog(FL,"ERR %s() not defined for type '%s'",FCT,
566             TSTR(T)); return IJ;
567     };
568
569     wbMatrix& SkipTiny_float(T ref __attribute__((unused)) ==-1) {
570         return *this;
571     };
572
573     void reset() {
574         if (dim1 && dim2) MEM_SET<T>(data,dim1*dim2);
575         isdiag=0;
576     }
577
578     size_t nnz() const {
579         size_t n=0,i,s=dim1*dim2;
580         for (i=0; i<s; ++i) if (data[i]!=T(0)) ++n;
581         return n;
582     }
583
584     wbMatrix& CAT(C_UINT d12, const wbMatrix** M, C_UINT len);
585     wbMatrix& CAT(C_UINT d12, const wbvector< wbMatrix > &M0);
586     wbMatrix& CAT(C_UINT d12, wbvector< wbMatrix const* > M);
587
588     wbMatrix& CAT(C_UINT d12, const wbvector< wbvector<T> > &M);
589     wbMatrix& CAT(C_UINT d12, const wbvector< wbvector<T> const* > &M);
590
591     wbMatrix& cat(C_UINT d12, const wbMatrix&);
592     wbMatrix& cat(C_UINT d12, const wbMatrix&, const wbMatrix&);
593     wbMatrix& cat(C_UINT d12, const wbMatrix&, const wbMatrix&, const wbMatrix&);
594
595     wbMatrix& Cat(C_UINT d12,
596         const wbMatrix&, const wbMatrix&);

```

```

597     wbMatrix& Cat(C_UINT d12,
598         const wbMatrix&, const wbMatrix&, const wbMatrix&);
599     wbMatrix& Cat(C_UINT d12,
600         const wbMatrix&, const wbMatrix&, const wbMatrix&, const wbMatrix&);
601
602     wbMatrix<T>& CAT(const unsigned dim,
603         wbvector< wbMatrix<T> const* > M,
604         wbvector< WBINDEX const* > I
605     );
606
607     wbMatrix& Cat(const unsigned dim,
608         const wbMatrix &M1, const wbindex &I1,
609         const wbMatrix &M2, const wbindex &I2
610     ){
611         wbvector< wbMatrix<T> const* > M(2);
612         wbvector< WBINDEX const* > I(2);
613         M.data[0]=&M1; I.data[0]=&I1;
614         M.data[1]=&M2; I.data[1]=&I2;
615         return CAT(dim,M,I);
616     };
617
618     void split(C_UINT d12, wbvector< wbMatrix* > &M) const;
619     void split(C_UINT d12, wbMatrix** M, C_UINT len) const;
620     void split(C_UINT d12, wbMatrix &M1, wbMatrix &M2, wbMatrix &M3) const;
621
622     void swap(wbMatrix &B) {
623         if (this!=&B) {
624             if (isref || B.isref) wblog(FL,
625                 "ERR %s() got arrays with isref=(%d,%d)",FCT,isref,B.isref);
626             SWAP(data, B.data );
627             SWAP(dim1, B.dim1 );
628             SWAP(dim2, B.dim2 );
629             SWAP(isdiag, B.isdiag);
630         }
631     };
632
633     const T& operator() (size_t i, size_t j, const T&x) const {
634         if (i<dim1 && j<dim2) return data[i*dim2+j];
635         else return x;
636     };
637     T& operator() (size_t i, size_t j, const T&x) {
638         if (i<dim1 && j<dim2) return data[i*dim2+j];
639         else return x;
640     };
641
642     T& operator() (size_t i, size_t j) { return data[i*dim2+j]; };
643     const T& operator() (size_t i, size_t j) const {
644         return data[i*dim2+j];
645     };
646
647     const T& el(size_t i, size_t j) const {
648         if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
649             "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
650         return data[i*dim2+j];
651     };
652     T& el(size_t i, size_t j) {
653         if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
654             "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
655         return data[i*dim2+j];
656     };
657
658     const T& els(long i, long j) const {
659         if (i<0) i+=dim1;
660         if (j<0) j+=dim2;
661         if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
662             "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
663         return data[i*dim2+j];
664     };
665     T& els(long i, long j) {
666         if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
667             "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
668         return data[i*dim2+j];
669     };
670
671     const T* operator() (size_t i) const { return data+i*dim2; };
672     T* operator() (size_t i) { return data+i*dim2; };
673
674     const T& operator[] (size_t i) const { return data[i]; };
675     T& operator[] (size_t i) { return data[i]; };
676
677     const T* ref(size_t i, size_t j=0) const {
678         if (int(j)<0) j=dim2+j;
679         if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
680             "index out of bounds (%ld/%ld, %ld/%ld)",FCT,i,dim1,j,dim2);
681         return data+(i*dim2+j);
682     };
683     T* ref(size_t i, size_t j=0) {

```

```

684     if (int(j)<0) j=dim2+j;
685     if (i>=dim1 || j>dim2) wblog(FL, "ERR %s() "
686         "index out of bounds (%ld/%ld, %ld/%ld)", FCT, i, dim1, j, dim2);
687     return data+(i*dim2+j);
688 };
689
690 const T* rec(size_t i) const {
691     if (i>=dim1) wblog(FL, "ERR index out of bounds (%d/%d)", i, dim1);
692     return data+i*dim2;
693 };
694
695 T* rec(size_t i) {
696     if (i>=dim1) wblog(FL,
697         "ERR %s() index out of bounds (%d/%d)", FCT, i, dim1);
698     return data+i*dim2;
699 };
700
701 void getDiag(wbvector<T> &D) const {
702     size_t i, n=MIN(dim1, dim2); D.init(n);
703     for (i=0; i<n; ++i) D[i]=data[i*dim2+i];
704 };
705 wbvector<T> getDiag() const {
706     wbvector<T> D; getDiag(D); return D;
707 };
708
709 void appendRows (size_t n, const T* =NULL);
710 void appendRow(const wbvector<T> &v);
711
712 mxArray* toMx (char raw=0) const;
713 mxArray* toMxP(char flag=0) const;
714 mxArray* toMx_base(char raw=0) const;
715 mxArray* toMx_base_d(char raw=0) const;
716
717 mxArray* toMx_Struct() const;
718 mxArray* toMx_StructP() const;
719 mxArray* toMx_CellP() const;
720
721 mxArray* toMxT(char raw=0) const { return toMx(); }
722
723 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
724 void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
725
726 void toMxStruct(mxArray* S, const char *vname) const;
727 void mat2mx (mxArray* &a) const { a=toMx(); };
728 void mat2mxs(mxArray* a, char field_nr) const;
729
730 void info(const char *istr="ans") const;
731 void print(const char *istr="", char mflag=0) const;
732 void Print(const char *istr="", char mflag=0) const;
733
734 void recPrint(size_t i, const char *istr="", char mflag=0) const;
735
736 void printdata(
737     const char *istr, const char *dbl_fmt=" %8.4g", const char *rsep="\n") const;
738
739 wbstring toStr(
740     const char *fmt="", const char *sep=" ", const char *rsep=";", "
741     size_t stride=0, const char *sep2=""
742 ) const;
743
744 wbstring rec2Str(
745     size_t k, const char *fmt="", const char *sep=" ",
746     size_t stride=0, const char *sep2=""
747 ) const;
748
749 wbstring sizeStr() const {
750     size_t l=0, n=16; char s[n];
751     l=snprintf(s, n, "%ldx%ld", dim1, dim2);
752     if (l>=n) wblog(FL, "ERR %s() string out of bounds (%ld/%ld)", FCT, l, n);
753     return s;
754 };
755
756 void put(const char *F, int L,
757     const char *vname="ans", const char *ws="caller", const char raw=0
758 ) const {
759     mxArray *a=toMx(raw);
760     mxPutAndDestroy(F_L, a, vname, ws);
761 };
762
763 void put(const char *vname="ans", const char *ws="caller",
764     const char raw=0) const { put(0, 0, vname, ws, raw); }
765
766 void putg(const char *vname="ans", const char raw=0
767 ) const { put(vname, "global", raw); return; };
768
769 void putx(const char *vname="ans", const char raw=0
770 ) const { put(vname, "caller", raw); return; };

```

```

771
772 void put0(const char *vname="ans", const char *ws="caller") const {
773     put(vname,ws,'r');
774 }
775
776 void getReal(wbMatrix<double> &R) const;
777 void getImag(wbMatrix<double> &I) const;
778 void set(const wbMatrix<double> &R, const wbMatrix<double> &I);
779 wbMatrix& Conj();
780
781 void setRec (size_t r, const T x) {
782     T* d=data+r*dim2;
783     if (r>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",r,dim1);
784     for (size_t i=0; i<dim2; ++i) d[i]=x;
785 };
786
787 void setRec (size_t r, const T x1, const T x2) {
788     size_t i=0, m=dim2%2, n=dim2-m; T* d=data+r*dim2;
789
790     if (r>=dim1) wblog(FL,
791         "ERR index out of bounds (%d/%d)",r,dim1);
792
793     for (; i<n; i+=2) { d[i]=x1; d[i+1]=x2; }
794     if (m) d[i]=x1;
795 };
796
797 void recSet (size_t idest, size_t isrc);
798
799 void recSet (size_t, const wbvector<T>&);
800 void recSet (size_t, const wbMatrix&, size_t);
801 void recSet (size_t i, const wbvector<T>&, const wbvector<T>&);
802
803 void recSetP(size_t, const T*, size_t n=-1);
804 void recSetP(size_t, const T*, size_t, const T*, size_t);
805 void recSetP(size_t,
806     const T*, const wbindex &i1, const T*, const wbindex &i2);
807
808 void recSetB(size_t, size_t, size_t D, const T*,
809     const T* =NULL, const T bfac=1);
810 void recSetB(size_t, const wbindex&, size_t D, const T*);
811
812 void recSet(size_t k,
813     const wbvector<T> &a, const wbindex &ia,
814     const wbvector<T> &b, const wbindex &ib
815 ){ return recSet(a.data,ia,b.data,ib); };
816
817 void recSet(size_t k,
818     const T* a, const wbindex &ia,
819     const T* b, const wbindex &ib
820 );
821
822 template<class T2>
823 void setCol (size_t k, const wbvector<T2> &v);
824
825 template<class T2>
826 void setCol (size_t k, const T2 *v, size_t stride=1);
827
828 void setCol (size_t k, const T& x);
829
830 template <class T2>
831 void setColT(size_t k, const wbvector<T2> &v);
832 template <class T2>
833 void setColT(size_t k, const T2* v, size_t stride=1);
834
835 void setLastCol(const T& x) { setCol(dim2-1,x); };
836
837 wbMatrix& scaleCol(size_t k, const T& fac);
838 wbMatrix& scaleRow(size_t k, const T& fac);
839
840 void recAddP(size_t, const T*);
841 int recLess (size_t, size_t, char lex=1) const;
842 int recLessE(size_t, size_t, char lex=1) const;
843 int recEqual(size_t, size_t) const;
844 int recEqual(size_t j1, const T *d) const;
845
846 T recDiff2(size_t i1, size_t i2, size_t n=-1) const;
847 T recDiff2P(size_t i1, const T* b, size_t n=-1) const;
848
849 bool recAllEqual(const T *v) {
850     for (size_t i=0; i<dim1; ++i) { if (recCompareP(i,v)) return 0; }
851     return 1;
852 };
853
854 bool recAllEqual() {
855     for (size_t i=1; i<dim1; ++i) { if (recCompare(i,0)) return 0; }
856     return 1;
857 };

```



```

858
859 bool recIsZero(size_t r) {
860     T *d=data+r*dim2;
861     if (r>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",r,dim1);
862     for (size_t i=0; i<dim2; ++i) if (d[i]!=0) return 0;
863     return 1;
864 };
865
866 char recCompare(
867     size_t, size_t, size_t m=-1, char lex=1, T eps=0) const;
868 char recCompare(
869     size_t, const wbvector<T> &r2, char lex=1) const;
870 char recCompareP(
871     size_t, const T*, size_t m=-1, char lex=1, T eps=0) const;
872
873 void set2Rec(size_t i) { recSet(0,i); Resize(1,dim2); };
874
875 wbMatrix& Set2Recs(const WBINDEX &I) {
876     wbMatrix X; save2(X);
877     return X.getRecs(I,*this);
878 };
879
880 wbvector<T>& getRec (size_t j0, wbvector<T> &v) const;
881 wbvector<T> getRec (size_t j0) const {
882     wbvector<T> v; getRec(j0,v); return v;
883 };
884
885 size_t UnionRecs(const wbMatrix &B, wbindex &Ia);
886
887 bool gotRecOverlap (const wbMatrix &B) const;
888 bool gotRecOverlapSA(const wbMatrix &B) const;
889
890 size_t findValsCol(
891     const size_t k, const wbvector<T> &v, wbindex &Ia) const;
892
893 wbMatrix& getRecs(const WBINDEX &, wbMatrix &M) const;
894 wbMatrix getRecs(const WBINDEX &I) const {
895     wbMatrix M; getRecs(I,M);
896     return M;
897 };
898
899 wbMatrix& getRecs(
900     size_t j1, size_t j2, wbMatrix &M, size_t m=0) const;
901
902 wbMatrix getRecs(
903     size_t i1, size_t i2, size_t m=0) const {
904     wbMatrix M; getRecs(i1,i2,M,m);
905     return M;
906 };
907
908 wbvector<T>& getCol(const size_t c, wbvector<T> &v) const;
909 wbvector<T> getCol(const size_t c) const {
910     wbvector<T> v; getCol(c,v); return v;
911 };
912
913 wbMatrix getCols(const size_t j1, const size_t j2) const {
914     wbMatrix M; getCols(j1,j2,M);
915     return M;
916 };
917
918 wbMatrix& getCols(size_t j1, size_t j2, wbMatrix &M) const;
919
920 template <class TI>
921 wbMatrix& getCols(size_t n, const TI *I, wbMatrix<T> &M) const;
922
923 wbMatrix& getCols(const wbpem &p, wbMatrix &M) const {
924     return getCols(p.len,p.data,M);
925 };
926
927 wbMatrix& getCols(
928     const wbindex &I, wbMatrix &M, char iflag=0) const {
929
930     if (!iflag) { return getCols(I.len,I.data,M); }
931     else {
932         wbindex I2; I.invert(dim2,I2);
933         return getCols(I2.len,I2.data,M);
934     }
935 };
936
937 wbMatrix& set2Cols(
938     const size_t j1, const size_t j2);
939
940 wbMatrix& set2Cols(const wbindex &J, char iflag=0) {
941     wbMatrix X; this->save2(X); X.getCols(J,*this,iflag);
942     return *this;
943 };
944

```

```

945     wbMatrix& recPermute(const wbperm &P, wbMatrix &M, char iflag=0) const;
946     wbMatrix& recPermute(const wbperm &P, char iflag=0) {
947         wbMatrix M(*this);
948         return M.recPermute(P,*this,iflag);
949     };
950
951     wbMatrix& colPermute(const wbperm &P, wbMatrix &M, char iflag=0) const;
952     wbMatrix& colPermute(const wbperm &P, char iflag=0){
953         wbMatrix M; save2(M); return M.colPermute(P,*this,iflag);
954     };
955
956     wbMatrix& FlipRecs() { wbperm P(dim1,'r'); return recPermute(P); };
957     wbMatrix& FlipCols() { wbperm P(dim2,'r'); return colPermute(P); };
958
959     wbMatrix& cols2Front(
960         const WBINDEX &I1, wbMatrix &M) const;
961     wbMatrix& cols2End(
962         const WBINDEX &I1, wbMatrix &M) const;
963
964     void blockPermute(const wbperm &P, wbMatrix &M) const;
965     void BlockPermute(const wbperm &P){
966         wbMatrix X; save2(X); X.blockPermute(P,*this);
967     };
968
969     wbvector<T>& recProd(wbvector<T> &p) const;
970     T recProd(size_t) const;
971
972     T recSum (size_t) const;
973     wbvector<T>& recSum(wbvector<T> &) const;
974     wbvector<T>& recSumA(wbvector<T> &) const;
975     wbvector<T> recSum() const { wbvector<T> s; return recSum(s); };
976     T colSum (size_t) const;
977
978     wbMatrix& repmat(
979         size_t m, size_t n, wbMatrix &Q,
980         size_t pad1=0, size_t pad2=0
981     ) const;
982
983     wbMatrix& Repmat(
984         size_t m, size_t n,
985         size_t pad1=0, size_t pad2=0
986     ){ wbMatrix X(*this); return X.repmat(m,n,*this,pad1,pad2); };
987
988     void ColKron(
989         const wbMatrix &Q,
990         int d=-1
991     );
992
993     wbMatrix& getBlocks(
994         size_t j1, size_t j2,
995         const size_t D, wbMatrix &M
996     ) const;
997
998     wbMatrix& getBlock(
999         size_t j1,
1000         const size_t D, wbMatrix &M) const;
1001
1002     wbMatrix getBlock(
1003         size_t j1,
1004         const size_t D
1005     ) const {
1006         wbMatrix M; getBlock(j1,D,M);
1007         return M;
1008     };
1009
1010     template <class T2>
1011     wbMatrix<T>& getBlock(
1012         size_t k, size_t D,
1013         const wbMatrix<T2>& R, size_t k2, size_t D2,
1014         wbMatrix &M
1015     ) const;
1016
1017     wbMatrix& getBlock(
1018         size_t i, size_t j,
1019         size_t n, size_t m,
1020         wbMatrix &M
1021     ) const;
1022
1023     wbMatrix& blockSum(size_t D, wbMatrix&) const;
1024     void blockSum(size_t, size_t, const size_t, wbMatrix<double>&) const;
1025     void blockSum(const wbindex &J, const size_t, wbMatrix<double>&) const;
1026
1027     void setBlock(size_t k, const size_t D, const T*);
1028     void SetBlock(size_t i0, size_t j0, const wbMatrix &M);
1029
1030     WBIDXMAT& toBlockIndex(
1031         widx_t D, WBIDXMAT &II,

```

```

1032     wbvector< wbMatrix > *QI=NULL
1033 ) const;
1034
1035 template <class T2>
1036 void toBlockIndex(widx_t D,
1037     wbMatrix<T2> &R, widx_t DR,
1038     WBIDXMAT *II=NULL, WBIDXMAT *IS=NULL,
1039     WBIDXMAT *SS=NULL, wbvector< wbMatrix > *QI=NULL
1040 ) const;
1041
1042 void groupRecs(
1043     wbperm &P, WBINDEX &d,
1044     size_t m=-1,
1045     char lex=1,
1046     wbindex *Ig=NULL
1047 );
1048
1049 void groupRecs(wbindex &Ig, widx_t m=-1, char lex=1) {
1050     wbperm P; wbvector<widx_t> d;
1051     groupRecs(P,d,m,lex,&Ig);
1052 };
1053
1054 void groupRecs(
1055     groupIndex<widx_t> &IG, size_t m=-1, char lex=1,
1056     wbindex *Ig=NULL
1057 ){
1058     wbperm P; wbvector<widx_t> d;
1059     groupRecs(P,d,m,lex,Ig);
1060     IG.initX(P,d);
1061 };
1062
1063 void groupRecs(
1064     groupIndex<widx_t> &IG, size_t m, char lex, const wperm_t **p
1065 ){
1066     wbperm P; wbvector<widx_t> d;
1067     groupRecs(P,d,m,lex);
1068     IG.initX(P,d,p);
1069 };
1070
1071 void groupRecs(
1072     wbperm &P, WBINDEX &d, wbMatrix &B,
1073     size_t m=-1, char lex=1, wbindex *Ig=NULL
1074 ) const { B=*this; B.groupRecs(P,d,m,lex,Ig); };
1075
1076 void groupRecs(
1077     wbperm &P, WBINDEX &d,
1078     const WBINDEX *I, char lex=1
1079 );
1080
1081 template <class T2>
1082 void groupRecs_rdeg(wbperm &P, WBINDEX &D, const wbMatrix<T2> &R,
1083     WBINDEX *Ib=NULL,
1084     WBINDEX *I2=NULL,
1085     WBINDEX *Sb=NULL,
1086     char iflag=0
1087 );
1088
1089 void groupRecs(
1090     wbperm &P, WBINDEX &D, size_t nc,
1091     WBINDEX &Ib,
1092     WBINDEX &I2,
1093     WBINDEX &Sb,
1094     char iflag=0
1095 );
1096
1097 void groupSortedRecs(
1098     WBINDEX &d, char keepall=0, size_t m=-1, char lex=1);
1099
1100 void groupSortedRecs(
1101     WBINDEX &d, const WBINDEX *I);
1102
1103 void groupSortedRecs(
1104     WBINDEX &d,
1105     size_t mc,
1106     char lex,
1107     WBINDEX &Ib,
1108     WBINDEX &I2,
1109     WBINDEX &Sb
1110 );
1111
1112 void makeUnique() { if (dim1<2) return;
1113     wbperm P; WBINDEX D;
1114     groupRecs(P,D);
1115 };
1116 void makeUnique(wbperm &P, WBINDEX &D) {
1117     if (dim1<2) {
1118         P.init(dim1); D.init(dim1); if (dim1) D[0]=1;

```

```

1119         return;
1120     }
1121     groupRecs(P,D);
1122 };
1123
1124 void makeUnique(wbindex &I) {
1125     if (dim1<2) { I.init(dim1); }
1126     else {
1127         size_t i=0, l=0, d; wbpPerm P; WBINDEX D;
1128         groupRecs(P,D); I.init(D.len);
1129         for (; i<D.len; ++i, l+=d) { d=D[i]; I[i]=P[l]; }
1130     }
1131 };
1132
1133 void findUnique(wbindex &I) const {
1134     if (dim1<2) { I.init(dim1); return; }
1135
1136     wbMatrix X(*this);
1137     wbpPerm P; WBINDEX D; X.groupRecs(P,D);
1138     I.init(D.numEQ(T(1)));
1139     for (size_t d, l=0, j=0, i=0; i<D.len; ++i, l+=d) {
1140         if ((d=D[i])==1) I[j++]=P[l];
1141     }
1142 };
1143
1144 size_t findUniqueRecSorted1(size_t n=-1, T eps=T(1E-12)) const;
1145
1146 void makeUnique_ig(wbindex &Iu){
1147     wbpPerm P; wbvector<widx_t> D; groupRecs(P,D);
1148     Iu.initGroup(P,D);
1149 };
1150 void makeUnique(groupIndex<widx_t> &IG){ groupRecs(IG); };
1151
1152 wbMatrix& SortRecs(wbpPerm &P,
1153     char dir=+1,
1154     char lex=1
1155 );
1156
1157 wbMatrix& sortRecs(
1158     wbMatrix &B, wbpPerm &P, char dir=+1, char lex=1
1159 ){
1160     B=*this;
1161     return B.SortRecs(P,dir,lex);
1162 };
1163
1164 wbMatrix& SortRecs(char dir=+1, char lex=1) {
1165     wbpPerm P; return SortRecs(P,dir,lex);
1166 };
1167
1168 wbMatrix& sortRecs_float(wbpPerm &P, char dir=+1) {
1169     P.init(dim1); return *this;
1170 };
1171 wbMatrix& sortRecs_float() { wbpPerm P; return sortRecs_float(P); };
1172
1173 char findRecsInSet(const wbMatrix &S, wbvector<int> &I) const;
1174 int findRecSorted(const T* r, size_t n=-1, char lex=1) const;
1175 int findRec(const T* r, size_t n=-1, char lex=1) const;
1176
1177 T *data;
1178
1179 size_t dim1, dim2;
1180
1181 char isdiag, isref;
1182
1183 protected:
1184
1185 void NEW_DATA(T* d=NULL) { size_t s=dim1*dim2;
1186
1187     if (data) { WB_DELETE(data); }
1188     if (s) {
1189         WB_NEW(data,s);
1190         MEM_COPY<T>(data,s,d);
1191     }
1192     else data=NULL;
1193 };
1194
1195 void RENEW(
1196     const size_t &d1, const size_t &d2, const T* dd=NULL, char iflag=1
1197 ){
1198     const size_t s=d1*d2; isdiag=0;
1199
1200     if (isref) {
1201         if (d1==0 || d2==0) {
1202             if (d1 || d2) wblog(FL,
1203                 "WRN wbMatrix is declared as reference "
1204                 " (%dx%d => %dx%d; %d)",dim1,dim2,d1,d2,isref
1205             );

```

```

1206
1207         dim1=d1; dim2=d2; isref=0;
1208
1209         data=NULL;
1210
1211         return;
1212     }
1213     else {
1214         wblog(FL,"ERR wbMatrix is declared as reference (%dx%d; %d)",
1215             d1,d2, isref);
1216     }
1217 }
1218
1219 if (d1!=dim1 || d2!=dim2) {
1220
1221     dim1=d1; dim2=d2; if (data) {
1222         if (dd==data) wblog(FL,
1223             "ERR init space equals *this (use Resize instead)");
1224         WB_DELETE(data);
1225     }
1226     if (d1==0 || d2==0) return;
1227     WB_NEW(data,s);
1228 }
1229
1230 if (!iflag && !dd) return;
1231 if (s) MEM_COPY<T>(data,s,dd);
1232 };
1233
1234 private:
1235
1236     bool isSym_aux(
1237         const wbMatrix &B, double eps, double *xref,
1238         const char symflag='s'
1239     ) const;
1240 };
1241
1242 template <class T>
1243 class wbRecs {
1244
1245     const T* data;
1246     size_t n,lda; bool asc, lex;
1247
1248 public:
1249
1250     wbRecs(const wbMatrix<T> &A, wbperm &P, char dir_=1, char lex_=1)
1251         : data(A.data), n(A.dim1), lda(A.dim2),
1252         asc(dir_> 0 ? 1 : 0), lex(lex_> 0 ? 1 : 0)
1253     {
1254         P.init(n);
1255     };
1256
1257     ~wbRecs() {};
1258
1259     bool operator()(const size_t &pa, const size_t &pb) const {
1260
1261         if (pa>n || pb>n) wblog(FL,
1262             "ERR %s() index out of bounds (%d,%d; %d)",FCT,pa,pb,n);
1263
1264         const T *a=data+pa*lda, *b=data+pb*lda;
1265         if (lex) { size_t i=0;
1266             if (asc) {
1267                 for (; i<lda; ++i) {
1268                     if (a[i]<b[i]) return 1;
1269                     if (a[i]>b[i]) return 0;
1270                 }
1271             }
1272             else {
1273                 for (; i<lda; ++i) {
1274                     if (a[i]<b[i]) return 0;
1275                     if (a[i]>b[i]) return 1;
1276                 }
1277             }
1278         }
1279         else { size_t i=lda-1;
1280             if (asc) {
1281                 for (; i<lda; --i) {
1282                     if (a[i]<b[i]) return 1;
1283                     if (a[i]>b[i]) return 0;
1284                 }
1285             }
1286             else {
1287                 for (; i<lda; --i) {
1288                     if (a[i]<b[i]) return 0;
1289                     if (a[i]>b[i]) return 1;
1290                 }
1291             }
1292         }
1293     }

```

```

1293         if (asc) return (pa<pb);
1294         else return (pb<pa);
1295     };
1296 };
1297
1298 #ifdef QS_USING_OMP
1299
1300 template <class T>
1301 void getSortPerm_OMP(
1302     const wbMatrix<T> &A, wbperm &P, char dir, char lex
1303 ){
1304     if (!A.dim2) { P.init(); return; }
1305     wbRecs<T> R(A,P,dir,lex);
1306
1307     #ifdef WB_SPARSE_CLOCK
1308     Wb::UseClock gr3(&wbMat_group3);
1309     #endif
1310
1311     if (A.dim1<128) {
1312         sort(P.data, P.data+A.dim1, R);
1313         return;
1314     }
1315
1316     int id=0, i,l, im=0, mp=MIN(
1317         1 « unsigned(floor(log2(double(A.dim1)))-6),
1318         omp_get_max_threads()
1319     );
1320
1321     double nsub=double(A.dim1)/mp;
1322
1323     wbindex idx(mp+1); wbperm PX(P.len);
1324     for (i=0; i<mp; ++i) { idx[i]=size_t(i*nsub+0.5); }
1325     idx[i]=A.dim1;
1326
1327     #pragma omp parallel for
1328     for (i=0; i<mp; ++i) {
1329         size_t i1=idx.data[i], i2=idx.data[i+1];
1330         sort(P.data+i1, P.data+i2, R);
1331         id=MAX(id,omp_get_thread_num());
1332     }
1333
1334     while (idx.len>2) { int m2=mp/2; P.swap(PX); ++im;
1335
1336     #pragma omp parallel for
1337     for (i=0; i<m2; ++i) {
1338         size_t k=2*i, i1=idx.data[k], i2=idx.data[k+1], i3=idx.data[k+2];
1339         merge(
1340             PX.data+i1, PX.data+i2,
1341             PX.data+i2, PX.data+i3, P.data+i1, R
1342         );
1343         id=MAX(id,omp_get_thread_num());
1344     }
1345
1346     if (mp%2) {
1347         size_t i1=idx.data[mp-3], i2=idx.data[mp-1], i3=idx.data[mp];
1348         memcpy(PX.data+i1,P.data+i1,(i2-i1)*sizeof(size_t));
1349         merge(
1350             PX.data+i1, PX.data+i2,
1351             PX.data+i2, PX.data+i3, P.data+i1, R
1352         );
1353         idx[mp-1]=idx[mp]; idx.len=(mp--);
1354     }
1355
1356     for (l=1, i=2; i<mp; i+=2, ++l) { idx[l]=idx[i]; }
1357     idx[l]=idx[mp]; mp=l; idx.len=l+1;
1358 }
1359
1360 };
1361 };
1362
1363 #endif
1364
1365 template <class T>
1366 bool wbMatrix<T>::deepEqual(const wbMatrix<T> &B) const {
1367     if (dim1!=B.dim1 || dim2!=B.dim2) return 0;
1368
1369     if (data!=B.data) {
1370         size_t i=0, n=numel();
1371         for (; i<n; ++i) { if (data[i]!=B.data[i]) continue;
1372             if (!data[i] || !B.data[i]) return 0; else
1373             if (!(*data[i])==(*B.data[i])) return 0;
1374         }
1375     }
1376
1377     return 1;
1378 };
1379

```

```

1380 template <class T> inline
1381 bool wbMatrix<T>::isDiag() const {
1382     size_t i, j;
1383
1384     for (i=0; i<dim1; ++i)
1385         for (j=0; j<dim2; ++j) if (i!=j) if (data[i*dim2+j]!=0.) return 0;
1386
1387     return 1;
1388 }
1389
1390 template <class T>
1391 bool wbMatrix<T>::isIdentity(const double &eps, double &maxdiff) const {
1392
1393     if (dim1!=dim2) return 0;
1394
1395     size_t i, j, l=0; double d; maxdiff=0;
1396
1397     for (j=0; j<dim2; ++j)
1398         for (i=0; i<dim1; ++i, ++l) {
1399             d = ABS( data[l] - (i!=j ? 0 : 1) );
1400             if (maxdiff<d) maxdiff=d;
1401         }
1402
1403     return (maxdiff<eps);
1404 }
1405
1406 template <class T> inline
1407 bool wbMatrix<T>::isSym_aux(
1408     const wbMatrix<T> &B, double eps, double *xref,
1409     const char symflag
1410 ) const {
1411
1412     const wbMatrix<T> &A=(*this);
1413     char issame = (&A==&B);
1414     size_t i, j, k;
1415     double x;
1416
1417     if (A.dim1!=B.dim2 || A.dim2!=B.dim1) return 0;
1418     if (xref) *xref=0;
1419
1420     if (symflag=='s') {
1421         if (eps==0) {
1422             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
1423                 for (j=k; j<dim2; ++j) {
1424                     if (A(i,j)!=CONJ(B(j,i))) {
1425                         if (xref) {
1426                             x = ABS(A(i,j) - CONJ(B(j,i)));
1427                             xref[0] = MAX(*xref,x);
1428                         }
1429                         else return 0;
1430                     }
1431                 }
1432             }
1433         }
1434         else {
1435             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
1436                 for (j=k; j<dim2; ++j) {
1437                     x = ABS(A(i,j) - CONJ(B(j,i)));
1438                     if (x>eps) { if (xref) xref[0]=MAX(*xref,x); else return 0; }
1439                 }
1440             }
1441         }
1442     }
1443     else if (symflag=='a') {
1444         if (eps==0) {
1445             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
1446                 for (j=k; j<dim2; ++j) {
1447                     if (A(i,j)!=-CONJ(B(j,i))) {
1448                         if (xref) {
1449                             x = ABS(A(i,j) + CONJ(B(j,i)));
1450                             xref[0] = MAX(*xref,x);
1451                         }
1452                         else return 0;
1453                     }
1454                 }
1455             }
1456         }
1457         else {
1458             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
1459                 for (j=k; j<dim2; ++j) {
1460                     x = ABS(A(i,j) + CONJ(B(j,i)));
1461                     if (x>eps) { if (xref) xref[0]=MAX(*xref,x); else return 0; }
1462                 }
1463             }
1464         }
1465     }
1466     else wblog(FL,"ERR invalid symflag=%c<%d>",symflag,symflag);
1467
1468     if (xref && (*xref)>eps) return 0;
1469
1470     return 1;

```

```

1467 }
1468
1469 template <class T> inline
1470 bool wbMatrix<T>::isComplex() const { return 0; };
1471
1472 template<> inline
1473 bool wbMatrix<wbcomplex>::isComplex() const {
1474     for (size_t s=dim1*dim2, i=0; i<s; ++i)
1475         if (data[i].i!=0.) return 1;
1476     return 0;
1477 }
1478
1479 template <class T> inline
1480 bool wbMatrix<T>::isUnique() const {
1481     wbMatrix<T> Q(*this);
1482     wbperm P;
1483     if (dim2==0) return (dim1<=1);
1484     Q.SortRecs(P);
1485     return Q.isUniqueSorted();
1486 };
1487
1488 template <class T> inline
1489 bool wbMatrix<T>::isUniqueSorted(char dir, char lex) const {
1490     if (dim1<=1 || !dim2) return 1;
1491     size_t i=1; char c=0;
1492     if (!dir) { if (dim1<=2) return 1;
1493         c=recCompare(1,0,-1,lex); if (!c) { return 0; }
1494         i=2;
1495     }
1496     else if (abs(int(dir))>1) {
1497         if (dir=='A') c+=1; else
1498         if (dir=='D') c-=1; else
1499         wblog(FL,"ERR %s() invalid c='%c'<%d> !?",FCT,c,c);
1500     }
1501     else { c=dir; }
1502     for (; i<dim1; ++i) {
1503         if (recCompare(i,i-1,-1,lex)!=c) return 0;
1504     }
1505     return 1;
1506 };
1507
1508 template <class T> inline
1509 int wbMatrix<T>::isSorted(char dir, char lex, size_t m) const {
1510     if (!dim1 || !dim2) return -1;
1511     if (dim1==1) return 3;
1512     size_t j=1, ndeg=0; char c=0;
1513     if (!dir) {
1514         for (; j<dim1; ++j, ++ndeg) {
1515             if ((c=recCompare(j,j-1,m,lex))) { dir=c; ++j; break; }
1516         }
1517     }
1518     else if (abs(int(dir))>1) {
1519         wblog(FL,"ERR %s() invalid direction s=%d",FCT,dir);
1520     }
1521     for (; j<dim1; ++j) {
1522         if ((c=recCompare(j,j-1,m,lex))) { if (c!=dir) return 0; }
1523         else { ++ndeg; }
1524     }
1525     return (1 | (ndeg ? 0 : 2) | (c<0 ? 4 : 0));
1526 };
1527
1528 template <class T> inline
1529 wbMatrix<T>& wbMatrix<T>::ResizeXRecs(size_t nr) {
1530     ref_check(FLF);
1531     if (nr) {
1532         if (nr<=dim1 && nr/double(dim1)>0.10) dim1=nr;
1533         else Resize(nr,dim2);
1534     }
1535     else init(0,dim2);
1536     return *this;
1537 };
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553

```



```

1554 template <class T> inline
1555 wbMatrix<T>& wbMatrix<T>::resize(
1556     size_t m, size_t n, wbMatrix<T> &M
1557 ) const {
1558
1559     if (this==&M) return M.Resize(m,n);
1560
1561     M.init(m,n);
1562     m=MIN(m,dim1); n=MIN(n,dim2); if (m==0 || n==0) return M;
1563
1564     for (size_t i=0; i<m; ++i)
1565         MEM_COPY<T>(M.data+i*M.dim2, n, data+i*dim2);
1566
1567     return M;
1568 };
1569
1570 template <class T> inline
1571 wbMatrix<T>& wbMatrix<T>::Resize(size_t nr, size_t nc, const T* dx) {
1572
1573     size_t d1,d2,s=nr*nc; T *d0=data;
1574     ref_check(FLF);
1575
1576     d1=MIN(dim1,nr); d2=MIN(dim2,nc);
1577
1578     if (dim1==nr && dim2==nc) return *this;
1579     if (d1==0 || d2==0) { RENEW(nr,nc); return *this; }
1580
1581     WB_NEW(data,s);
1582
1583     if (nc==dim2) {
1584         MEM_COPY<T>(data,d1*dim2,d0);
1585         if (nr>dim1) {
1586             if (dx)
1587                 MEM_COPY<T>(data+dim1*dim2, (nr-dim1)*dim2,dx);
1588             else MEM_SET<T>(data+dim1*dim2, (nr-dim1)*dim2);
1589         }
1590     }
1591     else {
1592         size_t i; T *p=data, *p0=d0;
1593         for (i=0; i<d1; ++i, p+=nc, p0+=dim2) {
1594             MEM_COPY<T>(p,d2,p0); if (d2<nc) {
1595                 MEM_SET<T>(p+d2,nc-d2); }
1596         }
1597         if (nr>dim1) {
1598             if (dx) wblog(FL,"ERR %s() "
1599                 "allows no reference data if dim2=%d->%d",FCT,dim2,nc);
1600             MEM_SET<T>(data+dim1*dim2, (nr-dim1)*dim2);
1601         }
1602     }
1603
1604     dim1=nr; dim2=nc;
1605     WB_DELETE(d0);
1606
1607     return *this;
1608 };
1609
1610 template <class T> inline
1611 wbMatrix<T>& wbMatrix<T>::Resize2Mult(const WBINDEX &M) {
1612
1613     size_t m=M.sum();
1614
1615     if (M.len!=dim1) wblog(FL,
1616         "ERR %s() size mismatch (%d/%dx%d)",FCT,M.len,dim1,dim2);
1617     if (!m || !dim2) { return init(m,dim2); } else
1618     if (int(m)<0) wblog(FL,"ERR %s() got m=%d !?",FCT,m);
1619
1620     size_t i,j;
1621     wbMatrix<T> X(m,dim2);
1622     const T *x0=data; T *x=X.data;
1623
1624     for (i=0; i<M.len; ++i, x0+=dim2) { m=M.data[i];
1625         for (j=0; j<m; ++j, x+=dim2) MEM_COPY<T>(x, dim2, x0); }
1626
1627     return X.save2(*this);
1628 };
1629
1630 template <class T> inline
1631 wbMatrix<T>& wbMatrix<T>::unRef() {
1632
1633     if (!isref) return *this;
1634     if (!dim1 || !dim2) {
1635         if (data) wblog(FL,"ERR data=%1X (%dx%d) !?",data,dim1,dim2);
1636         isref=0; return *this;
1637     }
1638
1639     size_t s=dim1*dim2;
1640     T const* const d0=data;

```

```

1641
1642     WB_NEW(data,s);
1643
1644     MEM_COPY<T>(data,s,d0);
1645     isref=0;
1646
1647     return *this;
1648 };
1649
1650 template <class T> inline
1651 wbMatrix<T>& wbMatrix<T>::transpose(wbMatrix<T> &M) const {
1652
1653     if (this==&M) return M.Transpose();
1654
1655     size_t i,j,k=0; M.init(dim2,dim1);
1656
1657     if (isdiag) {
1658         M.isdiag=1; k=MIN(dim1,dim2);
1659         for (i=0; i<k; ++i) M.data[i*dim1+i]=data[i*dim2+i];
1660         return M;
1661     }
1662
1663     for (j=0; j<dim2; ++j)
1664         for (i=0; i<dim1; ++i) M.data[k++]=data[i*dim2+j];
1665
1666     return M;
1667 };
1668
1669 template <class T> inline
1670 wbMatrix<T>& wbMatrix<T>::Reshape(size_t nr, size_t nc) {
1671
1672     if (dim1!=nr || dim2!=nc) {
1673         if (dim1*dim2!=nr*nc) wblog(FL,
1674             "ERR %s() size not preserved (%dx%d => %dx%d)",
1675             FCT,dim1,dim2,nr,nc
1676         );
1677         dim1=nr; dim2=nc;
1678     }
1679     return *this;
1680 };
1681
1682 template <class T> inline
1683 wbMatrix<T>& wbMatrix<T>::add(
1684     const wbMatrix<T> &M,
1685     const T& c,
1686     const char zflag
1687 ){
1688     size_t i,s=dim1*dim2;
1689
1690     if (data==NULL && zflag) {
1691         RENEW(M.dim1, M.dim2, M.data); (*this)*=c;
1692         isdiag=M.isdiag;
1693         return *this;
1694     }
1695
1696     if (dim1!=M.dim1 || dim2!=M.dim2)
1697         wberror(FL,"Dimension mismatch.");
1698
1699     if (isdiag) if (!M.isdiag) isdiag=0;
1700
1701     if (c==T(+1)) { for (i=0; i<s; ++i) data[i]+=M.data[i]; } else
1702     if (c==T( 0)) { return *this; } else
1703     if (c==T(-1)) { for (i=0; i<s; ++i) data[i]-=M.data[i]; }
1704     else          { for (i=0; i<s; ++i) data[i]+=(c * M.data[i]); }
1705
1706     return *this;
1707 }
1708
1709 template <class T> inline
1710 wbMatrix<T>& wbMatrix<T>::minus(
1711     const wbMatrix<T> &M,
1712     const T& c,
1713     const char zflag
1714 ){
1715     size_t i,s=dim1*dim2;
1716
1717     if (data==NULL && zflag) {
1718         RENEW(M.dim1, M.dim2, M.data); (*this)*=(-c);
1719         isdiag=M.isdiag;
1720         return *this;
1721     }
1722
1723     if (dim1!=M.dim1 || dim2!=M.dim2)
1724         wberror(FL,"Dimension mismatch.");
1725
1726     if (isdiag) if (!M.isdiag) isdiag=0;
1727

```

```

1728     if (c==T(+1)) { for (i=0; i<s; ++i) data[i]-=M.data[i]; } else
1729     if (c==T( 0)) { return *this; } else
1730     if (c==T(-1)) { for (i=0; i<s; ++i) data[i]+=M.data[i]; }
1731     else          { for (i=0; i<s; ++i) data[i]-(c * M.data[i]); }
1732
1733     return *this;
1734 }
1735
1736 template <class T> inline
1737 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
1738     const wbMatrix<T> &M2
1739 ){
1740     const wbMatrix<T>* M0[] = {this, &M2};
1741     wbvector< wbMatrix<T> > const* > M; M.init(2,M0);
1742     return CAT(dim,M);
1743 };
1744
1745 template <class T> inline
1746 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
1747     const wbMatrix<T> &M2, const wbMatrix<T> &M3
1748 ){
1749     const wbMatrix<T>* M0[] = {this, &M2, &M3};
1750     wbvector< wbMatrix<T> > const* > M; M.init(3,M0);
1751     return CAT(dim,M);
1752 };
1753
1754 template <class T> inline
1755 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
1756     const wbMatrix<T> &M2, const wbMatrix<T> &M3, const wbMatrix<T> &M4
1757 ){
1758     const wbMatrix<T>* M0[] = {this, &M2, &M3, &M4};
1759     wbvector< wbMatrix<T> > const* > M; M.init(4,M0);
1760     return CAT(dim,M);
1761 };
1762
1763 template <class T> inline
1764 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
1765     const wbMatrix<T> &M1, const wbMatrix<T> &M2
1766 ){
1767     wbMatrix<T> const* M0[] = {&M1, &M2};
1768     wbvector< wbMatrix<T> > const* > M; M.init(2,M0);
1769     return CAT(dim,M);
1770 };
1771
1772 template <class T> inline
1773 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
1774     const wbMatrix<T> &M1, const wbMatrix<T> &M2,
1775     const wbMatrix<T> &M3
1776 ){
1777     wbMatrix<T> const* M0[] = {&M1, &M2, &M3};
1778     wbvector< wbMatrix<T> > const* > M; M.init(3,M0);
1779     return CAT(dim,M);
1780 };
1781
1782 template <class T> inline
1783 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
1784     const wbMatrix<T> &M1, const wbMatrix<T> &M2,
1785     const wbMatrix<T> &M3, const wbMatrix<T> &M4
1786 ){
1787     wbMatrix<T> const* M0[] = {&M1, &M2, &M3, &M4};
1788     wbvector< wbMatrix<T> > const* > M; M.init(4,M0);
1789     return CAT(dim,M);
1790 };
1791
1792 template <class T> inline
1793 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
1794     wbMatrix<T> const* M[], const unsigned len
1795 ){
1796     wbvector< wbMatrix<T> > const* > MM;
1797     MM.init(len,M);
1798
1799     return CAT(dim,MM);
1800 };
1801
1802 template <class T> inline
1803 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
1804     const wbvector< wbMatrix<T> > &M0
1805 ){
1806     wbvector< wbMatrix<T> > const* > M(M0.len);
1807     for (unsigned i=0; i<M0.len; ++i) M[i]=&M0[i];
1808
1809     return CAT(dim,M);
1810 };
1811
1812 template <class T> inline
1813 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
1814     const wbvector< wbvector<T> > &V0

```

```

1815 ){
1816     wbvector< wbMatrix<T> > M0(V0.len);
1817     wbvector< wbMatrix<T> const* > M(V0.len);
1818
1819     for (size_t i=0; i<V0.len; ++i) {
1820         if (dim==1)
1821             M0[i].init2ref(1,V0[i].len,V0[i].data);
1822         else M0[i].init2ref(V0[i].len,1,V0[i].data);
1823
1824         M[i]=&M0[i];
1825     }
1826
1827     return CAT(dim,M);
1828 };
1829
1830 template <class T> inline
1831 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
1832     const wbvector< wbvector<T> const* > &V0
1833 ){
1834     wbvector< wbMatrix<T> > M0(V0.len);
1835     wbvector< wbMatrix<T> const* > M(V0.len);
1836
1837     for (size_t i=0; i<V0.len; ++i) {
1838         if (dim==1)
1839             M0[i].init2ref(1,V0[i]->len,V0[i]->data);
1840         else M0[i].init2ref(V0[i]->len,1,V0[i]->data);
1841
1842         M[i]=&M0[i];
1843     }
1844
1845     return CAT(dim,M);
1846 };
1847
1848 template <class T>
1849 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
1850     wbvector< wbMatrix<T> const* > M
1851 ){
1852     wbMatrix<T> X;
1853     size_t i,k;
1854
1855     if (isdiag) { wblog(FL,"WRN unsetting isdiag"); isdiag=0; }
1856
1857     if (M.len==0) { this->init(); return *this; }
1858
1859     for (k=i=0; i<M.len; ++i) {
1860         if (M[i]==this) {
1861             if (!(k++)) { this->save2(X); }
1862             M[i]=&X;
1863         }
1864     }
1865
1866     for (k=i=0; i<M.len; ++i) {
1867         if (M[i] && (M[i]->dim1 || M[i]->dim2)) {
1868             if (k<i) { M[k]=M[i]; }; ++k;
1869             if (M[i]->isdiag) wblog(FL,
1870                 "ERR %s() got diagonal matrix (i=%d)",FCT, i+1
1871             );
1872         }
1873     }
1874     if (k<M.len) {
1875         if (k) M.len=k;
1876         else { this->init(); return *this; }
1877     }
1878
1879     if (dim==1) {
1880         size_t m,d1,D1,d2=0;
1881
1882         for (D1=k=0; k<M.len; ++k) { D1+=(M[k]->dim1);
1883             if (k==0) d2=(M[k]->dim2); else
1884                 if (d2!=(M[k]->dim2)) wblog(FL,
1885                     "ERR dimension mismatch (%d/%d)",d2,M[k]->dim2
1886                 );
1887         }
1888
1889         this->init(D1,d2);
1890
1891         for (m=k=0; k<M.len; ++k) { d1=(M[k]->dim1);
1892             MEM_COPY<T>(data+d2*m, d1*d2, M[k]->data);
1893             m+=d1;
1894         }
1895     }
1896     else if (dim==2) {
1897         size_t i,m,d1=0,d2,D2;
1898         T *p, *p2;
1899
1900         for (D2=k=0; k<M.len; ++k) { D2+=(M[k]->dim2);
1901             if (k==0) d1=(M[k]->dim1); else

```

```

1902         if (d1!=(M[k]->dim1)) wblog(FL,
1903             "ERR dimension mismatch (%d/%d)",d1,M[k]->dim1
1904         );
1905     }
1906
1907     this->init(d1,D2);
1908
1909     for (m=k=0; k<M.len; ++k) { d2=(M[k]->dim2);
1910         p=data+m; p2=(M[k]->data);
1911
1912         for (i=0; i<d1; ++i)
1913             MEM_COPY<T>(p+D2*i, d2, p2+d2*i);
1914
1915         m+=d2;
1916     }
1917 }
1918 else wblog(FL,"ERR invalid cat dim=%d",dim);
1919
1920 return *this;
1921 };
1922
1923 template <class T>
1924 wbMatrix<T>& wbMatrix<T>::CAT(
1925     const unsigned dim,
1926     wbvector< wbMatrix<T> const* > M,
1927     wbvector< WBINDEX const* > I
1928 ){
1929     wbMatrix<T> X;
1930     size_t i,k;
1931
1932     if (M.len!=I.len) wblog(FL,
1933         "ERR %s() size mismatch %d/%d",FCT,M.len,I.len);
1934     if (!M.len) { this->init(); return *this; }
1935
1936     for (k=i=0; i<M.len; ++i) {
1937         if (M[i]==this) {
1938             if (!(k++)) { this->save2(X); }
1939             M[i]=&X;
1940         }
1941     }
1942
1943     for (k=i=0; i<M.len; ++i) {
1944         if (M[i] && (M[i]->dim1 || M[i]->dim2 || I[i]->len)) {
1945             if (k<i) { M[k]=M[i]; I[k]=I[i]; }; ++k;
1946             if (M[i]->isdiag) wblog(FL,
1947                 "ERR %s() got diagonal matrix (i=%d)",FCT, i+1
1948             );
1949         }
1950     }
1951     if (k<M.len) {
1952         if (k) { M.len=k; I.len=k; }
1953         else { this->init(); return *this; }
1954     }
1955
1956     if (isdiag) isdiag=0;
1957
1958     if (dim==1) {
1959         size_t d1,d2=0, D1=0;
1960
1961         for (k=0; k<M.len; ++k) { D1+=(I[k]->len);
1962             if (k==0) d2=(M[k]->dim2); else
1963             if (d2!=(M[k]->dim2)) wblog(FL,
1964                 "ERR dimension mismatch (vcat: dim2=%d/%d)",d2,M[k]->dim2
1965             );
1966         }
1967
1968         this->init(D1,d2);
1969         T *x=data;
1970
1971         for (k=0; k<M.len; ++k) {
1972             const T* x0=M[k]->data;
1973             const wid_x_t *Ik=I[k]->data; D1=M[k]->dim1; d1=I[k]->len;
1974
1975             for (i=0; i<d1; ++i, x+=d2) {
1976                 if (Ik[i]>=D1) wblog(FL,
1977                     "ERR %s() index out of bounds (%d/%d)",FCT,Ik[i]+1,D1);
1978                 MEM_COPY<T>(x, d2, x0+Ik[i]*d2);
1979             }
1980         }
1981     }
1982     else if (dim==2) {
1983         size_t i,d1=0,d2,D1,D2=0;
1984
1985         for (k=0; k<M.len; ++k) { D2+=(d2=M[k]->dim2);
1986             if (k==0) d1=(I[k]->len); else
1987             if (d1!=(I[k]->len)) wblog(FL,
1988                 "ERR dimension mismatch (%d/%d)",d1,I[k]->len

```

```

1989         );
1990     }
1991
1992     this->init(d1,D2);
1993     T *x=data;
1994
1995     for (k=0; k<M.len; ++k) {
1996         const T* x0=M[k]->data;
1997         const widx_t *Ik=I[k]->data; D1=M[k]->dim1; d2=(M[k]->dim2);
1998
1999         for (i=0; i<d1; ++i) {
2000             if (Ik[i]>=D1) wblog(FL,
2001                 "ERR %s() index out of bounds (%d/%d)",FCT,Ik[i]+1,D1);
2002             MEM_COPY<T>(x+i*D2, d2, x0+Ik[i]*d2);
2003             ++x+=d2;
2004         }
2005     }
2006     else wblog(FL,"ERR invalid cat dim=%d",dim);
2007
2008     return *this;
2009 };
2010
2011 template <class T> inline
2012 void wbMatrix<T>::split(
2013     const unsigned dim,
2014     wvector< wbMatrix<T>* > &M) const {
2015
2016     split(dim,M.data,M.len);
2017     return;
2018 }
2019
2020 template <class T> inline
2021 void wbMatrix<T>::split(
2022     const unsigned dim,
2023     wbMatrix<T> &M1,
2024     wbMatrix<T> &M2,
2025     wbMatrix<T> &M3
2026 ) const {
2027
2028     wbMatrix<T>* M[] = {&M1,&M2,&M3};
2029     split(dim,M,3);
2030     return;
2031 }
2032
2033 template <class T>
2034 void wbMatrix<T>::split(
2035     const unsigned dim, wbMatrix<T>** M, const unsigned len
2036 )const {
2037
2038     unsigned k;
2039
2040     for (k=0; k<len; ++k) {
2041         if (M[k]==this)
2042             wberror(FL, "This matrix itself is included in split list!");
2043     }
2044
2045     if (dim==1) {
2046         size_t m,d1,D1;
2047
2048         for (D1=k=0; k<len; ++k) D1+=(*M[k]).dim1;
2049
2050         if (D1!=dim1)
2051             wberror(FL, "Dimension mismatch.");
2052
2053         for (k=0; k<len; ++k) {
2054             if (M[k]->dim2!=dim2) M[k]->init(M[k]->dim1, dim2);
2055         }
2056
2057         for (m=k=0; k<len; ++k) { d1=M[k]->dim1;
2058             MEM_COPY<T>(M[k]->data, d1*dim2, data+dim2*m);
2059             m+=d1;
2060         }
2061     }
2062     else if (dim==2) {
2063         size_t i,m,d2,D2;
2064         T *p, *p2;
2065
2066         for (D2=k=0; k<len; ++k) D2+=M[k]->dim2;
2067
2068         if (D2!=dim2)
2069             wblog(FL,"ERR dimension mismatch (%d/%d)",D2,dim2);
2070
2071         for (k=0; k<len; ++k) {
2072             if (M[k]->dim1!=dim1) M[k]->init(dim1,M[k]->dim2);
2073         }
2074
2075         for (m=k=0; k<len; ++k) {

```

```

2076         d2=M[k]->dim2; p=data+m; p2=M[k]->data;
2077
2078         for (i=0; i<dim1; ++i)
2079             MEM_COPY<T>(p2+d2*i, d2, p+D2*i);
2080
2081         m+=d2;
2082     }
2083 }
2084 else wblog(FL,"ERR invalid cat index (%d).", dim);
2085 };
2086
2087 template <class T> inline
2088 int wbMatrix<T>::recLess(size_t i1, size_t i2, char lex) const {
2089     size_t i;
2090     T *r1=data+i1*dim2, *r2=data+i2*dim2;
2091
2092     if (i1>=dim1 || i2>=dim1) wblog(FL,
2093         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
2094
2095     if (lex>0) {
2096         for (i=0; i<dim2; ++i)
2097             if (r1[i]<r2[i]) return 1; else
2098                 if (r1[i]>r2[i]) return 0;
2099     }
2100     else {
2101         for (i=dim2-1; i<dim2; --i)
2102             if (r1[i]<r2[i]) return 1; else
2103                 if (r1[i]>r2[i]) return 0;
2104     }
2105     return 0;
2106 }
2107
2108 }
2109
2110 template <class T> inline
2111 int wbMatrix<T>::recLessE(size_t i1, size_t i2, char lex) const {
2112     size_t i;
2113     T *r1=data+i1*dim2, *r2=data+i2*dim2;
2114
2115     if (i1>=dim1 || i2>=dim1) wblog(FL,
2116         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
2117
2118     if (lex) {
2119         for (i=0; i<dim2; ++i)
2120             if (r1[i]<r2[i]) return 1; else
2121                 if (r1[i]>r2[i]) return 0;
2122     }
2123     else {
2124         for (i=dim2-1; i<dim2; --i)
2125             if (r1[i]<r2[i]) return 1; else
2126                 if (r1[i]>r2[i]) return 0;
2127     }
2128     return 1;
2129 }
2130
2131 }
2132
2133 template <class T> inline
2134 int wbMatrix<T>::recEqual(size_t i1, const T *d) const {
2135     if (i1>=dim1) wblog(FL,
2136         "ERR %s() index out of bounds (%d/%d)",FCT,i1,dim1);
2137     if (data==d) return 1;
2138     else return (!memcmp(data+i1*dim2, d, dim2*sizeof(T)));
2139 };
2140
2141 template <class T> inline
2142 int wbMatrix<T>::recEqual(size_t i1, size_t i2) const {
2143     if (i1>=dim1 || i2>=dim1) wblog(FL,
2144         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
2145     if (i1==i2) return 1;
2146     else return (!memcmp(data+i1*dim2, data+i2*dim2, dim2*sizeof(T)));
2147 };
2148
2149 template <class T> inline
2150 T wbMatrix<T>::recDiff2(size_t i1, size_t i2, size_t n) const {
2151     if (i1>=dim1 || i2>=dim1) wblog(FL,
2152         "ERR %s() index out of bounds (%d,%d/%d)",i1,i2,dim1);
2153     if (i1==i2) return 0;
2154     return Wb::rangeNormDiff2(
2155         data+i1*dim2, data+i2*dim2, int(n)<0 ? dim2 : n
2156     );
2157 };
2158
2159 template <class T> inline
2160 T wbMatrix<T>::recDiff2P(size_t i1, const T *b, size_t n) const {
2161     if (i1>=dim1) wblog(FL,
2162         "ERR %s() index out of bounds (%d/%d)",i1,dim1);

```

```

2163     return Wb::rangeNormDiff2(
2164         data+i1*dim2, b, int(n)<0 ? dim2 : n
2165     );
2166 };
2167
2168 template <class T> inline
2169 char wbMatrix<T>::recCompare(
2170     size_t i1, size_t i2,
2171     size_t m,
2172     char lex,
2173     T eps
2174 ) const {
2175
2176     if (i1>=dim1 || i2>=dim1) wblog(FL,
2177         "ERR %s() index out of bounds (%d,%d)",FCT,i1,i2,dim1);
2178     if (!dim2) wblog(FL,
2179         "ERR %s() got nothing to compare (dim2=%d)",FCT,dim2);
2180
2181     if (int(m)<0) m=dim2; else
2182     if (!m || m>dim2) wblog(FL,"ERR size out of bounds (%d/%d)",m,dim2);
2183
2184     i1*=dim2; i2*=dim2;
2185     if (lex<=0) {
2186         i1+=(dim2-m);
2187         i2+=(dim2-m);
2188     }
2189
2190     if (eps<=T(0))
2191         return Wb::recCompare(data+i1, data+i2, m, lex);
2192     else return Wb::recCompare(data+i1, data+i2, m, lex, eps);
2193 };
2194
2195 template <class T> inline
2196 char wbMatrix<T>::recCompare(
2197     size_t i1, const wbvector<T> &v, char lex
2198 ) const {
2199
2200     if (v.len!=dim2) wblog(FL,
2201         "ERR %s() size mismatch (%d,%d)",FCT,v.len,dim2);
2202     return Wb::recCompare(data+i1*dim2, v.data, dim2, lex);
2203 };
2204
2205 template <class T> inline
2206 char wbMatrix<T>::recCompareP(
2207     size_t i1, const T* v,
2208     size_t m,
2209     char lex,
2210     T eps
2211 ) const {
2212
2213     if (i1>=dim1) wblog(FL,
2214         "ERR %s() index out of bounds (%d,%d)",FCT,i1,dim1);
2215     if (!dim2) wblog(FL,
2216         "ERR %s() got nothing to compare (dim2=%d)",FCT,dim2);
2217
2218     if (int(m)<0) m=dim2; else
2219     if (!m || m>dim2) wblog(FL,"ERR size out of bounds (%d/%d)",m,dim2);
2220
2221     i1*=dim2;
2222     if (lex<=0) i1+=(dim2-m);
2223
2224     return Wb::recCompare(data+i1, v, m, lex, eps);
2225 };
2226
2227 template <class T> inline
2228 wbvector<T>& wbMatrix<T>::recProd(wbvector<T> &pp) const {
2229     size_t i=0, j; T *d=data, p;
2230
2231     pp.init(dim1);
2232
2233     if (!dim2) wblog(FL,"ERR %s() records of length 0",FCT);
2234
2235     for (; i<dim1; ++i, d+=dim2) {
2236         for (p=d[0], j=1; j<dim2; ++j) { p*=d[j]; }; pp[i]=p; }
2237
2238     return pp;
2239 };
2240
2241 template <class T> inline
2242 T wbMatrix<T>::recProd(size_t i) const {
2243
2244     if (i>=dim1 || dim2==0) { wblog(FL,
2245         "ERR %s() got empty record length (%dx%d; i)",FCT,dim1,dim2,i);
2246         return 0;
2247     }
2248
2249     const T *d=data+i*dim2; T p=d[0];

```



```

2250     for (size_t j=1; j<dim2; ++j) { p*=d[j]; }
2251     return p;
2252 };
2253
2254 template <class T>
2255 inline T wbMatrix<T>::colSum(size_t c) const {
2256     T *d=data+c, x=0;
2257
2258     if (c>=dim2) wblog(FL,"ERR index out of bounds (%d/%d)", c, dim2);
2259     if (dim1==0) wblog(FL,
2260         "WRN Sum over column of length zero (%d/%d).", c, dim2);
2261
2262     for (size_t i=0; i<dim1; ++i, d+=dim2) x+=(*d);
2263     return x;
2264 }
2265
2266 template <class T> inline
2267 T wbMatrix<T>::recSum(size_t i) const {
2268
2269     if (i>=dim1) wblog(FL,
2270         "ERR %s() index out of bounds (%d,%dx%d)",FCT,i+1,dim1,dim2);
2271     if (!dim2) wblog(FL,
2272         "ERR %s() got empty object (%d,%dx%d)",i,dim1,dim2);
2273
2274     return Wb::addRange(rec(i),dim2);
2275 };
2276
2277 template <class T> inline
2278 wbvector<T>& wbMatrix<T>::recSum(wbvector<T> &s) const {
2279
2280     s.init(dim1);
2281     if (!dim2) { if (dim1) wblog(FL,
2282         "WRN %s() got empty object (%dx%d)",FCT,dim1,dim2);
2283         return s;
2284     }
2285
2286     size_t i=0; const T* d=data;
2287     for (; i<dim1; ++i, d+=dim2) { s[i]=Wb::addRange(d,dim2); }
2288
2289     return s;
2290 };
2291
2292 template <class T> inline
2293 wbvector<T>& wbMatrix<T>::recSumA(wbvector<T> &s) const {
2294
2295     s.init(dim1);
2296     if (!dim2) { if (dim1) wblog(FL,
2297         "WRN %s() got empty object (%dx%d)",FCT,dim1,dim2);
2298         return s;
2299     }
2300
2301     size_t i=0; const T* d=data; double q;
2302     for (; i<dim1; ++i, d+=dim2) {
2303         q=Wb::addRange2(d,dim2);
2304         s[i]=std::sqrt(q);
2305     }
2306
2307     return s;
2308 };
2309
2310 template <class T>
2311 wbMatrix<T>& wbMatrix<T>::setRand(double fac, double shift) {
2312     size_t i,s=dim1*dim2;
2313     static char first_call=1;
2314
2315     if (first_call) { wb_srand(); first_call=0; }
2316     fac/=(double)RAND_MAX; isdiag=0;
2317
2318     if (shift==0.)
2319         for (i=0; i<s; ++i) data[i]=(T)(fac*rand());
2320     else for (i=0; i<s; ++i) data[i]=(T)(fac*rand()+shift);
2321
2322     return *this;
2323 }
2324
2325 template<>
2326 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::setRand(double fac, double shift) {
2327     size_t i,s=dim1*dim2;
2328     static char first_call=1;
2329
2330     if (first_call) { wb_srand(); first_call=0; }
2331     fac/=(double)RAND_MAX; isdiag=0;
2332
2333     if (shift==0.) {
2334         for (i=0; i<s; ++i)
2335             data[i].set(fac*rand(), fac*rand());
2336     }

```

```

2337     else {
2338         for (i=0; i<s; ++i)
2339             data[i].set(fac*rand()+shift, fac*rand()+shift);
2340     }
2341
2342     return *this;
2343 }
2344
2345 template<>
2346 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::Symmetrize() {
2347     size_t i,j,r,s;
2348     if (dim1!=dim2) wblog(FL,
2349         "ERR Symmetrize() called with %dx%d matrix !?",dim1,dim2);
2350
2351     for (j=0; j<dim2; ++j)
2352         for (i=j+1; i<dim1; ++i) { r=i*dim2+j; s=j*dim2+i;
2353             data[r]=data[s]=wbcomplex(
2354                 0.5*(data[r].r+data[s].r),
2355                 0.5*(data[r].i-data[s].i)
2356             );
2357             data[s].Conj();
2358         }
2359
2360     return *this;
2361 };
2362
2363 template <class T>
2364 inline void wbMatrix<T>::setDiagRand(double fac, double shift) {
2365     size_t i, s=MIN(dim1,dim2);
2366     static char first_call=1;
2367
2368     if (data==NULL) return;
2369
2370     if (first_call) { wb_srand(); first_call=0; }
2371     fac/=(double)RAND_MAX;
2372
2373     MEM_SET<T>(data,dim1*dim2);
2374
2375     if (shift==0.)
2376         for (i=0; i<s; ++i) data[i*dim2+i]=(T)(fac*rand());
2377     else for (i=0; i<s; ++i) data[i*dim2+i]=(T)(fac*rand()+shift);
2378
2379     isdiag=1;
2380 }
2381
2382 template <class T>
2383 mxArray* wbMatrix<T>::toMx_base_d(char raw) const {
2384     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2385     return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
2386 };
2387
2388 template <class T>
2389 mxArray* wbMatrix<T>::toMx_base(char raw) const {
2390     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2391     return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
2392 };
2393
2394 template <>
2395 mxArray* wbMatrix<wbcomplex>::toMx_base(char raw) const {
2396     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2397     return Mx::Array<wbcomplex>(d1,d2,!raw).copyFromTR(data);
2398 };
2399
2400 template <>
2401 mxArray* wbMatrix<char>::toMx_base(char raw) const {
2402     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2403     return Mx::Array<mxChar>(d1,d2,!raw).copyFromTR(data);
2404 };
2405
2406 template <>
2407 mxArray* wbMatrix<long>::toMx_base(char raw) const {
2408     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2409     if (double(amax())<1E15)
2410         return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
2411     else return Mx::Array<int64_T>(d1,d2,!raw).copyFromTR(data);
2412 };
2413
2414 template <>
2415 mxArray* wbMatrix<unsigned long>::toMx_base(char raw) const {
2416     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2417     if (double(amax())<1E15)
2418         return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
2419     else return Mx::Array<uint64_T>(d1,d2,!raw).copyFromTR(data);
2420 };
2421
2422 template <class T> inline
2423 mxArray* wbMatrix<T>::toMx(char raw) const {

```

```

2424     if (raw) wblog(FL,
2425         "WRN raw flag will be ignored for typeid=%s", TSTR(T));
2426     return toMx_Struct();
2427 };
2428
2429 template <class T> inline
2430 mxArray* wbMatrix<T>::toMxP(char flag) const {
2431
2432     if (!flag || flag=='s' || flag=='S') return toMx_StructP();
2433     if (      flag=='c' || flag=='C') return toMx_CellP();
2434
2435     wblog(FL, "ERR %s() invalid flag (%s)", FCT, cSTR(flag));
2436     return 0;
2437 };
2438
2439 template <> inline
2440 mxArray* wbMatrix<unsigned>::toMx(char raw) const {
2441     return toMx_base(raw);
2442 };
2443 template <> inline
2444 mxArray* wbMatrix<unsigned>::toMxT(char raw) const {
2445     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2446     return Mx::Array<uint32_T>(d1,d2,!raw).copyFromTR(data);
2447 };
2448
2449 template <> inline
2450 mxArray* wbMatrix<int>::toMx(char raw) const {
2451     return toMx_base(raw);
2452 };
2453 template <> inline
2454 mxArray* wbMatrix<int>::toMxT(char raw) const {
2455     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2456     return Mx::Array<int32_T>(d1,d2,!raw).copyFromTR(data);
2457 };
2458
2459 template <> inline
2460 mxArray* wbMatrix<unsigned long>::toMx(char raw) const {
2461     return toMx_base(raw);
2462 };
2463 template <> inline
2464 mxArray* wbMatrix<unsigned long>::toMxT(char raw) const {
2465     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2466     return Mx::Array<uint64_T>(d1,d2,!raw).copyFromTR(data);
2467 };
2468
2469 template <> inline
2470 mxArray* wbMatrix<long>::toMx(char raw) const {
2471     return toMx_base(raw);
2472 };
2473 template <> inline
2474 mxArray* wbMatrix<long>::toMxT(char raw) const {
2475     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2476     return Mx::Array<int64_T>(d1,d2,!raw).copyFromTR(data);
2477 };
2478
2479 template <> inline
2480 mxArray* wbMatrix<unsigned char>::toMx(char raw) const {
2481     return toMx_base(raw);
2482 };
2483 template <> inline
2484 mxArray* wbMatrix<unsigned char>::toMxT(char raw) const {
2485     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2486     return Mx::Array<uint8_T>(d1,d2,!raw).copyFromTR(data);
2487 };
2488
2489 template <> inline
2490 mxArray* wbMatrix<char>::toMx(char raw) const {
2491     return toMx_base(raw);
2492 };
2493 template <> inline
2494 mxArray* wbMatrix<char>::toMxT(char raw) const {
2495     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
2496     return Mx::Array<int8_T>(d1,d2,!raw).copyFromTR(data);
2497 };
2498
2499 template <> inline
2500 mxArray* wbMatrix<double>::toMx(char raw) const {
2501     return toMx_base(raw);
2502 };
2503
2504 template <> inline
2505 mxArray* wbMatrix<wbcomplex>::toMx(char raw) const {
2506     return toMx_base(raw);
2507 };
2508
2509 template<class T>
2510 mxArray* wbMatrix<T>::toMx_Struct() const {

```

```

2511
2512     mxArray *S=data->mxCreateStruct(dim1,dim2);
2513
2514     size_t i,j;
2515     for (i=0; i<dim1; ++i)
2516         for (j=0; j<dim2; ++j) data[j+i*dim2].add2MxStruct(S,i+j*dim1);
2517
2518     return S;
2519 };
2520
2521 template<class T>
2522 mxArray* wbMatrix<T>::toMx_StructP() const {
2523
2524     if (data==NULL) {
2525         return mxCreateStructMatrix(dim1,dim2,0,NULL);
2526     }
2527     else {
2528         mxArray *S=(*data)->mxCreateStruct(dim1,dim2);
2529
2530         size_t i,j;
2531         for (i=0; i<dim1; ++i)
2532             for (j=0; j<dim2; ++j) data[j+i*dim2]->add2MxStruct(S,i+j*dim1);
2533
2534         return S;
2535     }
2536 };
2537
2538 template<class T>
2539 mxArray* wbMatrix<T>::toMx_CellP() const {
2540
2541     if (data==NULL) {
2542         return mxCreateCellMatrix(dim1,dim2);
2543     }
2544     else {
2545         mxArray *S=(*data)->mxCreateCell(dim1,dim2);
2546
2547         size_t i,j;
2548         for (i=0; i<dim1; ++i)
2549             for (j=0; j<dim2; ++j) data[j+i*dim2]->add2MxCell(S,i+j*dim1);
2550
2551         return S;
2552     }
2553 };
2554
2555 template<class T>
2556 mxArray* wbMatrix<T>::mxCreateStruct(unsigned m, unsigned n) const {
2557     return mxCreateCellMatrix(m,n);
2558 }
2559
2560 template<class T>
2561 void wbMatrix<T>::add2MxStruct(mxArray *S, unsigned i, char tst) const {
2562
2563     if (tst) {
2564         size_t s=0;
2565         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s) wblog(FL,
2566             "ERR %s() must follow mxCreateCell()\n%lx, %d/%d",FCT,S,i+1,s);
2567     }
2568
2569     mxSetCell(S,i,toMx());
2570 }
2571
2572
2573 template <class T>
2574 void wbMatrix<T>::info(const char *istr) const {
2575
2576     size_t l=0, n=16; char s[n];
2577     snprintf(s,n,"%lix%li",dim1,dim2);
2578     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
2579     fprintf(stdout,"  %12s %10s @ 0x%p double array\n", istr, s, data);
2580 };
2581
2582 template <class T>
2583 void wbMatrix<T>::print(const char *istr, char mflag) const {
2584
2585     mxArray *a=toMx_base(0);
2586
2587     if (istr && istr[0]) {
2588         if (mflag) { wb_printf("\n%s = [\n",istr); }
2589         else {
2590             wb_printf("\n%s = [%dx%d double]\n\n",istr,dim1,dim2);
2591         }
2592     } else { wb_printf("\n"); }
2593
2594     Wb::CallMatlab(0,NULL,1,&a,"disp");
2595     wb_printf("%s\n", mflag && istr && istr[0] ? " ";":"");
2596
2597     mxDestroyArray(a);

```

```

2598 };
2599
2600 template <class T>
2601 void wbMatrix<T>::Print(const char *istr, char mflag) const {
2602     size_t i,j;
2603
2604     if (!mflag) {
2605         if (istr[0])
2606             printf("\n%s = [%ldx%ld double]\n\n", istr, dim1, dim2);
2607         else printf("\n");
2608     }
2609     else
2610         printf("\n%s = [\n", istr[0] ? istr : "ans");
2611
2612     for (i=0; i<dim1; ++i) {
2613         for (j=0; j<dim2; ++j) printf(" %8.3g", (double)data[i*dim2+j]);
2614         printf("\n");
2615     }
2616     if (mflag) printf("]\n");
2617 }
2618
2619 }
2620
2621 template <class T>
2622 void wbMatrix<T>::printdata(
2623     const char *istr, const char *dbl_fmt, const char *rsep
2624 ) const {
2625     size_t i=0,j;
2626     if (istr && istr[0]) printf("%s = [\n ",istr); else printf("[");
2627
2628     for (; i<dim1; ++i) { if (i) printf(" %s ",rsep);
2629         for (j=0; j<dim2; ++j) { printf(dbl_fmt, double(data[i*dim2+j])); }
2630     }
2631     printf(" ]\n");
2632 }
2633
2634 }
2635
2636 template <class T>
2637 void wbMatrix<T>::appendRow(const wbvector<T> &v) {
2638     if (dim1) {
2639         if (dim2!=v.len) wblog(FL,
2640             "ERR Dimension mismatch (%dx%d : %d)", dim1,dim2, v.len);
2641         appendRows(1, v.data);
2642     }
2643     else RENEW(1,v.len,v.data);
2644 }
2645
2646
2647 template <class T>
2648 void wbMatrix<T>::appendRows(size_t n, const T* v) {
2649     size_t s=dim1*dim2, ds=n*dim2;
2650     T *d0=data;
2651
2652     dim1+=n; if (!n || !dim2) return;
2653
2654     WB_NEW(data,s+ds);
2655     MEM_COPY<T>(data,s+ds,s,d0,v);
2656     WB_DELETE(d0);
2657 }
2658
2659
2660 template<class T>
2661 void wbMatrix<T>::getReal(wbMatrix<double> &R) const {
2662     if (typeid(T)!=typeid(double)) {
2663         R.init(dim1,dim2);
2664         for (size_t n=dim1*dim2, i=0; i<n; ++i) { R.data[i]=double(data[i]); }
2665     }
2666     else { R.init(*this); }
2667     return;
2668 }
2669
2670 template<class T>
2671 void wbMatrix<T>::getImag(wbMatrix<double> &I) const {
2672     wblog(FL,"ERR wbMatrix::getImag not defined for type %s.",
2673         TSTR(T));
2674 }
2675
2676 template<class T>
2677 void wbMatrix<T>::set(const wbMatrix<double> &R, const wbMatrix<double> &I) {
2678     wblog(FL,"ERR wbMatrix::set(R,I) not defined for type %s.",
2679         TSTR(T));
2680 }
2681
2682 template<class T>
2683 wbMatrix<T>& wbMatrix<T>::Conj() { return *this; }
2684

```

```

2685 template<class T>
2686 double wbMatrix<T>::normReal() const { return Sqrt(norm2()); };
2687
2688 template<class T>
2689 double wbMatrix<T>::normImag() const { return 0; };
2690
2691 template<>
2692 double wbMatrix<wbcomplex>::normReal() const {
2693     double x=0;
2694     for (size_t n=dim1*dim2, i=0; i<n; ++i) x+=NORM2(data[i].r);
2695     return Sqrt(x);
2696 };
2697
2698 template<>
2699 double wbMatrix<wbcomplex>::normImag() const {
2700     double x=0;
2701     for (size_t n=dim1*dim2, i=0; i<n; ++i) x+=NORM2(data[i].i);
2702     return Sqrt(x);
2703 };
2704
2705 template<>
2706 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::Conj() {
2707     for (size_t s=dim1*dim2, i=0; i<s; ++i)
2708         data[i].i = -data[i].i;
2709     return *this;
2710 }
2711
2712 template<>
2713 void wbMatrix<wbcomplex>::getReal(wbMatrix<double> &R) const {
2714     R.init(dim1, dim2);
2715
2716     if (isdiag) {
2717         size_t i,k,s=MIN(dim1,dim2);
2718         for (i=0; i<s; ++i) { k=i*dim2+i; R.data[k]=data[k].r; }
2719         R.isdiag=isdiag;
2720     }
2721     else {
2722         for (size_t s=dim1*dim2, i=0; i<s; ++i)
2723             R.data[i]=data[i].r;
2724     }
2725 }
2726
2727 template<>
2728 void wbMatrix<wbcomplex>::getImag(wbMatrix<double> &I) const {
2729     I.init(dim1, dim2);
2730
2731     if (isdiag) {
2732         size_t i,k,s=MIN(dim1,dim2);
2733         for (i=0; i<s; ++i) { k=i*dim2+i; I.data[k]=data[k].i; }
2734         I.isdiag=isdiag;
2735     }
2736     else {
2737         for (size_t s=dim1*dim2, i=0; i<s; ++i)
2738             I.data[i]=data[i].i;
2739     }
2740 }
2741
2742 template<>
2743 void wbMatrix<wbcomplex>::set(
2744     const wbMatrix<double> &R,
2745     const wbMatrix<double> &I
2746 ){
2747     if (!R.hasSameSize(I)) wblog(FL,
2748         "ERR Dimension mismatch (%dx%d; %dx%d)!",R.dim1,R.dim2,I.dim1,I.dim2);
2749
2750     init(R.dim1,R.dim2);
2751
2752     for (size_t s=dim1*dim2, i=0; i<s; ++i)
2753         data[i].set(R.data[i], I.data[i]);
2754 }
2755
2756 template <class T> inline
2757 void matchIndexU(const char *F, int L,
2758     const wbMatrix<T> &QA, const wbMatrix<T> &QB,
2759     wbindex &Ib, const char force=1
2760 );
2761
2762 template<class T>
2763 int matchIndex(C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
2764     char lex=1,
2765     widx_t *ma=NULL, widx_t *mb=NULL,
2766     T eps=0
2767 );
2768
2769 template <class T> inline
2770 int matchIndex(
2771     const wbvector<T> &qA, const wbvector<T> &qB,

```

```

2772     wbindex &Ia, wbindex &Ib,
2773     char lex=1,
2774     widx_t *ma=NULL, widx_t *mb=NULL,
2775     T eps=0
2776 ){
2777     wbMatrix<T> QA, QB; int r;
2778     QA.init2ref(qA,'t');
2779     QB.init2ref(qB,'t'); if (QA.dim2!=1 || QB.dim2!=1) wblog(FL,"ERR");
2780
2781     r=matchIndex(QA,QB,Ia,Ib,lex,ma,mb,eps);
2782     return r;
2783 };
2784
2785 template <class T> inline
2786 int matchSortedIdx(
2787     const T *da, size_t lda, size_t na,
2788     const T *db, size_t ldb, size_t nb, wbindex &Ia, wbindex &Ib,
2789     widx_t m=-1,
2790     char lex=1
2791 ){
2792     widx_t m1=0,m2=0;
2793     return Wb::matchSortedIdx(da,lda,na, db,ldb,nb, Ia,Ib,m,lex,&m1,&m2);
2794 };
2795
2796 template <class T> inline
2797 int matchSortedIdx(C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
2798     widx_t m=-1,
2799     char lex=1,
2800     widx_t *ma=NULL, widx_t *mb=NULL,
2801     T eps=0
2802 ){
2803     return Wb::matchSortedIdx(
2804         QA.data, QA.dim2, QA.dim1,
2805         QB.data, QB.dim2, QB.dim1, Ia,Ib, m, lex, ma, mb, eps
2806     );
2807 };
2808
2809 template <class T> inline
2810 int matchSortedIdx(C_TMAT &QA, C_TMAT &QB,
2811     wbindex &Ia, wbindex &Ib, wbvector<widx_t> &D,
2812     widx_t m=-1,
2813     char lex=1,
2814     T eps=0
2815 ){
2816     return Wb::matchSortedIdx(
2817         QA.data, QA.dim2, QA.dim1,
2818         QB.data, QB.dim2, QB.dim1, Ia,Ib,D, m, lex, eps
2819     );
2820 };
2821
2822 template <class T> inline
2823 int matchSortedIdxU(
2824     const char *F, int L,
2825     C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
2826     widx_t m=-1, char lex=1
2827 );
2828
2829 template <class T> inline
2830 int matchSortedIdxU(
2831     C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib, widx_t m=-1
2832 ){ return matchSortedIdxU(FL,QA,QB,Ia,Ib,m); };
2833
2834 #endif
2835

```

5.30 wbMatrix_blas.hh

```

1 #ifndef __WB_MATRIX_BLAS_HH__
2 #define __WB_MATRIX_BLAS_HH__
3
4 // ----- //
5 // ----- //
6
7 void WbEigenSymmetric (const wbMatrix<double>&M,
8     wbMatrix<double>&U, wbvector<double> &E);
9
10 namespace Wb {
11
12 template<class T>
13 void MatProd(
14     const wbMatrix<T> &A,
15     const wbMatrix<T> &B, wbMatrix<T> &C,
16     char aflag='N', char bflag='N',

```

```

17     const T afac=1., const T cfac=0.,
18     const char i00flag=0
19 );
20
21 template<class TA, class TB, class TC>
22 void MatProd(
23     const wbMatrix<TA> &A,
24     const wbvector<TB> &B, wbvector<TC> &C,
25     char aflag='N', const TA afac=1., const TC cfac=0.,
26     const char i00flag=0
27 );
28
29 template<class T>
30 void SVD(
31     const wbMatrix<T> &A,
32     wbMatrix<T> &U,
33     wbMatrix<double> &S,
34     wbMatrix<T> &Vt
35 );
36
37 };
38
39 #endif
40

```

5.31 wbmpr.h

```

1  #ifndef __WB_MPFR_HH__
2  #define __WB_MPFR_HH__
3
4  /* ----- *
5   * wrapper class for MPFR (multiple precision floating point library)
6   * Wb, Feb01, 12
7   *
8   * -----
9   * Docu: see $SRC/MPFR/doc/mpfr.pdf
10  * -----
11
12  MPFR basics
13  * before assigning, MPFR variables need to be initialized.
14  * when done with a variable, it needs to be cleared out.
15  * variable will have the same allocated space during all its life.
16
17  * MPFR functions may create intermediate caches
18  => use mpfr_free_cache() to free
19  It is strongly advised to do that before terminating a thread
20  to avoid memory leaks.
21
22  * The semantics of a calculation in MPFR is specified as follows:
23  Compute the requested operation exactly (with "infinite accuracy"),
24  and round the result to the precision of the destination variable,
25  with the given rounding mode.
26
27  * if two variables are used to store only a few significant bits,
28  and their product is stored in a variable with large precision,
29  then MPFR will still compute the result with full precision.
30
31  -----
32  General rules:
33  * all MPFR functions expect output arguments before input arguments;
34  typicall: output var, input vars ..., rounding
35  * ok to use the same variable for both input and output in the same expression
36  * functions returning int: ternary value
37  0: value stored in output is the exact result of given mathematical function.
38  pos. (neg): value stored in output is greater (lower) than the exact result.
39
40  -----
41  MPFR rounding modes (similar to GMP rounding modes
42  GMP_RNDD, GMP_RNDU, GMP_RNDN, ...):
43
44  MPFR_RNDN: nearest: round to nearest (roundTiesToEven in IEEE 754-2008)
45  MPFR_RNDZ: zero : round toward zero (roundTowardZero in IEEE 754-2008)
46  MPFR_RNDU: up : round toward plus infity (roundTowardPositive in IEEE 754-2008)
47  MPFR_RNDD: down : round toward minus infity (roundTowardNegative in IEEE 754-2008)
48  MPFR_RNDA: away 0 : round away from zero.
49
50  -----
51  Initialization routines can clearance
52
53  void mpfr_init(mpfr_t x)
54  => initialize x to default precision and set its value to NaN
55
56  void mpfr_inits2(mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)

```



```

57  => initialize set of variables to default precision
58  => must be terminated by null-pointer (whose type must also be mpf_ptr)
59
60  void mpfr_init2(mpf_t x, mpfr_prec_t prec)
61  void mpfr_inits2(mpf_prec_t prec, mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
62  => same as previous init() functions, but with specified exact precision
63
64  void mpfr_clear(mpf_t x)
65  => free the space occupied by the significand of x.
66      Make sure to call this function for all mpfr_t variables
67      when you are done with them.
68
69  void mpfr_clears(mpf_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
70  => clear sequence of variables
71  => must be terminated by null-pointer (whose type must also be mpf_ptr)
72
73  void mpfr_set_default_prec(mpf_prec_t prec)
74  => if MPFR was compiled with --enable-thread-safe,
75      the default precision is local to each thread.
76
77  mpfr_prec_t mpfr_get_default_prec(void)
78  => return the current default MPFR precision in bits.
79
80  void mpfr_set_prec (mpfr_t x, mpfr_prec_t prec)
81  mpfr_prec_t mpfr_get_prec (mpfr_t x)
82  => set/get variable specific precision
83
84  * ----- */
85
86 #define DEF_RND GMP_RNDN
87
88 namespace Wb {
89
90     gmp_randstate_t wb_rstate;
91
92     template <unsigned P=128>
93     class mpfr__ {
94     public:
95
96         mpfr__(double val=0) {
97             mpfr_init2(f,P); init(FL,val);
98             #pragma omp atomic
99             ++num_vars;
100         };
101
102         mpfr__(const char *F, int L, double val) {
103             mpfr_init2(f,P); init(F_L,val);
104             #pragma omp atomic
105             ++num_vars;
106         };
107
108         mpfr__(const mpfr__ & b) {
109             mpfr_init2(f,P); mpfr_set(f,b.f,DEF_RND);
110             #pragma omp atomic
111             ++num_vars;
112         };
113
114         ~mpfr__() {
115             mpfr_clear(f);
116
117             #pragma omp atomic
118             --num_vars;
119
120             if (!num_vars) {
121                 Cleanup(FL);
122             }
123         };
124
125         void Cleanup(const char *F=NULL, int L=0) const;
126
127         mpfr__ & init (const char *F, int L, double val, char lflag=0);
128         mpfr__ & init_s(const char *F, int L, const char *s, unsigned base=10);
129         mpfr__ & init_d(const char *F, int L, double val, char dcheck=1);
130
131         mpfr__ & set_pi () { mpfr_const_pi (f,DEF_RND); return *this; };
132         mpfr__ & set_log2() { mpfr_const_log2(f,DEF_RND); return *this; };
133         mpfr__ & set_e () { init(FL,1); mpfr_exp(f,f,DEF_RND); return *this; };
134
135         mpfr__ & set_nan () { mpfr_set_nan(f); return *this; };
136         mpfr__ & set_inf (int s=+1) { mpfr_set_inf(f,s); return *this; };
137         mpfr__ & set_zero(int s=+1) { mpfr_set_zero(f,s); return *this; };
138
139         int isnan () const { return mpfr_nan_p(f); };
140         int isinf () const { return mpfr_inf_p(f); };
141         int iszero () const { return mpfr_zero_p(f); };
142         int isnumber () const { return mpfr_number_p(f); };

```

```

144 int isnonzero() const { return mpfr_regular_p(f); };
145
146 int sign() { return mpfr_sgn(f); };
147
148 void swap(mpfr__<P> &b) { mpfr_swap(f,b.f); };
149
150 int cmp(const mpfr__ &b ) const { return mpfr_cmp (f,b.f); };
151 int cmp(const double &val) const { return mpfr_cmp_d (f,val); };
152 int cmp(const int &val) const { return mpfr_cmp_si(f,val); };
153 int cmp(const long &val) const { return mpfr_cmp_si(f,val); };
154 int cmp(const unsigned &val) const { return mpfr_cmp_ui(f,val); };
155 int cmp(const unsigned long &val) const { return mpfr_cmp_ui(f,val); };
156
157 bool operator> (const mpfr__ &x) const { return (cmp(x)> 0); };
158 bool operator> (const double &x) const { return (cmp(x)> 0); };
159 bool operator> (const int &x) const { return (cmp(x)> 0); };
160 bool operator> (const long &x) const { return (cmp(x)> 0); };
161 bool operator> (const unsigned &x) const { return (cmp(x)> 0); };
162 bool operator> (const unsigned long &x) const { return (cmp(x)> 0); };
163
164 bool operator< (const mpfr__ &x) const { return (cmp(x)< 0); };
165 bool operator< (const double &x) const { return (cmp(x)< 0); };
166 bool operator< (const int &x) const { return (cmp(x)< 0); };
167 bool operator< (const long &x) const { return (cmp(x)< 0); };
168 bool operator< (const unsigned &x) const { return (cmp(x)< 0); };
169 bool operator< (const unsigned long &x) const { return (cmp(x)< 0); };
170
171 bool operator>= (const mpfr__ &x) const { return (cmp(x)>=0); };
172 bool operator>= (const double &x) const { return (cmp(x)>=0); };
173 bool operator>= (const int &x) const { return (cmp(x)>=0); };
174 bool operator>= (const long &x) const { return (cmp(x)>=0); };
175 bool operator>= (const unsigned &x) const { return (cmp(x)>=0); };
176 bool operator>= (const unsigned long &x) const { return (cmp(x)>=0); };
177
178 bool operator<= (const mpfr__ &x) const { return (cmp(x)<=0); };
179 bool operator<= (const double &x) const { return (cmp(x)<=0); };
180 bool operator<= (const int &x) const { return (cmp(x)<=0); };
181 bool operator<= (const long &x) const { return (cmp(x)<=0); };
182 bool operator<= (const unsigned &x) const { return (cmp(x)<=0); };
183 bool operator<= (const unsigned long &x) const { return (cmp(x)<=0); };
184
185 bool operator!= (const mpfr__ &x) const { return (cmp(x)!=0); };
186 bool operator!= (const double &x) const { return (cmp(x)!=0); };
187 bool operator!= (const int &x) const { return (cmp(x)!=0); };
188 bool operator!= (const long &x) const { return (cmp(x)!=0); };
189 bool operator!= (const unsigned &x) const { return (cmp(x)!=0); };
190 bool operator!= (const unsigned long &x) const { return (cmp(x)!=0); };
191
192 bool operator== (const mpfr__ &x) const { return (cmp(x)==0); };
193 bool operator== (const double &x) const { return (cmp(x)==0); };
194 bool operator== (const int &x) const { return (cmp(x)==0); };
195 bool operator== (const long &x) const { return (cmp(x)==0); };
196 bool operator== (const unsigned &x) const { return (cmp(x)==0); };
197 bool operator== (const unsigned long &x) const { return (cmp(x)==0); };
198
199 mpfr__& operator= (double val) { return init(FL,val); };
200 mpfr__& operator= (const mpfr__& b) {
201     mpfr_set(f,b.f,DEF_RND);
202     return *this;
203 };
204
205 mpfr__& times(const mpfr__& b, mpfr__& c) const {
206     mpfr_mul(c.f,f,b.f,DEF_RND); return c; };
207 mpfr__& divby(const mpfr__& b, mpfr__& c) const {
208     mpfr_div(c.f,f,b.f,DEF_RND); return c; };
209
210 mpfr__& operator*= (const mpfr__& b) {
211     mpfr_mul(f,f,b.f,DEF_RND); return *this; };
212 mpfr__& operator/= (const mpfr__& b) {
213     mpfr_div(f,f,b.f,DEF_RND); return *this; };
214 mpfr__& operator+= (const mpfr__& b) {
215     mpfr_add(f,f,b.f,DEF_RND); return *this; };
216 mpfr__& operator-= (const mpfr__& b) {
217     mpfr_sub(f,f,b.f,DEF_RND); return *this; };
218
219 mpfr__& operator*= (double val) { mpfr__ b(FL,val); return (*this)*=b; };
220 mpfr__& operator/= (double val) { mpfr__ b(FL,val); return (*this)/=b; };
221 mpfr__& operator+= (double val) { mpfr__ b(FL,val); return (*this)+=b; };
222 mpfr__& operator-= (double val) { mpfr__ b(FL,val); return (*this)-=b; };
223
224 mpfr__ operator* (const mpfr__& b) const {
225     mpfr__ x; mpfr_mul(x.f,f,b.f,DEF_RND); return x; };
226 mpfr__ operator/ (const mpfr__& b) const {
227     mpfr__ x; mpfr_div(x.f,f,b.f,DEF_RND); return x; };
228 mpfr__ operator+ (const mpfr__& b) const {
229     mpfr__ x; mpfr_add(x.f,f,b.f,DEF_RND); return x; };
230 mpfr__ operator- (const mpfr__& b) const {

```

```

231     mpfr__ x; mpfr_sub(x.f,f,b.f,DEF_RND); return x; };
232
233 mpfr__ operator* (int n) const {
234     mpfr__ x; mpfr_mul_si(x.f,f,n,DEF_RND); return x; };
235 mpfr__ operator/ (int n) const {
236     mpfr__ x; mpfr_div_si(x.f,f,n,DEF_RND); return x; };
237 mpfr__ operator+ (int n) const {
238     mpfr__ x; mpfr_add_si(x.f,f,n,DEF_RND); return x; };
239 mpfr__ operator- (int n) const {
240     mpfr__ x; mpfr_sub_si(x.f,f,n,DEF_RND); return x; };
241
242 mpfr__ operator* (unsigned n) const {
243     mpfr__ x; mpfr_mul_ui(x.f,f,n,DEF_RND); return x; };
244 mpfr__ operator/ (unsigned n) const {
245     mpfr__ x; mpfr_div_ui(x.f,f,n,DEF_RND); return x; };
246 mpfr__ operator+ (unsigned n) const {
247     mpfr__ x; mpfr_add_ui(x.f,f,n,DEF_RND); return x; };
248 mpfr__ operator- (unsigned n) const {
249     mpfr__ x; mpfr_sub_ui(x.f,f,n,DEF_RND); return x; };
250
251 mpfr__ operator* (long n) const {
252     mpfr__ x; mpfr_mul_si(x.f,f,n,DEF_RND); return x; };
253 mpfr__ operator/ (long n) const {
254     mpfr__ x; mpfr_div_si(x.f,f,n,DEF_RND); return x; };
255 mpfr__ operator+ (long n) const {
256     mpfr__ x; mpfr_add_si(x.f,f,n,DEF_RND); return x; };
257 mpfr__ operator- (long n) const {
258     mpfr__ x; mpfr_sub_si(x.f,f,n,DEF_RND); return x; };
259
260 mpfr__ operator* (unsigned long n) const {
261     mpfr__ x; mpfr_mul_ui(x.f,f,n,DEF_RND); return x; };
262 mpfr__ operator/ (unsigned long n) const {
263     mpfr__ x; mpfr_div_ui(x.f,f,n,DEF_RND); return x; };
264 mpfr__ operator+ (unsigned long n) const {
265     mpfr__ x; mpfr_add_ui(x.f,f,n,DEF_RND); return x; };
266 mpfr__ operator- (unsigned long n) const {
267     mpfr__ x; mpfr_sub_ui(x.f,f,n,DEF_RND); return x; };
268
269 mpfr__ & Sqrt() { mpfr_sqrt(f,f,DEF_RND); return *this; };
270 mpfr__ & Exp() { mpfr_exp(f,f,DEF_RND); return *this; };
271 mpfr__ & Log() { mpfr_log(f,f,DEF_RND); return *this; };
272 mpfr__ & Log2() { mpfr_log2(f,f,DEF_RND); return *this; };
273 mpfr__ & Atan() { mpfr_atan(f,f,DEF_RND); return *this; };
274
275 mpfr__ & Tanh() { mpfr_tanh(f,f,DEF_RND); return *this; };
276 mpfr__ & Atanh() { mpfr_atanh(f,f,DEF_RND); return *this; };
277
278 mpfr__ & Rand() {
279     mpfr_urandomb(f,wb_rstate);
280     return *this;
281 };
282
283 mpfr__ & Log10() { mpfr_log10(f,f,DEF_RND); return *this; };
284 mpfr__ & Round() { mpfr_round(f,f); return *this; };
285
286 mpfr__ sqrt() const {
287     mpfr__<P> x(*this); mpfr_sqrt(x.f,x.f,DEF_RND); return x; };
288 mpfr__ exp() const {
289     mpfr__<P> x(*this); mpfr_exp(x.f,x.f,DEF_RND); return x; };
290 mpfr__ log() const {
291     mpfr__<P> x(*this); mpfr_log(x.f,x.f,DEF_RND); return x; };
292 mpfr__ log2() const {
293     mpfr__<P> x(*this); mpfr_log2(x.f,x.f,DEF_RND); return x; };
294 mpfr__ atan() const {
295     mpfr__<P> x(*this); mpfr_atan(x.f,x.f,DEF_RND); return x; };
296 mpfr__ log10() const {
297     mpfr__<P> x(*this); mpfr_log10(x.f,x.f,DEF_RND); return x; };
298 mpfr__ round() const {
299     mpfr__<P> x(*this); mpfr_round(x.f,x.f); return x; };
300
301 explicit operator double() const { return mpfr_get_d(f,DEF_RND); };
302
303 explicit operator wbcplx() const;
304
305 explicit operator bool() const {
306     return (mpfr_zero_p(f) ? 0 : 1);
307 };
308
309 double getval_d() const { return mpfr_get_d(f,DEF_RND); };
310
311 bool operator< (double val) const { return (getval_d()< val); };
312 bool operator<= (double val) const { return (getval_d()<=val); };
313 bool operator> (double val) const { return (getval_d()> val); };
314 bool operator>= (double val) const { return (getval_d()>=val); };
315
316 int prec(unsigned base=2) const {
317     int p=mpfr_get_prec(f);

```

```

318         if (base!=2) { p = p* (::log(2) / ::log(base)) + 1; }
319         return p;
320     };
321
322     mpfr__ eps() {
323         mpfr__ x;
324         mpfr_set_uj_2exp(x.f, 1, -prec(2), DEF_RND);
325         return x;
326     };
327
328     mpfr__ & FlipSign() {
329         mpfr_setsign(f,f, mpfr_signbit(f) ? 0 : 1, DEF_RND);
330         return *this;
331     };
332
333     mpfr__ & Abs() {
334         mpfr_setsign(f,f, 0, DEF_RND);
335         return *this;
336     };
337
338     mpfr__ abs() const { return mpfr__(*this).Abs(); };
339     mpfr__ fabs() const { return mpfr__(*this).Abs(); };
340     mpfr__ abs2() const {
341         mpfr__ x(*this); x*=(*this);
342         return x;
343     };
344
345     void print(const char *vname="", const char *istr=0, char sep=0) const {
346         if (sep) printf("-----"
347             "0-----1-----2-----3-----4-----"
348             "5-----6-----7-----\n"
349         );
350         if (vname && vname[0]) printf("%6s = ",vname);
351         else printf("      ");
352
353         mpfr_out_str (stdout,
354             10,
355             0,
356             f,
357             GMP_RNDU
358         );
359
360         if (!istr || !istr[0]) {
361             long p,q; unsigned niter; double r, x=getval_d();
362             int e=Rational(x,p,q,&r,&niter,NULL,6,1000000,1E-10,1E-12);
363             if (!e && p) { char s[32];
364                 if (q!=1) snprintf(s,32," = %ld/%ld",p,q);
365                 else snprintf(s,32," = %ld",p);
366                 istr=s;
367             }
368         }
369         printf(" %s\n", istr && istr[0] ? istr : "");
370     };
371
372     size_t to_str(char *s, size_t n_, int base=10) const;
373     wstring to_str(int base=10) const;
374
375     void toStr(
376         char *s, unsigned n, int base=10, int nd=-1, char align=0) const;
377
378     wstring& toStr(
379         wstring &sout,int base=10, int nd=-1, char align=0) const;
380
381     mpfr_t f;
382
383     static unsigned long num_vars;
384
385 private:
386
387 };
388
389 typedef mpfr__<128> quad;
390 typedef mpfr__<128> quad2;
391
392 typedef mpfr__<192> quad3;
393 typedef mpfr__<256> quad4;
394 typedef mpfr__<256> qqquad;
395
396 template<>
397 unsigned long quad::num_vars=0;
398
399 quad mpfr_dummy_quad;
400
401 template <unsigned P>
402 mpfr__<P> operator+(const mpfr__<P> &a) { return mpfr__<P>(a); };
403
404 template <unsigned P>

```

```

405 mpfr__<P> operator-(const mpfr__<P> &a) { return mpfr__<P>(a).FlipSign(); };
406
407 template <class T, unsigned P>
408 mpfr__<P> operator*(const T& x_, const mpfr__<P> &a) {
409     mpfr__<P> x(x_); return (x*=a);
410 };
411
412 template <class T, unsigned P>
413 mpfr__<P> operator+(const T& x_, const mpfr__<P> &a) {
414     mpfr__<P> x(x_); return (x+=a);
415 };
416
417 template <class T, unsigned P>
418 mpfr__<P> operator-(const T& x_, const mpfr__<P> &a) {
419     mpfr__<P> x(x_); return (x-=a);
420 };
421
422 template <class T, unsigned P>
423 mpfr__<P> operator/(const T& x_, const mpfr__<P> &a) {
424     mpfr__<P> x(x_); return (x/=a);
425 };
426
427 template <unsigned P> inline
428 int isnan(const mpfr__<P> &a) { return a.isnan(); };
429
430 template <unsigned P> inline
431 int isinf(const mpfr__<P> &a) { return a.isinf(); };
432
433 template <unsigned P> inline
434 int iszero(const mpfr__<P> &a) { return a.iszero(); };
435
436 template <unsigned P> inline
437 int isnumber(const mpfr__<P> &a) { return a.isnumber(); };
438
439 template <unsigned P> inline
440 int isfinite(const mpfr__<P> &a) { return a.isfinite(); };
441
442 template <unsigned P>
443 mpfr__<P> fabs (const mpfr__<P> &x) { return x.fabs(); };
444
445 template <unsigned P>
446 mpfr__<P> sqrt (const mpfr__<P> &x) { return x.sqrt(); };
447
448 template <unsigned P>
449 mpfr__<P> exp (const mpfr__<P> &x) { return x.exp(); };
450
451 template <unsigned P>
452 mpfr__<P> log (const mpfr__<P> &x) { return x.log(); };
453
454 template <unsigned P>
455 mpfr__<P> log2 (const mpfr__<P> &x) { return x.log2(); };
456
457 template <unsigned P>
458 mpfr__<P> atan (const mpfr__<P> &x) { return x.atan(); };
459
460 template <unsigned P>
461 mpfr__<P> log10(const mpfr__<P> &x) { return x.log10(); };
462
463 template <unsigned P>
464 mpfr__<P> round(const mpfr__<P> &x) { return x.round(); };
465
466 class cc_quad {
467 public:
468
469     cc_quad(long double l=0) : f(l) { if (l!=0) {
470         long double r1=std::sqrt(l), r2=std::sqrt(f);
471
472         printf("1> %22.20Lf => %22.20Lf (%.3Lg)\n", l, r1, r1*r1-l);
473         printf("2> %22.20Lf => %22.20Lf (%.3Lg)\n", f, r2, r2*r2-f);
474
475         r2=std::sqrt(f);
476         printf("3> %22.20Lf => %22.20Lf (%.3Lg)\n\n", f, r2, r2*r2-f);
477     } }
478
479 };
480
481 cc_quad sqrt() const { cc_quad x(*this); x.f=std::sqrt(f); return x; };
482 cc_quad fabs() const { cc_quad x(*this); x.f=std::fabs(f); return x; };
483 cc_quad abs() const { cc_quad x(*this); x.f=std::fabs(f); return x; };
484
485 cc_quad operator+(const cc_quad& b) const { return (f+b.f); };
486 cc_quad operator-(const cc_quad& b) const { return (f-b.f); };
487 cc_quad operator*(const cc_quad& b) const { return (f*b.f); };
488 cc_quad operator/(const cc_quad& b) const { return (f/b.f); };
489
490 cc_quad operator+=(const cc_quad& b) { return (f+=b.f); };
491 cc_quad operator-=(const cc_quad& b) { return (f-=b.f); };

```

```

492     cc_quad operator*=(const cc_quad& b) { return (f*=b.f); };
493     cc_quad operator/=(const cc_quad& b) { return (f/=b.f); };
494
495     long double f;
496
497 private:
498 };
499 };
500
501     cc_quad sqrt(const cc_quad &a) { return a.sqrt(); }
502
503 };
504
505 #endif
506

```

5.32 wbmsg.hh

```

1  #ifndef __WB_MSG_HH__
2  #define __WB_MSG_HH__
3
4  // ----- //
5  // Messaging class -> automates deletion of message queue!!
6  // Wb, Dec03, 2003
7
8  #define WB_MSG_PRIV 0666
9
10     typedef struct __wb_msgbuf {
11         long    type;
12         time_t   time;
13         char    text[128];
14         double   val;
15     } wbMsgBuf;
16
17     const size_t WBMSGSZ = sizeof(wbMsgBuf) - sizeof(long);
18
19     class wbMsg {
20
21     public:
22
23         wbMsg(const char* file, int line,
24             const char *istr="Message queue", int key_=0
25         ){
26             info=istr;
27
28             key = (key_>0) ? key_ : getpid();
29             msgflg = WB_MSG_PRIV;
30
31             if ((msqid = msgget(key, IPC_CREAT | msgflg)) < 0) {
32                 wblog(FL,"ERR msgget returns %d", -msqid);
33                 exit(-1);
34             }
35
36             if (istr && istr[0]) wblog(file,line,"%NMSG create queue "
37                 "%s (key=%d (0x%X), qid=%d)", info.data,key,key,msqid
38             );
39         };
40
41         ~wbMsg () {
42             if (info.data && info.data[0]) wblog (FL,"MSG remove queue "
43                 "%s (key=%d (0x%X), qid=%d)", info.data,key,key,msqid);
44             msgctl (msqid, IPC_RMID, 0);
45         };
46
47         int Rec(const char*,int, wbMsgBuf &rbuf,
48             unsigned flag=IPC_NOWAIT
49         );
50
51         int Snd(const char*,int, wbMsgBuf &sbuf,
52             const char *istr, double val=0., unsigned type=1
53         );
54
55         wstring info;
56         int msqid, msgflg;
57         key_t key;
58
59     protected:
60     private:
61     };
62
63     int wbMsg::Rec (
64         const char* file, int line,
65         wbMsgBuf &rbuf,

```

```

66     unsigned flag
67 ){
68
69     memset(&rbuf,0,sizeof(wbMsgBuf));
70
71     int i = msgrcv(msqid, &rbuf, WBMSGSZ, 1, flag);
72     if (i < 0) {
73         if (i!=-1)
74             wblog(file,line,
75                 "ERR can't read message port %d (%d)", msqid, i);
76         return -i;
77     }
78
79     sprintf(str,"%s", ctime(&rbuf.time));
80     str[strlen(str)-1] = 0;
81
82     wblog (file,line,"%NNB! RECEIVED MESSAGE %s (%d; 0x%lX %d)\n\n"
83         "   time : %s\n"
84         "   text : %s\n"
85         "   val  : %g\n"
86         "   type : %ld\n",
87         info.data ? info.data : "", key, key, msqid,
88         str, rbuf.text, rbuf.val, rbuf.type);
89
90     return 0;
91 }
92
93 int wbMsg::Snd (
94     const char* file, int line,
95     wbMsgBuf &sbuf,
96     const char *istr, double val,
97     unsigned type
98 ){
99
100     int i;
101
102     memset(&sbuf,0,sizeof(wbMsgBuf));
103
104     sbuf.type = type;
105     sbuf.time = time((time_t*)NULL);
106     strncpy(sbuf.text,istr,127);
107     sbuf.val = val;
108
109     i = msgsnd(msqid, &sbuf, WBMSGSZ, IPC_NOWAIT);
110     if (i<0) {
111         wblog(FL,"ERR can't send to message port %d (err=%d)", msqid, i);
112         return -1;
113     }
114
115     sprintf(str,"%s", ctime(&sbuf.time));
116     str[strlen(str)-1] = 0;
117
118     wblog (file,line,
119         "Message sent ...\n"
120         "   type : %ld\n"
121         "   time : %s\n"
122         "   text : %s\n"
123         "   val  : %g",
124         sbuf.type, str, sbuf.text, sbuf.val);
125
126     return 0;
127 }
128
129 int WbMSGsnd (
130     const char* file, int line,
131     unsigned key,
132     char *text, double val,
133     unsigned type=1
134 ){
135
136     wbMsgBuf sbuf;
137     int i, msqid;
138
139     if ((msqid = msgget(key, IPC_CREAT | WB_MSG_PRIV)) < 0) {
140         wblog(FL,"ERR can't read message port %d (err=%d)", key, -msqid);
141         return -1;
142     }
143
144     sbuf.type = type;
145     sbuf.time = time((time_t*)NULL);
146     strcpy (sbuf.text, text);
147     sbuf.val = val;
148
149     i = msgsnd(msqid, &sbuf, WBMSGSZ, IPC_NOWAIT);
150     if (i<0) {
151         wblog(FL,"ERR can't send to message port %d (err=%d)", msqid, i);
152         return -1;

```

```

153     }
154
155     sprintf (str, "%s", ctime(&sbuf.time));
156     str[strlen(str)-1] = 0;
157
158     wblog(file,line,
159           "Message sent (%d; 0x%lX %d) ...\n"
160           "   type   : %ld\n"
161           "   time   : %s\n"
162           "   text   : %s\n"
163           "   val    : %g", key, key, msqid,
164           sbuf.type, str, sbuf.text, sbuf.val);
165
166     return 0;
167 }
168
169 #endif
170

```

5.33 wbomp.hh

```

1  #ifndef __WB_OMP_HH__
2  #define __WB_OMP_HH__
3
4  #ifdef QS_USING_OMP
5
6  // ----- //
7  // auxilliary routines for OpenMP // Wb,Jan30,17
8  // see also ~/Source/MPI/tst_omp_lock.cc etc.
9  // ----- //
10
11 namespace Wb {
12
13 std::string getMyOMP_ID();
14
15 class ompLockNest {
16
17 public:
18
19     ompLockNest(const char *s=NULL)
20         : level(0), owner(-1), istr(0), nactive(0), ntot(0) {
21         omp_init_nest_lock(&_nlk);
22         if (s && s[0]) { set_istr(s); }
23     };
24
25     ~ompLockNest() {
26         if (level || nactive) {
27             wblog(FI,"WRN %s() got lock @ l=%d, n=%d (id=%d/%d) !?",
28                 FCT,level,nactive,omp_get_thread_num(),omp_get_num_threads());
29             if (level==1 && !nactive) { release(); }
30         }
31         if (istr) { delete [] istr; istr=0; }
32         owner=-1;
33         omp_destroy_nest_lock(&_nlk);
34     };
35
36     void set_istr(const char *s);
37
38     wstring toStr(const char *istr=NULL) const;
39
40     wstring get_istr() const;
41
42     int acquire();
43
44     int test_acquire();
45
46     int release();
47
48     int level;
49     int owner;
50
51     char *istr;
52
53     int nactive;
54
55     long ntot;
56
57 private:
58
59     omp_nest_lock_t _nlk;
60
61     ompLockNest(const ompLockNest &);
62     void operator=(const ompLockNest &);

```



```

63
64 #ifdef LOAD_CGC_QSPACE
65     wstring check_BUF_size() const;
66 #endif
67 };
68
69 class ompGuard {
70
71 public:
72
73     ompGuard() : lp(0) { };
74
75     ompGuard(const char *F, int L, ompLockNest &lk, int lmax=99)
76         : lp(0) {
77         if (omp_get_num_threads()>1) {
78             acquire(F,L,lk,lmax);
79         }
80     };
81
82     ompGuard(ompLockNest &lk, int lmax=99) : lp(0) {
83         if (omp_get_num_threads()>1) {
84             acquire(0,0,lk,lmax);
85         }
86     };
87
88     void acquire(ompLockNest &lk, int lmax=99) { acquire(0,0,lk,lmax); };
89     void acquire(const char *F, int L, ompLockNest &lk, int lmax=99);
90
91     ~ompGuard() { if (lp) { lp->release(); } };
92
93     int level() { return (lp ? lp->level : -1); };
94
95 private:
96
97     ompLockNest *lp;
98
99     ompGuard(const ompGuard &) {
100         wblog(FL,"ERR %s() forbidden copy",FCT); };
101     void operator=(const ompGuard &) {
102         wblog(FL,"ERR %s() forbidden copy",FCT); };
103 };
104
105 class nthreads_aux {
106 public:
107     nthreads_aux() : np_curr(0), np_max(0) { };
108
109     nthreads_aux& increment(const char *fl) {
110         int q=omp_get_num_threads();
111         if (np_max>q) {
112             fprintf(stdout,"TST \e[31m%s() having nq=%d->%d\e[0m\n",
113                 fl? fl:"(null)",np_max,q);
114             np_max=q;
115         }
116         if (np_max<(++np_curr)) {
117             fprintf(stdout,"TST \e[35m%s() having nq=%d->%d\e[0m\n",
118                 fl? fl:"(null)",np_max,np_curr);
119             np_max=np_curr;
120         }
121         return *this;
122     };
123
124     nthreads_aux& decrement(const char *fl) {
125         if (--np_curr<0)
126             fprintf(stdout,"\e[35mWRN %s() having nq=%d/%d\e[0m\n",
127                 fl? fl:"(null)",np_curr,np_max);
128         return *this;
129     };
130
131 private:
132     int np_curr, np_max;
133 };
134
135 map< std::string, nthreads_aux > np_hist;
136
137 class checkMaxThreads {
138
139 public:
140
141     checkMaxThreads(const char *F, int L) : im(0) {
142         std::string fl=shortFL(F,L);
143         #pragma omp critical (checkMaxThreads__)
144         { np_hist[fl].increment(fl.c_str()); im=np_hist.find(fl); }
145     };
146
147     ~checkMaxThreads() {
148         #pragma omp critical (checkMaxThreads__)
149         { im->second.decrement(im->first.c_str()); }

```

```

150     };
151
152 private:
153
154     map< std::string, nthreads_aux >::iterator im;
155 };
156
157     class thread_xlink;
158     map< int, thread_xlink* > txmap;
159
160     class thread_xlink {
161     public:
162
163         thread_xlink(const char *F=NULL, int L=0) : tx(0) {
164             t0=omp_get_thread_num();
165             nt=omp_get_num_threads();
166
167             if (F) { fl=shortFL(F,L); }
168         };
169
170         ~thread_xlink() {
171             auto it = txmap.find(t0);
172             if (it!=txmap.end()) {
173                 #pragma omp critical (thread_xlink__)
174                 txmap.erase(t0);
175             }
176         };
177
178         void init(const char *F=NULL, int L=0) {
179             if (nt<=1) {
180                 t0=omp_get_thread_num();
181                 nt=omp_get_num_threads();
182             }
183             else {
184                 unsigned t2=omp_get_thread_num(); if (t2!=t0) {
185                     wblog(F_L,"WRN %s() thread-id changed %d->%d / %d->%d",
186                         FCT, t0,t2,nt,omp_get_num_threads());
187                 }
188             }
189             if (!tx) { if (F || fl.empty()) fl=shortFL(F_L);
190                 #pragma omp critical (thread_xlink__)
191                 txmap[t0]=this;
192             }
193             tx=(1<t0);
194         };
195
196         int add(const char *F, int L, size_t h);
197         int add(const char *F, int L, const ompLockNest *lk);
198
199         static int check_deadlocks();
200
201         unsigned t0, nt;
202
203         size_t tx;
204         std::string fl;
205
206     private:
207 };
208
209 };
210
211     Wb::ompLockNest wblog_lock("wblog");
212     Wb::ompLockNest rclog_lock("rclog");
213     Wb::ompLockNest mxapi_lock("mxapi");
214
215     Wb::ompLockNest XS_buf("XS_buf");
216
217     Wb::ompLockNest CS_buf("CS_buf");
218
219     Wb::ompLockNest CG_locks("CG_locks");
220
221 #else
222 #endif
223
224 #endif
225

```

5.34 wbopts.hh

```

1 #ifndef __WB_OPTS_HCC__
2 #define __WB_OPTS_HCC__
3
4 /* ----- */

```

```

5  /* ----- */
6  /* class to manage optional input arguments */
7
8  class OPTS {
9
10     public:
11
12         OPTS () : mto(0) {};
13
14         OPTS (const mxArray** ain, const unsigned n)
15             : mto(0) { init(ain,n); };
16
17         OPTS (const char *fname)
18             : mto(0) { init(fname); };
19
20         void init (const mxArray** ain, const unsigned n);
21         void init (const char *fname);
22
23     ~OPTS () { init(); }
24     void init() {
25         name.init();
26
27         if (MAT.mfp && aa.len)
28             for (unsigned i=0; i<aa.len; ++i) mxDestroyArray(aa[i]);
29
30         aa.init();
31     };
32
33     int len() { return ( MAT.mfp ? -1 : (int)aa.len ); };
34
35     template <class T>
36     bool getOpt(const char*, int,
37                const char* s, T &x, char force=0, const char *istr=0);
38
39     template <class T>
40     bool getOpt(const char*, int,
41                const char* s, wbvector<T> &x, char force=0, const char *istr=0);
42
43     template <class T>
44     bool getOpt(const char*, int,
45                const char* s, wbMatrix<T> &x, char force=0, const char *istr=0);
46
47     bool getOpt(const char*, int, const char* s, const char *istr=0);
48
49     template <class T>
50     bool getOpt(const char* s, T &x, char force=0, const char *istr=0) {
51         return getOpt(NULL,0, s,x,force,istr);
52     };
53
54     template <class T>
55     bool getOpt(const char* s,
56                wbvector<T> &x, char force=0, const char *istr=0) {
57         return getOpt(NULL,0,x,force,istr);
58     };
59
60     template <class T>
61     bool getOpt(const char* s,
62                wbMatrix<T> &x, char force=0, const char *istr=0) {
63         return getOpt(NULL,0,x,force,istr);
64     };
65
66     bool getOpt(const char* s, const char *istr=0) {
67         return getOpt(NULL,0,s,istr);
68     };
69
70     void checkAnyLeft(const char *MAT=NULL, int L=0);
71
72     protected:
73     private:
74
75         wbvector<mxArray*> aa;
76         wbvector<wbstring> name;
77
78         unsigned mto;
79
80         Wb::matFile MAT;
81
82         int findOpt(const char* s);
83     };
84
85     int OPTS::findOpt(const char* s) {
86
87         unsigned i, n=name.len;
88         int k=-1;
89
90         for (i=0; i<n; ++i)
91             if (name[i].len) if (!strcmp(name[i].data,s)) {

```

```

92     if (k<0) k=i; else ++mto;
93 };
94
95 return k;
96 }
97
98 void OPTS::init (const mxArray** ain, const unsigned n) {
99
100     char s[128];
101     if (MAT.mfp) wblog(FL,
102         "ERR OPTS already set by file >%s<", MAT.fname.data);
103
104     aa.init(n, (mxArray**)ain);
105     name.init(n);
106
107     for (unsigned i=0; i<aa.len; ++i) {
108         if (!mxIsChar(aa[i])) continue; else
109         if (mxGetString(aa[i],s,128)) {
110             wblog(FL,"WRN Error reading string in OPTS!?");
111             continue;
112         }
113
114         name[i].init(s);
115         aa[i]=NULL;
116     }
117 }
118
119 void OPTS::init (const char *fname) {
120
121     if (aa.len) wblog(FL,
122         "ERR OPTS already set by mxArray[] (%d)", aa.len);
123
124     MAT.Open(FL, fname, "r");
125 }
126
127 bool OPTS::getOpt (
128     const char *F, int L, const char* s, const char *istr
129 ){
130     unsigned r=0;
131
132     if (MAT.mfp) {
133         mxArray *a = matGetVariable(MAT.mfp, s); r=(a!=NULL);
134         if (a) {
135             double dbl=0.;
136             if (mxGetNumber(a,dbl)) wblog(FL,"ERR %s",str);
137             if (dbl==0) r=0;
138         }
139         mxDestroyArray(a);
140     }
141     else {
142         int i=findOpt(s); r=(i>=0);
143         if (r) name[i].init();
144     }
145
146     if (r && F) {
147         if (istr && istr[0])
148             wblog(F,L," * %s",istr);
149         else wblog(F,L," * %s",s);
150     }
151
152     return r;
153 }
154
155 template <class T>
156 bool OPTS::getOpt (
157     const char *F, int L, const char* s, T &x0,
158     char force, const char *istr
159 ){
160     unsigned k,e=0;
161     mxArray *a; T x=0;
162
163     if (MAT.mfp) {
164         a = matGetVariable(MAT.mfp, s);
165         if (!a) {
166             if (force) wblog(F_L,"ERR missing input %s (%s)",s,istr?istr:"");
167             return 0;
168         }
169     }
170     else {
171         int i=findOpt(s);
172         if (i<0) {
173             if (force) wblog(F_L,"ERR missing input %s (%s)",s,istr?istr:"");
174             return 0;
175         }
176
177         k=(unsigned)(i+1);
178

```

```

179         if (k>=name.len) { ++e; } else
180         if (aa[k]==NULL) { ++e; }
181         if (e) wblog(F_L,"ERR input double missing in OPTS (%s)",s);
182
183         a=aa[k];
184
185         name[k-1].init(); aa[k]=NULL;
186     }
187
188     if (mxGetNumber(a,x)) wblog(F_L,
189         "ERR reading OPT %s (%s)", s, str);
190
191     if (F && x0!=x) {
192         if (istr && istr[0])
193             wblog(F,L," * %-8s: %s [%s]", s,NSTR(x),istr);
194         else wblog(F,L," * %-8s: %s", s, NSTR(x));
195     }
196
197     x0=x;
198
199     if (MAT.mfp) mxDestroyArray(a);
200
201     return 1;
202 };
203
204 template<>
205 bool OPTS::getOpt(const char *F, int L,
206     const char* s, wstring &x, char force, const char *istr
207 ){
208     int lflag=0;
209     if (MAT.mfp) {
210         char s[128];
211         mxArray *a = matGetVariable(MAT.mfp, s);
212         if (!a) {
213             if (force) wblog(F_L,"ERR missing input %s", s);
214             return 0;
215         }
216
217         if (!mxIsChar(a)) wblog(F_L,
218             "ERR Invalid type '%s' in OPTS (%s)", mxGetClassName(a), s);
219         if (mxGetString(a,s,128)) wblog(F_L,
220             "ERR Error reading string '%s' for OPTS (max.len=128)", s);
221
222         if (x.data && !strcmp(x.data,s)) ++lflag;
223         x=s; mxDestroyArray(a);
224     }
225     else {
226         int i; unsigned k,e=0;
227
228         i=findOpt(s);
229         if (i<0) {
230             if (force) wblog(F_L,"ERR missing input %s", s);
231             return 0;
232         }
233         k=unsigned(i+1);
234
235         if (k>=name.len) { ++e; } else
236         if (aa[k]!=NULL) { ++e; }
237         if (e) wblog(F_L,"ERR missing input string in OPTS (%s)",s);
238
239         if (x.data && !strcmp(x.data,name[k].data)) ++lflag;
240         x=name[k];
241
242         name[k-1].init(); name[k].init();
243     }
244
245     if (F && lflag) {
246         if (istr && istr[0])
247             wblog(F,L," * %-8s: %s [%s]",s,x.data,istr);
248         else wblog(F,L," * %-8s: %s", s, x.data);
249     }
250
251     return 1;
252 };
253
254 template<>
255 bool OPTS::getOpt(const char *F, int L,
256     const char* s, mxArray* &a, char force, const char *istr
257 ){
258     if (MAT.mfp) {
259         a=matGetVariable(MAT.mfp, s);
260         if (a) aa.Append((mxArray*)a);
261     }
262     else {
263         int i=findOpt(s);
264
265         if (i<0) a=NULL;

```

```

266         else {
267             unsigned e=0, k=unsigned(i+1);
268
269             if (k>=name.len) { ++e; } else
270             if (aa[k]==NULL) { ++e; }
271             if (e) wblog(F_L,"ERR Input double missing in OPTS (%s)",s);
272
273             a=aa[k];
274
275             name[k-1].init(); aa[k]=NULL;
276         }
277     }
278
279     if (!a) {
280         if (force) wblog(F_L,"ERR missing input %s", s);
281         return 0;
282     }
283
284     if (F) {
285         if (istr && istr[0])
286             wblog(F,L," * %-8s: (%s) [%s]",s,mxGetClassName(a),istr);
287         else wblog(F,L," * %-8s: (%s)",s,mxGetClassName(a));
288     }
289
290     return 1;
291 };
292
293 template<>
294 bool OPTS::getOpt(
295     const char *F __attribute__((unused)),
296     int L __attribute__((unused)),
297     const char* s, iTag &t, char force, const char *istr
298 ){
299     wstring q;
300     getOpt(s,q,force,istr);
301     if (q) { t.init(0,0,q.data); return 1; }
302     return 0;
303 };
304
305 template<class T>
306 bool OPTS::getOpt(const char *F, int L,
307     const char* s, wvector<T>&x_, char force, const char *istr
308 ){
309     int lflag=0;
310     if (MAT.mfp) {
311         mxArray *a = matGetVariable(MAT.mfp, s);
312         if (!a) {
313             if (force) wblog(F_L,"ERR missing input %s", s);
314             return 0;
315         }
316
317         if (!mxIsDouble(a)) wblog(F_L,
318             "ERR invalid type '%s' in OPTS (%s)", mxGetClassName(a), s);
319
320         wvector<T> x; x.init(FL,a);
321         if (x!=x_) { x.save2(x_); ++lflag; }
322         mxDestroyArray(a);
323     }
324     else {
325         int i; unsigned k,e=0;
326
327         i=findOpt(s);
328         if (i<0) {
329             if (force) wblog(F_L,"ERR missing input %s", s);
330             return 0;
331         }
332         k=unsigned(i+1);
333
334         if (k>=name.len) { ++e; } else
335         if (aa[k]==NULL) { ++e; }
336         if (e) wblog(F_L,"ERR missing input data (vector) in OPTS (%s)",s);
337
338         wvector<T> x; x.init(FL,aa[k]);
339         if (x!=x_) { x.save2(x_); ++lflag; }
340
341         name[k-1].init(); aa[k]=NULL;
342     }
343
344     if (F && lflag) {
345         if (istr && istr[0])
346             wblog(F,L," * %-8s: [%s] (%s)", s, x_.toStr().data,istr);
347         else wblog(F,L," * %-8s: [%s]", s, x_.toStr().data);
348     }
349
350     return 1;
351 }
352

```

```

353 template<class T>
354 bool OPTS::getOpt(const char *F, int L,
355   const char* s, wbMatrix<T>&x_, char force, const char *istr
356 ){
357   int lflag=0;
358   if (MAT.mfp) {
359     mxArray *a = matGetVariable(MAT.mfp, s);
360     if (!a) {
361       if (force) wblog(F_L,"ERR missing input %s", s);
362       return 0;
363     }
364
365     if (!mxIsDouble(a)) wblog(F_L,
366       "ERR Invalid type '%s' in OPTS (%s)", mxGetClassName(a), s);
367
368     wbMatrix<T> x; x.init(F_L,a);
369     if (x!=x_) { x.save2(x_); ++lflag; }
370     mxDestroyArray(a);
371   }
372   else {
373     int i; unsigned k,e=0;
374
375     i=findOpt(s);
376     if (i<0) {
377       if (force) wblog(F_L,"ERR missing input %s", s);
378       return 0;
379     }
380     k=unsigned(i+1);
381
382     if (k>=name.len) { ++e; } else
383     if (aa[k]==NULL) { ++e; }
384     if (e) wblog(F_L,"ERR missing input data (matrix) in OPTS (%s)",s);
385
386     wbMatrix<T> x; x.init(F_L,aa[k]);
387     if (x!=x_) { x.save2(x_); ++lflag; }
388
389     name[k-1].init(); aa[k]=NULL;
390   }
391
392   if (F && lflag) {
393     if (istr && istr[0])
394       wblog(F_L," * %-8s: [%s] (%s)", s, x_.toStr().data,istr);
395     else wblog(F_L," * %-8s: [%s]", s, x_.toStr().data);
396   }
397
398   return 1;
399 }
400
401 void OPTS::checkAnyLeft(const char *F, int L) {
402   unsigned i,e=0;
403
404   if (MAT.mfp) {
405     MAT.init();
406     for (i=0; i<aa.len; ++i) mxDestroyArray(aa[i]);
407     return;
408   }
409
410   for (i=0; i<name.len; ++i) { if (name[i].len || aa[i]) {
411     if ((++e)==1) printf("\n");
412
413     if (name[i].len) {
414       printf("%5d: '%s'\n", i+1, name[i].data);
415     }
416     else if (aa[i]) {
417       int m=mxGetM(aa[i]), n=mxGetN(aa[i]);
418       if (mxIsCell(aa[i]))
419         printf("%5d: {%dx%d %s}\n", i+1,m,n,mxGetClassName(aa[i]));
420       else printf("%5d: [%dx%d %s]\n", i+1,m,n,mxGetClassName(aa[i]));
421     }
422   } }
423
424   if (e) {
425     if (mto)
426       wblog(F_L,"WRN some options appear more than once !?");
427
428     printf("\n");
429     wblog(F_L,"ERR unused/unrecognized options");
430   }
431 }
432 }
433
434 #endif
435

```

5.35 wbp perm.hh

```

1  #ifndef __WB_PERM_HH__
2  #define __WB_PERM_HH__
3
4  // ----- //
5  // permutation class
6  // Wb, Apr19, 06
7  // ----- //
8
9  class wbp perm : public wbvector<wperm_t> {
10
11  public:
12
13      wbp perm(wperm_t n=0, char reverse=0) { Index(n,reverse); };
14
15      wbp perm(const wbp perm &P, char iflag=0, unsigned r=-1)
16          : WBPPerm() { init(P,iflag,r); };
17
18      wbp perm(const wbp perm &P, size_t r) { init_pad(P,r); };
19
20      template<class T>
21      wbp perm(const wbvector<T> &b, wperm_t offset=0) {
22          init(FL,b,offset);
23      };
24
25      wbp perm(const wbindex &P);
26
27      wbp perm(const char *s, wperm_t offset=1) {
28          initStr(FL,s,offset);
29      }
30
31      wperm_t ell(wperm_t i) const {
32          if (len) {
33              if (i>=len) wblog(FL,
34                  "ERR element index out of bounds (%d/%d)",i+1/len);
35              return data[i];
36          }
37          else return i;
38      };
39
40      wbp perm& operator= (const char *s) {
41          initStr(FL,s,0); return *this;
42      };
43
44      int initStr(
45          const char *F, int L, const char *s, wperm_t offset=1);
46
47      wbp perm& init(const wbp perm &P, char iflag=0, unsigned r=-1) {
48          if (&P!=this) {
49              if (iflag) { P.invert(*this); }
50              else NEW(P.len,P.data);
51          }
52          else if (iflag) Invert();
53
54          if (int(r)>=0) { Extend(r); }
55
56          return *this;
57      };
58
59      wbp perm& init_trafo(const wbp perm &p1_, const wbp perm &p2_);
60
61      wbp perm& init_pad(const wbp perm &P, wperm_t r) {
62          if (r<P.len) wblog(FL,
63              "ERR invalid %s construction (%d/%d)",FCT,r,P.len);
64          NEW(r); memcpy(data,P.data,P.len*sizeof(P.data[0]));
65          for (wperm_t i=P.len; i<r; i++) data[i]=i;
66          if (!isValidPerm()) dispInvalidPerm(FL);
67          return *this;
68      };
69
70      wbp perm& init(wperm_t n=0, char reverse=0) {
71          return Index(n,reverse);
72      };
73
74      wbp perm& init(wperm_t n, const wperm_t* d) {
75          RENEW(n,d); isValidPerm(FL);
76          return *this;
77      };
78
79      wbp perm& init(const char *F, int L, const mxArray* a){
80          return init(F,L,a,mxIsChar(a) ? 1 : 0);
81      };
82
83      wbp perm& init(const char *F, int L,
84          const mxArray* a, wperm_t offset
85      ){

```



```

86     if (mxIsNumeric(a)) {
87         WBPPerm::init(F,L,a); if (offset) { operator+=(offset); }
88         isValidPerm(F_L);
89     }
90     else if (mxIsChar(a)) {
91         int l=mxGetNumberOfElements(a)+1;
92         char s[l]; mxGetString(a,s,l);
93         initStr(F_L,s,offset);
94     }
95     else wblog(FL,
96         "ERR %s() invalid input (%s) !?",FCT,mxGetClassName(a));
97     return *this;
98 };
99
100 template<class T>
101 WBPPerm<T> init(
102     const char *F, int L, const Wbvector<T> &b, wperm_t offset=0
103 ){
104     initT(b.len,b.data);
105     if (offset) { (*this)-=offset; }; isValidPerm(F_L);
106     return *this;
107 };
108
109 WBPPerm<T> initTranspose(wperm_t r) {
110     WBPPerm::init(r); wperm_t r2=r/2;
111     if (r%2) wblog(FL,"ERR %s() even input rank required (%d)",FCT,r);
112     for (wperm_t i=0; i<r2; ++i) { data[i]=r2+i; data[r2+i]=i; }
113     return *this;
114 };
115
116 WBPPerm<T> initPerm2(wperm_t r) {
117     WBPPerm::init(r); wperm_t r2=r/2;
118     if (r%2) wblog(FL,"ERR %s() even input rank required (%d)",FCT,r);
119     for (wperm_t l=0,i=0; i<r2; ++i) {
120         data[l++]=i;
121         data[l++]=r2+i;
122     }
123     return *this;
124 };
125
126 WBPPerm<T> initFirstTo (wperm_t k, wperm_t N) {
127     Index(N); mvFirstTo(k);
128     return *this;
129 };
130
131 WBPPerm<T> initLastTo (wperm_t k, wperm_t N) {
132     Index(N); mvLastTo(k);
133     return *this;
134 };
135
136 WBPPerm<T> init2Front(const WBPPerm &I, wperm_t N);
137 WBPPerm<T> init2End  (const WBPPerm &I, wperm_t N);
138
139 WBPPerm<T> init2FrontB(wperm_t m, wperm_t N);
140 WBPPerm<T> init2EndB  (wperm_t m, wperm_t N);
141
142 WBPPerm<T> init2Front(wperm_t k, wperm_t N);
143 WBPPerm<T> init2End  (wperm_t k, wperm_t N);
144
145 WBPPerm<T> Cycle(wperm_t k1, wperm_t k2);
146 WBPPerm<T> initCycle(wperm_t n, wperm_t k1, wperm_t k2);
147
148 WBPPerm<T> Index(wperm_t n, char reverse=0) {
149     RENEW(n); if (n==0) return *this;
150     if (reverse)
151         for (wperm_t l=n-1, i=0; i<n; ++i) data[i]=l-i;
152     else for (wperm_t i=0; i<n; ++i) data[i]=i;
153     return *this;
154 };
155
156 WBPPerm<T> Index(wperm_t i1, wperm_t i2) {
157     if (i1>i2) { RENEW(0); return *this; }
158
159     RENEW(i2-i1+1);
160     for (wperm_t i=0; i<len; i++) data[i]=i+i1;
161
162     return *this;
163 };
164
165 char isValidPerm(const char *F, int L, wperm_t r=-1) const;
166 char isValidPerm(wperm_t r=-1) const { return isValidPerm(0,0,r); };
167
168 WBPPerm<T> Invert() {
169     WBPPerm X; this->invert(X); X.save2(*this);
170     return *this;
171 };
172

```

```

173     wperm_t &invert(wperm_t &iP) const {
174         if (&iP==this) {
175             wperm_t X; this->invert(X); X.save2(iP);
176             return iP;
177         }
178
179         #ifndef SKIP_WB_ASSERT
180         isValidPerm(FL);
181         #endif
182
183         iP.RENEW(len);
184
185         for (wperm_t *p=iP.data, i=0; i<len; ++i) {
186             if (data[i]>=len) wblog(FL, "ERR %s() "
187                 "permutation out of bounds (%d/%d)", FCT, data[i], len);
188             p[data[i]]=i;
189         }
190         return iP;
191     };
192
193     wperm_t Rotate(wperm_t l);
194     wperm_t initRotate(wperm_t n, wperm_t k);
195
196     wperm_t Permute(const wperm_t &P, char iflag, wperm_t r);
197     wperm_t Permute(const wperm_t &P, char iflag=0) {
198         return Permute(P, iflag, len>P.len ? len : P.len);
199     };
200
201     wperm_t Extend(wperm_t r) {
202         if (r<len) wblog(FL,
203             "ERR %s() invalid decrease in size (%d/%d)", FCT, r, len);
204         if (r>len) {
205             wperm_t Px(r);
206             if (len) { memcpy(Px.data, data, len*sizeof(wperm_t)); }
207             Px.save2(*this);
208         }
209         return *this;
210     };
211
212     bool sameAs(const wperm_t &b) const {
213         if (!len) {
214             if (b.len) return b.isIdentityPerm();
215             else return 1;
216         }
217         if (!b.len) return isIdentityPerm();
218         else return (*this)==b;
219     };
220
221     bool isIdentityPerm() const {
222         bool q=1; wperm_t i=0;
223         for (; i<len; ++i) { if (data[i]!=i) { q=0;
224             #ifndef SKIP_WB_ASSERT
225                 for (; i<len; ++i) { if (data[i]>=len) wblog(FL,
226                     "ERR %s() perm out of bounds (%d: %d/%d)",
227                     FCT, i, data[i], len);
228                 }
229             #endif
230             break;
231         } }
232         return q;
233     };
234
235     bool isIdentityPerm(const char *F, int L, wperm_t r=-1) const {
236         if (int(r)>=0 && len!=r) { if (F) wblog(F, L,
237             "ERR %s() invalid permutation (len=%d/%d)", FCT, len, r);
238             return 0;
239         }
240         bool q=1; char mark[len]; memset(mark, 0, len);
241         wperm_t i=0;
242
243         for (; i<len; ++i) {
244             if (data[i]==i) { ++mark[i]; } else { q=0; break; }
245         }
246         for (; i<len; ++i) {
247             if (data[i]>=len) wblog(F, L, "ERR %s() invalid permutation\n"
248                 "(index out of range %d/%d)", FCT, data[i], len);
249             if ((++mark[data[i]])>1) wblog(F, L, "ERR %s() "
250                 "invalid permutation (non-unique index)", FCT);
251         }
252         return q;
253     };
254
255     bool isReversePerm() const {
256         if (len==0) return 0;
257         for (wperm_t m=len-1, i=0; i<len; i++) if (data[i]!=m-i) return 0;
258         return 1;
259     };

```

```

260
261     bool isCyclic2F(wperm_t m, wperm_t n=-1,
262         char iflag=0
263     ) const {
264
265         if (swperm_t(n)<0) { n = (len>m ? (len-m) : 0); }
266         if (!len) { return (m || n ? 0 : 1); }
267         if (len!=m+n) wblog(FL,
268             "ERR %s() length mismatch (%d / %d + %d)",FCT,len,m,n);
269
270         if (iflag) { SWAP(m,n); }
271
272         wperm_t i=0;
273         for (; i<m; ++i) if (data[i]!=i+n) { return 0; }
274         for (; i<len; ++i) if (data[i]!=i-m) { return 0; }
275         return 1;
276     };
277
278     bool isOpTranspose() const {
279
280         if (len!=3 && len%2) { return 0; }
281
282         for (wperm_t i=0, r=len/2; i<r; ++i) {
283             if (data[i]!=i+r || data[i+r]!=i) return 0;
284         }
285         return 1;
286     };
287
288     wbperrm& times(const wbperrm &p2, wbperrm &Pout) const {
289         if (!isValidPerm()) wblog(FL,
290             "ERR %s() invalid permutation (%s)",__FUNCTION__,toStr().data);
291         if (!p2.isValidPerm()) wblog(FL,
292             "ERR %s() invalid permutation (%s)",__FUNCTION__,STR(p2));
293         if (len!=p2.len) wblog(FL,
294             "ERR %s() incompatible permutations (%d/%d)",
295             __FUNCTION__,len,p2.len);
296
297         wbperrm P(len);
298         for (wperm_t i=0; i<len; i++) P.data[i]=data[p2.data[i]];
299
300         P.save2(Pout); return Pout;
301     };
302
303     wbperrm& flipIdx(wbperrm &P) const;
304     wbperrm& FlipIdx();
305     wbperrm flipIdx() const {
306         wbperrm P; flipIdx(P);
307         return P;
308     };
309
310     void ToRaw() { ToRaw(len); };
311     void ToRaw(wperm_t n) {
312         if (n<len) wblog(FL,"WRN ToRaw() with n=%d/%d !?",n,len);
313         if (n==0) return; else n--;
314
315         Flip();
316
317         for (wperm_t i=0; i<len; i++) {
318             if (i>n) wblog(FL,"ERR wbperrm - invalid n=%d (%d)",n,i);
319             data[i]=n-data[i];
320         }
321     }
322
323     wbststring toStr() const {
324
325         wbststring s_;
326         unsigned i=0; char *s;
327
328         if (len<10) {
329             s_.init(len+1); s=s_.data;
330             for (; i<len; ++i) { s[i]='1'+data[i]; }
331             s[i]=0;
332         }
333         else {
334             unsigned l=0, n=len*(log10(double(len))+2);
335             s_.init(n); s=s_.data;
336             for (; i<len && l<n; ++i) {
337                 l+=sprintf(s+l,n-l, " %d", (int) (data[i]+1));
338             }
339             if (l+3>=n) wblog(FL,"WRN %s() "
340                 "string out of bounds (%d/%d; %d/%d)",FCT,l,n,i+1,len);
341             else { s[l]=' '; s[l+1]=0; }
342         }
343         return s_;
344     };
345
346     mxArray* toMx(const char tflag=0) const {

```

```

347         return toMx_offset(1,tflag);
348     };
349
350 protected:
351 private:
352
353     void dispInvalidPerm(const char* F, int L) const {
354         wblog(F,L,"WRN invalid permutation [%s]",toStr().data); };
355
356 };
357
358 int isValidPerm(const char *s, wperm_t r=-1) {
359     wbperm P; int i;
360     if ((i=P.initStr(0,0,s))<0) {
361         return -1;
362     }
363     if (!P.isValidPerm(r)) return -2;
364     return P.len;
365 };
366
367 wbperm& wbperm::flipIdx(wbperm &P) const {
368
369     if (&P==this) wblog(FL,
370         "ERR flipIdx(P) calls itself\nused flipIdx() instead!");
371
372     P.RENEW(len); if (!len) return P;
373
374     for (wperm_t m=len-1, i=0; i<len; i++)
375         P.data[i]=m-data[i];
376
377     return P;
378 };
379
380 inline wbperm& wbperm::FlipIdx() {
381
382     for (wperm_t m=len-1, i=0; i<len; ++i) {
383         if (data[i]>m) wblog(FL,"ERR index out of bounds (%d/%d)",data[i],m);
384         data[i]=m-data[i];
385     }
386
387     return *this;
388 };
389
390 #endif
391

```

5.36 wbsort.hh

```

1  #ifndef __WB_REC_HPSORT_HH__
2  #define __WB_REC_HPSORT_HH__
3
4  namespace Wb {
5
6  /* ----- */
7  // generalized matrix/record sort
8  // sorted by rows/records of length m, with actual leading length lda
9  /* ----- */
10
11 template<class T>
12 int hpsort(
13     T *A,
14     size_t lda,
15     size_t N,
16     wbperm &P,
17     char dir=+1,
18     char lex=+1
19 );
20
21 template<class T>
22 int hpsort(
23     wbMatrix<T> &ra,
24     wbperm &P,
25     char dir=+1,
26     char lex=+1
27 );
28
29 template<class T>
30 void hpsort(
31     wbvector<T> &ra,
32     wbperm &P,
33     char dir=+1
34 );
35

```

```

36 template<class T>
37 void hpsort(wbvector<T> &ra);
38
39 template <class T> inline
40 size_t matchSortedIdx_grp(
41     const T *da, int lda, size_t na,
42     const T *db, int ldb, size_t nb,
43     wbindex &gra, wbindex &grb, size_t &mtot,
44     int m,
45     char lex,
46     T eps=0
47 );
48
49 template <class T> inline
50 size_t matchSortedIdx(
51     const T *da, int lda, size_t na,
52     const T *db, int ldb, size_t nb, wbindex &Ia, wbindex &Ib,
53     int m, char lex,
54     widx_t *ma=NULL,
55     widx_t *mb=NULL,
56     T eps=0
57 );
58
59 template <class T> inline
60 size_t matchSortedIdx(
61     const T *da, int lda, size_t na,
62     const T *db, int ldb, size_t nb,
63     wbindex &Ia, wbindex &Ib, wbvector<widx_t> &D,
64     int m, char lex, T eps=0
65 );
66
67 };
68
69 #endif
70

```

5.37 wbsparray.hh

```

1  #ifndef __WB_SPARRAY_HH__
2  #define __WB_SPARRAY_HH__
3
4  // #define WB_SPARSE_CLOCK
5
6  // ===== //
7  // wbsparray - N-D sparse array class //
8  // //
9  // generic N-dimensional class that allows permutations //
10 // and reshaping. Matrix elements are stored as follows: //
11 // //
12 // [ full index element 1, value1 //
13 // full index element 2, value2 //
14 // ... //
15 // ]; //
16 // //
17 // Wb, Jan05, 11 //
18 // ===== //
19
20 int sp_num_threads = 0;
21
22 bool mxIsWbsparray(
23     const char *F, int L, const mxArray *a, unsigned k=0);
24
25 bool mxIsWbsparray(
26     const char *F, int L, const mxArray *a, unsigned k,
27     const mxArray **as, const mxArray **ad, const mxArray **ai
28 );
29
30 bool mxIsWbsparray(const mxArray *a) { return mxIsWbsparray(0,0,a); }
31
32 #ifdef WB_SPARSE_CLOCK
33 Wb::Clock wbc_sparse_cont("sp::contract");
34 Wb::Clock wbc_sparse_cntf("spF::contract");
35 Wb::Clock wbc_sparse_cnt2("sp2::contract");
36 Wb::Clock wbc_sparse_cnt3("sp3::contract");
37 Wb::Clock wbc_sparse_cnt4("sp4::contract");
38 Wb::Clock wbc_sparse_cmpr("sp::compress");
39 #endif
40
41 #define wbsparray wbsparray<double>
42 #define wbsparrayTD wbsparray<TD>
43
44 template<class TD>
45 class wbsparray {

```

```

46
47 public:
48
49     wbsparray() : isref(0) {};
50
51     wbsparray(SPIDX_T d1) : isref(0) { if (d1) init(d1); };
52     wbsparray(SPIDX_T d1, SPIDX_T d2) : isref(0) { init(d1,d2); };
53     wbsparray(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) : isref(0) { init(d1,d2,d3); };
54     wbsparray(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
55         : isref(0) { init(d1,d2,d3,d4); };
56
57     wbsparray(const wbvector<SPIDX_T> &d, SPIDX_T l=0)
58         : isref(0) { init(d,l); };
59
60     wbsparray(char *sidx, SPIDX_T l=0)
61         : isref(0) { init(Str2Idx(sidx),l); };
62
63     wbsparray(const wbsparray &a)
64         : isref(0) { SIZE=a.SIZE; IDX=a.IDX; D=a.D; };
65
66     wbsparray(const char *F, int L, const mxArray *a)
67         : isref(0) { init(F_L,a); };
68
69     wbsparray(const mxArray *a) : isref(0) { init(F_L,a); };
70
71     wbsparray& init(
72         const char *F, int L, const wbarray<TD> &A, TD eps=0
73     );
74
75     template<class TA>
76     wbarray<TA>& toFull(wbarray<TA> &A) const;
77
78     template<class TA>
79     wbvector<TA>& toFull(wbvector<TA> &A) const;
80
81     template<class DB>
82     wbsparray& init(const wbsparray<DB>& a) {
83         if ((void*)this!=(void*)&a) {
84             SIZE=a.SIZE; IDX=a.IDX; D=a.D;
85             isref=0;
86         }
87         return *this;
88     };
89
90     SPIDX_T catRecs(SPIDX_T i0, const wbsparray &a, SPIDX_T id);
91
92     wbsparray& init(const wbvector<SPIDX_T> &S0, SPIDX_T l=0) {
93         SIZE=S0; D.init(l);
94         if (S0.len) IDX.init(l,S0.len);
95         else {
96             if (l>1) wblog(F_L,
97                 "WRN %s() use initDiag() with S=[] and nz=%d",FCT,l);
98             IDX.init();
99         }
100         return *this;
101     };
102
103     template<class I2>
104     wbsparray& init(const wbvector<I2> &S_, SPIDX_T l=0) {
105         wbvector<SPIDX_T> S0(S_);
106         return init(S0,l);
107     };
108
109     wbsparray& init() {
110         SIZE.init(); IDX.init(); D.init(); isref=0;
111         return *this;
112     };
113
114     wbsparray& init(SPIDX_T d1) {
115         if (d1) SIZE.init(1,&d1); else SIZE.init();
116         IDX.init(); D.init(); isref=0;
117         return *this;
118     };
119
120     wbsparray& init(SPIDX_T d1, SPIDX_T d2) {
121         SPIDX_T s[] = {d1,d2}; SIZE.init(2,s); IDX.init(); D.init();
122         return *this;
123     };
124
125     wbsparray& init(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) {
126         SPIDX_T s[] = {d1,d2,d3}; SIZE.init(3,s); IDX.init(); D.init();
127         return *this;
128     };
129
130     wbsparray& init(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4) {
131         SPIDX_T s[] = {d1,d2,d3,d4}; SIZE.init(4,s); IDX.init(); D.init();
132         return *this;

```

```

133     };
134
135     wbsparray& initz() {
136         IDX.init(0,SIZE.len); D.init();
137         return *this;
138     };
139
140     wbsparray& initz(SPIDX_T d1, SPIDX_T n) {
141         SIZE.init(1,&d1); IDX.init(n,SIZE.len); D.init(n);
142         return *this;
143     };
144
145     wbsparray& initz(SPIDX_T d1, SPIDX_T d2, SPIDX_T n) {
146         SPIDX_T s[] = {d1,d2}; SIZE.init(2,s); IDX.init(n,SIZE.len); D.init(n);
147         return *this;
148     };
149
150     wbsparray& initz(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T n) {
151         SPIDX_T s[] = {d1,d2,d3}; SIZE.init(3,s); IDX.init(n,SIZE.len); D.init(n);
152         return *this;
153     };
154
155     wbsparray& init_rzd(SPIDX_T r, SPIDX_T n) {
156         SIZE.init(r); IDX.init(n,r); D.init(n); isref=0;
157         return *this;
158     };
159
160     wbsparray& init_nnz(SPIDX_T n=0, char kflag=0) {
161         if (isref) {
162             if (SIZE.isref) wblog(FL,"ERR %s() got isref "
163                 "(%d,%d,%d,%d)",FCT,isref,SIZE.isref,IDX.isref,D.isref);
164             isref=0;
165         }
166         if (!n) {
167             if (!SIZE.len) { IDX.init(); }
168             else {
169                 if (kflag) { IDX.init(n,SIZE.len/2); }
170                 else { IDX.init(n,SIZE.len); }
171             }
172             D.init(); return *this;
173         }
174
175         if (!SIZE.len) wblog(FL,
176             "ERR sparse::%s() got empty size SIZE!",FCT);
177         if (kflag) {
178             if (SIZE.len%2) wblog(FL,
179                 "ERR %s() even rank expected (%d)",FCT,SIZE.len);
180             IDX.init(n,SIZE.len/2);
181         }
182         else { IDX.init(n,SIZE.len); }
183
184         D.init(n); return *this;
185     };
186
187     wbsparray& setRand(double x=1, char sflag=0);
188     wbsparray& setRand_data(double x=1, char sflag=0);
189
190     template<class DB>
191     wbsparray& init(const char *F, int L,
192         SPIDX_T d1, SPIDX_T d2, const wbsparray<DB> &B);
193
194     template<class DB>
195     wbsparray& init(const char *F, int L,
196         SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const wbsparray<DB> &B);
197
198     template<class DB>
199     wbsparray& init(const char *F, int L,
200         SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const wbsparray<DB> &B);
201
202     wbsparray& initEmpty(const wbsparray &a) {
203         if (this!=&a) SIZE=a.SIZE;
204         IDX.init(0,SIZE.len); D.init();
205         return *this;
206     };
207
208     wbsparray& init(
209         const char *F, int L, const mxArray *a, unsigned k=0);
210
211     wbsparray& init2ref(const wbsparray &a) {
212         SIZE=a.SIZE; IDX.init2ref(a.IDX); D.init2ref(a.D);
213         isref=1; return *this;
214     };
215
216     wbsparray& unRef() {
217         if (!isref) return *this;
218         IDX.unRef(); D.unRef(); isref=0;
219         return *this;

```

```

220     };
221
222     wbsparray& initCAT(const char *F, int L,
223         wbvector< wbsparray* > &X,
224         wbvector<SPIDX_T> *IO=NULL, char del=0
225     );
226
227     wbsparray& initCAT(const char *F, int L,
228         wbvector< wbsparray > &X, const wbvector<SPIDX_T> *IO=NULL
229     ){
230         wbvector< wbsparray* > xp(X.len);
231         for (unsigned i=0; i<X.len; ++i) xp[i]=(&X[i]);
232         return initCAT(F,L,xp,IO);
233     };
234
235     wbsparray& initCAT(
236         const char *F, int L, const wbvector< const wbsparray* > &X,
237         unsigned dim
238     );
239
240     wbsparray& initCAT(
241         const char *F, int L, const wbvector< wbsparray > &X,
242         unsigned dim
243     ){
244         wbvector< const wbsparray* > xp(X.len);
245         for (unsigned i=0; i<X.len; ++i) xp[i]=(&X[i]);
246         return initCAT(F,L,xp,dim);
247     };
248
249     wbsparray& Cat(const char *F, int L,
250         const wbsparray &B, unsigned dim
251     );
252
253     wbsparray& init_kron(
254         const wbvector<SPIDX_T> &sa, const wbvector<SPIDX_T> &sb,
255         SPIDX_T nab, char kflag
256     ){
257         setSIZE_kron(FL,sa,sb);
258         return init_nnz(nab,kflag);
259     };
260
261     void setSIZE_kron(const char *F, int L,
262         const wbvector<SPIDX_T> &sa, const wbvector<SPIDX_T> &sb,
263         char kflag=0);
264     void setSIZE_kron(const char *F, int L, char kflag=0);
265
266     wbsparray& setRec(SPIDX_T i, SPIDX_T j) {
267         if (i>=D.len || j>=D.len || IDX.dim1!=D.len) wblog(FL,"ERR %s() "
268             "index out of bounds (%d,%d/%d,%d",FCT,i,j,IDX.dim1,D.len);
269         if (i!=j) { D[i]=D[j]; IDX.recSet(i,j); }
270         return *this;
271     };
272
273     template<class T2>
274     wbsparray& setRec(SPIDX_T i, SPIDX_T i1, SPIDX_T i2, const T2& x) {
275         if (IDX.dim2!=2) wblog(FL,"ERR %s() "
276             "rank-2 array required (%d: %s)",FCT,SIZE.len,sizeStr().data);
277         if (i>=D.len || IDX.dim1!=D.len) wblog(FL,"ERR %s() "
278             "index out of bounds (%d/%d,%d)",FCT,i,IDX.dim1,D.len);
279         SPIDX_T *I=IDX.rec(i);
280         I[0]=i1; I[1]=i2; D[i]=x;
281         return *this;
282     };
283
284     wbsparray& setRecP(SPIDX_T i, const SPIDX_T *I, const TD &x) {
285         if (i>=D.len && IDX.dim1!=D.len) wblog(FL,"ERR %s() "
286             "index out of bounds (%d /%d,%d",FCT,i,IDX.dim1,D.len);
287         IDX.recSetP(i,I); D[i]=x;
288         return *this;
289     };
290
291     wbsparray& setRec_kron(
292         SPIDX_T l, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2,
293         unsigned kflag=0
294     );
295
296     wbsparray& Resize(SPIDX_T l) {
297         IDX.Resize(l,IDX.dim2); D.Resize(l);
298         return *this;
299     };
300
301     wbsparray& REDSIZE_NNZ(SPIDX_T l) {
302         if (l) {
303             if (IDX.dim1) {
304                 if (l>IDX.dim1) wblog(FL,"ERR %s() "
305                     "range out of bounds (IDX: %ld/%ld)",FCT,l,IDX.dim1);
306                 IDX.dim1=l;

```



```

307         }
308         if (D.len) {
309             if (l>D.len) wblog(FL,"ERR %s() "
310                 "range out of bounds (D: %ld/%ld)",FCT,l,D.len);
311             D.len=l;
312         }
313     }
314     else { IDX.init(0,SIZE.len); D.init(); }
315     return *this;
316 };
317
318 wbsparray& Resize(const wbvector<SPIDX_T> &S);
319
320 wbsparray& operator= (const wbsparray &a) {
321     if ((void*)this!=(void*)&a) init(a);
322     return *this;
323 };
324 template<class DB>
325 wbsparray& operator= (const wbsparray<DB> &a) {
326     if ((void*)this!=(void*)&a) init(a);
327     return *this;
328 };
329
330 wbsparray& save2(wbsparray &a, char ref=0) {
331     if (this!=&a) {
332         SIZE.save2(a.SIZE); IDX.save2(a.IDX,ref); D.save2(a.D);
333         a.isref=isref; isref=0;
334     }
335     return a;
336 };
337
338 SPIDX_T sub2ind(SPIDX_T k) const {
339     return Wb::sub2ind(SIZE.data,IDX.rec(k),SIZE.len);
340 };
341
342 void ind2sub(SPIDX_T k, SPIDX_T *I) const {
343     Wb::ind2sub(k, SIZE.data, I, SIZE.len);
344 };
345
346 void ind2sub(SPIDX_T k, wbindex &I) const {
347     if (I.len!=SIZE.len) I.init(SIZE.len);
348     Wb::ind2sub(k, SIZE.data, I.data, SIZE.len);
349 };
350
351 wbsparray<TD>& make2D(const wbindex &ic, wbsparray<TD> &B,
352     char pos=1
353 ) const;
354
355 sparseIndex2D<SPIDX_T>& toIndex2D(
356     const wbindex &ic, sparseIndex2D<SPIDX_T> &a,
357     char pos,
358     char use_rmaj=0
359 ) const;
360
361 wbsparray& initDiag(SPIDX_T d) {
362     D.init(d); SIZE.init(); IDX.init();
363     return *this;
364 };
365
366 wbsparray& initDiag(const wbvector<TD> &dd) {
367     D.init(dd); SIZE.init(); IDX.init();
368     return *this;
369 };
370
371 wbsparray& initDiag2(SPIDX_T d, TD dval=1) {
372     initz(d,d, d); D.set(dval); {
373         SPIDX_T i=0, l=d-1, *idx=IDX.data;
374         for (; i<d; ++i, idx+=2) { idx[0]=l-i; idx[1]=i; }
375     }
376     return *this;
377 };
378
379 bool isDiagMatrix(TD eps=1E-14) const;
380
381 bool isDiag(const char *F=0, int L=0) const {
382     if (SIZE.len) { checkSize(F,LF); }
383     return (!IDX.data && D.len ? 1 : 0);
384 };
385
386 wbvector<TD>& getDiag(const char *F, int L, wbvector<TD> &dd) const;
387
388 TD getDiag(const char *F, int L, SPIDX_T k) const;
389
390 wbsparray& getDiag(const char *F, int L, wbsparray &A) const {
391     A.init(); getDiag(F,L,A.D);
392     return A;
393 };

```

```

394
395     wbsparray& diag2reg(const char *F=0, int L=0, unsigned r=-1);
396     wbsparray& diag2reg(
397         const char *F, int L, wbsparray &A, unsigned r=-1) const;
398
399     wbsparray& toDiag(const char *F=0, int L=0) {
400         wbsparray X; getDiag(F_L,X);
401         return X.save2(*this);
402     };
403
404     int toDiagSafe(const char *F, int L, TD eps=1E-14) {
405         if (!isDiagMatrix(eps)) { if (F) {
406             wblog(F_L, "ERR %s() got non-diagonal matrix",FCT); }
407             return 0;
408         }
409         wbsparray X; getDiag(F_L,X);
410         X.save2(*this); return 1;
411     };
412
413     wbsparray& initIdentity(SPIDX_T d, TD dval=1){
414         initDiag(d); D.set(dval);
415         return *this;
416     };
417
418     wbsparray& initIdentity(SPIDX_T d1, SPIDX_T d2, sSPIDX_T m=0, TD dval=1){
419         if (d1==d2 && !m) { return initIdentity(d1,dval); }
420
421         sSPIDX_T k1=0, k2=0, i1=0, i2=d1, j1=0, j2=d2;
422         SPIDX_T l, s[] = {d1,d2};
423         if (m>0) { j1=-m; j2=-m; } else { i1=m; i2+=m; }
424
425         k2=MIN(i2,j2); l=(k2>k1 ? k2-k1 : 0);
426         SIZE.init(2,s); IDX.init(l,2); D.init2val(l,dval);
427
428         if (l) { SPIDX_T *idx=IDX.data;
429             for (SPIDX_T i=0; i<l; ++i, ++k1, idx+=2) {
430                 idx[0]=k1-i1;
431                 idx[1]=k1-j1;
432             }
433         }
434         return *this;
435     };
436
437     wbsparray& initIdentity(
438         const wvector<SPIDX_T> &S0, int m=-1, sSPIDX_T k=0, char fflag='f');
439     wbsparray& initIdentityB3(
440         SPIDX_T d1, SPIDX_T d2, SPIDX_T D,
441         sSPIDX_T i0=0, char fflag='f'
442     );
443
444     unsigned rank(const char *F=NULL, int L=0) const {
445         checkSize(F_LF);
446         if (!SIZE.len) {
447             return (isDiag() ? 2 : 0);
448         }
449         return SIZE.len;
450     };
451
452     SPIDX_T dim() const {
453         if (isDiag()) return D.len;
454         if (SIZE.len!=2 || SIZE[0]!=SIZE[1]) wblog(FL,
455             "ERR %s() got %s array",FCT,sizeStr().data);
456         return SIZE[0];
457     };
458
459     SPIDX_T dim(unsigned k) const {
460         if (!k) wblog(FL,
461             "ERR %s() input dim not 1-based (%d)",FCT,k);
462         return dim0(--k);
463     };
464
465     SPIDX_T dim0(unsigned k) const {
466         if (isDiag() && k<2) return D.len;
467         if (k<SIZE.len) return SIZE[k];
468
469         wblog(FL,"ERR %s() "
470             "dim=%d out of bounds (%s)",FCT,k+1,sizeStr().data);
471         return 0;
472     };
473
474     SPIDX_T matDim(unsigned k=1) {
475         if (!k || k>2) wblog(FL,
476             "ERR %s() index out of bounds (%d/%d)",FCT,k);
477         if (isDiag(FL)) { return D.len; } else
478             if (k==1) { return Wb::prodRange(SIZE.data,SIZE.len/2); }
479         else {
480

```

```

481         unsigned l=SIZE.len/2;
482         return Wb::prodRange(SIZE.data+l,SIZE.len-1);
483     }
484 };
485
486 SPIDX_T maxDim(unsigned r=-1) const {
487     if (int(r)<0) {
488         if (!SIZE.len)
489             { return (isDiag(FL) ? D.len : 0); }
490         else { return SIZE.max(); }
491     }
492     else {
493         if (SIZE.len) {
494             if (!r || r>SIZE.len) wblog(FL,
495                 "ERR %s() invalid dim range (%d/%d)",FCT,r,SIZE.len);
496             unsigned i=0, d=SIZE[0];
497             for (; i<r; ++i) { if (d<SIZE[i]) d=SIZE[i]; }
498             return d;
499         }
500         else {
501             if (isDiag(FL)) {
502                 if (!r || r>2) wblog(FL,
503                     "ERR %s() invalid dim range (%d/2)",FCT,r);
504                 return D.len;
505             }
506             else {
507                 if (r) wblog(FL,
508                     "ERR %s() invalid dim range (%d/0)",FCT,r);
509                 return 0;
510             }
511         }
512     }
513 };
514
515 TD norm2(char tnorm=0) const {
516     return Wb::overlap(D.data,D.data,D.len,1,tnorm);
517 };
518
519 wbststring sizeStr() const {
520     if (SIZE.len) return SIZE.toStrD();
521     else if (D.len) {
522         wbststring s(16); s.pushf(FL,"%dx%d",D.len,D.len);
523         return s;
524     }
525     else return wbststring("");
526 };
527
528 TD SkipTinyCols(TD eps=1E-14) {
529     unsigned r=rank(); if (!r && isEmpty()) return 0;
530     SPIDX_T l=0, i=0; TD e2=0;
531     wbvector<TD> a2; norm2vec(r-1,a2);
532     WBINDEX P(a2.len);
533
534     for (; i<a2.len; ++i) {
535         if (a2[i]>eps) { P[l++]=i; } else { e2+=a2[i]; }
536     }
537     if (l) P.len=l; else P.init();
538     Select0(P,r-1);
539
540     return e2;
541 };
542
543 TD SkipTiny(TD eps=1E-14) {
544     TD e2=0; SPIDX_T l=0, i=0;
545     if (eps) {
546         Wb::scale_eps(eps,D.data,D.len);
547         for (i=0; i<D.len; ++i) {
548             if (ABS(D[i])>eps) { if (l<i) { setRec(l,i); }; ++l; }
549             else { e2+=NORM2(D[i]); }
550         }
551     }
552     else {
553         for (i=0; i<D.len; ++i) {
554             if (D[i]!=eps) { if (l<i) { setRec(l,i); }; ++l; }
555         }
556     }
557
558     if (l<i) { IDX.ResizeXRecs(l); D.ResizeX(l); }
559     return e2;
560 };
561
562 bool isVector(const char *F=0, int L=0) const {
563     unsigned r=rank(F_L); if (r==1) return 1;
564     return (SIZE.count(1)>=(r-1));
565 };
566
567 bool isMatrix(const char *F=0, int L=0) const { return rank(F_L)==2; };

```

```

568     bool isRank(unsigned r) const { return (rank(FL)==r); };
569
570     bool isSMatrix(const char *F=0, int L=0, SPIDX_T *d=NULL) const {
571         unsigned r=rank(F,L);
572         if (r!=2 || (SIZE.len && SIZE[0]!=SIZE[1])) return 0;
573         if (d) {
574             SPIDX_T d0=(SIZE.len ? SIZE[0] : D.len);
575             if (sSPIDX_T(*d)>=0) { if ((*d)!=d0) return 0; }
576             else (*d)=d0;
577         }
578         return 1;
579     };
580
581     bool isSMatrix(const char *F, int L, SPIDX_T d) const {
582         unsigned r=rank(F,L); SPIDX_T d0=(SIZE.len ? SIZE[0] : D.len);
583         if (r!=2 || (SIZE.len && SIZE[0]!=SIZE[1]) || d!=d0) return 0;
584         return 1;
585     };
586
587     bool isEmpty() const {
588         return (SIZE.len || IDX.dim1 || D.len ? 0 : 1);
589     };
590
591     bool isEmpty_() const {
592         if (SIZE.len || IDX.dim1 || D.len) {
593             if (IDX.dim1!=D.len || IDX.dim2!=SIZE.len) wblog(FL,
594                 "ERR %s() size mismatch (%dx%d / %s)",
595                 FCT,D.len,SIZE.len,SSTR(IDX));
596             return 0;
597         }
598         return 1;
599     };
600
601     explicit operator bool() const { return !isEmpty_(); }
602     bool operator! () const { return isEmpty_(); }
603
604     bool isZero(TD eps=0, char flag=0) const;
605
606     SPIDX_T nnz() const {
607         if (isScalar(FL))
608             return (D.len==1 && D.data[0]!=0 ? 1 : 0);
609         else return D.len;
610     };
611
612     SPIDX_T nnz(const TD eps1, const TD eps2=1E9) const {
613         SPIDX_T i=0, n=0; TD a,e=0;
614         for (i=0; i<D.len; ++i) {
615             a=ABS(D[i]); if (a>eps1) ++n;
616             else if (a>eps2 && e<a) e=a;
617         }
618         if (e) wblog(FL,
619             "WRN %s() got small value(s) %.3g (%g;%g)",FCT,
620             double(e), double(eps1), double(eps2));
621         return n;
622     };
623
624     SPIDX_T nnz_diag() const {
625         if (isDiag(FL)) return D.nnz();
626         if (IDX.dim2!=2) wblog(FL,
627             "ERR %s() requires rank-2 object (S=%s)",FCT,sizeStr().data);
628         SPIDX_T i=0, n=0; const SPIDX_T *I=IDX.data;
629         for (; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1] && D[i]!=0) ++n; }
630         return n;
631     };
632
633     SPIDX_T numel() const {
634         return (isDiag(FL) ? D.len*D.len : SIZE.prod(0));
635     };
636
637     TD norm(char tnorm=0) const {
638         return SQRT(Wb::overlap(D.data,D.data,D.len,1,tnorm));
639     };
640
641     template<class T2>
642     wbvector<T2>& norm2vec(unsigned k,
643         wbvector<T2> &A, char tnorm=0
644     ) const;
645
646     void swap(wbsparray &B) {
647         if (this!=&B) {
648             if (isref || B.isref) wblog(FL,
649                 "ERR %s() got arrays with isref=(%d,%d)",FCT,isref,B.isref);
650             SIZE.swap(B.SIZE);
651             IDX.swap(B.IDX);
652             D.swap(B.D);
653         }
654     };

```

```

655
656 wbsparray& TimesEl(const wbsparray &B, char tnorm=0);
657
658 TD dotProd(
659     const char *F, int L, const wbsparray &B, char tnorm=0) const;
660
661 TD froNorm2(const wbsparray &B) const { return dotProd(0,0,B); };
662
663 double normDiff2(const wbsparray &B) const {
664     wbsparray X(*this); X-=B; return (double)X.norm2();
665 };
666 double normDiff (const wbsparray &B) const {
667     wbsparray X(*this); X-=B; return (double)X.norm();
668 };
669
670 TD Normalize(char tnorm=0, char qflag=0);
671 TD NormalizeCol(SPIDX_T k, char tnorm=0, char qflag=0);
672
673 wbsparray& QRdecomp(const char *F, int L,
674     wbsparray &Q,
675     char posR=0,
676     TD eps=1E-14
677 );
678
679 wbsparray& OrthoNormalizeColsQR(
680     const char *F=NULL, int L=0,
681     TD eps=1E-14
682 );
683
684 wbsparray& OrthoNormalizeColsGS(
685     const char *F=NULL, int L=0, char tnorm=0,
686     char qxflag=0, TD eps=1E-14, unsigned np=1
687 );
688
689 wbsparray& setCol(
690     const char *F, int L, SPIDX_T k, const wbsparray &v);
691
692 wbsparray& setCol(SPIDX_T k, const wbsparray &v) {
693     return setCol(0,0,k,v);
694 };
695
696 double sparsity() const {
697     if (!D.len) return 0;
698     if (isScalar(FL)) return (D.len==1 && D.data[0]!=0 ? 1 : 0);
699     return D.len/double(numel());
700 };
701
702 int checkTrailingSingletons(const char *F=0, int L=0, unsigned *r=NULL) const;
703
704 int skipTrailingSingletons(const char *F=0, int L=0, unsigned r=-1);
705 int skipTrailingSingletons(
706     const char *F, int L, wbsparray &c, unsigned r=-1) const;
707
708 bool hasSingletons(const wbindex &I) const;
709
710 wbsparray& skipSingletons(
711     const char *F, int L, const wbindex &I, wbsparray &A
712 ) const;
713
714 wbsparray& AddTrailingSingletons(
715     const char *F, int L, unsigned r);
716
717 wbsparray& initScalar(const TD x=0, unsigned r=-1) {
718     if (int(r)<=0) {
719         if (D.len!=1) { D.init(1); }; D[0]=x;
720         if (SIZE.len) { SIZE.init(); }
721         IDX.init();
722     }
723     else {
724         SIZE.init(r).set(1);
725         if (x) {
726             if (D.len!=1) { D.init(1); }; D[0]=x;
727             IDX.init(1,r);
728         }
729         else { IDX.init(0,r); D.init(); }
730     }
731     isref=0; return *this;
732 };
733
734 char isScalar(const char *F=0, int L=0, char zflag=0) const {
735     if (!SIZE.len) {
736         if (IDX.dim1 || IDX.dim2) sperror_this(F_L);
737         return (zflag ? D.len<=1 : D.len==1);
738     }
739     return SIZE.allEqual(1);
740 };
741

```

```

742     wbsparray& toScalar(const char *F=0, int L=0, unsigned r=-1);
743
744     TD getScalar(const char *F=0, int L=0) const;
745
746     char checkSize(
747         const char *F=0, int L=0, const char *fct=0, const char *istr=0
748     ) const;
749
750     char checkSize2(const char *F, int L, const wbsparray &B) const;
751
752     void check_IDX_range(const char *F=0, int L=0) const;
753
754     bool isProptoId() const { TD q=0; return isProptoId(q); }
755     bool isProptoId(TD &q, TD eps=1E-14) const;
756
757     bool isIdentityMatrix(TD eps=1E-14) const {
758         TD q=1; return isProptoId(q,eps);
759     };
760
761     char isIdentity(TD eps=1E-14) const;
762     char isIdentity(const wbperrm &P, TD *dval=NULL, TD eps=1E-14) const;
763
764     bool hasSize(SPIDX_T d1) const;
765     bool hasSize(SPIDX_T d1, SPIDX_T d2) const;
766     bool hasSize(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const;
767
768     template<class T>
769     wbvector<T>& getSize(wbvector<T> &S) const {
770         if (SIZE.len) { return S.initT(SIZE); }
771         if (D.len && isDiag()) { return S.init(2).set(D.len); }
772         return S.init();
773     };
774
775     char fitsSize(const wbsparray &, char strict=0) const;
776
777     char sameSize(const wbsparray &, char strict=0) const;
778     char sameSizeUp2Singletons(
779         const wbvector<SPIDX_T> &S, char strict=0) const;
780
781     char matchWithSingletons(const char *F, int L,
782         const wbvector<SPIDX_T> &S, wbvector<int> &M
783     ) const;
784
785     char sameUptoFac(
786         const wbsparray &B, TD *fac=NULL, char lex=-1,
787         TD eps=1E-12) const;
788
789     bool operator== (const wbsparray &B) const {
790         if (this==&B) return 1;
791         if (SIZE!=B.SIZE) return 0;
792         if (IDX==B.IDX && D==B.D) return 1;
793         return (normDiff2(B)<1E-20);
794     };
795
796     bool operator!= (const wbsparray &B) const {
797         return !(*this==B);
798     };
799
800     wbsparray& Plus(const char *F, int L,
801         const wbsparray &B, TD bfac=1, TD afac=1, TD eps=0
802     );
803
804     TD sum() const { return D.sum(); };
805     TD aMin() const { return D.aMin(1); };
806     TD aMax() const { return D.aMax(); };
807
808     SPIDX_T findRecSortedP(SPIDX_T *idx, SPIDX_T n,
809         char lex=-1
810     );
811
812     wbsparray& Eldiv(const char *F, int L, const wbvector<TD> &x_) {
813         SPIDX_T i=0, j;
814         const SPIDX_T *idx=IDX.data;
815         const TD *x=x_.data;
816
817         if (!isVector()) wblog(F_L,
818             "ERR %s() invalid vector (%s)",FCT,sizeStr().data);
819         for (; i<SIZE.len; ++i) { if (SIZE[i]>1) { idx+=i; break; } }
820
821         for (i=0; i<IDX.dim1; ++i, idx+=IDX.dim2) { j=(*idx);
822             if (j>=x_.len) wblog(F_L,
823                 "ERR %s() index out of bounds (%d/%d)",FCT,j+1,x_.len);
824             if (x[j]==0) wblog(F_L,
825                 "ERR %s() div/0 !? (%g)",FCT,double(x[j]));
826             D.data[i]/=x[j];
827         }
828         return *this;

```

```

829     };
830
831     wbsparray& operator+=(const wbsparray &B) { return Plus(FL,B,+1); }
832     wbsparray& operator-=(const wbsparray &B) { return Plus(FL,B,-1); }
833
834     wbsparray& Conj() { D.Conj(); return *this; }
835
836     wbsparray& operator*=(TD x) {
837         if (isref) {
838             D*=x;
839         }
840         else {
841             if (x!=0) { if (x!=1) D*=x; }
842             else init_nnz();
843         }
844         return *this;
845     };
846
847     wbsparray& operator/=(TD x) {
848         if (x==0) wblog(FL,"ERR %s() div/0",FCT);
849         if (x!=1) D/=x;
850         return *this;
851     };
852
853     const TD& operator[](SPIDX_T i) const {
854         if (i>=D.len) {
855             wblog(FL,"ERR %s() index out of bounds (%d/%d; %s)",
856                 FCT,i,D.len,sizeStr().data);
857         }
858         return D.data[i];
859     };
860
861     TD& operator[](SPIDX_T i) {
862         if (i>=D.len) {
863             if (i==0 && IDX.dim1==0) {
864                 if (SIZE.len && SIZE.allEqual(1)) {
865                     IDX.init(1,SIZE.len);
866                     D.init(1); return D.data[i];
867                 }
868                 else if (!SIZE.len && IDX.dim2==0) {
869                     IDX.init();
870                     D.init(1); return D.data[i];
871                 }
872             }
873             wblog(FL,"ERR %s() index out of bounds (%d/%d; %s)",
874                 FCT,i,D.len,sizeStr().data);
875         }
876         return D.data[i];
877     };
878
879     TD value(const SPIDX_T *idx, SPIDX_T n, char lex=-1
880 ) const {
881         SPIDX_T i=IDX.findRecSorted(idx,n,lex);
882         return (sSPIDX_T(i)<0 ? TD(0.) : D.data[i]);
883     };
884
885     wbsting info2Str(const char *istr=0) const {
886         wbsting s(32);
887         if (istr) {
888             s.pushf(FL,"%s(%s; %dx%d; %d)",istr,
889                 sizeStr().data, IDX.dim1,IDX.dim2,D.len);
890         }
891         else {
892             s.pushf(FL,"%s; %dx%d; %d",
893                 sizeStr().data, IDX.dim1,IDX.dim2,D.len);
894         }
895         return s;
896     };
897
898     size_t memSize() const {
899         size_t m=sizeof(TD);
900 #ifdef QS_USING_MPFR
901         if (typeid(TD)==typeid(Wb::quad)) {
902             m+=16;
903         }
904 #endif
905         return ( sizeof(isref)
906             + sizeof(SIZE) + SIZE.len*sizeof(SPIDX_T)
907             + sizeof(IDX) + IDX.numel()*sizeof(SPIDX_T)
908             + sizeof(D) + D.len*m
909         );
910     };
911
912     void info(const char* istr="") const { return info(0,0,istr); };
913     void info(const char* F, int L, const char* "=") const;
914     void print(const char* istr="", const char *fmt="") const;
915

```

```

916 void print(
917     const char *F, int L,
918     const char *istr="", const char *fmt=""
919 ) const;
920
921 void printdata(const char *istr="ans", const char *fmt="") const;
922
923 void sperror_this(
924     const char *F, int L, const char *fct=0, const char *istr=0
925 ) const {
926     wblog(F_L,"ERR %s() %s S=%s (%dx%d; %d) %s",
927         fct ? fct : "sparse", istr ? istr : "got",
928         SIZE.len ? SIZE.toStrD().data : "[]",
929         IDX.dim1,IDX.dim2,D.len, isref ? " **REF** ":"");
930 };
931 };
932
933 TD trace() const;
934
935 template<class T2>
936 wbvector<T2>& trace(unsigned k, wbvector<T2> &A) const;
937
938 bool isSym_aux(
939     const char *file, int line, const char *fct,
940     const wbsparray &B, TD eps, TD* xref,
941     const char symflag='s',
942     const char lflag=1
943 ) const;
944
945 bool isHConj(
946     const wbsparray &B, TD eps=1E-12, TD* xref=NULL
947 ) const { return isSym_aux(FLF, B, eps, xref, 's'); };
948
949 bool isHConj(
950     TD eps=1E-12, TD* xref=NULL
951 ) const { return isSym_aux(FLF,*this,eps,xref,'s'); };
952
953 bool isAHerm(
954     const wbsparray &B, TD eps=1E-12, TD* xref=NULL
955 ) const { return isSym_aux(FLF, B, eps, xref, 'a'); };
956
957 bool isAHerm(
958     TD eps=1E-12, TD* xref=NULL
959 ) const { return isSym_aux(FLF,*this,eps,xref,'a'); };
960
961 wbsparray& Sort(char lex=-1) {
962     if (IDX.data) {
963         if (IDX.dim1!=D.len) wblog(FL,
964             "ERR %s() %d/%d !?",FCT,IDX.dim1,D.len);
965         wbpem P; IDX.SortRecs(P,+1,lex); D.Select(P);
966     }
967     return *this;
968 };
969
970 double Compress(
971     const char *F=0, int L=0, TD eps=0,
972     char lex=-1,
973     const wbpem *P=NULL
974 );
975
976 bool isCompressed(
977     char lex=-1, char dir=0
978 ) const { return IDX.isUniqueSorted(dir,lex); };
979
980 int splitSparseCM(
981     const char *F, int L, wbvector< wbsparray > &X,
982     TD eps=0, char ref=0, char tnorm=0);
983
984 wbsparray& reshape(
985     const wbvector<SPIDX_T> &S, wbsparray &A) const;
986
987 wbsparray& Reshape(const wbvector<SPIDX_T> &S){
988     wbsparray X; reshape(S,X); return X.save2(*this);
989 };
990
991 wbsparray& Reshape(SPIXD_T s1, SPIDX_T s2){
992     wbsparray X; wbvector<SPIDX_T> S(2); S[0]=s1; S[1]=s2;
993     reshape(S,X); return X.save2(*this);
994 };
995
996 wbsparray& Reshape(SPIXD_T s1, SPIDX_T s2, SPIDX_T s3){
997     wbsparray X; wbvector<SPIDX_T> S(3); S[0]=s1; S[1]=s2; S[2]=s3;
998     reshape(S,X); return X.save2(*this);
999 };
1000
1001 wbvector<TD>& getCol(SPIXD_T k, wbvector<TD> &v) const;
1002 wbvector<TD>& getRow(SPIXD_T k, wbvector<TD> &v) const;

```



```

1003
1004 wbsparray& getBlock(const char *F, int L,
1005     SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2,
1006     wbsparray<TD> &B
1007 ) const;
1008
1009 wbsparray& getCol(SPIDX_T k, wbsparray<TD> &b) {
1010     if (k>=dim0(k)) wblog(FL,"ERR %s() "
1011         "col-index out of bounds (%ld / %s)",FCT,k,SSTR_(this));
1012     return getBlock(FL,0,-1,k,k);
1013 };
1014
1015 wbsparray& splitBlock(const char *F, int L,
1016     sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2,
1017     wbsparray<TD> &B
1018 );
1019
1020 wbsparray& SplitQR(sSPIDX_T k, sSPIDX_T k_, wbsparray<TD> &X) {
1021     save2(X); X.SplitBlock(FL,k,-1,k_,-1,*this);
1022     return *this;
1023 };
1024
1025 TD getHouseholderVec(const char *F, int L,
1026     SPIDX_T k, SPIDX_T j, wbsparray<TD> &b,
1027     TD eps2=0
1028 ) const;
1029
1030 wbsparray& IDX_Shift(sSPIDX_T i1, sSPIDX_T i2);
1031
1032 wbsparray<TD>& LProject1(
1033     const wbsparray<TD> &b, wbsparray<TD> &X,
1034     TD xfac=1, char tnorm=0) const;
1035
1036 SPIDX_T LContractVec_omp(
1037     const wbsparray<TD> &b, wvector<TD> &x, char tnorm) const;
1038
1039 wbsparray& splitLast(SPIDX_T k, wbsparray &a) const;
1040
1041 wbsparray& select0(
1042     const WBINDEX &,
1043     unsigned dim,
1044     wbsparray&
1045 ) const;
1046
1047 wbsparray& Select0(const WBINDEX &I, unsigned dim) {
1048     wbsparray X; save2(X); X.select0(I,dim,*this);
1049     return *this;
1050 };
1051
1052 wbsparray& Permute(const char* s, char iflag=0);
1053 wbsparray& Permute(const wbperm&, char iflag=0);
1054 wbsparray& permute(const char* s, wbsparray&, char iflag=0) const;
1055 wbsparray& permute(const wbperm&, wbsparray&, char iflag=0) const;
1056
1057 wbsparray& transpose(const char *F, int L, wbsparray &A) const;
1058 wbsparray& Transpose(const char *F=0, int L=0) {
1059     wbsparray X; transpose(F,L,X); X.save2(*this);
1060     return *this;
1061 };
1062
1063 wbsparray& htranspose(const char *F, int L, wbsparray &A) const {
1064     transpose(F,L,A); A.D.Conj();
1065     return A;
1066 };
1067 wbsparray& hTranspose(const char *F=0, int L=0) {
1068     wbsparray X; htranspose(F,L,X); X.save2(*this);
1069     return *this;
1070 };
1071
1072 wbsparray& ColPermute(const wbperm &P, char iflag=0);
1073 wbsparray& MatPermute(const wbperm &P, char iflag=0);
1074
1075 wbsparray& mtimes(
1076     const char *F, int L, const wbsparray&B, wbsparray&C,
1077     char sflag=0, const TD eps=0
1078 ) {
1079     const wbsparray &A=*this;
1080     if (A.rank()!=2 || B.rank()!=2) wblog(FL,
1081         "ERR %s() requires rank-2 wbsparrays (%s * %s)",
1082         FCT,A.sizeStr().data,B.sizeStr().data);
1083     wbindx ia(1), ib(1); ia[0]=1; ib[0]=0;
1084
1085     return contract(F,L,ia,B,ib,C,wbperm(),1,0,sflag,eps);
1086 };
1087
1088 char contract_scalar(
1089     const char* F, int L, const ctrIdx &ica,

```

```

1090     const wbsparray &B, const ctrIdx &icb, wbsparray &C,
1091     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
1092 ) const;
1093
1094 wbsparray& contract_diag_diag(
1095     const char* F, int L, const ctrIdx &ica,
1096     const wbsparray &B, const ctrIdx &icb, wbsparray &C,
1097     const wbperm &pfinal, const TD& afac, const TD& cfac
1098 ) const;
1099
1100 char contract_check_2full(
1101     const char* F, int L, const ctrIdx &ica,
1102     const wbsparray &B, const ctrIdx &icb, wbsparray &C,
1103     const wbperm &pfinal, const TD& afac, const TD& cfac,
1104     char flag, TD eps, SPIDX_T *nnz_C=NULL
1105 ) const;
1106
1107 wbsparray& contract(const char *F, int L, const ctrIdx &ica,
1108     const wbsparray &B, const ctrIdx &icb, wbsparray &C,
1109     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.,
1110     char sflag=0, const TD eps=0
1111 ) const;
1112
1113 wbsparray& contract(const char *F, int L, const char *ica,
1114     const wbsparray &B, const char *icb, wbsparray &C,
1115     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.,
1116     char sflag=0, const TD eps=0
1117 ) const {
1118
1119     return contract(F,L,
1120         ctrIdx(FL,ica), B, ctrIdx(FL,icb), C,
1121         pfinal,afac,cfac,sflag,eps
1122     );
1123 };
1124
1125 wbararray<TD>& contract(const char* F, int L, const ctrIdx &ica,
1126     const wbsparray &B, const ctrIdx &icb, wbararray<TD> &C,
1127     const wbperm &pfinal, const TD& afac, const TD& cfac
1128 ) const;
1129
1130 wbsparray& contract(const char *F, int L,
1131     unsigned il, const wvector<TD> &B, wbsparray &C
1132 ) const;
1133
1134 wbsparray& comm(const char *F, int L,
1135     const wbsparray &B, wbsparray &C,
1136     char aflag='N', char bflag='N'
1137 ) const;
1138
1139 wbsparray& tensorProdX(
1140     const char *F, int L, const wbsparray& B, wbsparray& C,
1141     char conja=0, const TD &afac=1, char conjb=0, const TD &cfac=0,
1142     const wbperm *Pfinal=NULL
1143 ) const;
1144
1145 wbsparray& tensorProd(
1146     const char *F, int L, const wbsparray& B, wbsparray& C,
1147     char aflag='N', char bflag='N', char kflag=0
1148 ) const;
1149
1150 wbsparray& Kron(const wbsparray& B,
1151     char aflag='N', char bflag='N', char kflag='k'
1152 ) {
1153     wbsparray X; tensorProd(FL,B,X,aflag,bflag,kflag);
1154     return X.save2(*this);
1155 };
1156
1157 wbsparray& kron(
1158     const char *F, int L, const wbsparray& B, wbsparray& C,
1159     char aflag='N', char bflag='N', char kflag='k'
1160 ) const {
1161     return tensorProd(F,L,B,C,aflag,bflag,kflag);
1162 };
1163
1164 mxArray* toMxSp() const;
1165
1166 mxArray* toMx(int m=-99) const {
1167     unsigned r=rank(FL);
1168
1169     if (m==-99 && r<=2 && Wb::isBaseType(typeid(TD))) {
1170         return toMxSp();
1171     }
1172
1173     mxArray *a=mxCreateStruct(1,1);
1174     add2MxStruct(a,0,m);
1175     return a;
1176 };

```

```

1177
1178     mxArray* IDtoMx() const;
1179
1180     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
1181     void add2MxStruct(mxArray *a, unsigned i, int ma=-99) const;
1182
1183     mxArray* mxCreateCell(unsigned m, unsigned n) const;
1184     void add2MxCell(mxArray *a, unsigned i) const;
1185
1186     mxArray* toMx0(int m=-99) const {
1187         mxArray *a=mxCreateStructX(1,1,m);
1188         add2MxStructX(a,0,m);
1189         return a;
1190     };
1191     mxArray* mxCreateStructX(unsigned m, unsigned n, int ma=-99) const;
1192     void add2MxStructX(mxArray *a, unsigned i, int ma=-99) const;
1193
1194     mxArray* toMX() const {
1195         mxArray *a=mxCreateSTRUCT(1,1);
1196         return add2MxSTRUCT(a,0);
1197     };
1198     mxArray* mxCreateSTRUCT(unsigned m, unsigned n) const;
1199     mxArray* add2MxSTRUCT(mxArray *a, unsigned i) const;
1200
1201     void put(const char *vname, unsigned ma=-99, const char *ws="caller"
1202 ) const { put(0,0,vname,ma,ws); };
1203
1204     void put0(const char *F, int L,
1205         const char *vname, const char *ws="caller"
1206 ) const { put(F,L,vname,-99,ws,'X'); };
1207
1208     void put(const char *F, int L, const char *vname,
1209         unsigned ma=-99, const char *ws="caller", char flag=0
1210 ) const {
1211         mxArray *a=0; {
1212             if (!flag) a=toMx(ma); else
1213             if (flag=='X') a=toMX(); else
1214             if (flag=='0') a=toMx0(); else
1215             if (flag=='') a=toMx0(); else
1216             wblog(FL,"ERR %s() invalid flag=%s",FCT,Wb::char2Str(flag).data);
1217         }
1218         int i=mexPutVariable(ws,vname,a);
1219
1220         if (i) wblog(F_L,
1221             "ERR failed to write variable '%s' (%d)",vname,i);
1222         if (F) wblog(F,L,
1223             "I/O putting variable '%s' to %s",vname,ws);
1224         mxDestroyArray(a);
1225     };
1226
1227     wbvector<SPIDX_T> SIZE;
1228     wbMatrix<SPIDX_T> IDX;
1229
1230     wbvector<TD> D;
1231
1232     char isref;
1233
1234 protected:
1235
1236     unsigned get2DSize(
1237         const char *F, int L,
1238         SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const;
1239
1240     unsigned get2DIndex(
1241         wbMatrix<SPIDX_T> &IJ, SPIDX_T &d1, SPIDX_T &d2,
1242         int m=-1) const;
1243
1244 private:
1245
1246     TD dotProd_full_diag(const wbsparray &B, char cflag) const;
1247
1248     wbsparray& timesEl_full_diag(
1249         const wbsparray &B, wbsparray &X, char cflag
1250 ) const;
1251
1252 };
1253
1254 template<class TD> inline
1255 wbsparray<TD> operator+(const wbsparray<TD> &A) { return A; };
1256
1257 template<class TD> inline
1258 wbsparray<TD> operator-(const wbsparray<TD> &A) {
1259     wbsparray<TD> X(A); TD *x=X.D.data;
1260     for (SPIDX_T n=X.D.len, i=0; i<n; ++i) { x[i]=-x[i]; }
1261 }

```

```

1264     return X;
1265 };
1266
1267 template<class TD>
1268 class indexSparseRef {
1269
1270 public:
1271
1272     indexSparseRef(const wbspararray<TD> &A_, unsigned M2=1)
1273         : M(0), A(NULL) { init(FL,A_,M2); };
1274
1275     indexSparseRef(
1276         const char *F, int L, const wbspararray<TD> &A_, unsigned M2=1)
1277         : M(0), A(NULL) { init(F_L,A_,M2); };
1278
1279     indexSparseRef& init(const char *F, int L,
1280         const wbspararray<TD> &A_, unsigned M2=1);
1281
1282     SPIDX_T numCols() const;
1283     SPIDX_T numRows() const;
1284     SPIDX_T nnz(SPIDX_T k) const;
1285
1286     TD LContractVec(
1287         const wbspararray<TD> &b, SPIDX_T k, char tnorm=0);
1288
1289     mxArray* toMx() const;
1290
1291     unsigned M;
1292
1293     wbvector<SPIDX_T> JA;
1294
1295     wbvector<sSPIDX_T> JJ;
1296
1297     const wbspararray<TD> *A;
1298
1299 protected:
1300 private:
1301
1302 };
1303
1304 template <class TD>
1305 indexSparseRef<TD>& indexSparseRef<TD>::init(
1306     const char *F, int L, const wbspararray<TD> &A_, unsigned M2
1307 ){
1308     unsigned i, r=A_.rank(FL);
1309
1310     if (r<2 || A_.SIZE.len!=r || A_.IDX.dim2!=r) wblog(FL,
1311         "ERR %s() got empty or diagonal wbspararray !? (%s)",FCT,SSTR(A_));
1312
1313     if (!M2 || M2>=r) wblog(FL,
1314         "ERR %s() M2 out of bounds (%d/%d)",FCT,M2,r);
1315
1316     M=r-M2; A=(&A_);
1317
1318     const SPIDX_T *s=A_.SIZE.data, *idx=A_.IDX.data;
1319     SPIDX_T l, n=A_.IDX.dim1, D1=s[0], D2=s[M];
1320
1321     sSPIDX_T j,I,J, Ip=-1, Jp=-1;
1322
1323     for (i=1; i<M; ++i) D1*=s[i];
1324     for (++i; i<r; ++i) D2*=s[i];
1325
1326     #ifndef WB_SKIP_ASSERT
1327     { double d1=s[0], d2=s[M];
1328       for (i=1; i<M; ++i) d1*=s[i];
1329       for (++i; i<r; ++i) d2*=s[i];
1330       if ((1+ceil(log2(d1))) >= 8*sizeof(SPIDX_T) ||
1331           (1+ceil(log2(d2))) >= 8*sizeof(SPIDX_T)) wblog(FL,"ERR %s() "
1332               "got possible indexSparseRef overrun (%.3g, %.3g / %.3g)",
1333               FCT, log2(d1), log2(d2), 8*sizeof(SPIDX_T)
1334           );
1335     }
1336     #endif
1337
1338     if (!D1 || !D2) wblog(FL,"ERR %s() got empty wbspararray !?",FCT);
1339
1340     JA.init(D2+1);
1341     JJ.init2val(MIN(D1,D2),-1);
1342
1343     for (l=0; l<n; ++l, idx+=r) {
1344         I=Wb::sub2ind(s,idx,M); J=Wb::sub2ind(s+M,idx+M,M2);
1345
1346         if (J>Jp) { for (j=Jp+1; j<=J; ++j) JA[j]=1; }
1347         else {
1348             bool i=(J==Jp); if ((i && I<=Ip) || !i) wblog(FL,
1349                 "ERR %s() wbspararray not sorted/col-major !?",FCT);
1350         }
1351     }

```

```

1351
1352     if (I==J) JJ[J]=1;
1353
1354     Ip=I; Jp=J;
1355 }
1356
1357 J=D2; if (J>Jp) { for (j=Jp+1; j<=J; ++j) JA[j]=1; }
1358
1359 return *this;
1360 };
1361
1362 template <class TD> inline
1363 SPIDX_T indexSparseRef<TD>::numCols() const {
1364     if (!A || JA.len<1) wblog(FL,
1365         "ERR %s() not yet initialized !?",FCT);
1366     return (JA.len-1);
1367 };
1368
1369 template <class TD> inline
1370 SPIDX_T indexSparseRef<TD>::numRows() const {
1371     if (!A || !M || A->SIZE.len<=M) wblog(FL,
1372         "ERR %s() got uninitialized object !?",FCT);
1373     const SPIDX_T *S=A->SIZE.data; SPIDX_T D1=S[0];
1374     for (unsigned i=1; i<M; ++i) D1*=S[i];
1375     return D1;
1376 };
1377
1378 template <class TD> inline
1379 SPIDX_T indexSparseRef<TD>::nnz(SPIDX_T k) const {
1380     if (k+1>=JA.len) wblog(FL,
1381         "ERR %s() index out of bounds (%d/%d) !?",FCT,k,JA.len);
1382     return (JA[k+1]-JA[k]);
1383 };
1384
1385 template <class TD> inline
1386 TD indexSparseRef<TD>::LContractVec(
1387     const wbsparray<TD> &B, SPIDX_T k, char tnorm
1388 ){
1389     TD x=0;
1390
1391     if (!A) wblog(FL,"ERR %s() got empty A !?",FCT);
1392     if (A->SIZE.len!=2 || M!=1) wblog(FL,"ERR %s() "
1393         "only applies to rank-2 objects (%s; %d)",FCT,SSTR_(A),M);
1394     if (k+1>=JA.len) wblog(FL,
1395         "ERR %s() index out of bounds (%d/%d) !?",FCT,k,JA.len);
1396
1397     SPIDX_T d1=A->SIZE[0], d2=A->SIZE[1];
1398     SPIDX_T i=0, i2=B.IDX.dim1, j=JA[k], j2=JA[k+1];
1399
1400     if (k>=d2) wblog(FL,
1401         "ERR %s() index out of bounds (%d/%d) !?",FCT,k,d2);
1402     if (B.SIZE.len!=1 || B.SIZE[0]!=d1) wblog(FL,
1403         "ERR %s() size mismatch (%s <> %s)",FCT,SSTR_(A),SSTR_(B));
1404     if (j==j2) return x;
1405
1406     SPIDX_T r=2, jr=j*r;
1407     char cflag = ((tnorm || WbUtil<TD>::hasNoConj()) ? 0 : 1);
1408     const SPIDX_T *I=B.IDX.data, *J=A->IDX.data;
1409     const TD *a=A->D.data, *b=B.D.data;
1410
1411     while (i<i2 && j<j2) {
1412         if (I[i]<J[jr]) { ++i; } else
1413         if (J[jr]<I[i]) { ++j; jr+=r; } else {
1414             if (cflag)
1415                 { x+=CONJ(b[i])*a[j]; }
1416             else { x+= b[i] *a[j]; }
1417
1418             ++i; ++j; jr+=r;
1419
1420             if (i<i2 && I[i]<=I[i-1]) wblog(FL,
1421                 "ERR %s() A: input sparse IDX not sorted !?",FCT);
1422             if (j<j2 && J[jr]<=J[jr-r]) wblog(FL,
1423                 "ERR %s() B: input sparse IDX not sorted !?",FCT);
1424         }
1425     }
1426
1427     return x;
1428 };
1429
1430 template <class TD>
1431 mxArray* indexSparseRef<TD>::toMx() const {
1432
1433     if (A==NULL) wblog(FL,"ERR %s() A not yet initialized !?",FCT);
1434
1435     const char *fields[]={"SIZE","M","JA","JJ"};
1436     mxArray *a=mxCreatStructMatrix(1,1,4,fields);
1437

```

```

1438     mxSetFieldByNumber(a,0, 0, A->SIZE.toMx());
1439     mxSetFieldByNumber(a,0, 1, numtoMx(M));
1440     mxSetFieldByNumber(a,0, 2, JA.toMx());
1441     mxSetFieldByNumber(a,0, 3, JJ.toMx());
1442
1443     return a;
1444 };
1445
1446 template<class TD>
1447 class iterSparseCMref {
1448
1449 public:
1450
1451     iterSparseCMref(const char *F, int L, wbspararray<TD> &A);
1452
1453     SPIDX_T next(wbspararray<TD> &a) {
1454         if ((++it)+1>=I0.len) { it=-1; return (it+1); }
1455         init(it,a,'r'); return (it+1);
1456     };
1457
1458     wbspararray<TD>& getIter(SPIDX_T i, wbspararray<TD> &a, char ref=0) const;
1459     TD norm2(SPIDX_T i, char tnorm=0) const;
1460
1461     SPIDX_T numIter() const {
1462         if (!IX.len) wblog(FL,
1463             "ERR %s() got non-initialized/empty object",FCT);
1464         return IX.len-1;
1465     };
1466
1467     bool gotLast() const {
1468         if (sSPIDX_T(it)<0 || it+1>=IX.len) wblog(FL,
1469             "ERR %s() not within loop (%d/%d)",FCT,it,IX.len-1);
1470         return (it+2==IX.len);
1471     };
1472
1473     SPIDX_T index() const {
1474         if (sSPIDX_T(it)<0 || it+1>=IX.len) wblog(FL,
1475             "ERR %s() not within loop (%d/%d)",FCT,it,IX.len-1);
1476         return IX[it];
1477     };
1478
1479     mxArray* toMx() const;
1480
1481     SPIDX_T it;
1482
1483     wbvector<SPIDX_T> I0;
1484
1485     wbvector<SPIDX_T> IX;
1486
1487     wbMatrix<SPIDX_T> IDX;
1488
1489 protected:
1490 private:
1491
1492     unsigned r;
1493     const SPIDX_T *S;
1494     const TD* data;
1495 };
1496
1497 template <class TD>
1498 mxArray* iterSparseCMref<TD>::toMx() const {
1499
1500     const char *fields[]={"SIZE","r","it","I0","IX","IDX","data"};
1501     mxArray *a=mxCreateStructMatrix(1,1,7,fields);
1502
1503     mxSetFieldByNumber(a,0, 0, wbvector<SPIDX_T>(r,S,'r').toMx());
1504     mxSetFieldByNumber(a,0, 1, numtoMx(r));
1505     mxSetFieldByNumber(a,0, 2, numtoMx(it));
1506     mxSetFieldByNumber(a,0, 3, I0.toMx());
1507     mxSetFieldByNumber(a,0, 4, IX.toMx());
1508     mxSetFieldByNumber(a,0, 5, IDX.toMx());
1509     mxSetFieldByNumber(a,0, 6, wbvector<TD>(IDX.dim1,data,'r').toMx());
1510
1511     return a;
1512 };
1513
1514 template <class TD>
1515 iterSparseCMref<TD>::iterSparseCMref(
1516     const char *F, int L, wbspararray<TD> &A) : r(A.SIZE.len) {
1517
1518     unsigned l=r-1; SPIDX_T D;
1519     if (r<2 || !(D=A.SIZE.end())) wblog(F_L,
1520         "ERR %s() got %s (%d)",FCT,A.sizeStr().data,D);
1521
1522     A.Compress(F_L);
1523     S=A.SIZE.data; data=A.D.data; it=-1;
1524

```

```

1525     if (A.IDX.dim1!=A.D.len || A.SIZE.len!=A.IDX.dim2)
1526         A.spererror_this(F_LF);
1527
1528     if (A.IDX.dim1==0) {
1529         IO.init(1);
1530         IX.init(1); IX[0]=D;
1531         IDX.init(0,r-1); return;
1532     }
1533     if (A.IDX.dim1==1) {
1534         IO.init(2); IO[1]=1;
1535         IX.init(2); IX[0]=A.IDX(0,1); IX[1]=D;
1536         IDX.init2ref(1,1,A.IDX.data);
1537         return;
1538     }
1539
1540     SPIDX_T i0=0, i=1, k=1, *I=A.IDX.ref(1,1);
1541     const int d2=A.IDX.dim2;
1542     wbvector<SPIDX_T> Dg;
1543
1544     for (k=1; i<A.IDX.dim1; ++i, I+=d2) { if (I[0]!=I[-d2]) ++k; }
1545     Dg.init(k); IX.init(k+1);
1546
1547     k=0; I=A.IDX.ref(0,1);
1548     IX[0]=I[0]; I+=d2;
1549
1550     for (i=1; i<A.IDX.dim1; ++i, I+=d2) {
1551         if (I[0]!=I[-d2]) { Dg.data[k++]=i-i0; i0=i; IX[k]=I[0]; }
1552     }
1553     Dg.data[k++]=i-i0; IX[k]=D;
1554
1555     Dg.cumsum_(IO,'N');
1556     A.IDX.getCols(0,A.IDX.dim2-2,IDX);
1557 };
1558
1559 template <class TD>
1560 wbsparray<TD>& iterSparseCMref<TD>::getIter(
1561     SPIDX_T i, wbsparray<TD> &a, char ref) const {
1562
1563     if (i+1>=IO.len) wblog(FL,
1564         "ERR %s() index out of bounds (%d/%d)",FCT,i,IO.len);
1565
1566     SPIDX_T i0=IO[i], n=IO[i+1]-i0;
1567     if (sSPIDX_T(n)<=0) wblog(FL,"ERR %s() got IO(%d:%d) !?",FCT,i0,i0+n);
1568
1569     if (ref=='r' || ref=='R') {
1570         a.SIZE.init2ref(r-1,S);
1571         a.D.init2ref(n, data+i0);
1572         a.IDX.init2ref(n,IDX.dim2,IDX.rec(i0));
1573         a.isref=1;
1574     }
1575     else {
1576         a.SIZE.init(r-1,S);
1577         a.D.init(n, data+i0);
1578         a.IDX.init(n,IDX.dim2,IDX.rec(i0));
1579     }
1580
1581     return a;
1582 };
1583
1584 template <class TD>
1585 TD iterSparseCMref<TD>::norm2(SPIDX_T i, char tnorm) const {
1586
1587     if (i+1>=IO.len) wblog(FL,
1588         "ERR %s() index out of bounds (%d/%d)",FCT,i,IO.len);
1589
1590     SPIDX_T i0=IO[i], n=IO[i+1]-i0;
1591
1592     return Wb::overlap(data+i0,data+i0,n,1,tnorm);
1593 };
1594
1595 template <class T>
1596 inline int checkContract(const char* F, int L,
1597     const wbvector<T> &SA, const ctrIdx &ica,
1598     const wbvector<T> &SB, const ctrIdx &icb,
1599     const wbpem *pfinal=NULL, wbvector<T> *SC=NULL
1600 );
1601
1602 template <class TD>
1603 wbararray<TD>& contract(const char *F, int L,
1604     const wbsparray<TD>&, const wbindex&,
1605     const wbararray<TD>&, const wbindex&, wbararray<TD>&,
1606     const wbpem &pfinal=wbpem(), const TD& afac=1., const TD& cfac=0.
1607 );
1608
1609 template <class TD>
1610 wbararray<TD>& contract(const char *F, int L,
1611     const wbararray<TD>&, const wbindex&,

```

```

1612     const wbsparray<TD>&, const wbindex&, wbararray<TD>&,
1613     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
1614 );
1615
1616 namespace Wb {
1617
1618 template <class TD>
1619 TD VMatVprod(
1620     const wbsparray<TD> &v1, const wbsparray<TD> &M,
1621     const wbsparray<TD> &v2
1622 );
1623
1624 template <class TD>
1625 wbsparray<TD>& MatProd(
1626     const wbsparray<TD> &A, const wbsparray<TD> &B, wbsparray<TD> &C,
1627     char aflag='N', char bflag='N', TD afac=TD(1.), TD cfac=TD(0.)
1628 );
1629
1630 };
1631
1632 template <class TD> inline
1633 wbararray<TD>& contract(const char *F, int L,
1634     const wbsparray<TD> &A, const char* s1,
1635     const wbsparray<TD> &B, const char* s2, wbsparray<TD> &C,
1636     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
1637 ){
1638     return
1639     A.contract(F_L, wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
1640 };
1641
1642 template <class TD> inline
1643 wbararray<TD>& contract(const char *F, int L,
1644     const wbsparray<TD> &A, const char* s1,
1645     const wbararray<TD> &B, const char* s2, wbararray<TD> &C,
1646     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
1647 ){
1648     return
1649     contract(F,L, A,wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
1650 };
1651
1652 template <class TD> inline
1653 wbararray<TD>& contract(const char *F, int L,
1654     const wbararray<TD> &A, const char* s1,
1655     const wbsparray<TD> &B, const char* s2, wbararray<TD> &C,
1656     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
1657 ){
1658     return
1659     contract(F,L, A,wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
1660 };
1661
1662 template<class TD> inline
1663 void wbsparray<TD>::info(const char* F, int L, const char* istr) const {
1664
1665     SPIIDX_T nz=D.len; checkSize(F_L);
1666
1667     if (!D.len) {
1668         if (SIZE.len) {
1669             wblog(F_L,"sparse %3s %12s (all zero)",
1670                 istr && istr[0] ? istr:"", SIZE.toStrD().data);
1671         }
1672         else {
1673             wblog(F_L,"sparse %3s (empty)",
1674                 istr && istr[0] ? istr:"", SIZE.toStrD().data);
1675         }
1676         return;
1677     }
1678
1679     if (isDiag(F_L)) {
1680         if (nz>4) {
1681             wblog(F_L,"sparse %3s %12s (diag)",
1682                 istr && istr[0] ? istr:"", SIZE.toStrD().data);
1683         }
1684         else if (nz>1) {
1685             wblog(F_L,"sparse %3s %8sdiag( [%s] )",
1686                 istr && istr[0] ? istr:"", SIZE.toStrD().data, STR(D));
1687         }
1688         else {
1689             wblog(F_L,"sparse %3s %5s(scalar) %-8.4g",
1690                 istr && istr[0] ? istr:"", SIZE.toStrD().data, double(D[0]));
1691         }
1692         return;
1693     }
1694
1695     if (nz>4) {
1696         wblog(F_L,"sparse %3s %12s (nz=%d @ %-6.3g)",
1697             istr && istr[0] ? istr:"", SIZE.toStrD().data, nnz(), sparsity());
1698     }

```



```

1699     else {
1700         wblog(F_L,"sparse %3s %12s {%s}",
1701             istr && istr[0] ? istr:"", SIZE.toStrD().data, D.toStrf("%.4g"," ", "").data);
1702     }
1703 };
1704
1705 template<class TD> inline
1706 void wbsparray<TD>::printdata(const char *istr, const char *fmt0) const {
1707
1708     SPIDX_T i; const char *fmt; wstring FMT;
1709
1710     if (fmt0 && fmt0[0]) fmt=fmt0;
1711     else { FMT.init2Fmt((double)0); fmt=FMT.data; }
1712
1713     checkSize(FLF);
1714
1715     if (isScalar()) {
1716         printf("%s = %4.8g (scalar)\n",istr,D.len ? double(D[0]) : 0);
1717     }
1718     else if (isDiag()) {
1719         printf("%s = diag([",istr);
1720         for (i=0; i<D.len; ++i) printf(" %4.8g",double(D[i]));
1721         printf(" ] (D=%ld)\n",D.len);
1722     }
1723     else if (SIZE.len==1) {
1724         printf("%s = [",istr);
1725         for (i=0; i<IDX.dim1; ++i) {
1726             printf(" %d %8g;",IDX(i,1)+1,double(D[i]));
1727         }
1728         printf(" ] (vector, D=%ld)\n",SIZE[0]);
1729     }
1730     else if (SIZE.len==2) {
1731         printf("%s = [ %d x %d ]\n",istr,SIZE[0],SIZE[1]);
1732         for (i=0; i<IDX.dim1; ++i) {
1733             printf(" %5d %4d %4.8g\n",IDX(i,0)+1,IDX(i,1)+1,double(D[i]));
1734         }
1735         printf("];\n");
1736     }
1737     else {
1738         wblog(FL,"ERR %s() only applies to rank<=2 (%s)",
1739             FCT,sizeStr().data);
1740     }
1741 };
1742
1743 template<class TD> inline
1744 unsigned wbsparray<TD>::get2DSize(
1745     const char *F, int L,
1746     SPIDX_T &d1, SPIDX_T &d2, int m_,
1747     char fflag
1748 ) const {
1749
1750     const SPIDX_T* const s=SIZE.data;
1751     unsigned m=0, i=0, r=rank(F_L);
1752
1753     if (isDiag()) wblog(FL,"ERR %s() got DIAG_REP",FCT);
1754
1755     if (!r) { d1=d2=0; return 0; }
1756
1757     if (m_<0) {
1758         if (r%2 && fflag) wblog(F,L,
1759             "ERR sparse::%s() even rank or 2nd argument required (%s)",
1760             FCT,sizeStr().data);
1761         else {
1762             SPIDX_T dm=SIZE[0], d=std::sqrt(double(numel()));
1763             for (m=1; m<SIZE.len; ++m) {
1764                 if (dm>=d) break; else dm*=SIZE[m];
1765             }
1766         }
1767     }
1768     else { m=m_;
1769         if (m>r) wblog(F,L,
1770             "ERR sparse::%s() invalid m=%d (%s)",FCT,m,sizeStr().data);
1771     }
1772
1773     if (m>0) { for (d1=s[i++]; i<m; ++i) { d1*=s[i]; } } else d1=1;
1774     if (m<r) { for (d2=s[i++]; i<r; ++i) { d2*=s[i]; } } else d2=1;
1775     return m;
1776 };
1777
1778 template<class TD> inline
1779 unsigned wbsparray<TD>::get2DIndex(
1780     wbMatrix<SPIDX_T> &IJ, SPIDX_T &d1, SPIDX_T &d2, int m_
1781 ) const {
1782
1783     if (isDiag(FL) || !D.len) {

```

```

1786         d1=d2=D.len; IJ.init(d1,2);
1787         for (SPIDX_T *I=IJ.data, i=0; i<d1; ++i, I+=2) { I[0]=I[1]=i; }
1788         return 1;
1789     }
1790     if (!IDX.dim2) sperror_this(FLF);
1791
1792     SPIDX_T i,j,k, n=IDX.dim1;
1793     unsigned r=rank(FL), l=r-1, m=get2DSize(FL,d1,d2,m_);
1794
1795     IJ.init(IDX.dim1,2);
1796
1797     if (!d1 || !d2 || !r) {
1798         if (IJ.data || D.data) wblog(FL,
1799             "ERR %s() got d1=%d, d2=%d, yet S=[%s] (%d) !?",
1800             FCT,d1,d2,STR(SIZE),D.len);
1801         return m;
1802     }
1803
1804     SPIDX_T *ij=IJ.data; char e=0;
1805     const SPIDX_T *idx=IDX.data, *s=SIZE.data;
1806
1807     if (!m || m==r) {
1808         for (i=0; i<n; ++i, ij+=2, idx+=r) {
1809             for (j=1, k=idx[j--]; j<r; --j) { k=k*s[j]+idx[j]; };
1810             if (m==r)
1811                 { ij[0]=k; if (k>=d1) { e=1; break; } }
1812             else { ij[1]=k; if (k>=d2) { e=2; break; } }
1813         }
1814     }
1815     else {
1816         for (i=0; i<n; ++i, ij+=2, idx+=r) {
1817             j=1;
1818
1819             for (k=idx[j--]; j>=m; --j) { k=k*s[j]+idx[j]; };
1820             ij[1]=k;
1821
1822             for (k=idx[j--]; j<r; --j) { k=k*s[j]+idx[j]; };
1823             ij[0]=k;
1824
1825             if (ij[0]>=d1 || ij[1]>=d2) { e=3; break; }
1826         }
1827     }
1828
1829     if (e) wblog(FL,
1830         "ERR %s() index out of bounds (%d: (%d,%d) %dx%d; %s: %d; %d)",
1831         FCT, i+1, ij[0]+1, ij[1]+1, d1,d2,sizeStr().data,m, e
1832     );
1833
1834     return m;
1835 };
1836
1837 template<class TD> inline
1838 wbsparray<TD>& wbsparray<TD>::initIdentity(
1839     const wbvector<SPIDX_T> &S0,
1840     int m,
1841     sSPIDX_T id,
1842     char fflag
1843 ){
1844     SPIDX_T d1=1,d2=1;
1845     wbindx I(S0.len); SIZE=S0; init(S0);
1846
1847     get2DSize(FL,d1,d2,m);
1848
1849     if (d1 && d2) { SPIDX_T i=0, n=0;
1850         if (id==0) {
1851             n=MIN(d1,d2); init_nnz(n); D.set(1);
1852             for (; i<n; ++i) ind2sub(i+d1*i,IDX.rec(i));
1853         }
1854         else if (id>0) { SPIDX_T k=id; if (k<d2) {
1855             n=MIN(d1,d2-k); init_nnz(n); D.set(1);
1856             for (; i<n; ++i) ind2sub(i+d1*(i+k),IDX.rec(i));
1857         } }
1858         else if (id<0) { SPIDX_T k=-id; if (k<d1) {
1859             n=MIN(d1-k,d2); init_nnz(n); D.set(1);
1860             for (; i<n; ++i) ind2sub((i+k)+d1*i,IDX.rec(i));
1861         } }
1862     }
1863
1864     if (!D.len && fflag)
1865         wblog(FL,"ERR sparse::%s() to empty (k=%d given %s)",
1866             FCT, id, sizeStr().data
1867     );
1868
1869     return *this;
1870 };
1871
1872 template<class TD> inline

```

```

1873 wbsparray<TD>& wbsparray<TD>::initIdentityB3(
1874     SPIDX_T d1, SPIDX_T d2, SPIDX_T D,
1875     sSPIDX_T i0,
1876     char fflag
1877 ){
1878     SPIDX_T s[3]={d1,d2,D}; SIZE.init(3,s);
1879     return initIdentity(SIZE,2,i0,fflag);
1880 };
1881
1882 template<class TD>
1883 class sparseCollector2D {
1884
1885 public:
1886
1887     sparseCollector2D(SPIDX_T m=0, SPIDX_T n=0)
1888         : M(m), N(n), nnz(0), NIJ(NULL), data(NULL) { SPIDX_T MN=M*N;
1889         if (MN) {
1890             WB_NEW(data,MN); MEM_SET<TD>(data,MN);
1891             WB_NEW(NIJ,MN); memset(NIJ,0,MN*sizeof(SPIDX_T));
1892         }
1893     };
1894
1895     ~sparseCollector2D() { clear(); }
1896
1897     void clear() {
1898         if ((data==NULL) ^ (NIJ==NULL)) wblog(FL,
1899             "ERR %s() got partial initialization (%lX, %lX) !?",FCT,data,NIJ);
1900         if (data) { WB_DELETE(data); }
1901         if (NIJ) { WB_DELETE(NIJ); }
1902         M=N=nnz=0;
1903     };
1904
1905     void init(SPIDX_T m=0, SPIDX_T n=0) { SPIDX_T MN=M*N;
1906         clear(); M=m; N=n; nnz=0;
1907         if (MN) {
1908             WB_NEW(data,MN); MEM_SET<TD>(data,MN);
1909             WB_NEW(NIJ,MN); memset(NIJ,0,MN*sizeof(SPIDX_T));
1910         }
1911     };
1912
1913     sparseCollector2D& add(
1914         const char *F, int L, SPIDX_T i, SPIDX_T j, const TD& a
1915     ){
1916         if (i>=M || j>=N || !data) wblog(F,L,
1917             "ERR %s() index out of bounds (%d,%d)/(%dx%d) [%lX]",
1918             FCT,i,j,M,N,data);
1919         SPIDX_T k=i*N+j;
1920         data[k]+=a; if ((++NIJ[k])==1) ++nnz;
1921         return *this;
1922     };
1923
1924     void getData(wbMatrix<SPIDX_T> &IJ, wbvector<TD> &X) {
1925         SPIDX_T i,j, k=0, l=0;
1926         IJ.init(nnz,2); X.init(nnz);
1927
1928         for (i=0; i<M; ++i)
1929             for (j=0; j<N; ++j, ++l) if (data[l]!=0) {
1930                 if (k>=nnz || !NIJ[l]) wblog(FL,"ERR %s() "
1931                     "possibly incorrect nnz (%g/%g; %d) !?",FCT,k,nnz,NIJ[l]);
1932                 X.data[k]=data[l];
1933                 IJ(k,0)=i; IJ(k,1)=j; ++k;
1934             }
1935         if (nnz && k<nnz) {
1936             if (k) { IJ.dim1=k; X.len=k; }
1937             else {
1938                 wblog(FL,"WRN %s() got all-zero data",FCT);
1939                 IJ.init(0,2); X.init();
1940             }
1941         }
1942     };
1943
1944     mxArray* toMx() const {
1945         const char *fields[]={ "M","N","nnz","data","NIJ" };
1946         mxArray *S=mxCreateStructMatrix(1,1,5,fields);
1947
1948         mxSetFieldByNumber(S,0,0, numtoMx(M));
1949         mxSetFieldByNumber(S,0,1, numtoMx(N));
1950         mxSetFieldByNumber(S,0,2, numtoMx(nnz));
1951         mxSetFieldByNumber(S,0,3, wbMatrix<TD>(M,N,data,'r').toMx());
1952         mxSetFieldByNumber(S,0,4, wbMatrix<SPIDX_T>(M,N,NIJ,'r').toMx());
1953
1954         return S;
1955     };
1956
1957 protected:
1958 private:
1959

```

```

1960     SPIDX_T M,N,nnz,*NIJ;
1961     TD* data;
1962 };
1963
1964 #endif
1965

```

5.38 wbststring.hh

```

1  #ifndef __WB_STRING_HH__
2  #define __WB_STRING_HH__
3
4  namespace Wb {
5
6  // NB! convert (possible) long long on input to double
7  // NB! see header in wblibx.h
8  char* memsize2Str(double x, char *s, unsigned l);
9
10 wbststring repHome(const wbststring &file);
11
12 int findRegEx(const char *s, const char *pat, char icalse=0);
13 };
14
15 class wbststring : public wbvector<char> {
16
17 public:
18
19     wbststring (unsigned n=0) : wbvector<char>(n) {};
20
21     wbststring (unsigned n, const char c)
22         : wbvector<char>(n+1) { set(c); data[len-1]=0; };
23
24     wbststring (const char* i, unsigned n)
25         : wbvector<char>(n+1,i) { data[len-1]=0; };
26
27     wbststring (const wbststring &s) : wbvector<char>() { init(s.data); };
28     wbststring (const char* s1) : wbvector<char>() { init(s1); };
29
30     wbststring (
31         const char* s1, const char* s2,
32         const char* s3=NULL, const char* s4=NULL
33     ) : wbvector<char>() {
34         unsigned
35             l1 = (s1? strlen(s1) : 0),
36             l2 = (s2? strlen(s2) : 0),
37             l3 = (s3? strlen(s3) : 0),
38             l4 = (s4? strlen(s4) : 0);
39
40         NEW(l1+l2+l3+l4+1);
41         if (l1) { strcpy(data, s1); }
42         if (l2) { strcpy(data+l1, s2); }
43         if (l3) { strcpy(data+l1+l2, s3); }
44         if (l4) { strcpy(data+l1+l2+l3, s4); }
45         data[len-1]=0;
46     };
47
48 #ifndef NOMEX
49     wbststring(const mxArray *a) : wbvector<char>() { init(0,0,a); };
50     wbststring& init(const mxArray *a) { return init(0,0,a); }
51 #endif
52
53     wbststring (const char *F, int L, const mxArray *a)
54         : wbvector<char>() { init(F,L,a); };
55
56 #if 0
57     ~wbstring() {
58         if (len) {
59             if (data[len-1]) {
60                 unsigned i=0; for (; i<len && data[i]; ++i) { };
61                 if (i>=len) { wblog(FL,
62                     "WRN %s() string out of bounds? (len=%d/%d)",FCT,i,len);
63                     fprintf(stdout,"\n");
64                     for (i=0; i<len && data[i]; ++i) fputc(data[i],stdout);
65                     fprintf(stdout,"\n\n");
66                 }
67             }
68         }
69     };
70 #endif
71
72     wbststring& init(unsigned n=0) { RENEW(n); return *this; };
73
74     wbststring& init(const char *s) {

```

```

75     if (s)
76     { RENEW(strlen(s)+1, s); data[len-1]=0; }
77     else { init(); }
78     return *this;
79 };
80
81 wbststring& init(const char *F, int L, const mxArray *a) {
82     int i,n;
83     if (!a || !(n=mxGetNumberOfElements(a))) {
84         RENEW(0); return *this;
85     };
86
87     if (!mxIsChar(a)) wblog(F_L,
88         "ERR initializing string with type '%s'",mxGetClassName(a));
89
90     RENEW(++n);
91     i=mxGetString(a,data,n);
92
93     if (i) wblog(F_L,"ERR failed to read string >%s<",data);
94     return *this;
95 };
96
97 wbststring& initrep(const char* s, unsigned n) {
98     if (s && s[0] && n) {
99         unsigned l=strlen(s); init(l+n+1); data[len-1]=0;
100        if (l==1) { memset(data,s[0],n); }
101        else {
102            for (unsigned i=0; i<n; ++i) { strcpy(data+i*n,s); }
103        }
104    }
105    else { init(); }
106    return *this;
107 };
108
109 wbststring& initrep(char c, unsigned n) {
110     if (n && isprint(c))
111         { init(n+1); data[len-1]=0; memset(data,c,n); }
112     else { init(); }
113     return *this;
114 };
115
116 void set(char c) {
117     if (len) {
118         memset(data, c, len-1); data[len-1]=0; }
119 };
120
121 wbststring& operator= (const char* i) {
122     RENEW(strlen(i)+1, i); data[len-1]=0; return *this;
123 };
124
125 bool gotTerm0() {
126     for (unsigned i=0; i<len; ++i) { if (data[i]==0) return 1; }
127     return 0;
128 };
129
130 bool strchr(const char *s) {
131     if (data && data[0]) {
132         if (!s || !s[0]) wblog(F_L,"WRN %s() got empty char set !?",FCT);
133         else { for (; *s; ++s) { if (::strchr(data,*s)) return 1; } }
134     }
135     return 0;
136 };
137
138 wbststring& push(const char *F, int L, const char* s, char extend=1);
139 wbststring& push(const char* s, char extend=1) {
140     return push(0,0,s,extend); };
141
142 wbststring& pushf(const char *F, int L, const char *fmt, ...);
143 wbststring& printf(const char *F, int L, const char *fmt, ...);
144 wbststring& cpy(const char *F, int L, const char *s);
145
146 wbststring& time() { return time::time(NULL); };
147 wbststring& time(const time_t &t) { NEW(32);
148     #pragma omp critical (got_CPTR_TIME)
149     { ctime_r(&t,data); }
150     return *this;
151 };
152
153 wbststring& time_sys() { return time_sys::time(NULL); };
154 wbststring& time_sys(const time_t &t) { NEW(32);
155     #pragma omp critical (got_CPTR_TIME)
156     { strftime(data,31,"%a %b %d %H:%M:%S %Z %Y",localtime(&t)); }
157     return *this;
158 };
159
160 wbststring& Upper() {
161     for (unsigned n=strlen(data), i=0; i<n; i++)

```

```

162         data[i]=std::toupper(data[i]);
163         return *this;
164     };
165
166     wstring& Lower() {
167         for (unsigned n=strlen(data), i=0; i<n; i++)
168             data[i]=std::tolower(data[i]);
169         return *this;
170     };
171
172     wstring toupper() const {
173         wstring sout(*this); sout.Upper(); return sout; };
174
175     wstring tolower() const {
176         wstring sout(*this); sout.Lower(); return sout; };
177
178     wstring& Chomp() {
179         if (data && data[0]) { int i=strlen(data);
180             for (; i>=0; --i) {
181                 if (data[i]>20 && data[i]!=' ') { data[i+1]=0; break; }
182             }
183             if (i<0) data[0]=0;
184         }
185         return *this;
186     };
187
188     char getopt(char c) {
189         for (unsigned i=0; data[i]; i++) {
190             if (data[i]==c) {
191                 data[i]=-1;
192                 return 1;
193             }
194         }
195         return 0;
196     }
197     char getopt() {
198         for (unsigned i=0; data[i]; i++) {
199             if (data[i]>0) { return data[i]; }
200         }
201         return 0;
202     }
203
204     const char* basename(const char c='/') const {
205         if (!data) return data;
206         return Wb::basename(data,c);
207     };
208
209     mxArray* toMx() const { return mxCreateString(data); };
210
211     void add2MxStruct(mxAarray *S, unsigned i) const {
212         mxSetCell(S,i,toMx());
213     };
214
215     template <class T>
216     char* init2Fmt(const T& x __attribute__((unused)), int n=-1, int p=-1){
217         (*this)=wstring(Wb::num2Fmt<T>(FL,n,p));
218         return data;
219     };
220
221     template <class T> wstring& operator<< (const T& x);
222     template <class T> wstring& operator<< (const T& x);
223
224     wstring operator+ (const wstring &s) const { return (*this)+s.data; };
225     wstring operator+ (const char *s) const {
226         wstring sout;
227         unsigned n1=data? strlen(data):0, n2=s && s[0]? strlen(s):0;
228         sout.init(n1+n2);
229         if (n1) { strcpy(sout.data, data); }
230         if (n2) { strcpy(sout.data+n1, s); }
231         return sout;
232     };
233
234     wstring& operator+=(const char *s) {
235         if (data && s && s[0]) {
236             unsigned n1=strlen(data), n2=strlen(s);
237             if (n2) { Resize(n1+n2); strcpy(data+n1, s); }
238         }
239         else if (s) { (*this)=s; }
240         return *this;
241     };
242
243     wstring& operator+=(const wstring &s) {
244         if (s.data) { (*this)+=s.data; }
245         return *this;
246     };
247
248     unsigned cat(unsigned &k, const char *s) {

```

```

249         if (s) { unsigned i=0;
250             for (; s[i]!=0 && k<len; ++i) { data[k++]=s[i]; }
251             if (s[i]) wblog(FL,"ERR wbststring::cat out of bounds (%d/%d)",k,len);
252         }
253         data[k]=0; return k;
254     };
255
256     wbststring toStr() const {
257         if (data) { return *this; }
258         else { return wbststring("(null)"); }
259     }
260
261     char& operator[] (unsigned i) const { return data[i]; };
262
263     unsigned strlen_() const {
264         return (data && data[0] ? strlen(data) : 0); };
265
266     bool isEmpty() const { return (data && data[0] ? 0 : 1); };
267
268     bool operator! () const { return (data && data[0] ? 0 : 1); }
269     explicit operator bool() const { return (data && data[0] ? 1 : 0); };
270
271     unsigned isName(int L=-1) const {
272         unsigned i, n=strlen(data);
273         if (L>=0 && L<(int)n) return 0;
274         for (i=0; i<n; i++) if (!isalnum(data[i])) return 0;
275         return n;
276     };
277
278     int operator== (const char *s) const {
279         return (strcmp(data, s)==0);
280     };
281     int operator!= (const char *s) const {
282         return (strcmp(data, s)!=0);
283     };
284
285     int operator== (const wbststring &s) const {
286         return strcmp(data, s.data)==0;
287     };
288     int operator!= (const wbststring &s) const {
289         return strcmp(data, s.data)!=0;
290     };
291     int operator< (const wbststring &s) const {
292         return strcmp(data, s.data)<0;
293     };
294     int operator<= (const wbststring &s) const {
295         return strcmp(data, s.data)<=0;
296     };
297     int operator> (const wbststring &s) const {
298         return strcmp(data, s.data)>0;
299     };
300     int operator>= (const wbststring &s) const {
301         return strcmp(data, s.data)>=0;
302     };
303
304     int firstLocation (const wbststring &, unsigned k=0);
305     int firstLocation (const char*, unsigned offset=0);
306
307     std::istream & getline (std::istream &istr, char c='\n');
308
309     protected:
310     private:
311 };
312
313 std::istream & wbststring::getline (std::istream &istr, char c) {
314     const unsigned maxlen=256;
315     char tmp[maxlen];
316     istr.getline(tmp, maxlen-1, c);
317
318     init(strlen(tmp)+1); strcpy (data, tmp);
319
320     return istr;
321 }
322
323 int wbststring::firstLocation (const wbststring &s, unsigned k) {
324     return firstLocation(s.data,k);
325 }
326
327 int wbststring::firstLocation (const char *s, unsigned offset) {
328     int i, j, n=strlen(data), m=strlen(s);
329
330     for (j=0, i=offset; i<n && j<m; i++, j++) {
331         if (data[i] != s[j]) {
332             i-=j;
333             j=-1;
334         }
335     }

```

```

336
337     if (j==m) return i-m;
338     else return -1;
339 }
340
341 namespace Wb {
342     template<class T>
343     wstring num2Str(const T &x, const char *fmt=0, unsigned l=0);
344
345     wstring int2Str(long x, const char *fmt=0, unsigned l=0);
346     wstring char2Str(int x, unsigned l=0);
347
348     wstring repStr(const char* s_, unsigned n) {
349         wstring s;
350         return s.initrep(s_,n);
351     };
352
353     class String : public wvector<char> {
354     public:
355
356         String(unsigned n)
357             : wvector<char>(n), ipos(0), fline(NULL) { if (n) { data[n]=0; } };
358
359         String(const char *F, int L, unsigned n)
360             : wvector<char>(n), ipos(0), fline(NULL) { if (n) { data[n]=0; }
361             if (F && F[0]) { fline=shortFL(F,L); }
362             };
363
364         int pushf(const char *fmt, ...);
365
366     protected:
367     private:
368
369         unsigned ipos;
370         const char *fline;
371     };
372
373 };
374
375 #include <queue>
376
377 class wblogbuf_struct {
378
379     public:
380
381     wblogbuf_struct() : file(NULL), line(0), text(NULL) {};
382
383     wblogbuf_struct (const char *s) : file(NULL), line(0), text(NULL) {
384         if (s) {
385             unsigned n=strlen(s);
386             WB_NEW(text,n+1); strcpy(text,s);
387         }
388     };
389
390     wblogbuf_struct (const char *F, int L, const char *s)
391         : file(NULL), line(L), text(NULL) { unsigned i=0,k=0,n;
392         if (F) {
393             for (; F[i]; i++) { if (F[i]=='/' || F[i]=='\\') k=i+1; }
394             n=strlen(F+k); WB_NEW(file,n+1); strcpy(file,F+k);
395         }
396         if (s) { n=strlen(s); WB_NEW(text,n+1); strcpy(text,s); }
397     };
398
399     wblogbuf_struct(const wblogbuf_struct &S)
400         : file(S.file), line(S.line), text(S.text)
401     ){
402     };
403
404     ~wblogbuf_struct () {
405         if (file) { WB_DELETE(file); }
406         if (text) { WB_DELETE(text); }
407         line=0;
408     };
409
410     void unset() { file=NULL; line=0; text=NULL; };
411
412     char *file; int line; char *text;
413
414     protected:
415     private:
416 };
417
418 class wblogbuffer {
419
420     public:
421
422     void push(const char *F, int L, const char *s) {

```



```

423     wblogbuf_struct S(F,L,s);
424     buf.push(S); S.unset();
425 };
426
427 void flush() {
428     while (!buf.empty()) {
429         wblogbuf_struct S=buf.front();
430         wblog(
431             S.file ? S.file : __FILE__,
432             S.line ? S.line : __LINE__, S.text
433         );
434         S.unset();
435         buf.pop();
436     }
437     doflush();
438 };
439
440 protected:
441 private:
442
443     queue<wblogbuf_struct> buf;
444 };
445
446     wblogbuffer wblogBuf;
447
448 #endif
449

```

5.39 wbutil.hh

```

1  #ifndef __WBUTIL_HCC__
2  #define __WBUTIL_HCC__
3
4  // NB! util.hh cannot use wbsting etc. as members of class
5  // definitions here since wbsting.hh etc. need to be fully
6  // defined (if an object is used as a member of a class,
7  // the compiler wants to know its full definition, so the
8  // forward declarations in wblib.h do not suffice)
9  // Wb,Sep03,16
10
11 // eventually set in wblib.h *after* WB_VERBOSE is initialized!
12 // Wb,Jun09,19
13 int OMP_NUM_THREADS = 0; // 1 suggest it was actually set to this value
14
15 namespace Wb {
16
17 class Path {
18 public:
19
20     Path() {};
21     Path(const char *s) { init(FL,s); };
22     Path(const char *F, int L, const char *s) { init(F,L,s); };
23
24     int init() { buf.init(); idx.init(); return 0; };
25
26     int init(
27         const char *F, int L, const char *s, const char *delim=":",
28         unsigned n_=-1
29     );
30
31     int append(
32         const char *F, int L, const char *s, const char *delim=":"
33     );
34
35     char* operator[] (unsigned i) {
36         if (i+1>=idx.len) wblog(FL,
37             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,idx.len-1);
38         return buf.data+idx[i];
39     };
40
41     const char* operator[] (unsigned i) const {
42         if (i+1>=idx.len) wblog(FL,
43             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,idx.len-1);
44         return buf.data+idx[i];
45     };
46
47     const char* last() const {
48         if (idx.len<2) {
49             if (!idx.len)
50                 wblog(FL,"ERR %s() path got uninitialized path !?",FCT);
51             else
52                 wblog(FL,"ERR %s() path got path with idx.len=%d !?",FCT,idx.len);
53         }
54         return (*this)[idx.len-2];
55     };
56

```

```

54
55     unsigned numel() const {
56         if (idx.len<2) {
57             if (!idx.len) wblog(FL,
58                 "ERR %s() got uninitialized Path (%d,%d)",FCT,idx.len,buf.len);
59             else wblog(FL,"ERR %s() got Path with idx.len=%d",FCT,idx.len);
60         }
61         return (idx.len-1);
62     };
63
64     Wb::iterator<Path,char*> begin() {
65         return Wb::iterator<Path,char*>(this);
66     };
67     Wb::citerator<Path,char*> cbegin() const {
68         return Wb::citerator<Path,char*>(this);
69     };
70
71     void print(const char *istr="") const;
72
73     wbstring buf;
74     wbvector<unsigned> idx;
75
76 protected:
77 private:
78
79 };
80
81 void Path::print(const char *istr) const {
82     unsigned i=0, n=numel();
83
84     if (istr && istr[0]) wblog(FL,"==> Contents of %s%N",istr);
85     for (; i<n; ++i) { printf("  %4d. %s\n",i+1,(*this)[i]); }
86     printf("\n");
87 };
88
89 int Path::init(
90     const char *F, int L, const char *s, const char *delim,
91     unsigned n_
92 ){
93
94     if (!s || !s[0]) {
95         if (int(n_)>0) buf.init(n_); else buf.init();
96         idx.init(); return 0;
97     }
98
99     if (!delim || !delim[0]) wblog(F_L,
100         "ERR %s() invalid delim '%s' !?",FCT,delim?delim:"(null)");
101
102     unsigned i,m0,m=0, n=strlen(s); char *b,*b0;
103
104     if (int(n_)<0) { buf=s; }
105     else {
106         if (n_<n) wblog(F_L,"ERR %s() "
107             "input path length out of bounds %d/%d !?",FCT,n,n_);
108         buf.init(n_); strcpy(buf.data,s);
109     }
110
111     b0=b=buf.data;
112
113     while ((b0=strsep(&b,delim))) { ++m; }
114
115     idx.init(m+1); m0=m; b=buf.data;
116
117     for (m=i=0; i<=n; ++i) {
118         idx[m++]=i;
119
120         if (!b[i]) continue;
121
122         do { ++i; } while (b[i] && i<=n);
123     }
124
125     if (m!=m0) wblog(FL,"ERR %s() m=%d/%d !?",FCT,m,m0);
126     idx[m]=n;
127
128     if (idx.end()-1>buf.len) wblog(FL,
129         "ERR %s() index out of bounds (%d/%d) !?",FCT,idx.end()-1,buf.len);
130     return m;
131 };
132
133 int Path::append(
134     const char *F, int L, const char *s, const char *delim
135 ){
136     if (!idx.len) { return init(F,L,s,delim); }
137
138     if (!s || !s[0]) { buf.init(); idx.init(); return 0; }
139     if (!delim || !delim[0]) wblog(FL,
140         "ERR %s() invalid delim '%s' !?",FCT,delim?delim:"(null)");

```

```

141
142     unsigned i,l=idx.len-1,m0,m=0, n=strlen(s);
143     unsigned i0=idx[l]+1;
144     char *b=buf.data+i0, *b0=b;
145
146     if (i0+n>=buf.len) wblog(FL,
147         "ERR %s() index out of bounds (%d+%d/%d) !?",FCT,i0,n,buf.len);
148     strcpy(b0,s);
149
150     while ((b0=strsep(&b,delim))) { ++m; }
151
152     idx.Resize(idx.len+m);
153     m0=m; b=buf.data+i0;
154
155     for (m=i=0; i<=n; ++i) {
156         idx[l+(m++)]=i0+i;
157
158         if (!b[i]) continue;
159
160         do { ++i; } while (b[i] && i<=n);
161     }
162
163     if (m!=m0) wblog(FL,"ERR %s() m=%d/%d !?",FCT,m,m0);
164     idx[l+m]=n;
165
166     if (idx.end()-1>=buf.len) wblog(FL,
167         "ERR %s() index out of bounds (%d/%d) !?",FCT,idx.end()-1,buf.len);
168
169     return m;
170 };
171
172 double getTimeNow() {
173
174     struct timespec now;
175     int e=clock_gettime(CLOCK_REALTIME,&now);
176     if (e<0) wblog(FL,"ERR %s() gettimeofday returned error (e=%d)",FCT,e);
177
178     return (now.tv_sec + (now.tv_nsec/1E9));
179 };
180
181 };
182
183 #endif

```

5.40 wbvector.hh

```

1 #ifndef __WB_SIMPLE_VECTOR_HCC__
2 #define __WB_SIMPLE_VECTOR_HCC__
3
4 template <class T> class wbvector;
5
6 /* ----- *
7  * class wbvector
8  * AWb (C) Jan 2006
9  * ----- */
10
11 WBINDEX Index(widx_t i1, widx_t i2);
12 char isUniqueIdxSet(const WBINDEX &I, widx_t imax);
13 char validPerm(const WBPERM &P, wperm_t d=-1);
14 void getIPerm (const WBPERM &P, WBPERM &iP);
15
16 bool mxIsWbvector(
17     const char *F, int L, const mxArray *a,
18     size_t *n=NULL, const char *istr=NULL, char dflag='d');
19
20 bool mxIsWbvector(const mxArray *a) {
21     return mxIsWbvector(0,1,a);
22 };
23
24 template <class T>
25 class wbvector {
26
27 public:
28
29     wbvector() : data(NULL), len(0), isref(0) {};
30
31     wbvector(size_t l, const T* d0=NULL, char ref_flag=0)
32         : data(NULL), len(0), isref(0) {
33         if (ref_flag && d0)
34             { init2ref(l,d0); }
35         else { NEW(l,d0); }
36     };
37

```

```

38     wbvector(const wbvector &v)
39     : data(NULL), len(0), isref(0) { NEW(v.len, v.data); };
40
41     template <class T2>
42     wbvector(const wbvector<T2> &v)
43     : data(NULL), len(0), isref(0) { initT(v.len,v.data); }
44
45     wbvector(const wbvector &v1, const wbvector &v2)
46     : data(NULL), len(0), isref(0) {
47         size_t i, k=0;
48         NEW(v1.len+v2.len);
49
50         for (i=0; i<v1.len; ++i, ++k) data[k]=v1[i];
51         for (i=0; i<v2.len; ++i, ++k) data[k]=v2[i];
52     };
53
54     wbvector(const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
55     : data(NULL), len(0), isref(ref) {
56         init(F,L,a,NULL,check_type,ref);
57     };
58
59     wbvector(const mxArray *a, char check_type=1, char ref=0)
60     : data(NULL), len(0), isref(ref) {
61         init(FL,a,NULL,check_type,ref);
62     };
63
64     virtual ~wbvector() {
65         if (!isref && data) { WB_DELETE(data); }
66     };
67
68     wbvector& init(size_t l=0) { return RENEW(l); };
69     wbvector& init(size_t l, const T* d0) { return RENEW(l,d0); };
70
71     wbvector& init(size_t l, const T* d0, char ref) {
72         if (ref)
73             return init2ref(l,d0);
74         else return RENEW(l,d0);
75     };
76
77     wbvector& init2val(size_t l, const T& x) {
78         return RENEW_VAL(l,x); };
79
80     wbvector& init_stride(
81         const T* d0, size_t n, size_t m, size_t D0
82     ){
83         RENEW(n*m);
84         Wb::cpyStride(data, d0, n, m, -1, D0);
85         return *this;
86     };
87
88     wbvector& init(const wbvector &v) {
89         RENEW(v.len);
90         for (size_t i=0; i<len; i++) data[i]=v.data[i];
91         return *this;
92     };
93
94     wbvector& init(size_t n, const wbvector &v) {
95         RENEW(n); if (n>v.len) n=v.len;
96         for (size_t i=0; i<n; ++i) data[i]=v.data[i];
97         return *this;
98     };
99
100     wbvector& init_data(const T *d0) {
101         for (size_t i=0; i<len; ++i) { data[i]=d0[i]; }
102         return *this;
103     };
104
105     template<class T0>
106     wbvector& init_data(const T0 *d0) {
107         for (size_t i=0; i<len; ++i) { data[i]=T(d0[i]); }
108         return *this;
109     };
110
111     template<class T0>
112     wbvector& init_data(const char *F, int L, const T0 *d0) {
113         for (size_t i=0; i<len; ++i) { data[i]=T(d0[i]);
114             if (T0(data[i])!=d0[i]) wblog(F,L,
115                 "ERR type conversion changes value (%g->%g) !?",
116                 double(d0[i]), double(data[i])
117             );
118         }
119         return *this;
120     };
121
122     wbvector<T>& init(const wbspararray<T> &S) { return initT(S); };
123
124     template <class T2>

```

```

125     wbvector<T>& initT(const wbspararray<T2> &S);
126
127     template <class T2>
128     wbvector<T>& initT(size_t l, const T2* v) {
129         RENEW(l);
130         for (size_t i=0; i<len; ++i) data[i]=T(v[i]);
131         return *this;
132     };
133
134     template <class T2>
135     wbvector<T>& initT(const wbvector<T2> &v) {
136         RENEW(v.len);
137         for (size_t i=0; i<len; ++i) data[i]=T(v.data[i]);
138         return *this;
139     };
140
141     template <class T2>
142     wbvector<T>& initT_(const wbvector<T2> &v) {
143         RENEW(v.len);
144         for (size_t i=0; i<len; ++i) data[i]=T(v.data[i], '!');
145         return *this;
146     };
147
148     template <class T2>
149     wbvector<T>& initT(const char *F, int L, const wbvector<T2> &v);
150
151     wbvector& init2ref(const wbvector &v) {
152         return init2ref(v.len, v.data);
153     };
154
155     wbvector& init2ref(size_t l, const T* d0) {
156         if (data && !isref) { RENEW(0); }; isref=1;
157         len=l; data=(T*)d0;
158
159         if (len && !data) wblog(FL,
160             "ERR init2ref with data=0 (%d) !?", len);
161         return *this;
162     };
163
164     wbvector& unRef();
165
166     wbvector& init(size_t l1, const T* d1, size_t l2, const T* d2) {
167         RENEW(l1+l2); MEM_COPY<T>(data, len, l1, d1, d2);
168         return *this;
169     };
170
171     int init(
172         const char *F, int L, const mxArray *a, const char *istr=NULL,
173         char check_type __attribute__((unused)) =1,
174         char ref=0
175     ) { return init_Struct(F, L, a, istr, ref); };
176
177     int init_Struct(
178         const char *F, int L, const mxArray *a,
179         const char *istr=NULL, char ref=0
180     );
181
182     int init_base(
183         const char *F, int L, const mxArray *a,
184         const char *istr=NULL, char check_type=1, char ref=0
185     );
186
187     int init(
188         const mxArray *a, const char *istr=NULL, char check_type=1
189     ) { return init(0, 0, a, istr, check_type); };
190
191     int init_mpr(
192         const char *F, int L, const mxArray *a, char base=-1,
193         const char *istr="", char tcheck=1
194     );
195
196     void initDef(size_t n) {
197         if (data) { WB_DELETE(data); }
198         len=n; if (n) { WB_NEW(data, n); }
199     };
200
201     void initDef(const wbvector &v) {
202         if (len!=v.len) initDef(v.len);
203         for (size_t i=0; i<v.len; ++i) data[i]=v[i];
204     };
205
206     wbvector& operator= (const wbvector &v) {
207         if (this!=&v) { RENEW(v.len, v.data); }
208         return *this;
209     };
210
211     template<class TB>

```

```

212     wbvector<T>& operator= (const wbvector<TB> &v) {
213         init(v.len);
214         for (size_t i=0; i<v.len; ++i) { data[i]=T(v.data[i]); }
215         return *this;
216     };
217
218     wbvector& set(const T &x) {
219         for (size_t i=0; i<len; ++i) data[i]=x;
220         return *this;
221     };
222
223     wbvector& setMin(const T &x) {
224         for (size_t i=0; i<len; ++i) { if (data[i]<x) data[i]=x; }
225         return *this;
226     };
227     wbvector& setMax(const T &x) {
228         for (size_t i=0; i<len; ++i) { if (data[i]>x) data[i]=x; }
229         return *this;
230     };
231
232     wbvector& set(const wbvector<size_t> &I, const T &x);
233     wbvector& add(const wbvector<size_t> &I, const T &x);
234
235     wbvector& Set(const wbvector<size_t> &I, const wbvector &v);
236
237     void Sort(wbperm &p, char dir=+1) { Wb::hpsort(*this,p,dir); };
238     void sort(wbvector &v2, wbperm &p) const { v2=(*this); Wb::hpsort(v2,p); };
239
240     void Sort(char dir=+1);
241     void sort(wbvector &v2) const;
242
243     wbperm& pSort(wbperm &p) const;
244
245     bool isSorted() const {
246         for (size_t i=1; i<len; ++i) if (data[i-1]> data[i]) return 0;
247         return 1;
248     };
249     bool isSortedU() const {
250         for (size_t i=1; i<len; ++i) if (data[i-1]>=data[i]) return 0;
251         return 1;
252     };
253
254     bool isUnique() const;
255
256     int contains(const T& x) const {
257         for (size_t i=0; i<len; ++i) { if (data[i]==x) return 1; }
258         return 0;
259     };
260
261     bool isNormal() const;
262
263     bool isFinite() const {
264         for (size_t i=0; i<len; ++i) {
265             if (data[i] && !isfinite(data[i])) return 0; }
266         return 1;
267     };
268
269     size_t find1(const T& x) const {
270         for (size_t i=0; i<len; ++i) { if (data[i]==x) return i; }
271         return -1;
272     };
273     size_t findl() const {
274         for (size_t i=0; i<len; ++i) { if (data[i]) return i; }
275         return -1;
276     };
277
278     size_t count(const T& x) const {
279         size_t i=0, n=0;
280         for (; i<len; ++i) if (data[i]==x) ++n;
281         return n;
282     };
283
284     size_t icount(const T& x) const {
285         size_t i=0,n=0;
286         for (; i<len; ++i) if (data[i]!=x) ++n;
287         return n;
288     };
289
290     size_t nnz() const {
291         size_t i=0, n=0;
292         for (; i<len; ++i) if (data[i]) ++n;
293         return n;
294     };
295     size_t nnz(T eps) const {
296         if (eps==0) { return nnz(); }
297         else { return len-numZeros(eps); }
298     };

```

```

299
300     size_t numZeros(T eps=T(1E-14)) const;
301     size_t numEQ(const T&) const;
302     size_t numGT(const T&) const;
303     size_t numLT(const T&) const;
304
305     windex& find(const T &x, windex &I, char iflag=0) const;
306     windex& find(const T &x, windex &I, windex &Ix) const;
307     windex& findGT(const T &x, windex &I, char iflag=0) const;
308     windex& findGT(const T &x, windex &I, windex &Ix) const;
309
310     windex& findzeros(windex &I) const { return find(0,I); }
311     windex& find(windex &I) const { return find(0,I,'i'); }
312
313     windex& find(windex &I, windex &Ix) const {
314         find(0,Ix,I); return I; }
315
316     wbvector<size_t> find() const {
317         wbvector<size_t> I;
318         return find(I);
319     };
320
321     windex& findRange(
322         const T &x1, const T &x2, windex &I, const char *w=NULL
323     ) const;
324
325     int findClosestSorted(T r) const;
326     int findClosest(T r) const;
327
328     size_t findValues(const wbvector &v, windex &I) const;
329
330     wbvector& setp(const T *x) {
331         for (size_t i=0; i<len; i++) data[i]=x[i];
332         return *this;
333     };
334
335     void set(size_t offset, const T *x, size_t n) {
336         if (offset+n>=len) wblog(FL,
337             "ERR index out of bounds (%d+%d; %d)",offset,n,len);
338         MEM_COPY<T>(data+offset,n,x);
339     };
340
341     void reset() { if (len) MEM_SET<T>(data,len); };
342
343     void cpy(const T* v) {
344         if (len) MEM_COPY<T>(data,len,v);
345     };
346
347     void cpy(const T *x, size_t m) {
348         if (m>len) { wblog(FL,"ERR index out of bounds (%d/%d)",m,len); m=len; }
349         if (m) MEM_COPY<T>(data,m,x);
350     };
351
352     wbvector& save2(wbvector &v);
353
354     void swap (wbvector &v) {
355         if (this!=&v) {
356             SWAP(len, v.len );
357             SWAP(data, v.data );
358             SWAP(isref, v.isref);
359         }
360     };
361
362     wbvector& swap(size_t i, size_t j) {
363         if (i<len && j<len)
364             { T x=data[i]; data[i]=data[j]; data[j]=x; }
365         else { wblog(FL,
366             "ERR %s() index out of bounds (%d,%d/%d)",FCT,i+1,j+1,len); }
367         return *this;
368     };
369
370     wbvector& Collect(T& A1, T& A2) {
371         init(2); A1.save2(data[0]); A2.save2(data[1]);
372         return *this;
373     };
374
375     wbvector& Collect(T& A1, T& A2, T& A3) {
376         init(3); A1.save2(data[0]); A2.save2(data[1]); A3.save2(data[2]);
377         return *this;
378     };
379
380     wbvector& Skip(size_t);
381     wbvector& Skip(const WINDEX &, char rflag=0);
382
383     wbvector& Move2_FE (const WINDEX &I, char FE_flag);
384     wbvector& Move2front(const WINDEX &I) { return Move2_FE(I,'F'); }
385     wbvector& Move2end (const WINDEX &I) { return Move2_FE(I,'E'); }

```

```

386
387     wbvector& Move2front (size_t k) {
388         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
389         T xk=data[k];
390         for (size_t i=k; i>0; i--) data[i]=data[i-1];
391         data[0]=xk; return *this;
392     }
393
394     wbvector& Move2end (size_t k) {
395         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
396         T xk=data[k];
397         for (size_t i=k+1; i<len; i--) data[i-1]=data[i];
398         data[len-1]=xk; return *this;
399     }
400
401     wbvector& mvLastTo (size_t k) {
402         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
403         T xk=data[len-1];
404         for (size_t i=len-1; i>k; i--) data[i]=data[i-1];
405         data[k]=xk; return *this;
406     }
407
408     wbvector& mvFirstTo (size_t k) {
409         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
410         T xk=data[0];
411         for (size_t i=0; i<k; i++) data[i]=data[i+1];
412         data[k]=xk; return *this;
413     }
414
415     bool checkSameLength(
416         const wbvector &a,
417         const char *F=NULL, int L=0, const char *istr=""
418     ) const;
419
420     bool checkSameLength(
421         const wbvector &a,
422         const wbvector &b,
423         const char *F=NULL, int L=0, const char *istr=""
424     ) const;
425
426     template <class T_>
427     void operator+= (const T_ &c) {
428         if (c==T(0)) return;
429         for (size_t i=0; i<len; ++i) data[i]+=c;
430     };
431
432     template <class T_>
433     void operator-= (const T_ &c) {
434         if (c==T(0)) return;
435         for (size_t i=0; i<len; ++i) data[i]-=c;
436     };
437
438     template <class T_>
439     void operator*= (const T_ &c) const {
440         if (c==T(1)) return;
441         if (c==T(0)) MEM_SET<T>(data,len); else
442         if (c==T(-1))
443             for (size_t i=0; i<len; ++i) data[i]=-data[i];
444         else for (size_t i=0; i<len; ++i) data[i]*=c;
445     };
446
447     template <class T_>
448     void operator/= (const T_ &c) {
449         if (c==T(1)) return;
450         if (c==T(0)) wblog(FL,"ERR c=%g DIV/0", double(c));
451         if (c==T(-1))
452             for (size_t i=0; i<len; ++i) data[i]=-data[i];
453         else for (size_t i=0; i<len; ++i) data[i]/=c;
454     };
455
456     template <class T_>
457     wbvector operator* (const T_ &c) const {
458         wbvector u(*this); if (c!=1) u*=c;
459         return u;
460     };
461
462     template <class T_>
463     wbvector operator+ (const T_ &c) const {
464         wbvector u(*this);
465         if (c!=0) for (size_t i=0; i<u.len; i++) u.data[i]+=c;
466         return u;
467     };
468
469     template <class T_>
470     wbvector operator- (const T_ &c) const {
471         wbvector u(*this);
472         if (c!=0) for (size_t i=0; i<u.len; i++) u.data[i]-=c;

```



```

473     return u;
474 };
475
476 wbvector operator+ (const wbvector &b) const {
477     wbvector u(*this); checkSameLength(b,FLF);
478     for (size_t i=0; i<u.len; i++) u.data[i]+=b.data[i];
479     return u;
480 };
481
482 wbvector operator- (const wbvector &b) const {
483     wbvector u(*this); checkSameLength(b,FLF);
484     for (size_t i=0; i<u.len; i++) u.data[i]-=b.data[i];
485     return u;
486 };
487
488 void operator+= (const wbvector &b) {
489     checkSameLength(b,FLF);
490     for (size_t i=0; i<len; ++i) data[i]+=b.data[i];
491 };
492
493 void operator-= (const wbvector &b) {
494     checkSameLength(b,FLF);
495     for (size_t i=0; i<len; ++i) data[i]-=b.data[i];
496 };
497
498 wbvector& Plus (const T* b) {
499     for (size_t i=0; i<len; ++i) data[i]+=b[i];
500     return *this;
501 };
502
503 wbvector& Plus (const T* b, T bfac) {
504     size_t i=0;
505     if (bfac==+1) for (; i<len; ++i) data[i]+=b[i]; else
506     if (bfac==-1) for (; i<len; ++i) data[i]-=b[i]; else
507     if (bfac!= 0) for (; i<len; ++i) data[i]+=(bfac*b[i]);
508     return *this;
509 };
510
511 wbvector& Plus (const wbvector &b, T bfac=1) {
512     checkSameLength(b,FLF);
513     return Plus(b.data,bfac);
514 };
515
516 void plus (
517     const wbvector &b, wbvector &c, T bfac=1, T cfac=0
518 ) const {
519     if (cfac) {
520         checkSameLength(b,c,FLF); c*=cfac; c.Plus(*this); }
521     else { checkSameLength(b, FLF); c=*this; }
522
523     c.Plus(b,bfac);
524 };
525
526 wbvector& times(T fac, wbvector &x) const {
527     x=*this; x*=fac; return x;
528 };
529
530 wbvector& RevertSigns();
531
532 int isum() const {
533     int s=0; if (len==0) return s;
534
535     s=data[0]; for (size_t i=1; i<len; i++) s+=data[i];
536     return s;
537 };
538
539 T sum() const {
540     T s=T(); if (len==0) return s;
541
542     s=data[0]; for (size_t i=1; i<len; i++) s+=data[i];
543     return s;
544 };
545
546 T sum(size_t i1, const char *estr) const {
547     if (!estr || strcmp(estr,"end")) wblog(FL,
548         "ERR Invalid end string '%s'.", estr);
549
550     return sum(i1,len-1);
551 };
552
553 T sum(size_t i1, size_t i2) const {
554     T s=T();
555
556     if (len==0) if (i1==0 && i2==0) return s;
557     if (i1>=len || i2>=len) wblog(FL,
558         "ERR index out of bounds (%d,%d; %d)",i1,i2,len);
559     if (i1>i2) return s;

```

```

560
561     s=data[i1];
562     for (size_t i=i1+1; i<=i2; i++) s+=data[i];
563
564     return s;
565 };
566
567 T cumsum_(wbvector &cs, char xflag=0) const {
568     size_t i=0, l=(xflag ? len+1 : len);
569     cs.init(l); if (!len) return 0;
570     for (--l; i<l; ++i) { cs[i+1]=cs[i]+data[i]; }
571     return (xflag ? cs[i] : cs[i]+data[i]);
572 };
573
574 T Cumsum(char zflag=0) const {
575     if (!len) return 0;
576     if (zflag) {
577         T q,x=data[0]; data[0]=0;
578         for (size_t i=1; i<len; ++i) {
579             q=data[i]; data[i]=data[i-1]+x; x=q;
580         }
581         return data[len-1]+x;
582     }
583     else {
584         for (size_t i=1; i<len; ++i) { data[i]+=data[i-1]; }
585         return data[len-1];
586     }
587 };
588
589 template <class T2>
590 T cumsum0prod(wbvector<T2> &b, wbvector<T> &cs, char xflag=0) const {
591     size_t i=0, l=(xflag ? len+1 : len);
592     if (b.len!=len) wblog(FL,
593         "ERR %s() got length mismatch (%d/%d)",FCT,len,b.len);
594     cs.init(l); if (!len) return 0;
595
596     for (--l; i<l; ++i) { cs[i+1]=cs[i]+data[i]*b.data[i]; }
597     return (xflag ? cs[i] : cs[i]+data[i]*b.data[i]);
598 };
599
600 wbvector& Eldiv( const wbvector &v) const {
601     if (len!=v.len) wblog(FL,
602         "ERR %s() size mismatch (%d/%d)",FCT,len,v.len);
603     for (size_t i=0; i<len; ++i) {
604         data[i] /= v.data[i];
605         if (v.data[i]==0) wblog(FL,
606             "ERR %s() div/0 !? (%g)",FCT,double(data[i])
607         );
608     }
609     return *this;
610 };
611
612 wbvector& tensorProd(
613     const wbvector &v2,
614     wbvector &vv
615 ) const {
616
617     vv.init(len*v2.len); if (!vv.isEmpty()) {
618         size_t i,j,l=0;
619         const T *d2=v2.data; T *d=vv.data;
620
621         for (j=0; j<v2.len; ++j)
622             for (i=0; i<len; ++i, ++l) d[l]=data[i]*d2[j];
623     }
624
625     return vv;
626 };
627
628 T scalarProd(const wbvector &v) const {
629     if (len!=v.len) wblog(FL,
630         "ERR scalarProd - length mismatch (%d/%d)",len,v.len);
631
632     if (len) return scalarProd(v.data);
633     else return 0;
634 };
635
636 T scalarProd(const T *v) const {
637     T s=T(); for (size_t i=0; i<len; ++i) s+=(data[i]*CONJ(v[i]));
638     return s;
639 };
640
641 T norm() const { return SQRT(norm2()); }
642
643 T norm2() const {
644     T s=0; if (len==0) return s;
645     s=NORM2(data[0]);
646

```

```

647         for (size_t i=1; i<len; i++) s+=NORM2(data[i]);
648         return s;
649     };
650
651     double normDiff(const wbvector &v, T fac=1) const {
652         return std::sqrt(double(normDiff2(v,fac)));
653     };
654
655     T normDiff2 (const wbvector &v, T fac=1) const;
656     double normDiff2_(const wbvector &v, double fac=1) const;
657
658     T GSProject(const T *v, char isnorm=0, char tflag=0) const {
659         return gs_project_range(data,v,len,1,isnorm,tflag);
660     };
661
662     wbvector& Normalize(const char *F=NULL, int L=0) {
663         double n=std::sqrt(norm2());
664         if (n==0) {
665             if (F) wblog(F,L,"WRN normalize zero vector!? - randomize");
666             setRand(1,-0.5); return Normalize(F,L);
667         }
668         (*this)*=(1./n); return *this;
669     };
670
671     wbvector& add(const T* d) {
672         for (size_t i=0; i<len; ++i) data[i]+=d[i];
673         return *this;
674     };
675
676     wbvector& subtract(const T* d) {
677         for (size_t i=0; i<len; ++i) data[i]-=d[i];
678         return *this;
679     };
680
681     void getReal(wbvector<double> &R) const;
682     void getImag(wbvector<double> &I) const;
683     void set(const wbvector<double> &R, const wbvector<double> &I);
684     wbvector& Conj() { return *this; };
685
686     wbvector& applyFlag(char &flag, const T &fac, wbvector &a);
687
688     bool allEqual() const {
689         for (size_t i=1; i<len; ++i) if (data[i]!=data[0]) return 0;
690         return 1;
691     }
692
693     bool allZero() const { return allEqual(T(0)); };
694     bool allOnes() const { return allEqual(T(1)); };
695
696     bool allEqual(const T &a) const {
697         for (size_t i=0; i<len; ++i) if (data[i]!=a) return 0;
698         return 1;
699     }
700
701     bool allEqual2(const T &a, T eps=1E-15) const {
702         if (eps==0) return allEqual(a);
703         else {
704             T x, x2=0; if (a!=0) eps*=a;
705             if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
706             for (size_t i=0; i<len; ++i) { x=data[i]-a; x2+=(x*x); }
707             return (ABS(x2)<eps);
708         }
709     };
710
711     bool allUnequal(const T &a) const {
712         for (size_t i=0; i<len; ++i) if (data[i]==a) return 0;
713         return 1;
714     }
715
716     bool any() const {
717         for (size_t i=0; i<len; ++i) if (data[i]) return 1;
718         return 0;
719     }
720
721     bool anyUnequal(const T &a) const {
722         for (size_t i=0; i<len; ++i) if (data[i]!=a) return 1;
723         return 0;
724     }
725
726     bool anyEqual(const T &a, size_t *I=NULL) const {
727         for (size_t i=0; i<len; ++i) {
728             if (data[i]==a) { if (I) (*I)=i; return 1; }
729         }
730         return 0;
731     }
732
733     bool anyGT(const T &a) const {

```

```

734     for (size_t i=0; i<len; ++i) if (data[i]>a) return 1;
735     return 0; }
736 bool anyGE(const T &a) const {
737     for (size_t i=0; i<len; ++i) if (data[i]>=a) return 1;
738     return 0;
739 };
740
741 bool anyLT(const T &a) const {
742     for (size_t i=0; i<len; ++i) if (data[i]<a) return 1;
743     return 0; };
744 bool anyLE(const T &a) const {
745     for (size_t i=0; i<len; ++i) if (data[i]<=a) return 1;
746     return 0;
747 };
748
749 bool allIn(const T &a1, const T &a2) const {
750     for (size_t i=0; i<len; ++i) {
751         if (data[i]<a1 || data[i]>a2) return 0; }
752     return 1;
753 };
754
755 bool anySmallVals(const T eps=1E-15) const {
756     size_t i=0; if (eps>0)
757         { for (; i<len; ++i) if (ABS(data[i])<eps) return 1; }
758     else { for (; i<len; ++i) if (data[i]) return 1; }
759     return 0;
760 };
761
762 bool isEqual(const T* v) const {
763     return (!memcmp(data, v, len*sizeof(T)));
764 }
765
766 bool anyEqual(const wbvector<size_t> &I, const T &x,
767     size_t *k=NULL) const;
768 bool anyUnequal(const wbvector<size_t> &I, const T &x,
769     size_t *k=NULL) const;
770
771 bool isEmpty() const {
772     if (data==NULL) {
773         if (len) wblog(FL,"ERR severe data inconsistency (%d,0x%lX)",len,data);
774         return 1;
775     }
776     else if (len==0)
777         wblog(FL,"WRN data[] not initialized !? (%d, 0X%lX)",len,data);
778
779     return 0;
780 };
781
782 explicit operator bool() const { return !isEmpty(); }
783 bool operator! () const { return isEmpty(); }
784
785 bool operator==(const wbvector &v) const { return isEqual(v); };
786 bool operator!=(const wbvector &v) const { return !isEqual(v); };
787
788 bool isEqual(const wbvector &v) const {
789     if (this!=&v) {
790         if (len!=v.len) return 0;
791         if (data!=v.data) {
792             for (size_t i=0; i<len; ++i) {
793                 if (data[i]!=v.data[i]) return 0;
794             }
795         }
796     }
797     return 1;
798 };
799
800 bool isEqualR(const wbvector &v) const {
801     if (len!=v.len) return 0;
802     for (size_t l=len-1, i=0; i<len; ++i) {
803         if (data[i]!=v.data[l-i]) return 0;
804     }
805     return 1;
806 };
807
808 bool operator^(const wbvector &v) const {
809     if (this==&v) { return (len ? 0 : 1); }
810     if (len!=v.len) return 0;
811
812     for (size_t i=0; i<len; ++i) {
813         if (!(data[i]==0) ^ (v.data[i]==0)) return 0; }
814
815     return 1;
816 };
817
818 bool deepEqualP(const wbvector &B) const;
819
820 char cmp(const wbvector &v) const;

```

```

821
822 bool operator< (const wbvector &v) const { return cmp(v)< 0; };
823 bool operator<= (const wbvector &v) const { return cmp(v)<=0; };
824 bool operator> (const wbvector &v) const { return cmp(v)> 0; };
825 bool operator>= (const wbvector &v) const { return cmp(v)>=0; };
826
827 int operator< (const T &r) const {
828     for (size_t i=0; i<len; ++i) if (data[i]>=r) return 0;
829     return (len ? 1 : 0);
830 };
831
832 T& el(size_t i) const {
833     if (i>=len) wblog(FL,
834         "ERR element index out of bounds (%d/%d)", i+1, len);
835     return data[i];
836 };
837
838 T elx(size_t i, const T &x) const {
839     return (i<len ? data[i] : x);
840 };
841
842 const T& operator[] (size_t i) const { return data[i]; };
843 T& operator[] (size_t i) { return data[i]; };
844
845 T& end() const { return last(); }
846 T& last() const {
847     if (!len) wblog(FL, "ERR %s() got empty %s vector", FCT, TSTR(T));
848     return data[len-1];
849 };
850
851 T& p2last() const {
852     if (!len) wblog(FL, "ERR %s() got empty %s vector", FCT, TSTR(T));
853     return data+(len-1);
854 };
855
856 T* ref(size_t i) const { return (data+i); };
857
858 wbvector& setRand(double fac=1., double shift=0.);
859 void setIdx (double shift=0., double fac=1.);
860
861 int set2Group(const wbperm &P,
862     const WBINDEX &D, const T* S0=NULL, char iflag=0
863 );
864
865 wbvector& ResizeX(size_t n) {
866     if (isref) wblog(FL, "ERR %s() got reference (%d)", FCT, isref);
867
868     if (n) {
869         if (n<=len && (len<32 || n*16>len)) { len=n; }
870         else Resize(n);
871     }
872     else init();
873
874     return *this;
875 };
876
877 wbvector& Resize(size_t n, const T* d=NULL) {
878     if (isref) wblog(FL, "ERR %s() got reference (%d)", FCT, isref);
879
880     if (n==len) { return *this; }
881     if (!len) { return RENEW(n,d); };
882     if (!n) { len=0; if (data) WB_DELETE(data); return *this; }
883
884     T *d0=data;
885     WB_NEW(data,n);
886     MEM_COPY<T>(data, n, (n<len ? n : len), d0, d);
887     len=n;
888     WB_DELETE(d0); return *this;
889 };
890
891 wbvector& resize(size_t n, wbvector& X) const {
892     if (!n) { X.init(); } else
893     if (n==len) { X=(*this) ; }
894     else {
895         WB_NEW(X.data,n);
896         if (len) MEM_COPY<T>(X.data, MIN(n,len), data);
897         if (n>len) MEM_SET<T>(X.data+len,n-len);
898     }
899
900     return X;
901 };
902
903 wbvector& cat(const wbvector &v2) {
904     Resize(len+v2.len, v2.data);
905     return *this;
906 };
907 wbvector& Cat(const wbvector &v1, const wbvector &v2) {

```

```

908     RENEW(v1.len+v2.len);
909     MEM_COPY<T>(data, len, v1.len, v1.data, v2.data);
910     return *this;
911 };
912 wbvector& Cat(const wbvector<wbvector> > &vv);
913 wbvector& Cat(const wbvector<wbvector* > &vv);
914
915 template <class T1, class T2, class T3, class T4>
916 wbvector& Cat(
917     const T1* v1, size_t l1, const T2* v2, size_t l2,
918     const T3* v3=0, size_t l3=0,
919     const T4* v4=0, size_t l4=0
920 );
921
922 template <class T1, class T2, class T3, class T4>
923 wbvector& Cat(
924     const wbvector<T1> *v1, const wbvector<T2> *v2,
925     const wbvector<T3> *v3=0,
926     const wbvector<T3> *v4=0
927 );
928
929 wbvector& Append(const wbvector &v2) {
930     Resize(len+v2.len, v2.data);
931     return *this;
932 };
933 wbvector& Append(const T &x) {
934     Resize(len+1, &x);
935     return *this;
936 };
937 wbvector& Append(size_t d2, const T *x) {
938     Resize(len+d2, x);
939     return *this;
940 };
941
942 wbvector append(const wbvector &v) const {
943     wbvector v2; append(v.data, v.len, v2);
944     return v2;
945 };
946
947 wbvector& append(const wbvector &v2, wbvector &vout) const {
948     append(v2.data, v2.len, vout);
949     return vout;
950 };
951 void append(const T *x, size_t d2, wbvector &vout) const {
952     vout.RENEW(len+d2);
953     MEM_COPY<T>(vout.data, vout.len, len, data, x);
954 };
955
956 int recLess (const T* v) const;
957 int recLessE(const T* v) const;
958 int recCompare(const T* v) const;
959
960 T max() const;
961 T max(size_t *k) const;
962 T max(unsigned &k) const { T x;
963     size_t k_; x=max(&k_); Wb::safeConvert (FL, k_, k);
964     return x;
965 };
966
967 T min() const;
968 T min(size_t *k) const;
969 T min(unsigned &k) const { T x;
970     size_t k_; x=min(&k_); Wb::safeConvert (FL, k_, k);
971     return x;
972 };
973
974 T aMax(size_t *k=NULL) const;
975
976 T aMax(unsigned &k) const { T x;
977     size_t k_; x=aMax(&k_); Wb::safeConvert (FL, k_, k);
978     return x;
979 };
980
981 T aMin(char zflag=0, size_t *k=NULL) const;
982
983 T maxneg(size_t *k=NULL) const;
984 T minpos(size_t *k=NULL) const;
985
986 T avg() const;
987 T std() const;
988
989 void range(T &xmin, T&xmax) const;
990 T range() const;
991
992 wbvector& Permute(const wbperm &P, char iflag=0);
993 wbvector& permute(wbvector &, const wbperm &P, char iflag=0) const;
994

```

```

995     wbvector permute(const wbperm &P, char iflag=0) const {
996         wbvector<T> X; permute(X,P,iflag);
997         return X;
998     };
999
1000     wbvector& blockPermute(
1001         const wbperm &P, wbvector &, char iflag=0) const;
1002
1003     wbvector& BlockPermute(const wbperm &P, char iflag=0) {
1004         wbvector<T> x(*this);
1005         return x.blockPermute(P,*this,iflag);
1006     };
1007
1008     int blockCompare(
1009         const wbperm &pa, const wbvector &b, const wbperm &pb) const;
1010
1011     void get(const WBINDEX &I0, wbvector &v) const {
1012         const wid_x_t *I=I0.data;
1013         size_t i=0, n=I0.len; v.init(n);
1014
1015         for (; i<n; i++) {
1016             if (I[i]>=len) wblog(FL,
1017                 "ERR index out of bounds (%d/%d) !?",I[i],len);
1018             v.data[i]=data[I[i]];
1019         }
1020     };
1021
1022     void get(const wbvector<size_t> &I, T* v) const {
1023         for (size_t i=0; i<I.len; i++) {
1024             if (I[i]>=len)
1025                 wblog(FL,"ERR index out of bounds (%d/%d) !?", I[i],len);
1026
1027             v[i]=data[I[i]];
1028         }
1029         return;
1030     };
1031
1032     void get(size_t i1, size_t i2, wbvector &v) const {
1033
1034         if (i1>=len || i2>=len) wberror(FL,"index out of range.");
1035         if (i2<i1)
1036             v.init();
1037         else v.init(i2-i1, data+i1);
1038     };
1039
1040     void selectSU(const WBINDEX &I);
1041     void select(const WBINDEX &I, T* r) const;
1042     wbvector& select(const WBINDEX &I, wbvector &a) const;
1043     wbvector& Select(const WBINDEX &I);
1044     wbvector& BlockSelect(const WBINDEX &I, const WBINDEX &D);
1045
1046     wbvector select(const WBINDEX &I) const {
1047         wbvector dI; select(I,dI);
1048         return dI;
1049     };
1050
1051     wbvector& getI(size_t k, wbvector &v) const;
1052     wbvector& getI(const wbindex &k, wbvector &v, char uflag=1) const;
1053
1054     wbvector& getI(const WBINDEX &k, wbvector &v, char uflag=1) const {
1055         return getI((const wbindex&)k,v,uflag);
1056     };
1057
1058     wbvector& initI(const wbvector &a, const wbindex &ia){
1059         return a.getI(ia,*this); };
1060     wbvector& initI(
1061         const wbvector &a, const wbindex &ia,
1062         const wbvector &b, const wbindex &ib,
1063         char uflag=1
1064     );
1065
1066     wbvector& Flip() {
1067         size_t i, m2=len/2, l=len-1;
1068         for (i=0; i<m2; i++) SWAP(data[i],data[l-i]);
1069         return *this;
1070     };
1071
1072     void flip(wbvector &a) const {
1073         if (&a==this) { a.Flip(); return; }
1074         a.init(len);
1075         for (size_t l=len-1, i=0; i<len; ++i) a.data[i]=data[l-i];
1076     };
1077
1078     wbvector flip() const {
1079         wbvector a; flip(a);
1080         return a;
1081     };

```

```

1082
1083     wbvector operator[] (const wbvector<size_t>& I) const {
1084         wbvector v(I.len);
1085         size_t i=0, *const k=I.data;
1086
1087         for (; i<I.len; ++i) {
1088             if (k[i]>=len) wblog(FL,
1089                 "ERR index out of bounds (%d: %d/%d)", i, I.max(), len);
1090             v[i]=data[k[i]];
1091         }
1092
1093         return v;
1094     };
1095
1096     T prod() const {
1097         T p=0;
1098
1099         if (len>0) {
1100             p=data[0];
1101             for (size_t i=1; i<len; i++) p*=data[i];
1102         }
1103         else wblog(FL, "WRN %s() empty data set", FCT);
1104
1105         return p;
1106     }
1107
1108     T prod(const T& p0) const {
1109         T p; if (len==0) return p0;
1110
1111         p=data[0]; for (size_t i=1; i<len; i++) p*=data[i];
1112         return p;
1113     }
1114
1115     T prod(size_t i1, size_t i2) const {
1116         size_t i; T p=0;
1117
1118         if (i1<len && i2<len && i1<=i2) {
1119             for (p=data[i1], i=i1+1; i<len && i<=i2; i++)
1120                 p*=data[i];
1121         }
1122         else wblog(FL, "ERR empty index set (%d,%d/%d)", i1, i2, len);
1123         return p;
1124     };
1125
1126     template <class TI>
1127     T prod(const TI *I, size_t n) const {
1128         T p=0; TI k;
1129         if (n) { p=1;
1130             for (size_t i=0; i<n; ++i) { k=I[i];
1131                 if (k>=len) wblog(FL,
1132                     "ERR %s() index out of bounds (%d/%d)", FCT, k, len);
1133                 p*=data[k];
1134             }
1135         }
1136         return p;
1137     };
1138
1139     template <class T1, class T2>
1140     T2 prod(const T1 *I, size_t n, T2 &s2) const {
1141         size_t i, k;
1142         char m[len]; for (i=0; i<len; ++i) m[i]=0;
1143         for (i=0; i<n; ++i) { k=I[i];
1144             if (k>=len) wblog(FL,
1145                 "ERR %s() index out of bounds (%d/%d)", FCT, k, len);
1146             if ((++m[k])>1) wblog(FL, "ERR %s() index not unique", FCT);
1147         }
1148         if (!len) { s2=0; return 0; }
1149         else {
1150             T2 s1=1; s2=1;
1151             for (i=0; i<len; ++i) { (m[i] ? s1 : s2) *= data[i]; }
1152             return s1;
1153         }
1154     };
1155
1156     void prod2(T &p1, T &p2, const T &p0=0) const {
1157         size_t l=len/2; if (len%2) wblog(FL,
1158             "ERR cannot split vector into two equal parts (%d)", len);
1159
1160         if (len==0) { p1=p0; p2=p0; return; }
1161         p1=data[0]; p2=data[1];
1162
1163         for (size_t i=1; i<1; i++) { p1*=data[i]; p2*=data[i+1]; }
1164     }
1165
1166     void info(const char *istr) const;
1167     void print(const char *istr, char mflag=0) const;
1168     void printdata(const char *istr=0) const {

```



```

1169         if (istr && istr[0]) printf("%s = ",istr);
1170         printf("[ %s ]\n",toStr().data);
1171     };
1172
1173     void put(
1174         const char *F, int L,
1175         const char *vname="ans", char tflag=0, const char *ws="base"
1176     ) const;
1177
1178     void put(const char *vname="ans", char tflag=0, const char *ws="base"
1179     ) const { put(0,0,vname,tflag,ws); }
1180
1181     void dispWithMsg(const char* file, int line,
1182         const char *msg, const char* vname="") const {
1183         print(vname);
1184         wblog(file,line,msg);
1185     }
1186
1187     void mat2mx(mxArray* &a, char dim=2) const;
1188     void mat2mxs(mxArray* a, char field_nr, char dim=2) const;
1189
1190     mxArray* toMx_Struct(const char tflag=0) const;
1191     mxArray* toMx_StructP(const char tflag=0) const;
1192
1193     mxArray* toMx(const char tflag=0) const { return toMx_Struct(tflag); };
1194     mxArray* toMxP(const char tflag=0) const { return toMx_StructP(tflag); };
1195
1196     mxArray* toMx_base(const char tflag=0) const {
1197         size_t m=1, n=len; if (tflag) { m=n; n=1; }
1198         return Mx::Array<double>(m,n).copyFromTR(data);
1199     };
1200
1201     mxArray* toMx_base_d(const char tflag=0) const {
1202         size_t m=1, n=len; if (tflag) { m=n; n=1; }
1203         return Mx::Array<double>(m,n).copyFromTR(data);
1204     };
1205
1206     mxArray* toMx_offset(const T& offset, const char tflag=0) const;
1207
1208     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
1209     void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const;
1210     void add2MxStruct(mxArray *S, const char *vname, char tflag=0) const;
1211
1212     mxArray* mxCreateCell(unsigned m, unsigned n) const;
1213     void add2MxCell(mxArray *S, unsigned i, char tflag=0) const;
1214
1215     void putg(const char *vname="ans", char tflag=0) const {
1216         put(vname,tflag,"global");
1217         return;
1218     };
1219
1220     void putx(const char *vname="ans", char tflag=0) const {
1221         put(vname,tflag,"base");
1222         return;
1223     };
1224
1225     wbsttring toStr(int n=-1, const char *sep=" ") const;
1226     wbsttring toStrD() const;
1227     wbsttring toStrf(
1228         const char *fmt0="",
1229         const char *sep=" ",
1230         const char stride=0,
1231         const char *sep2=""
1232     ) const;
1233
1234     T *data;
1235
1236     size_t len;
1237
1238     char isref;
1239
1240 protected:
1241
1242     void NEW(size_t n, const T* d=NULL) {
1243         len=n; data=NULL; isref=0;
1244         if (len) { WB_NEW(data,len); MEM_COPY<T>(data,len,d); }
1245     };
1246
1247     wbvector& RENEW(size_t n, const T* d=NULL) {
1248         if (isref) { data=NULL; isref=0; } else
1249             if (n!=len && data) { WB_DELETE(data); }
1250
1251         len=n; if (len) {
1252             if (!data) { WB_NEW(data,len); }
1253             MEM_COPY<T>(data,len,d);
1254         }
1255         return *this;

```

```

1256     };
1257
1258     wbvector& RENEW_VAL(size_t n, const T& x) {
1259         if (isref) { data=NULL; isref=0; } else
1260             if (n!=len && data) { WB_DELETE(data); }
1261
1262         len=n;
1263         if (len) {
1264             if (!data) { WB_NEW(data,len); }
1265             for (size_t i=0; i<len; ++i) data[i]=x;
1266         }
1267         return *this;
1268     };
1269
1270 private:
1271
1272 };
1273
1274 template <class T>
1275 wbvector<T>& wbvector<T>::unRef() {
1276
1277     if (!isref) return *this;
1278     if (!len) {
1279         if (data) wblog(FL,"ERR data=%lX (%d)",data,len);
1280         isref=0; return *this;
1281     }
1282
1283     T const* const d0=data;
1284
1285     WB_NEW(data,len); MEM_COPY<T>(data,len,d0);
1286     isref=0;
1287
1288     return *this;
1289 };
1290
1291 template <class T>
1292 bool wbvector<T>::deepEqualP(const wbvector<T> &B) const {
1293     if (len!=B.len) return 0;
1294
1295     if (data!=B.data) {
1296         for (size_t i=0; i<len; ++i) { if (data[i]==B.data[i]) continue;
1297             if (!!data[i] || !B.data[i]) return 0; else
1298                 if (!(data[i]==B.data[i])) return 0;
1299         }
1300     }
1301
1302     return 1;
1303 };
1304
1305 template <class T> inline
1306 char wbvector<T>::cmp(const wbvector<T> &v) const {
1307
1308     size_t i=0, n=MIN(len,v.len); const T *b=v.data;
1309     for (; i<n; ++i) {
1310         if (data[i]<b[i]) return -1;
1311         if (data[i]>b[i]) return +1;
1312     }
1313
1314     if (len<v.len) return -1;
1315     if (len>v.len) return +1; else return 0;
1316 };
1317
1318 template <class T> inline
1319 bool wbvector<T>::checkSameLength(
1320     const wbvector<T> &a,
1321     const char *F, int L, const char *istr
1322 ) const {
1323
1324     bool i=(len==a.len);
1325     if (!i && F) {
1326         if (istr && istr[0]) sprintf(str,"%s() - ",istr); else str[0]=0;
1327         wblog(F,L, "ERR %svector length mismatch (%d/%d)",str,len,a.len);
1328     }
1329     return i;
1330 };
1331
1332 template <class T> inline
1333 bool wbvector<T>::checkSameLength(
1334     const wbvector<T> &a,
1335     const wbvector<T> &b,
1336     const char *F, int L, const char *istr
1337 ) const {
1338
1339     bool i=(len==a.len && len==b.len);
1340     if (!i && F) {
1341         if (istr && istr[0]) sprintf(str,"%s() - ",istr); else str[0]=0;
1342         wblog(F,L,"ERR %svector length mismatch (%d,%d/%d)",str,a.len,b.len,len);

```

```

1343     }
1344     return i;
1345 };
1346
1347 template <class T> inline
1348 void wbvector<T>::getReal(wbvector<double> &R) const {
1349     R.init(len);
1350     for (size_t i=0; i<len; ++i) { R.data[i]=double(data[i]); }
1351 };
1352
1353 template <> inline
1354 void wbvector<wbcomplex>::getReal(wbvector<double> &R) const {
1355     R.init(len);
1356     for (size_t i=0; i<len; ++i) { R.data[i]=data[i].r; }
1357 };
1358
1359 template <class T> inline
1360 void wbvector<T>::getImag(wbvector<double> &I) const {
1361     I.init(len);
1362 };
1363
1364 template <> inline
1365 void wbvector<wbcomplex>::getImag(wbvector<double> &I) const {
1366     I.init(len);
1367     for (size_t i=0; i<len; ++i) { I.data[i]=data[i].i; }
1368 };
1369
1370 template <> inline
1371 void wbvector<wbcomplex>::set(
1372     const wbvector<double> &R, const wbvector<double> &I) {
1373
1374     size_t i=0; const double *re=R.data, *im=I.data;
1375
1376     init(R.len ? R.len : I.len);
1377
1378     if (R.len) {
1379         if (I.len) {
1380             if (R.len!=I.len) wblog(FL,
1381                 "ERR %s() got length inconsistency (%d/%d)",FCT,R.len,I.len);
1382             for (; i<len; ++i) { data[i].init(re[i],im[i]); }
1383         }
1384         else { for (; i<len; ++i) { data[i].init(re[i],0); } }
1385     }
1386     else if (I.len) { for (; i<len; ++i) { data[i].init(0,im[i]); } }
1387 };
1388
1389 template <> inline
1390 wbvector<wbcomplex>& wbvector<wbcomplex>::Conj() {
1391     for (size_t i=0; i<len; i++) data[i].Conj();
1392     return *this;
1393 };
1394
1395 template <class T> inline
1396 wbvector<T>& wbvector<T>::applyFlag(
1397     char &flag, const T &fac, wbvector<T> &a
1398 ){
1399     if (!strchr("NCTc",flag))
1400         wblog(FL,"ERR %s() invalid flag %c<%d>",__FUNCTION__,flag,flag);
1401
1402     if (flag=='c') flag='C';
1403     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='N';
1404
1405     if (fac!=1 || flag=='C') { a=(*this);
1406         if (flag=='C') a.Conj();
1407         if (fac!=1) a*=fac;
1408     }
1409     else a<<*this;
1410 };
1411
1412 template <class T> inline
1413 wbvector<T>& wbvector<T>::save2(wbvector<T> &v) {
1414
1415     if (this!=&v) {
1416         if (v.data) v.init();
1417         v.data=data; v.len=len; v.isref=isref;
1418         data=NULL; len=0; isref=0;
1419     }
1420
1421     return v;
1422 };
1423
1424 template <class T> inline
1425 T wbvector<T>::min() const {
1426
1427     if (!len) wblog(FL,"ERR %s() got empty vector",FCT);
1428     T x=data[0];
1429     for (size_t i=1; i<len; ++i) { if (x>data[i]) x=data[i]; }

```

```

1430     return x;
1431 };
1432
1433 template <class T> inline
1434 T wbvector<T>::min(size_t *k) const {
1435     if (!k) return min();
1436     if (!len) wblog(FL,"ERR %s() got empty vector",FCT);
1437
1438     T x=data[0]; *k=0;
1439     for (size_t i=1; i<len; ++i) { if (x>data[i]) { x=data[i]; *k=i; }}
1440     return x;
1441 };
1442
1443 template <class T> inline
1444 T wbvector<T>::max() const {
1445
1446     T x=T(0);
1447     if (!len) { wblog(FL,"ERR %s() got empty vector",FCT); }
1448     else { x=data[0];
1449         for (unsigned i=1; i<len; ++i) { if (x<data[i]) x=data[i]; }
1450     }
1451     return x;
1452 };
1453
1454 template <class T> inline
1455 T wbvector<T>::max(size_t *k) const {
1456     if (!k) return max();
1457     if (!len) wblog(FL,"ERR %s() got empty vector",FCT);
1458
1459     T x=data[0]; *k=0;
1460     for (size_t i=1; i<len; ++i) { if (x<data[i]) { x=data[i]; *k=i; }}
1461     return x;
1462 };
1463
1464 template <class T> inline
1465 T wbvector<T>::minpos(size_t *k) const {
1466     T x=0; bool found=0; if (k) (*k)=-1;
1467
1468     if (len==0) wblog(FL,"WRN %s() of null vector",FCT);
1469     else {
1470         for (size_t i=0; i<len; i++)
1471             if (data[i]>0) {
1472                 if (found) { if (x>data[i]) { x=data[i]; if (k) (*k)=i; }}
1473                 else { x=data[i]; if (k) (*k)=i; found=1; }
1474             }
1475     }
1476     return x;
1477 };
1478
1479 template <class T> inline
1480 T wbvector<T>::maxneg(size_t *k) const {
1481     T x=0; bool found=0; if (k) (*k)=-1;
1482
1483     if (len==0) wblog(FL,"WRN %s() of null vector",FCT);
1484     else {
1485         for (size_t i=0; i<len; i++)
1486             if (data[i]<0) {
1487                 if (found) { if (x<data[i]) { x=data[i]; if (k) (*k)=i; }}
1488                 else { x=data[i]; if (k) (*k)=i; found=1; }
1489             }
1490     }
1491
1492     return x;
1493 };
1494
1495 template <class T> inline
1496 T wbvector<T>::aMax(size_t *k_) const {
1497
1498     if (len) {
1499         size_t k=-1, i=0; T a,x=0;
1500         for (; i<len; ++i) {
1501             a=ABS(data[i]); if (x<a) { x=a; k=i; }
1502         }
1503         if (k_) (*k_)=k;
1504         return x;
1505     }
1506     else {
1507         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got null vector",FCT); }
1508         return 0;
1509     }
1510 };
1511
1512 template <class T> inline
1513 T wbvector<T>::aMin(char zflag, size_t *k_) const {
1514
1515     if (!len) {
1516         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got null vector",FCT); }

```

```

1517     return 0;
1518 }
1519
1520 size_t i=0, k=-1; double a,x=0;
1521
1522 if (zflag==0) {
1523     x=ABS(data[i++]); k=0; if (x!=0) {
1524         for (; i<len; ++i) { a=ABS(data[i]); if (x>a) {
1525             x=a; k=i; if (x==0) break;
1526         }}
1527     }
1528 }
1529 else {
1530     for (; i<len; ++i) {
1531         a=ABS(data[i]); if (a) { x=a; k=i; break; }
1532     }
1533     for (; i<len; ++i) { a=ABS(data[i]);
1534         if (a && x>a) { x=a; k=i; }
1535     }
1536 }
1537 if (k_) (*k_)=k;
1538 return x;
1539 };
1540
1541 template <class T> inline
1542 void wbvector<T>::range(T &xmin, T&xmax) const {
1543     if (len==0) {
1544         xmin=xmax=0;
1545         wblog(FL,"WRN %s() of null vector",FCT);
1546     }
1547     else {
1548         xmin=xmax=data[0];
1549         for (size_t i=1; i<len; i++) {
1550             if (xmin>data[i]) xmin=data[i];
1551             if (xmax<data[i]) xmax=data[i];
1552         }
1553     }
1554 };
1555
1556 template <class T> inline
1557 T wbvector<T>::range() const {
1558     T xmin, xmax; range(xmin,xmax);
1559     return (xmax-xmin);
1560 };
1561
1562 template <class T> inline
1563 T wbvector<T>::avg() const {
1564
1565     if (!len) {
1566         wblog(FL,"WRN %s() of null vector !?",FCT);
1567         return 0;
1568     }
1569     else {
1570         T x=data[0];
1571         for (size_t i=1; i<len; ++i) { x+=data[i]; }
1572         return (x/T(len));
1573     }
1574 };
1575
1576 template <class T> inline
1577 T wbvector<T>::std() const {
1578
1579     if (len<2) {
1580         wblog(FL,"WRN %s() of vector of length %d !?",FCT,len);
1581         return 0;
1582     }
1583     else {
1584         T x=data[0], x2=data[0]*data[0];
1585         for (size_t i=1; i<len; ++i) { x+=data[i]; x2+=data[i]*data[i]; }
1586         return SQRT( ( x2 - (x*x)/T(len) ) / T(len-1) );
1587     }
1588 };
1589
1590 template <class T> inline
1591 int wbvector<T>::findClosest(T r) const {
1592     if (len==0) return -1;
1593
1594     T dbl, eps=ABS(data[0]-r); size_t i,k=0;
1595
1596     for (i=1; i<len; i++) {
1597         dbl = ABS(data[i]-r);
1598         if (dbl<eps) { k=i; eps=dbl; if (eps==0) break; }
1599     }
1600
1601     return k;
1602 }
1603

```

```

1604 template <class T>
1605 int wvector<T>::findClosestSorted(T r) const {
1606     if (len==0) return -1;
1607
1608     size_t i, k, n1=0, n2=len, e=0;
1609     char c, cref;
1610
1611     if (len==0) return -1;
1612     if (len==1) return 0;
1613
1614     cref=NUMCMP(data[0], data[len-1]);
1615
1616     if (cref) {
1617         if (len>6) {
1618             for (i=1; i<4; i++) if (NUMCMP(data[i-1],data[i])==cref) e++;
1619             for (i=len-3; i<len; i++)
1620                 if (NUMCMP(data[i-1],data[i])==cref) e++;
1621         }
1622         else
1623             for (i=1; i<len; i++) if (NUMCMP(data[i-1],data[i])==cref) e++;
1624     }
1625     else for (i=1; i<len; i++) if (NUMCMP(data[i-1],data[i])) e++;
1626
1627     if (e) wblog(FL,"ERR Data not sorted! (%d)", e);
1628     if (cref==0) { wblog(FL,
1629         "WRN find closest: all data constant!?\n(%g; %g %dx)",
1630         (double)r, (double)data[0], len); return 0;
1631     }
1632
1633     c=NUMCMP(data[len-1],r); if (c!=cref) return len-1;
1634     c=NUMCMP(data[0],r); if (c!= cref) return 0;
1635
1636     for (k=0;;) {
1637         if (c==cref) {
1638             n1=n2/2; n2-=n1; if (n1) k+=n1; else {
1639                 if (k+1<len && (ABS(r-data[k]) > ABS(r-data[k+1]))) k++;
1640                 break;
1641             }
1642         }
1643         else if (c) {
1644             n2=n1/2; n1-=n2; if (n2) k-=n2; else {
1645                 if (k) if (ABS(r-data[k]) > ABS(r-data[k-1])) k--;
1646                 break;
1647             }
1648         }
1649         else break;
1650
1651         c=NUMCMP(data[k],r);
1652     }
1653
1654     return k;
1655 }
1656
1657 template <class T>
1658 wvector<T>& wvector<T>::setRand(double fac, double shift) {
1659     size_t i;
1660     static char first_call=1;
1661
1662     if (first_call) { wb_srand(); first_call=0; }
1663     fac/=(double)RAND_MAX;
1664
1665     if (shift==0.)
1666         for (i=0; i<len; ++i) data[i]=(T)(fac*rand());
1667     else for (i=0; i<len; ++i) data[i]=(T)(fac*rand()+shift);
1668
1669     return *this;
1670 };
1671
1672 template<>
1673 wvector<wbcomplex>& wvector<wbcomplex>::setRand(double fac, double shift) {
1674     size_t i;
1675     static char first_call=1;
1676
1677     if (first_call) { wb_srand(); first_call=0; }
1678     fac/=(double)RAND_MAX;
1679
1680     if (shift==0.) {
1681         for (i=0; i<len; ++i)
1682             data[i].set(fac*rand(), fac*rand());
1683     }
1684     else {
1685         for (i=0; i<len; ++i)
1686             data[i].set(fac*rand()+shift, fac*rand()+shift);
1687     }
1688
1689     return *this;
1690 };

```

```

1691
1692 template <class T> inline
1693 void wbvector<T>::setIdx(double shift, double fac) {
1694     size_t i;
1695
1696     if (shift==0.) {
1697         if (fac==1.)
1698             for (i=0; i<len; i++) data[i]=(T)(i);
1699         else for (i=0; i<len; i++) data[i]=(T)(fac*i);
1700     }
1701     else {
1702         if (fac==1.)
1703             for (i=0; i<len; i++) data[i]=(T)(i+shift);
1704         else for (i=0; i<len; i++) data[i]=(T)(fac*i+shift);
1705     }
1706 };
1707
1708 template <class T>
1709 void wbvector<T>::mat2mx(mxArray* &a, char dim) const {
1710
1711     if (dim==1) a=mxCreateDoubleMatrix(len,1,mxREAL); else
1712     if (dim==2) a=mxCreateDoubleMatrix(1,len,mxREAL);
1713     else wblog(FL,"ERR %s() invalid dim=%d",FCT,dim);
1714
1715     double *d=mxGetDoubles(a);
1716     for (size_t i=0; i<len; ++i) d[i]=(double)data[i];
1717 };
1718
1719 template <class T>
1720 void wbvector<T>::mat2mxs(mxArray* a, char fid, char dim) const {
1721
1722     size_t i=-1, m=1, n=len;
1723
1724     if (!mxIsStruct(a) || (i=mxGetNumberOfElements(a))!=1) wblog(FL,
1725     "ERR %s() invalid input structure (n=%d)",FCT,i);
1726     if (fid>=int(i=mxGetNumberOfFields(a))) wblog(FL,
1727     "ERR %s() field index out of bounds (%d/%d)",FCT,fid,i);
1728
1729     if (dim==1) { m=len; n=1; } else
1730     if (dim==2) { m=1; n=len; } else
1731     wblog(FL,"ERR %s() invalid dim=%d",FCT,dim);
1732
1733     mxArray *b=mxCreateDoubleMatrix(m,n,mxREAL);
1734     double *d=mxGetDoubles(b);
1735
1736     for (i=0; i<len; ++i) { d[i]=(double)data[i]; }
1737
1738     mxSetFieldByNumber(a,0,fid,b);
1739 };
1740
1741 template <class T> inline
1742 int wbvector<T>::recLess(const T* v) const {
1743
1744     for (size_t i=0; i<len; i++)
1745         if (data[i]<v[i]) return 1; else
1746         if (data[i]>v[i]) return 0;
1747
1748     return 0;
1749 }
1750
1751 template <class T> inline
1752 int wbvector<T>::recLessE(const T* v) const {
1753
1754     for (size_t i=0; i<len; i++)
1755         if (data[i]<v[i]) return 1; else
1756         if (data[i]>v[i]) return 0;
1757
1758     return 1;
1759 }
1760
1761 template <class T> inline
1762 int wbvector<T>::recCompare(const T* v) const {
1763
1764     for (size_t i=0; i<len; i++)
1765         if (data[i]<v[i]) return -1; else
1766         if (data[i]>v[i]) return +1;
1767
1768     return 0;
1769 }
1770
1771 template<>
1772 mxArray* wbvector<long>::toMx_base(const char tflag) const {
1773     size_t m=1, n=len; if (tflag) { m=n; n=1; }
1774     return Mx::Array<long>(m,n).copyFromTR(data);
1775 };
1776
1777 template<>

```

```

1778 mxArray* wbvector<size_t>::toMx_base(const char tflag) const {
1779     size_t m=1, n=len; if (tflag) { m=n; n=1; }
1780     return Mx::Array<size_t>(m,n).copyFromTR(data);
1781 };
1782
1783 template<>
1784 mxArray* wbvector<wbcomplex>::toMx_base(const char tflag) const {
1785     size_t m=1, n=len; if (tflag) { m=n; n=1; }
1786     return Mx::Array<wbcomplex>(m,n).copyFromTR(data);
1787 };
1788
1789 template<> int wbvector<double>::init(
1790     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
1791 { return init_base(F,L,a,istr,tcheck,ref); }
1792
1793 template<> int wbvector<wbcomplex>::init(
1794     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
1795 { return init_base(F,L,a,istr,tcheck,ref); }
1796
1797 template<> int wbvector<char>::init(
1798     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
1799 { return init_base(F,L,a,istr,tcheck,ref); }
1800
1801 template<> int wbvector<int>::init(
1802     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
1803 { return init_base(F,L,a,istr,tcheck,ref); }
1804
1805 template<> int wbvector<unsigned>::init(
1806     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
1807 { return init_base(F,L,a,istr,tcheck,ref); }
1808
1809 template<> int wbvector<size_t>::init(
1810     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
1811 { return init_base(F,L,a,istr,tcheck,ref); }
1812
1813 template<> mxArray* wbvector<double>::toMx(const char tflag) const {
1814     return toMx_base(tflag); }
1815 template<> mxArray* wbvector<wbcomplex>::toMx(const char tflag) const {
1816     return toMx_base(tflag); }
1817
1818 template<> mxArray* wbvector<unsigned>::toMx(const char tflag) const {
1819     return toMx_base(tflag); }
1820 template<> mxArray* wbvector<int>::toMx(const char tflag) const {
1821     return toMx_base(tflag); }
1822 template<> mxArray* wbvector<char>::toMx(const char tflag) const {
1823     return toMx_base(tflag); }
1824 template<> mxArray* wbvector<uint8_T>::toMx(const char tflag) const {
1825     return toMx_base(tflag); }
1826 template<> mxArray* wbvector<short>::toMx(const char tflag) const {
1827     return toMx_base(tflag); }
1828 template<> mxArray* wbvector<uint16_T>::toMx(const char tflag) const {
1829     return toMx_base(tflag); }
1830 template<> mxArray* wbvector<long>::toMx(const char tflag) const {
1831     return toMx_base(tflag); }
1832 template<> mxArray* wbvector<size_t>::toMx(const char tflag) const {
1833     return toMx_base(tflag); }
1834
1835 template<class T>
1836 mxArray* wbvector<T>::toMx_Struct(const char tflag) const {
1837
1838     mxArray *S;
1839
1840     if (tflag)
1841         S=data->mxCreatStruct(len,1);
1842     else S=data->mxCreatStruct(1,len);
1843
1844     for (size_t i=0; i<len; ++i) { data[i].add2MxStruct(S,i); }
1845     return S;
1846 }
1847
1848 template<class T>
1849 mxArray* wbvector<T>::toMx_StructP(const char tflag) const {
1850
1851     mxArray *S;
1852
1853     if (tflag)
1854         S=(*data)->mxCreatStruct(len,1);
1855     else S=(*data)->mxCreatStruct(1,len);
1856
1857     for (size_t i=0; i<len; ++i) { data[i]->add2MxStruct(S,i); }
1858     return S;
1859 };
1860
1861 template<class T>
1862 mxArray* wbvector<T>::toMx_offset(const T& offset, const char tflag)
1863 const {
1864     if (!len) { return mxCreateDoubleMatrix(0,0,mxREAL); }

```



```

1865
1866     mxArray *a=(tflag ?
1867         mxCreateDoubleMatrix(len,1,mxREAL) :
1868         mxCreateDoubleMatrix(1,len,mxREAL)
1869     );
1870
1871     double *d=mxGetDoubles(a);
1872     for (size_t i=0; i<len; ++i) { d[i]=double(data[i]+offset); }
1873
1874     return a;
1875 };
1876
1877 template <class T>
1878 void wbvector<T>::put(
1879     const char *file, int line, const char *vname, char tflag,
1880     const char* ws
1881 ) const {
1882
1883     size_t i;
1884     mxArray *a=toMx(tflag);
1885
1886     if (file) wblog(file,line,
1887         "I/O putting vector '%s' to %s%s",vname,ws,tflag ? " (t)" : ".");
1888
1889     i=mexPutVariable(ws, vname, a);
1890
1891     if (i==1) {
1892         info("this"); wblog(__FILE__,
1893             "ERR failed to put variable %s into workspace %s.",vname,ws);
1894     }
1895
1896     mxDestroyArray(a);
1897     return;
1898 }
1899
1900 inline WBINDEX Index(size_t i1, size_t i2) {
1901     WBINDEX ii;
1902
1903     if (i1>i2) return ii;
1904
1905     ii.init(i2-i1+1);
1906     for (size_t i=0; i<ii.len; i++) ii[i]=i+i1;
1907
1908     return ii;
1909 };
1910
1911 inline char isUniqueIdxSet(const WBINDEX &I0, widx_t imax) {
1912
1913     size_t i, n=I0.len, e=0;
1914     const widx_t *I=I0.data;
1915     wbvector<char> mark(imax); char *m=mark.data;
1916
1917     for (i=0; i<n; i++) if (I[i]<imax) m[I[i]]++; else e++;
1918     for (i=0; i<imax; i++) if (m[i]>1) e++;
1919
1920     return (e==0);
1921 }
1922
1923 inline char validPerm(const WBPPerm &P, wperm_t d) {
1924
1925     size_t i,n=P.len;
1926     wbvector<char> f(n);
1927
1928     if (swperm_t(d)>=0) if (P.len!=d) return 0;
1929
1930     for (i=0; i<n; i++) {
1931         if (P[i]>=n) return 0;
1932         f[P[i]]++;
1933     }
1934
1935     for (i=0; i<n; i++) if (f[i]!=1) return 0;
1936
1937     return 1;
1938 }
1939
1940 inline void getIPerm(const WBPPerm &P, WBPPerm &iP) {
1941
1942     size_t i,n=P.len;
1943
1944     #ifdef __WBDEBUG__
1945     if (!validPerm(P))
1946         werror(FLINE,"Invalid permutation.");
1947     #endif
1948
1949     iP.init(n);
1950
1951

```

```

1952     for (i=0; i<n; i++) iP[P[i]]=i;
1953
1954     return;
1955 }
1956
1957 char isIdentityPerm(const WBPERM &P) {
1958
1959     for (size_t i=0; i<P.len; i++) if (P[i]!=i) return 0;
1960     return 1;
1961 }
1962
1963 #endif
1964

```

5.41 xmap.hh

```

1  #ifndef __WB_XMAP_HCC__
2  #define __WB_XMAP_HCC__
3
4  /* ----- *
5  * class XMAP - map to store contraction pattern
6  * AW (C) Jun 2006
7  * ----- */
8
9  // ----- //
10 // ----- //
11 // central element for contraction map
12
13 class XMAP_Node {
14
15 public:
16
17     XMAP_Node ()
18     : Z(NULL), R(NULL), I(NULL),
19       tflag(0), ic(0), i1(0), i2(0), isref(0), z(1.) {};
20
21     ~XMAP_Node () {};
22
23     free() {
24         if (isref) { Z=R=NULL; }
25         else {
26             if (Z) { delete Z; Z=NULL; }
27             if (R) { delete R; R=NULL; }
28         }
29     };
30
31     void init(const mxArray *a);
32
33     wbvector<unsigned> size() const {
34         if (Z) return (*Z).SIZE; else
35         if (R) return (*R).SIZE;
36         else return wbvector<unsigned>();
37     };
38
39     unsigned rank() const {
40         if (Z) return (*Z).rank(); else
41         if (R) return (*R).rank();
42         else return 0;
43     };
44
45     bool hasSameSize(const XMAP_Node &B) const {
46         if ((!R ^ !Z) || (!B.R ^ !B.Z)) wblog(FL,
47         "ERR Severe XMAP inconsistency.");
48
49         return (
50             (R ? (*R).SIZE : (*Z).SIZE) !=
51             (B.R ? (*B.R).SIZE : (*B.Z).SIZE)
52         );
53     };
54
55     template<class T>
56     void set(
57         const wbArray<T> &A,
58         unsigned ic, unsigned i1, unsigned i2, wbcomplex z, char ref=1
59     );
60
61     void set(
62         unsigned ic, unsigned i1, unsigned i2, wbcomplex z, char ref=1
63     );
64
65     wbArray<wbcomplex> *Z;
66     wbArray<double> *R;
67

```

```

68     char tflag;
69     wbperrm P;
70
71     unsigned ic;
72     unsigned il;
73     unsigned i2;
74
75     unsigned isref;
76
77     wbcomplex z;
78
79 protected:
80 private:
81
82 };
83
84 class XMAP : public wbvector<XMAP_Node> {
85 public:
86     XMAP(unsigned n) : isref(1) { initDef(n); };
87
88     ~XMAP() {}
89
90     free() {
91         for (unsigned i=0; i<len; i++)
92             data[i].free();
93     };
94
95     void init(unsigned n) { free(); initDef(n); };
96
97     template<class T>
98     void set(
99         unsigned k, const wbArray<T> &A,
100         unsigned ic, unsigned il, unsigned i2, wbcomplex z
101     ) { data[k].set(A,ic,il,i2,z, isref); };
102
103     void set(
104         unsigned k,
105         unsigned ic, unsigned il, unsigned i2, wbcomplex z
106     ) { data[k].set(ic,il,i2,z, isref); };
107
108     char isref;
109
110 protected:
111 private:
112 };
113
114 void XMAP_Node::init(const mxArray *a) {
115
116     if (!a) wblog(FL,
117         "ERR Input mxArray is NULL.");
118
119     if (!mxIsDouble(a) || mxIsSparse(a)) wblog(FL,
120         "ERR Need full double array upon input.");
121
122     free();
123     (*(mxIsComplex(a) ? Z : R)).init(a);
124 }
125
126 template<class T>
127 void XMAP_Node::set(
128     unsigned k, const wbArray<T> &A,
129     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
130 ){
131     wblog(FL,"ERR XMAP::set() not available for data type %s",
132         TSTR(T));
133 };
134
135 void XMAP_Node::set(
136     unsigned k, const wbArray<wbcomplex> &A,
137     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
138 ){
139     ic=ic_; il=il_; i2=i2_; z=z_;
140
141     if (A.rank()!=2) wblog(FL,
142         "ERR expecting rank-2 objects to contract.");
143
144     if (ref) {
145         Z=&A; isref=1;
146     }
147     else {
148         if (!Z) Z = new wbArray<wbcomplex>;
149         (*Z)=A; (*Z)*=z;
150         z=1.;
151     }
152 };
153
154 void XMAP_Node::set(

```

```

155     const wbArray<double> &A,
156     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
157 ){
158     ic=ic_; il=il_; i2=i2_; z=z_;
159
160     if (A.rank()!=2) wblog(FL,
161     "ERR expecting rank-2 objects to contract.");
162
163     if (ref) {
164         R=&A; isref=1;
165     }
166     else {
167         if (z.i==0.) {
168             if (!R) R = new wbArray<double>;
169             (*R)=A; (*R)*=z.r;
170         }
171         else {
172             if (!Z) Z = new wbArray<wbcomplex>;
173             (*Z).set(A, wbArray<double>());
174             (*Z)*=z;
175         }
176         z=1.;
177     }
178 };
179
180 void XMAP_Node::set (
181     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_
182 ){
183     ic=ic_; il=il_; i2=i2_; z=z_;
184
185     if (Z && R) wblog(FL,
186     "ERR Both Z and R are set (0x%X, 0x%X) ???", Z, R);
187
188     if (Z) (*Z)*=z; else
189     if (R) {
190         if (z.i==0) (*R)*=z.r;
191         else {
192             Z = new wbArray<wbcomplex>;
193             Z.set(R, wbArray<double>()); (*Z)*=z;
194             delete R; R=NULL;
195         }
196     };
197
198     z=1.;
199 };
200
201 void XMAP::init(const mxArray *a) {
202     unsigned i,N;
203
204     if (mxIsCell(a)) {
205         N=mxGetNumberOfElements(a);
206         init(N);
207
208         for (i=0; i<N; i++)
209             data[i].init(mxGetCell(a,i));
210     }
211     else
212     if (mxIsDouble(a)) {
213         N=1; init(N); data[0].init(a);
214     }
215
216     for (i=0; i<N; i++) {
217         if (data[i].rank()!=2) wblog(FL,
218         "ERR Expecting rank-2 objects (%d)", data[i].rank());
219     }
220 }
221
222 #endif
223

```

Index

[__wb_msgbuf, 11](#)

[blockSpaceQS< TQ, TD >, 14](#)

[cdata< TD >, 17](#)

[CData< TQ, TD >, 15](#)

[CG::FileLock, 30](#)

[cgc_contract_id< TM >, 18](#)

[cgdStatus, 19](#)

[cgrType, 20](#)

[clebsch::binomial_t, 13](#)

[clebsch::coefficients, 23](#)

[clebsch::decomposition, 30](#)

[clebsch::index_adapter, 34](#)

[clebsch::pattern, 54](#)

[clebsch::weight, 117](#)

[CPAT< TQ, TD >, 23](#)

[CPAT_Set< TQ, TD >, 24](#)

[cpu_time, 25](#)

[CRef< TQ >, 25](#)

[CStore< TQ >, 27](#)

[ctrIdx, 29](#)

[DY::Symmetry, 77](#)

[DY::weight_info< TQ >, 118](#)

[DY::Weights< TQ >, 118](#)

[genRG_base< TQ, TD >, 31](#)

[genRG_struct< TQ, TD >, 31](#)

[groupIndex< T >, 33](#)

[icFlags, 33](#)

[IndexLabels, 34](#)

[indexSparseRef< TD >, 35](#)

[is_pointer< T >, 36](#)

[is_pointer< T * >, 37](#)

[is_pointer< T ** >, 37](#)

[is_pointer< T *** >, 37](#)

[iTag, 37](#)

[iterSparseCMref< TD >, 39](#)

[MEM_SET< T >, 42](#)

[MEM_SET< T * >, 42](#)

[MHash< TM >, 42](#)

[Mx::Array< T >, 12](#)

[mxArray_buf, 46](#)

[MXPut, 46](#)

[mxStruct, 47](#)

[NRGData< TQ, TD >, 48](#)

[NRGIndex< TQ, TD >, 50](#)

[opFlags< T >, 51](#)

[OPTS, 52](#)

[OrthoCGS< TQ >, 52](#)

[pairPatternCG< T >, 53](#)

[QBlock< TQ, TD, TE >, 55](#)

[QDir, 55](#)

[QHash< TQ >, 56](#)

[QHash_, 57](#)

[QMap< TQ >, 57](#)

[QSet< TQ >, 58](#)

[qset< TQ >, 59](#)

[QSpace< TQ, TD >, 61](#)

[QType, 67](#)

[QVec, 69](#)

[RCStore, 71](#)

[RIXS_para, 71](#)

[RSpecData< TS >, 72](#)

[RStore< TQ, TD >, 72](#)

[sparseCollector2D< TD >, 73](#)

[sparseIndex2D< T >, 74](#)

[Spectral< TS >, 74](#)

[STAT_DGEMM, 76](#)

[SVDDData< TQ, TD >, 77](#)

[TDSpectral< TS >, 78](#)

[Wb::ARGV, 11](#)

[Wb::Bitset, 13](#)

[Wb::bitset< T >, 13](#)

[Wb::cc_quad, 14](#)

[Wb::citerator< T, T2 >, 21](#)

[Wb::citerator< T, T2 * >, 21](#)

[Wb::CleanUp, 22](#)

[Wb::Clock, 22](#)

[Wb::Clock_resume, 22](#)

[Wb::gpara_, 32](#)

[Wb::IOstat, 36](#)

[Wb::iterator< T, T2 >, 38](#)

[Wb::iterator< T, T2 * >, 39](#)

[Wb::line_stat_, 40](#)

[Wb::LogException, 40](#)

[Wb::matFile, 41](#)

[Wb::mpfr__< P >, 43](#)

[Wb::MREC, 45](#)

[Wb::myname_, 48](#)

[Wb::num2Fmt< T >, 51](#)

[Wb::Path, 53](#)

Wb::Profile, [54](#)
Wb::rand, [70](#)
Wb::sptr< T >, [75](#)
Wb::String, [76](#)
Wb::tmpFile, [79](#)
Wb::UseClock, [79](#)
Wb::WbListMTRACK, [90](#)
wbarry< T >, [79](#)
wbarrRef< T >, [86](#)
wbcomplex, [87](#)
wbdebug_dummy, [88](#)
wbhist, [88](#)
wbIndex, [89](#)
wbindex, [88](#)
wblog::SBUF, [73](#)
wblog::stdio_buf, [76](#)
wblogbuf_struct, [91](#)
wblogbuffer, [91](#)
wbMatrix< T >, [92](#)
wbMsg, [99](#)
wbperm, [99](#)
wbRecs< T >, [101](#)
wbREF< TX >, [101](#)
wbSigHandler, [101](#)
wbsparray< TD >, [102](#)
wbstring, [107](#)
wbsys, [109](#)
wbtopy, [110](#)
WbUtil< T >, [110](#)
wbvector< T >, [111](#)

X3Map< TQ, TD >, [119](#)
x3map< TQ, TD >, [119](#)
XMAP, [120](#)
XMAP_CMat, [121](#)
XMAP_Node, [121](#)